

第三篇 系统移植篇

在上一篇中我们学习了如何进行 I.MX6U 的裸机开发，通过 21 个裸机例程我们掌握了 I.MX6U 的常用外设。通过裸机的学习我们掌握了外设的底层原理，这样在以后进行 Linux 驱动开发的时候就只需要将精力放到 Linux 驱动框架上，在进行 Linux 驱动开发之前肯定需要先将 Linux 系统移植到开发板上去。如果学习过 UCOS/FreeRTOS 应该知道，UCOS/FreeRTOS 移植就是在官方的 SDK 包里面找一个和自己所使用的芯片一样的工程编译一下，然后下载到开发板就可以了。那么 Linux 的移植是不是也是这样的，下载 Linux 源码，然后找个和我们所使用的芯片一样的工程编译一下就可以了？很明显不是的！Linux 的移植要复杂的多，在移植 Linux 之前我们需要先移植一个 bootloader 代码，这个 bootloader 代码用于启动 Linux 内核，bootloader 有很多，常用的就是 U-Boot。移植好 U-Boot 以后再移植 Linux 内核，移植完 Linux 内核以后 Linux 还不能正常启动，还需要再移植一个根文件系统(rootfs)，根文件系统里面包含了一些最常用的命令和文件。所以 U-Boot、Linux kernel 和 rootfs 这三者一起构成了一个完整的 Linux 系统，一个可以正常使用、功能完善的 Linux 系统。在本篇我们就来讲解 U-Boot、Linux Kernel 和 rootfs 的移植，与其说是“移植”，倒不如说是“适配”，因为大部分的移植工作都由 NXP 完成了，我们这里所谓的“移植”主要是使其能够在 I.MX6U-ALPHA 开发板上跑起来。

第三十章 U-Boot 使用实验

在移植 U-Boot 之前,我们肯定要先使用一下 U-Boot,得先体验一下 U-Boot 是个什么东西。I.MX6U-ALPHA 开发板光盘资料里面已经提供了一个正点原子团队已经移植好的 U-Boot,本章我们就直接编译这个移植好的 U-Boot,然后烧写到 SD 卡里面启动,启动 U-Boot 以后就可以学习使用 U-Boot 的命令。

30.1 U-Boot 简介

Linux 系统要启动就必须需要一个 bootloader 程序, 也就是说芯片上电以后先运行一段 bootloader 程序。这段 bootloader 程序会先初始化 DDR 等外设, 然后将 Linux 内核从 flash(NAND, NOR FLASH, SD, MMC 等)拷贝到 DDR 中, 最后启动 Linux 内核。当然了, bootloader 的实际工作要复杂的多, 但是它最主要的工作就是启动 Linux 内核, bootloader 和 Linux 内核的关系就跟 PC 上的 BIOS 和 Windows 的关系一样, bootloader 就相当于 BIOS。所以我们要先搞定 bootloader, 很庆幸, 有很多现成的 bootloader 软件可以使用, 比如 U-Boot、vivi、RedBoot 等等, 其中以 U-Boot 使用最为广泛, 为了方便书写, 本书会将 U-Boot 写为 uboot。

uboot 的全称是 Universal Boot Loader, uboot 是一个遵循 GPL 协议的开源软件, uboot 是一个裸机代码, 可以看作是一个裸机综合例程。现在的 uboot 已经支持液晶屏、网络、USB 等高级功能。uboot 官网为 <http://www.denx.de/wiki/U-Boot/>, 如图 30.1.1 所示:

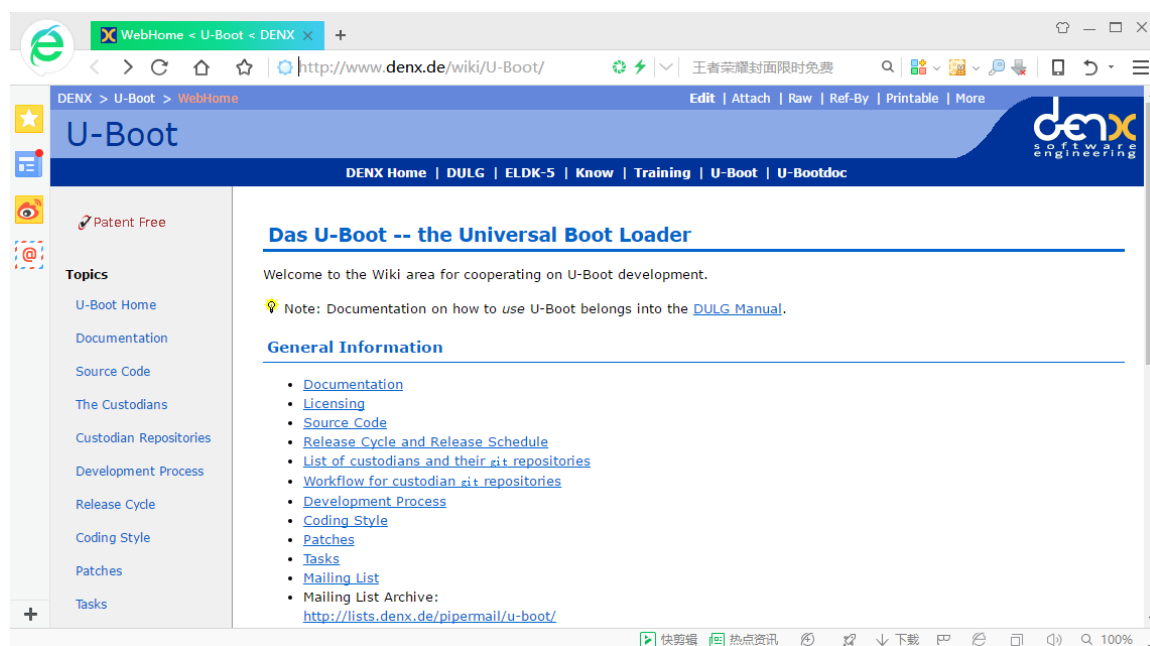


图 30.1.1 uboot 官网

我们可以在 uboot 官网下载 uboot 源码, 点击图 30.1.1 中左侧 Topics 中的“Source Code”, 打开如图 30.1.2 所示界面:

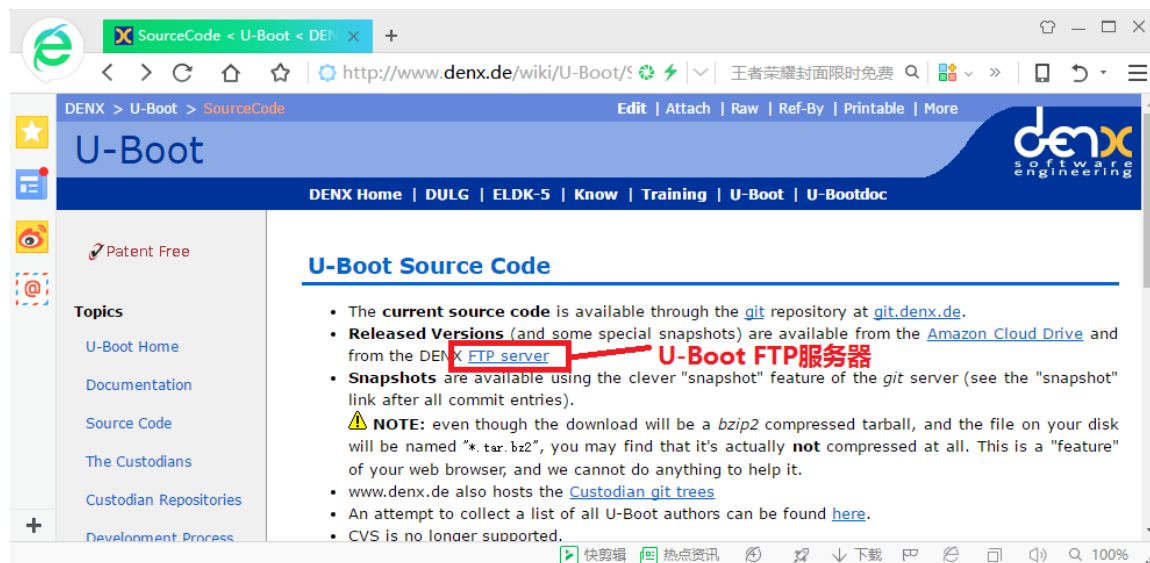


图 30.1.2 uboot 源码界面

点击图 30.1.2 中的“FTP Server”，进入其 FTP 服务器即可看到 uboot 源码，如图 30.1.3 所示：

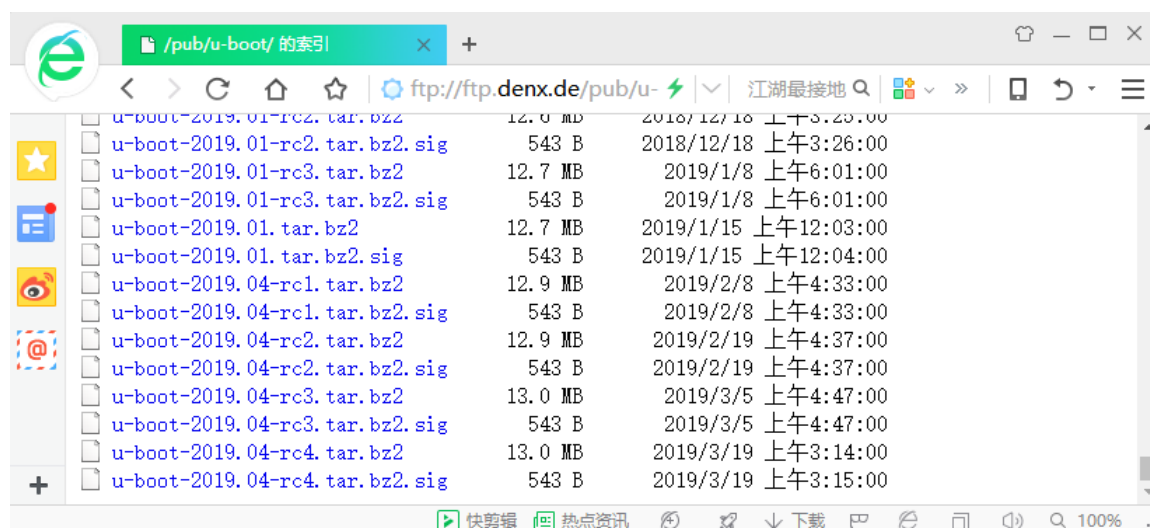


图 30.1.3 uboot 源码

图 30.1.3 中就是 uboot 原汁原味的源码文件，目前最新的版本是 2019.04。但是我们一般不会直接用 uboot 官方的 U-Boot 源码的。uboot 官方的 uboot 源码是给半导体厂商准备的，半导体厂商会下载 uboot 官方的 uboot 源码，然后将自家相应的芯片移植进去。也就是说半导体厂商会自己维护一个版本的 uboot，这个版本的 uboot 相当于是他们定制的。既然是定制的，那么肯定对自家的芯片支持会很全，虽然 uboot 官网的源码中一般也会支持他们的芯片，但是绝对是没有半导体厂商自己维护的 uboot 全面。

NXP 就维护的 2016.03 这个版本的 uboot，下载地址为：
http://git.freescale.com/git/cgit.cgi/imx/uboot-imx.git/tag/?h=imx_v2016.03_4.1.15_2.0.0_ga&id=rel_imx_4.1.15_2.1.0_ga，下载界面如图 30.1.4 所示：

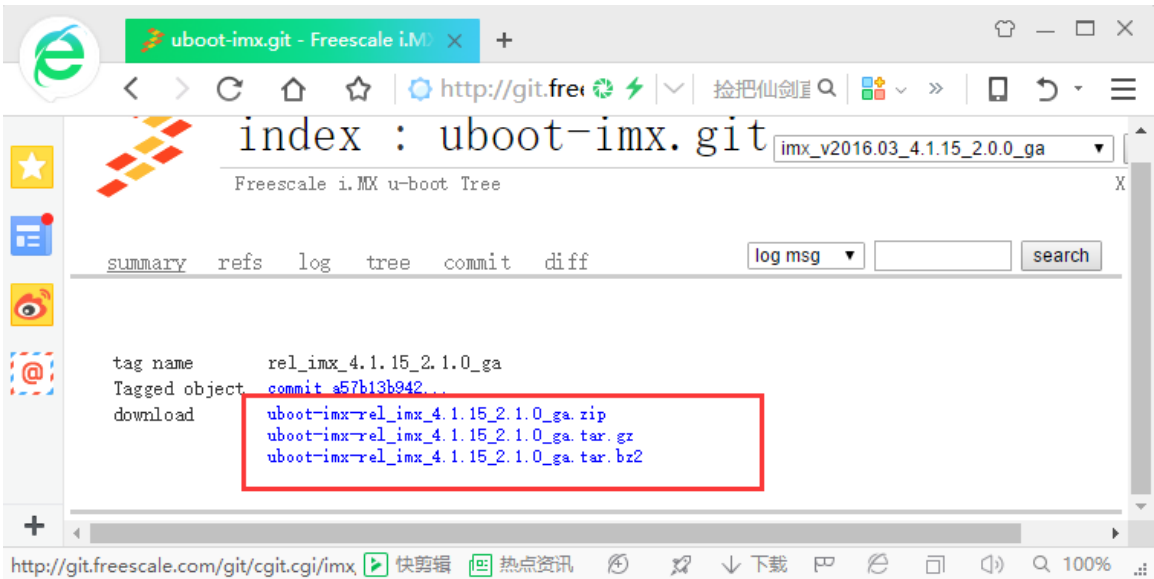


图 30.1.4 NXP 官方 uboot 下载界面

图 30.1.4 中的 uboot-imx_rel_imx4.1.15_2.1.0_ga.xx(xx 为 zip、tar.gz 或 tar.bz2)就是 NXP 官方维护的 uboot，后面我们学习 uboot 移植的时候就是使用的图 30.1.4 中的 uboot，下载 uboot-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2。我们已经放到了开发板光盘中，路径为：**开发板光盘->1、程序源码->4、NXP 官方原版 Uboot 和 Linux->uboot-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2**。图 30.1.4 中的 uboot 基本支持了 NXP 当前所有可以跑 Linux 的芯片，而且支持各种启动方式，比如 EMMC、NAND、NOR FLASH 等等，这些都是 uboot 官方所不支持的。但是图 30.1.4 中的 uboot 是针对 NXP 自家评估板的，如果是我们自己做的板子就需要修改 NXP 官方的 uboot，使其支持我们自己做板子，正点原子的 I.MX6U 开发板就是自己做的板子，虽然大部分都参考了 NXP 官方的 I.MX6ULLEVK 开发板，但是还是有很多不同的地方，所以需要修改 NXP 官方的 uboot，使其适配正点原子的 I.MX6U 开发板。所以当我们拿到开发板以后，是有三种 uboot 的，这三种 uboot 的区别如表 30.1.1 所示：

种类	描述
uboot 官方的 uboot 代码	由 uboot 官方维护开发的 uboot 版本，版本更新快，基本包含所有常用的芯片。
半导体厂商的 uboot 代码	半导体厂商维护的一个 uboot，专门针对自家的芯片，在对自家芯片支持上要比 uboot 官方的好。
开发板厂商的 uboot 代码	开发板厂商在半导体厂商提供的 uboot 基础上加入了对自家开发板的支持。

表 30.1.1 三种 uboot 的区别

那么这三种 uboot 该如何选择呢？首先 uboot 官方的基本是不会用的，因为支持太弱了。最常用的就是半导体厂商或者开发板厂商的 uboot，如果你用的半导体厂商的评估板，那么就使用半导体厂商的 uboot，如果你是购买的第三方开发板，比如正点原子的 I.MX6ULL 开发板，那么就使用正点原子提供的 uboot 源码（也是在半导体厂商的 uboot 上修改的）。当然了，你也可以在购买了第三方开发板以后使用半导体厂商提供的 uboot，只不过有些外设驱动可能不支持，需要自己移植，这个就是我们常说的 uboot 移植。

本节是 uboot 的使用，所以就直接使用正点原子已经移植好的 uboot，这个已经放到了开发板光盘中了，路径为：**开发板光盘->1、程序源码->3、正点原子 Uboot 和 Linux 出厂源码->uboot-imx-2016.03-2.1.0-ge468cdc-v1.5.tar.bz2**。

30.2 U-Boot 初次编译

首先在 Ubuntu 中安装 ncurses 库，否则编译会报错，安装命令如下：

```
sudo apt-get install libncurses5-dev
```

在 Ubuntu 中创建存放 uboot 的目录，比如我的是 /home/\$USER/linux/uboot，然后在此目录下新建一个名为“alientek_uboot”的文件夹用于存放正点原子提供的 uboot 源码。alientek_uboot 文件夹创建成功以后使用 FileZilla 软件将正点原子提供的 uboot 源码拷贝到此目录中，正点原子提供的 uboot 源码已经放到了开发板光盘中，路径为：**开发板光盘->1、例程源码->3、正点原子 Uboot 和 Linux 出厂源码->uboot-imx-2016.03-2.1.0-ge468cdc-v1.5.tar.bz2**。将其拷贝到 Ubuntu 中新建的 alientek_uboot 文件夹下，完成以后如图 30.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$ ls
uboot-imx-2016.03-2.1.0-g8b546e4.tar.bz2
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$
```

图 30.2.1 将 uboot 拷贝到 Ubuntu 中

使用如下命令对其进行解压缩：

```
tar -xjf uboot-imx-2016.03-2.1.0-g8b546e4.tar.bz2
```

解压完成以后 alientek_uboot 文件夹内容如图 30.2.2 所示：

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$ ls
api      config.mk  dts        Kconfig    Makefile   snapshot.commit
arch     configs   examples   lib         net        test
board    disk      fs          Licenses   post       tools
cmd       doc       include    MAINTAINERS README     uboot-imx-2016.03-2.1.0-g8b546e4.tar.bz2
common   drivers   Kbuild     MAKEALL     scripts
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$
```

图 30.2.2 解压后的 uboot

图 30.2.2 中除了 uboot-imx-2016.03-2.1.0-g8b546e4.tar.bz2 这个正点原子提供的 uboot 源码压缩包以外，其他的文件和文件夹都是解压出来的 uboot 源码。

1、512MB(DDR3)+8GB(EMMC)核心板

如果使用的是 512MB+8GB 的 EMMC 核心板，使用如下命令来编译对应的 uboot：

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- (加空格)
mx6ull_14x14_ddr512_emmc_defconfig
make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j12
```

这三条命令中 ARCH=arm 设置目标为 arm 架构，CROSS_COMPILE 指定所使用的交叉编译器。第一条命令相当于“make distclean”，目的是清除工程，一般在第一次编译的时候最好清理一下工程。第二条指令相当于“make mx6ull_14x14_ddr512_emmc_defconfig”，用于配置 uboot，配置文件为 mx6ull_14x14_ddr512_emmc_defconfig。最后一条指令相当于“make -j12”也就是使用 12 核来编译 uboot。当这三条命令执行完以后 uboot 也就编译成功了，如图 30.2.3 所示：

```
/mx6ullevk/imximage-ddr512.cfg.cfgtmp board/freescale/mx6ullevk/imximage-ddr512.cfg
./tools/mkimage -n board/freescale/mx6ullevk/imximage-ddr512.cfg.cfgtmp -T imximage -e 0x87
800000 -d u-boot.bin u-boot.imx
Image Type:   Freescale IMX Boot Image
Image Ver:    2 (i.MX53/6/7 compatible)
Mode:         DCD
Data Size:    425984 Bytes = 416.00 kB = 0.41 MB
Load Address: 877ff420
Entry Point:  87800000
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$ ls
```

图 30.2.3 编译完成

编译完成以后的 alientek_uboot 文件夹内容如图 30.2.4 所示:

```

zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$ ls
api          doc          lib          scripts      u-boot.imx
arch         drivers     Licenses    snapshot.commit  uboot-imx-2016.03-2.1.0-g8b546e4.tar.bz2
board        dts         MAINTAINERS System.map    u-boot.lds
cmd          examples    MAKEALL      test          u-boot.map
common       fs          Makefile     tools         u-boot-nodtb.bin
config.mk    include     net          u-boot       u-boot.srec
configs      Kbuild     post         u-boot.bin   u-boot.sym
disk         Kconfig    README      u-boot.cfg
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$

```

图 30.2.4 编译后的 uboot 源码

可以看出, 编译完成以后 uboot 源码多了一些文件, 其中 u-boot.bin 就是编译出来的 uboot 二进制文件。uboot 是个裸机程序, 因此需要在其前面加上头部(IVT、DCD 等数据)才能在 I.MX6U 上执行, 图 30.2.4 中的 u-boot.imx 文件就是添加头部以后的 u-boot.bin, u-boot.imx 就是我们最终要烧写到开发板中的 uboot 镜像文件。

每次编译 uboot 都要输入一长串命令, 为了简单起见, 我们可以新建一个 shell 脚本文件, 将这些命令写到 shell 脚本文件里面, 然后每次只需要执行 shell 脚本即可完成编译工作。新建名为 mx6ull_alientek_emmc.sh 的 shell 脚本文件, 然后在里面输入如下内容:

示例代码 30.2.1 mx6ull_alientek_emmc.sh 文件代码

```

1 #!/bin/bash
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- (加空格)
   mx6ull_14x14_ddr512_emmc_defconfig
4 make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j12

```

第 1 行是 shell 脚本要求的, 必须是 “#!/bin/bash” 或者 “#!/bin/sh”。

第 2 行使用了 make 命令, 用于清理工程, 也就是每次在编译 uboot 之前都清理一下工程。这里的 make 命令带有三个参数, 第一个是 ARCH, 也就是指定架构, 这里肯定是 arm; 第二个参数 CROSS_COMPILE 用于指定编译器, 只需要指明编译器前缀就行了, 比如 arm-linux-gnueabihf-gcc 编译器的前缀就是 “arm-linux-gnueabihf-”; 最后一个参数 distclean 就是清除工程。

第 3 行也使用了 make 命令, 用于配置 uboot。同样有三个参数, 不同的是, 最后一个参数是 mx6ull_14x14_ddr512_emmc_defconfig。前面说了 uboot 是 bootloader 的一种, 可以用来引导 Linux, 但是 uboot 除了引导 Linux 以外还可以引导其它的系统, 而且 uboot 还支持其它的架构和外设, 比如 USB、网络、SD 卡等。这些都是可以配置的, 需要什么功能就使能什么功能。所以在编译 uboot 之前, 一定要根据自己的需求配置 uboot。mx6ull_14x14_ddr512_emmc_defconfig 就是正点原子针对 I.MX6U-ALPHA 的 EMMC 核心板编写的配置文件, 这个配置文件在 uboot 源码的 configs 目录中。在 uboot 中, 通过 “make xxx_defconfig” 来配置 uboot, xxx_defconfig 就是不同板子的配置文件, 这些配置文件都在 uboot/configs 目录中。

第 4 行有 4 个参数, 用于编译 uboot, 通过第 3 行配置好 uboot 以后就可以直接 “make” 编译 uboot 了。其中 V=1 用于设置编译过程的信息输出级别; -j 用于设置主机使用多少线程编译 uboot, 最好设置成我们虚拟机所设置的核心数, 如果在 VMware 里面给虚拟就分配了 4 个核, 那么使用 -j4 是最合适的, 这样 4 个核都会一起编译。

使用 chmod 命令给予 mx6ull_alientek_emmc.sh 文件可执行权限, 然后就可以使用这个 shell 脚本文件来重新编译 uboot, 命令如下:

```
./mx6ull_alientek_emmc.sh
```

2、256MB(DDR3)+ 512MB(NAND)核心板

如果用的 256MB+512MB 的 NAND 核心板, 新建名为 mx6ull_alientek_nand.sh 的 shell 脚本文件, 然后在里面输入如下内容:

示例代码 30.2.2 mx6ull_alientek_nand.sh 文件代码

```
1 #!/bin/bash
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- (加空格)
   mx6ull_14x14_ddr256_nand_defconfig
4 make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j12
```

完成以后同样使用 chmod 指令给予 mx6ull_alientek_nand.sh 可执行权限, 然后输入如下命令即可编译 NAND 版本的 uboot:

```
./mx6ull_alientek_nand.sh
```

mx6ull_alientek_nand.sh 和 mx6ull_alientek_emmc.sh 类似, 只是 uboot 配置文件不同, 这里就不详细介绍了。

30.3 U-Boot 烧写与启动

uboot 编译好以后就可以烧写到板子上使用了, 这里我们跟前面裸机例程一样, 将 uboot 烧写到 SD 卡中, 然后通过 SD 卡来启动来运行 uboot。使用 imxdownload 软件烧写, 命令如下:

```
chmod 777 imxdownload //给予 imxdownload 可执行权限, 一次即可
./imxdownload u-boot.bin /dev/sdd //烧写到 SD 卡, 不能烧写到/dev/sda 或 sda1 设备里面!
```

等待烧写完成, 完成以后将 SD 卡插到 I.MX6U-ALPHA 开发板上, BOOT 设置从 SD 卡启动, 使用 USB 线将 USB_TTL 和电脑连接, 也就是将开发板的串口 1 连接到电脑上。打开 MobaXterm, 设置好串口参数并打开, 最后复位开发板。在 MobaXterm 上出现 “Hit any key to stop autoboot: ” 倒计时的时候按下键盘上的回车键, 默认是 3 秒倒计时, 在 3 秒倒计时结束以后如果没有按下回车键的话 uboot 就会使用默认参数来启动 Linux 内核了。如果在 3 秒倒计时结束之前按下回车键, 那么就会进入 uboot 的命令行模式, 如图 30.3.1 所示:

```
U-Boot 2016.03-gd3f0479 (Aug 07 2020 - 20:47:37 +0800)

CPU:   Freescale i.MX6ULL rev1.1 792 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 51C
Reset cause: POR
Board: I.MX6U ALPHA|MINI
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: ATK-LCD-7-1024x600 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc1(part 0) is current device
Net:    FEC1
Error: FEC1 address not set.

Normal Boot
Hit any key to stop autoboot:  0
=>
```

图 30.3.1 uboot 启动过程

从图 30.3.1 可以看出,当进入到 uboot 的命令行模式以后,左侧会出现一个“=>”标志。uboot 启动的时候会输出一些信息,这些信息如下所示:

示例代码 30.3.1 uboot 输出信息

```
1 U-Boot 2016.03-gd3f0479 (Aug 07 2020 - 20:47:37 +0800)
2
3 CPU: Freescale i.MX6ULL rev1.1 792 MHz (running at 396 MHz)
4 CPU: Industrial temperature grade (-40C to 105C) at 51C
5 Reset cause: POR
6 Board: I.MX6U ALPHA|MINI
7 I2C: ready
8 DRAM: 512 MiB
9 MMC: FSL_SDHC: 0, FSL_SDHC: 1
10 Display: ATK-LCD-7-1024x600 (1024x600)
11 Video: 1024x600x24
12 In: serial
13 Out: serial
14 Err: serial
15 switch to partitions #0, OK
16 mmc1(part 0) is current device
17 Net: FEC1
18 Error: FEC1 address not set.
19
20 Normal Boot
21 Hit any key to stop autoboot: 0
22 =>
```

第 1 行是 uboot 版本号和编译时间,可以看出,当前的 uboot 版本号是 2016.03,编译时间是 2020 年 8 月 7 日凌晨 20 点 47 分。

第 3 和第 4 行是 CPU 信息,可以看出当前使用的 CPU 是飞思卡尔的 I.MX6ULL (I.MX 以前属于飞思卡尔,然而飞思卡尔被 NXP 收购了),频率为 792MHz,但是此时运行在 396MHz。这颗芯片是工业级的,结温为-40° C~105° C。

第 5 行是复位原因,当前的复位原因是 POR。I.MX6ULL 芯片上有个 POR_B 引脚,将这个引脚拉低即可复位 I.MX6ULL。

第 6 行是板子名字,当前的板子名字为“I.MX6U ALPHA|MINI”。

第 7 行提示 I2C 准备就绪。

第 8 行提示当前板子的 DRAM(内存)为 512MB,如果是 NAND 版本的话内存为 256MB。

第 9 行提示当前有两个 MMC/SD 卡控制器: FSL_SDHC(0)和 FSL_SDHC(1)。I.MX6ULL 支持两个 MMC/SD,正点原子的 I.MX6ULL EMMC 核心板上 FSL_SDHC(0)接的 SD(TF)卡, FSL_SDHC(1)接的 EMMC。

第 10 和第 11 行是 LCD 型号,当前的 LCD 型号是 ATK-LCD-7-1024x600 (1024x600),分辨率为 1024x600,格式为 RGB888(24 位)。

第 12~14 是标准输入、标准输出和标准错误所使用的终端,这里都使用串口(serial)作为终端。

第 15 和 16 行是切换到 emmc 的第 0 个分区上, 因为当前的 uboot 是 emmc 版本的, 也就是从 emmc 启动的。我们只是为了方便将其烧写到了 SD 卡上, 但是它的“内心”还是 EMMC 的。所以 uboot 启动以后会将 emmc 作为默认存储器, 当然了, 你也可以将 SD 卡作为 uboot 的存储器, 这个我们后面会讲解怎么做。

第 17 行是网口信息, 提示我们当前使用的 FEC1 这个网口, I.MX6ULL 支持两个网口。

第 18 行提示 FEC1 网卡地址没有设置, 后面我们会讲解如何在 uboot 里面设置网卡地址。

第 20 行提示正常启动, 也就是说 uboot 要从 emmc 里面读取环境变量和参数信息启动 Linux 内核了。

第 21 行是倒计时提示, 默认倒计时 3 秒, 倒计时结束之前按下回车键就会进入 Linux 命令行模式。如果在倒计时结束以后没有按下回车键, 那么 Linux 内核就会启动, Linux 内核一旦启动, uboot 就会寿终正寝。

这个就是 uboot 默认输出信息的含义, NAND 版本的 uboot 也是类似的, 只是 NAND 版本的就没有 EMMC/SD 相关信息了, 取而代之的就是 NAND 的信息, 比如 NAND 容量大小信息。

uboot 是来干活的, 我们现在已经进入 uboot 的命令行模式了, 进入命令行模式以后就可以给 uboot 发号施令了。当然了, 不能随便发号施令, 得看看 uboot 支持哪些命令, 然后使用这些 uboot 所支持的命令来做一些工作。下一节就讲解 uboot 命令的使用。

30.4 U-Boot 命令使用

进入 uboot 的命令行模式以后输入“help”或者“?”, 然后按下回车即可查看当前 uboot 所支持的命令, 如图 30.4.1 所示:

```

=> help
? - alias for 'help'
base - print or set address offset
bdinfo - print Board Info structure
bmode - sd|sd2|qspi|normal|usb|sata|ecspi1:0|ecspi1:1|ecspi1:2|ecspi1:3|esdhc1|esdhc2|esdhc3|esdhc4 [noreset]
bmp - manipulate BMP image data
boot - boot default, i.e., run 'bootcmd'
bootd - boot default, i.e., run 'bootcmd'
bootelf - Boot from an ELF image in memory
bootm - boot application image from memory
bootp - boot image via network using BOOTP/TFTP protocol
bootvx - Boot vxWorks from an ELF image
  
```

图 30.4.1 uboot 命令列表

图 30.4.1 中只是 uboot 的一部分命令, 具体的命令列表以实际为准。图 30.4.1 中的命令并不是 uboot 所支持的所有命令, 前面说过 uboot 是可配置的, 需要什么命令就使能什么命令。所以图 30.4.1 中的命令是正点原子提供的 uboot 中使能的命令, uboot 支持的命令还有很多, 而且也可以在 uboot 中自定义命令。这些命令后面都跟有命令说明, 用于描述此命令的作用, 但是命令具体怎么用呢? 我们输入“help(或?) 命令名”既可以查看命令的详细用法, 以“bootz”这个命令为例, 我们输入如下命令即可查看“bootz”这个命令的用法:

```
? bootz 或 help bootz
```

结果如图 30.4.2 所示:

```

=> ? bootz
bootz - boot Linux zImage image from memory

Usage:
bootz [addr [initrd[:size]] [fdt]]
- boot Linux zImage stored in memory
  The argument 'initrd' is optional and specifies the address
  of the initrd in memory. The optional argument ':size' allows
  specifying the size of RAW initrd.
  When booting a Linux kernel which requires a flat device-tree
  a third argument is required which is the address of the
  device-tree blob. To boot that kernel without an initrd image,
  use a '-' for the second argument. If you do not pass a third
  a bd_info struct will be passed instead
=>

```

图 30.4.2 bootz 命令使用说明

图 30.4.2 中就详细的列出了“bootz”这个命令的详细，其它的命令也可以使用此方法查询具体的使用方法。接下来我们学习一下一些常用的 uboot 命令。

30.4.1 信息查询命令

常用的和信息查询有关的命令有 3 个：bdinfo、printenv 和 version。先来看一下 bdinfo 命令，此命令用于查看板子信息，直接输入“bdinfo”即可，结果如图 30.4.1.1 所示：

```

=> bdinfo
arch_number = 0x00000000
boot_params = 0x80000100
DRAM_bank   = 0x00000000
-> start    = 0x80000000
-> size      = 0x20000000
eth0name     = FEC1
ethaddr      = (not set)
current eth   = FEC1
ip_addr       = <NULL>
baudrate      = 115200 bps
TLB_addr     = 0x9FFF0000
relocaddr     = 0x9FF51000
reloc off    = 0x18751000
irq_sp       = 0x9EF4EEA0
sp_start     = 0x9EF4EE90
FB_base      = 0x00000000
=>

```

图 30.4.1.1 bdinfo 命令

从图 30.4.1.1 中可以得出 DRAM 的起始地址和大小、启动参数保存起始地址、波特率、sp(堆栈指针)起始地址等信息。

命令“printenv”用于输出环境变量信息，uboot 也支持 TAB 键自动补全功能，输入“print”然后按下 TAB 键就会自动补全命令，直接输入“print”也可以。输入“print”，然后按下回车键，环境变量如图 30.4.1.2 所示：

```
=> print
baudrate=115200
board_name=EVK
board_rev=14X14
boot_fdt=try
bootcmd=run findfdt;mmc dev ${mmcdev};mmc dev ${mmcdev}; if mmc rescan; then if run loadbootscript
; then run bootscript; else if run loadimage; then run mmcboot; else run netboot; fi; fi; else run
netboot; fi
bootcmd_mfg=run mfgtool_args;bootz ${loadaddr} ${initrd_addr} ${fdt_addr};
bootdelay=1
bootscript=echo Running bootscript from mmc ...; source
console=ttyMXC0
ethact=FEC1
ethprime=FEC
fdt_addr=0x83000000
fdt_file=imx6ull-14x14-emmc-7-1024x600-c.dtb
fdt_high=0xffffffff
findfdt=if test $fdt_file = undefined; then if test $board_name = EVK && test $board_rev = 9X9; th
en setenv fdt_file imx6ull-9x9-evk.dtb; fi; if test $board_name = EVK && test $board_rev = 14X14;
then setenv fdt_file imx6ull-14x14-evk.dtb; fi; if test $fdt_file = undefined; then echo WARNING:
Could not determine dtb to use; fi; fi;
image=zImage
initrd_addr=0x83800000
initrd_high=0xffffffff
ip_dyn=yes
loadaddr=0x80800000
loadbootscript=fatload mmc ${mmcdev}:${mmcpart} ${loadaddr} ${script};
loadfdt=fatload mmc ${mmcdev}:${mmcpart} ${fdt_addr} ${fdt_file}
loadimage=fatload mmc ${mmcdev}:${mmcpart} ${loadaddr} ${image}
logo_file=alienek.bmp
mfgtool_args=setenv bootargs console=${console},${baudrate} rdinit=/linuxrc g_mass_storage.stall=0
g_mass_storage.removable=1 g_mass_storage.file=/fat g_mass_storage.ro=1 g_mass_storage.idVendor=0
x066F g_mass_storage.idProduct=0x37FF g_mass_storage.iSerialNumber="" clk_ignore_unused
mmcargs=setenv bootargs console=${console},${baudrate} root=${mmccroot}
mmcautodetect=yes
mmcboot=echo Booting from mmc ...; run mmcargs; if test ${boot_fdt} = yes || test ${boot_fdt} = tr
y; then if run loadfdt; then bootz ${loadaddr} - ${fdt_addr}; else if test ${boot_fdt} = try; then
bootz; else echo WARN: Cannot load the DT; fi; fi; else bootz; fi;
mmcdev=1
mmcpart=1
mmccroot=/dev/mmcblkp2 rootwait rw
netargs=setenv bootargs console=${console},${baudrate} root=/dev/nfs ip=dhcp nfsroot=${serverip}:$
{nfsroot},v3,tcp
netboot=echo Booting from net ...; run netargs; if test ${ip_dyn} = yes; then setenv get_cmd dhcp;
else setenv get_cmd tftp; fi; ${get_cmd} ${image}; if test ${boot_fdt} = yes || test ${boot_fdt}
= try; then if ${get_cmd} ${fdt_addr} ${fdt_file}; then bootz ${loadaddr} - ${fdt_addr}; else if t
est ${boot_fdt} = try; then bootz; else echo WARN: Cannot load the DT; fi; fi; else bootz; fi;
panel=ATK-LCD-7-1024x600
script=boot.scr
splashimage=0x88000000
splashpos=m,m

Environment size: 2534/8188 bytes
=>
```

图 30.4.1.2 printenv 命令结果

在图 30.4.1.2 中有很多的环境变量,比如 baudrate、board_name、board_rev、boot_fdt、bootcmd 等等。uboot 中的环境变量都是字符串,既然叫做环境变量,那么它的作用就和“变量”一样。比如 bootdelay 这个环境变量就表示 uboot 启动延时时间,默认 bootdelay=3,也就默认延时 3 秒。前面说的 3 秒倒计时就是由 bootdelay 定义的,如果将 bootdelay 改为 5 的话就会倒计时 5s 了。uboot 中的环境变量是可以修改的,有专门的命令来修改环境变量的值,稍后我们会讲解。

命令 version 用于查看 uboot 的版本号,输入“version”,uboot 版本号如图 30.4.1.3 所示:

```
=> version

U-Boot 2016.03-gd3f0479 (Aug 07 2020 - 20:47:37 +0800)
arm-poky-linux-gnueabi-gcc (GCC) 5.3.0
GNU ld (GNU Binutils) 2.26.0.20160214
=>
```


图 30.4.1.3 version 命令结果

从图 30.4.1.3 可以看出, 当前 uboot 版本号为 2016.03, 2020 年 8 月 7 日编译的, 编译器为 arm-poky-linux-gnueabi-gcc, 这是 NXP 官方提供的编译器, 正点原子出厂系统用的此编译器编译的, 但是本教程我们统一使用 arm-linux-gnueabi-gcc。

30.4.2 环境变量操作命令

1、修改环境变量

环境变量的操作涉及到两个命令: setenv 和 saveenv, 命令 setenv 用于设置或者修改环境变量的值。命令 saveenv 用于保存修改后的环境变量, 一般环境变量是存放在外部 flash 中的, uboot 启动的时候会将环境变量从 flash 读取到 DRAM 中。所以使用命令 setenv 修改的是 DRAM 中的环境变量值, 修改以后要使用 saveenv 命令将修改后的环境变量保存到 flash 中, 否则的话 uboot 下一次重启会继续使用以前的环境变量值。

命令 saveenv 使用起来很简单, 格式为:

```
saveenv
```

比如我们要将环境变量 bootdelay 改为 5, 就可以使用如下所示命令:

```
setenv bootdelay 5
```

```
saveenv
```

上述命令执行过程如图 30.4.2.1 所示:

```
=> setenv bootdelay 5
=> saveenv
Saving Environment to MMC...
Writing to MMC(0)... done
=> █
```

图 30.4.2.1 环境变量修改

在图 30.4.2.1 中, 当我们使用命令 saveenv 保存修改后的环境变量的话会有保存过程提示信息, 根据提示可以看出环境变量保存到了 MMC(0)中, 也就是 SD 卡中。因为我们现在将 uboot 烧写到了 SD 卡里面, 所以会保存到 MMC(0)中。如果烧写到 EMMC 里面就会提示保存到 MMC(1), 也就是 EMMC 设备, 同理, 如果是 NAND 版本核心板的话就会提示保存到 NAND 中。

修改 bootdelay 以后, 重启开发板, uboot 就是变为 5 秒倒计时, 如图 30.4.2.2 所示:

```
U-Boot 2016.03 (Mar 12 2020 - 15:11:51 +0800)

CPU:   Freescale i.MX6ULL rev1.1 69 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 52C
Reset cause: POR
Board: MX6ULL ALIENTEK EMMC
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:    serial
Out:   serial
Err:   serial
switch to partitions #0, OK
mmc0 is current device
Net:   FEC1
Normal Boot
Hit any key to stop autoboot:  5 █
```

图 30.4.2.2 5 秒倒计时

有时候我们修改的环境变量值可能会有空格，比如 `bootcmd`、`bootargs` 等，这个时候环境变量值就得用单引号括起来，比如下面修改环境变量 `bootargs` 的值：

```
setenv bootargs 'console=ttymxc0,115200 root=/dev/mmcblk1p2 rootwait rw'
saveenv
```

上面命令设置 `bootargs` 的值为“`console=ttymxc0,115200 root=/dev/mmcblk1p2 rootwait rw`”，其中“`console=ttymxc0,115200`”、“`root=/dev/mmcblk1p2`”、“`rootwait`”和“`rw`”相当于四组“值”，这四组“值”之间用空格隔开，所以需要使用单引号 ‘ ’ 将其括起来，表示这四组“值”都属于环境变量 `bootargs`。

2、新建环境变量

命令 `setenv` 也可以用于新建命令，用法和修改环境变量一样，比如我们新建一个环境变量 `author`，`author` 的值为我的名字拼音：`zuozhongkai`，那么就可以使用如下命令：

```
setenv author zuozhongkai
saveenv
```

新建命令 `author` 完成以后重启 `uboot`，然后使用命令 `printenv` 查看当前环境变量，如图 30.4.2.3 所示：

```
=> print
author=zuozhongkai
baudrate=115200
board_name=EVK
board_rev=14X14
boot_fdt=try
```

图 30.4.2.3 环境变量

从图 30.4.2.3 可以看到新建的环境变量：`author`，其值为：`zuozhongkai`。

3、删除环境变量

既然可以新建环境变量，肯定也可以删除环境变量，删除环境变量也是使用命令 `setenv`，要删除一个环境变量只要给这个环境变量赋空值即可，比如我们删除掉上面新建的 `author` 这个环境变量，命令如下：

```
setenv author
saveenv
```

上面命令中通过 `setenv` 给 `author` 赋空值，也就是什么都不写来删除环境变量 `author`。重启 `uboot` 就会发现环境变量 `author` 没有了。

30.4.3 内存操作命令

内存操作命令就是用于直接对 `DRAM` 进行读写操作的，常用的内存操作命令有 `md`、`nm`、`mm`、`mw`、`cp` 和 `cmp`。我们依次来看一下这些命令都是做什么的。

1、md 命令

`md` 命令用于显示内存值，格式如下：

```
md[.b, .w, .l] address [# of objects]
```

命令中的 `[.b .w .l]` 对应 `byte`、`word` 和 `long`，也就是分别以 1 个字节、2 个字节、4 个字节来显示内存值。`address` 就是要查看的内存起始地址，`[# of objects]` 表示要查看的数据长度，这

个数据长度单位不是字节，而是跟你所选择的显示格式有关。比如你设置要查看的内存长度为 20(十六进制为 0x14)，如果显示格式为.b 的话那就表示 20 个字节；如果显示格式为.w 的话就表示 20 个 word，也就是 $20 \times 2 = 40$ 个字节；如果显示格式为.l 的话就表示 20 个 long，也就是 $20 \times 4 = 80$ 个字节。另外要注意：

uboot 命令中的数字都是十六进制的！不是十进制的！

比如你想查看以 0X80000000 开始的 20 个字节的内存值，显示格式为.b 的话，应该使用如下所示命令：

```
md.b 80000000 14
```

而不是：

```
md.b 80000000 20
```

上面说了，uboot 命令里面的数字都是十六进制的，所以可以不用写“0x”前缀，十进制的 20 其十六进制为 0x14，所以命令 md 后面的个数应该是 14，如果写成 20 的话就表示查看 32(十六进制为 0x20)个字节的数据。分析下面三个命令的区别：

```
md.b 80000000 10
```

```
md.w 80000000 10
```

```
md.l 80000000 10
```

上面这三个命令都是查看以 0X80000000 为起始地址的内存数据，第一个命令以.b 格式显示，长度为 0x10，也就是 16 个字节；第二个命令以.w 格式显示，长度为 0x10，也就是 $16 \times 2 = 32$ 个字节；最后一个命令以.l 格式显示，长度也是 0x10，也就是 $16 \times 4 = 64$ 个字节。这三个命令的执行结果如图 30.4.3.1 所示：

```
=> md.b 80000000 10
80000000: 31 e0 00 05 04 40 10 12 06 98 00 0b 3c 00 78 2e    1....@.....<.x.
=> md.w 80000000 10
80000000: e031 0500 4004 1210 9806 0b00 003c 2e78    1....@.....<.x.
80000010: 890b 0404 8110 2047 0404 6009 3300 2008    .....G ...`.3.
=> md.l 80000000 10
80000000: 0500e031 12104004 0b009806 2e78003c    1....@.....<.x.
80000010: 0404890b 20478110 60090404 20083300    .....G ...`.3.
80000020: e04b80f0 c0400200 24422000 81400401    ..K...@.. B$..@.
80000030: c4042402 0120e40d 4110700d 4000700c    .$.... ..p.A.p.@
=>
```

图 30.4.3.1 md 命令使用示例

2、nm 命令

nm 命令用于修改指定地址的内存值，命令格式如下：

```
nm [.b, .w, .l] address
```

nm 命令同样可以以.b、.w 和.l 来指定操作格式，比如现在以.l 格式修改 0x80000000 地址的数据为 0x12345678。输入命令：

```
nm.l 80000000
```

输入上述命令以后如图 30.4.3.2 所示：

```
=> nm.l 80000000
80000000: 0500e031 ?
```

图 30.4.3.2 nm 命令

在图 30.4.3.2 中，80000000 表示现在要修改的内存地址，0500e031 表示地址 0x80000000 现在的数据，? 后面就可以输入要修改后的数据 0x12345678，输入完成以后按下回车，然后再输入‘q’即可退出，如图 30.4.3.3 所示：

```
=> nm.l 80000000
80000000: 0500e031 ? 12345678
80000000: 12345678 ? q
=> █
```

图 30.4.3.3 修改内存数据

修改完成以后在使用命令 md 来查看一下有没有修改成功，如图 30.4.3.4 所示：

```
=> md.l 80000000 1
80000000: 12345678 xV4.
=> █
```

图 30.4.3.4 查看修改后的值

从图 30.4.3.4 可以看出，此时地址 0X80000000 的值变为了 0x12345678。

3、mm 命令

mm 命令也是修改指定地址内存值的，使用 mm 修改内存值的时候地址会自增，而使用命令 nm 的话地址不会自增。比如以 l 格式修改从地址 0x80000000 开始的连续 3 个内存块(3*4=12 个字节)的数据为 0X05050505，操作如图 30.4.3.5 所示：

```
=> mm.l 80000000
80000000: 15d84584 ? 05050505
80000004: 9211440c ? 05050505
80000008: 2b009906 ? 05050505
8000000c: ae78903f ? q
=> █
```

图 30.4.3.5 命令 mm

从图 30.4.3.5 可以看出，修改了地址 0X80000000、0X80000004 和 0X8000000C 的内容为 0x05050505。使用命令 md 查看修改后的值，结果如图 30.4.3.6 所示：

```
=> md.l 80000000 3
80000000: 05050505 05050505 05050505 .....
=> █
```

图 30.4.3.6 查看修改后的内存数据

从图 30.4.3.6 可以看出内存数据修改成功。

4、mw 命令

命令 mw 用于使用一个指定的数据填充一段内存，命令格式如下：

```
mw [.b, .w, .l] address value [count]
```

mw 命令同样可以以 .b、.w 和 .l 来指定操作格式，address 表示要填充的内存起始地址，value 为要填充的数据，count 是填充的长度。比如使用 l 格式将以 0X80000000 为起始地址的 0x10 个内存块(0x10 * 4=64 字节)填充为 0X0A0A0A0A，命令如下：

```
mw.l 80000000 0A0A0A0A 10
```

然后使用命令 md 来查看，如图 30.4.3.7 所示：

```
=> mw.l 80000000 0A0A0A0A 10
=> md.l 80000000 10
80000000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
=> █
```

图 30.4.3.7 查看修改后的内存数据

从图 30.4.3.7 可以看出内存数据修改成功。

5、cp 命令

cp 是数据拷贝命令，用于将 DRAM 中的数据从一段内存拷贝到另一段内存中，或者把 Nor Flash 中的数据拷贝到 DRAM 中。命令格式如下：

```
cp [.b, .w, .l] source target count
```

cp 命令同样可以以 .b、.w 和 .l 来指定操作格式，source 为源地址，target 为目的地址，count 为拷贝的长度。我们使用 .l 格式将 0x80000000 处的地址拷贝到 0x80000100 处，长度为 0x10 个内存块(0x10 * 4=64 个字节)，命令如下所示：

```
cp.l 80000000 80000100 10
```

结果如图 30.4.3.8 所示：

```
=> md.l 80000000 10
80000000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
=> md.l 80000100 10
80000100: 50050f02 3304990b 600a9310 600a1201 ...P...3...`...`
80000110: 42922a03 ac98746f e4d02004 d0460544 ...*.Bot... ..D.F.
80000120: a2079b06 1000f818 60131a80 00077003 .....`.p..
80000130: 0305f023 42901a02 f0c04402 d5262000 #.....B.D... &.
=> cp.l 80000000 80000100 10
=> md.l 80000100 10
80000100: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000110: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000120: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
80000130: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....
=>
```

图 30.4.3.8 cp 命令操作结果

在图 30.4.3.8 中，先使用 md.l 命令打印出地址 0x80000000 和 0x80000100 处的数据，然后使用命令 cp.l 将 0x80000100 处的数据拷贝到 0x80000100 处。最后使用命令 md.l 查看 0x80000100 处的数据有没有变化，检查拷贝是否成功。

6、cmp 命令

cmp 是比较命令，用于比较两段内存的数据是否相等，命令格式如下：

```
cmp [.b, .w, .l] addr1 addr2 count
```

cmp 命令同样可以以 .b、.w 和 .l 来指定操作格式，addr1 为第一段内存首地址，addr2 为第二段内存首地址，count 为要比较的长度。我们使用 .l 格式来比较 0x80000000 和 0x80000100 这两个地址数据是否相等，比较长度为 0x10 个内存块(16 * 4=64 个字节)，命令如下所示：

```
cmp.l 80000000 80000100 10
```

结果如图 30.4.3.9 所示：

```
=> cmp.l 80000000 80000100 10
Total of 16 word(s) were the same
=>
```

图 30.4.3.9 cmp 命令比较结果

从图 30.4.3.9 可以看出两段内存的数据相等。我们再随便挑两段内存比较一下，比如地址 0x80002000 和 0x800003000，长度为 0x10，比较结果如图 30.4.3.10 所示：

```

=> cmp.l 80002000 80003000 10
word at 0x80002000 (0xf0c06f00) != word at 0x80003000 (0x3fc0f414)
Total of 0 word(s) were the same
=> █
    
```

图 30.4.3.10 cmp 命令比较结果

从图 30.4.3.10 可以看出，0x80002000 处的数据和 0x80003000 处的数据就不一样。

30.4.4 网络操作命令

uboot 是支持网络的，我们在移植 uboot 的时候一般都要调通网络功能，因为在移植 linux kernel 的时候需要使用到 uboot 的网络功能做调试。uboot 支持大量的网络相关命令，比如 dhcp、ping、nfs 和 tftpboot，我们接下来依次学习一下这几个和网络有关的命令。

在使用 uboot 的网络功能之前先用网线将开发板的 ENET2 接口和电脑或者路由器连接起来，I.MX6U-ALPHA 开发板有两个网口：ENET1 和 ENET2，一定要连接 ENET2，不能连接错了，ENET2 接口如图 30.4.4.1 所示。

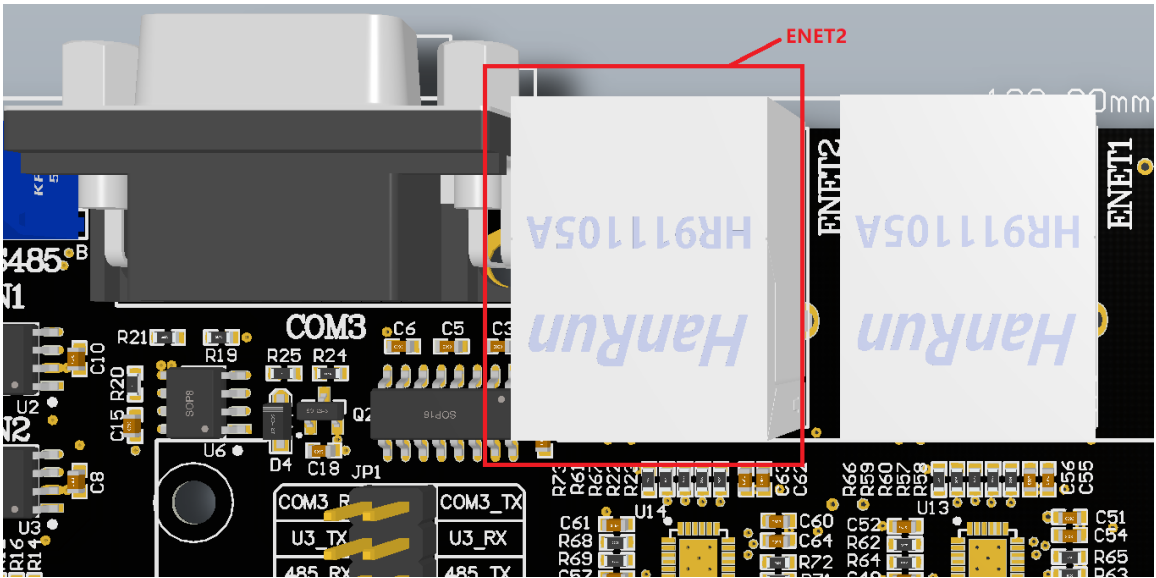


图 30.4.4.1 ENET2 网络接口

建议开发板和主机 PC 都连接到同一个路由器上！最后设置表 30.4.4.1 中所示的几个环境变量。

环境变量	描述
ipaddr	开发板 ip 地址，可以不设置，使用 dhcp 命令来从路由器获取 IP 地址。
ethaddr	开发板的 MAC 地址，一定要设置。
gatewayip	网关地址。
netmask	子网掩码。
serverip	服务器 IP 地址，也就是 Ubuntu 主机 IP 地址，用于调试代码。

表 30.4.4.1 网络相关环境变量

表 30.4.4.1 中环境变量设置命令如下所示：

```

setenv ipaddr 192.168.1.50
setenv ethaddr b8:ae:1d:01:00:00
setenv gatewayip 192.168.1.1
setenv netmask 255.255.255.0
    
```



```
setenv serverip 192.168.1.253
saveenv
```

注意，网络地址环境变量的设置要根据自己的实际情况，确保 Ubuntu 主机和开发板的 IP 地址在同一个网段内，比如我现在的开发板和电脑都在 192.168.1.0 这个网段内，所以设置开发板的 IP 地址为 192.168.1.50，我的 Ubuntu 主机的地址为 192.168.1.253，因此 serverip 就是 192.168.1.253。ethaddr 为网络 MAC 地址，是一个 48bit 的地址，如果在同一个网段内有多个开发板的话一定要保证每个开发板的 ethaddr 是不同的，否则通信会有问题！设置好网络相关的环境变量以后就可以使用网络相关命令了。

1、ping 命令

开发板的网络能否使用，是否可以和服务端(Ubuntu 主机)进行通信，通过 ping 命令就可以验证，直接 ping 服务器的 IP 地址即可，比如我的服务器 IP 地址为 192.168.1.253，命令如下：

```
ping 192.168.1.253
```

结果如图 30.4.4.2 所示：

```
=> ping 192.168.1.253
Using FEC1 device
host 192.168.1.253 is alive
=> █
```

图 30.4.4.2 ping 命令

从图 30.4.4.2 可以看出，192.168.1.253 这个主机存在，说明 ping 成功，uboot 的网络工作正常。

注意！只能在 uboot 中 ping 其他的机器，其他机器不能 ping uboot，因为 uboot 没有对 ping 命令做处理，如果用其他的机器 ping uboot 的话会失败！

2、dhcp 命令

dhcp 用于从路由器获取 IP 地址，前提得开发板连接到路由器上的，如果开发板是和电脑直连的，那么 dhcp 命令就会失效。直接输入 dhcp 命令即可通过路由器获取到 IP 地址，如图 30.4.4.3 所示：

```
=> dhcp
BOOTP broadcast 1
BOOTP broadcast 2
BOOTP broadcast 3
BOOTP broadcast 4
BOOTP broadcast 5
DHCP client bound to address 192.168.1.137 (7962 ms)
*** Warning: no boot file name; using 'C0A80189.img'
Using FEC1 device
TFTP from server 192.168.1.1; our IP address is 192.168.1.137
Filename 'C0A80189.img'.
Load address: 0x80800000
Loading: T T T T █
```

图 30.4.4.3 dhcp 命令

从图 30.4.4.3 可以看出，开发板通过 dhcp 获取到的 IP 地址为 192.168.1.137。同时在图 30.4.4.3 中可以看到“warning: no boot file name;”、“TFTP from server 192.168.1.1”这样的字样。这是因为 DHCP 不单单是获取 IP 地址，其还会通过 TFTP 来启动 linux 内核，输入“? dhcp”即可查看 dhcp 命令详细的信息，如图 30.4.4.4 所示：

```
=> ? dhcp
dhcp - boot image via network using DHCP/TFTP protocol

Usage:
dhcp [loadAddress] [[hostIPAddr:]bootfilename]
=> █
```

图 30.4.4.4 dhcp 命令使用查询

3、nfs 命令

nfs(Network File System)网络文件系统，通过 nfs 可以在计算机之间通过网络来分享资源，比如我们将 linux 镜像和设备树文件放到 Ubuntu 中，然后在 uboot 中使用 nfs 命令将 Ubuntu 中的 linux 镜像和设备树下载到开发板的 DRAM 中。这样做的目的是为了更方便调试 linux 镜像和设备树，也就是网络调试，通过网络调试是 Linux 开发中最常用的调试方法。原因是嵌入式 linux 开发不像单片机开发，可以直接通过 JLINK 或 STLink 等仿真器将代码直接烧写到单片机内部的 flash 中，嵌入式 Linux 通常是烧写到 EMMC、NAND Flash、SPI Flash 等外置 flash 中，但是嵌入式 Linux 开发也没有 MDK, IAR 这样的 IDE，更没有烧写算法，因此不可能通过点击一个“download”按钮就将固件烧写到外部 flash 中。虽然半导体厂商一般都会提供一个烧写固件的软件，但是这个软件使用起来比较复杂，这个烧写软件一般用于量产的。其远没有 MDK、IAR 的一键下载方便，在 Linux 内核调试阶段，如果用这个烧写软件的话将会非常浪费时间，而这个时候网络调试的优势就显现出来了，可以通过网络将编译好的 linux 镜像和设备树文件下载到 DRAM 中，然后就可以直接运行。

我们一般使用 uboot 中的 nfs 命令将 Ubuntu 中的文件下载到开发板的 DRAM 中，在使用之前需要开启 Ubuntu 主机的 NFS 服务，并且要新建一个 NFS 使用的目录，以后所有要通过 NFS 访问的文件都需要放到这个 NFS 目录中。Ubuntu 的 NFS 服务开启我们在 4.2.1 小节已经详细讲解过了，包括 NFS 文件目录的创建，如果忘记的话可以去查看一下 4.2.1 小节。我设置的 `/home/zuozhongkai/linux/nfs` 这个目录为我的 NFS 文件目录。uboot 中的 nfs 命令格式如下所示：

```
nfs [loadAddress] [[hostIPAddr:]bootfilename]
```

loadAddress 是要保存的 DRAM 地址，[[hostIPAddr:]bootfilename]是要下载的文件地址。这里我们将正点原子官方编译出来的 Linux 镜像文件 zImage 下载到开发板 DRAM 的 0x80800000 这个地址处。正点原子编译出来的 zImage 文件已经放到了开发板光盘中，路径为：**8、系统镜像->1、出厂系统镜像->2、kernel 镜像linux-imx-4.1.15-2.1.0-gbfed875-v1.6 ->zImage**。将文件 zImage 通过 FileZilla 发送到 Ubuntu 中的 NFS 目录下，比如我的就是放到 `/home/zuozhongkai/linux/nfs` 这个目录下，完成以后的 NFS 目录如图 30.4.4.5 所示：

```
zuozhongkai@ubuntu:~/linux/nfs$ ls -l
总用量 3598528
drwxr-xr-x 20 zuozhongkai zuozhongkai 4096 Mar 26 12:23 debian_rootfs
-rw-r--r-- 1 zuozhongkai zuozhongkai 37162 Apr 15 12:39 imx6ul-14x14-evk.dtb
-rw-r--r-- 1 zuozhongkai zuozhongkai 36626 Mar 26 12:09 imx6ull-14x14-evk.dtb
drwxr-xr-x 30 zuozhongkai zuozhongkai 4096 Apr 16 18:21 imx_rootfs
drwxr-xr-x 3 zuozhongkai zuozhongkai 4096 Mar 26 12:25 imx_rootfs_alsa
-rw-r--r-- 1 zuozhongkai zuozhongkai 526147441 Mar 26 12:11 imx_rootfs_alsa.tar.bz2
-rw-r--r-- 1 zuozhongkai zuozhongkai 529069027 Mar 26 12:10 imx_rootfs.tar.bz2
-rw-r--r-- 1 zuozhongkai zuozhongkai 625374408 Mar 26 12:11 linaro-jessie-alip-20160428-22.tar.gz
drwxr-xr-x 19 zuozhongkai zuozhongkai 4096 Mar 26 12:24 nxp_rootfs
-rw-r--r-- 1 zuozhongkai zuozhongkai 1986363826 Mar 26 12:10 rootfs_nogpu.tar.bz2
drwxr-xr-x 23 zuozhongkai zuozhongkai 4096 Mar 26 12:21 ubuntu_rootfs
-rw-r--r-- 1 zuozhongkai zuozhongkai 6071136 Apr 18 12:18 zImage
-rwxr-xr-x 1 zuozhongkai zuozhongkai 6071136 Apr 17 18:17 zImage_imx6ul
-rwxr-xr-x 1 zuozhongkai zuozhongkai 5662880 Mar 26 12:09 zImage_imx6ull
```

图 30.4.4.5 NFS 目录中的 zImage 文件

准备好以后就可以使用 `nfs` 命令来将 `zImage` 下载到开发板 DRAM 的 `0X80800000` 地址处, 命令如下:

```
nfs 80800000 192.168.1.253:/home/zuozhongkai/linux/nfs/zImage
```

命令中的“80800000”表示 zImage 保存地址，“192.168.1.253:/home/zuozhongkai/linux/nfs/zImage”表示 zImage 在 192.168.1.253 这个主机中，路径为/home/zuozhongkai/linux/nfs/zImage。下载过程如图 30.4.4.6 所示：

[illegible]

图 30.4.4.6 nfs 命令下载 zImage 过程

在图 30.4.4.6 中会以“#”提示下载过程，下载完成以后会提示下载的数据大小，这里下载的 6785272 字节(出厂系统在不断的更更新中，因此以实际的 zImage 大小为准)，而 zImage 的大小就是 6785272 字节，如图 30.4.4.7 所示：

```
zuozhongkai@ubuntu:~/linux/nfs$ ls zImage -l
-rw----- 1 zuozhongkai zuozhongkai 6785272 Mar 22 16:43 zImage
zuozhongkai@ubuntu:~/linux/nfs$
```

图 30.4.4.7 zImage 大小

下载完成以后查看 0x80800000 地址处的数据，使用命令 `md.b` 来查看前 0x100 个字节的数据，如图 30.4.4.8 所示：

```
=> md.b 80800000 100
80800000: 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 .....
80800010: 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 .....
80800020: 03 00 00 ea 18 28 6f 01 00 00 00 00 f8 88 67 00 .....(o.....g.
80800030: 01 02 03 04 00 90 0f e1 e8 04 00 eb 01 70 a0 e1 .....p..
80800040: 02 80 a0 e1 00 20 0f e1 03 00 12 e3 01 00 00 1a .....
80800050: 17 00 a0 e3 56 34 12 ef 00 00 0f e1 1a 00 20 e2 ....V4.....
80800060: 1f 00 10 e3 1f 00 c0 e3 d3 00 80 e3 04 00 00 1a .....
80800070: 01 0c 80 e3 0c e0 8f e2 00 f0 6f e1 0e f3 2e e1 .....o....
80800080: 6e 00 60 e1 00 f0 21 e1 09 f0 6f e1 00 00 00 00 n.....!...o....
80800090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
808000a0: 0f 40 a0 e1 3e 43 04 e2 02 49 84 e2 0f 00 a0 e1 .@..>C...I.....
808000b0: 04 00 50 e1 ac 01 9f 35 0f 00 80 30 00 00 54 31 ..P....5...0..T1
808000c0: 01 40 84 33 6d 00 00 2b 5e 0f 8f e2 4e 1c 90 e8 .@.3m..+^...N...
808000d0: 1c d0 90 e5 01 00 40 e0 00 60 86 e0 00 a0 8a e0 .....@..
808000e0: 00 90 da e5 01 e0 da e5 0e 94 89 e1 02 e0 da e5 .....
808000f0: 03 a0 da e5 0e 98 89 e1 0a 9c 89 e1 00 d0 8d e0 .....
=> █
```

图 30.4.4.8 下载的数据

在使用 winhex 软件来查看 zImage，检查一下前面的数据是否和图 30.4.4.8 只的一致，结果如图 30.4.4.9 所示：

zImage																
Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00000000	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1
00000010	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1
00000020	03	00	00	EA	18	28	6F	01	00	00	00	00	F8	88	67	00
00000030	01	02	03	04	00	90	0F	E1	E8	04	00	EB	01	70	A0	E1
00000040	02	80	A0	E1	00	20	0F	E1	03	00	12	E3	01	00	00	1A
00000050	17	00	A0	E3	56	34	12	EF	00	00	0F	E1	1A	00	20	E2
00000060	1F	00	10	E3	1F	00	C0	E3	D3	00	80	E3	04	00	00	1A
00000070	01	0C	80	E3	0C	E0	8F	E2	00	F0	6F	E1	0E	F3	2E	E1
00000080	6E	00	60	E1	00	F0	21	E1	09	F0	6F	E1	00	00	00	00
00000090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000000A0	0F	40	A0	E1	3E	43	04	E2	02	49	84	E2	0F	00	A0	E1
000000B0	04	00	50	E1	AC	01	9F	35	0F	00	80	30	00	00	54	31
000000C0	01	40	84	33	6D	00	00	2B	5E	0F	8F	E2	4E	1C	90	E8
000000D0	1C	D0	90	E5	01	00	40	E0	00	60	86	E0	00	A0	8A	E0
000000E0	00	90	DA	E5	01	E0	DA	E5	0E	94	89	E1	02	E0	DA	E5
000000F0	03	A0	DA	E5	0E	98	89	E1	0A	9C	89	E1	00	D0	8D	E0

图 30.4.4.9 winhex 查看 zImage

可以看出图 30.4.4.8 和图 30.4.4.9 中的前 100 个字节的数据一致，说明 nfs 命令下载到的 zImage 是正确的。

4、tftp 命令

tftp 命令的作用和 nfs 命令一样，都是用于通过网络下载东西到 DRAM 中，只是 tftp 命令使用的 TFTP 协议，Ubuntu 主机作为 TFTP 服务器。因此需要在 Ubuntu 上搭建 TFTP 服务器，需要安装 tftp-hpa 和 tftpd-hpa，命令如下：

```
sudo apt-get install tftp-hpa tftpd-hpa
sudo apt-get install xinetd
```

和 NFS 一样，TFTP 也需要一个文件夹来存放文件，在用户目录下新建一个目录，命令如下：

```
mkdir /home/zuozhongkai/linux/tftpboot
chmod 777 /home/zuozhongkai/linux/tftpboot
```

这样我就在我的电脑上创建了一个名为 `tftpboot` 的目录(文件夹), 路径为 `/home/zuozhongkai/linux/tftpboot`。注意! 我们要给 `tftpboot` 文件夹权限, 否则的话 `uboot` 不能从 `tftpboot` 文件夹里面下载文件。

最后配置 `tftp`, 安装完成以后新建文件 `/etc/xinetd.d/tftp`, 如果没有 `/etc/xinetd.d` 目录的话自行创建, 然后在里面输入如下内容:

示例代码 30.4.4.1 `/etc/xinetd.d/tftp` 文件内容

```
1 server tftp
2 {
3     socket_type      = dgram
4     protocol         = udp
5     wait             = yes
6     user             = root
7     server           = /usr/sbin/in.tftpd
8     server_args      = -s /home/zuozhongkai/linux/tftpboot/
9     disable          = no
10    per_source       = 11
11    cps              = 100 2
12    flags            = IPv4
13 }
```

完了以后启动 `tftp` 服务, 命令如下:

```
sudo service tftpd-hpa start
```

打开 `/etc/default/tftpd-hpa` 文件, 将其修改为如下所示内容:

示例代码 30.4.4.2 `/etc/default/tftpd-hpa` 文件内容

```
1 # /etc/default/tftpd-hpa
2
3 TFTP_USERNAME="tftp"
4 TFTP_DIRECTORY="/home/zuozhongkai/linux/tftpboot"
5 TFTP_ADDRESS=":69"
6 TFTP_OPTIONS="-l -c -s"
```

`TFTP_DIRECTORY` 就是我们上面创建的 `tftp` 文件夹目录, 以后我们就将所有需要通过 TFTP 传输的文件都放到这个文件夹里面, 并且要给予这些文件相应的权限。

最后输入如下命令, 重启 `tftp` 服务器:

```
sudo service tftpd-hpa restart
```

`tftp` 服务器已经搭建好了, 接下来就是使用了。将 `zImage` 镜像文件拷贝到 `tftpboot` 文件夹中, 并且给予 `zImage` 相应的权限, 命令如下:

```
cp zImage /home/zuozhongkai/linux/tftpboot/
cd /home/zuozhongkai/linux/tftpboot/
chmod 777 zImage
```

万事俱备, 只剩验证了, `uboot` 中的 `tftp` 命令格式如下:

```
tftpboot [loadAddress] [[hostIPAddr:]bootfilename]
```

看起来和 `nfs` 命令格式一样的, `loadAddress` 是文件在 DRAM 中的存放地址, `[[hostIPAddr:]bootfilename]` 是要从 Ubuntu 中下载的文件。但是和 `nfs` 命令的区别在于, `tftp` 命令

不需要输入文件在 Ubuntu 中的完整路径，只需要输入文件名即可。比如我们现在将 tftpboot 文件夹里面的 zImage 文件下载到开发板 DRAM 的 0X80800000 地址处，命令如下：

tftp 80800000 zImage

下载过程如图 30.4.4.10 所示:

[illegible]

图 30.4.4.10 tftp 命令下载过程

从图 30.4.4.10 可以看出，zImage 下载成功了，网速为 1.4MibB/s，文件大小为 6071136 字节。同样的，可以使用 md.b 命令来查看前 100 个字节的数据是否和图 30.4.4.9 中的相等。有时候使用 tftp 命令从 Ubuntu 中下载文件的时候会出现如图 30.4.4.11 所示的错误提示：

```
=> tftp 80800000 zImage
Using FEC1 device
TFTP from server 192.168.1.253; our IP address is 192.168.1.137
Filename 'zImage'.
Load address: 0x80800000
Loading: *
TFTP error: 'Permission denied' (0)
Starting again

=>
```

图 30.4.4.11 tftp 下载出错

在图 30.4.4.11 中可以看到“TFTP error: 'Permission denied' (0)”这样的错误提示，提示没有权限，出现这个错误一般有两个原因：

- ①、在 Ubuntu 中创建 `tftpboot` 目录的时候没有给予 `tftpboot` 相应的权限。
- ②、`tftpboot` 目录中要下载的文件没有给予相应的权限。

针对上述两个问题，使用命令“`chmod 777 xxx`”来给予权限，其中“xxx”就是要给予权限的文件或文件夹。

好了，uboot 中关于网络的命令就讲解到这里，我们最常用的就是 ping、nfs 和 tftp 这三个命令。使用 ping 命令来查看网络的连接状态，使用 nfs 和 tftp 命令来从 Ubuntu 主机中下载文件。

30.4.5 EMMC 和 SD 卡操作命令

uboot 支持 EMMC 和 SD 卡，因此也要提供 EMMC 和 SD 卡的操作命令。一般认为 EMMC 和 SD 卡是同一个东西，所以没有特殊说明，本教程统一使用 MMC 来代指 EMMC 和 SD 卡。uboot 中常用于操作 MMC 设备的命令为“mmc”。

mmc 是一系列的命令，其后可以跟不同的参数，输入“? mmc”即可查看 mmc 有关的命令，如图 30.4.5.1 所示：


```
=> ? mmc
mmc - MMC sub system

Usage:
mmc info - display info of the current MMC device
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
mmc rescan
mmc part - lists available partition on current mmc device
mmc dev [dev] [part] - show or set current mmc device [partition]
mmc list - lists available devices
mmc hwpartition [args...] - does hardware partitioning
    arguments (sizes in 512-byte blocks):
        [user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes
        [gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition
        [check|set|complete] - mode, complete set partitioning completed
    WARNING: Partitioning is a write-once setting once it is set to complete.
    Power cycling is required to initialize partitions after set to complete.
mmc bootbus dev boot_bus_width reset_boot_bus_width boot_mode
    - Set the BOOT_BUS_WIDTH field of the specified device
mmc bootpart-resize <dev> <boot part size MB> <RPMB part size MB>
    - Change sizes of boot and RPMB partitions of specified device
mmc partconf dev boot_ack boot_partition partition_access
    - Change the bits of the PARTITION_CONFIG field of the specified device
mmc rst-function dev value
    - Change the RST_n_FUNCTION field of the specified device
    WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.
mmc setdsr <value> - set DSR register value

=> █
```

图 30.4.5.1 mmc 命令

从图 30.4.5.1 可以看出，mmc 后面跟不同的参数可以实现不同的功能，如表 30.4.5.1 所示：

命令	描述
mmc info	输出 MMC 设备信息
mmc read	读取 MMC 中的数据。
mmc wirte	向 MMC 设备写入数据。
mmc rescan	扫描 MMC 设备。
mmc part	列出 MMC 设备的分区。
mmc dev	切换 MMC 设备。
mmc list	列出当前有效的所有 MMC 设备。
mmc hwpartition	设置 MMC 设备的分区。
mmc bootbus.....	设置指定 MMC 设备的 BOOT_BUS_WIDTH 域的值。
mmc bootpart.....	设置指定 MMC 设备的 boot 和 RPMB 分区的大小。
mmc partconf.....	设置指定 MMC 设备的 PARTITION_CONFG 域的值。
mmc rst	复位 MMC 设备
mmc setdsr	设置 DSR 寄存器的值。

表 30.4.5.1 mmc 命令

1、mmc info 命令

mmc info 命令用于输出当前选中的 mmc info 设备的信息，输入命令“mmc info”即可，如图 30.4.5.2 所示：

```
=> mmc info
Device: FSL_SDHC
Manufacturer ID: 13
OEM: 14e
Name: Q2J55
Tran Speed: 52000000
Rd Block Len: 512
MMC version 5.0
High Capacity: Yes
Capacity: 7.1 GiB
Bus Width: 8-bit
Erase Group Size: 512 KiB
HC WP Group Size: 8 MiB
User Capacity: 7.1 GiB WRREL
Boot Capacity: 16 MiB ENH
RPMB Capacity: 4 MiB ENH
=>
```

图 30.4.5.2 mmc info 命令

从图 30.4.5.2 可以看出,当前选中的 MMC 设备是 EMMC,版本为 5.0,容量为 7.1GiB(EMMC 为 8GB),速度为 52000000Hz=52MHz,8 位宽的总线。还有一个与 mmc info 命令相同功能的命令: mmccinfo,“mmc”和“info”之间没有空格。实际量产的 EMMC 核心板所使用的 EMMC 芯片是多厂商供应的,因此 EMMC 信息以实际为准,但是容量都为 8GB 的!

2、mmc rescan 命令

mmc rescan 命令用于扫描当前开发板上所有的 MMC 设备,包括 EMMC 和 SD 卡,输入“mmc rescan”即可。

3、mmc list 命令

mmc list 命令用于来查看当前开发板一共有几个 MMC 设备,输入“mmc list”,结果如图 30.4.5.3 所示:

```
=> mmc list
FSL_SDHC: 0
FSL_SDHC: 1 (eMMC)
=>
```

图 30.4.5.3 扫描 MMC 设备

可以看出当前开发板有两个 MMC 设备: FSL_SDHC:0 和 FSL_SDHC:1 (eMMC),这是因为我现在用的是 EMMC 版本的核心板,加上 SD 卡一共有两个 MMC 设备,FSL_SDHC:0 是 SD 卡,FSL_SDHC:1(eMMC)是 EMMC,。默认会将 EMMC 设置为当前 MMC 设备,这就是为什么输入“mmc info”查询到的是 EMMC 设备信息,而不是 SD 卡。要想查看 SD 卡信息,就要使用命令“mmc dev”来将 SD 卡设置为当前的 MMC 设备。

4、mmc dev 命令

mmc dev 命令用于切换当前 MMC 设备,命令格式如下:

```
mmc dev [dev] [part]
```

[dev]用来设置要切换的 MMC 设备号,[part]是分区号。如果不写分区号的话默认为分区 0。

使用如下命令切换到 SD 卡:

```
mmc dev 0 //切换到 SD 卡, 0 为 SD 卡, 1 为 eMMC
```

结果如图 30.4.5.4 所示:

```
=> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
=>
```

图 30.4.5.4 切换到 SD 卡

从图 30.4.5.4 可以看出，切换到 SD 卡成功，mmc0 为当前的 MMC 设备，输入命令“mmc info”即可查看 SD 卡的信息，结果如图 30.4.5.5 所示：

```
=> mmc info
Device: FSL_SDHC
Manufacturer ID: 3
OEM: 5344
Name: SC16G
Tran Speed: 50000000
Rd Block Len: 512
SD version 3.0
High Capacity: Yes
Capacity: 14.8 GiB
Bus Width: 4-bit
Erase Group Size: 512 Bytes
=>
```

图 30.4.5.5 SD 信息

从图 30.4.5.5 可以看出当前 SD 卡为 3.0 版本的，容量为 14.8GiB(16GB 的 SD 卡)，4 位宽的总线。

5、mmc part 命令

有时候 SD 卡或者 EMMC 会有多个分区，可以使用命令“mmc part”来查看其分区，比如查看 EMMC 的分区情况，输入如下命令：

```
mmc dev 1    //切换到 EMMC
mmc part     //查看 EMMC 分区
```

结果如图 30.4.5.6 所示：

```
=> mmc dev 1
switch to partitions #0, OK
mmc1(part 0) is current device
=> mmc part

Partition Map for MMC device 1  --  Partition Type: DOS

Part   Start Sector   Num Sectors   UUID                Type
  1     20480         262144        6aa037b6-01         0c
  2     282624        14594048       6aa037b6-02         83
=>
```

图 30.4.5.6 查看 EMMC 分区

从图 30.4.5.6 中可以看出，此时 EMMC 有两个分区，第一个分区起始扇区为 20480，长度为 262144 个扇区；第二个分区起始扇区为 282624，长度为 14594048 个扇区。如果 EMMC 里面烧写了 Linux 系统的话，EMMC 是有 3 个分区的，第 0 个分区存放 uboot，第 1 个分区存放 Linux 镜像文件和设备树，第 2 个分区存放根文件系统。但是在图 30.4.5.6 中只有两个分区，那是因为第 0 个分区没有格式化，所以识别不出来，实际上第 0 个分区是存在的。一个新的 SD 卡默认只有一个分区，那就是分区 0，所以前面讲解的 uboot 烧写到 SD 卡，其实就是将 u-boot.bin

烧写到了 SD 卡的分区 0 里面。后面学习 Linux 内核移植的时候再讲解怎么在 SD 卡中创建并格式化第二个分区, 并将 Linux 镜像文件和设备树文件存放到第二个分区中。

如果要将 EMMC 的分区 2 设置为当前 MMC 设备, 可以使用如下命令:

```
mmc dev 1 2
```

结果如图 30.4.5.7 所示:

```
=> mmc dev 1 2
switch to partitions #2, OK
mmc1(part 2) is current device
=> █
```

图 30.4.5.7 设置 EMMC 分区 2 为当前设备

6、mmc read 命令

mmc read 命令用于读取 mmc 设备的数据, 命令格式如下:

```
mmc read addr blk# cnt
```

addr 是数据读取到 DRAM 中的地址, blk 是要读取的块起始地址(十六进制), 一个块是 512 字节, 这里的块和扇区是一个意思, 在 MMC 设备中我们通常说扇区, cnt 是要读取的块数量(十六进制)。比如从 EMMC 的第 1536(0x600)个块开始, 读取 16(0x10)个块的数据到 DRAM 的 0X80800000 地址处, 命令如下:

```
mmc dev 1 0          //切换到 MMC 分区 0
mmc read 80800000 600 10 //读取数据
```

结果如图 30.4.5.8 所示:

```
=> mmc dev 1 0
switch to partitions #0, OK
mmc1(part 0) is current device
=> mmc read 80800000 600 10

MMC read: dev # 1, block # 1536, count 16 ... 16 blocks read: OK
=> █
```

图 30.4.5.8 mmc read 命令

这里我们还看不出来读取是否正确, 通过 md.b 命令查看 0x80800000 处的数据就行了, 查看 16*512=8192(0x2000)个字节的数据, 命令如下:

```
md.b 80800000 2000
```

结果如图 30.4.5.9 所示:

```
=> md.b 80800000 2000
80800000: 8d c4 5f 28 62 61 75 64 72 61 74 65 3d 31 31 35    ..(baudrate=115
80800010: 32 30 30 00 62 6f 61 72 64 5f 72 65 76 3d 31 34 58    200.board_name=E
80800020: 56 4b 00 62 6f 61 72 64 5f 72 65 76 3d 31 34 58    VK.board_rev=14X
80800030: 31 34 00 62 6f 6f 74 5f 66 64 74 3d 74 72 79 00    14.boot_fdt=try.
80800040: 62 6f 6f 74 63 6d 64 3d 72 75 6e 20 66 69 6e 64    bootcmd=run find
80800050: 66 64 74 3b 6d 6d 63 20 64 65 76 20 24 7b 6d 6d    fdt;mmc dev ${mm
80800060: 63 64 65 76 7d 3b 6d 6d 63 20 64 65 76 20 24 7b    cdev};mmc dev ${
80800070: 6d 6d 63 64 65 76 7d 3b 20 69 66 20 6d 6d 63 20    mmcdev}; if mmc
80800080: 72 65 73 63 61 6e 3b 20 74 68 65 6e 20 69 66 20    rescan; then if
80800090: 72 75 6e 20 6c 6f 61 64 62 6f 6f 74 73 63 72 69    run loadbootscri
808000a0: 70 74 3b 20 74 68 65 6e 20 72 75 6e 20 62 6f 6f    pt; then run boo
808000b0: 74 73 63 72 69 70 74 3b 20 65 6c 73 65 20 69 66    tscript; else if
808000c0: 20 72 75 6e 20 6c 6f 61 64 69 6d 61 67 65 3b 20    run loadimage;
808000d0: 74 68 65 6e 20 72 75 6e 20 6d 6d 63 62 6f 6f 74    then run mmcboot
808000e0: 3b 20 65 6c 73 65 20 72 75 6e 20 6e 65 74 62 6f    ; else run netbo
808000f0: 6f 74 3b 20 66 69 3b 20 66 69 3b 20 65 6c 73 65    ot; fi; fi; else
```

图 30.4.5.9 读取到的数据(部分截图)

从图 30.4.5.9 可以看到 “baudrate=115200.board_name=EVK.board_rev=14X14.” 等字样, 这个就是 uboot 中的环境变量。EMMC 核心板 uboot 环境变量的存储起始地址就是 $1536 \times 512 = 786432$ 。

7、mmc write 命令

要将数据写到 MMC 设备里面, 可以使用命令 “mmc write”, 格式如下:

```
mmc write addr blk# cnt
```

addr 是要写入 MMC 中的数据在 DRAM 中的起始地址, blk 是要写入 MMC 的块起始地址 (十六进制), cnt 是要写入的块大小, 一个块为 512 字节。我们可以使用命令 “mmc write” 来升级 uboot, 也就是在 uboot 中更新 uboot。这里要用到 nfs 或者 tftp 命令, 通过 nfs 或者 tftp 命令将新的 u-boot.bin 下载到开发板的 DRAM 中, 然后再使用命令 “mmc write” 将其写入到 MMC 设备中。我们就来更新一下 SD 中的 uboot, 先查看一下 SD 卡中的 uboot 版本号, 注意编译时间, 输入命令:

```
mmc dev 0    //切换到 SD 卡
version      //查看版本号
```

结果如图 30.4.5.10 所示:

```
=> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
=> version

U-Boot 2016.03 (Mar 12 2020 - 15:11:51 +0800)
arm-linux-gnueabi-hf-gcc (Linaro GCC 4.9-2017.01) 4.9.4
GNU ld (Linaro_Binutils-2017.01) 2.24.0.20141017 Linaro 2014_11-3-git
=>
```

图 30.4.5.10 uboot 版本查询

可以看出当前 SD 卡中的 uboot 是 2020 年 3 月 12 日 15:11:51 编译的。我们现在重新编译一下 uboot, 然后将编译出来的 u-boot.imx(u-boot.bin 前面加了一些头文件)拷贝到 Ubuntu 中的 tftpboot 目录下。最后使用 tftp 命令将其下载到 0x80800000 地址处, 命令如下:

```
tftp 80800000 u-boot.imx
```

下载过程如图 30.4.5.11 所示:

```
=> tftp 80800000 u-boot.imx
FEC1 Waiting for PHY auto negotiation to complete.... done
Using FEC1 device
TFTP from server 192.168.1.253; our IP address is 192.168.1.137
Filename 'u-boot.imx'.
Load address: 0x80800000
Loading: #####
          2.4 MiB/s
done
Bytes transferred = 379904 (5cc00 hex)
=>
```

图 30.4.5.11 u-boot.imx 下载过程

可以看出, u-boot.imx 大小为 379904 字节, $379904/512=742$, 所以我们要向 SD 卡中写入 742 个块, 如果有小数的话就要加 1 个块。使用命令 “mmc write” 从 SD 卡分区 0 第 2 个块(扇区)开始烧写, 一共烧写 742(0x2E6)个块, 命令如下:

```
mmc dev 0 0
mmc write 80800000 2 2E6
```

烧写过程如图 30.4.5.12 所示:


```
=> mmc dev 0 0
switch to partitions #0, OK
mmc0 is current device
=> mmc write 80800000 3 32E

MMC write: dev # 0, block # 3, count 814 ... 814 blocks written: OK
=> █
```

图 30.4.5.12 烧写过程

烧写成功，重启开发板(从 SD 卡启动)，重启以后再输入 version 来查看版本号，结果如图 30.4.5.13 所示：

```
=> version

U-Boot 2016.03 (Oct 27 2020 - 11:44:31 +0800)
arm-linux-gnueabi-hf-gcc (Linaro GCC 4.9-2017.01) 4.9.4
GNU ld (Linaro_Binutils-2017.01) 2.24.0.20141017 Linaro 2014_11-3-git
=> █
```

图 30.4.5.13 uboot 版本号

从图 30.4.5.13 可以看出，此时的 uboot 是 2020 年 10 月 27 号 11:44:31 编译的，说明 uboot 更新成功。这里我们就学会了如何在 uboot 中更新 uboot 了，如果要更新 EMMC 中的 uboot 也是一样的。

同理，如果要在 uboot 中更新 EMMC 对应的 uboot，可以使用如下所示命令：

```
mmc dev 1 0           //切换到 EMMC 分区 0
tftp 80800000 u-boot.imx //下载 u-boot.imx 到 DRAM
mmc write 80800000 2 32E //烧写 u-boot.imx 到 EMMC 中
mmc partconf 1 1 0 0   //分区配置，EMMC 需要这一步！
```

千万不要写 SD 卡或者 EMMC 的前两个块(扇区)，里面保存着分区表！

8、mmc erase 命令

如果要擦除 MMC 设备的指定块就是用命令“mmc erase”，命令格式如下：

```
mmc erase blk# cnt
```

blk 为要擦除的起始块，cnt 是要擦除的数量。没事不要用 mmc erase 来擦除 MMC 设备!!!

关于 MMC 设备相关的命令就讲解到这里，表 30.4.5.1 中还有一些跟 MMC 设备操作有关的命令，但是很少用到，这里就不讲解了，感兴趣的可以上网查一下，或者在 uboot 中查看这些命令的使用方法。

30.4.6 FAT 格式文件系统操作命令

有时候需要在 uboot 中对 SD 卡或者 EMMC 中存储的文件进行操作，这时候就要用到文件操作命令，跟文件操作相关的命令有：fatinfo、fatls、fstype、fatload 和 fatwrite，但是这些文件操作命令只支持 FAT 格式的文件系统!!

1、fatinfo 命令

fatinfo 命令用于查询指定 MMC 设备分区的文件系统信息，格式如下：

```
fatinfo <interface> [<dev[:part]>]
```

interface 表示接口，比如 mmc，dev 是查询的设备号，part 是要查询的分区。比如我们要查询 EMMC 分区 1 的文件系统信息，命令如下：

```
fatinfo mmc 1:1
```

结果如图 30.4.6.1 所示：

```
=> fatinfo mmc 1:1
Interface: MMC
Device 1: Vendor: Man 000013 Snr 0c9ecc76 Rev: 1.0 Prod: Q2J55L
Type: Removable Hard Disk
Capacity: 7264.0 MB = 7.0 GB (14876672 x 512)
Filesystem: FAT32 "NO NAME"
=>
```

图 30.4.6.1 emmc 分区 1 文件系统信息

从上图可以看出，EMMC 分区 1 的文件系统为 FAT32 格式的。

2、fatls 命令

fatls 命令用于查询 FAT 格式设备的目录和文件信息，命令格式如下：

```
fatls <interface> [<dev[:part]>] [directory]
```

interface 是要查询的接口，比如 mmc，dev 是要查询的设备号，part 是要查询的分区，directory 是要查询的目录。比如查询 EMMC 分区 1 中的所有的目录和文件，输入命令：

```
fatls mmc 1:1
```

结果如图 30.4.6.2 所示：

```
=> fatls mmc 1:1
6785272 zimage
38859 imx6ull-14x14-emmc-4.3-480x272-c.dtb
38859 imx6ull-14x14-emmc-4.3-800x480-c.dtb
38859 imx6ull-14x14-emmc-7-800x480-c.dtb
38859 imx6ull-14x14-emmc-7-1024x600-c.dtb
38859 imx6ull-14x14-emmc-10.1-1280x800-c.dtb
39691 imx6ull-14x14-emmc-hdmi.dtb
39599 imx6ull-14x14-emmc-vga.dtb

8 file(s), 0 dir(s)
=>
```

图 30.4.6.2 EMMC 分区 1 文件查询

从上图可以看出，emmc 的分区 1 中存放着 8 个文件。

3、fstype 命令

fstype 用于查看 MMC 设备某个分区的文件系统格式，命令格式如下：

```
fstype <interface> <dev>[:<part>]
```

正点原子 EMMC 核心板上的 EMMC 默认有 3 个分区，我们来查看一下这三个分区的文件系统格式，输入命令：

```
fstype mmc 1:0
```

```
fstype mmc 1:1
```

```
fstype mmc 1:2
```

结果如图 30.4.6.3 所示：

```
=> fstype mmc 1:0
Failed to mount ext2 filesystem...
** Unrecognized filesystem type **
=> fstype mmc 1:1
fat
=> fstype mmc 1:2
ext4
=>
```

图 30.4.6.3 fstype 命令

从上图可以看出,分区 0 格式未知,因为分区 0 存放的 uboot,并且分区 0 没有格式化,所以文件系统格式未知。分区 1 的格式为 fat,分区 1 用于存放 linux 镜像和设备树。分区 2 的格式为 ext4,用于存放 Linux 的根文件系统(rootfs)。

4、fatload 命令

fatload 命令用于将指定的文件读取到 DRAM 中,命令格式如下:

```
fatload <interface> [<dev[:part]> [<addr> [<filename> [bytes [pos]]]]]
```

interface 为接口,比如 mmc, dev 是设备号, part 是分区, addr 是保存在 DRAM 中的起始地址, filename 是要读取的文件名字。bytes 表示读取多少字节的数据,如果 bytes 为 0 或者省略的话表示读取整个文件。pos 是要读的文件相对于文件首地址的偏移,如果为 0 或者省略的话表示从文件首地址开始读取。我们将 EMMC 分区 1 中的 zImage 文件读取到 DRAM 中的 0X80800000 地址处,命令如下:

```
fatload mmc 1:1 80800000 zImage
```

操作过程如图 30.4.6.4 所示:

```
=> fatload mmc 1:1 80800000 zImage
reading zImage
6785272 bytes read in 225 ms (28.8 MiB/s)
=> █
```

图 30.4.6.4 读取过程

从上图可以看出在 225ms 内读取了 6785272 个字节的数据,速度为 28.8MiB/s,速度是非常快的,因为这是从 EMMC 里面读取的,而 EMMC 是 8 位的,速度肯定会很快的。

5、fatwrite 命令

注意! uboot 默认没有使能 fatwrite 命令,需要修改板子配置头文件,比如 mx6ullevk.h、mx6ull_alientek_emmc.h 等等,板子不同,其配置头文件也不同。找到自己开发板对应的配置头文件然后添加如下一行宏定义来使能 fatwrite 命令:

```
#define CONFIG_FAT_WRITE /* 使能 fatwrite 命令 */
```

fatwrite 命令用于将 DRAM 中的数据写入到 MMC 设备中,命令格式如下:

```
fatwrite <interface> <dev[:part]> <addr> <filename> <bytes>
```

interface 为接口,比如 mmc, dev 是设备号, part 是分区, addr 是要写入的数据在 DRAM 中的起始地址, filename 是写入的数据文件名字, bytes 表示要写入多少字节的数据。我们可以通过 fatwrite 命令在 uboot 中更新 linux 镜像文件和设备树。我们以更新 linux 镜像文件 zImage 为例,首先将正点原子 I.MX6U-ALPHA 开发板提供的 zImage 镜像文件拷贝到 Ubuntu 中的 tftpboot 目录下,zImage 镜像文件放到了开发板光盘中,路径为:开发板光盘->8、系统镜像->1、出厂系统镜像->2、kernel 镜像->linux-imx-4.1.15-2.1.0-gbfed875-v1.6->zImage。

拷贝完成以后使用命令 tftp 将 zImage 下载到 DRAM 的 0X80800000 地址处,命令如下:

```
tftp 80800000 zImage
```

下载过程如图 30.4.6.5 所示:

[illegible]

图 30.4.6.5 zImage 下载过程

zImage 大小为 6785272(0X6788f8)个字节(注意, 由于开发板系统在不断的更新中, 因此 zImage 大小不是固定的, 一切以实际大小为准), 接下来使用命令 fatwrite 将其写入到 EMMC 的分区 1 中, 文件名字为 zImage, 命令如下:

```
fatwrite mmc 1:1 80800000 zImage 6788f8
```

结果如图 30.4.6.6 所示:

```
=> fatwrite mmc 1:1 80800000 zImage 6788f8
writing zImage
6785272 bytes written
=>
```

图 30.4.6.6 将 zImage 烧写到 EMMC 扇区 1 中

完成以后使用“fatls”命令查看一下 EMMC 分区 1 里面的文件，结果如图 30.4.6.7 所示：

```
=> fatls mmc 1:1
6785272    zimage
38859      imx6ull-14x14-emmc-4.3-480x272-c.dtb
38859      imx6ull-14x14-emmc-4.3-800x480-c.dtb
38859      imx6ull-14x14-emmc-7-800x480-c.dtb
38859      imx6ull-14x14-emmc-7-1024x600-c.dtb
38859      imx6ull-14x14-emmc-10.1-1280x800-c.dtb
39691      imx6ull-14x14-emmc-hdmi.dtb
39599      imx6ull-14x14-emmc-vga.dtb

8 file(s), 0 dir(s)

=>
```

图 30.4.6.7 EMMC 分区 1 里面的文件

30.4.7 EXT 格式文件系统操作命令

uboot 有 ext2 和 ext4 这两种格式的文件系统的操作命令，常用的就四个命令，分别为：ext2load、ext2ls、ext4load、ext4ls 和 ext4write。这些命令的含义和使用与 fatload、fatls 和 fatwrite 一样，只是 ext2 和 ext4 都是针对 ext 文件系统的。比如 ext4ls 命令，EMMC 的分区 2 就是 ext4 格式的，使用 ext4ls 就可以查询 EMMC 的分区 2 中的文件和目录，输入命令：

```
ext4ls mmc 1:2
```

结果如图 30.4.7.1 所示:

```
=> ext4ls mmc 1:2
<DIR>      4096 .
<DIR>      4096 ..
<DIR>    16384 lost+found
<DIR>      4096 bin
<DIR>      4096 boot
<DIR>      4096 dev
<DIR>      4096 etc
<DIR>      4096 home
<DIR>      4096 lib
<DIR>      4096 media
<DIR>      4096 mnt
<DIR>      4096 opt
<DIR>      4096 proc
<DIR>      4096 run
<DIR>      4096 sbin
<DIR>      4096 sys
<SYM>         8 tmp
<DIR>      4096 usr
<DIR>      4096 var
=>
```

图 30.4.7.1 ext4ls 命令

关于 ext 格式文件系统其他命令的操作参考 30.4.6 小节的即可，这里就不讲解了。

30.4.8 NAND 操作命令

uboot 是支持 NAND Flash 的，所以也有 NAND Flash 的操作命令，前提是使用的 NAND 版本的核心板，并且编译 NAND 核心板对应的 uboot，然后使用 imxdownload 软件将 u-boot.bin 烧写到 SD 卡中，最后通过 SD 卡启动。一般情况下 NAND 版本的核心板已经烧写好了 uboot、linux kernel 和 rootfs 这些文件，所以可以将 BOOT 拨到 NAND，然后直接从 NAND Flash 启动即可。

NAND 版核心板启动信息如图 30.4.8.1 所示：

```
U-Boot 2016.03-gd3f0479 (Aug 07 2020 - 20:47:45 +0800)

CPU:   Freescale i.MX6ULL rev1.1 792 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 45C
Reset cause: POR
Board: I.MX6U ALPHA|MINI
I2C:   ready
DRAM:  256 MiB
NAND:  512 MiB
MMC:   FSL_SDHC: 0
*** Warning - bad CRC, using default environment

Display: ATK-LCD-7-1024x600 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
Net:    FEC1
Error:  FEC1 address not set.

Normal Boot
Hit any key to stop autoboot:  0
=>
```

图 30.4.8.1 NAND 核心板启动信息

从图 30.4.8.1 可以看出, 当前开发板的 NAND 容量为 512MiB。输入 “? nand” 即可查看 NAND 相关命令, 如图 30.4.8.2 所示:

```
=> ? nand
nand - NAND sub-system

Usage:
nand info - show available NAND devices
nand device [dev] - show or set current device
nand read - addr off|partition size
nand write - addr off|partition size
    read/write 'size' bytes starting at offset 'off'
    to/from memory address 'addr', skipping bad blocks.
nand read.raw - addr off|partition [count]
nand write.raw - addr off|partition [count]
    Use read.raw/write.raw to avoid ECC and access the flash as-is.
nand write.trimffs - addr off|partition size
    write 'size' bytes starting at offset 'off' from memory address
    'addr', skipping bad blocks and dropping any pages at the end
    of eraseblocks that contain only 0xFF
nand erase[.spread] [clean] off size - erase 'size' bytes from offset 'off'
    With '.spread', erase enough for given file size, otherwise,
    'size' includes skipped bad blocks.
nand erase.part [clean] partition - erase entire mtd partition'
nand erase.chip [clean] - erase entire chip'
nand bad - show bad blocks
nand dump[.oob] off - dump page
nand scrub [-y] off size | scrub.part partition | scrub.chip
    really clean NAND erasing bad blocks (UNSAFE)
nand markbad off [...] - mark bad block(s) at offset (UNSAFE)
nand biterr off - make a bit error at offset (UNSAFE)
=> █
```

图 30.4.8.2 NAND 相关操作命令

可以看出, NAND 相关的操作命令不少, 本节我们讲解一些常用的命令。

1、nand info 命令

此命令用户打印 NAND Flash 信息, 输入 “nand info”, 结果如图 30.4.8.3 所示:

```
=> nand info

Device 0: nand0, sector size 128 KiB
Page size      2048 b
OOB size       64 b
Erase size     131072 b
subpagesize    2048 b
options        0x40000200
bbt options    0x 8000
=> █
```

图 30.4.8.3 nand 信息

图 30.4.8.3 中给出了 NAND 的页大小、OOB 域大小, 擦除大小等信息。可以对照着所使用的 NAND Flash 数据手册来查看一下这些信息是否正确。

2、nand device 命令

nand device 用于切换 NAND Flash, 如果你的板子支持多片 NAND 的话就可以使用此命令来设置当前所使用的 NAND。这个需要你的 CPU 有两个 NAND 控制器, 并且两个 NAND 控制器各接一片 NAND Flash。就跟 I.MX6U 有两个 SDIO 接口, 这两个 SDIO 接口可以接两个 MMC 设备一样。不过一般情况下 CPU 只有一个 NAND 接口, 而且在使用中只接一片 NAND。

3、nand erase 命令

nand erase 命令用于擦除 NAND Flash, NAND Flash 的特性决定了在向 NAND Flash 写数据之前一定要先对要写入的区域进行擦除。“nand erase”命令有三种形式:

```
nand erase[.spread] [clean] off size //从指定地址开始(off)开始, 擦除指定大小(size)的区域。
nand erase.part [clean] partition //擦除指定的分区
nand erase.chip [clean] //全篇擦除
```

NAND 的擦除命令一般是配合写命令的, 后面讲解 NAND 写命令的时候在演示如何使用“nand erase”。

4、nand write 命令

此命令用于向 NAND 指定地址写入指定的数据, 一般和“nand erase”命令配置使用来更新 NAND 中的 uboot、linux kernel 或设备树等文件, 命令格式如下:

```
nand write addr off size
```

addr 是要写入的数据首地址, off 是 NAND 中的目的地址, size 是要写入的数据大小。

由于 I.MX6ULL 要求 NAND 对应的 uboot 可执行文件还需要另外包含 BCB 和 DBBT, 因此直接编译出来的 uboot.imx 不能直接烧写到 NAND 里面。关于 BCB 和 DBBT 的详细介绍请参考《I.MX6ULL 参考手册》的 8.5.2.2 小节, 笔者目前没有详细去研究 BCB 和 DBBT, 因此我们不能在 NAND 版的 uboot 里面更新 uboot 自身。除非大家去研究一下 I.MX6ULL 的 BCB 和 DBBT, 然后在 u-boot.imx 前面加上相应的信息, 否则即使将 uboot 烧进去了也不能运行。我们使用 mfgtool 烧写系统到 NAND 里面的时候, mfgtool 会使用一个叫做“kogs-ng”的工具完成 BCB 和 DBBT 的添加。

但是我们可以在 uboot 里面使用“nand write”命令烧写 kernel 和 dtb。先编译出来 NAND 版本的 kernel 和 dtb 文件, 在烧写之前要先对 NAND 进行分区, 也就是规划好 uboot、linux kernel、设备树和根文件系统的存储区域, I.MX6U-ALPHA 开发板出厂系统 NAND 分区如下:

```
0x000000000000-0x00000003FFFFFF : "boot"
0x000000040000-0x000000041FFFF : "env"
0x000000042000-0x000000051FFFF : "logo"
0x000000052000-0x000000061FFFF : "dtb"
0x000000062000-0x0000000E1FFFF : "kernel"
0x0000000E2000-0x00000020000000 : "rootfs"
```

一共有六个分区, 第一个分区存放 uboot, 地址范围为 0x0~0x3FFFFFF(共 4MB); 第二个分区存放 env (环境变量), 地址范围为 0x400000~0x420000(共 128KB); 第三个分区存放 logo(启动图标), 地址范围为 0x420000~0x51FFFF(共 1MB); 第四个分区存放 dtb(设备树), 地址范围为 0x520000~0x61FFFF(共 1MB); 第五个分区存放 kernel(也就是 linux kernel), 地址范围为 0x620000~0xE1FFFF(共 8MB); 剩下的所有存储空间全部作为最后一个分区, 存放 rootfs(根文件系统)。

可以看出 kernel 是从地址 0x620000 开始存放的, 将 NAND 版本 kernel 对应的 zImage 文件放到 Ubuntu 中的 tftpboot 目录中, 然后使用 tftp 命令将其下载到开发板的 0X87800000 地址处, 最终使用“nand write”将其烧写到 NAND 中, 命令如下:

```
tftp 0x87800000 zImage //下载 zImage 到 DRAM 中
nand erase 0x620000 0x800000 //从地址 0x620000 开始擦除 8MB 的空间
nand write 0x87800000 0x620000 0x800000 //将接收到的 zImage 写到 NAND 中
```

这里我们擦除了 8MB 的空间, 因为一般 zImage 就是 6,7MB 左右, 8MB 肯定够了, 如果不够的话就再多擦除一点就行了。

同理, 最后烧写设备树(dtb)文件文件, 命令如下:

```
tftp 0x87800000 imx6ull-14x14-emmc-7-1024x600-c.dtb //下载 dtb 到 DRAM 中
nand erase 0x520000 0x100000 //从地址 0x520000 开始擦除 1MB 的空间
nand write 0x87800000 0x520000 0x100000 //将接收到的 dtb 写到 NAND 中
```

dtb 文件一般只有几十 KB，所以擦除 1M 是绰绰有余的了。**注意！正点原子出厂系统在 NAND 里面烧写了很多种设备树文件！这里只是举例烧写一种的方法，我们在实际产品开发中肯定只有一种设备树。**

根文件系统(rootfs)就不要在 uboot 中更新了，还是使用 NXP 提供的 Mfgtool 工具来烧写，因为根文件系统太大！很有可能超过开发板 DRAM 的大小，这样连下载都没法下载，更别说更新了。

4、nand read 命令

此命令用于从 NAND 中的指定地址读取指定大小的数据到 DRAM 中，命令格式如下：

```
nand read addr off size
```

addr 是目的地址，off 是要读取的 NAND 中的数据源地址，size 是要读取的数据大小。比如我们读取设备树(dtb)文件到 0x83000000 地址处，命令如下：

```
nand read 0x83000000 0x520000 0x19000
```

过程如图 30.4.8.4 所示：

```
=> nand read 83000000 520000 19000

NAND read: device 0 offset 0x520000, size 0x19000
102400 bytes read: OK
=> █
```

图 30.4.8.4 nand read 读取过程

设备树文件读取到 DRAM 中以后就可以使用 fdt 命令来对设备树进行操作了，首先设置 fdt 的地址，fdt 地址就是 DRAM 中设备树的首地址，命令如下：

```
fdt addr 83000000
```

设置好以后可以使用“fdt header”来查看设备树的头信息，输入命令：

```
fdt header
```

结果如图 30.4.8.5 所示：

```
=> fdt header
magic:                0xd00dfeed
totalsize:             0x9975 (39285)
off_dt_struct:         0x38
off_dt_strings:        0x8f0c
off_mem_rsvmap:        0x28
version:               17
last_comp_version:     16
boot_cpuid_phys:       0x0
size_dt_strings:       0xa69
size_dt_struct:        0x8ed4
number_mem_rsv:        0x0
=> █
```

图 30.4.8.5 设备树头信息

输入命令“fdt print”就可以查看设备树文件的内容，输入命令：

```
fdt print
```

结果如图 30.4.8.6 所示：

```

=> fdt print
/ {
    #address-cells = <0x00000001>;
    #size-cells = <0x00000001>;
    model = "Freescale i.MX6 ULL 14x14 EVK Board";
    compatible = "fsl,imx6ull-14x14-evk", "fsl,imx6ull";
    chosen {
        stdout-path = "/soc/aips-bus@02000000/spba-bus@02000000/serial@02020000";
    };
    aliases {
        can0 = "/soc/aips-bus@02000000/can@02090000";
        can1 = "/soc/aips-bus@02000000/can@02094000";
        ethernet0 = "/soc/aips-bus@02100000/ethernet@02188000";
        ethernet1 = "/soc/aips-bus@02000000/ethernet@020b4000";
        gpio0 = "/soc/aips-bus@02000000/gpio@0209c000";
        gpio1 = "/soc/aips-bus@02000000/gpio@020a0000";
        gpio2 = "/soc/aips-bus@02000000/gpio@020a4000";
        gpio3 = "/soc/aips-bus@02000000/gpio@020a8000";
        gpio4 = "/soc/aips-bus@02000000/gpio@020ac000";
        i2c0 = "/soc/aips-bus@02100000/i2c@021a0000";
        i2c1 = "/soc/aips-bus@02100000/i2c@021a4000";
        i2c2 = "/soc/aips-bus@02100000/i2c@021a8000";
        i2c3 = "/soc/aips-bus@02100000/i2c@021f8000";
        mmc0 = "/soc/aips-bus@02100000/usdhc@02190000";
        mmc1 = "/soc/aips-bus@02100000/usdhc@02194000";
    };
}

```

图 30.4.8.6 设备树文件

图 30.4.8.6 中的文件就是我们写到 NAND 中的设备树文件，至于设备树文件的详细内容我们后面会有专门的章节来讲解，这里大家知道这个文件就行了。

NAND 常用的操作命令就是擦除、读和写，至于其他的命令大家可以自行研究一下，一定不要尝试全片擦除 NAND 的指令!! 否则 NAND 就被全部擦除掉了，什么都没有了，又得重头烧整个系统。

30.4.9 BOOT 操作命令

uboot 的本质工作是引导 Linux，所以 uboot 肯定有相关的 boot(引导)命令来启动 Linux。常用的跟 boot 有关的命令有：bootz、bootm 和 boot。

1、bootz 命令

要启动 Linux，需要先将 Linux 镜像文件拷贝到 DRAM 中，如果使用到设备树的话也需要将设备树拷贝到 DRAM 中。可以从 EMMC 或者 NAND 等存储设备中将 Linux 镜像和设备树文件拷贝到 DRAM，也可以通过 nfs 或者 tftp 将 Linux 镜像文件和设备树文件下载到 DRAM 中。不管用那种方法，只要能将 Linux 镜像和设备树文件存到 DRAM 中就行，然后使用 bootz 命令来启动，bootz 命令用于启动 zImage 镜像文件，bootz 命令格式如下：

```
bootz [addr [initrd[:size]] [fdt]]
```

命令 bootz 有三个参数，addr 是 Linux 镜像文件在 DRAM 中的位置，initrd 是 initrd 文件在 DRAM 中的地址，如果不使用 initrd 的话使用 '-' 代替即可，fdt 就是设备树文件在 DRAM 中的地址。现在我们使用网络 and EMMC 两种方法来启动 Linux 系统，首先将 I.MX6U-ALPHA 开发板的 Linux 镜像和设备树发送到 Ubuntu 主机中的 tftpboot 文件夹下。Linux 镜像文件前面已经放到了 tftpboot 文件夹中，现在把设备树文件放到 tftpboot 文件夹里面。由于不同的屏幕其设备树不同，因此我们出厂系统提供了很多设备树，路径为：**开发板光盘->8、系统镜像->1、出厂系统镜像->2、kernel 镜像->linux-imx-4.1.15-2.1.0-gbfed875-v1.6**，所有设备树文件如图 30.4.9.1 所示：

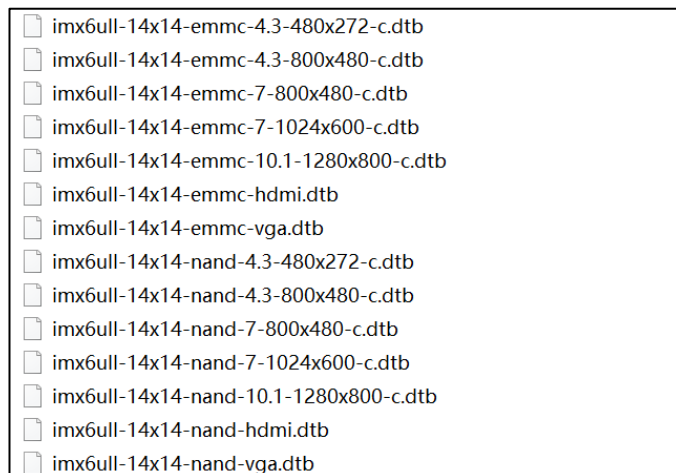


图 30.4.9.1 正点原子出厂设备树

从图 30.4.9.1 可以看出,我们提供了 14 种设备树,笔者正在使用的是 EMMC 核心板,7 寸 1024×600 分辨率的屏幕,所以需要使用 imx6ull-14x14-emmc-7-1024x600-c.dtb 这个设备树。将 imx6ull-14x14-emmc-7-1024x600-c.dtb 发送到 Ubuntu 主机中的 tftpboot 文件夹里面,完成以后的 tftpboot 文件夹如图 30.4.9.2 所示:

```
zuozhongkai@ubuntu:~/linux/tftpboot$ ls
imx6ull-14x14-emmc-7-1024x600-c.dtb  u-boot.imx  zImage
zuozhongkai@ubuntu:~/linux/tftpboot$
```

图 30.4.9.2 tftpboot 文件夹

给予 imx6ull-14x14-emmc-7-1024x600-c.dtb 可执行权限, 命令如下:

```
chmod 777 imx6ull-14x14-emmc-7-1024x600-c.dtb
```

Linux 镜像文件和设备树都准备好了,我们先学习如何通过网络启动 Linux,使用 tftp 命令将 zImage 下载到 DRAM 的 0X80800000 地址处,然后将设备树 imx6ull-14x14-emmc-7-1024x600-c.dtb 下载到 DRAM 中的 0X83000000 地址处,最后之后命令 bootz 启动, 命令如下:

```
tftp 80800000 zImage
tftp 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb
bootz 80800000 - 83000000
```

命令运行结果如图 30.4.9.3 所示:

[illegible]

图 30.4.9.3 通过网络启动 Linux

上图就是我们通过 `tftp` 和 `bootz` 命令来从网络启动 Linux 系统，如果我们要从 EMMC 中启动 Linux 系统的话只需要使用命令 `fatload` 将 `zImage` 和 `imx6ull-14x14-emmc-7-1024x600-c.dtb` 从 EMMC 的分区 1 中拷贝到 DRAM 中，然后使用命令 `bootz` 启动即可。先使用命令 `fatls` 查看要下 EMMC 的分区 1 中有没有 Linux 镜像文件和设备树文件，如果没有的话参考 30.4.6 小节中讲解的 `fatwrite` 命令将 `tftpboot` 中的 `zImage` 和 `imx6ull-14x14-emmc-7-1024x600-c.dtb` 文件烧写到 EMMC 的分区 1 中。然后使用命令 `fatload` 将 `zImage` 和 `imx6ull-14x14-emmc-7-1024x600-c.dtb` 文件拷贝到 DRAM 中，地址分别为 `0X80800000` 和 `0X83000000`，最后使用 `bootz` 启动，命令如下：

```
fatload mmc 1:1 80800000 zImage
fatload mmc 1:1 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb
bootz 80800000 - 83000000
```

命令运行结果如图 30.4.9.4 所示:


```

=> fatload mmc 1:1 80800000 zImage ——— 1、读取zImage
reading zImage
6785272 bytes read in 224 ms (28.9 MiB/s)
=> fatload mmc 1:1 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb ——— 2、读取设备树
reading imx6ull-14x14-emmc-7-1024x600-c.dtb
38859 bytes read in 24 ms (1.5 MiB/s)
=> bootz 80800000 - 83000000 ——— 3、使用bootz启动
Kernel image @ 0x80800000 [ 0x000000 - 0x6788f8 ]
## Flattened Device Tree blob at 83000000
Booting using the fdt blob at 0x83000000
Using Device Tree in place at 8300c7ca ——— 4、Linux启动信息

Starting kernel ...

[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 4.1.15-gbfed875 (alientek@ubuntu) (gcc version 5.3.0 (GCC) )
t Jan 30 15:53:28 CST 2021
[ 0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c53c7d
[ 0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[ 0.000000] Machine model: Freescale i.MX6 ULL 14x14 EVK Board

```

图 30.4.9.4 从 EMMC 中启动 Linux

2、bootm 命令

bootm 和 bootz 功能类似，但是 bootm 用于启动 uImage 镜像文件。如果不使用设备树的话启动 Linux 内核的命令如下：

```
bootm addr
```

addr 是 uImage 镜像在 DRAM 中的首地址。

如果要使用设备树，那么 bootm 命令和 bootz 一样，命令格式如下：

```
bootm [addr [initrd[:size]]] [fdt]
```

其中 addr 是 uImage 在 DRAM 中的首地址，initrd 是 initrd 的地址，fdt 是设备树(.dtb)文件在 DRAM 中的首地址，如果 initrd 为空的话，同样是用“-”来替代。

3、boot 命令

boot 命令也是用来启动 Linux 系统的，只是 boot 会读取环境变量 bootcmd 来启动 Linux 系统，bootcmd 是一个很重要的环境变量！其名字分为“boot”和“cmd”，也就是“引导”和“命令”，说明这个环境变量保存着引导命令，其实就是启动的命令集合，具体的引导命令内容是可以修改的。比如我们要想使用 tftp 命令从网络启动 Linux 那么就可以设置 bootcmd 为“tftp 80800000 zImage; tftp 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb; bootz 80800000 - 83000000”，然后使用 saveenv 将 bootcmd 保存起来。然后直接输入 boot 命令即可从网络启动 Linux 系统，命令如下：

```

setenv bootcmd 'tftp 80800000 zImage; tftp 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb;
bootz 80800000 - 83000000'

saveenv

boot

```

运行结果如图 30.4.9.5 所示：

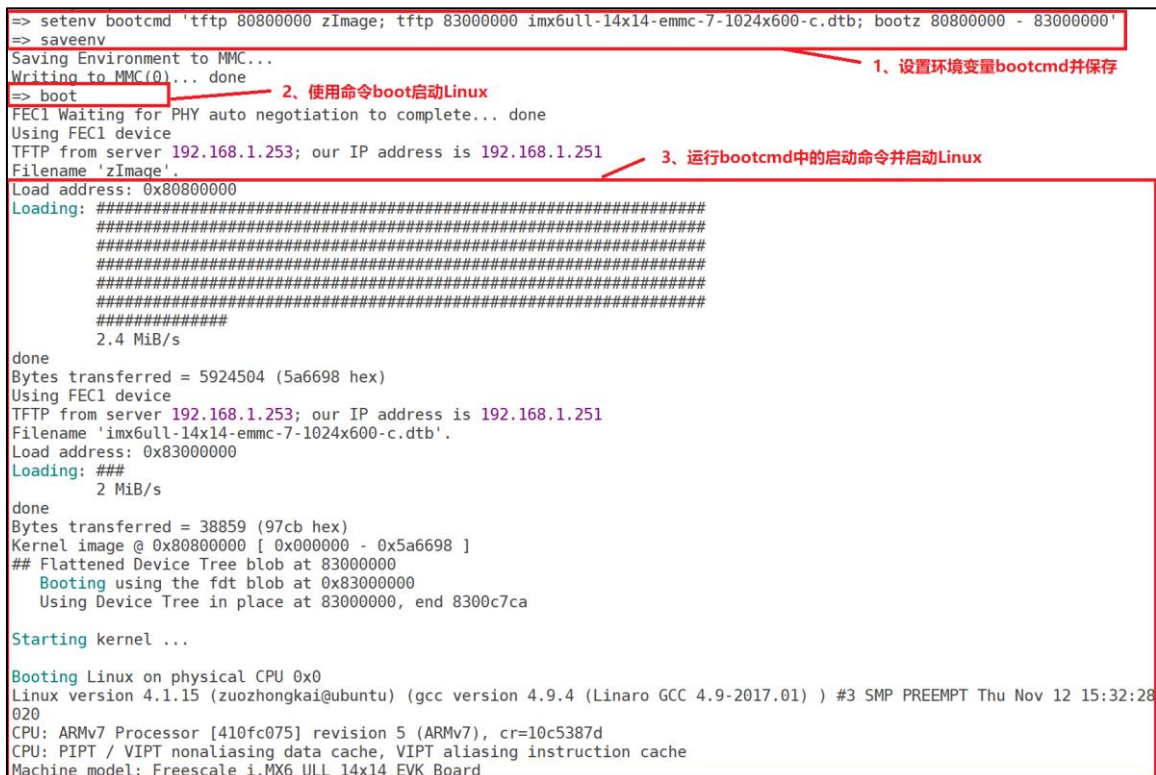


图 30.4.9.5 设置 bootcmd 从网络启动 Linux

前面说过 uboot 倒计时结束以后就会启动 Linux 系统，其实就是执行的 bootcmd 中的启动命令。只要不修改 bootcmd 中的内容，以后每次开机 uboot 倒计时结束以后都会使用 tftp 命令从网络下载 zImage 和 imx6ull-14x14-ermmc-7-1024x600-c.dtb，然后启动 Linux。

如果想从 EMMC 启动那就设置 bootcmd 为 “fatload mmc 1:1 80800000 zImage; fatload mmc 1:1 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb; bootz 80800000 - 83000000”，然后使用 boot 命令启动即可，命令如下：

```
setenv bootcmd 'fatload mmc 1:1 80800000 zImage; fatload mmc 1:1 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb; bootz 80800000 - 83000000'
```

savenev

boot

运行结果如图 30.4.9.6 所示:



图 30.4.9.6 设置 bootcmd 从 EMMC 启动 Linux

如果不修改 bootcmd 的话，每次开机 uboot 倒计时结束以后都会自动从 EMMC 里面读取 zImage 和 imx6ull-14x14-emmc-7-1024x600-c.dtb，然后启动 Linux。

在启动 Linux 内核的时候可能会遇到如下错误:

“Kernel panic - not Syncing: VFS: Unable to mount root fs on unknown-block(0,0)”

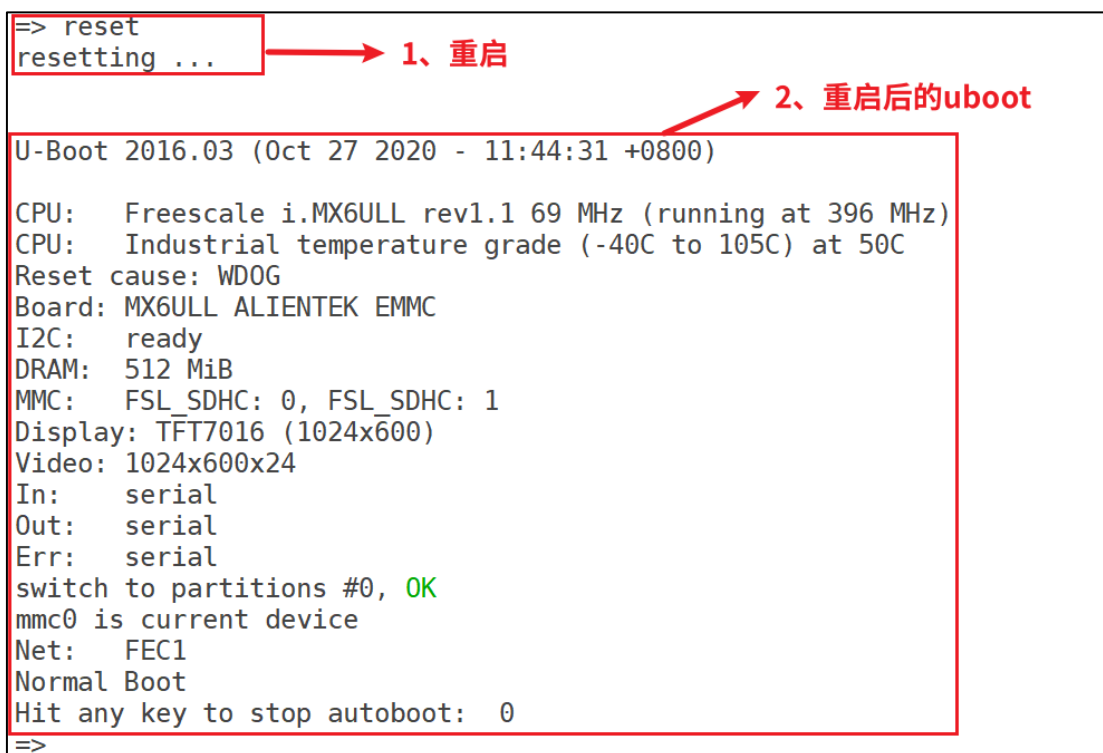
这个错误的原因是 linux 内核没有找到根文件系统, 这个很正常, 因为没有设置 uboot 的 bootargs 环境变量, 关于 bootargs 环境变量后面会讲解! 此处我们重点是验证 boot 命令, linux 内核已经成功启动了, 说明 boot 命令工作正常。

30.4.10 其他常用命令

uboot 中还有其他一些常用的命令, 比如 reset、go、run 和 mtest 等。

1、reset 命令

reset 命令顾名思义就是复位的, 输入 “reset” 即可复位重启, 如图 30.4.10.1 所示:



```
=> reset
resetting ...

U-Boot 2016.03 (Oct 27 2020 - 11:44:31 +0800)

CPU:   Freescale i.MX6ULL rev1.1 69 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 50C
Reset cause: WDOG
Board: MX6ULL ALIENTEK EMMC
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:    serial
Out:   serial
Err:   serial
switch to partitions #0, OK
mmc0 is current device
Net:   FEC1
Normal Boot
Hit any key to stop autoboot:  0
=>
```

图 30.4.10.1 reset 命令运行结果

2、go 命令

go 命令用于跳到指定的地址处执行应用, 命令格式如下:

```
go addr [arg ...]
```

addr 是应用在 DRAM 中的首地址, 我们可以编译一下裸机例程的 [实验 13_printf](#), 然后将编译出来的 printf.bin 拷贝到 Ubuntu 中的 tftpboot 文件夹里面, 注意, 这里要拷贝 printf.bin 文件, 不需要在前面添加 IVT 信息, 因为 uboot 已经初始化好了 DDR 了。使用 tftp 命令将 printf.bin 下载到开发板 DRAM 的 0X87800000 地址处, 因为裸机例程的链接首地址就是 0X87800000, 最后使用 go 命令启动 printf.bin 这个应用, 命令如下:

```
tftp 87800000 printf.bin
go 87800000
```

结果如图 30.4.10.2 所示:

```

=> tftp 87800000 printf.bin
FEC1 Waiting for PHY auto negotiation to complete.... done
Using FEC1 device
TFTP from server 192.168.1.250; our IP address is 192.168.1.137
Filename 'printf.bin'.
Load address: 0x87800000
Loading: #
        433.6 KiB/s
done
Bytes transferred = 11569 (2d31 hex)
=> go 87800000
## Starting application at 0x87800000 ...
输入两个整数, 使用空格隔开:2 3
数据2 + 3 = 5

输入两个整数, 使用空格隔开:54 6
数据54 + 6 = 60

输入两个整数, 使用空格隔开:

```

1、下载printf.bin

2、运行printf.bin

3、printf.bin运行正常

图 30.4.10.2 go 命令运行裸机例程

从图 30.4.10.2 可以看出, 通过 go 命令我们就可以在 uboot 中运行裸机例程。

3、run 命令

run 命令用于运行环境变量中定义的命令, 比如可以通过“run bootcmd”来运行 bootcmd 中的启动命令, 但是 run 命令最大的作用在于运行我们自定义的环境变量。在后面调试 Linux 系统的时候常常要在网络启动和 EMMC/NAND 启动之间来回切换, 而 bootcmd 只能保存一种启动方式, 如果要换另外一种启动方式的话就得重写 bootcmd, 会很麻烦。这里我们就可以通过自定义环境变量来实现不同的启动方式, 比如定义环境变量 mybootemmc 表示从 emmc 启动, 定义 mybootnet 表示从网络启动, 定义 mybootnand 表示从 NAND 启动。如果要切换启动方式的话只需要运行“run mybootxxx(xxx 为 emmc、net 或 nand)”即可。

说干就干, 创建环境变量 mybootemmc、mybootnet 和 mybootnand, 命令如下:

```

setenv mybootemmc 'fatload mmc 1:1 80800000 zImage; fatload mmc 1:1 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb;bootz 80800000 - 83000000'
setenv mybootnand 'nand read 80800000 4000000 800000;nand read 83000000 6000000 1000000;bootz 80800000 - 83000000'
setenv mybootnet 'tftp 80800000 zImage; tftp 83000000 imx6ull-14x14-emmc-7-1024x600-c.dtb;bootz 80800000 - 83000000'
saveenv

```

创建环境变量成功以后就可以使用 run 命令来运行 mybootemmc、mybootnet 或 mybootnand 来实现不同的启动:

```
run mybootemmc
```

或

```
run mytoobnand
```

或

```
run mybootnet
```

4、mtest 命令

mtest 命令是一个简单的内存读写测试命令, 可以用来测试自己开发板上的 DDR, 命令格式如下:

```
mtest [start [end [pattern [iterations]]]]
```

start 是要测试的 DRAM 开始地址, end 是结束地址, 比如我们测试 0X80000000~0X80001000 这段内存, 输入 “mtest 80000000 80001000”, 结果如图 30.4.10.3 所示:

```
=> mtest 80000000 80001000
Testing 80000000 ... 80001000:
Pattern FFFFFFFF Writing...      ading...Iteration:    486
```

图 30.4.10.3 mtest 命令运行结果

从图 30.4.10.3 可以看出, 测试范围为 0X80000000~0X80001000, 已经测试了 486 次, 如果要结束测试就按下键盘上的 “Ctrl+C” 键。

至此, uboot 常用的命令就讲解完了, 如果要使用 uboot 的其他命令, 可以查看 uboot 中的帮助信息, 或者上网查询一下相应的资料。

第三十一章 U-Boot 顶层 Makefile 详解

上一章我们详细的讲解了 uboot 的使用方法，其实就是各种命令的使用，学会 uboot 使用以后就可以尝试移植 uboot 到自己的开发板上了，但是在移植之前需要我们先分析一遍 uboot 的启动流程源码，得捋一下 uboot 的启动流程，否则移植的时候都不知道该修改那些文件。本章我们就来分析一下正点原子提供的 uboot 源码，重点是分析 uboot 启动流程，而不是整个 uboot 源码，uboot 整个源码非常大，我们只看跟我们关心的部分即可。

31.1 U-Boot 工程目录分析

本书我们以 EMMC 版本的核心板为例讲解,为了方便,uboot 启动源码分析就在 Windows 下进行,将正点原子提供的 uboot 源码进行解压,解压完成以后的目录如图 31.1.1 所示:

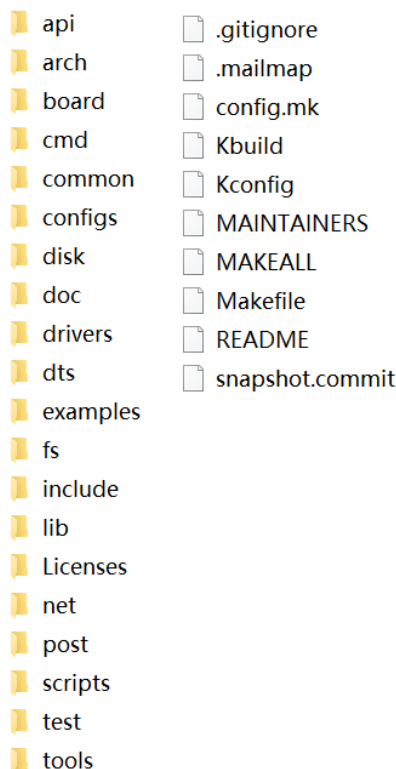


图 31.1.1 未编译的 uboot

图 31.1.1 是正点原子提供的未编译的 uboot 源码目录,我们在分析 uboot 源码之前一定要先在 Ubuntu 中编译一下 uboot 源码,因为编译过程会生成一些文件,而生成的这些恰恰是分析 uboot 源码不可或缺的文件。使用上一章创建的 shell 脚本来完成编译工作,命令如下:

```
cd alientek_uboot           //进入正点原子 uboot 源码目录
./mx6ull_alientek_emmc.sh   //编译 uboot
cd ../                      //返回上一级目录
tar -vcjf alientek_uboot.tar.bz2 alientek_uboot //压缩
```

最终会生成一个名为 alientek_uboot.tar.bz2 的压缩包,将 alientek_uboot.tar.bz2 拷贝到 windows 系统中并解压,解压后的目录如图 31.1.2 所示:

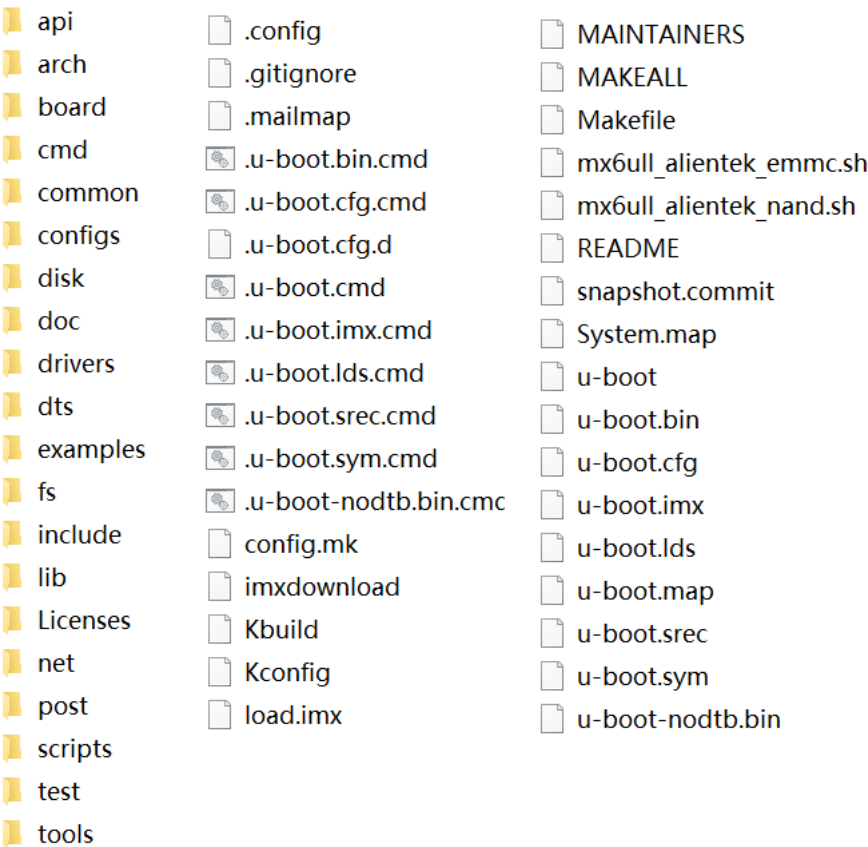


图 31.1.2 编译后的 uboot 源码文件

对比图 31.1.2 和图 31.1.1，可以看出编译后的 uboot 要比没编译之前多了好多文件，这些文件夹或文件的含义见表 31.1.1 所示：

类型	名字	描述	备注
文件夹	api	与硬件无关的 API 函数。	uboot 自带
	arch	与架构体系有关的代码。	
	board	不同板子(开发板)的定制代码。	
	cmd	命令相关代码。	
	common	通用代码。	
	configs	配置文件。	
	disk	磁盘分区相关代码	
	doc	文档。	
	drivers	驱动代码。	
	dts	设备树。	
	examples	示例代码。	
	fs	文件系统。	
	include	头文件。	
	lib	库文件。	
	Licenses	许可证相关文件。	
	net	网络相关代码。	

	post	上电自检程序。	
	scripts	脚本文件。	
	test	测试代码。	
	tools	工具文件夹。	
文件	.config	配置文件，重要的文件。	编译生成的文件
	.gitignore	git 工具相关文件。	uboot 自带
	.mailmap	邮件列表。	
	.u-boot.xxx.cmd (一系列)	这是一系列的文件，用于保存着一些命令。	编译生成的文件
	config.mk	某个 Makefile 会调用此文件。	uboot 自带
	imxdownload	正点原子编写的 SD 卡烧写软件。	正点原子提供
	Kbuild	用于生成一些和汇编有关的文件。	uboot 自带
	Kconfig	图形配置界面描述文件。	
	MAINTAINERS	维护者联系方式文件。	
	MAKEALL	一个 shell 脚本文件，帮助编译 uboot 的。	
	Makefile	主 Makefile，重要文件！	上一章编写的。
	mx6ull_alientek_emmc.sh	上一章编写的编译脚本文件	
	mx6ull_alientek_nand.sh	上一章编写的编译脚本文件	
	README	相当于帮助文档。	uboot 自带
	snapshot.commint	???	
	System.map	系统映射文件	编译出来的文件
	u-boot	编译出来的 u-boot 文件。	
	u-boot.xxx (一系列)	生成的一些 u-boot 相关文件，包括 u-boot.bin、u-boot.imx.等	

表 31.1.1 uboot 目录列表

表 31.1.1 中的很多文件夹和文件我们都不需要去关心，我们要关注的文件夹或文件如下：

1、arch 文件夹

这个文件夹里面存放着和架构有关的文件，如图 31.1.3 所示：

	arc	2019-04-22 21:07	文件夹	
	arm	2019-04-22 21:07	文件夹	
	avr32	2019-04-22 21:07	文件夹	
	blackfin	2019-04-22 21:07	文件夹	
	m68k	2019-04-22 21:07	文件夹	
	microblaze	2019-04-22 21:07	文件夹	
	mips	2019-04-22 21:07	文件夹	
	nds32	2019-04-22 21:07	文件夹	
	nios2	2019-04-22 21:07	文件夹	
	openrisc	2019-04-22 21:07	文件夹	
	powerpc	2019-04-22 21:07	文件夹	
	sandbox	2019-04-22 21:07	文件夹	
	sh	2019-04-22 21:07	文件夹	
	sparc	2019-04-22 21:07	文件夹	
	x86	2019-04-22 21:07	文件夹	
	.gitignore	2019-03-26 15:49	GITIGNORE 文件	1 KB
	Kconfig	2019-03-26 15:49	文件	5 KB

图 31.1.3 arch 文件夹

从图 31.1.3 可以看出有很多架构，比如 arm、avr32、m68k 等，我们现在用的是 ARM 芯片，所以只需要关心 arm 文件夹即可，打开 arm 文件夹里面内容如图 31.1.4 所示：

	cpu	2019-04-22 21:07	文件夹	
	dts	2019-04-22 21:07	文件夹	
	imx-common	2019-04-22 21:07	文件夹	
	include	2019-04-22 21:07	文件夹	
	lib	2019-04-22 21:07	文件夹	
	mach-at91	2019-04-22 21:07	文件夹	
	mach-bcm283x	2019-04-22 21:07	文件夹	
	mach-davinci	2019-04-22 21:07	文件夹	
	mach-exynos	2019-04-22 21:07	文件夹	
	mach-highbank	2019-04-22 21:07	文件夹	
	mach-integrator	2019-04-22 21:07	文件夹	
	mach-keystone	2019-04-22 21:07	文件夹	

图 31.1.4 arm 文件夹

图 31.1.4 只截取了一部分，还有一部分 mach-xxx 的文件夹。mach 开头的文件夹是跟具体的设备有关的，比如“mach-exynos”就是跟三星的 exynos 系列 CPU 有关的文件。我们使用的是 I.MX6ULL，所以要关注“imx-common”这个文件夹。另外“cpu”这个文件夹也是和 cpu 架构有关的，打开以后如图 31.1.5 所示：

arm11	2019-04-22 21:07	文件夹	
arm720t	2019-04-22 21:07	文件夹	
arm920t	2019-04-22 21:07	文件夹	
arm926ejs	2019-04-22 21:07	文件夹	
arm946es	2019-04-22 21:07	文件夹	
arm1136	2019-04-22 21:07	文件夹	
arm1176	2019-04-22 21:07	文件夹	
armv7	2019-04-22 21:07	文件夹	
armv7m	2019-04-22 21:07	文件夹	
armv8	2019-04-22 21:07	文件夹	
pxa	2019-04-22 21:07	文件夹	
sa1100	2019-04-22 21:07	文件夹	
.built-in.o.cmd	2019-04-22 20:52	Windows 命令脚本	1 KB
built-in.o	2019-04-22 20:52	O 文件	1 KB
Makefile	2019-03-26 15:49	文件	1 KB
u-boot.lds	2019-03-26 15:49	LDS 文件	3 KB
u-boot-spl.lds	2019-03-26 15:49	LDS 文件	2 KB

图 31.1.5 cpu 文件夹

从图 31.1.5 可以看出有多种 ARM 架构相关的文件夹, I.MX6ULL 使用的 Cortex-A7 内核, Cortex-A7 属于 armv7, 所以我们要关心“armv7”这个文件夹。cpu 文件夹里面有个名为“u-boot.lds”的链接脚本文件, 这个就是 ARM 芯片所使用的 u-boot 链接脚本文件! armv7 这个文件夹里面的文件都是跟 ARMV7 架构有关的, 是我们分析 uboot 启动源码的时候需要重点关注的。

2、board 文件夹

board 文件夹就是和具体的板子有关的, 打开此文件夹, 里面全是不同的板子, 毫无疑问正点原子的开发板肯定也在里面(正点原子添加的), board 文件夹里面有个名为“freescale”的文件夹, 如图 31.1.6 所示:

evb_rk303b	2019-04-22 21:07	文件夹
firefly	2019-04-22 21:07	文件夹
freescale	2019-04-22 21:07	文件夹
gaisler	2019-04-22 21:07	文件夹

图 31.1.6 freescale 文件夹

所有使用 freescale 芯片的板子都放到此文件夹中, IMX 系列以前属于 freescale, 只是 freescale 后来被 NXP 收购了。打开此 freescale 文件夹, 在里面找到和 mx6u(I.MX6UL/ULL)有关的文件夹, 如图 31.1.7 所示:

mx6ul_14x14_ddr3_arm2	2019-09-03 0:17	文件夹
mx6ul_14x14_evk	2019-09-03 0:17	文件夹
mx6ul_14x14_lpddr2_arm2	2019-09-03 0:17	文件夹
mx6ull_ddr3_arm2	2019-09-03 0:17	文件夹
mx6ullevk	2019-09-03 0:17	文件夹

图 31.1.7 mx6u 相关板子

图 31.1.7 中有 5 个文件夹, 这 5 个文件夹对应 5 种板子, 以“mx6ul”开头的表示使用 I.MX6UL 芯片的板子, 以 mx6ull 开头的表示使用 I.MX6ULL 芯片的板子。mx6ullevk 是 NXP 官方的 I.MX6ULL 开发板, 正点原子的 ALPHA 开发板就是在这个基础上开发的, 因此 mx6ullevk 也是正点原子的开发板。我们后面移植 uboot 到时候就是参考 NXP 官方的开发板, 也就是要参

考 mx6ullevk 这个文件夹来定义我们的板子。

3、configs 文件夹

此文件夹为 uboot 配置文件，uboot 是可配置的，但是你要是自己从头开始一个一个项目的配置，那就太麻烦了，因此一般半导体或者开发板厂商都会制作好一个配置文件。我们可以在这个做好的配置文件基础上来添加自己想要的功能，这些半导体厂商或者开发板厂商制作好的配置文件统一命名为“xxx_defconfig”，xxx 表示开发板名字，这些 defconfig 文件都存放在 configs 文件夹，因此，NXP 官方开发板和正点原子的开发板配置文件肯定也在这个文件夹中，如图 31.1.8 所示：

mx6ull_14x14_ddr3_arm2_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_emmc_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_epdc_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_nand_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_qspi1_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_spinor_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr3_arm2_tsc_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr256_emmc_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr256_nand_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr256_nand_sd_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr512_emmc_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr512_nand_defconfig	2019-08-31 11:46	文件	1 KB
mx6ull_14x14_ddr512_nand_sd_defconfig	2019-08-31 11:46	文件	1 KB

图 31.1.8 正点原子开发板配置文件

图 31.1.8 中这 6 个文件就是正点原子 I.MX6U-ALPHA 开发板所对应的 uboot 默认配置文件。我们只关心 mx6ull_14x14_ddr512_emmc_defconfig 和 mx6ull_14x14_ddr256_nand_defconfig 这两个文件，分别是正点原子 I.MX6ULL EMMC 核心板和 NAND 核心板的配置文件。使用“make xxx_defconfig”命令即可配置 uboot，比如：

```
make mx6ull_14x14_ddr512_emmc_defconfig
```

上述命令就是配置正点原子的 I.MX6ULL EMMC 核心板所使用的 uboot。

在编译 uboot 之前一定要使用 defconfig 来配置 uboot！

在 mx6ull_alientek_emmc.sh 中就有下面这一句：

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mx6ull_14x14_ddr512_emmc_defconfig
```

这个就是调用 mx6ull_14x14_ddr512_emmc_defconfig 来配置 uboot，只是这个命令还带了一些其它参数而已。

4、.u-boot.xxx_cmd 文件

.u-boot.xxx_cmd 是一系列的文件，这些文件都是编译生成的，都是一些命令文件，比如文件.u-boot.bin.cmd，看名字应该是和 u-boot.bin 有关的，此文件的内容如下：

示例代码 31.1.1 .u-boot.bin.cmd 代码

```
1 cmd_u-boot.bin := cp u-boot-nodtb.bin u-boot.bin
```

.u-boot.bin.cmd 里面定义了一个变量：cmd_u-boot.bin，此变量的值为“cp u-boot-nodtb.bin u-boot.bin”，也就是拷贝一份 u-boot-nodtb.bin 文件，并且重命名为 u-boot.bin，这个就是 u-boot.bin 的来源，来自于文件 u-boot-nodtb.bin。

那么 u-boot-nodtb.bin 是怎么来的呢？文件.u-boot-nodtb.bin.cmd 就是用于生成 u-boot.nodtb.bin 的，此文件内容如下：

示例代码 31.1.2 .u-boot-nodtb.bin.cmd 代码

```
1 cmd_u-boot-nodtb.bin := arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -
j .text -j .secure_text -j .rodata -j .hash -j .data -j .got -
j .got.plt -j .u_boot_list -j .rel.dyn -O binary u-boot u-boot-
nodtb.bin
```

这里用到了 arm-linux-gnueabi-hf-objcopy, 使用 objcopy 将 ELF 格式的 u-boot 文件转换为二进制的 u-boot-nodtb.bin 文件。

文件 u-boot 是 ELF 格式的文件, 文件 u-boot.cmd 用于生成 u-boot, 文件内容如下:

示例代码 31.1.3 .u-boot.cmd 代码

```
1 cmd_u-boot := arm-linux-gnueabi-hf-ld.bfd -pie --gc-sections -
Bstatic -Ttext 0x87800000 -o u-boot -T u-boot.lds
arch/arm/cpu/armv7/start.o --start-group arch/arm/cpu/built-in.o
arch/arm/cpu/armv7/built-in.o arch/arm/imx-common/built-in.o
arch/arm/lib/built-in.o board/freescale/common/built-in.o
board/freescale/mx6ull_alientek_emmc/built-in.o cmd/built-in.o
common/built-in.o disk/built-in.o drivers/built-in.o
drivers/dma/built-in.o drivers/gpio/built-in.o drivers/i2c/built-
in.o drivers/mmc/built-in.o drivers/mtd/built-in.o
drivers/mtd/onenand/built-in.o drivers/mtd/spi/built-in.o
drivers/net/built-in.o drivers/net/phy/built-in.o drivers/pci/built-
in.o drivers/power/built-in.o drivers/power/battery/built-in.o
drivers/power/fuel_gauge/built-in.o drivers/power/mfd/built-in.o
drivers/power/pmic/built-in.o drivers/power/regulator/built-in.o
drivers/serial/built-in.o drivers/spi/built-in.o
drivers/usb/dwc3/built-in.o drivers/usb/emul/built-in.o
drivers/usb/eth/built-in.o drivers/usb/gadget/built-in.o
drivers/usb/gadget/udc/built-in.o drivers/usb/host/built-in.o
drivers/usb/musb-new/built-in.o drivers/usb/musb/built-in.o
drivers/usb/phy/built-in.o drivers/usb/ulpi/built-in.o fs/built-in.o
lib/built-in.o net/built-in.o test/built-in.o test/dm/built-in.o --
end-group arch/arm/lib/eabi_compat.o -L /usr/local/arm/gcc-linaro-
4.9.4-2017.01-x86_64_arm-linux-gnueabi-hf/bin/../lib/gcc/arm-linux-
gnueabi-hf/4.9.4 -lgcc -Map u-boot.map
```

.u-boot.cmd 使用到了 arm-linux-gnueabi-hf-ld.bfd, 也就是链接工具, 使用 ld.bfd 将各个 built-in.o 文件链接在一起就形成了 u-boot 文件。u-boot 在编译的时候会将同一个目录中的所有.c 文件都编译在一起, 并命名为 built-in.o, 相当于将众多的.c 文件对应的.o 文件集合在一起, 这个就是 u-boot 文件的来源。

如果我们要用 NXP 提供的 MFGTools 工具向开发板烧写 uboot, 此时烧写的是 u-boot.imx 文件, 而不是 u-boot.bin 文件。u-boot.imx 是在 u-boot.bin 文件的头部添加了 IVT、DCD 等信息。这个工作是由文件 u-boot.imx.cmd 来完成的, 此文件内容如下:

示例代码 31.1.4 .u-boot.imx.cmd 代码


```
1 cmd_u-boot.imx := ./tools/mkimage -n
board/freescale/mx6ull_alientek_emmc/imximage.cfg.cfgtmp -T imximage -
e 0x87800000 -d u-boot.bin u-boot.imx
```

可以看出, 这里用到了工具 `tools/mkimage`, 而 IVT、DCD 等数据保存在了文件 `board/freescale/mx6ullevk/imximage-ddr512.cfg.cfgtmp` 中(如果是 NAND 核心板的话就是 `imximage-ddr256.cfg.cfgtmp`), 工具 `mkimage` 就是读取文件 `imximage-ddr512.cfg.cfgtmp` 里面的信息, 然后将其添加到文件 `u-boot.bin` 的头部, 最终生成 `u-boot.imx`。

文件 `u-boot.lds.cmd` 就是用于生成 `u-boot.lds` 链接脚本的, 由于 `u-boot.lds.cmd` 文件内容太多, 这里就不列出来了。`u-boot` 根目录下的 `u-boot.lds` 链接脚本就是来源于 `arch/arm/cpu/u-boot.lds` 文件。

还有一些其它的 `u-boot.lds.xxx.cmd` 文件, 大家自行分析一下, 关于 `u-boot.lds.xxx.cmd` 文件就讲解到这里。

5、Makefile 文件

这个是顶层 Makefile 文件, Makefile 是支持嵌套的, 也就是顶层 Makefile 可以调用子目录中的 Makefile 文件。Makefile 嵌套在大项目中很常见, 一般大项目里面所有的源代码都不会放到同一个目录中, 各个功能模块的源代码都是分开的, 各自存放在各自的目录中。每个功能模块目录下都有一个 Makefile, 这个 Makefile 只处理本模块的编译链接工作, 这样所有的编译链接工作就不用全部放到一个 Makefile 中, 可以使得 Makefile 变得简洁明了。

`u-boot` 源码根目录下的 Makefile 是顶层 Makefile, 他会调用其它的模块的 Makefile 文件, 比如 `drivers/adcc/Makefile`。当然了, 顶层 Makefile 要做的工作可远不止调用子目录 Makefile 这么简单, 关于顶层 Makefile 的内容我们稍后会有详细的讲解。

6、u-boot.xxx 文件

`u-boot.xxx` 同样也是一系列文件, 包括 `u-boot`、`u-boot.bin`、`u-boot.cfg`、`u-boot.imx`、`u-boot.lds`、`u-boot.map`、`u-boot.srec`、`u-boot.sym` 和 `u-boot-nodtb.bin`, 这些文件的含义如下:

u-boot: 编译出来的 ELF 格式的 `u-boot` 镜像文件。

u-boot.bin: 编译出来的二进制格式的 `u-boot` 可执行镜像文件。

u-boot.cfg: `u-boot` 的另外一种配置文件。

u-boot.imx: `u-boot.bin` 添加头部信息以后的文件, NXP 的 CPU 专用文件。

u-boot.lds: 链接脚本。

u-boot.map: `u-boot` 映射文件, 通过查看此文件可以知道某个函数被链接到了哪个地址上。

u-boot.srec: S-Record 格式的镜像文件。

u-boot.sym: `u-boot` 符号文件。

u-boot-nodtb.bin: 和 `u-boot.bin` 一样, `u-boot.bin` 就是 `u-boot-nodtb.bin` 的复制文件。

7、.config 文件

`u-boot` 配置文件, 使用命令 “`make xxx_defconfig`” 配置 `u-boot` 以后就会自动生成, `.config` 内容如下:

示例代码 31.1.5 .config 代码

```
1 #
2 # Automatically generated file; DO NOT EDIT.
3 # U-Boot 2016.03 Configuration
4 #
5 CONFIG_CREATE_ARCH_SYMLINK=y
```

```
6 CONFIG_HAVE_GENERIC_BOARD=y
7 CONFIG_SYS_GENERIC_BOARD=y
8 # CONFIG_ARC is not set
9 CONFIG_ARM=y
10 # CONFIG_AVR32 is not set
11 # CONFIG_BLACKFIN is not set
12 # CONFIG_M68K is not set
13 # CONFIG_MICROBLAZE is not set
14 # CONFIG_MIPS is not set
15 # CONFIG_NDS32 is not set
16 # CONFIG_NIOS2 is not set
17 # CONFIG_OPENRISC is not set
18 # CONFIG_PPC is not set
19 # CONFIG_SANDBOX is not set
20 # CONFIG_SH is not set
21 # CONFIG_SPARC is not set
22 # CONFIG_X86 is not set
23 CONFIG_SYS_ARCH="arm"
24 CONFIG_SYS_CPU="armv7"
25 CONFIG_SYS_SOC="mx6"
26 CONFIG_SYS_VENDOR="freescale"
27 CONFIG_SYS_BOARD="mx6ull_alientek_emmc"
28 CONFIG_SYS_CONFIG_NAME="mx6ull_alientek_emmc"
29
30 .....
31
32 #
33 # Boot commands
34 #
35 CONFIG_CMD_BOOTD=y
36 CONFIG_CMD_BOOTM=y
37 CONFIG_CMD_ELFS=y
38 CONFIG_CMD_GO=y
39 CONFIG_CMD_RUN=y
40 CONFIG_CMD_IMI=y
41 CONFIG_CMD_IMLS=y
42 CONFIG_CMD_XIMG=y
43
44 #
45 # Environment commands
46 #
47 CONFIG_CMD_EXPORTENV=y
48 CONFIG_CMD_IMPORTENV=y
```

```

49 CONFIG_CMD_EDITENV=y
50 CONFIG_CMD_SAVEENV=y
51 CONFIG_CMD_ENV_EXISTS=y
52
53 .....
54
55 #
56 # Library routines
57 #
58 # CONFIG_CC_OPTIMIZE_LIBS_FOR_SPEED is not set
59 CONFIG_HAVE_PRIVATE_LIBGCC=y
60 # CONFIG_USE_PRIVATE_LIBGCC is not set
61 CONFIG_SYS_HZ=1000
62 # CONFIG_USE_TINY_PRINTF is not set
63 CONFIG_REGEX=y
    
```

可以看出.config 文件中都是以“CONFIG_”开始的配置项，这些配置项就是 Makefile 中的变量，因此后面都跟有相应的值，uboot 的顶层 Makefile 或子 Makefile 会调用这些变量值。在.config 中会有大量的变量值为‘y’，这些为‘y’的变量一般用于控制某项功能是否使能，为‘y’的话就表示功能使能，比如：

CONFIG_CMD_BOOTM=y

如果使能了 bootd 这个命令的话，CONFIG_CMD_BOOTM 就为‘y’。在 cmd/Makefile 中有如下代码：

示例代码 31.1.6 cmd/Makefile 代码

```

1 ifndef CONFIG_SPL_BUILD
2 # core command
3 obj-y += boot.o
4 obj-$(CONFIG_CMD_BOOTM) += bootm.o
5 obj-y += help.o
6 obj-y += version.o
    
```

在示例代码 31.1.6 中，有如下所示一行代码：

obj-\$(CONFIG_CMD_BOOTM) += bootm.o

CONFIG_CMD_BOOTM=y，将其展开就是：

obj-y += bootm.o

也就是给 obj-y 追加了一个“bootm.o”，obj-y 包含着所有要编译的文件对应的.o 文件，这里表示需要编译文件 cmd/bootm.c。相当于通过“CONFIG_CMD_BOOTD=y”来使能 bootm 这个命令，进而编译 cmd/bootm.c 这个文件，这个文件实现了命令 bootm。在 uboot 和 Linux 内核中都是采用这种方法来选择使能某个功能，编译对应的源码文件。

8、README

README 文件描述了 uboot 的详细信息，包括 uboot 该如何编译、uboot 中各文件夹的含义、相应的命令等等。建议大家详细的阅读此文件，可以进一步增加对 uboot 的认识。

关于 uboot 根目录中的文件和文件夹的含义就讲解到这里，接下来就要开始分析 uboot 的启动流程了。

31.2 VScode 工程创建

先在 Ubuntu 下编译一下 uboot，然后将编译后的 uboot 文件夹复制到 windows 下，并创建 VScode 工程。打开 VScode，选择：文件->打开文件夹...，选中 uboot 文件夹，如图 31.2.1 所示：

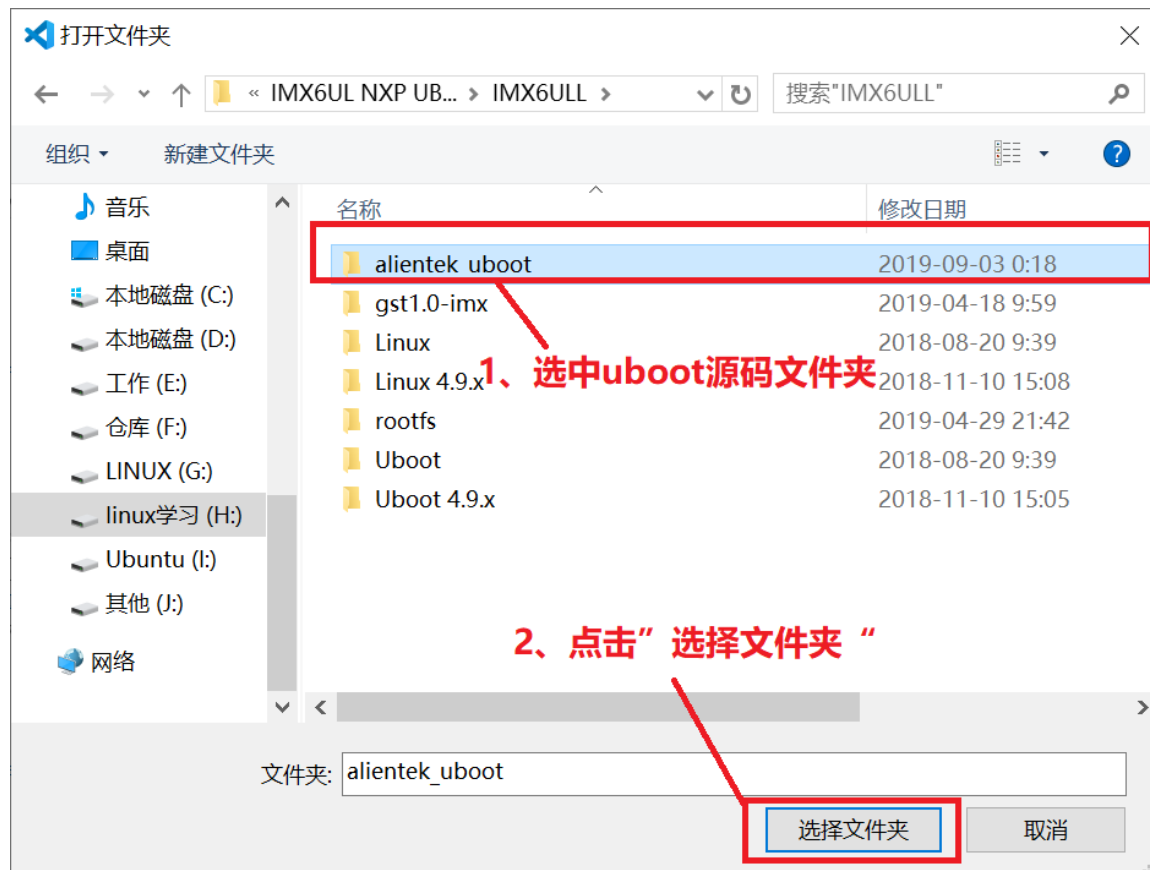


图 31.2.1 选择 uboot 源码文件夹

打开 uboot 目录以后，VSCode 界面如图 31.2.2 所示：



图 31.2.2 VScode 界面

点击“文件->将工作区另存为...”，打开保存工作区对话框，将工作区保存到 uboot 源码根目录下，设置文件名为“uboot”，如图 31.2.3 所示：

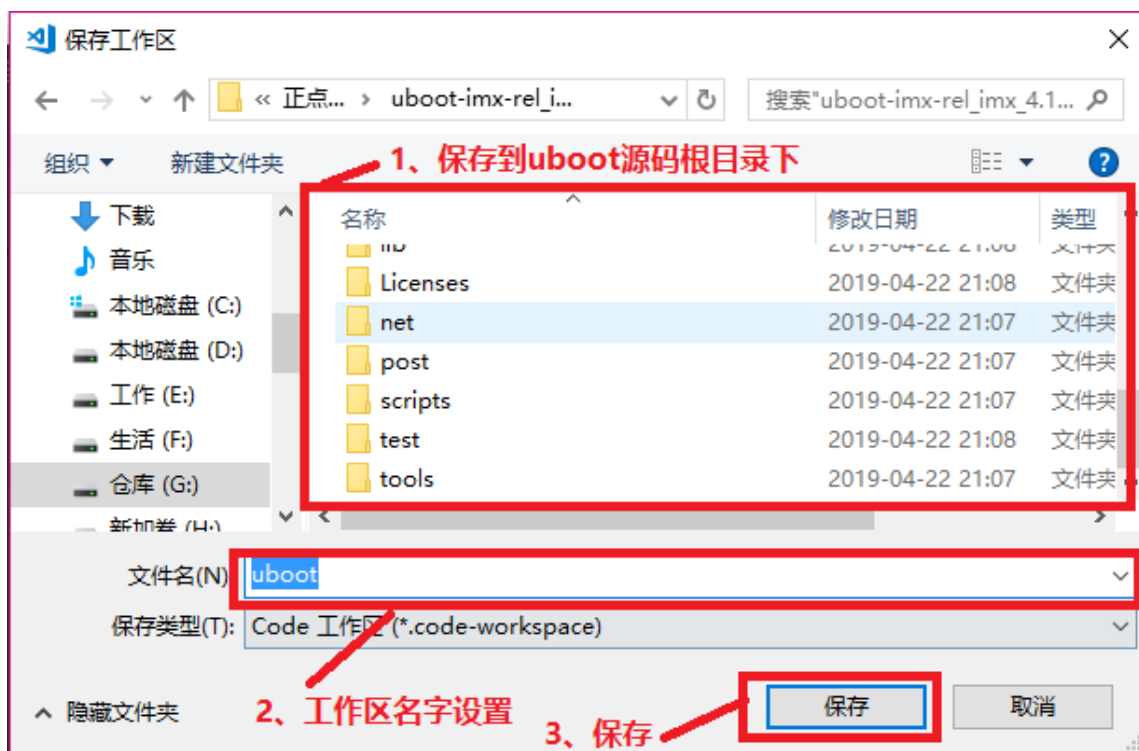


图 31.2.3 保存工作区

保存成功以后就会在 uboot 源码根目录下存在一个名为 uboot.code-workspace 的文件。这样一个完整的 VSCode 工程就建立起来了。但是这个 VSCode 工程包含了 uboot 的所有文件，uboot 中有些文件是不需要的，比如 arch 目录下是各种架构的文件夹，如图 31.2.4 所示：

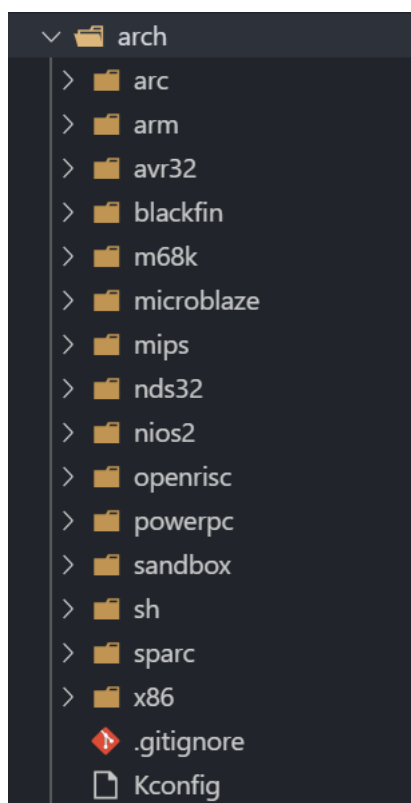


图 31.2.4 arch 目录

在 arch 目录下，我们只需要 arm 文件夹，所以需要将其余的目录从 VSCode 中给屏蔽掉，比如将 arch/avr32 这个目录给屏蔽掉。

在 VSCode 上建名为“.vscode”的文件夹，如图 31.2.5 所示：



图 31.2.5 新建.vscode 文件夹

输入新建文件夹的名字，完成以后如图 31.2.6 所示。

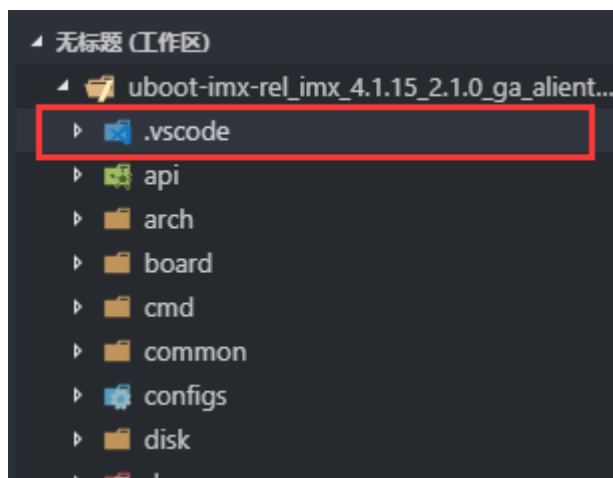


图 31.2.6 新建的.vscode 文件夹

在.vscode 文件夹中新建一个名为“settings.json”的文件，然后在 settings.json 中输入如下内容：

示例代码 31.2.1 settings.json 文件代码

```
1 {
2     "search.exclude": {
3         "**/node_modules": true,
4         "**/bower_components": true,
5     },
6     "files.exclude": {
7         "**/.git": true,
8         "**/.svn": true,
9         "**/.hg": true,
10        "**/CVS": true,
11        "**/.DS_Store": true,
12    }
13 }
```

结果如图 31.2.7 所示：

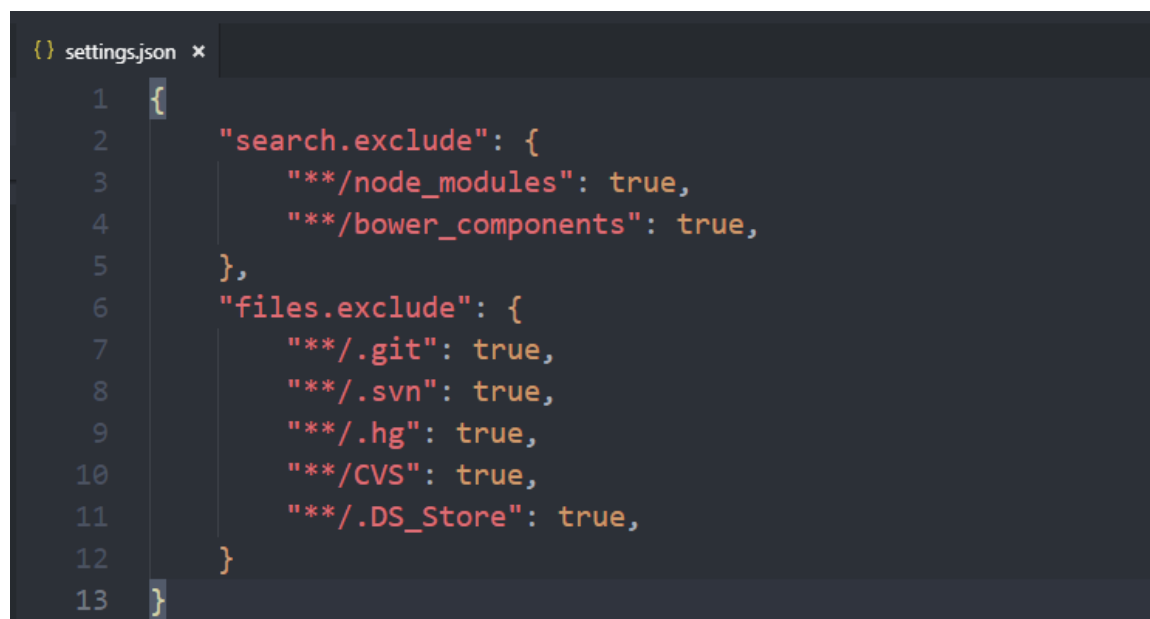
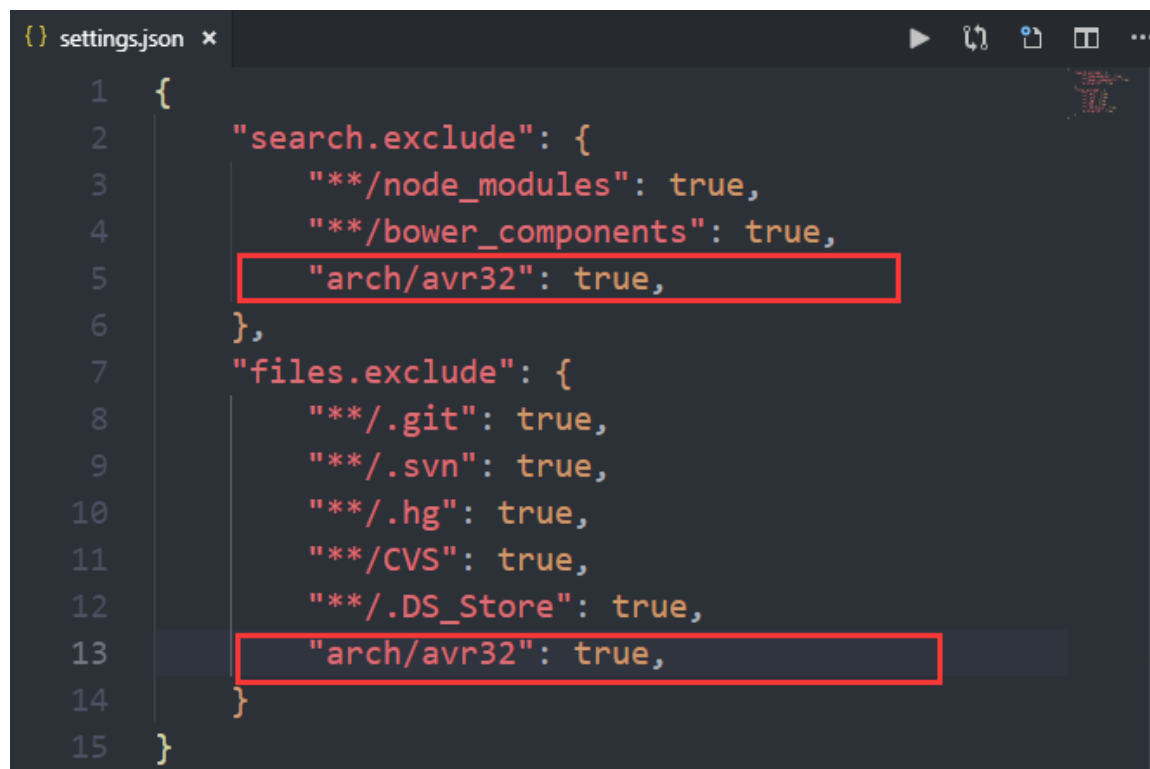


图 31.2.7 settings.json 文件默认内容

其中“search.exclude”里面是需要在搜索结果中排除的文件或者文件夹，“files.exclude”是左侧工程目录中需要排除的文件或者文件夹。我们需要将 arch/avr32 文件夹下的所有文件从搜索结果和左侧的工程目录中都排除掉，因此在“search.exclude”和“files.exclude”中输入如图 31.2.8 所示内容：



```
{
  "search.exclude": {
    "**/node_modules": true,
    "**/bower_components": true,
    "arch/avr32": true,
  },
  "files.exclude": {
    "**/.git": true,
    "**/.svn": true,
    "**/.hg": true,
    "**/CVS": true,
    "**/.DS_Store": true,
    "arch/avr32": true,
  }
}
```

图 31.2.8 排除 arch/avr32 目录

保存一下 settings.json 文件，然后再看一下左侧的工程目录，发现 arch 目录下没有 avr32 这个文件夹了，说明 avr32 这个文件夹被排除掉了，如图 31.2.9 所示：

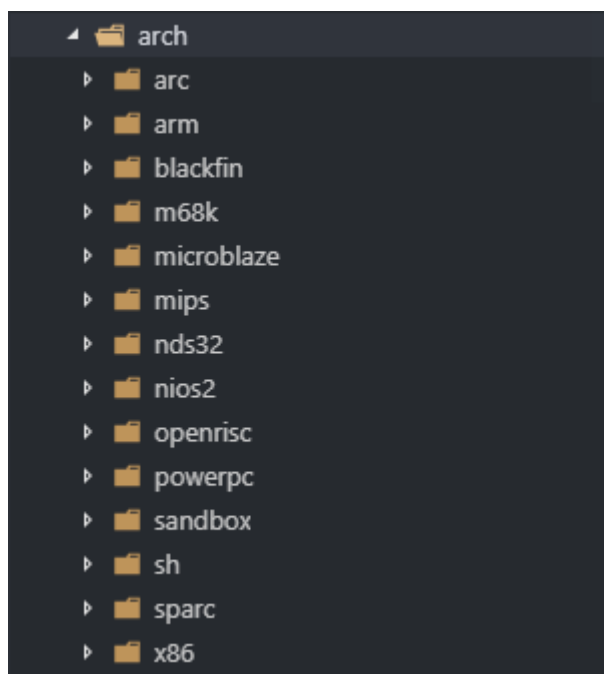


图 31.2.9 arch/avr32 目录排除

我们只是在"search.exclude"和"files.exclude"中加入了："arch/avr32": true，冒号前面的是要排除的文件或者文件夹，冒号后面为是否将文件排除，true 表示排除，false 表示不排除。用这种方法即可将不需要的文件，或者文件夹排除掉，对于本章我们分析 uboot 而言，在

"search.exclude"和"files.exclude"中需要输入的完成的内容如下:

示例代码 31.2.2 settings.json 文件代码

```
1  "**/*.o":true,
2  "**/*.su":true,
3  "**/*.cmd":true,
4  "arch/arc":true,
5  "arch/avr32":true,
6  "arch/blackfin":true,
7  "arch/m68k":true,
8  "arch/microblaze":true,
9  "arch/mips":true,
10 "arch/nds32":true,
11 "arch/nios2":true,
12 "arch/openrisc":true,
13 "arch/powerpc":true,
14 "arch/sandbox":true,
15 "arch/sh":true,
16 "arch/sparc":true,
17 "arch/x86":true,
18 "arch/arm/mach*":true,
19 "arch/arm/cpu/arm11*":true,
20 "arch/arm/cpu/arm720t":true,
21 "arch/arm/cpu/arm9*":true,
22 "arch/arm/cpu/armv7m":true,
23 "arch/arm/cpu/armv8":true,
24 "arch/arm/cpu/pxa":true,
25 "arch/arm/cpu/sa1100":true,
26 "board/[a-e]*":true,
27 "board/[g-z]*":true,
28 "board/[0-9]*":true,
29 "board/[A-Z]*":true,
30 "board/fir*":true,
31 "board/freescale/b*":true,
32 "board/freescale/l*":true,
33 "board/freescale/m5*":true,
34 "board/freescale/mp*":true,
35 "board/freescale/c29*":true,
36 "board/freescale/cor*":true,
37 "board/freescale/mx7*":true,
38 "board/freescale/mx2*":true,
39 "board/freescale/mx3*":true,
40 "board/freescale/mx5*":true,
41 "board/freescale/p*":true,
```

```
42 "board/freescale/q*":true,  
43 "board/freescale/t*":true,  
44 "board/freescale/v*":true,  
45 "configs/[a-l]*":true,  
46 "configs/[n-z]*":true,  
47 "configs/[A-Z]*":true,  
48 "configs/M[a-z]*":true,  
49 "configs/M[A-Z]*":true,  
50 "configs/M[0-9]*":true,  
51 "configs/m[a-w]*":true,  
52 "configs/m[0-9]*":true,  
53 "configs/[0-9]*":true,  
54 "include/configs/[a-l]*":true,  
55 "include/configs/[n-z]*":true,  
56 "include/configs/[A-Z]*":true,  
57 "include/configs/m[a-w]*":true,
```

上述代码用到了通配符“*”，比如“**/*.o”表示所有.o结尾的文件。“configs/[a-l]*”表示 configs 目录下所有以‘a’~‘l’开头的文件或者文件夹。上述配置只是排除了一部分文件夹，大家在实际的使用中可以根据自己的实际需求来选择将哪些文件或者文件夹排除掉。排除以后我们的工程就会清爽很多，搜索的时候也不会跳出很多文件了。

31.3 U-Boot 顶层 Makefile 分析

在阅读 uboot 源码之前，肯定是要先看一下顶层 Makefile，分析 gcc 版本代码的时候一定是先从顶层 Makefile 开始的，然后再是子 Makefile，这样通过层层分析 Makefile 即可了解整个工程的组织结构。顶层 Makefile 也就是 uboot 根目录下的 Makefile 文件，由于顶层 Makefile 文件内容比较多，所以我们将其分开来看。

31.3.1 版本号

顶层 Makefile 一开始是版本号，内容如下(为了方便分析，顶层 Makefile 代码段前段行号采用 Makefile 中的行号，因为 uboot 会更新，因此行号可能会与你所看的顶层 Makefile 有所不同)：

示例代码 31.3.1.1 顶层 Makefile 代码

```
5 VERSION = 2016  
6 PATCHLEVEL = 03  
7 SUBLEVEL =  
8 EXTRAVERSION =  
9 NAME =
```

VERSION 是主版本号，PATCHLEVEL 是补丁版本号，SUBLEVEL 是次版本号，这三个一起构成了 uboot 的版本号，比如当前的 uboot 版本号就是“2016.03”。EXTRAVERSION 是附加版本信息，NAME 是和名字有关的，一般不使用这两个。

31.3.2 MAKEFLAGS 变量

make 是支持递归调用的，也就是在 Makefile 中使用“make”命令来执行其他的 Makefile 文件，一般都是子目录中的 Makefile 文件。假如在当前目录下存在一个“subdir”子目录，这个子目录中又有其对应的 Makefile 文件，那么这个工程在编译的时候其主目录中的 Makefile 就可以调用子目录中的 Makefile，以此来完成所有子目录的编译。主目录的 Makefile 可以使用如下代码来编译这个子目录：

```
$(MAKE) -C subdir
```

\$(MAKE)就是调用“make”命令，-C 指定子目录。有时候我们需要向子 make 传递变量，这个时候使用“export”来导出要传递给子 make 的变量即可，如果不希望哪个变量传递给子 make 的话就使用“unexport”来声明不导出：

```
export VARIABLE ..... //导出变量给子 make 。
unexport VARIABLE..... //不导出变量给子 make。
```

有两个特殊的变量：“SHELL”和“MAKEFLAGS”，这两个变量除非使用“unexport”声明，否则的话在整个 make 的执行过程中，它们的值始终自动的传递给子 make。在 uboot 的主 Makefile 中有如下代码：

示例代码 31.3.2.1 顶层 Makefile 代码

```
20 MAKEFLAGS += -rR --include-dir=$(CURDIR)
```

上述代码使用“+=”来给变量 MAKEFLAGS 追加了一些值，“-rR”表示禁止使用内置的隐含规则和变量定义，“--include-dir”指明搜索路径，“\$(CURDIR)”表示当前目录。

31.3.3 命令输出

uboot 默认编译是不会在终端中显示完整的命令，都是短命令，如图 31.3.3 所示：

```
CC      examples/standalone/stubs.o
LD      examples/standalone/libstubs.o
CC      examples/standalone/hello_world.o
LD      examples/standalone/hello_world
OBJCOPY examples/standalone/hello_world.srec
OBJCOPY examples/standalone/hello_world.bin
LDS      u-boot.lds
LD      u-boot
OBJCOPY u-boot-nodtb.bin
COPY     u-boot.bin
CFG      board/freescale/mx6ull_alientek_emmc/imximage.cfg.cfgtmp
MKIMAGE u-boot.imx
OBJCOPY u-boot.srec
SYM      u-boot.sym
CFG      u-boot.cfg
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/uboot-imx-rel_imx_4.1.15_2.1.0_ga_alientek$
```

图 31.3.3.1 终端短命令输出

在终端中输出短命令虽然看起来很清爽，但是不利于分析 uboot 的编译过程。可以通过设置变量“V=1”来实现完整的命令输出，这个在调试 uboot 的时候很有用，结果如图 31.3.3.2 所示：

```

arm-linux-gnueabi-hf-ld.bfd -pie --gc-sections -Bstatic -Ttext 0x87800000 -o u-boot -T u-boot.lds arch/arm/c
pu/armv7/start.o --start-group arch/arm/cpu/built-in.o arch/arm/cpu/armv7/built-in.o arch/arm/imx-common/buil
t-in.o arch/arm/lib/built-in.o board/freescale/common/built-in.o board/freescale/mx6ull_alientek_emmc/built-i
n.o cmd/built-in.o common/built-in.o disk/built-in.o drivers/built-in.o drivers/dma/built-in.o drivers/gpi
o/built-in.o drivers/i2c/built-in.o drivers/mmc/built-in.o drivers/mtd/built-in.o drivers/mtd/onenand/built-
in.o drivers/mtd/spi/built-in.o drivers/net/built-in.o drivers/net/phy/built-in.o drivers/pci/built-in.o dr
ivers/power/built-in.o drivers/power/battery/built-in.o drivers/power/fuel_gauge/built-in.o drivers/power/mfd
/built-in.o drivers/power/pmic/built-in.o drivers/power/regulator/built-in.o drivers/serial/built-in.o drive
rs/spi/built-in.o drivers/usb/dwc3/built-in.o drivers/usb/emul/built-in.o drivers/usb/eth/built-in.o drivers
/usb/gadget/built-in.o drivers/usb/gadget/udc/built-in.o drivers/usb/host/built-in.o drivers/usb/musb-new/bui
lt-in.o drivers/usb/musb/built-in.o drivers/usb/phy/built-in.o drivers/usb/ulpi/built-in.o fs/built-in.o li
b/usb/usbip/built-in.o net/built-in.o test/built-in.o test/dm/built-in.o --end-group arch/arm/lib/eabi_compat.o -L /us
r/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi-hf/bin/./lib/gcc/arm-linux-gnueabi-hf/4.9.4 -lgcc -
Map u-boot.map
arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -j .text -j .secure_text -j .rodata -j .hash -j .data -j .got -j
.got.plt -j .u_boot_list -j .rel.dyn -O binary u-boot u-boot-nodtb.bin
cp u-boot-nodtb.bin u-boot.bin
make -f ./scripts/Makefile.build obj=arch/arm/imx-common u-boot.imx
mkdir -p board/freescale/mx6ull_alientek_emmc/
./tools/mkimage -n board/freescale/mx6ull_alientek_emmc/imximage.cfg.cfgtmp -T imximage -e 0x87800000 -d u-bo
t.bin u-boot.imx
Image Type:      Freescale IMX Boot Image
Image Ver:       2 (i.MX53/6/7 compatible)
Mode:            DCD
Data Size:       425984 Bytes = 416.00 kB = 0.41 MB
Load Address:    877ff420
Entry Point:     87800000
arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -j .text -j .secure_text -j .rodata -j .hash -j .data -j .got -j
.got.plt -j .u_boot_list -j .rel.dyn -O srec u-boot u-boot.srec
arm-linux-gnueabi-hf-objdump -t u-boot > u-boot.sym
zuo zhong kai@ubuntu:~/linux/IMX6ULL/uboot/uboot-imx-rel imx 4.1.15 2.1.0 qa alientek$

```

图 31.3.3.2 终端完整命令输出

顶层 Makefile 中控制命令输出的代码如下:

示例代码 31.3.3.1 顶层 Makefile 代码

```

73 ifeq ("$(origin V)", "command line")
74     KBUILD_VERBOSE = $(V)
75 endif
76 ifndef KBUILD_VERBOSE
77     KBUILD_VERBOSE = 0
78 endif
79
80 ifeq ($(KBUILD_VERBOSE), 1)
81     quiet =
82     Q =
83 else
84     quiet=quiet_
85     Q = @
86 endif

```

上述代码中先使用 ifeq 来判断 "\$(origin V)" 和 "command line" 是否相等。这里用到了 Makefile 中的函数 origin，origin 和其他的函数不一样，它不操作变量的值，origin 用于告诉你变量是哪来的，语法为：

\$(origin <variable>)

variable 是变量名，origin 函数的返回值就是变量来源，因此 \$(origin V) 就是变量 V 的来源。如果变量 V 是在命令行定义的那么它的来源就是 "command line"，这样 "\$(origin V)" 和 "command line" 就相等了。当这两个相等的时候变量 KBUILD_VERBOSE 就等于 V 的值，比如在命令行中输入 “V=1” 的话那么 KBUILD_VERBOSE=1。如果没有在命令行输入 V 的话 KBUILD_VERBOSE=0。

第 80 行判断 KBUILD_VERBOSE 是否为 1，如果 KBUILD_VERBOSE 为 1 的话变量 quiet

和 Q 都为空，如果 KBUILD_VERBOSE=0 的话变量 quiet 为 “quiet_”，变量 Q 为 “@”，综上所述：

V=1 的话：

KBUILD_VERBOSE=1

quiet= 空。

Q= 空。

V=0 或者命令行不定义 V 的话：

KBUILD_VERBOSE=0

quiet= quiet_。

Q= @。

Makefile 中会用到变量 quiet 和 Q 来控制编译的时候是否在终端输出完整的命令，在顶层 Makefile 中有很多如下所示的命令：

```
$(Q)$(MAKE) $(build)=tools
```

如果 V=0 的话上述命令展开就是 “@ make \$(build)=tools”，make 在执行的时候默认会在终端输出命令，但是在命令前面加上 “@” 就不会在终端输出命令了。当 V=1 的时候 Q 就为空，上述命令就是 “make \$(build)=tools”，因此在 make 执行的过程，命令会被完整的输出在终端上。

有些命令会有两个版本，比如：

```
quiet_cmd_sym ?= SYM      $@
```

```
cmd_sym ?= $(OBJDUMP) -t $< > $@
```

sym 命令分为 “quiet_cmd_sym” 和 “cmd_sym” 两个版本，这两个命令的功能都是一样的，区别在于 make 执行的时候输出的命令不同。quiet_cmd_xxx 命令输出信息少，也就是短命令，而 cmd_xxx 命令输出信息多，也就是完整的命令。

如果变量 quiet 为空的话，整个命令都会输出。

如果变量 quiet 为 “quiet_” 的话，仅输出短版本。

如果变量 quiet 为 “silent_” 的话，整个命令都不会输出。

31.3.4 静默输出

上一小节讲了，设置 V=0 或者在命令行中不定义 V 的话，编译 uboot 的时候终端中显示的短命令，但是还是会有命令输出，有时候我们在编译 uboot 的时候不需要输出命令，这个时候就可以使用 uboot 的静默输出功能。编译的时候使用 “make -s” 即可实现静默输出，顶层 Makefile 中相应的代码如下：

示例代码 31.3.4.1 顶层 Makefile 代码

```
88 # If the user is running make -s (silent mode), suppress echoing of
89 # commands
90
91 ifneq ($(filter 4.%, $(MAKE_VERSION)),) # make-4
92 ifneq ($(filter %s, $(firstword x$(MAKEFLAGS))),)
93     quiet=silent_
94 endif
95 else # make-3.8x
96 ifneq ($(filter s% -s%, $(MAKEFLAGS)),)
97     quiet=silent_
98 endif
```

```

99 endif
100
101 export quiet Q KBUILD_VERBOSE

```

第 91 行判断当前正在使用的编译器版本号是否为 4.x, 判断 $\$(filter\ 4.\%,\$(MAKE_VERSION))$ 和 “ ” (空) 是否相等, 如果不相等的话就成立, 执行里面的语句。也就是说 $\$(filter\ 4.\%,\$(MAKE_VERSION))$ 不为空的话条件就成立, 这里用到了 Makefile 中的 filter 函数, 这是个过滤函数, 函数格式如下:

$\$(filter\ <pattern...\>,\<text\>)$

filter 函数表示以 pattern 模式过滤 text 字符串中的单词, 仅保留符合模式 pattern 的单词, 可以有多个模式。函数返回值就是符合 pattern 的字符串。因此 $\$(filter\ 4.\%,\$(MAKE_VERSION))$ 的含义就是在字符串 “MAKE_VERSION” 中找出符合 “4.%” 的字符(%为通配符), MAKE_VERSION 是 make 工具的版本号, ubuntu16.04 里面默认自带的 make 工具版本号为 4.1, 大家可以输入 “make -v” 查看。因此 $\$(filter\ 4.\%,\$(MAKE_VERSION))$ 不为空, 条件成立, 执行 92~94 行的语句。

第 92 行也是一个判断语句, 如果 $\$(filter\ \%s,\$(firstword\ x\$(MAKEFLAGS)))$ 不为空的话条件成立, 变量 quiet 等于 “silent_”。这里也用到了函数 filter, 在 $\$(firstword\ x\$(MAKEFLAGS))$ 中过滤出符合 “%s” 的单词。到了函数 firstword, 函数 firstword 是获取首单词, 函数格式如下:

$\$(firstword\ <text\>)$

firstword 函数用于取出 text 字符串中的第一个单词, 函数的返回值就是获取到的单词。当使用 “make -s” 编译的时候, “-s” 会作为 MAKEFLAGS 变量的一部分传递给 Makefile。在顶层 Makefile 中添加如图 31.3.4.1 所示的代码:

```

85 Q = @
86 endif
87
88 # If the user is running make -s (silent mode), suppress echoing of
89 # commands
90
91 ifneq ($(filter 4.%, $(MAKE_VERSION)),) # make-4
92 ifneq ($(filter %s, $(firstword x$(MAKEFLAGS))),)
93   quiet=silent_
94 endif
95 else # make-3.8x
96 ifneq ($(filter %s -s%, $(MAKEFLAGS)),)
97   quiet=silent_
98 endif
99 endif
100
101 export quiet Q KBUILD_VERBOSE
102
103 mytest:
104   @echo 'firstword=' $(firstword x$(MAKEFLAGS))
105
106
107 # kbuild supports saving output files in a separate directory.

```

图 31.3.4.1 顶层 Makefile 添加代码

图 31.3.4.1 中的两行代码用于输出 $\$(firstword\ x\$(MAKEFLAGS))$ 的结果, 最后修改文件 mx6ull_alientek_emmc.sh, 在里面加入 “-s” 选项, 结果如图 31.3.4.2 所示:

```

zuozhongkai@ubuntu: ~/linux/IMX6ULL/u-boot/alientek_uboot
1 #!/bin/bash
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mx6ull_14x14_ddr512_emmc_defconfig
4 make -s ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j12

```

图 31.3.4.2 加入 -s 选项

修改完成以后执行 `mx6ull_alientek_emmc.sh`, 结果如图 31.3.4.3 所示:

```

zuo zhong kai@ubuntu:~/linux/IMX6ULL/uboot/uboot-imx-rel_imx_4.1.15_2.1.0_ga_alientek$ ./mx6ull_alientek_emmc.sh
CLEAN      scripts/basic
CLEAN      scripts/kconfig
CLEAN      include/config include/generated
CLEAN      .config include/autoconf.mk include/autoconf.mk.dep include/config.h
HOSTCC     scripts/basic/fixdep
HOSTCC     scripts/kconfig/conf.o
SHIPPED    scripts/kconfig/zconf.tab.c
SHIPPED    scripts/kconfig/zconf.lex.c
SHIPPED    scripts/kconfig/zconf.hash.c
HOSTCC     scripts/kconfig/zconf.tab.o
HOSTLD     scripts/kconfig/conf
#
# configuration written to .config
#
firstword= xrRs
zuo zhong kai@ubuntu:~/linux/IMX6ULL/uboot/uboot-imx-rel_imx_4.1.15_2.1.0_ga_alientek$

```

图 31.3.4.3 修改顶层 Makefile 后的执行结果

从图 31.3.4.3 可以看出第一个单词是“xrRs”，将`$(filter %s, $(firstword x$(MAKEFLAGS)))`展开就是`$(filter %s, xrRs)`，而`$(filter %s, xrRs)`的返回值肯定不为空，条件成立，`quiet=silent_`。第 101 行 使用 `export` 导出变量 `quiet`、`Q` 和 `KBUILD_VERBOSE`。

31.3.5 设置编译结果输出目录

uboot 可以将编译出来的目标文件输出到单独的目录中，在 `make` 的时候使用“O”来指定输出目录，比如“`make O=out`”就是设置目标文件输出到 `out` 目录中。这么做是为了将源文件和编译产生的文件分开，当然也可以不指定 `O` 参数，不指定的话源文件和编译产生的文件都在同一个目录内，一般我们指定 `O` 参数。顶层 Makefile 中相关的代码如下：

示例代码 31.3.5.1 顶层 Makefile 代码

```

103 # kbuild supports saving output files in a separate directory.
104 # To locate output files in a separate directory two syntaxes are
    supported.
105 # In both cases the working directory must be the root of the
    kernel src.
106 # 1) O=
107 # Use "make O=dir/to/store/output/files/"
108 #
109 # 2) Set KBUILD_OUTPUT
110 # Set the environment variable KBUILD_OUTPUT to point to the
    directory
111 # where the output files shall be placed.
112 # export KBUILD_OUTPUT=dir/to/store/output/files/
113 # make
114 #
115 # The O= assignment takes precedence over the KBUILD_OUTPUT
    environment
116 # variable.
117
118 # KBUILD_SRC is set on invocation of make in OBJ directory
119 # KBUILD_SRC is not intended to be used by the regular user (for
    now)
120 ifeq ($(KBUILD_SRC),)

```

```

121
122 # OK, Make called in directory where kernel src resides
123 # Do we want to locate output files in a separate directory?
124 ifeq ("$(origin O)", "command line")
125     KBUILD_OUTPUT := $(O)
126 endif
127
128 # That's our default target when none is given on the command line
129 PHONY := _all
130 _all:
131
132 # Cancel implicit rules on top Makefile
133 $(CURDIR)/Makefile Makefile: ;
134
135 ifneq ($(KBUILD_OUTPUT),)
136 # Invoke a second make in the output directory, passing relevant
variables
137 # check that the output directory actually exists
138 saved-output := $(KBUILD_OUTPUT)
139 KBUILD_OUTPUT := $(shell mkdir -p $(KBUILD_OUTPUT) && cd
$(KBUILD_OUTPUT) \
140                                     && /bin/pwd)
.....
155 endif # ifneq ($(KBUILD_OUTPUT),)
156 endif # ifeq ($(KBUILD_SRC),)

```

第 124 行判断“O”是否来自于命令行，如果来自命令行的话条件成立，KBUILD_OUTPUT 就为\$(O)，因此变量 KBUILD_OUTPUT 就是输出目录。

第 135 行判断 KBUILD_OUTPUT 是否为空。

第 139 行调用 mkdir 命令，创建 KBUILD_OUTPUT 目录，并且将创建成功以后的绝对路径赋值给 KBUILD_OUTPUT。至此，通过 O 指定的输出目录就存在了。

31.3.6 代码检查

uboot 支持代码检查，使用命令“make C=1”使能代码检查，检查那些需要重新编译的文件。“make C=2”用于检查所有的源码文件，顶层 Makefile 中的代码如下：

示例代码 31.3.6.1 顶层 Makefile 代码

```

166 # Call a source code checker (by default, "sparse") as part of the
167 # C compilation.
168 #
169 # Use 'make C=1' to enable checking of only re-compiled files.
170 # Use 'make C=2' to enable checking of *all* source files,
regardless
171 # of whether they are re-compiled or not.
172 #

```

```
173 # See the file "Documentation/sparse.txt" for more details,
including
174 # where to get the "sparse" utility.
175
176 ifeq ("$(origin C)", "command line")
177     KBUILD_CHECKSRC = $(C)
178 endif
179 ifndef KBUILD_CHECKSRC
180     KBUILD_CHECKSRC = 0
181 endif
```

第 176 行判断 C 是否来源于命令行, 如果 C 来源于命令行, 那就将 C 赋值给变量 KBUILD_CHECKSRC, 如果命令行没有 C 的话 KBUILD_CHECKSRC 就为 0。

31.3.7 模块编译

在 uboot 中允许单独编译某个模块, 使用命令 “make M=dir” 即可, 旧语法 “make SUBDIRS=dir” 也是支持的。顶层 Makefile 中的代码如下:

示例代码 31.3.7.1 顶层 Makefile 代码

```
183 # Use make M=dir to specify directory of external module to build
184 # Old syntax make ... SUBDIRS=$PWD is still supported
185 # Setting the environment variable KBUILD_EXTMOD take precedence
186 ifdef SUBDIRS
187     KBUILD_EXTMOD ?= $(SUBDIRS)
188 endif
189
190 ifeq ("$(origin M)", "command line")
191     KBUILD_EXTMOD := $(M)
192 endif
193
194 # If building an external module we do not care about the all: rule
195 # but instead _all depend on modules
196 PHONY += all
197 ifeq ($(KBUILD_EXTMOD),)
198     _all: all
199 else
200     _all: modules
201 endif
202
203 ifeq ($(KBUILD_SRC),)
204     # building in the source tree
205     srctree := .
206 else
207     ifeq ($(KBUILD_SRC)/,$(dir $(CURDIR)))
208         # building in a subdirectory of the source tree
```

```

209         srctree := ..
210     else
211         srctree := $(KBUILD_SRC)
212     endif
213 endif
214 objtree     := .
215 src        := $(srctree)
216 obj        := $(objtree)
217
218 VPATH       := $(srctree)$(if $(KBUILD_EXTMOD),,$(KBUILD_EXTMOD))
219
220 export srctree objtree VPATH

```

第 186 行判断是否定义了 SUBDIRS，如果定义了 SUBDIRS，变量 KBUILD_EXTMOD=SUBDIRS，这里是为了支持老语法“make SUBDIRS=dir”

第 190 行判断是否在命令行定义了 M，如果定义了的话 KBUILD_EXTMOD=\$(M)。

第 197 行判断 KBUILD_EXTMOD 时为空，如果为空的话目标_all 依赖 all，因此要先编译出 all。否则的话默认目标_all 依赖 modules，要先编译出 modules，也就是编译模块。一般情况下我们不会在 uboot 中编译模块，所以此处会编译 all 这个目标。

第 203 行判断 KBUILD_SRC 是否为空，如果为空的话就设置变量 srctree 为当前目录，即 srctree 为“.”，一般不设置 KBUILD_SRC。

第 214 行设置变量 objtree 为当前目录。

第 215 和 216 行分别设置变量 src 和 obj，都为当前目录。

第 218 行设置 VPATH。

第 220 行导出变量 srctree、objtree 和 VPATH。

31.3.8 获取主机架构和系统

接下来顶层 Makefile 会获取主机架构和系统，也就是我们电脑的架构和系统，代码如下：

示例代码 31.3.8.1 顶层 Makefile 代码

```

227 HOSTARCH := $(shell uname -m | \
228     sed -e s/i.86/x86/ \
229     -e s/sun4u/sparc64/ \
230     -e s/arm.*/arm/ \
231     -e s/sa110/arm/ \
232     -e s/ppc64/powerpc/ \
233     -e s/ppc/powerpc/ \
234     -e s/macppc/powerpc/ \
235     -e s/sh.*/sh/)
236
237 HOSTOS := $(shell uname -s | tr '[:upper:]' '[:lower:]' | \
238     sed -e 's/\(cygwin\).*/cygwin/')
239
240 export HOSTARCH HOSTOS

```

第 227 行定义了一个变量 HOSTARCH，用于保存主机架构，这里调用 shell 命令“uname -

m” 获取架构名称，结果如图 31.3.8.1 所示：

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_u-boot$ uname -m
x86_64
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_u-boot$
```

图 31.3.8.1 uname -m 命令

从图 31.3.8.1 可以看出当前电脑主机架构为“x86_64”，shell 中的“|”表示管道，意思是将左边的输出作为右边的输入，sed -e 是替换命令，“sed -e s/i.86/x86/”表示将管道输入的字符串中的“i.86”替换为“x86”，其他的“sed -e s”命令同理。对于我的电脑而言，HOSTARCH=x86_64。

第 237 行定义了变量 HOSTOS，此变量用于保存主机 OS 的值，先使用 shell 命令“uname -s”来获取主机 OS，结果如图 31.3.8.2 所示：

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_u-boot$ uname -s
Linux
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_u-boot$
```

图 31.3.8.2 uname -s 命令

从图 31.3.8.2 可以看出此时的主机 OS 为“Linux”，使用管道将“Linux”作为后面“tr '[:upper:]' '[:lower:]’”的输入，“tr '[:upper:]' '[:lower:]’”表示将所有的大写字母替换为小写字母，因此得到“linux”。最后同样使用管道，将“linux”作为“sed -e 's/(cygwin\).*/cygwin/'”的输入，用于将 cygwin.* 替换为 cygwin。因此，HOSTOS=linux。

第 240 行导出 HOSTARCH=x86_64，HOSTOS=linux。

31.3.9 设置目标架构、交叉编译器和配置文件

编译 u-boot 的时候需要设置目标板架构和交叉编译器，“make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-”就是用于设置 ARCH 和 CROSS_COMPILE，在顶层 Makefile 中代码如下：

示例代码 31.3.9.1 顶层 Makefile 代码

```
244 # set default to nothing for native builds
245 ifeq ($(HOSTARCH),$(ARCH))
246 CROSS_COMPILE ?=
247 endif
248
249 KCONFIG_CONFIG ?= .config
250 export KCONFIG_CONFIG
```

第 245 行判断 HOSTARCH 和 ARCH 这两个变量是否相等，主机架构(变量 HOSTARCH)是 x86_64，而我们编译的是 ARM 版本 u-boot，肯定不相等，所以 CROSS_COMPILE= arm-linux-gnueabi-。从示例代码 31.3.9.1 可以看出，每次编译 u-boot 的时候都要在 make 命令后面设置 ARCH 和 CROSS_COMPILE，使用起来很麻烦，可以直接修改顶层 Makefile，在里面加入 ARCH 和 CROSS_COMPILE 的定义，如图 31.3.9.1 所示：

```
244 # set default to nothing for native builds
245 ifeq ($(HOSTARCH),$(ARCH))
246 CROSS_COMPILE ?=
247 endif
248
249 ARCH ?= arm
250 CROSS_COMPILE ?= arm-linux-gnueabi-
251
252 KCONFIG_CONFIG ?= .config
253 export KCONFIG_CONFIG
```

图 31.3.9.1 定义 ARCH 和 CROSS_COMPILE

按照图 31.3.9.1 所示,直接在顶层 Makefile 里面定义 ARCH 和 CROSS_COMPILE,这样就不用每次编译的时候都要在 make 命令后面定义 ARCH 和 CROSS_COMPILE。

继续回到示例代码 31.3.9.1 中,第 249 行定义变量 KCONFIG_CONFIG,uboot 是可以配置的,这里设置配置文件为.config,.config 默认是没有的,需要使用命令“make xxx_defconfig”对 uboot 进行配置,配置完成以后就会在 uboot 根目录下生成.config。默认情况下.config 和 xxx_defconfig 内容是一样的,因为.config 就是从 xxx_defconfig 复制过来的。如果后续自行调整了 uboot 的一些配置参数,那么这些新的配置参数就添加到了.config 中,而不是 xxx_defconfig。相当于 xxx_defconfig 只是一些初始配置,而.config 里面的才是实时有效的配置。

31.3.10 调用 scripts/Kbuild.include

主 Makefile 会调用文件 scripts/Kbuild.include 这个文件,顶层 Makefile 中代码如下:

示例代码 31.3.10.1 顶层 Makefile 代码

```
327 # We need some generic definitions (do not try to remake the file).
328 scripts/Kbuild.include: ;
329 include scripts/Kbuild.include
```

示例代码 31.3.10.1 中使用“include”包含了文件 scripts/Kbuild.include,此文件里面定义了很多变量,如图 31.3.10.1 所示:

```
1 ###
2 # kbuild: Generic definitions
3
4 # Convenient variables
5 comma := ,
6 quote := "
7 squote := '
8 empty :=
9 space := $(empty) $(empty)
10
11 ###
12 # Name of target with a '.' as filename prefix. foo/bar.o => foo/.bar.o
13 dot-target = $(dir $@).$(notdir $@)
14
15 ###
16 # The temporary file to save gcc -MD generated dependencies must not
17 # contain a comma
18 depfile = $(subst $(comma),_,$(dot-target).d)
19
```

图 31.3.10.1 Kbuild.include 文件

在 uboot 的编译过程中会用到 scripts/Kbuild.include 中的这些变量,后面用到的时候再分析。

31.3.11 交叉编译工具变量设置

上面我们只是设置了 CROSS_COMPILE 的名字,但是交叉编译器其他的工具还没有设置,顶层 Makefile 中相关代码如下:

示例代码 31.3.11.1 顶层 Makefile 代码

```
331 # Make variables (CC, etc...)
332
333 AS = $(CROSS_COMPILE)as
334 # Always use GNU ld
335 ifneq ($(shell $(CROSS_COMPILE)ld.bfd -v 2> /dev/null),)
```

```

336 LD      = $(CROSS_COMPILE)ld.bfd
337 else
338 LD      = $(CROSS_COMPILE)ld
339 endif
340 CC      = $(CROSS_COMPILE)gcc
341 CPP     = $(CC) -E
342 AR      = $(CROSS_COMPILE)ar
343 NM      = $(CROSS_COMPILE)nm
344 LDR     = $(CROSS_COMPILE)ldr
345 STRIP   = $(CROSS_COMPILE)strip
346 OBJCOPY = $(CROSS_COMPILE)objcopy
347 OBJDUMP = $(CROSS_COMPILE)objdump

```

31.3.12 导出其他变量

接下来在顶层 Makefile 会导出很多变量，代码如下：

示例代码 31.3.12.1 顶层 Makefile 代码

```

368 export VERSION PATCHLEVEL SUBLEVEL UBOOTRELEASE UBOOTVERSION
369 export ARCH CPU BOARD VENDOR SOC CPUDIR BOARDDIR
370 export CONFIG_SHELL HOSTCC HOSTCFLAGS HOSTLDFLAGS CROSS_COMPILE AS
LD CC
371 export CPP AR NM LDR STRIP OBJCOPY OBJDUMP
372 export MAKE AWK PERL PYTHON
373 export HOSTCXX HOSTCXXFLAGS DTC CHECK CHECKFLAGS
374
375 export KBUILD_CPPFLAGS NOSTDINC_FLAGS UBOOTINCLUDE OBJCOPYFLAGS
LDFLAGS
376 export KBUILD_CFLAGS KBUILD_AFLAGS

```

这些变量中大部分都已经在前面定义了，我们重点来看一下下面这几个变量：

ARCH CPU BOARD VENDOR SOC CPUDIR BOARDDIR

这 7 个变量在顶层 Makefile 是找不到的，说明这 7 个变量是在其他文件里面定义的，先来看一下这 7 个变量都是什么内容，在顶层 Makefile 中输入如图 31.3.12.1 所示的内容：

```

372 export VERSION PATCHLEVEL SUBLEVEL UBOOTRELEASE UBOOTVERSION
373 export ARCH CPU BOARD VENDOR SOC CPUDIR BOARDDIR
374 export CONFIG_SHELL HOSTCC HOSTCFLAGS HOSTLDFLAGS CROSS_COMPILE AS LD CC
375 export CPP AR NM LDR STRIP OBJCOPY OBJDUMP
376 export MAKE AWK PERL PYTHON
377 export HOSTCXX HOSTCXXFLAGS DTC CHECK CHECKFLAGS
378
379 export KBUILD_CPPFLAGS NOSTDINC_FLAGS UBOOTINCLUDE OBJCOPYFLAGS LDFLAGS
380 export KBUILD_CFLAGS KBUILD_AFLAGS
381
382 mytest:
383     @echo 'ARCH=' $(ARCH)
384     @echo 'CPU=' $(CPU)
385     @echo 'BOARD=' $(BOARD)
386     @echo 'VENDOR=' $(VENDOR)
387     @echo 'SOC=' $(SOC)
388     @echo 'CPUDIR=' $(CPUDIR)
389     @echo 'BOARDDIR=' $(BOARDDIR)
390

```

输入这几行代码

图 31.3.12.1 输出变量值

修改好顶层 Makefile 以后执行如下命令：

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mytest
```

结果如图 31.3.12.2 所示:

```
ARCH= arm
CPU= armv7
BOARD= mx6ullevk
VENDOR= freescale
SOC= mx6
CPUDIR= arch/arm/cpu/armv7
BOARDDIR= freescale/mx6ullevk
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_uboot$
```

图 31.3.12.2 变量结果

从图 31.3.12.2 可以看到这 7 个变量的值, 这 7 个变量是从哪里来的呢? 在 uboot 根目录下有个文件叫做 config.mk, 这 7 个变量就是在 config.mk 里面定义的, 打开 config.mk 内容如下:

示例代码 31.3.12.2 config.mk 代码

```
1 #
2 # (C) Copyright 2000-2013
3 # Wolfgang Denk, DENX Software Engineering, wd@denx.de.
4 #
5 # SPDX-License-Identifier: GPL-2.0+
6 #
7
8 #####
9 # This file is included from ./Makefile and spl/Makefile.
10 # Clean the state to avoid the same flags added twice.
11 #
12 # (Tegra needs different flags for SPL.
13 # That's the reason why this file must be included from spl/Makefile
14 # If we did not have Tegra SoCs, build system would be much
15 # simpler...)
16 PLATFORM_RELFLAGS :=
17 PLATFORM_CPPFLAGS :=
18 PLATFORM_LDFLAGS :=
19 LDFLAGS :=
20 LDFLAGS_FINAL :=
21 OBJCOPYFLAGS :=
22 # clear VENDOR for tcsh
23 VENDOR :=
24
25 #####
26 ##
27
28 ARCH := $(CONFIG_SYS_ARCH:"%"=)
29 CPU := $(CONFIG_SYS_CPU:"%"=)
```

```
27 ifdef CONFIG_SPL_BUILD
28 ifdef CONFIG_TEGRA
29 CPU := arm720t
30 endif
31 endif
32 BOARD := $(CONFIG_SYS_BOARD:"%"=%)
33 ifneq ($(CONFIG_SYS_VENDOR),)
34 VENDOR := $(CONFIG_SYS_VENDOR:"%"=%)
35 endif
36 ifneq ($(CONFIG_SYS_SOC),)
37 SOC := $(CONFIG_SYS_SOC:"%"=%)
38 endif
39
40 # Some architecture config.mk files need to know what CPUDIR is set
to,
41 # so calculate CPUDIR before including ARCH/SOC/CPU config.mk files.
42 # Check if arch/$ARCH/cpu/$CPU exists, otherwise assume
arch/$ARCH/cpu contains
43 # CPU-specific code.
44 CPUDIR=arch/$(ARCH)/cpu$(if $(CPU),/$(CPU),)
45
46 sinclude $(srctree)/arch/$(ARCH)/config.mk
47 sinclude $(srctree)/$(CPUDIR)/config.mk
48
49 ifdef SOC
50 sinclude $(srctree)/$(CPUDIR)/$(SOC)/config.mk
51 endif
52 ifneq ($(BOARD),)
53 ifdef VENDOR
54 BOARDDIR = $(VENDOR)/$(BOARD)
55 else
56 BOARDDIR = $(BOARD)
57 endif
58 endif
59 ifdef BOARD
60 sinclude $(srctree)/board/$(BOARDDIR)/config.mk # include board
specific rules
61 endif
62
63 ifdef FTRACE
64 PLATFORM_CPPFLAGS += -finstrument-functions -DFTRACE
65 endif
66
```

```

67 # Allow use of stdint.h if available
68 ifneq ($(USE_STDINT),)
69 PLATFORM_CPPFLAGS += -DCONFIG_USE_STDINT
70 endif
71
72
#####
##
73
74 RELFLAGS := $(PLATFORM_RELFLAGS)
75
76 PLATFORM_CPPFLAGS += $(RELFLAGS)
77 PLATFORM_CPPFLAGS += -pipe
78
79 LDFLAGS += $(PLATFORM_LDFLAGS)
80 LDFLAGS_FINAL += -Bstatic
81
82 export PLATFORM_CPPFLAGS
83 export RELFLAGS
84 export LDFLAGS_FINAL

```

第 25 行定义变量 ARCH，值为\$(CONFIG_SYS_ARCH:"%"=%)，也就是提取 CONFIG_SYS_ARCH 里面双引号“”之间的内容。比如 CONFIG_SYS_ARCH=“arm”的话，ARCH=arm。

第 26 行定义变量 CPU，值为\$(CONFIG_SYS_CPU:"%"=%)。

第 32 行定义变量 BOARD，值为\$(CONFIG_SYS_BOARD:"%"=%)。

第 34 行定义变量 VENDOR，值为\$(CONFIG_SYS_VENDOR:"%"=%)。

第 37 行定义变量 SOC，值为\$(CONFIG_SYS_SOC:"%"=%)。

第 44 行定义变量 CPUDIR，值为 arch/\$(ARCH)/cpu\$(if \$(CPU),/\$(CPU),)。

第 46 行 **sinclude** 和 **include** 的功能类似，在 Makefile 中都是读取指定文件内容，这里读取文件\$(srctree)/arch/\$(ARCH)/config.mk 的内容。**sinclude** 读取的文件如果不存在的话不会报错。

第 47 行读取文件\$(srctree)/\$(CPUDIR)/config.mk 的内容。

第 50 行读取文件\$(srctree)/\$(CPUDIR)/\$(SOC)/config.mk 的内容。

第 54 行定义变量 BOARDDIR，如果定义了 VENDOR 那么 BOARDDIR=\$(VENDOR)/\$(BOARD)，否则的 BOARDDIR=\$(BOARD)。

第 60 行读取文件\$(srctree)/board/\$(BOARDDIR)/config.mk。

接下来需要找到 CONFIG_SYS_ARCH、CONFIG_SYS_CPU、CONFIG_SYS_BOARD、CONFIG_SYS_VENDOR 和 CONFIG_SYS_SOC 这 5 个变量的值。这 5 个变量在 uboot 根目录下的.config 文件中有定义，定义如下：

示例代码 31.3.12.3 .config 文件代码

```

23 CONFIG_SYS_ARCH="arm"
24 CONFIG_SYS_CPU="armv7"
25 CONFIG_SYS_SOC="mx6"
26 CONFIG_SYS_VENDOR="freescale"

```



```
27 CONFIG_SYS_BOARD="mx6ullevk "
28 CONFIG_SYS_CONFIG_NAME="mx6ullevk"
```

根据示例代码 31.3.12.3 可知:

```
ARCH = arm
CPU = armv7
BOARD = mx6ullevk
VENDOR = freescale
SOC = mx6
CPUDIR = arch/arm/cpu/armv7
BOARDDIR = freescale/mx6ullevk
```

在 config.mk 中读取的文件有:

```
arch/arm/config.mk
arch/arm/cpu/armv7/config.mk
arch/arm/cpu/armv7/mx6/config.mk (此文件不存在)
board/freescale/mx6ullevk/config.mk (此文件不存在)
```

31.3.13 make xxx_defconfig 过程

在编译 uboot 之前要使用 “make xxx_defconfig” 命令来配置 uboot, 那么这个配置过程是如何运行的呢? 在顶层 Makefile 中有如下代码:

示例代码 31.3.13.1 顶层 Makefile 代码段

```
414 # To make sure we do not include .config for any of the *config
targets
415 # catch them early, and hand them over to scripts/kconfig/Makefile
416 # It is allowed to specify more targets when calling make,
including
417 # mixing *config targets and build targets.
418 # For example 'make oldconfig all'.
419 # Detect when mixed targets is specified, and make a second
invocation
420 # of make so .config is not included in this case either (for
*config).
421
422 version_h := include/generated/version_autogenerated.h
423 timestamp_h := include/generated/timestamp_autogenerated.h
424
425 no-dot-config-targets := clean clobber mrproper distclean \
426     help %docs check% cooccicheck \
427     ubootversion backup
428
429 config-targets := 0
430 mixed-targets := 0
431 dot-config := 1
432
```

```

433 ifneq ($(filter $(no-dot-config-targets), $(MAKECMDGOALS)),)
434     ifeq ($(filter-out $(no-dot-config-targets), $(MAKECMDGOALS)),)
435         dot-config := 0
436     endif
437 endif
438
439 ifeq ($(KBUILD_EXTMOD),)
440     ifneq ($(filter config %config, $(MAKECMDGOALS)),)
441         config-targets := 1
442         ifneq ($(words $(MAKECMDGOALS)), 1)
443             mixed-targets := 1
444         endif
445     endif
446 endif
447
448 ifeq ($(mixed-targets), 1)
449 # =====
450 # We're called with mixed targets (*config and build targets).
451 # Handle them one by one.
452
453 PHONY += $(MAKECMDGOALS) __build_one_by_one
454
455 $(filter-out __build_one_by_one, $(MAKECMDGOALS)):
456     @:
457
458 __build_one_by_one:
459     $(Q)set -e; \
460     for i in $(MAKECMDGOALS); do \
461         $(MAKE) -f $(srctree)/Makefile $$i; \
462     done
463
464 else
465 ifeq ($(config-targets), 1)
466 # =====
467 # *config targets only - make sure prerequisites are updated, and
468 # descend
469 # in scripts/kconfig to make the *config target
470 KBUILD_DEFCONFIG := sandbox_defconfig
471 export KBUILD_DEFCONFIG KBUILD_KCONFIG
472
473 config: scripts_basic outputmakefile FORCE

```

```

474 $(Q)$(MAKE) $(build)=scripts/kconfig $@
475
476 %config: scripts_basic outputmakefile FORCE
477 $(Q)$(MAKE) $(build)=scripts/kconfig $@
478
479 else
480 #=====
481 # Build targets only - this includes vmlinux, arch specific
targets, clean
482 # targets and others. In general all targets except *config
targets.
483
484 ifeq ($(dot-config),1)
485 # Read in config
486 -include include/config/auto.conf
.....

```

第 422 行定义了变量 `version_h`，这变量保存版本号文件，此文件是自动生成的。文件 `include/generated/version_autogenerated.h` 内容如图 31.3.13.1 所示：

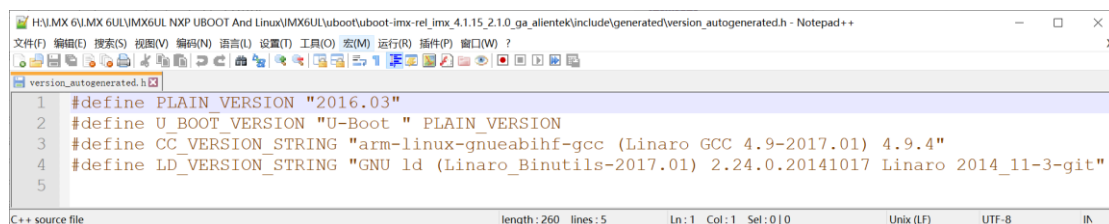


图 31.3.13.1 版本号文件

第 423 行定义了变量 `timestamp_h`，此变量保存时间戳文件，此文件也是自动生成的。文件 `include/generated/timestamp_autogenerated.h` 内容如图 31.3.13.2 所示：

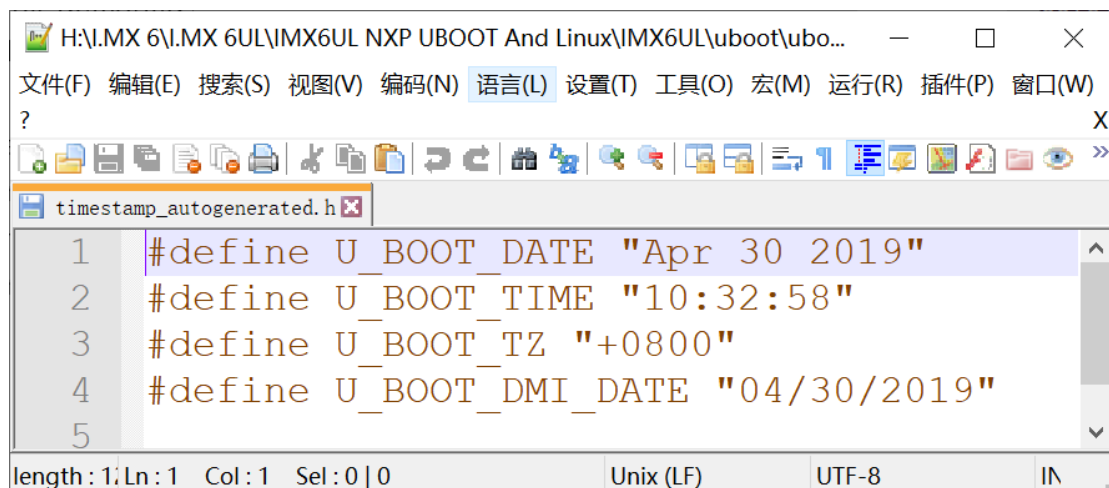


图 31.3.13.2 时间戳文件

第 425 行定义了变量 `no-dot-config-targets`。

第 429 行定义了变量 `config-targets`，初始值为 0。

第 430 行定义了变量 `mixed-targets`，初始值为 0。

第 431 行定义了变量 `dot-config`，初始值为 1。

第 433 行将 `MAKECMDGOALS` 中不符合 `no-dot-config-targets` 的部分过滤掉，剩下的如果不为空的话条件就成立。`MAKECMDGOALS` 是 `make` 的一个环境变量，这个变量会保存你所指定的终极目标列表，比如执行“`make mx6ull_alientek_emmc_defconfig`”，那么 `MAKECMDGOALS` 就为 `mx6ull_alientek_emmc_defconfig`。很明显过滤后为空，所以条件不成立，变量 `dot-config` 依旧为 1。

第 439 行判断 `KBUILD_EXTMOD` 是否为空，如果 `KBUILD_EXTMOD` 为空的话条件成立，经过前面的分析，我们知道 `KBUILD_EXTMOD` 为空，所以条件成立。

第 440 行将 `MAKECMDGOALS` 中不符合“`config`”和“`%config`”的部分过滤掉，如果剩下的部分不为空条件就成立，很明显此处条件成立，变量 `config-targets=1`。

第 442 行统计 `MAKECMDGOALS` 中的单词个数，如果不为 1 的话条件成立。此处调用 Makefile 中的 `words` 函数来统计单词个数，`words` 函数格式如下：

```
$(words <text>)
```

很明显，`MAKECMDGOALS` 的单词个数是 1 个，所以条件不成立，`mixed-targets` 继续为 0。综上所述，这些变量值如下：

```
config-targets = 1
mixed-targets = 0
dot-config = 1
```

第 448 行如果变量 `mixed-targets` 为 1 的话条件成立，很明显，条件不成立。

第 465 行如果变量 `config-targets` 为 1 的话条件成立，很明显，条件成立，执行这个分支。

第 473 行，没有目标与之匹配，所以不执行。

第 476 行，有目标与之匹配，当输入“`make xxx_defconfig`”的时候就会匹配到 `%config` 目标，目标“`%config`”依赖于 `scripts_basic`、`outputmakefile` 和 `FORCE`。`FORCE` 在顶层 Makefile 的 1610 行有如下定义：

示例代码 31.3.13.2 顶层 Makefile 代码段

```
1610 PHONY += FORCE
1611 FORCE:
```

可以看出 `FORCE` 是没有规则和依赖的，所以每次都会重新生成 `FORCE`。当 `FORCE` 作为其他目标的依赖时，由于 `FORCE` 总是被更新过的，因此依赖所在的规则总是会执行的。

依赖 `scripts_basic` 和 `outputmakefile` 在顶层 Makefile 中的内容如下：

示例代码 31.3.13.3 顶层 Makefile 代码段

```
394 # Basic helpers built in scripts/
395 PHONY += scripts_basic
396 scripts_basic:
397     $(Q)$(MAKE) $(build)=scripts/basic
398     $(Q)rm -f .tmp_quiet_recordmcount
399
400 # To avoid any implicit rule to kick in, define an empty command.
401 scripts/basic/%: scripts_basic ;
402
403 PHONY += outputmakefile
404 # outputmakefile generates a Makefile in the output directory, if
405 # using a separate output directory. This allows convenient use of
```

```

406 # make in the output directory.
407 outputmakefile:
408 ifneq ($(KBUILD_SRC),)
409     $(Q)ln -fsn $(srctree) source
410     $(Q)$(CONFIG_SHELL) $(srctree)/scripts/mkmakefile \
411         $(srctree) $(objtree) $(VERSION) $(PATCHLEVEL)
412 endif

```

第 408 行, 判断 KBUILD_SRC 是否为空, 只有变量 KBUILD_SRC 不为空的时候 outputmakefile 才有意义, 经过我们前面的分析 KBUILD_SRC 为空, 所以 outputmakefile 无效。只有 scripts_basic 是有效的。

第 396~398 行是 scripts_basic 的规则, 其对应的命令用到了变量 Q、MAKE 和 build, 其中:

Q=@或为空

MAKE=make

变量 build 是在 scripts/Kbuild.include 文件中有定义, 定义如下:

示例代码 31.3.13.3 Kbuild.include 代码段

```

177 ###
178 # Shorthand for $(Q)$(MAKE) -f scripts/Makefile.build obj=
179 # Usage:
180 # $(Q)$(MAKE) $(build)=dir
181 build := -f $(srctree)/scripts/Makefile.build obj

```

从示例代码 31.3.13.3 可以看出 build=-f\$(srctree)/scripts/Makefile.build obj, 经过前面的分析可知, 变量 srctree 为".", 因此:

build=-f ./scripts/Makefile.build obj

scripts_basic 展开以后如下:

scripts_basic:

@make -f ./scripts/Makefile.build obj=scripts/basic //也可以没有@, 视配置而定

@rm -f .tmp_quiet_recordmcount //也可以没有@

scripts_basic 会调用文件./scripts/Makefile.build, 这个我们后面在分析。

接着回到示例代码 31.3.13.1 中的%config 处, 内容如下:

%config: scripts_basic outputmakefile FORCE

\$(Q)\$(MAKE) \$(build)=scripts/kconfig \$@

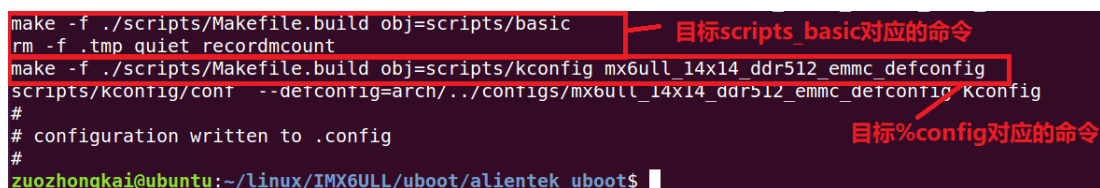
将命令展开就是:

@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig

同样也跟文件./scripts/Makefile.build 有关, 我们后面再分析此文件。使用如下命令配置 uboot, 并观察其配置过程:

make mx6ull_14x14_ddr512_emmc_defconfig V=1

配置过程如图 31.3.13.1 所示:



```

make -f ./scripts/Makefile.build obj=scripts/basic
rm -f .tmp_quiet_recordmcount
make -f ./scripts/Makefile.build obj=scripts/kconfig mx6ull_14x14_ddr512_emmc_defconfig
scripts/kconfig/conf --defconfig=arch/./configs/mx6ull_14x14_ddr512_emmc_defconfig Kconfig
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek uboot$

```

图 31.3.13.1 uboot 配置过程

从图 31.3.13.1 可以看出,我们的分析是正确的,接下来就要结合下面两行命令重点分析一下文件 scripts/Makefile.build。

①、scripts_basic 目标对应的命令

```
@make -f ./scripts/Makefile.build obj=scripts/basic
```

②、%config 目标对应的命令

```
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

31.3.14 Makefile.build 脚本分析

从上一小节可知,“make xxx_defconfig”配置 uboot 的时候如下两行命令会执行脚本 scripts/Makefile.build:

```
@make -f ./scripts/Makefile.build obj=scripts/basic
```

```
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

依次来分析一下:

1、scripts_basic 目标对应的命令

scripts_basic 目标对应的命令为: @make -f ./scripts/Makefile.build obj=scripts/basic。打开文件 scripts/Makefile.build, 有如下代码:

示例代码 31.3.14.1 Makefile.build 代码段

```
8 # Modified for U-Boot
9 prefix := tpl
10 src := $(patsubst $(prefix)/%,%, $(obj))
11 ifeq ($(obj), $(src))
12 prefix := spl
13 src := $(patsubst $(prefix)/%,%, $(obj))
14 ifeq ($(obj), $(src))
15 prefix := .
16 endif
17 endif
```

第 9 行定义了变量 prefix 值为 tpl。

第 10 行定义了变量 src, 这里用到了函数 patsubst, 此行代码展开后为:

```
$(patsubst tpl/%,%, scripts/basic)
```

patsubst 是替换函数, 格式如下:

```
$(patsubst <pattern>, <replacement>, <text>)
```

此函数用于在 text 中查找符合 pattern 的部分, 如果匹配的话就用 replacement 替换掉。pattern 是可以包含通配符 “%”, 如果 replacement 中也包含通配符 “%”, 那么 replacement 中的这个 “%” 将是 pattern 中的那个 “%” 所代表的字符串。函数的返回值为替换后的字符串。因此, 第 10 行就是在 “scripts/basic” 中查找符合 “tpl/%” 的部分, 然后将 “tpl/” 取消掉, 但是 “scripts/basic” 没有 “tpl/”, 所以 src= scripts/basic。

第 11 行判断变量 obj 和 src 是否相等, 相等的话条件成立, 很明显, 此处条件成立。

第 12 行和第 9 行一样, 只是这里处理的是 “spl”, “scripts/basic” 里面也没有 “spl/”, 所以 src 继续为 scripts/basic。

第 15 行因为变量 obj 和 src 相等, 所以 prefix=.

继续分析 scripts/Makefile.build, 有如下代码:

示例代码 31.3.14.2 Makefile.build 代码段

```

56 # The filename Kbuild has precedence over Makefile
57 kbuild-dir := $(if $(filter /%, $(src)), $(src), $(srctree)/$(src))
58 kbuild-file := $(if $(wildcard $(kbuild-dir)/Kbuild), $(kbuild-dir)/Kbuild, $(kbuild-dir)/Makefile)
59 include $(kbuild-file)

```

将 kbuild-dir 展开后为:

```
$(if $(filter /%, scripts/basic), scripts/basic, ./scripts/basic),
```

因为没有以 “/” 为开头的单词，所以 \$(filter /%, scripts/basic) 的结果为空，kbuild-dir=./scripts/basic。

将 kbuild-file 展开后为:

```
$(if $(wildcard ./scripts/basic/Kbuild), ./scripts/basic/Kbuild, ./scripts/basic/Makefile)
```

因为 scripts/basic 目录中没有 Kbuild 这个文件，所以 kbuild-file=./scripts/basic/Makefile。最后将 59 行展开，即:

```
include ./scripts/basic/Makefile
```

也就是读取 scripts/basic 下面的 Makefile 文件。

继续分析 scripts/Makefile.build，如下代码:

示例代码 31.3.14.3 Makefile.build 代码段

```

116 __build: $(if $(KBUILD_BUILTIN), $(builtin-target) $(lib-target) \
117           $(if $(KBUILD_MODULES), $(obj-m) $(modorder-target)) \
118           $(subdir-ym) $(always)
119 @:

```

__build 是默认目标，因为命令 “@make -f ./scripts/Makefile.build obj=scripts/basic” 没有指定目标，所以会使用到默认目标: __build。在顶层 Makefile 中，KBUILD_BUILTIN 为 1，KBUILD_MODULES 为 0，因此展开后目标 __build 为:

```
__build:$(builtin-target) $(lib-target) $(extra-y) $(subdir-ym) $(always)
@:
```

可以看出目标 __build 有 5 个依赖: builtin-target、lib-target、extra-y、subdir-ym 和 always。这 5 个依赖的具体内容我们就不通过源码来分析了，直接在 scripts/Makefile.build 中输入图 31.3.14.1 所示内容，将这 5 个变量的值打印出来:

```

117 __build: $(if $(KBUILD_BUILTIN), $(builtin-target) $(lib-target) $(extra-y)) \
118           $(if $(KBUILD_MODULES), $(obj-m) $(modorder-target)) \
119           $(subdir-ym) $(always)
120 @:
121 @echo builtin-target = $(builtin-target)
122 @echo lib-target = $(lib-target)
123 @echo extra-y = $(extra-y)
124 @echo subdir-ym = $(subdir-ym)
125 @echo always = $(always)
126

```

输出5个依赖的值

图 31.3.14.1 输出变量

执行如下命令:

```
make mx6ull_14x14_ddr512_emmc_defconfig V=1
```

结果如图 31.3.14.2 所示:

```

make -f ./scripts/Makefile.build obj=scripts/basic
builtin-target =
lib-target =
extra-y =
subdir-ym =
always = scripts/basic/fixdep
rm -rf .tmp/quiet_recordmcount
make -f ./scripts/Makefile.build obj=scripts/kconfig mx6ull_alientek_emmc_defconfig
scripts/kconfig/conf --defconfig=arch/./configs/mx6ull_alientek_emmc_defconfig Kconfig
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/uboot-imx-rel imx 4.1.15 2.1.0 ga_alientek$

```

图 31.3.14.2 输出结果

从上图可以看出，只有 always 有效，因此 __build 最终为：

```

__build: scripts/basic/fixdep
@:

```

__build 依赖于 scripts/basic/fixdep，所以要先编译 scripts/basic/fixdep.c，生成 fixdep，前面已经读取了 scripts/basic/Makefile 文件。

综上所述，scripts_basic 目标的作用就是编译出 scripts/basic/fixdep 这个软件。

2、%config 目标对应的命令

%config 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig，各个变量值如下：

```

src= scripts/kconfig
kbuild-dir = ./scripts/kconfig
kbuild-file = ./scripts/kconfig/Makefile
include ./scripts/kconfig/Makefile

```

可以看出，Makefile.build 会读取 scripts/kconfig/Makefile 中的内容，此文件有如下所示内容：

示例代码 31.3.14.4 scripts/kconfig/Makefile 代码段

```

113 %_defconfig: $(obj)/conf
114     $(Q) < $(silent) --defconfig=arch/$(SRCARCH)/configs/$(@
$(Kconfig)
115
116 # Added for U-Boot (backward compatibility)
117 %_config: %_defconfig
118     @:

```

目标 %_defconfig 刚好和我们输入的 xxx_defconfig 匹配，所以会执行这条规则。依赖为 \$(obj)/conf，展开后就是 scripts/kconfig/conf。接下来就是检查并生成依赖 scripts/kconfig/conf。conf 是主机软件，到这里我们就打住，不要纠结 conf 是怎么编译出来的，否则就越陷越深，太绕了，像 conf 这种主机所使用的工具类软件我们一般不关心它是如何编译产生的。如果一定要看是 conf 是怎么生成的，可以输入如下命令重新配置 uboot，在重新配置 uboot 的过程中就会输出 conf 编译信息。

```

make distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mx6ull_14x14_ddr512_emmc_
defconfig V=1

```

结果如图 31.3.14.3 所示：

```
cc -o scripts/kconfig/conf scripts/kconfig/conf.o scripts/kconfig/zconf.tab.o
scripts/kconfig/conf --defconfig=arch/./configs/mx6ull_14x14_ddr512_emmc_defconfig Kconfig
#
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/alientek_u-boot$
```

图 31.3.14.3 编译过程

得到 scripts/kconfig/conf 以后就要执行目标%_defconfig 的命令:

```
$(Q)$< $(silent) --defconfig=arch/$(SRCARCH)/configs/$@ $(Kconfig)
```

相关的变量值如下:

silent=-s 或为空

SRCARCH=..

Kconfig=Kconfig

将其展开就是:

```
@ scripts/kconfig/conf --defconfig=arch/./configs/xxx_defconfig Kconfig
```

上述命令用到了 xxx_defconfig 文件, 比如 mx6ull_alientek_emmc_defconfig。这里会将 mx6ull_alientek_emmc_defconfig 中的配置输出到.config 文件中, 最终生成 u-boot 根目录下的.config 文件。

这个就是命令 make xxx_defconfig 执行流程, 总结一下如图 31.3.14.4 所示:

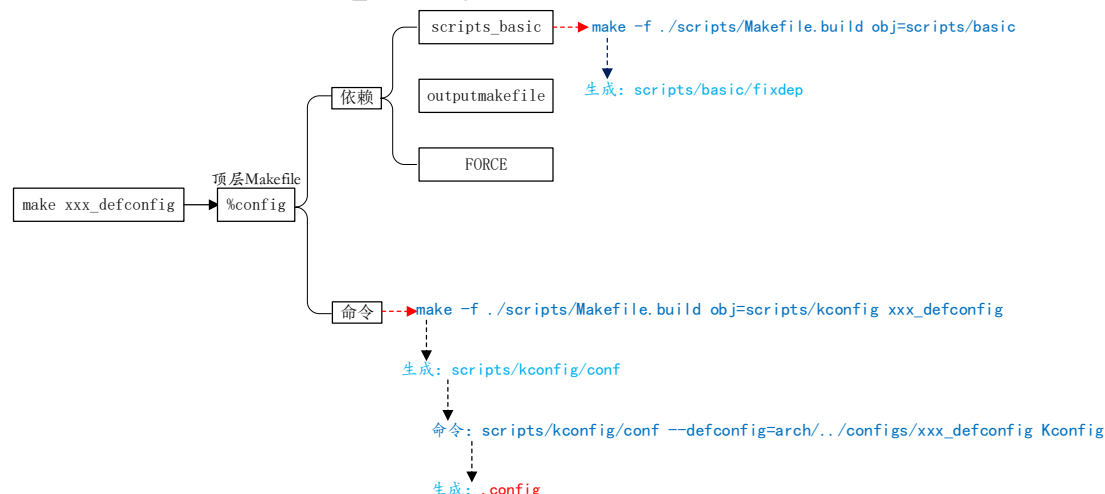


图 31.3.14.4 make xxx_defconfig 执行流程图

至此, make xxx_defconfig 就分析完了, 接下来就要分析一下 u-boot.bin 是怎么生成的了。

31.3.15 make 过程

配置好 u-boot 以后就可以直接 make 编译了, 因为没有指明目标, 所以会使用默认目标, 主 Makefile 中的默认目标如下:

示例代码 31.3.15.1 顶层 Makefile 代码段

```
128 # That's our default target when none is given on the command line
129 PHONY := _all
130 _all:
```

目标_all 又依赖于 all, 如下所示:

示例代码 31.3.15.2 顶层 Makefile 代码段

```
194 # If building an external module we do not care about the all: rule
195 # but instead _all depend on modules
```

```
196 PHONY += all
197 ifeq ($(KBUILD_EXTMOD),)
198 _all: all
199 else
200 _all: modules
201 endif
```

如果 KBUILD_EXTMOD 为空的话_all 依赖于 all。这里不编译模块，所以 KBUILD_EXTMOD 肯定为空，_all 的依赖就是 all。在主 Makefile 中 all 目标规则如下：

示例代码 31.3.15.2 顶层 Makefile 代码段

```
802 all:          $(ALL-y)
803 ifneq ($(CONFIG_SYS_GENERIC_BOARD),y)
804     @echo "===== WARNING ====="
805     @echo "Please convert this board to generic board."
806     @echo "Otherwise it will be removed by the end of 2014."
807     @echo "See doc/README.generic-board for further information"
808     @echo "===== WARNING ====="
809 endif
810 ifeq ($(CONFIG_DM_I2C_COMPAT),y)
811     @echo "===== WARNING ====="
812     @echo "This board uses CONFIG_DM_I2C_COMPAT. Please remove"
813     @echo "(possibly in a subsequent patch in your series)"
814     @echo "before sending patches to the mailing list."
815     @echo "===== WARNING ====="
816 endif
```

从 802 行可以看出，all 目标依赖\$(ALL-y)，而在顶层 Makefile 中，ALL-y 如下：

示例代码 31.3.15.3 顶层 Makefile 代码段

```
730 # Always append ALL so that arch config.mk's can add custom ones
731 ALL-y += u-boot.srec u-boot.bin u-boot.sym System.map u-boot.cfg
binary_size_check
732
733 ALL-$(CONFIG_ONENAND_U_BOOT) += u-boot-onenand.bin
734 ifeq ($(CONFIG_SPL_FSL_PBL),y)
735 ALL-$(CONFIG_RAMBOOT_PBL) += u-boot-with-spl-pbl.bin
736 else
737 ifneq ($(CONFIG_SECURE_BOOT), y)
738 # For Secure Boot The Image needs to be signed and Header must also
739 # be included. So The image has to be built explicitly
740 ALL-$(CONFIG_RAMBOOT_PBL) += u-boot.pbl
741 endif
742 endif
743 ALL-$(CONFIG_SPL) += spl/u-boot-spl.bin
744 ALL-$(CONFIG_SPL_FRAMEWORK) += u-boot.img
745 ALL-$(CONFIG_TPL) += tpl/u-boot-tpl.bin
```

```

746 ALL-$(CONFIG_OF_SEPARATE) += u-boot.dtb
747 ifeq ($(CONFIG_SPL_FRAMEWORK),y)
748 ALL-$(CONFIG_OF_SEPARATE) += u-boot-dtb.img
749 endif
750 ALL-$(CONFIG_OF_HOSTFILE) += u-boot.dtb
751 ifneq ($(CONFIG_SPL_TARGET),)
752 ALL-$(CONFIG_SPL) += $(CONFIG_SPL_TARGET:"%"=%)
753 endif
754 ALL-$(CONFIG_REMAKE_ELF) += u-boot.elf
755 ALL-$(CONFIG_EFI_APP) += u-boot-app.efi
756 ALL-$(CONFIG_EFI_STUB) += u-boot-payload.efi
757
758 ifneq ($(BUILD_ROM),)
759 ALL-$(CONFIG_X86_RESET_VECTOR) += u-boot.rom
760 endif
761
762 # enable combined SPL/u-boot/dtb rules for tegra
763 ifeq ($(CONFIG_TEGRA)$ (CONFIG_SPL),yy)
764 ALL-y += u-boot-tegra.bin u-boot-nodtb-tegra.bin
765 ALL-$(CONFIG_OF_SEPARATE) += u-boot-dtb-tegra.bin
766 endif
767
768 # Add optional build target if defined in board/cpu/soc headers
769 ifneq ($(CONFIG_BUILD_TARGET),)
770 ALL-y += $(CONFIG_BUILD_TARGET:"%"=%)
771 endif

```

从示例代码代码 31.3.15.3 可以看出, ALL-y 包含 u-boot.srec、u-boot.bin、u-boot.sym、System.map、u-boot.cfg 和 binary_size_check 这几个文件。根据 uboot 的配置情况也可能包含其他的文件, 比如:

```
ALL-$(CONFIG_ONENAND_U_BOOT) += u-boot-onenand.bin
```

CONFIG_ONENAND_U_BOOT 就是 uboot 中跟 ONENAND 配置有关的, 如果我们使能了 ONENAND, 那么在.config 配置文件中就会有“CONFIG_ONENAND_U_BOOT=y”这一句。相当于 CONFIG_ONENAND_U_BOOT 是个变量, 这个变量的值为“y”, 所以展开以后就是:

```
ALL-y += u-boot-onenand.bin
```

这个就是.config 里面的配置参数的含义, 这些参数其实都是变量, 后面跟着变量值, 会在顶层 Makefile 或者其他 Makefile 中调用这些变量。

ALL-y 里面有个 u-boot.bin, 这个就是我们最终需要的 uboot 二进制可执行文件, 所作的的所有工作就是为了它。在顶层 Makefile 中找到 u-boot.bin 目标对应的规则, 如下所示:

示例代码 31.3.15.4 顶层 Makefile 代码段

```

825 ifeq ($(CONFIG_OF_SEPARATE),y)
826 u-boot-dtb.bin: u-boot-nodtb.bin dts/dt.dtb FORCE
827     $(call if_changed,cat)
828

```

```

829 u-boot.bin: u-boot-dtb.bin FORCE
830     $(call if_changed,copy)
831 else
832 u-boot.bin: u-boot-nodtb.bin FORCE
833     $(call if_changed,copy)
834 endif

```

第 825 行判断 CONFIG_OF_SEPARATE 是否等于 y，如果相等，那条件就成立，在.config 中搜索“CONFIG_OF_SEPARAT”，没有找到，说明条件不成立。

第 832 行就是目标 u-boot.bin 的规则，目标 u-boot.bin 依赖于 u-boot-nodtb.bin，命令为\$(call if_changed,copy)，这里调用了 if_changed，if_changed 是一个函数，这个函数在 scripts/Kbuild.include 中有定义，而顶层 Makefile 中会包含 scripts/Kbuild.include 文件，这个前面已经说过了。

if_changed 在 Kbuild.include 中的定义如下：

示例代码 31.3.15.5 Kbuild.include 代码段

```

226 ###
227 # if_changed      - execute command if any prerequisite is newer than
228 #                  target, or command line has changed
229 # if_changed_dep - as if_changed, but uses fixdep to reveal
230 # dependencies including used config symbols
231 # if_changed_rule - as if_changed but execute rule instead
232 # See Documentation/kbuild/makefiles.txt for more info
233
234 ifneq ($(KBUILD_NOCMDDEP),1)
235 # Check if both arguments has same arguments. Result is empty
string if equal.
236 # User may override this check using make KBUILD_NOCMDDEP=1
237 arg-check = $(strip $(filter-out $(cmd_$(1)), $(cmd_$(2))) \
238                $(filter-out $(cmd_$(2)), $(cmd_$(1))))
239 else
240 arg-check = $(if $(strip $(cmd_$(2))),,1)
241 endif
242
243 # Replace >$< with >$$< to preserve $ when reloading the .cmd file
244 # (needed for make)
245 # Replace >#< with >\#< to avoid starting a comment in the .cmd
file
246 # (needed for make)
247 # Replace >'< with >'\'< to be able to enclose the whole string in
'...'
248 # (needed for the shell)
249 make-cmd = $(call escsq,$(subst \#,\\#, $(subst
$$,$$$$,$(cmd_$(1))))))
250

```



```

251 # Find any prerequisites that is newer than target or that does not
exist.
252 # PHONY targets skipped in both cases.
253 any-prereq = $(filter-out $(PHONY),$(?)) $(filter-out $(PHONY)
$(wildcard ^),^)
254
255 # Execute command if command has changed or prerequisite(s) are
updated.
256 #
257 if_changed = $(if $(strip $(any-prereq) $(arg-check)), \
258     @set -e;                                     \
259     $(echo-cmd) $(cmd_$(1));                       \
260     printf '%s\n' 'cmd_$$ := $(make-cmd)' > $(dot-target).cmd)
261

```

第 227 行为 if_changed 的描述, 根据描述, 在一些先决条件比目标新的时候, 或者命令行有改变的时候, if_changed 就会执行一些命令。

第 257 行就是函数 if_changed, if_changed 函数引用的变量比较多, 也比较绕, 我们只需要知道它可以从 u-boot-nodtb.bin 生成 u-boot.bin 就行了。

既然 u-boot.bin 依赖于 u-boot-nodtb.bin, 那么肯定要先生成 u-boot-nodtb.bin 文件, 顶层 Makefile 中相关代码如下:

示例代码 31.3.15.6 顶层 Makefile 代码段

```

866 u-boot-nodtb.bin: u-boot FORCE
867     $(call if_changed,objcopy)
868     $(call DO_STATIC_RELA,$<,$@,$(CONFIG_SYS_TEXT_BASE))
869     $(BOARD_SIZE_CHECK)

```

目标 u-boot-nodtb.bin 又依赖于 u-boot, 顶层 Makefile 中 u-boot 相关规则如下:

示例代码 31.3.15.7 顶层 Makefile 代码段

```

1170 u-boot:    $(u-boot-init) $(u-boot-main) u-boot.lds FORCE
1171     $(call if_changed,u-boot__)
1172 ifeq ($(CONFIG_KALLSYMS),y)
1173     $(call cmd,smap)
1174     $(call cmd,u-boot__) common/system_map.o
1175 endif

```

目标 u-boot 依赖于 u-boot_init、u-boot-main 和 u-boot.lds, u-boot_init 和 u-boot-main 是两个变量, 在顶层 Makefile 中有定义, 值如下:

示例代码 31.3.15.8 顶层 Makefile 代码段

```

678 u-boot-init := $(head-y)
679 u-boot-main := $(libs-y)

```

\$(head-y)跟 CPU 架构有关, 我们使用的是 ARM 芯片, 所以 head-y 在 arch/arm/Makefile 中被指定为:

```
head-y := arch/arm/cpu/$(CPU)/start.o
```

根据 31.3.12 小节的分析, 我们知道 CPU=armv7, 因此 head-y 展开以后就是:

```
head-y := arch/arm/cpu/armv7/start.o
```

因此:

u-boot-init= arch/arm/cpu/armv7/start.o

\$(libs-y)在顶层 Makefile 中被定义为 u-boot 所有子目录下 build-in.o 的集合, 代码如下:

示例代码 31.3.15.9 顶层 Makefile 代码段

```

620 libs-y += lib/
621 libs-$(HAVE_VENDOR_COMMON_LIB) += board/$(VENDOR)/common/
622 libs-$(CONFIG_OF_EMBED) += dts/
623 libs-y += fs/
624 libs-y += net/
625 libs-y += disk/
626 libs-y += drivers/
627 libs-y += drivers/dma/
628 libs-y += drivers/gpio/
629 libs-y += drivers/i2c/
.....
660 libs-y += cmd/
661 libs-y += common/
662 libs-$(CONFIG_API) += api/
663 libs-$(CONFIG_HAS_POST) += post/
664 libs-y += test/
665 libs-y += test/dm/
666 libs-$(CONFIG_UT_ENV) += test/env/
667
668 libs-y += $(if $(BOARD_DIR),board/$(BOARD_DIR)/)
669
670 libs-y := $(sort $(libs-y))
671
672 u-boot-dirs := $(patsubst %/,%, $(filter %/, $(libs-y))) tools
examples
673
674 u-boot-alldirs := $(sort $(u-boot-dirs)
$(patsubst %/,%, $(filter %/, $(libs-y))))
675
676 libs-y := $(patsubst %/, %/built-in.o, $(libs-y))

```

从上面的代码可以看出, libs-y 都是 u-boot 各子目录的集合, 最后:

libs-y := \$(patsubst %/, %/built-in.o, \$(libs-y))

这里调用了函数 patsubst, 将 libs-y 中的 “/” 替换为 “/built-in.o”, 比如 “drivers/dma/” 就变为了 “drivers/dma/built-in.o”, 相当于将 libs-y 改为所有子目录中 built-in.o 文件的集合。那么 u-boot-main 就等于所有子目录中 built-in.o 的集合。

这个规则就相当于将以 u-boot.lds 为链接脚本, 将 arch/arm/cpu/armv7/start.o 和各个子目录下的 built-in.o 链接在一起生成 u-boot。

u-boot.lds 的规则如下:

示例代码 31.3.15.10 顶层 Makefile 代码段

```
u-boot.lds: $(LDSCRIPT) prepare FORCE
$(call if_changed_dep,cpp_lds)
```

接下来的重点就是各子目录下的 built-in.o 是怎么生成的, 以 drivers/gpio/built-in.o 为例, 在 drivers/gpio/ 目录下会有个名为 built-in.o.cmd 的文件, 此文件内容如下:

示例代码 31.3.15.11 drivers/gpio/.built-in.o.cmd 代码

```
1 cmd_drivers/gpio/built-in.o := arm-linux-gnueabi-hf-ld.bfd -r -o
drivers/gpio/built-in.o drivers/gpio/mxc_gpio.o
```

从命令 “cmd_drivers/gpio/built-in.o” 可以看出, drivers/gpio/built-in.o 这个文件是使用 ld 命令由文件 drivers/gpio/mxc_gpio.o 生成而来的, mxc_gpio.o 是 mxc_gpio.c 编译生成的.o 文件, 这个是 NXP 的 IMX 系列的 GPIO 驱动文件。这里用到了 ld 的 “-r” 参数, 参数含义如下:

-r-relocatable: 产生可重定向的输出, 比如, 产生一个输出文件它可再次作为 ‘ld’ 的输入, 这经常被叫做 “部分链接”, 当我们需要将几个小的.o 文件链接成为一个.o 文件的时候, 需要使用此选项。

最终将各个子目录中的 built-in.o 文件链接在一起就形成了 u-boot, 使用如下命令编译 u-boot 就可以看到链接的过程:

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf- mx6ull_14x14_ddr512_emmc_
defconfig V=1
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf- V=1
```

编译的时候会有如图 31.3.15.1 所示内容输出:

```
arm-linux-gnueabi-hf-ld.bfd -pie --gc-sections -Bstatic -Ttext 0x87800000 -o u-boot -T u-boot.lds arch/ar
m/cpu/armv7/start.o --start-group arch/arm/cpu/built-in.o arch/arm/cpu/armv7/built-in.o arch/arm/imx-commo
n/built-in.o arch/arm/lib/built-in.o board/freescale/common/built-in.o board/freescale/mx6ull_alientek_emm
c/built-in.o cmd/built-in.o common/built-in.o disk/built-in.o drivers/built-in.o drivers/dma/built-in.o
drivers/gpio/built-in.o drivers/i2c/built-in.o drivers/mmc/built-in.o drivers/mtd/built-in.o drivers/mtd
/onenand/built-in.o drivers/mtd/spi/built-in.o drivers/net/built-in.o drivers/net/phy/built-in.o drivers/
pci/built-in.o drivers/power/built-in.o drivers/power/battery/built-in.o drivers/power/fuel_gauge/built-in
.o drivers/power/mfd/built-in.o drivers/power/pmic/built-in.o drivers/power/regulator/built-in.o drivers/
serial/built-in.o drivers/spi/built-in.o drivers/usb/dwc3/built-in.o drivers/usb/emul/built-in.o drivers/
usb/eth/built-in.o drivers/usb/gadget/built-in.o drivers/usb/gadget/udc/built-in.o drivers/usb/host/built-
in.o drivers/usb/musb-new/built-in.o drivers/usb/musb/built-in.o drivers/usb/phy/built-in.o drivers/usb/u
lpi/built-in.o fs/built-in.o lib/built-in.o net/built-in.o test/built-in.o test/dm/built-in.o --end-grou
p arch/arm/lib/eabi_compat.o -L /usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi-hf/bin/./li
b/gcc/arm-linux-gnueabi-hf/4.9.4 -lgcc -Map u-boot.map
```

图 31.3.15.1 编译内容输出

将其整理一下, 内容如下:

```
arm-linux-gnueabi-hf-ld.bfd -pie --gc-sections -Bstatic -Ttext 0x87800000 \
-o u-boot -T u-boot.lds \
arch/arm/cpu/armv7/start.o \
--start-group arch/arm/cpu/built-in.o \
arch/arm/cpu/armv7/built-in.o \
arch/arm/imx-common/built-in.o \
arch/arm/lib/built-in.o \
board/freescale/common/built-in.o \
board/freescale/mx6ull_alientek_emmc/built-in.o \
cmd/built-in.o \
common/built-in.o \
disk/built-in.o \
drivers/built-in.o \
drivers/dma/built-in.o \
drivers/gpio/built-in.o \
```

```
.....
drivers/spi/built-in.o \
drivers/usb/dwc3/built-in.o \
drivers/usb/emul/built-in.o \
drivers/usb/eth/built-in.o \
drivers/usb/gadget/built-in.o \
drivers/usb/gadget/udc/built-in.o \
drivers/usb/host/built-in.o \
drivers/usb/musb-new/built-in.o \
drivers/usb/musb/built-in.o \
drivers/usb/phy/built-in.o \
drivers/usb/ulpi/built-in.o \
fs/built-in.o \
lib/built-in.o \
net/built-in.o \
test/built-in.o \
test/dm/built-in.o \
--end-group arch/arm/lib/eabi_compat.o \
-L /usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabihf/bin/../lib/gcc/arm-linux-gnueabihf/4.9.4 -lgcc -Map u-boot.map
```

可以看出最终是用 `arm-linux-gnueabihf-ld.bfd` 命令将 `arch/arm/cpu/armv7/start.o` 和其他众多的 `built_in.o` 链接在一起，形成 `u-boot`。

目标 `all` 除了 `u-boot.bin` 以外还有其他的依赖，比如 `u-boot.srec`、`u-boot.sym`、`System.map`、`u-boot.cfg` 和 `binary_size_check` 等等，这些依赖的生成方法和 `u-boot.bin` 很类似，大家自行查看一下顶层 `Makefile`，我们就不详细的讲解了。

总结一下“make”命令的流程，如图 31.3.15.2 所示：

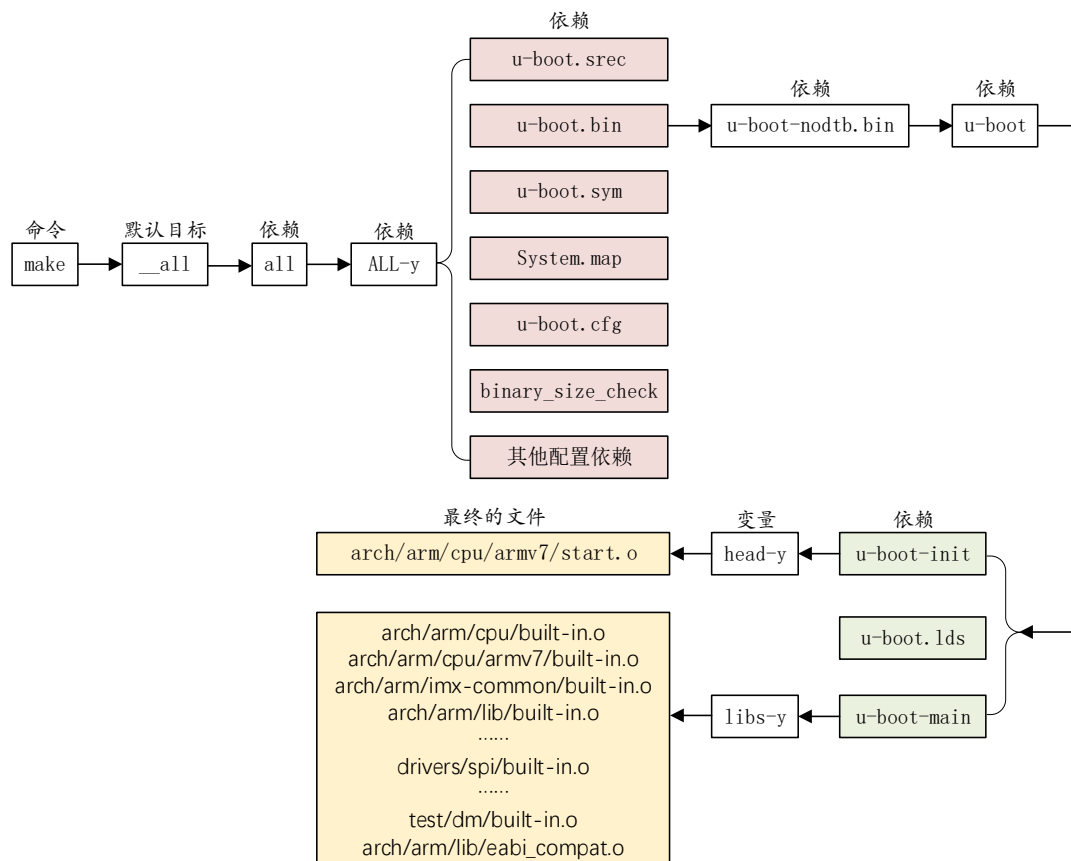


图 31.3.15.2 make 命令流程

图 31.3.15.2 就是“make”命令的执行流程，关于 uboot 的顶层 Makefile 就分析到这里，重点是“make xxx_defconfig”和“make”这两个命令的执行流程：

make xxx_defconfig: 用于配置 uboot，这个命令最主要的目的就是生成.config 文件。

make: 用于编译 uboot，这个命令的主要工作就是生成二进制的 u-boot.bin 文件和其他的一些与 uboot 有关的文件，比如 u-boot.imx 等等。

关于 uboot 的顶层 Makefile 就分析到这里，有些内容我们没有详细、深入的去研究，因为我们的重点是使用 uboot，而不是 uboot 的研究者，我们要做的是缕清 uboot 的流程。至于更具体的实现，有兴趣的可以参考一下其他资料。

第三十二章 U-Boot 启动流程详解

上一章我们详细的分析了 uboot 的顶层 Makefile, 理清了 uboot 的编译流程。本章我们来详细的分析一下 uboot 的启动流程, 理清 uboot 是如何启动的。通过对 uboot 启动流程的梳理, 我们就可以掌握一些外设是在哪里被初始化的, 这样当我们需要修改这些外设驱动的时候就会心里有数。另外, 通过分析 uboot 的启动流程可以了解 Linux 内核是如何被启动的。

32.1 链接脚本 u-boot.lds 详解

要分析 uboot 的启动流程，首先要找到“入口”，找到第一行程序在哪里。程序的链接是由链接脚本来决定的，所以通过链接脚本可以找到程序的入口。如果没有编译过 uboot 的话链接脚本为 arch/arm/cpu/u-boot.lds。但是这个不是最终使用的链接脚本，最终的链接脚本是在这个链接脚本的基础上生成的。编译一下 uboot，编译完成以后就会在 uboot 根目录下生成 u-boot.lds 文件，如图 32.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$ ls
api      disk      include  MAKEALL  scripts  u-boot.cfg
arch     doc       Kbuild   Makefile  snapshot.commit  u-boot.imx
board    drivers  Kconfig  mx6ull_alientek_emmc.sh  System.map  u-boot.lds
cmd      dts       lib       mx6ull_alientek_nand.sh  test        u-boot.map
common   examples licenses net        tools       u-boot-nodtb.bin
config.mk fs         load.imx  post       u-boot     u-boot.srec
configs  imxdownload  MAINTAINERS  README    u-boot.bin  u-boot.sym
zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/alientek_uboot$
```

图 32.1.1 链接脚本

只有编译 u-boot 以后才会在根目录下出现 u-boot.lds 文件！

打开 u-boot.lds，内容如下：

示例代码 32.1.1 u-boot.lds 文件代码

```
1 OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-littlearm")
2 OUTPUT_ARCH (arm)
3 ENTRY(_start)
4 SECTIONS
5 {
6   . = 0x00000000;
7   . = ALIGN(4);
8   .text :
9   {
10    *(__image_copy_start)
11    *(.vectors)
12    arch/arm/cpu/armv7/start.o (.text*)
13    *(.text*)
14  }
15  . = ALIGN(4);
16  .rodata : { *(SORT_BY_ALIGNMENT(SORT_BY_NAME(.rodata*))) }
17  . = ALIGN(4);
18  .data : {
19    *(.data*)
20  }
21  . = ALIGN(4);
22  . = .;
23  . = ALIGN(4);
24  .u_boot_list : {
25    KEEP(*(SORT(.u_boot_list*)));
26  }
27  . = ALIGN(4);
```

```

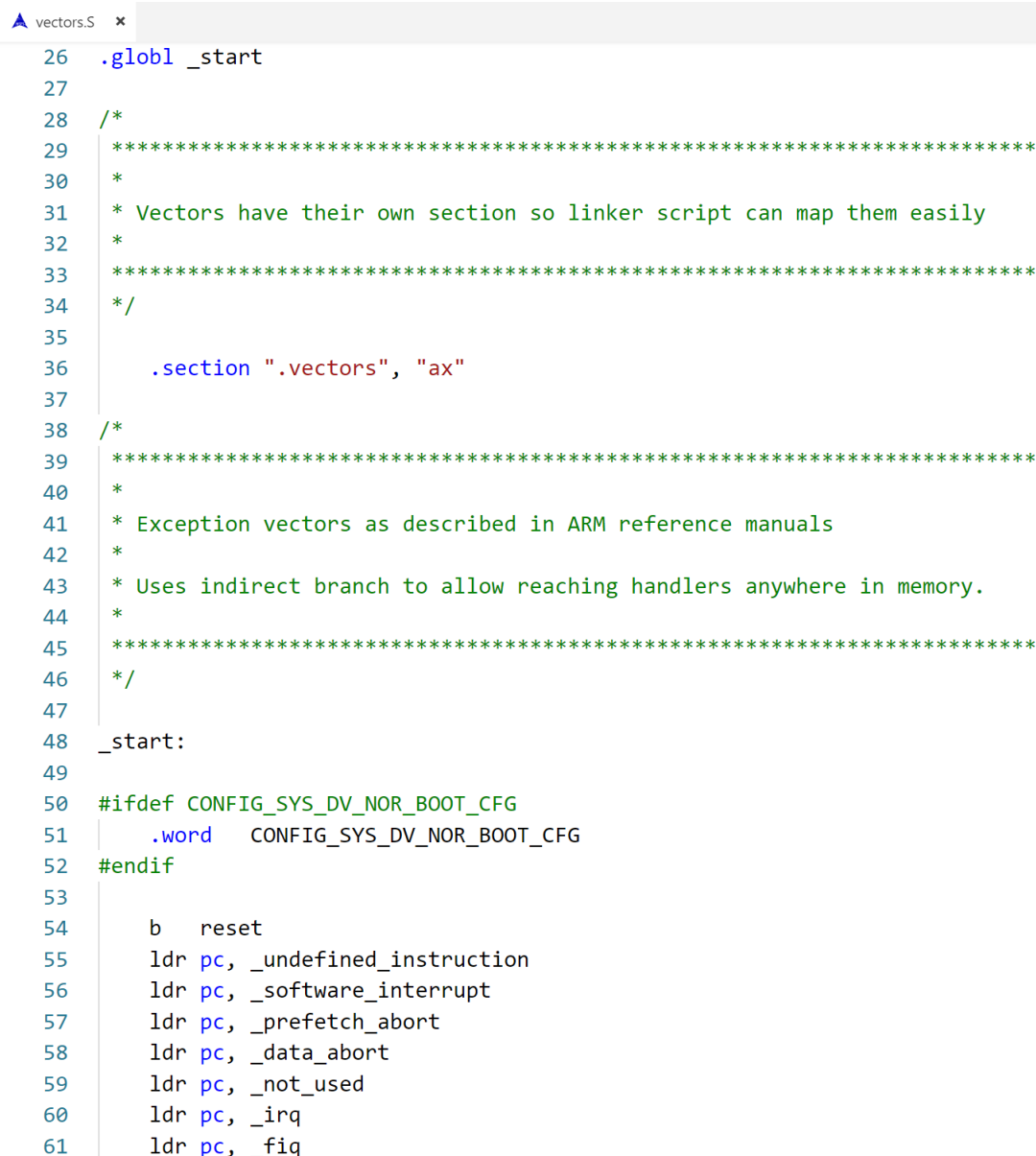
28 .image_copy_end :
29 {
30     *(__image_copy_end)
31 }
32 .rel_dyn_start :
33 {
34     *(__rel_dyn_start)
35 }
36 .rel.dyn : {
37     *(.rel*)
38 }
39 .rel_dyn_end :
40 {
41     *(__rel_dyn_end)
42 }
43 .end :
44 {
45     *(__end)
46 }
47 _image_binary_end = .;
48 . = ALIGN(4096);
49 .mmutable : {
50     *(.mmutable)
51 }
52 .bss_start __rel_dyn_start (OVERLAY) : {
53     KEEP(*(__bss_start));
54     __bss_base = .;
55 }
56 .bss __bss_base (OVERLAY) : {
57     *(.bss*)
58     . = ALIGN(4);
59     __bss_limit = .;
60 }
61 .bss_end __bss_limit (OVERLAY) : {
62     KEEP(*(__bss_end));
63 }
64 .dynsym _image_binary_end : { *(.dynsym) }
65 .dynbss : { *(.dynbss) }
66 .dynstr : { *(.dynstr*) }
67 .dynamic : { *(.dynamic*) }
68 .plt : { *(.plt*) }
69 .interp : { *(.interp*) }
70 .gnu.hash : { *(.gnu.hash) }
    
```

```

71 .gnu : { *(.gnu*) }
72 .ARM.exidx : { *(.ARM.exidx*) }
73 .gnu.linkonce.armexidx : { *(.gnu.linkonce.armexidx.*) }
74 }

```

第3行为代码当前入口点: `_start`, `_start` 在文件 `arch/arm/lib/vectors.S` 中有定义, 如图 32.1.2 所示:



```

26 .globl _start
27
28 /*
29  ****
30  *
31  * Vectors have their own section so linker script can map them easily
32  *
33  ****
34  */
35
36     .section ".vectors", "ax"
37
38 /*
39  ****
40  *
41  * Exception vectors as described in ARM reference manuals
42  *
43  * Uses indirect branch to allow reaching handlers anywhere in memory.
44  *
45  ****
46  */
47
48 _start:
49
50 #ifdef CONFIG_SYS_DV_NOR_BOOT_CFG
51     .word    CONFIG_SYS_DV_NOR_BOOT_CFG
52 #endif
53
54     b    reset
55     ldr pc, _undefined_instruction
56     ldr pc, _software_interrupt
57     ldr pc, _prefetch_abort
58     ldr pc, _data_abort
59     ldr pc, _not_used
60     ldr pc, _irq
61     ldr pc, _fiq

```

图 32.1.2 `_start` 入口

从图 32.1.2 中的代码可以看出, `_start` 后面就是中断向量表, 从图中的 `".section ".vectors", "ax"` 可以得到, 此代码存放在 `.vectors` 段里面。

使用如下命令在 `uboot` 中查找 `"__image_copy_start"`:

```
grep -nR "__image_copy_start"
```

搜索结果如图 32.1.3 所示:

```

arch/arm/lib/relocate.c:17:      memcpy((void *)gd->relocaddr, (void *)&__image_copy_start, len);
arch/arm/lib/relocate.c:48:      if (offset_ptr_rom >= (Elf32_Addr *)&__image_copy_start &&
arch/arm/lib/relocate.c:71:      if (val >= (unsigned int)&__image_copy_start && val <=
arch/arm/cpu/u-boot.lds:15:      *(__image_copy_start)
u-boot.lds:10:  *(__image_copy_start)
u-boot.map:931: *(__image_copy_start)
u-boot.map:932: . __image_copy_start
u-boot.map:934:      0x0000000087800000      __image_copy_start
zuozhongkai@ubuntu:~/Linux/IMX6ULL/uboot/uboot-imx-rel_imx_4.1.15_2.1.0_ga_aliante$

```

图 32.1.3 查找结果

打开 u-boot.map，找到如图 32.1.4 所示位置：

```

u-boot.map *
926 段 .text 的地址设置为 0x87800000
927      |      |      |      0x0000000000000000      . = 0x0
928      |      |      |      0x0000000000000000      . = ALIGN (0x4)
929
930 .text      0x0000000087800000      0x3e94c
931 *(__image_copy_start)
932 .__image_copy_start
933      |      |      |      0x0000000087800000      0x0 arch/arm/lib/built-in.o
934      |      |      |      0x0000000087800000      __image_copy_start
935 *(.vectors)
936 .vectors   0x0000000087800000      0x300 arch/arm/lib/built-in.o
937      |      |      |      0x0000000087800000      _start
938      |      |      |      0x0000000087800020      _undefined_instruction
939      |      |      |      0x0000000087800024      _software_interrupt
940      |      |      |      0x0000000087800028      _prefetch_abort
941      |      |      |      0x000000008780002c      _data_abort
942      |      |      |      0x0000000087800030      _not_used
943      |      |      |      0x0000000087800034      _irq
944      |      |      |      0x0000000087800038      _fiq
945      |      |      |      0x0000000087800040      IRQ_STACK_START_IN

```

图 32.1.4 u-boot.map

u-boot.map 是 uboot 的映射文件，可以从此文件看到某个文件或者函数链接到了哪个地址，从图 32.1.4 的 932 行可以看到 __image_copy_start 为 0X87800000，而 .text 的起始地址也是 0X87800000。

继续回到示例代码 32.1.1 中，第 11 行是 vectors 段，vectors 段保存中断向量表，从图 32.1.2 中我们知道了 vectors.S 的代码是存在 vectors 段中的。从图 32.1.4 可以看出，vectors 段的起始地址也是 0X87800000，说明整个 uboot 的起始地址就是 0X87800000，这也是为什么我们裸机例程的链接起始地址选择 0X87800000 了，目的就是为了和 uboot 一致。

第 12 行将 arch/arm/cpu/armv7/start.s 编译出来的代码放到中断向量表后面。

第 13 行为 text 段，其他的代码段就放到这里

在 u-boot.lds 中有一些跟地址有关的“变量”需要我们注意一下，后面分析 u-boot 源码的时候会用到，这些变量要最终编译完成才能确定的!!! 比如我编译完成以后这些“变量”的值如表 32.1.1 所示：

变量	数值	描述
__image_copy_start	0x87800000	uboot 拷贝的首地址
__image_copy_end	0x8785dd54	uboot 拷贝的结束地址
__rel_dyn_start	0x8785dd54	.rel.dyn 段起始地址
__rel_dyn_end	0x878668f4	.rel.dyn 段结束地址
_image_binary_end	0x878668f4	镜像结束地址

__bss_start	0x8785dd54	.bss 段起始地址
__bss_end	0x878a8e74	.bss 段结束地址

表 32.1.1 uboot 相关变量表

表 32.1.1 中的“变量”值可以在 u-boot.map 文件中查找,表 32.1.1 中除了 __image_copy_start 以外,其他的变量值每次编译的时候可能会变化,如果修改了 uboot 代码、修改了 uboot 配置、选用不同的优化等级等等都会影响到这些值。所以,一切以实际值为准!

32.2 U-Boot 启动流程详解

32.2.1 reset 函数源码详解

从 u-boot.lds 中我们已经知道了入口点是 arch/arm/lib/vectors.S 文件中的 _start,代码如下:

示例代码 32.2.1.1 vectors.S 代码段

```
38 /*
39 ****
40 *
41 * Exception vectors as described in ARM reference manuals
42 *
43 * Uses indirect branch to allow reaching handlers anywhere in
44 * memory.
45 ****
46 */
47
48 _start:
49
50 #ifdef CONFIG_SYS_DV_NOR_BOOT_CFG
51 .word CONFIG_SYS_DV_NOR_BOOT_CFG
52 #endif
53
54     b reset
55     ldr pc, _undefined_instruction
56     ldr pc, _software_interrupt
57     ldr pc, _prefetch_abort
58     ldr pc, _data_abort
59     ldr pc, _not_used
60     ldr pc, _irq
61     ldr pc, _fiq
```

第 48 行 _start 开始的是中断向量表,其中 54~61 行就是中断向量表,和我们裸机例程里面一样。54 行跳转到 reset 函数里面,reset 函数在 arch/arm/cpu/armv7/start.S 里面,代码如下:

示例代码 32.2.1.2 start.S 代码段

```
22 /*****
23 *
24 * Startup Code (reset vector)
25 *
```

```

26 * Do important init only if we don't start from memory!
27 * Setup memory and board specific bits prior to relocation.
28 * Relocate armboot to ram. Setup stack.
29 *
30 *****/
31
32 .globl reset
33 .globl save_boot_params_ret
34
35 reset:
36     /* Allow the board to save important registers */
37     b save_boot_params

```

第 35 行就是 reset 函数。

第 37 行从 reset 函数跳转到了 save_boot_params 函数，而 save_boot_params 函数同样定义在 start.S 里面，定义如下：

示例代码 32.2.1.3 start.S 代码段

```

91 /*****
92 *
93 * void save_boot_params(u32 r0, u32 r1, u32 r2, u32 r3)
94 * __attribute__((weak));
95 *
96 * Stack pointer is not yet initialized at this moment
97 * Don't save anything to stack even if compiled with -O0
98 *
99 *****/
100 ENTRY(save_boot_params)
101     b save_boot_params_ret @ back to my caller

```

save_boot_params 函数也是只有一句跳转语句，跳转到 save_boot_params_ret 函数，save_boot_params_ret 函数代码如下：

示例代码 32.2.1.4 start.S 代码段

```

38 save_boot_params_ret:
39     /*
40     * disable interrupts (FIQ and IRQ), also set the cpu to SVC32
41     * mode, except if in HYP mode already
42     */
43     mrs     r0, cpsr
44     and     r1, r0,     #0x1f @ mask mode bits
45     teq     r1,         #0x1a @ test for HYP mode
46     bicne   r0, r0,     #0x1f @ clear all mode bits
47     orrne   r0, r0,     #0x13 @ set SVC mode
48     orr     r0, r0,     #0xc0 @ disable FIQ and IRQ
49     msr     cpsr, r0

```

第 43 行，读取寄存器 cpsr 中的值，并保存到 r0 寄存器中。

第 44 行, 将寄存器 r0 中的值与 0X1F 进行与运算, 结果保存到 r1 寄存器中, 目的就是提取 cpsr 的 bit0~bit4 这 5 位, 这 5 位为 M4 M3 M2 M1 M0, M[4:0]这五位用来设置处理器的工作模式, 如表 32.2.1.1 所示:

M[4:0]	模式
10000	User(usr)
10001	FIQ(fiq)
10010	IRQ(irq)
10011	Supervisor(svc)
10110	Monitor(mon)
10111	Abort(abt)
11010	Hyp(hyp)
11011	Undefined(und)
11111	System(sys)

表 32.2.1.1 Cortex-A7 工作模式

第 45 行, 判断 r1 寄存器的值是否等于 0X1A(0b11010), 也就是判断当前处理器模式是否处于 Hyp 模式。

第 46 行, 如果 r1 和 0X1A 不相等, 也就是 CPU 不处于 Hyp 模式的话就将 r0 寄存器的 bit0~5 进行清零, 其实就是清除模式位

第 47 行, 如果处理器不处于 Hyp 模式的话就将 r0 寄存器的值与 0x13 进行或运算, 0x13=0b10011, 也就是设置处理器进入 SVC 模式。

第 48 行, r0 寄存器的值再与 0xC0 进行或运算, 那么 r0 寄存器此时的值就是 0xD3, cpsr 的 I 为和 F 位分别控制 IRQ 和 FIQ 这两个中断的开关, 设置为 1 就关闭了 FIQ 和 IRQ!

第 49 行, 将 r0 寄存器写回到 cpsr 寄存器中。完成设置 CPU 处于 SVC32 模式, 并且关闭 FIQ 和 IRQ 这两个中断。

继续执行下面的代码:

示例代码 32.2.1.5 start.S 代码段

```

51 /*
52  * Setup vector:
53  * (OMAP4 spl TEXT_BASE is not 32 byte aligned.
54  * Continue to use ROM code vector only in OMAP4 spl)
55  */
56 #if !(defined(CONFIG_OMAP44XX) && defined(CONFIG_SPL_BUILD))
57 /* Set V=0 in CP15 SCTLR register - for VBAR to point to vector */
58     mrc p15, 0, r0, c1, c0, 0    @ Read CP15 SCTLR Register
59     bic r0, #CR_V                @ V = 0
60     mcr p15, 0, r0, c1, c0, 0    @ Write CP15 SCTLR Register
61
62     /* Set vector address in CP15 VBAR register */
63     ldr r0, =_start
64     mcr p15, 0, r0, c12, c0, 0   @Set VBAR
65 #endif

```

第 56 行, 如果没有定义 CONFIG_OMAP44XX 和 CONFIG_SPL_BUILD 的话条件成立, 此处条件成立。

第 58 行读取 CP15 中 c1 寄存器的值到 r0 寄存器中, 根据 17.1.4 小节可知, 这里是读取 SCTLr 寄存器的值。

第 59 行, CR_V 在 arch/arm/include/asm/system.h 中有如下所示定义:

```
#define CR_V (1 << 13) /* Vectors relocated to 0xffff0000 */
```

因此这一行的目的就是清除 SCTLr 寄存器中的 bit13, SCTLr 寄存器结构如图 32.2.1.1 所示:

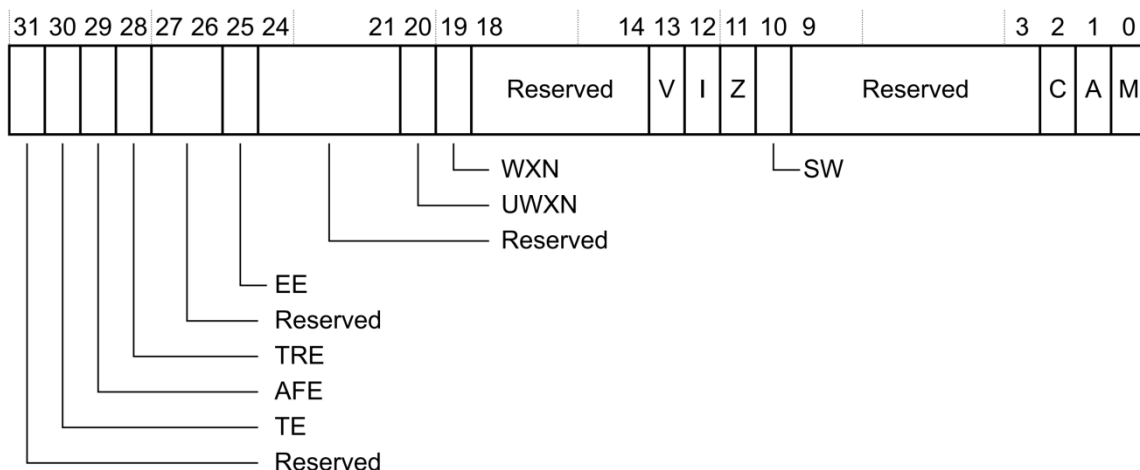


图 32.2.1.1 SCTLr 寄存器结构图

从图 32.2.1.1 可以看出, bit13 为 V 位, 此位是向量表控制位, 当为 0 的时候向量表基地址为 0X00000000, 软件可以重定位向量表。为 1 的时候向量表基地址为 0XFFFF0000, 软件不能重定位向量表。这里将 V 清零, 目的就是为了接下来的向量表重定位, 这个我们在第十七章有过详细的介绍了。

第 60 行将 r0 寄存器的值重写写入到寄存器 SCTLr 中。

第 63 行设置 r0 寄存器的值为 _start, _start 就是整个 uboot 的入口地址, 其值为 0X87800000, 相当于 uboot 的起始地址, 因此 0x87800000 也是向量表的起始地址。

第 64 行将 r0 寄存器的值(向量表值)写入到 CP15 的 c12 寄存器中, 也就是 VBAR 寄存器。因此第 58~64 行就是设置向量表重定位的。

代码继续往下执行:

示例代码 32.2.1.6 start.S 代码段

```
67 /* the mask ROM code should have PLL and others stable */
68 #ifndef CONFIG_SKIP_LOWLEVEL_INIT
69     bl cpu_init_cp15
70     bl cpu_init_crit
71 #endif
72
73 bl _main
```

第 68 行如果没有定义 CONFIG_SKIP_LOWLEVEL_INIT 的话条件成立。我们没有定义 CONFIG_SKIP_LOWLEVEL_INIT, 因此条件成立, 执行下面的语句。

示例代码 32.2.1.6 中的内容比较简单, 就是分别调用函数 cpu_init_cp15、cpu_init_crit 和 _main。

函数 cpu_init_cp15 用来设置 CP15 相关的内容, 比如关闭 MMU 啥的, 此函数同样在 start.S 文件中定义的, 代码如下:

示例代码 32.2.1.7 start.S 代码段

```

105 /*****
106  *
107  *  cpu_init_cp15
108  *
109  *  Setup CP15 registers (cache, MMU, TLBs). The I-cache is turned on
110  *  unless CONFIG_SYS_ICACHE_OFF is defined.
111  *
112  *****/
113 ENTRY(cpu_init_cp15)
114     /*
115     *  Invalidate L1 I/D
116     */
117     mov r0, #0                @ set up for MCR
118     mcr p15, 0, r0, c8, c7, 0 @ invalidate TLBs
119     mcr p15, 0, r0, c7, c5, 0 @ invalidate icache
120     mcr p15, 0, r0, c7, c5, 6 @ invalidate BP array
121     mcr p15, 0, r0, c7, c10, 4 @ DSB
122     mcr p15, 0, r0, c7, c5, 4 @ ISB
123
124     /*
125     *  disable MMU stuff and caches
126     */
127     mrc p15, 0, r0, c1, c0, 0
128     bic r0, r0, #0x00002000 @ clear bits 13 (--V-)
129     bic r0, r0, #0x00000007 @ clear bits 2:0 (-CAM)
130     orr r0, r0, #0x00000002 @ set bit 1 (--A-) Align
131     orr r0, r0, #0x00000800 @ set bit 11 (Z---) BTB
132 #ifdef CONFIG_SYS_ICACHE_OFF
133     bic r0, r0, #0x00001000 @ clear bit 12 (I) I-cache
134 #else
135     orr r0, r0, #0x00001000 @ set bit 12 (I) I-cache
136 #endif
137     mcr p15, 0, r0, c1, c0, 0
138
139     .....
140
255
256     mov pc, r5                @ back to my caller
257 ENDPROC(cpu_init_cp15)

```

函数 `cpu_init_cp15` 都是一些和 CP15 有关的内容，我们不用关心，有兴趣的可以详细的看一下。

函数 `cpu_init_crit` 也是在定义在 `start.S` 文件中，函数内容如下：

示例代码 32.2.1.8 start.S 代码段

```

260 /*****
261  *
262  * CPU_init_critical registers
263  *
264  * setup important registers
265  * setup memory timing
266  *
267  *****/
268 ENTRY(cpu_init_crit)
269  /*
270   * Jump to board specific initialization...
271   * The Mask ROM will have already initialized
272   * basic memory. Go here to bump up clock rate and handle
273   * wake up conditions.
274   */
275   b   lowlevel_init      @ go setup pll,mux,memory
276 ENDPROC(cpu_init_crit)

```

可以看出函数 `cpu_init_crit` 内部仅仅是调用了函数 `lowlevel_init`，接下来就是详细的分析一下 `lowlevel_init` 和 `_main` 这两个函数。

32.2.2 lowlevel_init 函数详解

函数 `lowlevel_init` 在文件 `arch/arm/cpu/armv7/lowlevel_init.S` 中定义，内容如下：

示例代码 32.2.2.1 `lowlevel_init.S` 代码段

```

14 #include <asm-offsets.h>
15 #include <config.h>
16 #include <linux/linkage.h>
17
18 ENTRY(lowlevel_init)
19  /*
20   * Setup a temporary stack. Global data is not available yet.
21   */
22   ldr sp, =CONFIG_SYS_INIT_SP_ADDR
23   bic sp, sp, #7          /* 8-byte alignment for ABI compliance */
24 #ifdef CONFIG_SPL_DM
25   mov r9, #0
26 #else
27   /*
28   * Set up global data for boards that still need it. This will be
29   * removed soon.
30   */
31 #ifdef CONFIG_SPL_BUILD
32   ldr r9, =gdata
33 #else

```

```

34     sub sp, sp, #GD_SIZE
35     bic sp, sp, #7
36     mov r9, sp
37 #endif
38 #endif
39     /*
40     * Save the old lr(passed in ip) and the current lr to stack
41     */
42     push    {ip, lr}
43
44     /*
45     * Call the very early init function. This should do only the
46     * absolute bare minimum to get started. It should not:
47     *
48     * - set up DRAM
49     * - use global_data
50     * - clear BSS
51     * - try to start a console
52     *
53     * For boards with SPL this should be empty since SPL can do all
54     * of this init in the SPL board_init_f() function which is
55     * called immediately after this.
56     */
57     bl s_init
58     pop {ip, pc}
59 ENDPROC(lowlevel_init)

```

第 22 行设置 sp 指向 CONFIG_SYS_INIT_SP_ADDR, CONFIG_SYS_INIT_SP_ADDR 在 include/configs/mx6ullevk.h 文件中, 在 mx6ullevk.h 中有如下所示定义:

示例代码 32.2.2.2 mx6ullevk.h 代码段

```

234 #define CONFIG_SYS_INIT_RAM_ADDR    IRAM_BASE_ADDR
235 #define CONFIG_SYS_INIT_RAM_SIZE    IRAM_SIZE
236
237 #define CONFIG_SYS_INIT_SP_OFFSET \
238     (CONFIG_SYS_INIT_RAM_SIZE - GENERATED_GBL_DATA_SIZE)
239 #define CONFIG_SYS_INIT_SP_ADDR \
240     (CONFIG_SYS_INIT_RAM_ADDR + CONFIG_SYS_INIT_SP_OFFSET)

```

示例代码 32.2.2.2 中的 IRAM_BASE_ADDR 和 IRAM_SIZE 在文件 arch/arm/include/asm/arch-mx6/imx-regs.h 中有定义, 如下所示, 其实就是 IMX6UL/IM6ULL 内部 ocram 的首地址和大小。

示例代码 32.2.2.3 imx-regs.h 代码段

```

71 #define IRAM_BASE_ADDR                0x00900000
.....
408 #if !(defined(CONFIG_MX6SX) || defined(CONFIG_MX6UL) || \

```

```

409     defined(CONFIG_MX6SLL) || defined(CONFIG_MX6SL))
410 #define IRAM_SIZE                0x00040000
411 #else
412 #define IRAM_SIZE                0x00020000
413 #endif

```

如果 408 行的条件成立的话 IRAM_SIZE=0X40000，当定义了 CONFIG_MX6SX、CONFIG_MX6U、CONFIG_MX6SLL 和 CONFIG_MX6SL 中的任意一个的话条件就不成立，在.config 中定义了 CONFIG_MX6UL，所以条件不成立，因此 IRAM_SIZE=0X20000=128KB。

结合示例代码 32.2.2.2，可以得到如下值：

CONFIG_SYS_INIT_RAM_ADDR = IRAM_BASE_ADDR = 0x00900000。

CONFIG_SYS_INIT_RAM_SIZE = 0x00020000 = 128KB。

还需要知道 GENERATED_GBL_DATA_SIZE 的值，在文件 include/generated/generic-asm-offsets.h 中有定义，如下：

示例代码 32.2.2.4 generic-asm-offsets.h 代码段

```

1  #ifndef __GENERIC_ASM_OFFSETS_H__
2  #define __GENERIC_ASM_OFFSETS_H__
3  /*
4   * DO NOT MODIFY.
5   *
6   * This file was generated by Kbuild
7   */
8
9  #define GENERATED_GBL_DATA_SIZE 256
10 #define GENERATED_BD_INFO_SIZE 80
11 #define GD_SIZE 248
12 #define GD_BD 0
13 #define GD_MALLOC_BASE 192
14 #define GD_RELOCADDR 48
15 #define GD_RELOC_OFF 68
16 #define GD_START_ADDR_SP 64
17
18 #endif

```

GENERATED_GBL_DATA_SIZE=256，GENERATED_GBL_DATA_SIZE 的含义为 (sizeof(struct global_data) + 15) & ~15。

综上所述，CONFIG_SYS_INIT_SP_ADDR 值如下：

CONFIG_SYS_INIT_SP_OFFSET = 0x00020000 - 256 = 0x1FF00。

CONFIG_SYS_INIT_SP_ADDR = 0x00900000 + 0x1FF00 = 0x0091FF00，

结果如下图所示：

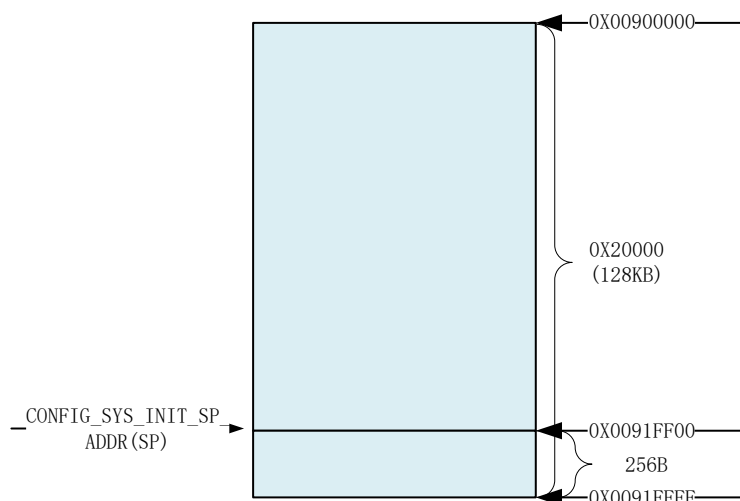


图 32.2.2.1 sp 值

此时 sp 指向 0X91FF00，这属于 IMX6UL/IMX6ULL 的内部 ram。

继续回到文件 lowlevel_init.S，第 23 行对 sp 指针做 8 字节对齐处理！

第 34 行，sp 指针减去 GD_SIZE，GD_SIZE 同样在 generic-asm-offsets.h 中定了，大小为 248，见示例代码 32.2.2.4 第 11 行。

第 35 行对 sp 做 8 字节对齐，此时 sp 的地址为 0X0091FF00-248=0X0091FE08，此时 sp 位置如图 32.2.2.2 所示：

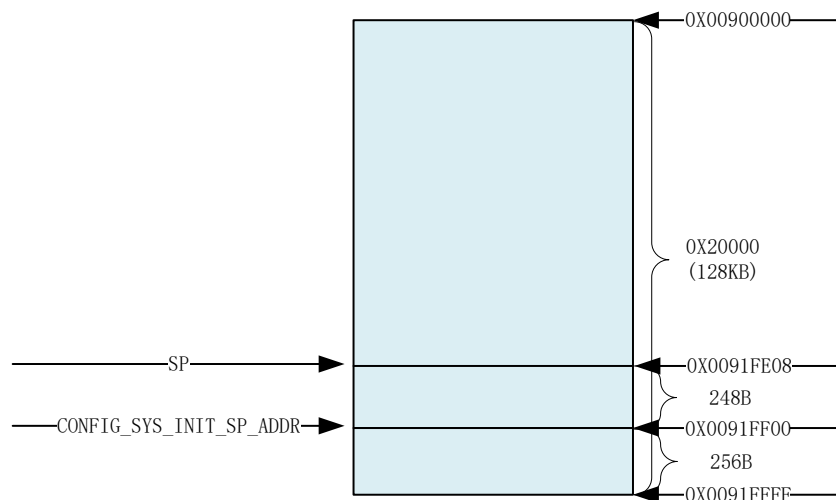


图 32.2.2.2 sp 值

第 36 行将 sp 地址保存在 r9 寄存器中。

第 42 行将 ip 和 lr 压栈

第 57 行调用函数 s_init，得，又来了一个函数。

第 58 行将第 36 行入栈的 ip 和 lr 进行出栈，并将 lr 赋给 pc。

32.2.3 s_init 函数详解

在上一小节中，我们知道 lowlevel_init 函数后面会调用 s_init 函数，s_init 函数定义在文件 arch/arm/cpu/armv7/mx6/soc.c 中，如下所示：

示例代码 32.2.3.1 soc.c 代码段

```
808 void s_init(void)
809 {
810     struct anatop_regs *anatop = (struct anatop_regs
*)ANATOP_BASE_ADDR;
811     struct mxc_ccm_reg *ccm = (struct mxc_ccm_reg *)CCM_BASE_ADDR;
812     u32 mask480;
813     u32 mask528;
814     u32 reg, periph1, periph2;
815
816     if (is_cpu_type(MXC_CPU_MX6SX) || is_cpu_type(MXC_CPU_MX6UL) ||
817         is_cpu_type(MXC_CPU_MX6ULL) || is_cpu_type(MXC_CPU_MX6SLL))
818         return;
819
820     /* Due to hardware limitation, on MX6Q we need to gate/ungate
821      * all PFDs to make sure PFD is working right, otherwise, PFDs
822      * may not output clock after reset, MX6DL and MX6SL have added
823      * 396M pfd workaround in ROM code, as bus clock need it
824      */
825
826     mask480 = ANATOP_PFD_CLKGATE_MASK(0) |
827         ANATOP_PFD_CLKGATE_MASK(1) |
828         ANATOP_PFD_CLKGATE_MASK(2) |
829         ANATOP_PFD_CLKGATE_MASK(3);
830     mask528 = ANATOP_PFD_CLKGATE_MASK(1) |
831         ANATOP_PFD_CLKGATE_MASK(3);
832
833     reg = readl(&ccm->cbcmr);
834     periph2 = ((reg & MXC_CCM_CBCMR_PRE_PERIPH2_CLK_SEL_MASK)
835         >> MXC_CCM_CBCMR_PRE_PERIPH2_CLK_SEL_OFFSET);
836     periph1 = ((reg & MXC_CCM_CBCMR_PRE_PERIPH_CLK_SEL_MASK)
837         >> MXC_CCM_CBCMR_PRE_PERIPH_CLK_SEL_OFFSET);
838
839     /* Checking if PLL2 PFD0 or PLL2 PFD2 is using for periph clock */
840     if ((periph2 != 0x2) && (periph1 != 0x2))
841         mask528 |= ANATOP_PFD_CLKGATE_MASK(0);
842
843     if ((periph2 != 0x1) && (periph1 != 0x1) &&
844         (periph2 != 0x3) && (periph1 != 0x3))
845         mask528 |= ANATOP_PFD_CLKGATE_MASK(2);
846
847     writel(mask480, &anatop->pfd_480_set);
848     writel(mask528, &anatop->pfd_528_set);
849     writel(mask480, &anatop->pfd_480_clr);
```



```

850     writel(mask528, &anatop->pfd_528_clr);
851 }

```

在第 816 行会判断当前 CPU 类型, 如果 CPU 为 MX6SX、MX6UL、MX6ULL 或 MX6SL 中的任意一种, 那么就会直接返回, 相当于 s_init 函数什么都没做。所以对于 I.MX6UL/I.MX6ULL 来说, s_init 就是个空函数。从 s_init 函数退出以后进入函数 lowlevel_init, 但是 lowlevel_init 函数也执行完成了, 返回到了函数 cpu_init_crit, 函数 cpu_init_crit 也执行完成了, 最终返回到 save_boot_params_ret, 函数调用路径如图 32.2.3.1 所示:

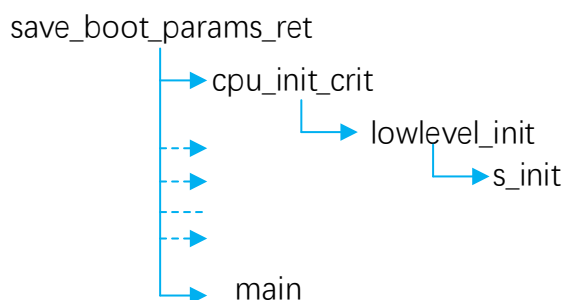


图 32.2.3.1 uboot 函数调用路径

从图 32.2.3.1 可知, 接下来要执行的是 save_boot_params_ret 中的 _main 函数, 接下来分析 _main 函数。

32.2.4 _main 函数详解

_main 函数定义在文件 arch/arm/lib/crt0.S 中, 函数内容如下:

示例代码 32.2.4.1 crt0.S 代码段

```

63  /*
64   * entry point of crt0 sequence
65   */
66
67  ENTRY(_main)
68
69  /*
70   * Set up initial C runtime environment and call board_init_f(0).
71   */
72
73  #if defined(CONFIG_SPL_BUILD) && defined(CONFIG_SPL_STACK)
74      ldr sp, =(CONFIG_SPL_STACK)
75  #else
76      ldr sp, =(CONFIG_SYS_INIT_SP_ADDR)
77  #endif
78  #if defined(CONFIG_CPU_V7M) /* v7M forbids using SP as BIC
destination */
79      mov r3, sp
80      bic r3, r3, #7
81      mov sp, r3
82  #else

```

```
83     bic sp, sp, #7 /* 8-byte alignment for ABI compliance */
84 #endif
85     mov r0, sp
86     bl board_init_f_alloc_reserve
87     mov sp, r0
88     /* set up gd here, outside any C code */
89     mov r9, r0
90     bl board_init_f_init_reserve
91
92     mov r0, #0
93     bl board_init_f
94
95 #if ! defined(CONFIG_SPL_BUILD)
96
97 /*
98  * Set up intermediate environment (new sp and gd) and call
99  * relocate_code(addr_moni). Trick here is that we'll return
100  * 'here' but relocated.
101  */
102
103     ldr sp, [r9, #GD_START_ADDR_SP] /* sp = gd->start_addr_sp */
104 #if defined(CONFIG_CPU_V7M) /* v7M forbids using SP as BIC
destination */
105     mov r3, sp
106     bic r3, r3, #7
107     mov sp, r3
108 #else
109     bic sp, sp, #7 /* 8-byte alignment for ABI compliance */
110 #endif
111     ldr r9, [r9, #GD_BD] /* r9 = gd->bd */
112     sub r9, r9, #GD_SIZE /* new GD is below bd */
113
114     adr lr, here
115     ldr r0, [r9, #GD_RELOC_OFF] /* r0 = gd->reloc_off */
116     add lr, lr, r0
117 #if defined(CONFIG_CPU_V7M)
118     orr lr, #1 /* As required by Thumb-only */
119 #endif
120     ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
121     b relocate_code
122 here:
123 /*
124  * now relocate vectors
```

```
125 */
126
127     bl    relocate_vectors
128
129 /* Set up final (full) environment */
130
131     bl    c_runtime_cpu_setup /* we still call old routine here */
132 #endif
133 #if !defined(CONFIG_SPL_BUILD) || defined(CONFIG_SPL_FRAMEWORK)
134 # ifdef CONFIG_SPL_BUILD
135     /* Use a DRAM stack for the rest of SPL, if requested */
136     bl    spl_relocate_stack_gd
137     cmp   r0, #0
138     movne sp, r0
139     movne r9, r0
140 # endif
141     ldr   r0, =__bss_start /* this is auto-relocated! */
142
143 #ifdef CONFIG_USE_ARCH_MEMSET
144     ldr   r3, =__bss_end /* this is auto-relocated! */
145     mov   r1, #0x00000000 /* prepare zero to clear BSS */
146
147     subs  r2, r3, r0 /* r2 = memset len */
148     bl    memset
149 #else
150     ldr   r1, =__bss_end /* this is auto-relocated! */
151     mov   r2, #0x00000000 /* prepare zero to clear BSS */
152
153 clbss_1: cmp r0, r1 /* while not at end of BSS */
154 #if defined(CONFIG_CPU_V7M)
155     itt  lo
156 #endif
157     strlo r2, [r0] /* clear 32-bit BSS word */
158     addlo r0, r0, #4 /* move to next */
159     blo  clbss_1
160 #endif
161
162 #if ! defined(CONFIG_SPL_BUILD)
163     bl    coloured_LED_init
164     bl    red_led_on
165 #endif
166 /* call board_init_r(gd_t *id, ulong dest_addr) */
167     mov   r0, r9 /* gd_t */
```

```
168     ldr r1, [r9, #GD_RELOCADDR] /* dest_addr */
169     /* call board_init_r */
170     #if defined(CONFIG_SYS_THUMB_BUILD)
171     ldr lr, =board_init_r /* this is auto-relocated! */
172     bx lr
173 #else
174     ldr pc, =board_init_r /* this is auto-relocated! */
175 #endif
176     /* we should not return here. */
177 #endif
178
179 ENDPROC(_main)
```

第 76 行, 设置 sp 指针为 CONFIG_SYS_INIT_SP_ADDR, 也就是 sp 指向 0X0091FF00。

第 83 行, sp 做 8 字节对齐。

第 85 行, 读取 sp 到寄存器 r0 里面, 此时 r0=0X0091FF00。

第 86 行, 调用函数 board_init_f_alloc_reserve, 此函数有一个参数, 参数为 r0 中的值, 也就是 0X0091FF00, 此函数定义在文件 common/init/board_init.c 中, 内容如下:

示例代码 32.2.4.2 board_init.c 代码段

```
56 ulong board_init_f_alloc_reserve(ulong top)
57 {
58     /* Reserve early malloc arena */
59     #if defined(CONFIG_SYS_MALLOC_F)
60     top -= CONFIG_SYS_MALLOC_F_LEN;
61     #endif
62     /* LAST : reserve GD (rounded up to a multiple of 16 bytes) */
63     top = rounddown(top-sizeof(struct global_data), 16);
64
65     return top;
66 }
```

函数 board_init_f_alloc_reserve 主要是留出早期的 malloc 内存区域和 gd 内存区域, 其中 CONFIG_SYS_MALLOC_F_LEN=0X400(在文件 include/generated/autoconf.h 中定义), sizeof(struct global_data)=248(GD_SIZE 值), 完成以后的内存分布如图 32.2.4.1 所示:

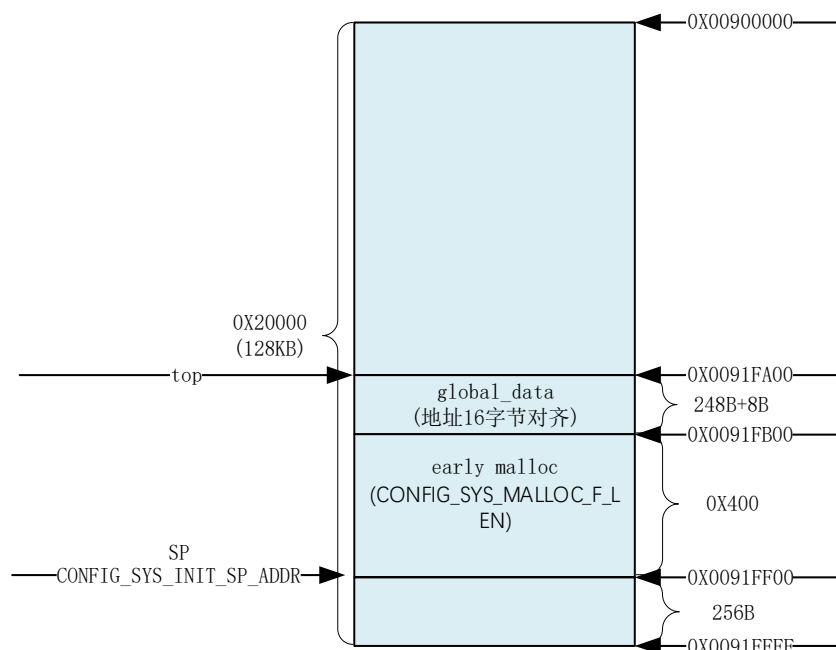


图 32.2.4.1 内存分布图

函数 `board_init_f_alloc_reserve` 是有返回值的，返回值为新的 `top` 值，从图 32.2.4.1 可知，此时 `top=0X0091FA00`。

继续回到示例代码 32.2.4.1 中，第 87 行，将 `r0` 写入到 `sp` 里面，`r0` 保存着函数 `board_init_f_alloc_reserve` 的返回值，所以这一句也就是设置 `sp=0X0091FA00`。

第 89 行，将 `r0` 寄存器的值写到寄存器 `r9` 里面，因为 `r9` 寄存器存放着全局变量 `gd` 的地址，在文件 `arch/arm/include/asm/global_data.h` 中有如图 32.2.4.2 所示宏定义：

```

83 #ifdef CONFIG_ARM64
84 #define DECLARE_GLOBAL_DATA_PTR    register volatile gd_t *gd asm ("x18")
85 #else
86 #define DECLARE_GLOBAL_DATA_PTR    register volatile gd_t *gd asm ("r9")
87 #endif
88 #endif

```

图 32.2.4.2 DECLARE_GLOBAL_DATA_PTR 宏定义

从图 32.2.4.2 可以看出，uboot 中定义了一个指向 `gd_t` 的指针 `gd`，`gd` 存放在寄存器 `r9` 里面的，因此 `gd` 是个全局变量。`gd_t` 是个结构体，在 `include/asm-generic/global_data.h` 里面有定义，`gd` 定义如下：

示例代码 32.2.4.3 global_data.h 代码段

```

27 typedef struct global_data {
28     bd_t *bd;
29     unsigned long flags;
30     unsigned int baudrate;
31     unsigned long cpu_clk; /* CPU clock in Hz! */
32     unsigned long bus_clk;
33     /* We cannot bracket this with CONFIG_PCI due to mpc5xxx */
34     unsigned long pci_clk;
35     unsigned long mem_clk;

```

```

36 #if defined(CONFIG_LCD) || defined(CONFIG_VIDEO)
37     unsigned long fb_base; /* Base address of framebuffer mem */
38 #endif
.....
121 #ifdef CONFIG_DM_VIDEO
122     ulong video_top;      /* Top of video frame buffer area */
123     ulong video_bottom;   /* Bottom of video frame buffer area */
124 #endif
125 } gd_t;

```

因此这一行代码就是设置 gd 所指向的位置，也就是 gd 指向 0X0091FA00。

继续回到示例代码 32.2.4.1 中，第 90 行调用函数 board_init_f_init_reserve，此函数在文件 common/init/board_init.c 中有定义，函数内容如下：

示例代码 32.2.4.4 board_init.c 代码段

```

110 void board_init_f_init_reserve(ulong base)
111 {
112     struct global_data *gd_ptr;
113 #ifndef _USE_MEMCPY
114     int *ptr;
115 #endif
116
117     /*
118      * clear GD entirely and set it up.
119      * Use gd_ptr, as gd may not be properly set yet.
120      */
121
122     gd_ptr = (struct global_data *)base;
123     /* zero the area */
124 #ifdef _USE_MEMCPY
125     memset(gd_ptr, '\0', sizeof(*gd));
126 #else
127     for (ptr = (int *)gd_ptr; ptr < (int *) (gd_ptr + 1); )
128         *ptr++ = 0;
129 #endif
130     /* set GD unless architecture did it already */
131 #if !defined(CONFIG_ARM)
132     arch_setup_gd(gd_ptr);
133 #endif
134     /* next alloc will be higher by one GD plus 16-byte alignment */
135     base += roundup(sizeof(struct global_data), 16);
136
137     /*
138      * record early malloc arena start.
139      * Use gd as it is now properly set for all architectures.

```

```

140     */
141
142 #if defined(CONFIG_SYS_MALLOC_F)
143     /* go down one 'early malloc arena' */
144     gd->malloc_base = base;
145     /* next alloc will be higher by one 'early malloc arena' size */
146     base += CONFIG_SYS_MALLOC_F_LEN;
147 #endif
148 }

```

可以看出, 此函数用于初始化 `gd`, 其实就是清零处理。另外, 此函数还设置了 `gd->malloc_base` 为 `gd` 基地址+`gd` 大小=`0X0091FA00+248=0X0091FAF8`, 在做 16 字节对齐, 最终 `gd->malloc_base=0X0091FB00`, 这个也就是 early malloc 的起始地址。

继续回到示例代码 32.2.4.1 中, 第 92 行设置 `R0` 为 0。

第 93 行, 调用 `board_init_f` 函数, 此函数定义在文件 `common/board_f.c` 中! 主要用来初始化 DDR, 定时器, 完成代码拷贝等等, 此函数我们后面在详细的分析。

第 103 行, 重新设置环境(`sp` 和 `gd`)、获取 `gd->start_addr_sp` 的值赋给 `sp`, 在函数 `board_init_f` 中会初始化 `gd` 的所有成员变量, 其中 `gd->start_addr_sp=0X9EF44E90`, 所以这里相当于设置 `sp=gd->start_addr_sp=0X9EF44E90`。0X9EF44E90 是 DDR 中的地址, 说明新的 `sp` 和 `gd` 将会存放在 DDR 中, 而不是内部的 RAM 了。GD_START_ADDR_SP=64, 参考示例代码 32.2.2.4。

第 109 行, `sp` 做 8 字节对齐。

第 111 行, 获取 `gd->bd` 的地址赋给 `r9`, 此时 `r9` 存放的是老的 `gd`, 这里通过获取 `gd->bd` 的地址来计算出新的 `gd` 的位置。GD_BD=0, 参考示例代码 32.2.2.4。

第 112 行, 新的 `gd` 在 `bd` 下面, 所以 `r9` 减去 `gd` 的大小就是新的 `gd` 的位置, 获取到新的 `gd` 的位置以后赋值给 `r9`。

第 114 行, 设置 `lr` 寄存器为 `here`, 这样后面执行其他函数返回的时候就返回到了第 122 行的 `here` 位置处。

第 115, 读取 `gd->reloc_off` 的值复制给 `r0` 寄存器, GD_RELOC_OFF=68, 参考示例代码 32.2.2.4。

第 116 行, `lr` 寄存器的值加上 `r0` 寄存器的值, 重新赋值给 `lr` 寄存器。因为接下来要重定位代码, 也就是把代码拷贝到新的地方去(现在的 `uboot` 存放的起始地址为 `0X87800000`, 下面要将 `uboot` 拷贝到 DDR 最后面的地址空间出, 将 `0X87800000` 开始的内存空出来), 其中就包括 `here`, 因此 `lr` 中的 `here` 要使用重定位后的位置。

第 120 行, 读取 `gd->relocaddr` 的值赋给 `r0` 寄存器, 此时 `r0` 寄存器就保存着 `uboot` 要拷贝的目的地址, 为 `0X9FF47000`。GD_RELOCADDR=48, 参考示例代码 32.2.2.4。

第 121 行, 调用函数 `relocate_code`, 也就是代码重定位函数, 此函数负责将 `uboot` 拷贝到新的地方去, 此函数定义在文件 `arch/arm/lib/relocate.S` 中稍后会详细分析此函数。

继续回到示例代码 32.2.4.1 第 127 行, 调用函数 `relocate_vectors`, 对中断向量表做重定位, 此函数定义在文件 `arch/arm/lib/relocate.S` 中, 稍后会详细分析此函数。

继续回到示例代码 32.2.4.1 第 131 行, 调用函数 `c_runtime_cpu_setup`, 此函数定义在文件 `arch/arm/cpu/armv7/start.S` 中, 函数内容如下:

示例代码 32.2.4.5 start.S 代码段

```

77 ENTRY(c_runtime_cpu_setup)
78 /*

```



```

79  * If I-cache is enabled invalidate it
80  */
81  #ifndef CONFIG_SYS_ICACHE_OFF
82      mcr p15, 0, r0, c7, c5, 0 @ invalidate icache
83      mcr    p15, 0, r0, c7, c10, 4 @ DSB
84      mcr    p15, 0, r0, c7, c5, 4 @ ISB
85  #endif
86
87      bx lr
88
89  ENDPROC(c_runtime_cpu_setup)

```

第 141~159 行，清除 BSS 段。

第 167 行，设置函数 board_init_r 的两个参数，函数 board_init_r 声明如下：

```
board_init_r(gd_t *id, ulong dest_addr)
```

第一个参数是 gd，因此读取 r9 保存到 r0 里面。

第 168 行，设置函数 board_init_r 的第二个参数是目的地址，因此 r1=gd->relocaddr。

第 174 行、调用函数 board_init_r，此函数定义在文件 common/board_r.c 中，稍后会详细的分析此函数。

这个就是 _main 函数的运行流程，在 _main 函数里面调用了 board_init_f、relocate_code、relocate_vectors 和 board_init_r 这 4 个函数，接下来依次看一下这 4 个函数都是干啥的。

32.2.5 board_init_f 函数详解

_main 中会调用 board_init_f 函数，board_init_f 函数主要有两个工作：

①、初始化一系列外设，比如串口、定时器，或者打印一些消息等。

②、初始化 gd 的各个成员变量，uboot 会将自己重定位到 DRAM 最后面的地址区域，也就是将自己拷贝到 DRAM 最后面的内存区域中。这么做的目的是给 Linux 腾出空间，防止 Linux kernel 覆盖掉 uboot，将 DRAM 前面的区域完整的空出来。在拷贝之前肯定要给 uboot 各部分分配好内存位置和大小，比如 gd 应该存放到哪个位置，malloc 内存池应该存放到哪个位置等等。这些信息都保存在 gd 的成员变量中，因此要对 gd 的这些成员变量做初始化。最终形成一个完整的内存“分配图”，在后面重定位 uboot 的时候就会用到这个内存“分配图”。

此函数定义在文件 common/board_f.c 中定义，代码如下：

示例代码 32.2.5.1 board_f.c 代码段

```

1035 void board_init_f(ulong boot_flags)
1036 {
1037     #ifdef CONFIG_SYS_GENERIC_GLOBAL_DATA
1038         /*
1039          * For some architectures, global data is initialized and used
1040          * before calling this function. The data should be preserved.
1041          * For others, CONFIG_SYS_GENERIC_GLOBAL_DATA should be defined
1042          * and use the stack here to host global data until relocation.
1043          */
1044         gd_t data;
1045     #endif

```

```

1046     gd = &data;
1047
1048     /*
1049      * Clear global data before it is accessed at debug print
1050      * in initcall_run_list. Otherwise the debug print probably
1051      * get the wrong vaule of gd->have_console.
1052      */
1053     zero_global_data();
1054 #endif
1055
1056     gd->flags = boot_flags;
1057     gd->have_console = 0;
1058
1059     if (initcall_run_list(init_sequence_f))
1060         hang();
1061
1062 #if !defined(CONFIG_ARM) && !defined(CONFIG_SANDBOX) && \
1063     !defined(CONFIG_EFI_APP)
1064     /* NOTREACHED - jump_to_copy() does not return */
1065     hang();
1066 #endif
1067 }

```

因为没有定义 CONFIG_SYS_GENERIC_GLOBAL_DATA, 所以第 1037~1054 行代码无效。

第 1056 行, 初始化 gd->flags=boot_flags=0。

第 1057 行, 设置 gd->have_console=0。

重点在第 1059 行! 通过函数 initcall_run_list 来运行初始化序列 init_sequence_f 里面的一些列函数, init_sequence_f 里面包含了一系列的初始化函数, init_sequence_f 也是定义在文件 common/board_f.c 中, 由于 init_sequence_f 的内容比较长, 里面有大量的条件编译代码, 这里为了缩小篇幅, 将条件编译部分删除掉了, 去掉条件编译以后的 init_sequence_f 定义如下:

示例代码 32.2.5.1 board_f.c 代码段

```

/*****去掉条件编译语句后的 init_sequence_f*****/
1 static init_fnc_t init_sequence_f[] = {
2     setup_mon_len,
3     initf_malloc,
4     initf_console_record,
5     arch_cpu_init,      /* basic arch cpu dependent setup */
6     initf_dm,
7     arch_cpu_init_dm,
8     mark_bootstage,     /* need timer, go after init dm */
9     board_early_init_f,
10    timer_init,          /* initialize timer */
11    board_postclk_init,
12    get_clocks,

```

```
13     env_init,                /* initialize environment          */
14     init_baud_rate,          /* initialize baudrate settings    */
15     serial_init,             /* serial communications setup     */
16     console_init_f,          /* stage 1 init of console        */
17     display_options,         /* say that we are here           */
18     display_text_info,       /* show debugging info if required */
19     print_cpuinfo,           /* display cpu info (and speed)    */
20     show_board_info,
21     INIT_FUNC_WATCHDOG_INIT
22     INIT_FUNC_WATCHDOG_RESET
23     init_func_i2c,
24     announce_dram_init,
25     /* TODO: unify all these dram functions? */
26     dram_init,                /* configure available RAM banks */
27     post_init_f,
28     INIT_FUNC_WATCHDOG_RESET
29     testdram,
30     INIT_FUNC_WATCHDOG_RESET
31     INIT_FUNC_WATCHDOG_RESET
32     /*
33     * Now that we have DRAM mapped and working, we can
34     * relocate the code and continue running from DRAM.
35     *
36     * Reserve memory at end of RAM for (top down in that order):
37     * - area that won't get touched by U-Boot and Linux (optional)
38     * - kernel log buffer
39     * - protected RAM
40     * - LCD framebuffer
41     * - monitor code
42     * - board info struct
43     */
44     setup_dest_addr,
45     reserve_round_4k,
46     reserve_mmu,
47     reserve_trace,
48     reserve_uboot,
49     reserve_malloc,
50     reserve_board,
51     setup_machine,
52     reserve_global_data,
53     reserve_fdt,
54     reserve_arch,
55     reserve_stacks,
```

```

56     setup_dram_config,
57     show_dram_config,
58     display_new_sp,
59     INIT_FUNC_WATCHDOG_RESET
60     reloc_fdt,
61     setup_reloc,
62     NULL,
63 };

```

接下来分析以上函数执行完以后的结果:

第 2 行, setup_mon_len 函数设置 gd 的 mon_len 成员变量, 此处为 __bss_end - _start, 也就是整个代码的长度。0X878A8E74-0x87800000=0XA8E74, 这个就是代码长度

第 3 行, initf_malloc 函数初始化 gd 中跟 malloc 有关的成员变量, 比如 malloc_limit, 此函数会设置 gd->malloc_limit=CONFIG_SYS_MALLOC_F_LEN=0X400。malloc_limit 表示 malloc 内存池大小。

第 4 行, initf_console_record, 如果定义了宏 CONFIG_CONSOLE_RECORD 和宏 CONFIG_SYS_MALLOC_F_LEN 的话此函数就会调用函数 console_record_init, 但是 IMX6ULL 的 uboot 没有定义宏 CONFIG_CONSOLE_RECORD, 所以此函数直接返回 0。

第 5 行, arch_cpu_init 函数。

第 6 行, initf_dm 函数, 驱动模型的一些初始化。

第 7 行, arch_cpu_init_dm 函数未实现。

第 8 行, mark_bootstage 函数应该是和啥标记有关的

第 9 行, board_early_init_f 函数, 板子相关的早期的一些初始化设置, I.MX6ULL 用来初始化串口的 IO 配置

第 10 行, timer_init, 初始化定时器, Cortex-A7 内核有一个定时器, 这里初始化的就是 Cortex-A 内核的那个定时器。通过这个定时器来为 uboot 提供时间。就跟 Cortex-M 内核 SysTick 定时器一样。关于 Cortex-A 内部定时器的详细内容, 请参考文档《ARM ArchitectureReference Manual ARMv7-A and ARMv7-R edition.pdf》的“Chapter B8 The Generic Timer”章节。

第 11 行, board_postclk_init, 对于 I.MX6ULL 来说是设置 VDDSOC 电压。

第 12 行, get_clocks 函数用于获取一些时钟值, I.MX6ULL 获取的是 sdhc_clk 时钟, 也就是 SD 卡外设的时钟。

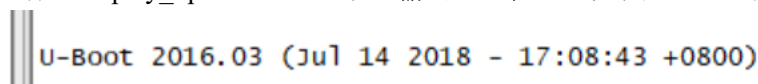
第 13 行, env_init 函数是和环境变量有关的, 设置 gd 的成员变量 env_addr, 也就是环境变量的保存地址。

第 14 行, init_baud_rate 函数用于初始化波特率, 根据环境变量 baudrate 来初始化 gd->baudrate。

第 15 行, serial_init, 初始化串口。

第 16 行, console_init_f, 设置 gd->have_console 为 1, 表示有个控制台, 此函数也将前面暂存在缓冲区中的数据通过控制台打印出来。

第 17 行、display_options, 通过串口输出一些信息, 如图 32.2.5.1 所示:



```

U-Boot 2016.03 (Jul 14 2018 - 17:08:43 +0800)

```

图 32.2.5.1 串口信息输出

第 18 行, display_text_info, 打印一些文本信息, 如果开启 UBOOT 的 DEBUG 功能的话就会输出 text_base、bss_start、bss_end, 形式如下:

```

debug("U-Boot code: %08IX -> %08IX BSS: -> %08IX\n",text_base,bss_start,bss_end);

```

结果如图 32.2.5.2 所示:

```
U-Boot 2016.03 (Aug 01 2018 - 09:44:06 +0800)
initcall: 878119cc
U-Boot code: 87800000 -> 878665E0 BSS: -> 878B1EF8
initcall: 878028ac
CPU: Freescale i.MX6ULL rev1.0 528 MHz (running at 396 MHz)
CPU: Commercial temperature grade (0C to 95C) malloc_simple: size=10, pt
uclass_find_device_by_seq: 0 -1
uclass_find_device_by_seq: 0 0
```

图 32.2.5.2 文本信息

第 19 行, print_cpuinfo 函数用于打印 CPU 信息, 结果如图 32.2.5.3 所示:

```
CPU: Freescale i.MX6ULL rev1.0 528 MHz (running at 396 MHz)
CPU: Commercial temperature grade (0C to 95C) at 47C
Reset cause: WDOG
```

图 32.2.5.3 CPU 信息

第 20 行, show_board_info 函数用于打印板子信息, 会调用 checkboard 函数, 结果如图 32.2.5.4 所示:

```
CPU: Freescale i.MX6ULL rev1.0 528 MHz (running at 396 MHz)
CPU: Commercial temperature grade (0C to 95C) at 42C
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C: ready
DRAM: 512 MiB
MMC: FSL_SDHC: 0, FSL_SDHC: 1
```

图 32.2.5.4 板子信息

第 21 行, INIT_FUNC_WATCHDOG_INIT, 初始化看门狗, 对于 I.MX6ULL 来说是空函数

第 22 行, INIT_FUNC_WATCHDOG_RESET, 复位看门狗, 对于 I.MX6ULL 来说是空函数

第 23 行, init_func_i2c 函数用于初始化 I2C, 初始化完成以后会输出如图 32.2.5.5 所示信息:

```
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C: ready
DRAM: 512 MiB
MMC: FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT43AB (480x272)
Video: 480x272x24
```

图 32.2.5.5 I2C 初始化信息输出

第 24 行, announce_dram_init, 此函数很简单, 就是输出字符串 “DRAM:”

第 26 行, dram_init, 并非真正的初始化 DDR, 只是设置 gd->ram_size 的值, 对于正点原子 I.MX6ULL 开发板 EMMC 版本核心板来说就是 512MB。

第 27 行, post_init_f, 此函数用来完成一些测试, 初始化 gd->post_init_f_time

第 29 行, testdram, 测试 DRAM, 空函数。

第 44 行, setup_dest_addr 函数, 设置目的地址, 设置 gd->ram_size, gd->ram_top, gd->relocaddr 这三个的值。接下来我们会遇到很多跟数值有关的设置, 如果直接看代码分析的话就太浪费时间了, 我可以修改 uboot 代码, 直接将这值通过串口打印出来, 比如这里我们修改文件 common/board_f.c, 因为 setup_dest_addr 函数定义在文件 common/board_f.c 中, 在 setup_dest_addr 函数输入如图 32.2.5.6 所示内容:

```

358     gd->ram_top += get_effective_memsizes();
359     gd->ram_top = board_get_usable_ram_top(gd->mon_len);
360     gd->relocaddr = gd->ram_top;
361     debug("Ram top: %08lx\n", (ulong)gd->ram_top);
362     #if defined(CONFIG_MP) && (defined(CONFIG_MPC86xx) || defined(CONFIG_E500))
363     /*
364      * We need to make sure the location we intend to put secondary core
365      * boot code is reserved and not used by any part of u-boot
366      */
367     if (gd->relocaddr > determine_mp_bootpg(NULL)) {
368         gd->relocaddr = determine_mp_bootpg(NULL);
369         debug("Reserving MP boot page to %08lx\n", gd->relocaddr);
370     }
371     #endif
372
373     printf("gd->ram_size %#x\r\n", gd->ram_size);
374     printf("gd->ram_top %#x\r\n", gd->ram_top);
375     printf("gd->relocaddr %#x\r\n", gd->relocaddr);
376
377     return 0;
378 }

```

通过串口输出这三个成员变量的值

图 32.2.5.6 添加 print 函数打印成员变量值

设置好以后重新编译 uboot，然后烧写到 SD 卡中，选择 SD 卡启动，重启开发板，打开 SecureCRT，uboot 会输出如图 32.2.5.7 所示信息：

```

DRAM: gd->ram_size 0x20000000
gd->ram_top 0xa0000000
gd->relocaddr 0xa0000000

```

图 32.2.5.7 信息输出

从图 32.2.5.7 可以看出：

```

gd->ram_size = 0X20000000    //ram 大小为 0X20000000=512MB
gd->ram_top = 0XA0000000    //ram 最高地址为 0X80000000+0X20000000=0XA0000000
gd->relocaddr = 0XA0000000  //重定位后最高地址为 0XA0000000

```

第 45 行，reserve_round_4k 函数用于对 gd->relocaddr 做 4KB 对齐，因为 gd->relocaddr=0XA0000000，已经是 4K 对齐了，所以调整后不变。

第 46 行，reserve_mmu，留出 MMU 的 TLB 表的位置，分配 MMU 的 TLB 表内存以后会对 gd->relocaddr 做 64K 字节对齐。完成以后 gd->arch.tlb_size、gd->arch.tlb_addr 和 gd->relocaddr 如图 32.2.5.8 所示：

```

gd->arch.tlb_size 0x4000
gd->arch.tlb_addr 0x9fff0000
gd->relocaddr 0x9fff0000

```

图 32.2.5.8 信息输出

从图 32.2.5.8 可以看出：

```

gd->arch.tlb_size = 0X4000    //MMU 的 TLB 表大小
gd->arch.tlb_addr = 0X9FFF0000 //MMU 的 TLB 表起始地址，64KB 对齐以后
gd->relocaddr = 0X9FFF0000    //relocaddr 地址

```

第 47 行，reserve_trace 函数，留出跟踪调试的内存，I.MX6ULL 没有用到！

第 48 行，reserve_uboot，留出重定位后的 uboot 所占用的内存区域，uboot 所占用大小由 gd->mon_len 所指定，留出 uboot 的空间以后还要对 gd->relocaddr 做 4K 字节对齐，并且重新设置 gd->start_addr_sp，结果如图 32.2.5.9 所示：


```
gd->mon_len = 0XA8EF4
gd->start_addr_sp = 0X9FF47000
gd->relocaddr = 0X9FF47000
```

图 32.2.5.9 信息输出

从图 32.2.5.9 可以看出:

```
gd->mon_len = 0XA8EF4
gd->start_addr_sp = 0X9FF47000
gd->relocaddr = 0X9FF47000
```

第 49 行, reserve_malloc, 留出 malloc 区域, 调整 gd->start_addr_sp 位置, malloc 区域由宏 TOTAL_MALLOC_LEN 定义, 宏定义如下:

```
#define TOTAL_MALLOC_LEN (CONFIG_SYS_MALLOC_LEN +
CONFIG_ENV_SIZE)
```

mx6ull_alientek_emmc.h 文件中定义宏 CONFIG_SYS_MALLOC_LEN 为 16MB=0X1000000, 宏 CONFIG_ENV_SIZE=8KB=0X2000, 因此 TOTAL_MALLOC_LEN=0X1002000。调整以后 gd->start_addr_sp 如图 32.2.5.10 所示:

```
TOTAL_MALLOC_LEN = 0X1002000
gd->start_addr_sp = 0X9EF45000
```

图 32.2.5.10 信息输出

从图 32.2.5.10 可以看出:

```
TOTAL_MALLOC_LEN=0X1002000
gd->start_addr_sp=0X9EF45000 //0X9FF47000-16MB-8KB=0X9EF45000
```

第 50 行, reserve_board 函数, 留出板子 bd 所占的内存区, bd 是结构体 bd_t, bd_t 大小为 80 字节, 结果如图 32.2.5.11 所示:

```
gd->bd = 0X9EF44FB0
gd->start_addr_sp = 0X9EF44FB0
```

图 32.2.5.10 信息输出

从图 32.2.5.11 可以看出:

```
gd->start_addr_sp=0X9EF44FB0
gd->bd=0X9EF44FB0
```

第 51 行, setup_machine, 设置机器 ID, linux 启动的时候会在这个机器 ID 匹配, 如果匹配的话 linux 就会启动正常。但是!! I.MX6ULL 不用这种方式了, 这是以前老版本的 uboot 和 linux 使用的, 新版本使用设备树了, 因此此函数无效。

第 52 行, reserve_global_data 函数, 保留出 gd_t 的内存区域, gd_t 结构体大小为 248B, 结果如图 32.2.5.11 所示:

```
gd->new_gd = 0x9ef44eb8
gd->start_addr_sp = 0x9ef44eb8
```

图 32.2.5.11 信息输出

```
gd->start_addr_sp=0X9EF44EB8 //0X9EF44FB0-248=0X9EF44EB8
gd->new_gd=0X9EF44EB8
```

第 53 行, reserve_fdt, 留出设备树相关的内存区域, I.MX6ULL 的 uboot 没有用到, 因此此函数无效。

第 54 行, reserve_arch 是个空函数。

第 55 行, reserve_stacks, 留出栈空间, 先对 gd->start_addr_sp 减去 16, 然后做 16 字节对

其。如果使能 IRQ 的话还要留出 IRQ 相应的内存，具体工作是由 arch/arm/lib/stack.c 文件中的函数 arch_reserve_stacks 完成。结果如图 32.2.5.12 所示：

```
gd->start_addr_sp = 0x9ef44e90
```

图 32.2.5.12 信息输出

在本 uboot 中并没有使用到 IRQ，所以不会留出 IRQ 相应的内存区域，此时：

```
gd->start_addr_sp=0X9EF44E90
```

第 56 行，setup_dram_config 函数设置 dram 信息，就是设置 gd->bd->bi_dram[0].start 和 gd->bd->bi_dram[0].size，后面会传递给 linux 内核，告诉 linux DRAM 的起始地址和大小。结果如图 32.2.5.13 所示：

```
gd->bd->bi_dram[0].start = 0x80000000
gd->bd->bi_dram[0].size = 0x20000000
```

图 32.2.5.13 信息输出

从图 32.2.5.13 可以看出，DRAM 的起始地址为 0X80000000，大小为 0X20000000(512MB)。

第 57 行，show_dram_config 函数，用于显示 DRAM 的配置，如图 32.2.5.14 所示：

```
Board: MX6ULL 14x14 EVK
I2C: ready
DRAM: 512 MiB
MMC: FSL_SDHC: 0, FSL_SDHC: 1
```

图 32.2.5.14 信息输出

第 58 行，display_new_sp 函数，显示新的 sp 位置，也就是 gd->start_addr_sp，不过要定义宏 DEBUG，结果如图 32.2.5.15 所示：

```
Reset cause: unknown reset
Board: MX6ULL 14x14 EVK
I2C: ready
DRAM: 512 MiB
New Stack Pointer is: 9ef44e90
MMC: FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT43AB (480x272)
Video: 480x272x24
In: serial
```

图 32.2.5.15 信息输出

图 32.2.5.15 中的 gd->start_addr_sp 值和我们前面分析的最后一次修改的值一致。

第 60 行，reloc_fdt 函数用于重定位 fdt，没有用到。

第 61 行，setup_reloc，设置 gd 的其他一些成员变量，供后面重定位的时候使用，并且将以前的 gd 拷贝到 gd->new_gd 处。需要使能 DEBUG 才能看到相应的信息输出，如图 32.2.5.16 所示：

```
DRAM: 512 MiB
Relocation offset is: 18747000
Relocating to 9ff47000, new gd at 9ef44eb8, sp at 9ef44e90
```

图 32.2.5.16 信息输出

从图 32.2.5.16 可以看出，uboot 重定位后的偏移为 0X18747000，重定位后的新地址为 0X9FF4700，新的 gd 首地址为 0X9EF44EB8，最终的 sp 为 0X9EF44E90。

至此，board_init_f 函数就执行完成了，最终的内存分配如图 32.2.5.16 所示：

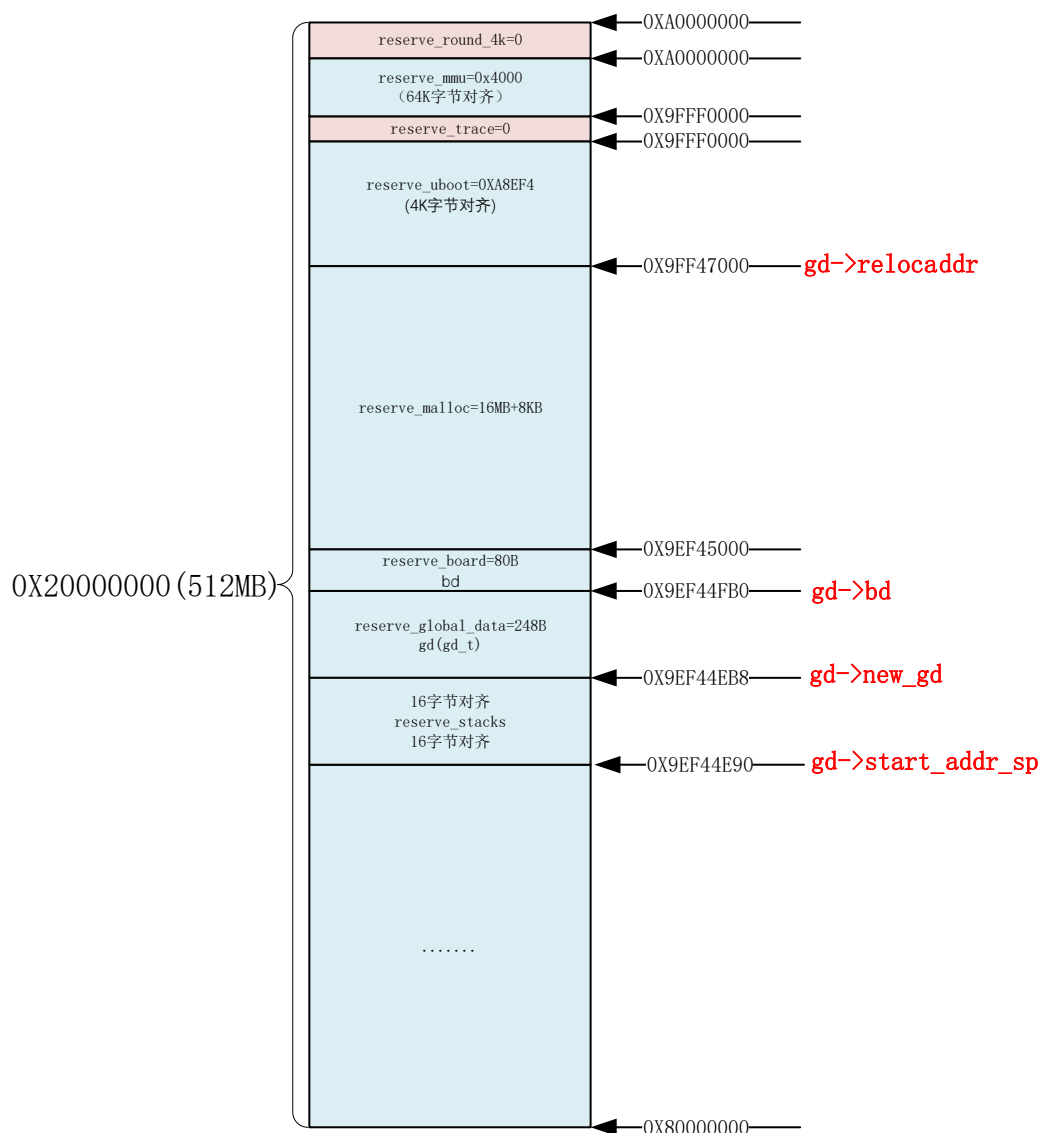


图 32.2.5.16 最终的内存分配图

32.2.6 relocate_code 函数详解

relocate_code 函数是用于代码拷贝的，此函数定义在文件 arch/arm/lib/relocate.S 中，代码如下：

示例代码 32.2.6.1 relocate.S 代码段

```
/*
 * void relocate_code(addr_moni)
 *
 * This function relocates the monitor code.
 *
 * NOTE:
 * To prevent the code below from containing references with an
 * R_ARM_ABS32 relocation record type, we never refer to linker-
```

```

* defined symbols directly. Instead, we declare literals which
* contain their relative location with respect to relocate_code,
* and at run time, add relocate_code back to them.
*/

79 ENTRY(relocate_code)
80     ldr r1, =__image_copy_start /* r1 <- SRC &__image_copy_start */
81     subs    r4, r0, r1          /* r4 <- relocation offset */
82     beq relocate_done          /* skip relocation */
83     ldr r2, =__image_copy_end  /* r2 <- SRC &__image_copy_end */
84
85 copy_loop:
86     ldmbia  r1!, {r10-r11}      /* copy from source address [r1] */
87     stmbia  r0!, {r10-r11}      /* copy to target address [r0] */
88     cmp r1, r2                  /* until source end address [r2] */
89     blo copy_loop
90
91     /*
92     * fix .rel.dyn relocations
93     */
94     ldr r2, =__rel_dyn_start /* r2 <- SRC &__rel_dyn_start */
95     ldr r3, =__rel_dyn_end  /* r3 <- SRC &__rel_dyn_end */
96 fixloop:
97     ldmbia  r2!, {r0-r1}        /* (r0,r1) <- (SRC location,fixup) */
98     and r1, r1, #0xff
99     cmp r1, #23                /* relative fixup? */
100    bne fixnext
101
102    /* relative fix: increase location by offset */
103    add r0, r0, r4
104    ldr r1, [r0]
105    add r1, r1, r4
106    str r1, [r0]
107 fixnext:
108    cmp r2, r3
109    blo fixloop
110
111 relocate_done:
112
113 #ifdef __XSCALE__
114     /*
115     * On xscale, icache must be invalidated and write buffers
116     * drained, even with cache disabled - 4.2.7 of xscale core

```

```

117     developer's manual */
118     mcr p15, 0, r0, c7, c7, 0 /* invalidate icache */
119     mcr p15, 0, r0, c7, c10, 4 /* drain write buffer */
120 #endif
121
122     /* ARMv4- don't know bx lr but the assembler fails to see that */
123
124 #ifdef __ARM_ARCH_4__
125     mov pc, lr
126 #else
127     bx lr
128 #endif
129
130 ENDPROC(relocate_code)

```

第 80 行, `r1=__image_copy_start`, 也就是 `r1` 寄存器保存源地址, 由表 31.4.1.1 可知, `__image_copy_start=0X87800000`。

第 81 行, `r0=0X9FF47000`, 这个地址就是 `uboot` 拷贝的目标首地址。`r4=r0-r1=0X9FF47000-0X87800000=0X18747000`, 因此 `r4` 保存偏移量。

第 82 行, 如果在第 81 中, `r0-r1` 等于 0, 说明 `r0` 和 `r1` 相等, 也就是源地址和目的地址是一样的, 那肯定就不需要拷贝了! 执行 `relocate_done` 函数

第 83 行, `r2=__image_copy_end`, `r2` 中保存拷贝之前的代码结束地址, 由表 31.4.1.1 可知, `__image_copy_end=0x8785dd54`。

第 84 行, 函数 `copy_loop` 完成代码拷贝工作! 从 `r1`, 也就是 `__image_copy_start` 开始, 读取 `uboot` 代码保存到 `r10` 和 `r11` 中, 一次就只拷贝这 2 个 32 位的数据。拷贝完成以后 `r1` 的值会更新, 保存下一个要拷贝的数据地址。

第 87 行, 将 `r10` 和 `r11` 的数据写到 `r0` 开始的地方, 也就是目的地址。写完以后 `r0` 的值会更新, 更新为下一个要写入的数据地址。

第 88 行, 比较 `r1` 是否和 `r2` 相等, 也就是检查是否拷贝完成, 如果不相等的话说明没有拷贝完成, 没有拷贝完成的话就跳转到 `copy_loop` 接着拷贝, 直至拷贝完成。

接下来的第 94 行~109 行是重定位 `.rel.dyn` 段, `.rel.dyn` 段是存放 `.text` 段中需要重定位地址的集合。重定位就是 `uboot` 将自身拷贝到 `DRAM` 的另一个地方去继续运行(`DRAM` 的高地址处)。我们知道, 一个可执行的 `bin` 文件, 其链接地址和运行地址要相等, 也就是链接到哪个地址, 在运行之前就要拷贝到哪个地址去。现在我们重定位以后, 运行地址就和链接地址不同了, 这样寻址的时候不会出问题吗? 为了分析这个问题, 我们需要在 `mx6ull_alientek_emmc.c` 中输入如下所示内容:

示例代码 32.2.6.2 `mx6ull_alientek_emmc.c` 新添代码段

```

1 static int rel_a = 0;
2
3 void rel_test(void)
4 {
5     rel_a = 100;
6     printf("rel_test\r\n");
7 }

```

最后还需要在 mx6ullevk.c 文件中的 board_init 函数里面调用 rel_test 函数, 否则 rel_reset 不会被编译进 uboot。修改完成后的 mx6ullevk.c 如图 32.2.6.1 所示:

```

826 static int rel_a = 0;
827
828 void rel_test(void)
829 {
830     rel_a = 100;
831     printf("rel_test\r\n");
832 }
833
834 int board_init(void)
835 {
836     rel_test();

```

图 32.2.6.1 加入 rel 测试相关代码

board_init 函数会调用 rel_test, rel_test 会调用全局变量 rel_a, 使用如下命令编译 uboot:

```
./mx6ull_alientek_emmc.sh
```

编译完成以后, 使用 arm-linux-gnueabi-hf-objdump 将 u-boot 进行反汇编, 得到 u-boot.dis 这个汇编文件, 命令如下:

```
arm-linux-gnueabi-hf-objdump -D -m arm u-boot > u-boot.dis
```

在 u-boot.dis 文件中找到 rel_a、rel_test 和 board_init, 相关内容如下所示:

示例代码 32.2.6.3 汇编文件代码段

```

1  87804184 <rel_test>:
2  87804184:    e59f300c    ldr r3, [pc, #12]    ; 87804198 <rel_test+0x14>
3  87804188:    e3a02064    mov r2, #100        ; 0x64
4  8780418c:    e59f0008    ldr r0, [pc, #8]    ; 8780419c <rel_test+0x18>
5  87804190:    e5832000    str r2, [r3]
6  87804194:    ea00d668    b    87839b3c <printf>
7  87804198:    8785da50    ; <UNDEFINED> instruction: 0x8785da50
8  8780419c:    878426a2    strhi r2, [r4, r2, lsr #13]
9
10 878041a0 <board_init>:
11 878041a0:    e92d4010    push {r4, lr}
12 878041a4:    ebfffff6    bl 87804184 <rel_test>
13
14 .....
15
16 8785da50 <rel_a>:
17 8785da50:    00000000    andeq r0, r0, r0

```

第 12 行是 board_init 调用 rel_test 函数, 用到了 bl 指令, 而 bl 指令是位置无关指令, bl 指令是相对寻址的(pc+offset), 因此 uboot 中函数调用是与绝对位置无关的。

再来看一下函数 rel_test 对于全局变量 rel_a 的调用, 第 2 行设置 r3 的值为 pc+12 地址处的值, 因为 ARM 流水线的原因, pc 寄存器的值为当前地址+8, 因此 pc=0X87804184+8=0X8780418C, r3=0X8780418C+12=0X87804198, 第 7 行就是 0X87804198 这个地址, 0X87804198 处的值为 0X8785DA50。根据第 17 行可知, 0X8785DA50 正是变量 rel_a 的地址, 最终 r3=0X8785DA50。

第 3 行, r2=100。

第 5 行, 将 r2 内的值写到 r3 地址处, 也就是设置地址 0X8785DA50 的值为 100, 这不就是示例代码 32.2.6.2 中的第 5 行: rel_a = 100。

总结一下 rel_a=100 的汇编执行过程:

①、在函数 rel_test 末尾处有一个地址为 0X87804198 的内存空间(示例代码 32.2.6.3 第 7 行), 此内存空间保存着变量 rel_a 的地址。

②、函数 rel_test 要想访问变量 rel_a, 首先访问末尾的 0X87804198 来获取变量 rel_a 的地址, 而访问 0X87804198 是通过偏移来访问的, 很明显是个位置无关的操作。

③、通过 0X87804198 获取到变量 rel_a 的地址, 对变量 rel_a 进行操作。

④、可以看出, 函数 rel_test 对变量 rel_a 的访问没有直接进行, 而是使用了一个第三方偏移地址 0X87804198, 专业术语叫做 Label。这个第三方偏移地址就是实现重定位后运行不会出错的重要原因!

uboot 重定位后偏移为 0X18747000, 那么重定位后函数 rel_test 的首地址就是 0X87804184+0X18747000=0X9FF4B184。保存变量 rel_a 地址的 Label 就是 0X9FF4B184+8+12=0X9FF4B198(既: 0X87804198+0X18747000), 变量 rel_a 的地址就为 0X8785DA50+0X18747000=0X9FFA4A50。重定位后函数 rel_test 要想正常访问变量 rel_a 就得设置 0X9FF4B198(重定位后的 Label)地址出的值为 0X9FFA4A50(重定位后的变量 rel_a 地址)。这样就解决了重定位后链接地址和运行地址不一致的问题。

可以看出, uboot 对于重定位后链接地址和运行地址不一致的解决方法就是采用位置无关码, 在使用 ld 进行链接的时候使用选项 “-pie” 生成位置无关的可执行文件。在文件 arch/arm/config.mk 下有如下代码:

示例代码 32.2.6.4 config.mk 文件代码段

```
82 # needed for relocation
83 LDFLAGS_u-boot += -pie
```

第 83 行就是设置 uboot 链接选项, 加入了 “-pie” 选项, 编译链接 uboot 的时候就会使用到 “-pie”, 如图 32.2.6.2 所示:

```
arm-linux-gnueabi-hf-ld.bfd -pie --gc-sections -Bstatic -Ttext 0x87800000 -o u-boot -T u-boot.lds arch/arm/cpu/armv7/start.o --start-group arch/arm/cpu/armv7/built-in.o arch/arm/lib/built-in.o arch/arm/lib/common/built-in.o board/freescale/common/built-in.o board/freescale/mx6ull-aliante emmc/built-in.o cmd/built-in.o common/built-in.o disk/built-in.o drivers/built-in.o drivers/dma/built-in.o drivers/gpio/built-in.o drivers/i2c/built-in.o drivers/mmc/built-in.o drivers/mtd/built-in.o drivers/mtd/onenand/built-in.o drivers/mtd/spi/built-in.o drivers/net/built-in.o drivers/net/phy/built-in.o drivers/pci/built-in.o drivers/power/built-in.o drivers/power/battery/built-in.o drivers/power/fuel_gauge/built-in.o drivers/power/mfd/built-in.o drivers/power/pmic/built-in.o drivers/power/regulator/built-in.o drivers/serial/built-in.o drivers/spi/built-in.o drivers/usb/dwc3/built-in.o drivers/usb/emul/built-in.o drivers/usb/eth/built-in.o drivers/usb/gadget/built-in.o drivers/usb/gadget/udc/built-in.o drivers/usb/host/built-in.o drivers/usb/musb-new/built-in.o drivers/usb/musb/built-in.o drivers/usb/phy/built-in.o drivers/usb/ulpi/built-in.o fs/built-in.o lib/built-in.o net/built-in.o test/built-in.o test/dm/built-in.o --end-group arch/arm/lib/eabi_compat.o -L /usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64-arm-linux-gnueabi-hf/bin/./lib/gcc/arm-linux-gnueabi-hf/4.9.4 -lgcc -Map u-boot.map arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -j .text -j .secure_text -j .rodata -j .hash -j .data -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -O binary u-boot u-boot-nodtb.bin
```

图 32.2.6.2 链接命令

使用 “-pie” 选项以后会生成一个 .rel.dyn 段, uboot 就是靠这个 .rel.dyn 来解决重定位问题的, 在 u-boot.dis 的 .rel.dyn 段中有如下所示内容:

示例代码 32.2.6.5 .rel.dyn 段代码段

```
1 Disassembly of section .rel.dyn:
2
3 8785da44 <__rel_dyn_end-0x8ba0>:
4 8785da44: 87800020 strhi r0, [r0, r0, lsr #32]
```



```

5 8785da48: 00000017    andeq    r0, r0, r7, lsl r0
6 .....
7 8785dfb4: 87804198      ; <UNDEFINED> instruction: 0x87804198
8 8785dfb8: 00000017    andeq    r0, r0, r7, lsl r0

```

先来看一下.rel.dyn 段的格式,类似第 7 行和第 8 行这样的是一组,也就是两个 4 字节数据为一组。高 4 字节是 Label 地址标识 0X17,低 4 字节就是 Label 的地址,首先判断 Label 地址标识是否正确,也就是判断高 4 字节是否为 0X17,如果是的话低 4 字节就是 Label 地址值。

第 7 行值为 0X87804198,第 8 行为 0X00000017,说明第 7 行的 0X87804198 是个 Label,这个正是示例代码 32.2.6.3 中存放变量 rel_a 地址的那个 Label。根据前面的分析,只要将地址 0X87804198+offset 处的值改为重定位后的变量 rel_a 地址即可。我们猜测的是否正确,看一下 uboot 对.rel.dyn 段的重定位即可(示例代码代码 32.2.6.1 中的第 94~109 行),.rel.dyn 段的重定位代码如下:

示例代码 32.2.6.6 relocate.S 代码段

```

91    /*
92    * fix .rel.dyn relocations
93    */
94    ldr r2, =__rel_dyn_start /* r2 <- SRC &__rel_dyn_start */
95    ldr r3, =__rel_dyn_end   /* r3 <- SRC &__rel_dyn_end */
96 fixloop:
97    ldmia r2!, {r0-r1}       /* (r0,r1) <- (SRC location,fixup) */
98    and r1, r1, #0xff
99    cmp r1, #23              /* relative fixup? */
100    bne fixnext
101
102    /* relative fix: increase location by offset */
103    add r0, r0, r4
104    ldr r1, [r0]
105    add r1, r1, r4
106    str r1, [r0]
107 fixnext:
108    cmp r2, r3
109    blo fixloop

```

第 94 行, r2=__rel_dyn_start, 也就是.rel.dyn 段的起始地址。

第 95 行, r3=__rel_dyn_end, 也就是.rel.dyn 段的终止地址。

第 97 行, 从.rel.dyn 段起始地址开始, 每次读取两个 4 字节的数据存放到 r0 和 r1 寄存器中, r0 存放低 4 字节的数据, 也就是 Label 地址; r1 存放高 4 字节的数据, 也就是 Label 标志。

第 98 行, r1 中给的值与 0xff 进行与运算, 其实就是取 r1 的低 8 位。

第 99 行, 判断 r1 中的值是否等于 23(0X17)。

第 100 行, 如果 r1 不等于 23 的话就说明不是描述 Label 的, 执行函数 fixnext, 否则的话继续执行下面的代码。

第 103 行, r0 保存着 Label 值, r4 保存着重定位后的地址偏移, r0+r4 就得到了重定位后的 Label 值。此时 r0 保存着重定位后的 Label 值, 相当于 0X87804198+0X18747000=0X9FF4B198。

第 104, 读取重定位后 Label 所保存的变量地址, 此时这个变量地址还是重定位前的(相当

于 `rel_a` 重定位前的地址 `0X8785DA50`), 将得到的值放到 `r1` 寄存器中。

第 105 行, `r1+r4` 即可得到重定位后的变量地址, 相当于 `rel_a` 重定位后的 `0X8785DA50+0X18747000=0X9FFA4A50`。

第 106 行, 重定位后的变量地址写入到重定位后的 `Label` 中, 相等于设置地址 `0X9FF4B198` 处的值为 `0X9FFA4A50`。

第 108 行, 比较 `r2` 和 `r3`, 查看 `.rel.dyn` 段重定位是否完成。

第 109 行, 如果 `r2` 和 `r3` 不相等, 说明 `.rel.dyn` 重定位还未完成, 因此跳到 `fixloop` 继续重定位 `.rel.dyn` 段。

可以看出, `uboot` 中对 `.rel.dyn` 段的重定位方法和我们猜想的一致。`.rel.dyn` 段的重定位比较复杂一点, 有点绕, 因为涉及到链接地址和运行地址的问题。

32.2.7 relocate_vectors 函数详解

函数 `relocate_vectors` 用于重定位向量表, 此函数定义在文件 `relocate.S` 中, 函数源码如下:

示例代码 32.2.7.1 `relocate.S` 代码段

```

27 ENTRY(relocate_vectors)
28
29 #ifdef CONFIG_CPU_V7M
30     /*
31      * On ARMv7-M we only have to write the new vector address
32      * to VTOR register.
33      */
34     ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
35     ldr r1, =V7M_SCB_BASE
36     str r0, [r1, V7M_SCB_VTOR]
37 #else
38 #ifdef CONFIG_HAS_VBAR
39     /*
40      * If the ARM processor has the security extensions,
41      * use VBAR to relocate the exception vectors.
42      */
43     ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
44     mcr p15, 0, r0, c12, c0, 0 /* Set VBAR */
45 #else
46     /*
47      * Copy the relocated exception vectors to the
48      * correct address
49      * CP15 c1 V bit gives us the location of the vectors:
50      * 0x00000000 or 0xFFFF0000.
51      */
52     ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
53     mrc p15, 0, r2, c1, c0, 0 /* V bit (bit[13]) in CP15 c1 */
54     ands r2, r2, #(1 << 13)
55     ldreq r1, =0x00000000 /* If V=0 */

```

```

56     ldrne    r1, =0xFFFF0000    /* If V=1    */
57     ldmia   r0!, {r2-r8,r10}
58     stmia   r1!, {r2-r8,r10}
59     ldmia   r0!, {r2-r8,r10}
60     stmia   r1!, {r2-r8,r10}
61 #endif
62 #endif
63     bx     lr
64
65 ENDPROC(relocate_vectors)

```

第 29 行, 如果定义了 CONFIG_CPU_V7M 的话就执行第 30~36 行的代码, 这是 Cortex-M 内核单片机执行的语句, 因此对于 LMX6ULL 来说是无效的。

第 38 行, 如果定义了 CONFIG_HAS_VBAR 的话就执行此语句, 这个是向量表偏移, Cortex-A7 是支持向量表偏移的。而且, 在 .config 里面定义了 CONFIG_HAS_VBAR, 因此会执行这个分支。

第 43 行, r0=gd->relocaddr, 也就是重定位后 uboot 的首地址, 向量表肯定是从这个地址开始存放的。

第 44 行, 将 r0 的值写入到 CP15 的 VBAR 寄存器中, 也就是将新的向量表首地址写入到寄存器 VBAR 中, 设置向量表偏移。

32.2.8 board_init_r 函数详解

第 32.2.5 小节讲解了 board_init_f 函数, 在此函数里面会调用一系列的函数来初始化一些外设和 gd 的成员变量。但是 board_init_f 并没有初始化所有的外设, 还需要做一些后续工作, 这些后续工作就是由函数 board_init_r 来完成的, board_init_r 函数定义在文件 common/board_r.c 中, 代码如下:

示例代码 32.2.8.1 board_r.c 代码段

```

991 void board_init_r(gd_t *new_gd, ulong dest_addr)
992 {
993     #ifdef CONFIG_NEEDS_MANUAL_RELOC
994         int i;
995     #endif
996
997     #ifdef CONFIG_AVR32
998         mmu_init_r(dest_addr);
999     #endif
1000
1001     #if !defined(CONFIG_X86) && !defined(CONFIG_ARM)
1002         && !defined(CONFIG_ARM64)
1003         gd = new_gd;
1004     #endif
1005     #ifdef CONFIG_NEEDS_MANUAL_RELOC
1006         for (i = 0; i < ARRAY_SIZE(init_sequence_r); i++)

```

```
1007     init_sequence_r[i] += gd->reloc_off;
1008 #endif
1009
1010     if (initcall_run_list(init_sequence_r))
1011         hang();
1012
1013     /* NOTREACHED - run_main_loop() does not return */
1014     hang();
1015 }
```

第 1010 行调用 `initcall_run_list` 函数来执行初始化序列 `init_sequence_r`, `init_sequence_r` 是一个函数集合, `init_sequence_r` 也定义在文件 `common/board_r.c` 中, 由于 `init_sequence_f` 的内容比较长, 里面有大量的条件编译代码, 这里为了缩小篇幅, 将条件编译部分删除掉了, 去掉条件编译以后的 `init_sequence_r` 定义如下:

示例代码 32.2.8.2 board_r.c 代码段

```
1  init_fnc_t init_sequence_r[] = {
2      initr_trace,
3      initr_reloc,
4      initr_caches,
5      initr_reloc_global_data,
6      initr_barrier,
7      initr_malloc,
8      initr_console_record,
9      bootstage_relocate,
10     initr_bootstage,
11     board_init, /* Setup chipselects */
12     stdio_init_tables,
13     initr_serial,
14     initr_announce,
15     INIT_FUNC_WATCHDOG_RESET
16     INIT_FUNC_WATCHDOG_RESET
17     INIT_FUNC_WATCHDOG_RESET
18     power_init_board,
19     initr_flash,
20     INIT_FUNC_WATCHDOG_RESET
21     initr_nand,
22     initr_mmc,
23     initr_env,
24     INIT_FUNC_WATCHDOG_RESET
25     initr_secondary_cpu,
26     INIT_FUNC_WATCHDOG_RESET
27     stdio_add_devices,
28     initr_jumptable,
29     console_init_r, /* fully init console as a device */
```

```

30     INIT_FUNC_WATCHDOG_RESET
31     interrupt_init,
32     initr_enable_interrupts,
33     initr_ethaddr,
34     board_late_init,
35     INIT_FUNC_WATCHDOG_RESET
36     INIT_FUNC_WATCHDOG_RESET
37     INIT_FUNC_WATCHDOG_RESET
38     initr_net,
39     INIT_FUNC_WATCHDOG_RESET
40     run_main_loop,
41 };

```

第 2 行, `initr_trace` 函数, 如果定义了宏 `CONFIG_TRACE` 的话就会调用函数 `trace_init`, 初始化和调试跟踪有关的内容。

第 3 行, `initr_reloc` 函数用于设置 `gd->flags`, 标记重定位完成。

第 4 行, `initr_caches` 函数用于初始化 `cache`, 使能 `cache`。

第 5 行, `initr_reloc_global_data` 函数, 初始化重定位后 `gd` 的一些成员变量。

第 6 行, `initr_barrier` 函数, I.MX6ULL 未用到。

第 7 行, `initr_malloc` 函数, 初始化 `malloc`。

第 8 行, `initr_console_record` 函数, 初始化控制台相关的内容, I.MX6ULL 未用到, 空函数。

第 9 行, `bootstage_relocate` 函数, 启动状态重定位。

第 10 行, `initr_bootstage` 函数, 初始化 `bootstage` 什么的。

第 11 行, `board_init` 函数, 板级初始化, 包括 74XX 芯片, I2C、FEC、USB 和 QSPI 等。这里执行的是 `mx6ull_alientek_emmc.c` 文件中的 `board_init` 函数。

第 12 行, `stdio_init_tables` 函数, `stdio` 相关初始化。

第 13 行, `initr_serial` 函数, 初始化串口。

第 14 行, `initr_announce` 函数, 与调试有关, 通知已经在 RAM 中运行。

第 18 行, `power_init_board` 函数, 初始化电源芯片, 正点原子的 I.MX6ULL 开发板没有用到。

第 19 行, `initr_flash` 函数, 对于 I.MX6ULL 而言, 没有定义宏 `CONFIG_SYS_NO_FLASH` 的话函数 `initr_flash` 才有效。但是 `mx6_common.h` 中定义了宏 `CONFIG_SYS_NO_FLASH`, 所以此函数无效。

第 21 行, `initr_nand` 函数, 初始化 NAND, 如果使用 NAND 版本核心板的话就会初始化 NAND。

第 22 行, `initr_mmc` 函数, 初始化 EMMC, 如果使用 EMMC 版本核心板的话就会初始化 EMMC, 串口输出如图 32.2.8.1 所示信息:

```

512 MiB
MMC: FSL_SDHC: 0, FSL_SDHC: 1

```

图 32.2.8.1 EMMC 信息输出

从图 32.2.8.1 可以看出, 此时有两个 EMMC 设备, `FSL_SDHC:0` 和 `FSL_SDHC:1`。

第 23 行, `initr_env` 函数, 初始化环境变量。

第 25 行, `initr_secondary_cpu` 函数, 初始化其他 CPU 核, I.MX6ULL 只有一个核, 因此此函数没用。

第 27 行, `stdio_add_devices` 函数, 各种输入输出设备的初始化, 如 LCD driver, I.MX6ULL 使用 `drv_video_init` 函数初始化 LCD。会输出如图 32.2.8.2 所示信息:

```
Display: ATK-LCD-7-1024x600 (1024x600)
Video: 1024x600x24
```

图 32.2.8.2 LCD 信息

第 28 行, `initr_jumptable` 函数, 初始化跳转表。

第 29 行, `console_init_r` 函数, 控制台初始化, 初始化完成以后此函数会调用 `stdio_print_current_devices` 函数来打印出当前的控制台设备, 如图 32.2.8.3 所示:

```
In:    serial
Out:   serial
Err:   serial
```

图 32.2.8.3 控制台信息

第 31 行, `interrupt_init` 函数, 初始化中断。

第 32 行, `initr_enable_interrupts` 函数, 使能中断。

第 33 行, `initr_ethaddr` 函数, 初始化网络地址, 也就是获取 MAC 地址。读取环境变量 “ethaddr” 的值。

第 34 行, `board_late_init` 函数, 板子后续初始化, 此函数定义在文件 `mx6ull_alientek_emmc.c` 中, 如果环境变量存储在 EMMC 或者 SD 卡中的话此函数会调用 `board_late_mmc_env_init` 函数初始化 EMMC/SD。会切换到正在时候用的 emmc 设备, 代码如图 32.2.8.4 所示:

```
30 void board_late_mmc_env_init(void)
31 {
32     char cmd[32];
33     char mmcblk[32];
34     u32 dev_no = mmc_get_env_dev();
35
36     if (!check_mmc_autodetect())
37         return;
38
39     setenv_ulong("mmcdev", dev_no);
40
41     /* Set mmcblk env */
42     sprintf(mmcblk, "/dev/mmcblk%dp2 rootwait rw",
43         mmc_map_to_kernel_blk(dev_no));
44     setenv("mmcroot", mmcblk);
45
46     sprintf(cmd, "mmc dev %d", dev_no);
47     run_command(cmd, 0);
48 }
```

图 32.2.8.4 board_late_mmc_env_init 函数

图 32.2.8.4 中的第 46 行和第 47 行就是运行 “mmc dev xx” 命令, 用于切换到正在使用的 EMMC 设备, 串口输出信息如图 32.2.8.5 所示:

```
switch to partitions #0, OK
mmc1(part 0) is current device
```

图 32.2.8.5 切换 mmc 设备

第 38 行, `initr_net` 函数, 初始化网络设备, 函数调用顺序为:

initr_net->eth_initialize->board_eth_init(), 串口输出如图 32.2.8.6 所示信息:

Net: FEC1

图 32.2.8.6 网络信息输出

第 40 行, run_main_loop 行, 主循环, 处理命令。

32.2.9 run_main_loop 函数详解

uboot 启动以后会进入 3 秒倒计时, 如果在 3 秒倒计时结束之前按下回车键, 那么就会进入 uboot 的命令模式, 如果倒计时结束以后都没有按下回车键, 那么就会自动启动 Linux 内核, 这个功能就是由 run_main_loop 函数来完成的。run_main_loop 函数定义在文件 common/board_r.c 中, 函数内容如下:

示例代码 32.2.9.1 board_r.c 文件代码段

```
753 static int run_main_loop(void)
754 {
755     #ifdef CONFIG_SANDBOX
756     sandbox_main_loop_init();
757     #endif
758     /* main_loop() can return to retry autoboot, if so just run it
again */
759     for (;;)
760         main_loop();
761     return 0;
762 }
```

第 759 行和第 760 行是个死循环, “for(;;)” 和 “while(1)” 功能一样, 死循环里面就一个 main_loop 函数, main_loop 函数定义在文件 common/main.c 里面, 代码如下:

示例代码 32.2.9.2 main.c 文件代码段

```
43 /* We come here after U-Boot is initialised and ready to process
commands */
44 void main_loop(void)
45 {
46     const char *s;
47
48     bootstage_mark_name(BOOTSTAGE_ID_MAIN_LOOP, "main_loop");
49
50 #ifndef CONFIG_SYS_GENERIC_BOARD
51     puts("Warning: Your board does not use generic board. Please
read\n");
52     puts("doc/README.generic-board and take action. Boards not\n");
53     puts("upgraded by the late 2014 may break or be removed.\n");
54 #endif
55
56 #ifdef CONFIG_VERSION_VARIABLE
57     setenv("ver", version_string); /* set version variable */
58 #endif /* CONFIG_VERSION_VARIABLE */
```

```

59
60     cli_init();
61
62     run_preboot_environment_command();
63
64 #if defined(CONFIG_UPDATE_TFTP)
65     update_tftp(0UL, NULL, NULL);
66 #endif /* CONFIG_UPDATE_TFTP */
67
68     s = bootdelay_process();
69     if (cli_process_fdt(&s))
70         cli_secure_boot_cmd(s);
71
72     autoboot_command(s);
73
74     cli_loop();
75 }

```

第 48 行, 调用 `bootstage_mark_name` 函数, 打印出启动进度。

第 57 行, 如果定义了宏 `CONFIG_VERSION_VARIABLE` 的话就会执行函数 `setenv`, 设置换将变量 `ver` 的值为 `version_string`, 也就是设置版本号环境变量。`version_string` 定义在文件 `cmd/version.c` 中, 定义如下:

```
const char __weak version_string[] = U_BOOT_VERSION_STRING;
```

`U_BOOT_VERSION_STRING` 是个宏, 定义在文件 `include/version.h`, 如下:

```
#define U_BOOT_VERSION_STRING U_BOOT_VERSION "(" U_BOOT_DATE " - " \
    U_BOOT_TIME " " U_BOOT_TZ ")" CONFIG_IDENT_STRING
```

`U_BOOT_VERSION` 定义在文件 `include/generated/version_autogenerated.h` 中, 文件 `version_autogenerated.h` 内如如下:

示例代码 32.2.9.4 `version_autogenerated.h` 文件代码

```

1 #define PLAIN_VERSION "2016.03"
2 #define U_BOOT_VERSION "U-Boot " PLAIN_VERSION
3 #define CC_VERSION_STRING "arm-linux-gnueabihf-gcc (Linaro GCC 4.9-
2017.01) 4.9.4"
4 #define LD_VERSION_STRING "GNU ld (Linaro_Binutils-2017.01)
2.24.0.20141017 Linaro 2014_11-3-git"

```

可以看出, `U_BOOT_VERSION` 为 “U-boot 2016.03”,
`U_BOOT_DATE`、`U_BOOT_TIME` 和 `U_BOOT_TZ` 这 定 义 在 文 件
`include/generated/timestamp_autogenerated.h` 中, 如下所示:

示例代码 32.2.9.5 `timestamp_autogenerated.h` 文件代码

```

1 #define U_BOOT_DATE "Apr 25 2019"
2 #define U_BOOT_TIME "21:10:53"
3 #define U_BOOT_TZ "+0800"
4 #define U_BOOT_DMI_DATE "04/25/2019"

```


宏 CONFIG_IDENT_STRING 为空, 所以 U_BOOT_VERSION_STRING 为“U-Boot 2016.03 (Apr 25 2019 - 21:10:53 +0800)”, 进入 uboot 命令模式, 输入命令“version”查看版本号, 如图 32.2.9.1 所示:

```
U-Boot 2016.03 (Apr 25 2019 - 21:10:53 +0800)
arm-linux-gnueabi-gcc (Linaro GCC 4.9-2017.01) 4.9.4
GNU ld (Linaro Binutils-2017.01) 2.24.0.20141017 Linaro 2014_11-3-git
=>
```

图 32.2.9.1 版本查询

图 32.2.9.1 中的第一行就是 uboot 版本号, 和我们分析的一致。

接着回到示例代码 32.2.9.2 中, 第 60 行, cli_init 函数, 跟命令初始化有关, 初始化 hush shell 相关的变量。

第 62 行, run_preboot_environment_command 函数, 获取环境变量 perboot 的内容, perboot 是一些预启动命令, 一般不使用这个环境变量。

第 68 行, bootdelay_process 函数, 此函数会读取环境变量 bootdelay 和 bootcmd 的内容, 然后将 bootdelay 的值赋值给全局变量 stored_bootdelay, 返回值为环境变量 bootcmd 的值。

第 69 行, 如果定义了 CONFIG_OF_CONTROL 的话函数 cli_process_fdt 就会实现, 如果没有定义 CONFIG_OF_CONTROL 的话函数 cli_process_fdt 直接返回一个 false。在本 uboot 中没有定义 CONFIG_OF_CONTROL, 因此 cli_process_fdt 函数返回值为 false。

第 72 行, autoboot_command 函数, 此函数就是检查倒计时是否结束? 倒计时结束之前有没有被打断? 此函数定义在文件 common/autoboot.c 中, 内容如下:

示例代码 32.2.9.5 auboboot.c 文件代码段

```
380 void autoboot_command(const char *s)
381 {
382     debug("### main_loop: bootcmd=\"%s\"\n", s ? s : "<UNDEFINED>");
383
384     if (stored_bootdelay != -1 && s && !abortboot(stored_bootdelay))
385     {
386         #if defined(CONFIG_AUTOBOOT_KEYED)
387         && !defined(CONFIG_AUTOBOOT_KEYED_CTRLIC)
388         int prev = disable_ctrlc(1);    /* disable Control C checking */
389     #endif
390
391     run_command_list(s, -1, 0);
392
393     #if defined(CONFIG_AUTOBOOT_KEYED)
394     && !defined(CONFIG_AUTOBOOT_KEYED_CTRLIC)
395     disable_ctrlc(prev);    /* restore Control C checking */
396     #endif
397 }
398
399 #ifdef CONFIG_MENUKEY
400     if (menukey == CONFIG_MENUKEY) {
401         s = getenv("menucmd");
402     }
403 }
```

```

399         if (s)
400             run_command_list(s, -1, 0);
401     }
402 #endif /* CONFIG_MENUKEY */
403 }

```

可以看出, `autoboot_command` 函数里面有很多条件编译, 条件编译一多就不利于我们阅读程序 (所以正点原子的例程基本是不用条件编译的, 就是为了方便大家阅读源码)! 宏 `CONFIG_AUTOBOOT_KEYED`、`CONFIG_AUTOBOOT_KEYED_CTRL` 和 `CONFIG_MENUKEY` 这三个宏在 I.MX6ULL 里面没有定义, 所以讲示例代码 32.2.9.5 进行精简, 得到如下代码:

示例代码 32.2.9.6 `autoboot_command` 函数精简版本

```

1 void autoboot_command(const char *s)
2 {
3     if (stored_bootdelay != -1 && s && !abortboot(stored_bootdelay)) {
4         run_command_list(s, -1, 0);
5     }
6 }

```

当一下三条全部成立的话, 就会执行函数 `run_command_list`。

- ①、`stored_bootdelay` 不等于 -1。
- ②、`s` 不为空。
- ③、函数 `abortboot` 返回值为 0。

`stored_bootdelay` 等于环境变量 `bootdelay` 的值; `s` 是环境变量 `bootcmd` 的值, 一般不为空, 因此前两个成立, 就剩下了函数 `abortboot` 的返回值, `abortboot` 函数也定义在文件 `common/autoboot.c` 中, 内容如下:

示例代码 32.2.9.7 `abortboot` 函数

```

283 static int abortboot(int bootdelay)
284 {
285 #ifdef CONFIG_AUTOBOOT_KEYED
286     return abortboot_keyed(bootdelay);
287 #else
288     return abortboot_normal(bootdelay);
289 #endif
290 }

```

因为宏 `CONFIG_AUTOBOOT_KEYED` 未定义, 因此执行函数 `abortboot_normal`, 好吧, 绕来绕去的! 接着来看函数 `abortboot_normal`, 此函数也定义在文件 `common/autoboot.c` 中, 内容如下:

示例代码 32.2.9.8 `abortboot_normal` 函数

```

225 static int abortboot_normal(int bootdelay)
226 {
227     int abort = 0;
228     unsigned long ts;
229
230 #ifdef CONFIG_MENU_PROMPT

```

```
231     printf(CONFIG_MENUUPROMPT);
232 #else
233     if (bootdelay >= 0)
234         printf("Hit any key to stop autoboot: %2d ", bootdelay);
235 #endif
236
237 #if defined CONFIG_ZERO_BOOTDELAY_CHECK
238     /*
239      * Check if key already pressed
240      * Don't check if bootdelay < 0
241      */
242     if (bootdelay >= 0) {
243         if (tstc()) { /* we got a key press */
244             (void) getc(); /* consume input */
245             puts("\b\b\b 0");
246             abort = 1; /* don't auto boot */
247         }
248     }
249 #endif
250
251     while ((bootdelay > 0) && (!abort)) {
252         --bootdelay;
253         /* delay 1000 ms */
254         ts = get_timer(0);
255         do {
256             if (tstc()) { /* we got a key press */
257                 abort = 1; /* don't auto boot */
258                 bootdelay = 0; /* no more delay */
259 #ifdef CONFIG_MENUKEY
260                 menukey = getc();
261 #else
262                 (void) getc(); /* consume input */
263 #endif
264                 break;
265             }
266             udelay(10000);
267         } while (!abort && get_timer(ts) < 1000);
268
269         printf("\b\b\b%2d ", bootdelay);
270     }
271
272     putc('\n');
273 }
```

```

274 #ifdef CONFIG_SILENT_CONSOLE
275     if (abort)
276         gd->flags &= ~GD_FLG_SILENT;
277 #endif
278
279     return abort;
280 }

```

函数 `abortboot_normal` 同样很多条件编译, 删除掉条件编译相关代码后 `abortboot_normal` 函数内容如下:

示例代码 32.2.9.9 `abortboot_normal` 函数精简

```

1 static int abortboot_normal(int bootdelay)
2 {
3     int abort = 0;
4     unsigned long ts;
5
6     if (bootdelay >= 0)
7         printf("Hit any key to stop autoboot: %2d ", bootdelay);
8
9     while ((bootdelay > 0) && (!abort)) {
10         --bootdelay;
11         /* delay 1000 ms */
12         ts = get_timer(0);
13         do {
14             if (tstc()) { /* we got a key press */
15                 abort = 1; /* don't auto boot */
16                 bootdelay = 0; /* no more delay */
17                 (void) getc(); /* consume input */
18                 break;
19             }
20             udelay(10000);
21         } while (!abort && get_timer(ts) < 1000);
22
23         printf("\b\b\b%2d ", bootdelay);
24     }
25     putc('\n');
26     return abort;
27 }

```

第 3 行的变量 `abort` 是函数 `abortboot_normal` 的返回值, 默认值为 0。

第 7 行通过串口输出 “Hit any key to stop autoboot” 字样, 如图 32.2.9.2 所示:

Hit any key to stop autoboot: 0

图 32.2.9.2 倒计时

第 9~21 行就是倒计时的具体实现。

第 14 行判断键盘是否有按下,也就是是否打断了倒计时,如果键盘按下的话就执行相应的分支。比如设置 abort 为 1, 设置 bootdelay 为 0 等,最后跳出倒计时循环。

第 26 行,返回 abort 的值,如果倒计时自然结束,没有被打断 abort 就为 0,否则的话 abort 的值就为 1。

回到示例代码 32.2.9.6 的 autoboot_command 函数中,如果倒计时自然结束那么就执行函数 run_command_list,此函数会执行参数 s 指定的一系列命令,也就是环境变量 bootcmd 的命令,bootcmd 里面保存着默认的启动命令,因此 linux 内核启动!这个就是 uboot 中倒计时结束以后自动启动 linux 内核的原理。如果倒计时结束之前按下了键盘上的按键,那么 run_command_list 函数就不会执行,相当于 autoboot_command 是个空函数。

回到“遥远”的示例代码 32.2.9.2 中的 main_loop 函数中,如果倒计时结束之前按下按键,那么就会执行第 74 行的 cli_loop 函数,这个就是命令处理函数,负责接收好处理输入的命令。

32.2.10 cli_loop 函数详解

cli_loop 函数是 uboot 的命令行处理函数,我们在 uboot 中输入各种命令,进行各种操作就是有 cli_loop 来处理的,此函数定义在文件 common/cli.c 中,函数内容如下:

示例代码 32.2.10.1 cli.c 文件代码段

```
202 void cli_loop(void)
203 {
204 #ifdef CONFIG_SYS_HUSH_PARSER
205     parse_file_outer();
206     /* This point is never reached */
207     for (;;) ;
208 #else
209     cli_simple_loop();
210 #endif /*CONFIG_SYS_HUSH_PARSER*/
211 }
```

在文件 include/configs/mx6_common.h 中有定义宏 CONFIG_SYS_HUSH_PARSER,而正点原子的 I.MX6ULL 开发板配置头文件 mx6ullevk.h 里面会引用 mx_common.h 这个头文件,因此宏 CONFIG_SYS_HUSH_PARSER 有定义。

第 205 行调用函数 parse_file_outer。

第 207 行是个死循环,永远不会执行到这里。

函数 parse_file_outer 定义在文件 common/cli_hush.c 中,去掉条件编译内容以后的函数内容如下:

示例代码 32.2.10.2 parse_file_outer 函数精简

```
1 int parse_file_outer(void)
2 {
3     int rcode;
4     struct in_str input;
5
6     setup_file_in_str(&input);
7     rcode = parse_stream_outer(&input, FLAG_PARSE_SEMICOLON);
8     return rcode;
9 }
```

第 6 行调用函数 `setup_file_in_str` 初始化变量 `input` 的成员变量。

第 7 行调用函数 `parse_stream_outer`, 这个函数就是 `hush shell` 的命令解释器, 负责接收命令行输入, 然后解析并执行相应的命令, 函数 `parse_stream_outer` 定义在文件 `common/cli_hush.c` 中, 精简版的函数内容如下:

示例代码 32.2.10.3 `parse_stream_outer` 函数精简

```
1 static int parse_stream_outer(struct in_str *inp, int flag)
2 {
3     struct p_context ctx;
4     o_string temp=NULL_O_STRING;
5     int rcode;
6     int code = 1;
7     do {
8         .....
9         rcode = parse_stream(&temp, &ctx, inp,
10                             flag & FLAG_CONT_ON_NEWLINE ? -1 : '\n');
11         .....
12         if (rcode != 1 && ctx.old_flag == 0) {
13             .....
14             run_list(ctx.list_head);
15             .....
16         } else {
17             .....
18         }
19         b_free(&temp);
20         /* loop on syntax errors, return on EOF */
21     } while (rcode != -1 && !(flag & FLAG_EXIT_FROM_LOOP) &&
22             (inp->peek != static_peek || b_peek(inp)));
23     return 0;
24 }
```

第 7~21 行中的 `do-while` 循环就是处理输入命令的。

第 9 行调用函数 `parse_stream` 进行命令解析。

第 14 行调用 `run_list` 函数来执行解析出来的命令。

函数 `run_list` 会经过一系列的函数调用, 最终通过调用 `cmd_process` 函数来处理命令, 过程如下:

示例代码 32.2.10.4 `run_list` 执行流程

```
1 static int run_list(struct pipe *pi)
2 {
3     int rcode=0;
4
5     rcode = run_list_real(pi);
6     .....
7     return rcode;
8 }
```

```

9
10 static int run_list_real(struct pipe *pi)
11 {
12     char *save_name = NULL;
13     .....
14     int if_code=0, next_if_code=0;
15     .....
16     rcode = run_pipe_real(pi);
17     .....
18     return rcode;
19 }
20
21 static int run_pipe_real(struct pipe *pi)
22 {
23     int i;
24
25     int nextin;
26     int flag = do_repeat ? CMD_FLAG_REPEAT : 0;
27     struct child_prog *child;
28     char *p;
29     .....
30     if (pi->num_progs == 1) child = & (pi->progs[0]);
31     .....
32     return rcode;
33 } else if (pi->num_progs == 1 && pi->progs[0].argv != NULL) {
34     .....
35     /* Process the command */
36     return cmd_process(flag, child->argc, child->argv,
37                       &flag_repeat, NULL);
38 }
39
40 return -1;
41 }

```

第 5 行, run_list 调用 run_list_real 函数。

第 16 行, run_list_real 函数调用 run_pipe_real 函数。

第 36 行, run_pipe_real 函数调用 cmd_process 函数。

最终通过函数 cmd_process 来处理命令, 接下来就是分析 cmd_process 函数。

32.2.11 cmd_process 函数详解

在学习 cmd_process 之前先看一下 uboot 中命令是如何定义的。uboot 使用宏 U_BOOT_CMD 来定义命令, 宏 U_BOOT_CMD 定义在文件 include/command.h 中, 定义如下:

示例代码 32.2.11.1 U_BOOT_CMD 宏定义

```
#define U_BOOT_CMD(_name, _maxargs, _rep, _cmd, _usage, _help) \
```



```
U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help,
NULL)
```

可以看出 U_BOOT_CMD 是 U_BOOT_CMD_COMPLETE 的特例，将 U_BOOT_CMD_COMPLETE 的最后一个参数设置成 NULL 就是 U_BOOT_CMD。宏 U_BOOT_CMD_COMPLETE 如下：

示例代码 32.2.11.2 U_BOOT_CMD_COMPLETE 宏定义

```
#define U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help,
_comp) \
    ll_entry_declare(cmd_tbl_t, _name, cmd) = \
        U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd, \
            _usage, _help, _comp);
```

宏 U_BOOT_CMD_COMPLETE 又用到了 ll_entry_declare 和 U_BOOT_CMD_MKENT_COMPLETE。ll_entry_declare 定义在文件 include/linker_lists.h 中，定义如下：

示例代码 32.2.11.3 ll_entry_declare 宏定义

```
#define ll_entry_declare(_type, _name, _list) \
    _type _u_boot_list_2_##_list##_2_##_name __aligned(4) \
        __attribute__((unused, \
            section(".u_boot_list_2_"#_list"2_"#_name)))
```

_type 为 cmd_tbl_t，因此 ll_entry_declare 就是定义了一个 cmd_tbl_t 变量，这里用到了 C 语言中的“##”连接符。其中的“##_list”表示用 _list 的值来替换，“##_name”就是用 _name 的值来替换。

宏 U_BOOT_CMD_MKENT_COMPLETE 定义在文件 include/command.h 中，内容如下：

示例代码 32.2.11.4 U_BOOT_CMD_MKENT_COMPLETE 宏定义

```
#define U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd, \
    _usage, _help, _comp) \
    { #_name, _maxargs, _rep, _cmd, _usage, \
        _CMD_HELP(_help) _CMD_COMPLETE(_comp) }
```

上述代码中的“#”表示将 _name 传递过来的值字符串化，U_BOOT_CMD_MKENT_COMPLETE 又用到了宏 _CMD_HELP 和 _CMD_COMPLETE，这两个宏的定义如下：

示例代码 32.2.11.5 _CMD_HELP 和 _CMD_COMPLETE 宏定义

```
1 #ifndef CONFIG_AUTO_COMPLETE
2 # define _CMD_COMPLETE(x) x,
3 #else
4 # define _CMD_COMPLETE(x)
5 #endif
6 #ifndef CONFIG_SYS_LONGHELP
7 # define _CMD_HELP(x) x,
8 #else
9 # define _CMD_HELP(x)
10 #endif
```

可以看出，如果定义了宏 CONFIG_AUTO_COMPLETE 和 CONFIG_SYS_LONGHELP 的

话, `_CMD_COMPLETE` 和 `_CMD_HELP` 就是取自自身的值, 然后在加上一个 ‘,’。
`CONFIG_AUTO_COMPLETE` 和 `CONFIG_SYS_LONGHELP` 这两个宏有定义在文件 `mx6_common.h` 中。

`U_BOOT_CMD` 宏的流程我们已经清楚了(一个 `U_BOOT_CMD` 宏就如此的绕来绕去的!), 我们就以一个具体的命令为例, 来看一下 `U_BOOT_CMD` 经过展开以后究竟是个什么模样的。以命令 `dhcp` 为例, `dhcp` 命令定义如下:

示例代码 32.2.11.6 dhcp 命令宏定义

```
U_BOOT_CMD(
    dhcp, 3, 1, do_dhcp,
    "boot image via network using DHCP/TFTP protocol",
    "[loadAddress] [[hostIPAddr:]bootfilename]"
);
```

将其展开, 结果如下:

示例代码 32.2.11.7 dhcp 命令展开

```
U_BOOT_CMD(
    dhcp, 3, 1, do_dhcp,
    "boot image via network using DHCP/TFTP protocol",
    "[loadAddress] [[hostIPAddr:]bootfilename]"
);
```

1、将 `U_BOOT_CMD` 展开后为:

```
U_BOOT_CMD_COMPLETE(dhcp, 3, 1, do_dhcp,
    "boot image via network using DHCP/TFTP protocol",
    "[loadAddress] [[hostIPAddr:]bootfilename]",
    NULL)
```

2、将 `U_BOOT_CMD_COMPLETE` 展开后为:

```
ll_entry_declare(cmd_tbl_t, dhcp, cmd) = \
U_BOOT_CMD_MKENT_COMPLETE(dhcp, 3, 1, do_dhcp, \
    "boot image via network using DHCP/TFTP protocol", \
    "[loadAddress] [[hostIPAddr:]bootfilename]", \
    NULL);
```

3、将 `ll_entry_declare` 和 `U_BOOT_CMD_MKENT_COMPLETE` 展开后为:

```
cmd_tbl_t _u_boot_list_2_cmd_2_dhcp __aligned(4) \
    __attribute__((unused, section(.u_boot_list_2_cmd_2_dhcp))) \
{ "dhcp", 3, 1, do_dhcp, \
    "boot image via network using DHCP/TFTP protocol", \
    "[loadAddress] [[hostIPAddr:]bootfilename]", \
    NULL}
```

从示例代码 32.2.11.7 可以看出, `dhcp` 命令最终展开结果为:

示例代码 32.2.11.8 dhcp 命令最终结果

```
1 cmd_tbl_t _u_boot_list_2_cmd_2_dhcp __aligned(4) \
```

```

2     __attribute__((unused,section(.u_boot_list_2_cmd_2_dhcp))) \
3     { "dhcp", 3, 1, do_dhcp, \
4       "boot image via network using DHCP/TFTP protocol", \
5       "[loadAddress] [[hostIPAddr:]bootfilename]",\
6       NULL}

```

第 1 行定义了一个 `cmd_tbl_t` 类型的变量，变量名为 `_u_boot_list_2_cmd_2_dhcp`，此变量 4 字节对齐。

第 2 行，使用 `__attribute__` 关键字设置变量 `_u_boot_list_2_cmd_2_dhcp` 存储在 `.u_boot_list_2_cmd_2_dhcp` 段中。`u-boot.lds` 链接脚本中有一个名为“`.u_boot_list`”的段，所有 `.u_boot_list` 开头的段都存放到 `.u_boot.list` 中，如图 32.2.11.1 所示：

```

21 | . = ALIGN(4);
22 | . = .;
23 | . = ALIGN(4);
24 | .u_boot_list : {
25 |     KEEP(*(SORT(.u_boot_list*)));
26 | }

```

图 32.2.11.1 u-boot.lds 中的 `.u_boot_list` 段

因此，第 2 行就是设置变量 `_u_boot_list_2_cmd_2_dhcp` 的存储位置。

第 3~6 行，`cmd_tbl_t` 是个结构体，因此第 3-6 行是初始化 `cmd_tbl_t` 这个结构体的各个成员变量。`cmd_tbl_t` 结构体定义在文件 `include/command.h` 中，内容如下：

示例代码 32.2.11.9 `cmd_tbl_t` 结构体

```

30 struct cmd_tbl_s {
31     char      *name;           /* Command Name                */
32     int       maxargs;         /* maximum number of arguments */
33     int       repeatable;      /* autorepeat allowed?         */
34     /* Implementation function */
35     int       (*cmd)(struct cmd_tbl_s *, int, int, char * const []);
36     char      *usage;          /* Usage message (short)       */
37 #ifdef CONFIG_SYS_LONGHELP
38     char      *help;           /* Help message (long)         */
39 #endif
40 #ifdef CONFIG_AUTO_COMPLETE
41     /* do auto completion on the arguments */
42     int       (*complete)(int argc, char * const argv[], char
last_char, int maxv, char *cmdv[]);
43 #endif
44 };
45
46 typedef struct cmd_tbl_s cmd_tbl_t;

```

结合实例代码 32.2.11.8，可以得出变量 `_u_boot_list_2_cmd_2_dhcp` 的各个成员的值如下所示：

```

_u_boot_list_2_cmd_2_dhcp.name = "dhcp"
_u_boot_list_2_cmd_2_dhcp.maxargs = 3
_u_boot_list_2_cmd_2_dhcp.repeatable = 1

```

```

_u_boot_list_2_cmd_2_dhcp.cmd = do_dhcp
_u_boot_list_2_cmd_2_dhcp.usage = "boot image via network using DHCP/TFTP protocol"
_u_boot_list_2_cmd_2_dhcp.help = "[loadAddress] [[hostIPAddr:]bootfilename]"
_u_boot_list_2_cmd_2_dhcp.complete = NULL

```

当我们在 uboot 的命令行中输入“dhcp”这个命令的时候，最终执行的是 do_dhcp 这个函数。总结一下，uboot 中使用 U_BOOT_CMD 来定义一个命令，最终的目的就是为了定义一个 cmd_tbl_t 类型的变量，并初始化这个变量的各个成员。uboot 中的每个命令都存储在 .u_boot_list 段中，每个命令都有一个名为 do_xxx(xxx 为具体的命令名)的函数，这个 do_xxx 函数就是具体的命令处理函数。

了解了 uboot 中命令的组成以后，再来看一下 cmd_process 函数的处理过程，cmd_process 函数定义在文件 common/command.c 中，函数内容如下：

示例代码 32.2.11.10 command.c 文件代码段

```

500 enum command_ret_t cmd_process(int flag, int argc,
501                                char * const argv[], int *repeatable, ulong *ticks)
502 {
503     enum command_ret_t rc = CMD_RET_SUCCESS;
504     cmd_tbl_t *cmdtp;
505
506     /* Look up command in command table */
507     cmdtp = find_cmd(argv[0]);
508     if (cmdtp == NULL) {
509         printf("Unknown command '%s' - try 'help'\n", argv[0]);
510         return 1;
511     }
512
513     /* found - check max args */
514     if (argc > cmdtp->maxargs)
515         rc = CMD_RET_USAGE;
516
517 #if defined(CONFIG_CMD_BOOTD)
518     /* avoid "bootd" recursion */
519     else if (cmdtp->cmd == do_bootd) {
520         if (flag & CMD_FLAG_BOOTD) {
521             puts("'bootd' recursion detected\n");
522             rc = CMD_RET_FAILURE;
523         } else {
524             flag |= CMD_FLAG_BOOTD;
525         }
526     }
527 #endif
528
529     /* If OK so far, then do the command */
530     if (!rc) {

```

```

531     if (ticks)
532         *ticks = get_timer(0);
533     rc = cmd_call(cmdtp, flag, argc, argv);
534     if (ticks)
535         *ticks = get_timer(*ticks);
536     *repeatable &= cmdtp->repeatable;
537 }
538 if (rc == CMD_RET_USAGE)
539     rc = cmd_usage(cmdtp);
540 return rc;
541 }

```

第 507 行, 调用函数 `find_cmd` 在命令表中找到指定的命令, `find_cmd` 函数内容如下:

示例代码 32.2.11.11 `command.c` 文件代码段

```

118 cmd_tbl_t *find_cmd(const char *cmd)
119 {
120     cmd_tbl_t *start = ll_entry_start(cmd_tbl_t, cmd);
121     const int len = ll_entry_count(cmd_tbl_t, cmd);
122     return find_cmd_tbl(cmd, start, len);
123 }

```

参数 `cmd` 就是所查找的命令名字, `uboot` 中的命令表其实就是 `cmd_tbl_t` 结构体数组, 通过函数 `ll_entry_start` 得到数组的第一个元素, 也就是命令表起始地址。通过函数 `ll_entry_count` 得到数组长度, 也就是命令表的长度。最终通过函数 `find_cmd_tbl` 在命令表中找到所需的命令, 每个命令都有一个 `name` 成员, 所以将参数 `cmd` 与命令表中每个成员的 `name` 字段都对比一下, 如果相等的话就说明找到了这个命令, 找到以后就返回这个命令。

回到示例代码 32.2.11.10 的 `cmd_process` 函数中, 找到命令以后肯定就要执行这个命令了, 第 533 行调用函数 `cmd_call` 来执行具体的命令, `cmd_call` 函数内容如下:

示例代码 32.2.11.12 `command.c` 文件代码段

```

490 static int cmd_call(cmd_tbl_t *cmdtp, int flag, int argc, char *
const argv[])
491 {
492     int result;
493
494     result = (cmdtp->cmd)(cmdtp, flag, argc, argv);
495     if (result)
496         debug("Command failed, result=%d\n", result);
497     return result;
498 }

```

在前面的分析中我们知道, `cmd_tbl_t` 的 `cmd` 成员就是具体的命令处理函数, 所以第 494 行调用 `cmdtp` 的 `cmd` 成员来处理具体的命令, 返回值为命令的执行结果。

`cmd_process` 中会检测 `cmd_tbl_t` 的返回值, 如果返回值为 `CMD_RET_USAGE` 的话就会调用 `cmd_usage` 函数输出命令的用法, 其实就是输出 `cmd_tbl_t` 的 `usage` 成员变量。

32.3 bootz 启动 Linux 内核过程

32.3.1 images 全局变量

不管是 bootz 还是 bootm 命令, 在启动 Linux 内核的时候都会用到一个重要的全局变量: images, images 在文件 cmd/bootm.c 中有如下定义:

示例代码 32.3.1.1 images 全局变量

```
43 bootm_headers_t images; /* pointers to os/initrd/fdt images */
```

images 是 bootm_headers_t 类型的全局变量, bootm_headers_t 是个 boot 头结构体, 在文件 include/image.h 中的定义如下(删除了一些条件编译代码):

示例代码 32.3.1.2 bootm_headers_t 结构体

```
304 typedef struct bootm_headers {
305     /*
306      * Legacy os image header, if it is a multi component image
307      * then boot_get_ramdisk() and get_fdt() will attempt to get
308      * data from second and third component accordingly.
309      */
310     image_header_t *legacy_hdr_os; /* image header pointer */
311     image_header_t legacy_hdr_os_copy; /* header copy */
312     ulong legacy_hdr_valid;
313
314     .....
333
334 #ifndef USE_HOSTCC
335     image_info_t os; /* OS 镜像信息 */
336     ulong ep; /* OS 入口点 */
337
338     ulong rd_start, rd_end; /* ramdisk 开始和结束位置 */
339
340     char *ft_addr; /* 设备树地址 */
341     ulong ft_len; /* 设备树长度 */
342
343     ulong initrd_start; /* initrd 开始位置 */
344     ulong initrd_end; /* initrd 结束位置 */
345     ulong cmdline_start; /* cmdline 开始位置 */
346     ulong cmdline_end; /* cmdline 结束位置 */
347     bd_t *kbd;
348 #endif
349
350     int verify; /* getenv("verify")[0] != 'n' */
351
352     #define BOOTM_STATE_START (0x00000001)
353     #define BOOTM_STATE_FINDOS (0x00000002)
354     #define BOOTM_STATE_FINDOTHER (0x00000004)
```

```

355 #define BOOTM_STATE_LOADOS      (0x00000008)
356 #define BOOTM_STATE_RAMDISK     (0x00000010)
357 #define BOOTM_STATE_FDT         (0x00000020)
358 #define BOOTM_STATE_OS_CMDLINE  (0x00000040)
359 #define BOOTM_STATE_OS_BD_T     (0x00000080)
360 #define BOOTM_STATE_OS_PREP     (0x00000100)
361 #define BOOTM_STATE_OS_FAKE_GO  (0x00000200) /* 'Almost' run the OS */
362 #define BOOTM_STATE_OS_GO       (0x00000400)
363     int      state;
364
365 #ifdef CONFIG_LMB
366     struct lmb lmb; /* 内存管理相关, 不深入研究 */
367 #endif
368 } bootm_headers_t;
    
```

第 335 行的 os 成员变量是 image_info_t 类型的, 为系统镜像信息。

第 352~362 行这 11 个宏定义表示 BOOT 的不同阶段。

接下来看一下结构体 image_info_t, 也就是系统镜像信息结构体, 此结构体在文件 include/image.h 中的定义如下:

示例代码 32.3.1.3 image_info_t 结构体

```

292 typedef struct image_info {
293     ulong start, end; /* blob 开始和结束位置 */
294     ulong image_start, image_len; /* 镜像起始地址 (包括 blob) 和长度 */
295     ulong load; /* 系统镜像加载地址 */
296     uint8_t comp, type, os; /* 镜像压缩、类型, OS 类型 */
297     uint8_t arch; /* CPU 架构 */
298 } image_info_t;
    
```

全局变量 images 会在 bootz 命令的执行中频繁使用到, 相当于 Linux 内核启动的“灵魂”。

32.3.2 do_bootz 函数

bootz 命令的执行函数为 do_bootz, 在文件 cmd/bootm.c 中有如下定义:

示例代码 32.3.2.1 do_bootz 函数

```

622 int do_bootz(cmd_tbl_t *cmdtp, int flag, int argc, char * const
argv[])
623 {
624     int ret;
625
626     /* Consume 'bootz' */
627     argc--; argv++;
628
629     if (bootz_start(cmdtp, flag, argc, argv, &images))
630         return 1;
631
632     /*
    
```



```

633     * We are doing the BOOTM_STATE_LOADOS state ourselves, so must
634     * disable interrupts ourselves
635     */
636     bootm_disable_interrupts();
637
638     images.os.os = IH_OS_LINUX;
639     ret = do_bootm_states(cmdtp, flag, argc, argv,
640                          BOOTM_STATE_OS_PREP | BOOTM_STATE_OS_FAKE_GO |
641                          BOOTM_STATE_OS_GO,
642                          &images, 1);
643
644     return ret;
645 }

```

第 629 行, 调用 bootz_start 函数, bootz_start 函数执行过程参考 32.3.3 小节。

第 636 行, 调用函数 bootm_disable_interrupts 关闭中断。

第 638 行, 设置 images.os.os 为 IH_OS_LINUX, 也就是设置系统镜像为 Linux, 表示我们要启动的是 Linux 系统! 后面会用到 images.os.os 来挑选具体的启动函数。

第 639 行, 调用函数 do_bootm_states 来执行不同的 BOOT 阶段, 这里要执行的 BOOT 阶段有: BOOTM_STATE_OS_PREP、BOOTM_STATE_OS_FAKE_GO 和 BOOTM_STATE_OS_GO。

32.3.3 bootz_start 函数

bootz_start 函数也定义在文件 cmd/bootm.c 中, 函数内容如下:

示例代码 32.3.3.1 bootz_start 函数

```

578 static int bootz_start(cmd_tbl_t *cmdtp, int flag, int argc,
579                        char * const argv[], bootm_headers_t *images)
580 {
581     int ret;
582     ulong zi_start, zi_end;
583
584     ret = do_bootm_states(cmdtp, flag, argc, argv,
585                          BOOTM_STATE_START, images, 1);
586
587     /* Setup Linux kernel zImage entry point */
588     if (!argc) {
589         images->ep = load_addr;
590         debug("* kernel: default image load address = 0x%08lx\n",
591              load_addr);
592     } else {
593         images->ep = simple_strtoul(argv[0], NULL, 16);
594         debug("* kernel: cmdline image address = 0x%08lx\n",
595              images->ep);
596     }
597

```

```

598     ret = bootz_setup(images->ep, &zi_start, &zi_end);
599     if (ret != 0)
600         return 1;
601
602     lmb_reserve(&images->lmb, images->ep, zi_end - zi_start);
603
604     /*
605      * Handle the BOOTM_STATE_FINDOTHER state ourselves as we do not
606      * have a header that provide this information.
607      */
608     if (bootm_find_images(flag, argc, argv))
609         return 1;
610
611     .....
619     return 0;
620 }

```

第 584 行, 调用函数 do_bootm_states, 执行 BOOTM_STATE_START 阶段。

第 593 行, 设置 images 的 ep 成员变量, 也就是系统镜像的入口点, 使用 bootz 命令启动系统的时候就会设置系统在 DRAM 中的存储位置, 这个存储位置就是系统镜像的入口点, 因此 images->ep=0X80800000。

第 598 行, 调用 bootz_setup 函数, 此函数会判断当前的系统镜像文件是否为 Linux 的镜像文件, 并且会打印出镜像相关信息, bootz_setup 函数稍后会讲解。

第 608 行, 调用函数 bootm_find_images 查找 ramdisk 和设备树(dtb)文件, 但是我们没有用到 ramdisk, 因此此函数在这里仅仅用于查找设备树(dtb)文件, 此函数稍后也会讲解。

先来看一下 bootz_setup 函数, 此函数定义在文件 arch/arm/lib/bootm.c 中, 函数内容如下:

示例代码 32.3.3.2 bootz_setup 函数

```

370 #define LINUX_ARM_ZIMAGE_MAGIC 0x016f2818
371
372 int bootz_setup(ulong image, ulong *start, ulong *end)
373 {
374     struct zimage_header *zi;
375
376     zi = (struct zimage_header *)map_sysmem(image, 0);
377     if (zi->zi_magic != LINUX_ARM_ZIMAGE_MAGIC) {
378         puts("Bad Linux ARM zImage magic!\n");
379         return 1;
380     }
381
382     *start = zi->zi_start;
383     *end = zi->zi_end;
384
385     printf("Kernel image @ %#08lx [ %#08lx - %#08lx ]\n", image,

```

```
386         *start, *end);
387
388     return 0;
389 }
```

第 370 行, 宏 LINUX_ARM_ZIMAGE_MAGIC 就是 ARM Linux 系统魔术数。

第 376 行, 从传递进来的参数 image(也就是系统镜像首地址)中获取 zimage 头。zImage 头结构体为 zimage_header。

第 377~380 行, 判断 image 是否为 ARM 的 Linux 系统镜像, 如果不是的话就直接返回, 并且打印出 “Bad Linux ARM zImage magic!”, 比如我们输入一个错误的启动命令:

```
bootz 80000000 - 900000000
```

因为我们并没有在 0X80000000 处存放 Linux 镜像文件(zImage), 因此上面的命令肯定会执行出错的, 结果如图 32.3.3.1 所示:

```
=> bootz 80000000 - 900000000
Bad Linux ARM zImage magic!
=>
```

图 32.3.3.1 启动出错

第 382、383 行初始化函数 bootz_setup 的参数 start 和 end。

第 385 行, 打印启动信息, 如果 Linux 系统镜像正常的话就会输出图 32.3.3.2 所示的信息:

```
[[Kernel] image @ 0x80800000 [ 0x000000 - 0x65ef68 ]
```

图 32.3.3.3 Linux 镜像信息

接下来看一下函数 bootm_find_images, 此函数定义在文件 common/bootm.c 中, 函数内容如下:

```
示例代码 32.3.3.3 bootm_find_images 函数
225 int bootm_find_images(int flag, int argc, char * const argv[])
226 {
227     int ret;
228
229     /* find ramdisk */
230     ret = boot_get_ramdisk(argc, argv, &images, IH_INITRD_ARCH,
231                             &images.rd_start, &images.rd_end);
232     if (ret) {
233         puts("Ramdisk image is corrupt or invalid\n");
234         return 1;
235     }
236
237 #if defined(CONFIG_OF_LIBFDT)
238     /* find flattened device tree */
239     ret = boot_get_fdt(flag, argc, argv, IH_ARCH_DEFAULT, &images,
240                         &images.ft_addr, &images.ft_len);
241     if (ret) {
242         puts("Could not find a valid device tree\n");
243         return 1;
244     }
```

```
244     }
245     set_working_fdt_addr((ulong)images.ft_addr);
246 #endif
.....
258     return 0;
259 }
```

第 230~235 行是跟查找 ramdisk, 但是我们没有用到 ramdisk, 因此这部分代码不用管。

第 237~244 行是查找设备树(dtb)文件, 找到以后就将设备树的起始地址和长度分别写到 images 的 ft_addr 和 ft_len 成员变量中。我们使用 bootz 启动 Linux 的时候已经指明了设备树在 DRAM 中的存储地址, 因此 images.ft_addr=0X83000000, 长度根据具体的设备树文件而定, 比如我现在使用的设备树文件长度为 0X8C81, 因此 images.ft_len=0X8C81。

bootz_start 函数就讲解到这里, bootz_start 主要用于初始化 images 的相关成员变量。

32.3.4 do_bootm_states 函数

do_bootz 最后调用的就是函数 do_bootm_states, 而且在 bootz_start 中也调用了 do_bootm_states 函数, 看来 do_bootm_states 函数还是个香饽饽。此函数定义在文件 common/bootm.c 中, 函数代码如下:

示例代码 32.3.4.1 do_bootm_states 函数

```
591 int do_bootm_states(cmd_tbl_t *cmdtp, int flag, int argc, char *
const argv[],
592                     int states, bootm_headers_t *images, int boot_progress)
593 {
594     boot_os_fn *boot_fn;
595     ulong iflag = 0;
596     int ret = 0, need_boot_fn;
597
598     images->state |= states;
599
600     /*
601      * Work through the states and see how far we get. We stop on
602      * any error.
603      */
604     if (states & BOOTM_STATE_START)
605         ret = bootm_start(cmdtp, flag, argc, argv);
606
607     if (!ret && (states & BOOTM_STATE_FINDOS))
608         ret = bootm_find_os(cmdtp, flag, argc, argv);
609
610     if (!ret && (states & BOOTM_STATE_FINDOTHER)) {
611         ret = bootm_find_other(cmdtp, flag, argc, argv);
612         argc = 0; /* consume the args */
613     }
614 }
```

```

615     /* Load the OS */
616     if (!ret && (states & BOOTM_STATE_LOADOS)) {
617         ulong load_end;
618
619         iflag = bootm_disable_interrupts();
620         ret = bootm_load_os(images, &load_end, 0);
621         if (ret == 0)
622             lmb_reserve(&images->lmb, images->os.load,
623                 (load_end - images->os.load));
624         else if (ret && ret != BOOTM_ERR_OVERLAP)
625             goto err;
626         else if (ret == BOOTM_ERR_OVERLAP)
627             ret = 0;
628 #if defined(CONFIG_SILENT_CONSOLE)
629 && !defined(CONFIG_SILENT_U_BOOT_ONLY)
630         if (images->os.os == IH_OS_LINUX)
631             fixup_silent_linux();
632 #endif
633     }
634
635     /* Relocate the ramdisk */
636 #ifdef CONFIG_SYS_BOOT_RAMDISK_HIGH
637     if (!ret && (states & BOOTM_STATE_RAMDISK)) {
638         ulong rd_len = images->rd_end - images->rd_start;
639
640         ret = boot_ramdisk_high(&images->lmb, images->rd_start,
641             rd_len, &images->initrd_start, &images->initrd_end);
642         if (!ret) {
643             setenv_hex("initrd_start", images->initrd_start);
644             setenv_hex("initrd_end", images->initrd_end);
645         }
646     }
647 #endif
648 #if defined(CONFIG_OF_LIBFDT) && defined(CONFIG_LMB)
649     if (!ret && (states & BOOTM_STATE_FDT)) {
650         boot_fdt_add_mem_rsv_regions(&images->lmb, images->ft_addr);
651         ret = boot_relocate_fdt(&images->lmb, &images->ft_addr,
652             &images->ft_len);
653     }
654 #endif
655
656     /* From now on, we need the OS boot function */
657     if (ret)

```

```

657     return ret;
658     boot_fn = bootm_os_get_boot_func(images->os.os);
659     need_boot_fn = states & (BOOTM_STATE_OS_CMDLINE |
660         BOOTM_STATE_OS_BD_T | BOOTM_STATE_OS_PREP |
661         BOOTM_STATE_OS_FAKE_GO | BOOTM_STATE_OS_GO);
662     if (boot_fn == NULL && need_boot_fn) {
663         if (iflag)
664             enable_interrupts();
665         printf("ERROR: booting os '%s' (%d) is not supported\n",
666             genimg_get_os_name(images->os.os), images->os.os);
667         bootstage_error(BOOTSTAGE_ID_CHECK_BOOT_OS);
668         return 1;
669     }
670
671     /* Call various other states that are not generally used */
672     if (!ret && (states & BOOTM_STATE_OS_CMDLINE))
673         ret = boot_fn(BOOTM_STATE_OS_CMDLINE, argc, argv, images);
674     if (!ret && (states & BOOTM_STATE_OS_BD_T))
675         ret = boot_fn(BOOTM_STATE_OS_BD_T, argc, argv, images);
676     if (!ret && (states & BOOTM_STATE_OS_PREP))
677         ret = boot_fn(BOOTM_STATE_OS_PREP, argc, argv, images);
678
679 #ifdef CONFIG_TRACE
680     /* Pretend to run the OS, then run a user command */
681     if (!ret && (states & BOOTM_STATE_OS_FAKE_GO)) {
682         char *cmd_list = getenv("fakegocmd");
683
684         ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_FAKE_GO,
685             images, boot_fn);
686         if (!ret && cmd_list)
687             ret = run_command_list(cmd_list, -1, flag);
688     }
689 #endif
690
691     /* Check for unsupported subcommand. */
692     if (ret) {
693         puts("subcommand not supported\n");
694         return ret;
695     }
696
697     /* Now run the OS! We hope this doesn't return */
698     if (!ret && (states & BOOTM_STATE_OS_GO))
699         ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_GO,

```

```

700             images, boot_fn);
.....
712     return ret;
713 }

```

函数 `do_bootm_states` 根据不同的 BOOT 状态执行不同的代码段,通过如下代码来判断 BOOT 状态:

```
states & BOOTM_STATE_XXX
```

在 `do_bootz` 函数中会用到 `BOOTM_STATE_OS_PREP`、`BOOTM_STATE_OS_FAKE_GO` 和 `BOOTM_STATE_OS_GO` 这三个 BOOT 状态, `bootz_start` 函数中会用到 `BOOTM_STATE_START` 这个 BOOT 状态。为了精简代码,方便分析,因此我们将示例代码 32.3.4.1 中的函数 `do_bootm_states` 进行精简,只留下下面这 4 个 BOOT 状态对应的处理代码:

```

BOOTM_STATE_OS_PREP
BOOTM_STATE_OS_FAKE_GO
BOOTM_STATE_OS_GO
BOOTM_STATE_START

```

精简以后的 `do_bootm_states` 函数如下所示:

示例代码 32.3.4.2 精简后的 `do_bootm_states` 函数

```

591 int do_bootm_states(cmd_tbl_t *cmdtp, int flag, int argc, char *
const argv[],
592             int states, bootm_headers_t *images, int boot_progress)
593 {
594     boot_os_fn *boot_fn;
595     ulong iflag = 0;
596     int ret = 0, need_boot_fn;
597
598     images->state |= states;
599
600     /*
601      * Work through the states and see how far we get. We stop on
602      * any error.
603      */
604     if (states & BOOTM_STATE_START)
605         ret = bootm_start(cmdtp, flag, argc, argv);
606     .....
654
655     /* From now on, we need the OS boot function */
656     if (ret)
657         return ret;
658     boot_fn = bootm_os_get_boot_func(images->os.os);
659     need_boot_fn = states & (BOOTM_STATE_OS_CMDLINE |
660             BOOTM_STATE_OS_BD_T | BOOTM_STATE_OS_PREP |
661             BOOTM_STATE_OS_FAKE_GO | BOOTM_STATE_OS_GO);
662     if (boot_fn == NULL && need_boot_fn) {

```



```

663     if (iflag)
664         enable_interrupts();
665     printf("ERROR: booting os '%s' (%d) is not supported\n",
666           genimg_get_os_name(images->os.os), images->os.os);
667     bootstage_error(BOOTSTAGE_ID_CHECK_BOOT_OS);
668     return 1;
669 }
670
671 .....
672 if (!ret && (states & BOOTM_STATE_OS_PREP))
673     ret = boot_fn(BOOTM_STATE_OS_PREP, argc, argv, images);
674
675 #ifdef CONFIG_TRACE
676     /* Pretend to run the OS, then run a user command */
677     if (!ret && (states & BOOTM_STATE_OS_FAKE_GO)) {
678         char *cmd_list = getenv("fakegocmd");
679
680         ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_FAKE_GO,
681                               images, boot_fn);
682         if (!ret && cmd_list)
683             ret = run_command_list(cmd_list, -1, flag);
684     }
685 #endif
686
687 /* Check for unsupported subcommand. */
688 if (ret) {
689     puts("subcommand not supported\n");
690     return ret;
691 }
692
693 /* Now run the OS! We hope this doesn't return */
694 if (!ret && (states & BOOTM_STATE_OS_GO))
695     ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_GO,
696                             images, boot_fn);
697
698 .....
699 return ret;
700 }

```

第 604、605 行，处理 BOOTM_STATE_START 阶段，bootz_start 会执行这一段代码，这里调用函数 bootm_start，此函数定义在文件 common/bootm.c 中，函数内容如下：

示例代码 32.3.4.2 bootm_start 函数

```

69 static int bootm_start(cmd_tbl_t *cmdtp, int flag, int argc,
70                        char * const argv[])
71 {

```

```

72     memset((void *)&images, 0, sizeof(images)); /* 清空 images */
73     images.verify = getenv_yesno("verify"); /* 初始化 verify 成员 */
74
75     boot_start_lmb(&images);
76
77     bootstage_mark_name(BOOTSTAGE_ID_BOOTM_START, "bootm_start");
78     images.state = BOOTM_STATE_START; /* 设置状态为 BOOTM_STATE_START */
79
80     return 0;
81 }

```

接着回到示例代码 32.3.4.2 中, 继续分析函数 do_bootm_states。第 658 行非常重要! 通过函数 bootm_os_get_boot_func 来查找系统启动函数, 参数 images->os.os 就是系统类型, 根据这个系统类型来选择对应的启动函数, 在 do_bootz 中设置 images.os.os= IH_OS_LINUX。函数返回值就是找到的系统启动函数, 这里找到的 Linux 系统启动函数为 do_bootm_linux, 关于此函数查找系统启动函数的过程请参考 32.3.5 小节。因此 boot_fn=do_bootm_linux, 后面执行 boot_fn 函数的地方实际上是执行的 do_bootm_linux 函数。

第 676 行, 处理 BOOTM_STATE_OS_PREP 状态, 调用函数 do_bootm_linux, do_bootm_linux 也是调用 boot_prep_linux 来完成具体的处理过程。boot_prep_linux 主要用于处理环境变量 bootargs, bootargs 保存着传递给 Linux kernel 的参数。

第 679~689 行是处理 BOOTM_STATE_OS_FAKE_GO 状态的, 但是我们要没用使用 TRACE 功能, 因此宏 CONFIG_TRACE 也就没有定义, 所以这段程序不会编译。

第 699 行, 调用函数 boot_selected_os 启动 Linux 内核, 此函数第 4 个参数为 Linux 系统镜像头, 第 5 个参数就是 Linux 系统启动函数 do_bootm_linux。boot_selected_os 函数定义在文件 common/bootm_os.c 中, 函数内容如下:

示例代码 32.3.4.3 boot_selected_os 函数

```

476 int boot_selected_os(int argc, char * const argv[], int state,
477                      bootm_headers_t *images, boot_os_fn *boot_fn)
478 {
479     arch_preboot_os();
480     boot_fn(state, argc, argv, images);
481     .....
490     return BOOTM_ERR_RESET;
491 }

```

第 480 行调用 boot_fn 函数, 也就是 do_bootm_linux 函数来启动 Linux 内核。

32.3.5 bootm_os_get_boot_func 函数

do_bootm_states 会调用 bootm_os_get_boot_func 来查找对应系统的启动函数, 此函数定义在文件 common/bootm_os.c 中, 函数内容如下:

示例代码 32.3.5.1 bootm_os_get_boot_func 函数

```

493 boot_os_fn *bootm_os_get_boot_func(int os)
494 {
495     #ifdef CONFIG_NEEDS_MANUAL_RELOC
496         static bool relocated;

```

```

497
498     if (!relocated) {
499         int i;
500
501         /* relocate boot function table */
502         for (i = 0; i < ARRAY_SIZE(boot_os); i++)
503             if (boot_os[i] != NULL)
504                 boot_os[i] += gd->reloc_off;
505
506         relocated = true;
507     }
508 #endif
509     return boot_os[os];
510 }

```

第 495~508 行是条件编译, 在本 uboot 中没有用到, 因此这段代码无效, 只有 509 行有效。在 509 行中 boot_os 是个数组, 这个数组里面存放着不同的系统对应的启动函数。boot_os 也定义在文件 common/bootm_os.c 中, 如下所示:

示例代码 32.3.5.2 boot_os 数组

```

435 static boot_os_fn *boot_os[] = {
436     [IH_OS_U_BOOT] = do_bootm_standalone,
437 #ifdef CONFIG_BOOTM_LINUX
438     [IH_OS_LINUX] = do_bootm_linux,
439 #endif
440     .....
441 #ifdef CONFIG_BOOTM_OPENRTOS
442     [IH_OS_OPENRTOS] = do_bootm_openrtos,
443 #endif
444 };

```

第 438 行就是 Linux 系统对应的启动函数: do_bootm_linux。

32.3.6 do_bootm_linux 函数

经过前面的分析, 我们知道了 do_bootm_linux 就是最终启动 Linux 内核的函数, 此函数定义在文件 arch/arm/lib/bootm.c, 函数内容如下:

示例代码 32.3.6.1 do_bootm_linux 函数

```

339 int do_bootm_linux(int flag, int argc, char * const argv[],
340                    bootm_headers_t *images)
341 {
342     /* No need for those on ARM */
343     if (flag & BOOTM_STATE_OS_BD_T || flag & BOOTM_STATE_OS_CMDLINE)
344         return -1;
345
346     if (flag & BOOTM_STATE_OS_PREP) {
347         boot_prep_linux(images);

```

```
348     return 0;
349 }
350
351 if (flag & (BOOTM_STATE_OS_GO | BOOTM_STATE_OS_FAKE_GO)) {
352     boot_jump_linux(images, flag);
353     return 0;
354 }
355
356 boot_prep_linux(images);
357 boot_jump_linux(images, flag);
358 return 0;
359 }
```

第351行,如果参数flag等于BOOTM_STATE_OS_GO或者BOOTM_STATE_OS_FAKE_GO的话就执行boot_jump_linux函数。boot_selected_os函数在调用do_bootm_linux的时候会将flag设置为BOOTM_STATE_OS_GO。

第352行,执行函数boot_jump_linux,此函数定义在文件arch/arm/lib/bootm.c中,函数内容如下:

示例代码 32.3.6.2 boot_jump_linux 函数

```
272 static void boot_jump_linux(bootm_headers_t *images, int flag)
273 {
274 #ifdef CONFIG_ARM64
275     .....
276 #else
277     unsigned long machid = gd->bd->bi_arch_number;
278     char *s;
279     void (*kernel_entry)(int zero, int arch, uint params);
280     unsigned long r2;
281     int fake = (flag & BOOTM_STATE_OS_FAKE_GO);
282
283     kernel_entry = (void (*)(int, int, uint))images->ep;
284
285     s = getenv("machid");
286     if (s) {
287         if (strict_strtoul(s, 16, &machid) < 0) {
288             debug("strict_strtoul failed!\n");
289             return;
290         }
291         printf("Using machid 0x%x from environment\n", machid);
292     }
293
294     debug("## Transferring control to Linux (at address %08lx)" \
295         "... \n", (ulong) kernel_entry);
296     bootstage_mark(BOOTSTAGE_ID_RUN_OS);
```

```

313     announce_and_cleanup(fake);
314
315     if (IMAGE_ENABLE_OF_LIBFDT && images->ft_len)
316         r2 = (unsigned long)images->ft_addr;
317     else
318         r2 = gd->bd->bi_boot_params;
319
320     .....
328         kernel_entry(0, machid, r2);
329     }
330 #endif
331 }

```

第 274~292 行是 64 位 ARM 芯片对应的代码, Cortex-A7 是 32 位芯片, 因此用不到。

第 293 行, 变量 machid 保存机器 ID, 如果不使用设备树的话这个机器 ID 会被传递给 Linux 内核, Linux 内核会在自己的机器 ID 列表里面查找是否存在与 uboot 传递进来的 machid 匹配的项目, 如果存在就说明 Linux 内核支持这个机器, 那么 Linux 就会启动! 如果使用设备树的话这个 machid 就无效了, 设备树存有一个“兼容性”这个属性, Linux 内核会比较“兼容性”属性的值(字符串)来查看是否支持这个机器。

第 295 行, 函数 kernel_entry, 看名字“内核_进入”, 说明此函数是进入 Linux 内核的, 也就是最终的大 boos!! 此函数有三个参数: zero, arch, params, 第一个参数 zero 同样为 0; 第二个参数为机器 ID; 第三个参数 ATAGS 或者设备树(DTB)首地址, ATAGS 是传统的方法, 用于传递一些命令行信息啥的, 如果使用设备树的话就要传递设备树(DTB)。

第 299 行, 获取 kernel_entry 函数, 函数 kernel_entry 并不是 uboot 定义的, 而是 Linux 内核定义的, Linux 内核镜像文件的第一行代码就是函数 kernel_entry, 而 images->ep 保存着 Linux 内核镜像的起始地址, 起始地址保存的正是 Linux 内核第一行代码!

第 313 行, 调用函数 announce_and_cleanup 来打印一些信息并做一些清理工作, 此函数定义在文件 arch/arm/lib/bootm.c 中, 函数内容如下:

示例代码 32.3.6.3 announce_and_cleanup 函数

```

72 static void announce_and_cleanup(int fake)
73 {
74     printf("\nStarting kernel ...%s\n\n", fake ?
75         "(fake run for tracing)" : "");
76     bootstage_mark_name(BOOTSTAGE_ID_BOOTM_HANDOFF, "start_kernel");
77     .....
87     cleanup_before_linux();
88 }

```

第 74 行, 在启动 Linux 之前输出“Starting kernel ...”信息, 如图 32.3.6.1 所示:

```

Kernel image @ 0x80800000 [ 0x000000 - 0x65ef68 ]
## Flattened Device Tree blob at 83000000
Booting using the fdt blob at 0x83000000
Using Device Tree in place at 83000000, end 8300bc80

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 4.1.15 (zuozhongkai@ubuntu) (gcc version 4.9.4 (Linaro GCC 4.
t May 25 12:32:15 CST 2019

```

图 32.3.6.1 系统启动提示信息

第 87 行调用 `cleanup_before_linux` 函数做一些清理工作。

继续回到示例代码 32.3.6.2 的函数 `boot_jump_linux`, 第 315~318 行是设置寄存器 `r2` 的值? 为什么要设置 `r2` 的值呢? Linux 内核一开始是汇编代码, 因此函数 `kernel_entry` 就是个汇编函数。向汇编函数传递参数要使用 `r0`、`r1` 和 `r2`(参数数量不超过 3 个的时候), 所以 `r2` 寄存器就是函数 `kernel_entry` 的第三个参数。

第 316 行, 如果使用设备树的话, `r2` 应该是设备树的起始地址, 而设备树地址保存在 `images` 的 `ftd_addr` 成员变量中。

第 317 行, 如果不使用设备树的话, `r2` 应该是 `uboot` 传递给 Linux 的参数起始地址, 也就是环境变量 `bootargs` 的值,

第 328 行, 调用 `kernel_entry` 函数进入 Linux 内核, 此行将一去不复返, `uboot` 的使命也就完成了, 它可以安息了!

总结一下 `bootz` 命令的执行过程, 如图 32.3.6.2 所示:

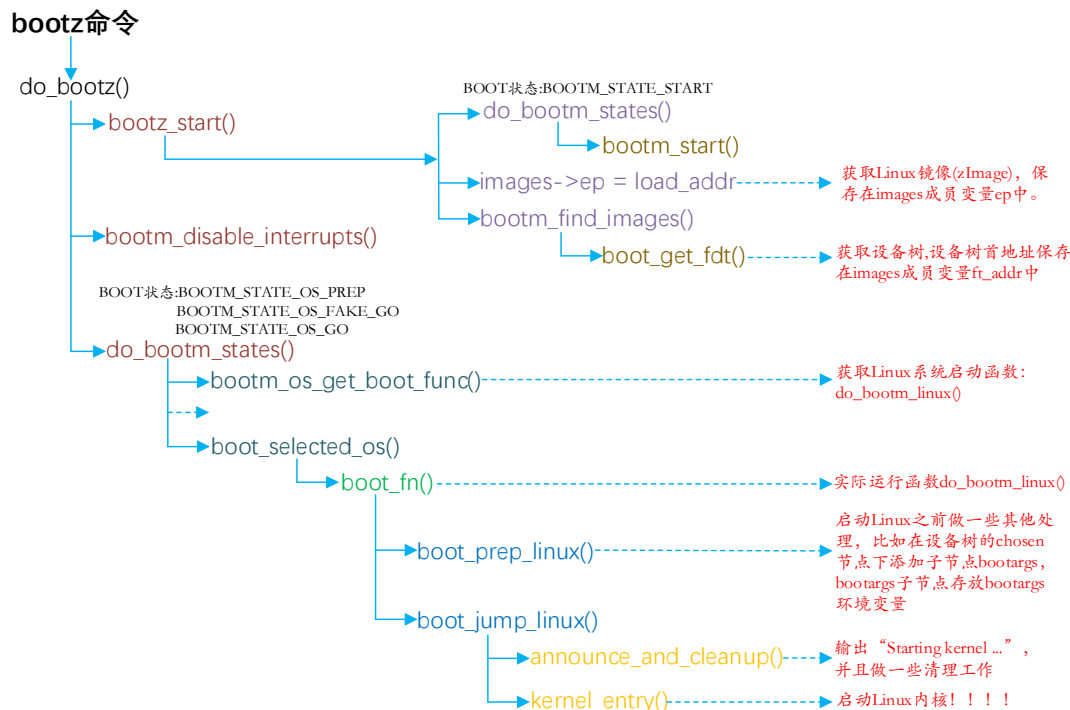


图 32.3.6.2 bootz 命令执行过程

到这里 `uboot` 的启动流程我们就讲解完成了, 加上 `uboot` 顶层 `Makefile` 的分析, 洋洋洒洒 100 多页, 还是不少的! 这也仅仅是 `uboot` 启动流程分析, 当缕清了 `uboot` 的启动流程以后, 后面移植 `uboot` 就会轻松很多。其实在工作中我们基本不需要这么详细的去了解 `uboot`, 半导体厂商提供给我们的 `uboot` 一般是可以直接用的, 只要能跑起来, 可以使用就可以了。但是作为学习, 我们是必须去详细的了解一下 `uboot` 的启动流程, 否则如果在工作中遇到问题我们连解决的方法都没有, 都不知道该从哪里看起。但是呢, 如果第一次就想弄懂 `uboot` 的整个启动流程还是有点困难的, 所以如果没有看懂的话, 不要紧! 不要气馁, 大多数人第一次看 `uboot` 启动流程基本都有各种各样的问题。

题外话:

相信大家看完本章以后基本都有一个感觉: 长、复杂、绕! 没错, 当我第一次学习 `uboot` 的时候看到 `uboot` 启动流程的时候也是这个感觉, 当时我也一脸懵逼, 怎么这么复杂, 这么长呢?

尤其前面的汇编代码部分，还要涉及到 ARM 处理器架构的内容，当时也怀疑过自己是不是搞这一块的料。不过好在自己坚持下来了，uboot 的启动流程我至少分析过 7,8 遍，各种版本的，零几年很古老的；12 年、14 年比较新的等等很多个版本的 uboot。就 I.MX6ULL 使用的这个 2016.03 版本 uboot 我至少详细的分析了 2 遍，直至写完本章，大概花了 1 个月的时间。这期间查阅了各种资料，看了不知道多少篇博客，在这里感谢那些无私奉献的网友们。

相信很多朋友看完本章可能会想：我什么时候也能这么厉害，能够这么详细的分析 uboot 启动流程。甚至可能会有挫败感，还是那句话：不要气馁！千里之行始于足下，所有你羡慕的人都曾经痛苦过，挫败过。脚踏实地，一步一个脚印，一点一滴的积累，最终你也会成为你所羡慕的人。在嵌入式 Linux 这条道路上，有众多的学习者陪着你，大家相互搀扶，终能踏出一条康庄大道，祝所有的同学终有所获！

第三十三章 U-Boot 移植

上一章节我们详细的分析了 uboot 的启动流程，对 uboot 有了一个初步的了解。前两章我们都是使用的正点原子提供的 uboot，本章我们就来学习如何将 NXP 官方的 uboot 移植到正点原子的 I.MX6ULL 开发板上，学习如何在 uboot 中添加我们自己的板子。

33.1 NXP 官方开发板 uboot 编译测试

33.1.1 查找 NXP 官方的开发板默认配置文件

uboot 的移植并不是说我们完完全全的从零开始将 uboot 移植到我们现在所使用的开发板或者开发平台上。这个对于我们来说基本是不可能的，这个工作一般是半导体厂商做的，半导体厂商负责将 uboot 移植到他们的芯片上，因此半导体厂商都会自己做一个开发板，这个开发板就叫做原厂开发板，比如大家学习 STM32 的时候听说过的 discover 开发板就是 ST 自己做的。半导体厂商会将 uboot 移植到他们自己的原厂开发板上，测试好以后就会将这个 uboot 发布出去，这就是大家常说的原厂 BSP 包。我们一般做产品的时候就会参考原厂的开发板做硬件，然后在原厂提供的 BSP 包上做修改，将 uboot 或者 linux kernel 移植到我们的硬件上。这个就是 uboot 移植的一般流程：

- ①、在 uboot 中找到参考的开发平台，一般是原厂的开发板。
- ②、参考原厂开发板移植 uboot 到我们所使用的开发板上。

正点原子的 I.MX6ULL 开发板参考的是 NXP 官方的 I.MX6ULL EVK 开发板做的硬件，因此我们在移植 uboot 的时候就可以以 NXP 官方的 I.MX6ULL EVK 开发板为蓝本。

本章我们是将 NXP 官方的 uboot 移植到正点原子的 I.MX6ULL 开发板上，NXP 官方的 uboot 放到了开发板光盘中，路径为：1、例程源码->4、NXP 官方原版 Uboot 和 Linux->uboot-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2。将 uboot-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2 发送到 Ubuntu 中并解压，然后创建 VSCode 工程。

在移植之前，我们先编译一下 NXP 官方 I.MX6ULL EVK 开发板对应的 uboot，首先是配置 uboot，configs 目录下有很多跟 I.MX6UL/6ULL 有关的配置如图 33.1.1.1 所示，

名称	修改日期	类型	大小
 mx6ul_9x9_evk_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_eimnor_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_emmc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_nand_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_ddr3_arm2_spinor_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_ddr_eol_brillo_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_ddr_eol_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_ddr_eol_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_emmc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_nand_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_evk_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_lpdrr2_arm2_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ul_14x14_lpdrr2_arm2_eimnor_defconf...	2017/5/2 10:45	文件	1 KB
 mx6ull_9x9_evk_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_9x9_evk_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_emmc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_epdc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_nand_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_qspi1_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_spinor_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_ddr3_arm2_tsc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_evk_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_evk_emmc_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_evk_nand_defconfig	2017/5/2 10:45	文件	1 KB
 mx6ull_14x14_evk_qspi1_defconfig	2017/5/2 10:45	文件	1 KB

图 33.1.1.1 NXP 官方 I.MX6UL/6ULL 默认配置文件

从图 33.1.1.1 可以看出有很多的默认配置文件，其中以 mx6ul 开头的是 I.MX6UL 芯片的，mx6ull 开头的是 I.MX6ULL 开发板的。I.MX6UL/6ULL 有 9x9mm 和 14x14mm 两种尺寸的，所以我们可以看到会有 mx6ull_9x9 和 mx6ull_14x14 开头的默认配置文件。我们使用的是 14x14mm 的芯片，所以关注 mx6ull_14x14 开头的默认配置文件。正点原子的 I.MX6ULL 有 EMMC 和 NAND 两个版本的，因此我们最终只需要关注 mx6ull_14x14_evk_emmc_defconfig 和 mx6ull_14x14_evk_nand_defconfig 这两个配置文件就行了。本章我们讲解 EMMC 版本的移植（NAND 版本移植很多类似），所以使用 mx6ull_14x14_evk_emmc_defconfig 作为默认配置文件。

33.1.2 编译 NXP 官方开发板对应的 uboot

找到 NXP 官方 I.MX6ULL EVK 开发板对应的默认配置文件以后就可以编译一下，使用如下命令编译 uboot:

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mx6ull_14x14_evk_emmc_defconfig
make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j16
```

编译完成以后结果如图 33.1.2.1 所示:

```

arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -j .text -j .secure_text -j .rodata -j .hash
-j .data -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -O binary u-boot u-boot-nodtb.bin
arm-linux-gnueabi-hf-objcopy --gap-fill=0xff -j .text -j .secure_text -j .rodata -j .hash
-j .data -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -O srec u-boot u-boot.srec
arm-linux-gnueabi-hf-objdump -t u-boot > u-boot.sym
cp u-boot-nodtb.bin u-boot.bin
make -f ./scripts/Makefile.build obj=arch/arm/imx-common u-boot.imx
mkdir -p board/freescale/mx6ullevk/
./tools/mkimage -n board/freescale/mx6ullevk/imximage.cfg.cfgtmp -T imximage -e 0x87800000
-d u-boot.bin u-boot.imx
Image Type:   Freescale IMX Boot Image
Image Ver:    2 (i.MX53/6/7 compatible)
Mode:        DCD
Data Size:    425984 Bytes = 416.00 kB = 0.41 MB
Load Address: 877ff420
Entry Point:  87800000
zuozhongkai@ubuntu:~/linux/IMX6ULL/u-boot/temp/u-boot-imx-rel imx 4.1.15 2.1.0 ga$

```

图 33.1.2.1 编译结果

从图 33.1.2.1 可以看出,编译成功。我们在编译的时候需要输入 ARCH 和 CROSS_COMPILE 这两个变量的值,这样太麻烦了。我们可以直接在顶层 Makefile 中直接给 ARCH 和 CROSS_COMPILE 赋值,修改如图 33.1.2.2 所示:

```

245 # set default to nothing for native builds
246 ifeq ($(HOSTARCH),$(ARCH))
247 CROSS_COMPILE ?=
248 endif
249
250 ARCH = arm
251 CROSS_COMPILE = arm-linux-gnueabi-hf-

```

图 33.1.2.2 添加 ARCH 和 CROSS_COMPILE 值

图 33.1.2.2 中的 250、251 行就是直接给 ARCH 和 CROSS_COMPILE 赋值,这样我们就可以使用如下简短的命令来编译 uboot 了:

```

make mx6ull_14x14_evk_emmc_defconfig
make V=1 -j16

```

如果既不想修改 uboot 的顶层 Makefile,又想编译的时候不用输入那么多,那么就直接创建个 shell 脚本就行了,shell 脚本名为 mx6ull_14x14_emmc.sh,然后在 shell 脚本里面输入如下内容:

示例代码 33.1.2.1 mx6ull_14x14_emmc.sh 文件

```

1 #!/bin/bash
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf-
mx6ull_14x14_evk_emmc_defconfig
4 make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf- -j16

```

记得给 mx6ull_14x14_emmc.sh 这个文件可执行权限,使用 mx6ull_14x14_emmc.sh 脚本编译 uboot 的时候每次都会清理一下工程,然后全部重新编译,编译的时候直接执行这个脚本就行了,命令如下:

```
./mx6ull_14x14_evk_emmc.sh
```

编译完成以后会生成 u-boot.bin、u-boot.imx 等文件,但是这些文件是 NXP 官方 I.MX6ULL EVK 开发板。能不能用到正点原子的 I.MX6ULL 开发板上呢?试一下不就知道了!

33.1.3 烧写验证与驱动测试

将 imxdownload 软件拷贝到 uboot 源码根目录下, 然后使用 imxdownload 软件将 u-boot.bin 烧写到 SD 卡中, 烧写命令如下:

```
chmod 777 imxdownload //给予 imxdownload 可执行权限
./imxdownload u-boot.bin /dev/sdd //烧写到 SD 卡中, 不能烧写到/dev/sda 或 sda1 里面
```

烧写完成以后将 SD 卡插入 I.MX6U-ALPHA 开发板的 TF 卡槽中, 最后设置开发板从 SD 卡启动。打开 SecureCRT, 设置好开发板所使用的串口并打开, 复位开发板, SecureCRT 接收到如下图 33.1.3.1 所示信息:

```
U-Boot 2016.03 (May 11 2019 - 15:52:17 +0800)
CPU: Freescale i.MX6ULL rev1.1 528 MHz (running at 396 MHz)
CPU: Industrial temperature grade (-40C to 105C) at 45C
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C: ready
DRAM: 512 MiB
MMC: FSL_SDHC: 0, FSL_SDHC: 1
unsupported panel TFT7016
In: serial
Out: serial
Err: serial
switch to partitions #0, OK
mmc0 is current device
Net: Board Net Initialization Failed
No ethernet found.
Normal Boot
Hit any key to stop autoboot: 0
No ethernet found.
No ethernet found.
Bad Linux ARM zImage magic!
=>
```

图 33.1.3.1 uboot 启动信息

从图 33.1.3.1 可以看出, uboot 启动正常, 虽然我们用的是 NXP 官方 I.MX6ULL 开发板的 uboot, 但是在正点原子的 I.MX6ULL 开发板上是可以正常启动的。而且 DRAM 识别正确, 为 512MB, 如果用的 NAND 版本的核心版的话 uboot 启动会失败! 因为 NAND 核心版用的 256MB 的 DRAM。

1、SD 卡和 EMMC 驱动检查

检查一下 SD 卡和 EMMC 驱动是否正常, 使用命令 mmc list 列出当前的 MMC 设备, 结果如图 33.1.3.2 所示:

```
=> mmc list
FSL_SDHC: 0 (SD)
FSL_SDHC: 1
=>
```

图 33.1.3.2 emmc 设备检查

从图 33.1.3.2 可以看出当前有两个 MMC 设备, 检查每个 MMC 设备信息, 先检查 MMC 设备 0, 输入如下命令:

```
mmc dev 0
mmc info
```

结果如图 33.1.3.3 所示:

```
=> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
=> mmc info
Device: FSL_SDHC
Manufacturer ID: 3
OEM: 5344
Name: SC16G
Tran Speed: 50000000
Rd Block Len: 512
SD version 3.0
High Capacity: Yes
Capacity: 14.8 GiB
Bus Width: 4-bit
Erase Group Size: 512 Bytes
```

图 33.1.3.3 mmc 设备 0 信息

从图 33.1.3.3 可以看出, mmc 设备 0 是 SD 卡, SD 卡容量为 14.8GB, 这个和我所使用的 SD 卡信息相符, 说明 SD 卡驱动正常。再来检查 MMC 设备 1, 输入如下命令:

```
mmc dev 1
mmc info
```

结果如图 33.1.3.4 所示:

```
=> mmc dev 1
switch to partitions #0, OK
mmc1(part 0) is current device
=> mmc info
Device: FSL_SDHC
Manufacturer ID: 15
OEM: 100
Name: 8GTF4
Tran Speed: 52000000
Rd Block Len: 512
MMC version 4.0
High Capacity: Yes
Capacity: 7.3 GiB
Bus Width: 4-bit
Erase Group Size: 512 KiB
=>
```

图 33.1.3.4 mmc 设备 1 信息

从图 33.1.3.4 可以看出, mmc 设备 1 为 EMMC, 容量为 7.3GB, 说明 EMMC 驱动也成功, SD 卡和 EMMC 的驱动都没问题。

2、LCD 驱动检查

如果 uboot 中的 LCD 驱动正确的话, 启动 uboot 以后 LCD 上应该会显示出 NXP 的 logo, 如下图 33.1.3.5 所示:

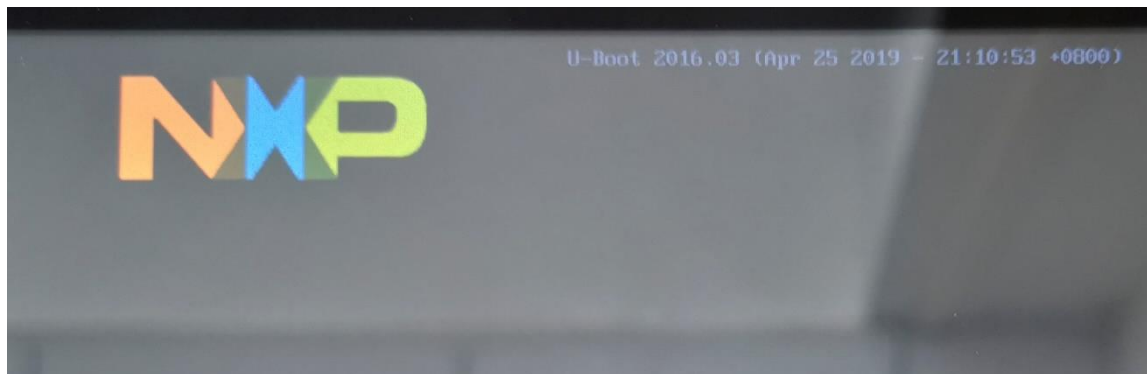


图 33.1.3.5 uboot LCD 界面

如果你用的不是正点原子的 4.3 寸 480x272 分辨率的屏幕的话，那么 LCD 就不会显示 33.1.3.5 所示 logo 界面。因为 NXP 官方 I.MX6ULL 开发板的屏幕就是 4.3 寸 480x272 分辨率的，所以 uboot 默认 LCD 驱动是 4.3 寸 480x272 分辨率的。如果使用其他分辨率的 LCD 就需要修改 LCD 驱动，这里我们先不修改 LCD 驱动了，稍后我们在讲解如何修改 uboot 中的 LCD 驱动，我们只需要记得，uboot 的 LCD 需要修改就行了。

3、网络驱动

uboot 启动的时候提示 “Board Net Initialization Failed” 和 “No ethernet found.” 这两行，说明网络驱动也有问题，正常情况下应该是如图 33.1.3.6 所示提示：

```
switch to partitions #0, OK
mmc0 is current device
Net: FEC0
Normal Boot
Hit any key to stop autoboot: 0
=>
```

图 33.1.3.6 网络信息

现在没有图 33.1.3.6 中的信息，那更别说 ping 一下 ubuntu 主机了，说明当前 uboot 的网络驱动也是有问题的，这是因为正点原子开发板的网络芯片复位引脚和 NXP 官方开发板不一样，因此需要修改驱动。

总结一下 NXP 官方 I.MX6ULL EVK 开发板的 uboot 在正点原子 EMMC 版本 I.MX6ULL 开发板上的运行情况：

①、uboot 启动正常，DRAM 识别正确，SD 卡和 EMMC 驱动正常。

②、uboot 里面的 LCD 驱动默认是给 4.3 寸 480x272 分辨率的，如果使用的其他分辨率的屏幕需要修改驱动。

③、网络不能工作，识别不出来网络信息，需要修改驱动。

接下来我们要做的工作如下：

①、前面我们一直使用着 NXP 官方开发板的 uboot 配置，接下来需要在 uboot 中添加我们自己的开发板，也就是正点原子的 I.MX6ULL 开发板。

②、解决 LCD 驱动和网络驱动的问题。

33.2 在 U-Boot 中添加自己的开发板

NXP 官方 uboot 中默认都是 NXP 自己的开发板，虽说我们可以直接在官方的开发板上直接修改，使 uboot 可以完整的运行在我们的板子上。但是从学习的角度来讲，这样我们就不能

了解到 uboot 是如何添加新平台的。接下来我们就参考 NXP 官方的 I.MX6ULL EVK 开发板，学习如何在 uboot 中添加我们的开发板或者开发平台。

33.2.1 添加开发板默认配置文件

先在 configs 目录下创建默认配置文件，复制 mx6ull_14x14_evk_emmc_defconfig，然后重命名为 mx6ull_alientek_emmc_defconfig，命令如下：

```
cd configs
cp mx6ull_14x14_evk_emmc_defconfig mx6ull_alientek_emmc_defconfig
```

然后将文件 mx6ull_alientek_emmc_defconfig 中的内容改成下面的：

示例代码 33.2.1.1 mx6ull_alientek_emmc_defconfig 文件

```
1 CONFIG_SYS_EXTRA_OPTIONS="IMX_CONFIG=board/freescale/mx6ull_alientek_
   emmc/imximage.cfg,MX6ULL_EVK_EMMC_REWORK"
2 CONFIG_ARM=y
3 CONFIG_ARCH_MX6=y
4 CONFIG_TARGET_MX6ULL_ALIENTEK_EMMC=y
5 CONFIG_CMD_GPIO=y
```

可以看出，mx6ull_alientek_emmc_defconfig 基本和 mx6ull_14x14_evk_emmc_defconfig 中的内容一样，只是第 1 行和第 4 行做了修改。

33.2.2 添加开发板对应的头文件

在目录 include/configs 下添加 I.MX6ULL-ALPHA 开发板对应的头文件，复制 include/configs/mx6ullevk.h，并重命名为 mx6ull_alientek_emmc.h，命令如下：

```
cp include/configs/mx6ullevk.h include/configs/mx6ull_alientek_emmc.h
```

拷贝完成以后将：

```
#ifndef __MX6ULLEVK_CONFIG_H
#define __MX6ULLEVK_CONFIG_H
```

改为：

```
#ifndef __MX6ULL_ALIENTEK_EMMC_CONFIG_H
#define __MX6ULL_ALIENTEK_EMMC_CONFIG_H
```

mx6ull_alientek_emmc.h 里面有很多宏定义，这些宏定义基本用于配置 uboot，也有一些 I.MX6ULL 的配置项目。如果我们自己要想使能或者禁止 uboot 的某些功能，那就在 mx6ull_alientek_emmc.h 里面做修改即可。mx6ull_alientek_emmc.h 里面的内容比较多，去掉一些用不到的配置，精简后的内容如下：

示例代码 33.2.2.1 mx6ull_alientek_emmc.h 文件

```
1  /*
2   * Copyright (C) 2016 Freescale Semiconductor, Inc.
3   *
4   * Configuration settings for the Freescale i.MX6UL 14x14 EVK board.
5   *
6   * SPDX-License-Identifier: GPL-2.0+
7   */
8  #ifndef __MX6ULL_ALEITENK_EMMC_CONFIG_H
9  #define __MX6ULL_ALEITENK_EMMC_CONFIG_H
```

```
10
11
12 #include <asm/arch/imx-regs.h>
13 #include <linux/sizes.h>
14 #include "mx6_common.h"
15 #include <asm/imx-common/gpio.h>
16
17 .....
28
29 #define is_mx6ull_9x9_evk() CONFIG_IS_ENABLED(TARGET_MX6ULL_9X9_EVK)
30
31 #ifdef CONFIG_TARGET_MX6ULL_9X9_EVK
32 #define PHYS_SDRAM_SIZE      SZ_256M
33 #define CONFIG_BOOTARGS_CMA_SIZE  "cma=96M "
34 #else
35 #define PHYS_SDRAM_SIZE      SZ_512M
36 #define CONFIG_BOOTARGS_CMA_SIZE  ""
37 /* DCDC used on 14x14 EVK, no PMIC */
38 #undef CONFIG_LDO_BYPASS_CHECK
39 #endif
40
41 /* SPL options */
42 /* We default not support SPL
43  * #define CONFIG_SPL_LIBCOMMON_SUPPORT
44  * #define CONFIG_SPL_MMC_SUPPORT
45  * #include "imx6_spl.h"
46  */
47
48 #define CONFIG_ENV_VARS_UBOOT_RUNTIME_CONFIG
49
50 #define CONFIG_DISPLAY_CPUINFO
51 #define CONFIG_DISPLAY_BOARDINFO
52
53 /* Size of malloc() pool */
54 #define CONFIG_SYS_MALLOC_LEN      (16 * SZ_1M)
55
56 #define CONFIG_BOARD_EARLY_INIT_F
57 #define CONFIG_BOARD_LATE_INIT
58
59 #define CONFIG_MXC_UART
60 #define CONFIG_MXC_UART_BASE      UART1_BASE
61
62 /* MMC Configs */
```

```

63 #ifdef CONFIG_FSL_USDHC
64 #define CONFIG_SYS_FSL_ESDHC_ADDR    USDHC2_BASE_ADDR
65
66 /* NAND pin conflicts with usdhc2 */
67 #ifdef CONFIG_SYS_USE_NAND
68 #define CONFIG_SYS_FSL_USDHC_NUM    1
69 #else
70 #define CONFIG_SYS_FSL_USDHC_NUM    2
71 #endif
72 #endif
73
74 /* I2C configs */
75 #define CONFIG_CMD_I2C
76 #ifdef CONFIG_CMD_I2C
77 #define CONFIG_SYS_I2C
78 #define CONFIG_SYS_I2C_MXC
79 #define CONFIG_SYS_I2C_MXC_I2C1    /* enable I2C bus 1 */
80 #define CONFIG_SYS_I2C_MXC_I2C2    /* enable I2C bus 2 */
81 #define CONFIG_SYS_I2C_SPEED        100000
82
83 .....
89
90 #define CONFIG_SYS_MMC_IMG_LOAD_PART    1
91
92 #ifdef CONFIG_SYS_BOOT_NAND
93 #define CONFIG_MFG_NAND_PARTITION "mtdparts=gpmi-
nand:64m(boot),16m(kernel),16m(dtb),1m(misc),-(rootfs) "
94 #else
95 #define CONFIG_MFG_NAND_PARTITION ""
96 #endif
97
98 #define CONFIG_MFG_ENV_SETTINGS \
99     "mfgtool_args=setenv bootargs console=${console},${baudrate} " \
100 .....
111     "bootcmd_mfg=run mfgtool_args;bootz ${loadaddr} ${initrd_addr}
${fdt_addr};\0" \
112
113 #if defined(CONFIG_SYS_BOOT_NAND)
114 #define CONFIG_EXTRA_ENV_SETTINGS \
115     CONFIG_MFG_ENV_SETTINGS \
116     "panel=TFT43AB\0" \
117 .....
126     "bootz ${loadaddr} - ${fdt_addr}\0"

```

```

127
128 #else
129 #define CONFIG_EXTRA_ENV_SETTINGS \
130     CONFIG_MFG_ENV_SETTINGS \
131     "script=boot.scr\0" \
132     .....
202     "fi;\0" \
203
204 #define CONFIG_BOOTCOMMAND \
205     "run findfdt;" \
206     .....
216     "else run netboot; fi"
217 #endif
218
219 /* Miscellaneous configurable options */
220 #define CONFIG_CMD_MEMTEST
221 #define CONFIG_SYS_MEMTEST_START    0x80000000
222 #define CONFIG_SYS_MEMTEST_END      (CONFIG_SYS_MEMTEST_START +
223 0x8000000)
224
224 #define CONFIG_SYS_LOAD_ADDR        CONFIG_LOADADDR
225 #define CONFIG_SYS_HZ                1000
226
227 #define CONFIG_STACKSIZE             SZ_128K
228
229 /* Physical Memory Map */
230 #define CONFIG_NR_DRAM_BANKS        1
231 #define PHYS_SDRAM                  MMDC0_ARB_BASE_ADDR
232
233 #define CONFIG_SYS_SDRAM_BASE        PHYS_SDRAM
234 #define CONFIG_SYS_INIT_RAM_ADDR    IRAM_BASE_ADDR
235 #define CONFIG_SYS_INIT_RAM_SIZE    IRAM_SIZE
236
237 #define CONFIG_SYS_INIT_SP_OFFSET \
238     (CONFIG_SYS_INIT_RAM_SIZE - GENERATED_GBL_DATA_SIZE)
239 #define CONFIG_SYS_INIT_SP_ADDR \
240     (CONFIG_SYS_INIT_RAM_ADDR + CONFIG_SYS_INIT_SP_OFFSET)
241
242 /* FLASH and environment organization */
243 #define CONFIG_SYS_NO_FLASH
244
245     .....
255

```

```
256 #define CONFIG_SYS_MMC_ENV_DEV      1    /* USDHC2 */
257 #define CONFIG_SYS_MMC_ENV_PART      0    /* user area */
258 #define CONFIG_MMCROOT                "/dev/mmcblk1p2" /* USDHC2 */
259
260 #define CONFIG_CMD_BMODE
261
262 .....
275
276 /* NAND stuff */
277 #ifdef CONFIG_SYS_USE_NAND
278 #define CONFIG_CMD_NAND
279 #define CONFIG_CMD_NAND_TRIMFFS
280
281 #define CONFIG_NAND_MXS
282 #define CONFIG_SYS_MAX_NAND_DEVICE    1
283 #define CONFIG_SYS_NAND_BASE          0x40000000
284 #define CONFIG_SYS_NAND_5_ADDR_CYCLE
285 #define CONFIG_SYS_NAND_ONFI_DETECTION
286
287 /* DMA stuff, needed for GPMI/MXS NAND support */
288 #define CONFIG_APBH_DMA
289 #define CONFIG_APBH_DMA_BURST
290 #define CONFIG_APBH_DMA_BURST8
291 #endif
292
293 #define CONFIG_ENV_SIZE                SZ_8K
294 #if defined(CONFIG_ENV_IS_IN_MMC)
295 #define CONFIG_ENV_OFFSET              (12 * SZ_64K)
296 #elif defined(CONFIG_ENV_IS_IN_SPI_FLASH)
297 #define CONFIG_ENV_OFFSET              (768 * 1024)
298 #define CONFIG_ENV_SECT_SIZE          (64 * 1024)
299 #define CONFIG_ENV_SPI_BUS             CONFIG_SF_DEFAULT_BUS
300 #define CONFIG_ENV_SPI_CS              CONFIG_SF_DEFAULT_CS
301 #define CONFIG_ENV_SPI_MODE            CONFIG_SF_DEFAULT_MODE
302 #define CONFIG_ENV_SPI_MAX_HZ          CONFIG_SF_DEFAULT_SPEED
303 #elif defined(CONFIG_ENV_IS_IN_NAND)
304 #undef CONFIG_ENV_SIZE
305 #define CONFIG_ENV_OFFSET              (60 << 20)
306 #define CONFIG_ENV_SECT_SIZE          (128 << 10)
307 #define CONFIG_ENV_SIZE                CONFIG_ENV_SECT_SIZE
308 #endif
309
310
```

```
311 /* USB Configs */
312 #define CONFIG_CMD_USB
313 #ifdef CONFIG_CMD_USB
314 #define CONFIG_USB_EHCI
315 #define CONFIG_USB_EHCI_MX6
316 #define CONFIG_USB_STORAGE
317 #define CONFIG_EHCI_HCD_INIT_AFTER_RESET
318 #define CONFIG_USB_HOST_ETHER
319 #define CONFIG_USB_ETHER_ASIX
320 #define CONFIG_MXC_USB_PORTSC      (PORT_PTS_UTMI | PORT_PTS_PTW)
321 #define CONFIG_MXC_USB_FLAGS      0
322 #define CONFIG_USB_MAX_CONTROLLER_COUNT 2
323 #endif
324
325 #ifdef CONFIG_CMD_NET
326 #define CONFIG_CMD_PING
327 #define CONFIG_CMD_DHCP
328 #define CONFIG_CMD_MII
329 #define CONFIG_FEC_MXC
330 #define CONFIG_MII
331 #define CONFIG_FEC_ENET_DEV      1
332
333 #if (CONFIG_FEC_ENET_DEV == 0)
334 #define IMX_FEC_BASE              ENET_BASE_ADDR
335 #define CONFIG_FEC_MXC_PHYADDR    0x2
336 #define CONFIG_FEC_XCV_TYPE       RMII
337 #elif (CONFIG_FEC_ENET_DEV == 1)
338 #define IMX_FEC_BASE              ENET2_BASE_ADDR
339 #define CONFIG_FEC_MXC_PHYADDR    0x1
340 #define CONFIG_FEC_XCV_TYPE       RMII
341 #endif
342 #define CONFIG_ETHPRIME           "FEC"
343
344 #define CONFIG_PHYLIB
345 #define CONFIG_PHY_MICREL
346 #endif
347
348 #define CONFIG_IMX_THERMAL
349
350 #ifndef CONFIG_SPL_BUILD
351 #define CONFIG_VIDEO
352 #ifdef CONFIG_VIDEO
353 #define CONFIG_CFB_CONSOLE
```

```

354 #define CONFIG_VIDEO_MXS
355 #define CONFIG_VIDEO_LOGO
356 #define CONFIG_VIDEO_SW_CURSOR
357 #define CONFIG_VGA_AS_SINGLE_DEVICE
358 #define CONFIG_SYS_CONSOLE_IS_IN_ENV
359 #define CONFIG_SPLASH_SCREEN
360 #define CONFIG_SPLASH_SCREEN_ALIGN
361 #define CONFIG_CMD_BMP
362 #define CONFIG_BMP_16BPP
363 #define CONFIG_VIDEO_BMP_RLE8
364 #define CONFIG_VIDEO_BMP_LOGO
365 #define CONFIG_IMX_VIDEO_SKIP
366 #endif
367 #endif
368
369 #define CONFIG_IOMUX_LPSR
370
.....
375 #endif

```

从示例代码 33.2.2.1 可以看出, `mx6ull_alientek_emmc.h` 文件中基本都是“CONFIG_”开头的宏定义, 这也说明 `mx6ull_alientek_emmc.h` 文件的主要功能就是配置或者裁剪 uboot。如果需要某个功能的话就在里面添加这个功能对应的 `CONFIG_XXX` 宏即可, 如果不需要某个功能的话就删除掉对应的宏即可。我们以示例代码 33.2.2.1 为例, 详细的看一下 `mx6ull_alientek_emmc.h` 中这些宏都是什么功能。

第 14 行, 添加了头文件 `mx6_common.h`, 如果在 `mx6ull_alientek_emmc.h` 中没有发现配置某个功能或命令, 但是实际却存在的话, 可以到 `mx6_common.h` 文件里面去找一下。

第 29~39 行, 设置 DRAM 的大小, 宏 `PHYS_SDRAM_SIZE` 就是板子上 DRAM 的大小, 如果用的 NXP 官方的 9X9 EVK 开发板的话 DRAM 大小就为 256MB。否则的话默认为 512MB, 正点原子的 I.MX6U-ALPHA 开发板用的是 512MB DDR3。

第 50 行, 定义宏 `CONFIG_DISPLAY_CPUINFO`, uboot 启动的时候可以输出 CPU 信息。

第 51 行, 定义宏 `CONFIG_DISPLAY_BOARDINFO`, uboot 启动的时候可以输出板子信息。

第 54 行, `CONFIG_SYS_MALLOC_LEN` 为 malloc 内存池大小, 这里设置为 16MB。

第 56 行, 定义宏 `CONFIG_BOARD_EARLY_INIT_F`, 这样 `board_init_f` 函数就会调用 `board_early_init_f` 函数。

第 57 行, 定义宏 `CONFIG_BOARD_LATE_INIT`, 这样 `board_init_r` 函数就会调用 `board_late_init` 函数。

第 59、60 行, 使能 I.MX6ULL 的串口功能, 宏 `CONFIG_MXC_UART_BASE` 表示串口寄存器基地址, 这里使用的串口 1, 基地址为 `UART1_BASE`, `UART1_BASE` 定义在文件 `arch/arm/include/asm/arch-mx6/imx-regs.h` 中, `imx-regs.h` 是 I.MX6ULL 寄存器描述文件, 根据 `imx-regs.h` 可得到 `UART1_BASE` 的值如下:

```

UART1_BASE=(ATZ1_BASE_ADDR + 0x20000)
            =AIPS1_ARB_BASE_ADDR + 0x20000
            =0x02000000 + 0x20000

```


=0X02020000

查阅 I.MX6ULL 参考手册, UART1 的寄存器基地址正是 0X02020000, 如图 33.2.2.1 所示:

202_0000	UART Receiver Register (UART1_URXD)	32	R	0000_0000h	55.15.1/ 3615
202_0040	UART Transmitter Register (UART1_UTXD)	32	W	0000_0000h	55.15.2/ 3617
202_0080	UART Control Register 1 (UART1_UCR1)	32	R/W	0000_0000h	55.15.3/ 3618
202_0084	UART Control Register 2 (UART1_UCR2)	32	R/W	0000_0001h	55.15.4/ 3620

图 33.2.2.1 UART1 寄存器地址表

第 63、64 行, EMMC 接在 I.MX6ULL 的 USDHC2 上, 宏 CONFIG_SYS_FSL_ESDHC_ADDR 为 EMMC 所使用接口的寄存器基地址, 也就是 USDHC2 的基地址。

第 67~72 行, 跟 NAND 相关的宏, 因为 NAND 和 USDHC2 的引脚冲突, 因此如果使用 NAND 的只能使用一个 USDHC 设备(SD 卡)。如果没有使用 NAND, 那么就有两个 USDHC 设备(EMMC 和 SD 卡), 宏 CONFIG_SYS_FSL_USDHC_NUM 表示 USDHC 数量。EMMC 版本的核心版没有用到 NAND, 所以 CONFIG_SYS_FSL_USDHC_NUM=2。

第 75~81, 和 I2C 有关的宏定义, 用于控制使能哪个 I2C, I2C 的速度为多少。

第 92~96 行, NAND 的分区设置, 如果使用 NAND 的话, 默认的 NAND 分区为: "mtdparts=gpmi-nand:64m(boot),16m(kernel),16m(dtb),1m(misc),-(rootfs)", 分区结果如表 33.2.2.1 所示:

范围	大小	分区
0~63M	64M	boot(uboot)
64~79M	16M	kernel(linux 内核)
80~94M	16M	dtb(设备树)
95M	1M	misc(杂项)
96M – end	剩余的所有空间	rootfs(根文件系统)

表 33.2.2.1 NAND 分区设置

NAND 的分区是可以调整的, 比如 boot 分区我们用不了 64M 这么大, 因此可以将其改小, 其他的分区一样的。

第 98~111 行, 宏 CONFIG_MFG_ENV_SETTINGS 定义了一些环境变量, 使用 MfgTool 烧写系统时候会用到这里面的环境变量。

第 113~202 行, 通过条件编译来设置宏 CONFIG_EXTRA_ENV_SETTINGS, 宏 CONFIG_EXTRA_ENV_SETTINGS 也是设置一些环境变量, 此宏会设置 bootargs 这个环境变量, 后面我们会详细分析这个宏定义。

第 204~217 行, 设置宏 CONFIG_BOOTCOMMAND, 此宏就是设置环境变量 bootcmd 的值。后面会详细的分析这个宏定义。

第 220~222 行, 设置命令 memtest 相关宏定义, 比如使能命令 memtest, 设置 memtest 测试的内存起始地址和内存大小。

第 224 行, 宏 CONFIG_SYS_LOAD_ADDR 表示 linux kernel 在 DRAM 中的加载地址, 也就是 linux kernel 在 DRAM 中的存储首地址, CONFIG_LOADADDR=0X80800000。

第 225 行, 宏 CONFIG_SYS_HZ 为系统时钟频率, 这里为 1000Hz。

第 227 行, 宏 CONFIG_STACKSIZE 为栈大小, 这里为 128KB。

第 230 行, 宏 CONFIG_NR_DRAM_BANKS 为 DRAM BANK 的数量, I.MX6ULL 只有一个 DRAM BANK, 我们也只用到了一个 BANK, 所以为 1。

第 231 行, 宏 PHYS_SDRAM 为 I.MX6ULL 的 DRAM 控制器 MMDC0 所管辖的 DRAM 范围起始地址, 也就是 0X80000000。

第 233 行, 宏 CONFIG_SYS_SDRAM_BASE 为 DRAM 的起始地址。

第 234 行, 宏 CONFIG_SYS_INIT_RAM_ADDR 为 I.MX6ULL 内部 IRAM 的起始地址(也就是 OCRAM 的起始地址), 为 0X00900000。

第 235 行, 宏 CONFIG_SYS_INIT_RAM_SIZE 为 I.MX6ULL 内部 IRAM 的大小(OCRAM 的大小), 为 0X00040000=128KB。

第 237~240 行, 宏 CONFIG_SYS_INIT_SP_OFFSET 和 CONFIG_SYS_INIT_SP_ADDR 与初始 SP 有关, 第一个为初始 SP 偏移, 第二个为初始 SP 地址。

第 256 行, 宏 CONFIG_SYS_MMC_ENV_DEV 为默认的 MMC 设备, 这里默认为 USDHC2, 也就是 EMMC。

第 257 行, 宏 CONFIG_SYS_MMC_ENV_PART 为模式分区, 默认为第 0 个分区。

第 258 行, 宏 CONFIG_MMROOT 设置进入 linux 系统的根文件系统所在的分区, 这里设置为 "/dev/mmcblk1p2", 也就是 EMMC 设备的第 2 个分区。第 0 个分区保存 uboot, 第 1 个分区保存 linux 镜像和设备树, 第 2 个分区为 Linux 系统的根文件系统。

第 277~291 行, 与 NAND 有关的宏定义, 如果使用 NAND 的话。

第 293 行, 宏 CONFIG_ENV_SIZE 为环境变量大小, 默认为 8KB。

第 294~308 行, 宏 CONFIG_ENV_OFFSET 为环境变量偏移地址, 这里的偏移地址是相对于存储器的首地址。如果环境变量保存在 EMMC 中的话, 环境变量偏移地址为 12*64KB。如果环境变量保存在 SPI FLASH 中的话, 偏移地址为 768*1024。如果环境变量保存在 NAND 中的话, 偏移地址为 60<<20(60MB), 并且重新设置环境变量的大小为 128KB。

第 312~323 行, 与 USB 相关的宏定义。

第 325~342 行, 与网络相关的宏定义, 比如使能 dhcp、ping 等命令。第 331 行的宏 CONFIG_FEC_ENET_DEV 指定 uboot 所使用的网口, I.MX6ULL 有两个网口, 为 0 的时候使用 ENET1, 为 1 的时候使用 ENET2。宏 IMX_FEC_BASE 为 ENET 接口的寄存器首地址, 宏 CONFIG_FEC_MXC_PHYADDR 为网口 PHY 芯片的地址。宏 CONFIG_FEC_XCV_TYPE 为 PHY 芯片所使用的接口类型, I.MX6U-ALPHA 开发板的两个 PHY 都使用的 RMII 接口。

第 344~END, 剩下的都是一些配置宏, 比如 CONFIG_VIDEO 宏用于开启 LCD, CONFIG_VIDEO_LOGO 使能 LOGO 显示, CONFIG_CMD_BMP 使能 BMP 图片显示指令。这样就可以在 uboot 中显示图片了, 一般用于显示 logo。

关于 mx6ull_alientek_emmc.h 就讲解到这里, 其中以 CONFIG_CMD 开头的宏都是用于使能相应命令的, 其他的以 CONFIG 开头的宏都是完成一些配置功能的。以后会频繁的和 mx6ull_alientek_emmc.h 这个文件打交道。

33.2.3 添加开发板对应的板级文件夹

uboot 中每个板子都有一个对应的文件夹来存放板级文件, 比如开发板上外设驱动文件等等。NXP 的 I.MX 系列芯片的所有板级文件夹都存放在 board/freescale 目录下, 在这个目录下有个名为 mx6ullevk 的文件夹, 这个文件夹就是 NXP 官方 I.MX6ULL EVK 开发板的板级文件夹。复制 mx6ullevk, 将其重命名为 mx6ull_alientek_emmc, 命令如下:

```
cd board/freescale/  
cp mx6ullevk/ -r mx6ull_alientek_emmc
```

进入 mx6ull_alientek_emmc 目录中, 将其中的 mx6ullevk.c 文件重命名为 mx6ull_alientek_emmc.c, 命令如下:

```
cd mx6ull_alientek_emmc
mv mx6ullevk.c mx6ull_alientek_emmc.c
```

我们还需要对 mx6ull_alientek_emmc 目录下的文件做一些修改:

1、修改 mx6ull_alientek_emmc 目录下的 Makefile 文件

将 mx6ull_alientek_emmc 下的 Makefile 文件内容改为如下所示:

示例代码 33.2.3.1 Makefile 文件

```
1 # (C) Copyright 2015 Freescale Semiconductor, Inc.
2 #
3 # SPDX-License-Identifier: GPL-2.0+
4 #
5
6 obj-y := mx6ull_alientek_emmc.o
7
8 extra-$(CONFIG_USE_PLUGIN) := plugin.bin
9 $(obj)/plugin.bin: $(obj)/plugin.o
10     $(OBJCOPY) -O binary --gap-fill 0xff $< $@
```

重点是第 6 行的 obj-y, 改为 mx6ull_alientek_emmc.o, 这样才会编译 mx6ull_alientek_emmc.c 这个文件。

2、修改 mx6ull_alientek_emmc 目录下的 imximage.cfg 文件

将 imximage.cfg 中的下面一句:

```
PLUGIN board/freescale/mx6ullevk/plugin.bin 0x00907000
```

改为:

```
PLUGIN board/freescale/mx6ull_alientek_emmc /plugin.bin 0x00907000
```

3、修改 mx6ull_alientek_emmc 目录下的 Kconfig 文件

修改 Kconfig 文件, 修改后的内容如下:

示例代码 33.2.3.2 Kconfig 文件

```
1 if TARGET_MX6ULL_ALIENTEK_EMMC
2
3 config SYS_BOARD
4     default "mx6ull_alientek_emmc"
5
6 config SYS_VENDOR
7     default "freescale"
8
9 config SYS_SOC
10    default "mx6"
11
12 config SYS_CONFIG_NAME
13    default "mx6ull_alientek_emmc"
14
15 endif
```

4、修改 mx6ull_alientek_emmc 目录下的 MAINTAINERS 文件

修改 MAINTAINERS 文件，修改后的内容如下：

```
1 MX6ULL_ALIENTEK_EMMC BOARD
2 M:   Peng Fan <peng.fan@nxp.com>
3 S:   Maintained
4 F:   board/freescale/mx6ull_alientek_emmc/
5 F:   include/configs/mx6ull_alientek_emmc.h
```

33.2.4 修改 U-Boot 图形界面配置文件

uboot 是支持图形界面配置，关于 uboot 的图形界面配置下一章会详细的讲解。修改文件 arch/arm/cpu/armv7/mx6/Kconfig(如果用的 I.MX6UL 的话，应该修改 arch/arm/Kconfig 这个文件)，在 207 行加入如下内容：

示例代码 33.2.4.1 Kconfig 文件

```
1 config TARGET_MX6ULL_ALIENTEK_EMMC
2     bool "Support mx6ull_alientek_emmc"
3     select MX6ULL
4     select DM
5     select DM_THERMAL
```

在最后一行的 endif 的前一行添加如下内容：

示例代码 33.2.4.2 Kconfig 文件

```
1 source "board/freescale/mx6ull_alientek_emmc/Kconfig"
```

添加完成以后的 Kconfig 文件如图 33.2.4.1 所示：

```
201 config TARGET_MX6ULL_9X9_EVK
202     bool "Support mx6ull_9x9_evk"
203     select MX6ULL
204     select DM
205     select DM_THERMAL
206
207 config TARGET_MX6ULL_ALIENTEK_EMMC
208     bool "Support mx6ull_alientek_emmc"
209     select MX6ULL
210     select DM
211     select DM_THERMAL
212
212 source "board/freescale/mx6sxscm/Kconfig"
293
294 source "board/freescale/mx6ull_alientek_emmc/Kconfig"
295
296 endif
```

图 33.2.4.1 修改后的 Kconfig 文件

到此为止，I.MX6U-ALPHA 开发板就已经添加到 uboot 中了，接下来就是编译这个新添加的开发板。

33.2.5 使用新添加的板子配置编译 uboot

在 uboot 根目录下新建一个名为 mx6ull_alientek_emmc.sh 的 shell 脚本，在这个 shell 脚本里面输入如下内容：

示例代码 33.2.5.1 mx6ull_alientek_emmc.sh 脚本文件

```

1 #!/bin/bash
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- (加空格)
    mx6ull_alientek_emmc_defconfig
4 make V=1 ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j16

```

第3行我们使用的默认配置文件就是33.2.1节中新建的mx6ull_alientek_emmc_defconfig这个配置文件。给予mx6ull_alientek_emmc.sh可执行权限，然后运行脚本来完成编译，命令如下：

```

chmod 777 mx6ull_alientek_emmc.sh      //给予可执行权限，一次即可
./mx6ull_alientek_emmc.sh              //运行脚本编译 uboot

```

等待编译完成，编译完成以后输入如下命令，查看一下33.2.2小节中添加的mx6ull_alientek_emmc.h这个头文件有没有被引用。

```
grep -nR "mx6ull_alientek_emmc.h"
```

如果有很多文件都引用了mx6ull_alientek_emmc.h这个头文件，那就说明新板子添加成功，如图33.2.5.1所示：

```

zuozhongkai@ubuntu:~/linux/IMX6ULL/uboot/temp/uboot-imx-rel_imx_4.1.15_2.1.0_ga/arch$ grep -nR "mx6ull_alientek_emmc.h"
arm/lib/.reset.o.cmd:133: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.cache-cp15.o.cmd:139: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.relocate.o.cmd:44: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.crt0.o.cmd:47: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.stack.o.cmd:137: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.vectors.o.cmd:42: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.eabi_compat.o.cmd:133: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.bootm-fdt.o.cmd:135: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.asm-offsets.s.cmd:138: include/configs/mx6ull_alientek_emmc.h \
arm/lib/.interrupts.o.cmd:135: include/configs/mx6ull_alientek_emmc.h \

```

图 33.2.5.1 查找结果

编译完成以后就使用 imxdownload 将新编译出来的 u-boot.bin 烧写到 SD 卡中测试，SecureCRT 输出结果如图33.2.5.2所示：

```

serial-com12 x
U-Boot 2016.03 (May 12 2019 - 22:17:47 +0800)

CPU:   Freescale i.MX6ULL rev1.1 528 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 48C
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C:   ready
DRAM:   512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
unsupported panel TFT7016
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc0 is current device
Net:   Board Net Initialization Failed
No ethernet found.
Normal Boot
Hit any key to stop autoboot:  0
=>

```

图 33.2.5.1 uboot 启动过程

从图33.2.5.1可以看出，此时的Board还是“MX6ULL 14x14 EVK”，因为我们参考的NXP官方的I.MX6ULL开发板来添加自己的开发板。如果接了LCD屏幕的话会发现LCD屏幕并没有显示NXP的logo，而且从图33.2.5.1可以看出此时的网络同样也没识别出来。前面已经说了，默认uboot中的LCD驱动和网络驱动在正点原子的I.MX6U-ALPHA开发板上是有问题的，需要修改。

33.2.6 LCD 驱动修改

一般 uboot 中修改驱动基本都是在 xxx.h 和 xxx.c 这两个文件中进行的, xxx 为板子名称, 比如 mx6ull_alientek_emmc.h 和 mx6ull_alientek_emmc.c 这两个文件。

一般修改 LCD 驱动重点注意以下几点:

- ①、LCD 所使用的 GPIO, 查看 uboot 中 LCD 的 IO 配置是否正确。
- ②、LCD 背光引脚 GPIO 的配置。
- ③、LCD 配置参数是否正确。

正点原子的 I.MX6U-ALPHA 开发板 LCD 原理图和 NXP 官方 I.MX6ULL 开发板一致, 也就是 LCD 的 IO 和背光 IO 都是一样的, 所以 IO 部分就不用修改了。需要修改的之后 LCD 参数, 打开文件 mx6ull_alientek_emmc.c, 找到如下所示内容:

示例代码 33.2.6.1 LCD 驱动参数

```
1 struct display_info_t const displays[] = {{
2     .bus = MX6UL_LCDIF1_BASE_ADDR,
3     .addr = 0,
4     .pixfmt = 24,
5     .detect = NULL,
6     .enable = do_enable_parallel_lcd,
7     .mode = {
8         .name           = "TFT43AB",
9         .xres           = 480,
10        .yres           = 272,
11        .pixclock       = 108695,
12        .left_margin    = 8,
13        .right_margin   = 4,
14        .upper_margin   = 2,
15        .lower_margin   = 4,
16        .hsync_len      = 41,
17        .vsync_len      = 10,
18        .sync            = 0,
19        .vmode           = FB_VMODE_NONINTERLACED
20 } } };
```

示例代码 33.2.6.1 中定义了一个变量 displays, 类型为 display_info_t, 这个结构体是 LCD 信息结构体, 其中包括了 LCD 的分辨率, 像素格式, LCD 的各个参数等。display_info_t 定义在文件 arch/arm/include/asm/imx-common/video.h 中, 定义如下:

示例代码 33.2.6.2 display_info 结构体

```
1 struct display_info_t {
2     int bus;
3     int addr;
4     int pixfmt;
5     int (*detect)(struct display_info_t const *dev);
6     void (*enable)(struct display_info_t const *dev);
7     struct fb_videomode mode;
```



```
8 };
```

pixfmt 是像素格式，也就是一个像素点是多少位，如果是 RGB565 的话就是 16 位，如果是 888 的话就是 24 位，一般使用 RGB888。结构体 display_info_t 还有个 mode 成员变量，此成员变量也是个结构体，为 fb_videomode，定义在文件 include/linux/fb.h 中，定义如下：

示例代码 33.2.6.3 fb_videomode 结构体

```
1 struct fb_videomode {
2     const char *name;      /* optional */
3     u32 refresh;           /* optional */
4     u32 xres;
5     u32 yres;
6     u32 pixclock;
7     u32 left_margin;
8     u32 right_margin;
9     u32 upper_margin;
10    u32 lower_margin;
11    u32 hsync_len;
12    u32 vsync_len;
13    u32 sync;
14    u32 vmode;
15    u32 flag;
16 };
```

结构体 fb_videomode 里面的成员变量为 LCD 的参数，这些成员变量函数如下：

name: LCD 名字，要和环境变量中的 panel 相等。

xres、yres: LCD X 轴和 Y 轴像素数量。

pixclock: 像素时钟，每个像素时钟周期的长度，单位为皮秒。

left_margin: HBP，水平同步后肩。

right_margin: HFP，水平同步前肩。

upper_margin: VBP，垂直同步后肩。

lower_margin: VFP，垂直同步前肩。

hsync_len: HSPW，行同步脉宽。

vsync_len: VSPW，垂直同步脉宽。

vmode: 大多数使用 FB_VMODE_NONINTERLACED，也就是不使用隔行扫描。

可以看出，这些参数和我们第二十四章讲解 RGB LCD 的时候参数基本一样，唯一不同的像素时钟 pixclock 的含义不同，以正点原子的 7 寸 1024*600 分辨率的屏幕(ATK7016)为例，屏幕要求的像素时钟为 51.2MHz，因此：

```
pixclock=(1/51200000)*10^12=19531
```

在根据其他的屏幕参数，可以得出 ATK7016 屏幕的配置参数如下：

示例代码 33.2.6.4 ATK7016 屏幕配置参数

```
1 struct display_info_t const displays[] = {{
2     .bus = MX6UL_LCDIF1_BASE_ADDR,
3     .addr = 0,
4     .pixfmt = 24,
5     .detect = NULL,
```



```

6     .enable = do_enable_parallel_lcd,
7     .mode   = {
8         .name           = "TFT7016",
9         .xres           = 1024,
10        .yres           = 600,
11        .pixclock       = 19531,
12        .left_margin    = 140,          //HBPD
13        .right_margin   = 160,          //HFPD
14        .upper_margin   = 20,           //VBPD
15        .lower_margin   = 12,           //VFBD
16        .hsync_len       = 20,           //HSPW
17        .vsync_len       = 3,            //VSPW
18        .sync            = 0,
19        .vmode            = FB_VMODE_NONINTERLACED
20    } } };

```

使用示例代码 33.2.6.4 中的屏幕参数替换掉 mx6ull_alientek_emmc.c 中 uboot 默认的屏幕参数。

打开 mx6ull_alientek_emmc.h, 找到所有如下语句:

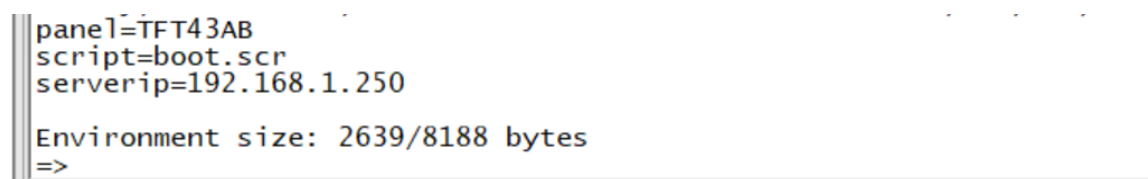
```
panel=TFT43AB
```

将其改为:

```
panel=TFT7016
```

也就是设置 panel 为 TFT7016, panel 的值要与示例代码 33.2.6.4 中的.name 成员变量的值一致。修改完成以后重新编译一遍 uboot 并烧写到 SD 中启动。

重启以后 LCD 驱动一般就会工作正常了, LCD 上回显示 NXP 的 logo。但是有可能会遇到 LCD 并没有工作, 还是黑屏, 这是什么原因呢? 在 uboot 命令模式输入 “print” 来查看环境变量 panel 的值, 会发现 panel 的值要是 TFT43AB(或其他, 反正不是 TFT7016), 如图 33.2.6.1 所示:



```

panel=TFT43AB
script=boot.scr
serverip=192.168.1.250

Environment size: 2639/8188 bytes
=>

```

图 33.2.6.1 panel 的值

这是因为之前有将环境变量保存到 EMMC 中, uboot 启动以后会先从 EMMC 中读取环境变量, 如果 EMMC 中没有环境变量的话才会使用 mx6ull_alientek_emmc.h 中的默认环境变量。如果 EMMC 中的环境变量 panel 不等于 TFT7016, 那么 LCD 显示肯定不正常, 我们只需要在 uboot 中修改 panel 的值为 TFT7016 即可, 在 uboot 的命令模式下输入如下命令:

```

setenv panel TFT7016
saveenv

```

上述命令修改环境变量 panel 为 TFT7016, 然后保存, 重启 uboot, 此时 LCD 驱动就工作正常了。如果 LCD 还是没有正常工作的, 那就要检查自己哪里有没有改错, 或者还有哪里没有修改。

33.2.7 V2.4 版本以前底板网络驱动修改

注意!正点原子 I.MX6U-ALPHA 开发板 V2.4 版本以前的底板使用的网络 PHY 为 LAN8720, V2.4 及其以后的版本使用的网络 PHY 为 SR8201F, 而且网络 PHY 地址有改变, 大家一定要看准自己所使用的底板版本号!

如果用的是 Mini 板, 那么 V2.2 及以后版本 Mini 开发板请参考 V2.4 版本的 ALPHA 开发板移植。

查看自己所使用的 ALPHA 底板版本号, 如果是 V2.4 以前版本, 那么就看本结!

1、I.MX6U-ALPHA 开发板网络简介

I.MX6UL/ULL 内部有个以太网 MAC 外设, 也就是 ENET, 需要外接一个 PHY 芯片来实现网络通信功能, 也就是内部 MAC+外部 PHY 芯片的方案。大家可能听过 DM9000 这个网络芯片, 在一些没有内部 MAC 的 CPU 中, 比如三星的 2440, 4412 等, 就会采用 DM9000 来实现联网功能。DM9000 提供了一个类似 SRAM 的访问接口, 主控 CPU 通过这个接口即可与 DM9000 进行通信, DM9000 就是一个 MAC+PHY 芯片。这个方案就相当于外部 MAC+外部 PHY, 那么 I.MX6U 这样的内部 MAC+PHY 芯片与 DM9000 方案比有什么优势吗? 那优势大了去了! 首先就是通信效率和速度, 一般 SOC 内部的 MAC 是带有一个专用 DMA 的, 专门用于处理网络数据包, 采用 SRAM 来读写 DM9000 的速度是压根就没法和内部 MAC+外部 PHY 芯片的速度比。采用外部 DM9000 完全是无奈之举, 谁让 2440, 4412 这些芯片内部没有以太网外设呢, 现在又想用有线网络, 没有办法只能找个 DM9000 的方案。从这里也可以看出, 三星的 2440、4412 这些芯片设计之初就不是给工业产品用的, 他们是给消费类电子使用的, 比如手机、平板等, 手机或平板要上网, 可以通过 WIFI 或者 4G, 我是没有见过哪个手机或者平板上网是要接根网线的。正点原子的 I.MX6U-ALPHA 开发板也可以通过 WIFI 或者 4G 上网, 这个是后话了。

I.MX6UL/ULL 有两个网络接口 ENET1 和 ENET2, 正点原子的 I.MX6U-ALPHA 开发板提供了这两个网络接口, 其中 ENET1 和 ENET2 都使用 LAN8720A 作为 PHY 芯片。NXP 官方的 I.MX6ULL EVK 开发板使用 KSZ8081 这颗 PHY 芯片, LAN8720A 相比 KSZ8081 具有体积小、外围器件少、价格便宜等优点。直接使用 KSZ8081 固然可以, 但是我们在实际的产品中不一定会使用 KSZ8081, 有时候为了降低成本会选择其他的 PHY 芯片, 这个时候就有个问题: 换了 PHY 芯片以后网络驱动怎么办? 为此, 正点原子的 I.MX6U-ALPHA 开发板将 ENET1 和 ENET2 的 PHY 换成了 LAN8720A, 这样就可以给大家讲解更换 PHY 芯片以后如何调整网络驱动, 使网络工作正常。

I.MX6U-ALPHA 开发板 ENET1 的网络原理图如图 33.2.7.1 所示:

ENET1

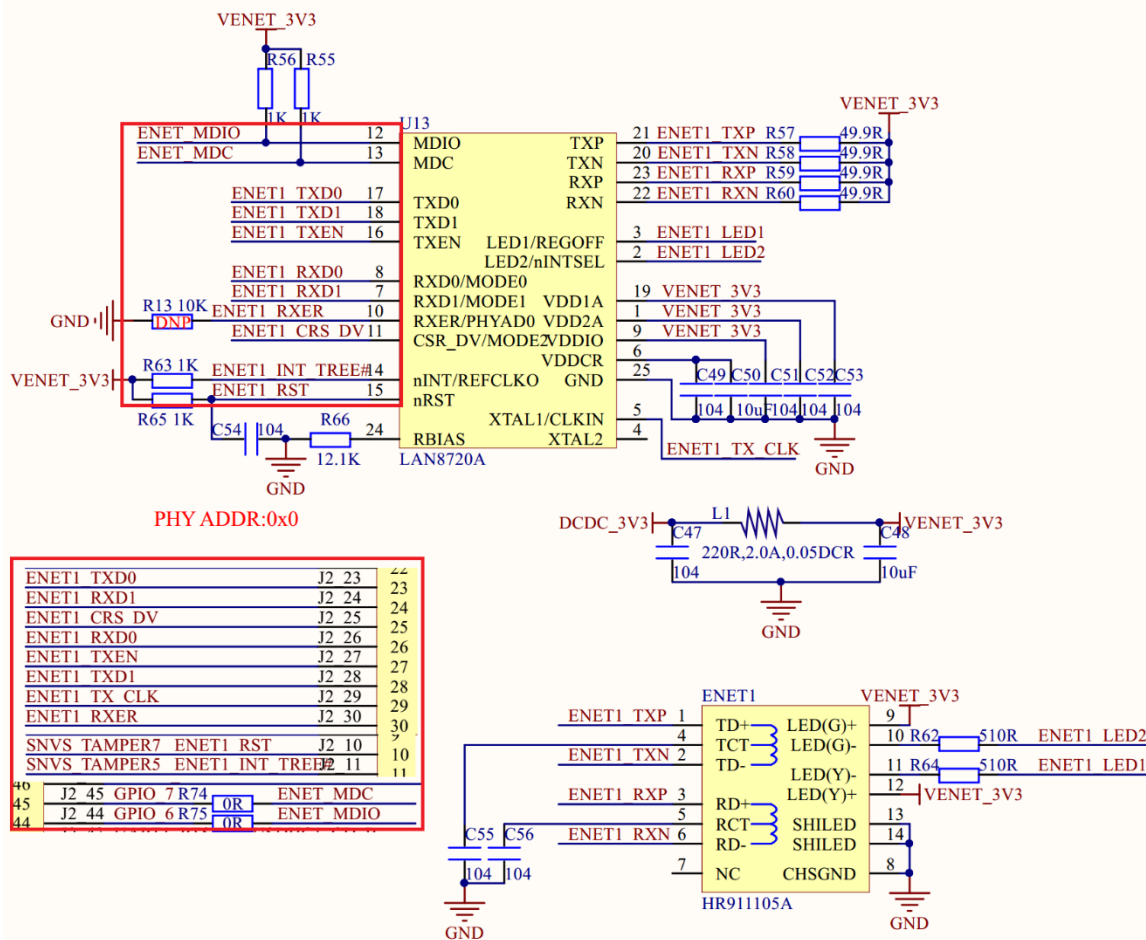


图 33.2.7.1 ENET1 原理图

ENET1 的网络 PHY 芯片为 LAN8720A，通过 RMII 接口与 I.MX6ULL 相连，正点原子 I.MX6U-ALPHA 开发板的 ENET1 引脚与 NXP 官方的 I.MX6ULL EVK 开发板基本一样，唯独复位引脚不同。从图 33.2.7.1 可以看出，正点原子 I.MX6U-ALPHA 开发板的 ENET1 复位引脚 ENET1_RST 接到了 I.M6ULL 的 SNVS_TAMPER7 这个引脚上。

LAN8720A 内部是有寄存器的，I.MX6ULL 会读取 LAN8720 内部寄存器来判断当前的物理链接状态、连接速度(10M 还是 100M)和双工状态(半双工还是全双工)。I.MX6ULL 通过 MDIO 接口来读取 PHY 芯片的内部寄存器，MDIO 接口有两个引脚，ENET_MDC 和 ENET_MDIO，ENET_MDC 提供时钟，ENET_MDIO 进行数据传输。一个 MDIO 接口可以管理 32 个 PHY 芯片，同一个 MDIO 接口下的这些 PHY 使用不同的器件地址来做区分，MIDO 接口通过不同的器件地址即可访问到相应的 PHY 芯片。I.MX6U-ALPHA 开发板 ENET1 上连接的 LAN8720A 器件地址为 0X0，所示我们要修改 ENET1 网络驱动的话重点就三点：

- ①、ENET1 复位引脚初始化。
- ②、LAN8720A 的器件 ID。
- ③、LAN8720 驱动

再来看一下 ENET2 的原理图，如图 33.2.7.2 所示：

ENET2

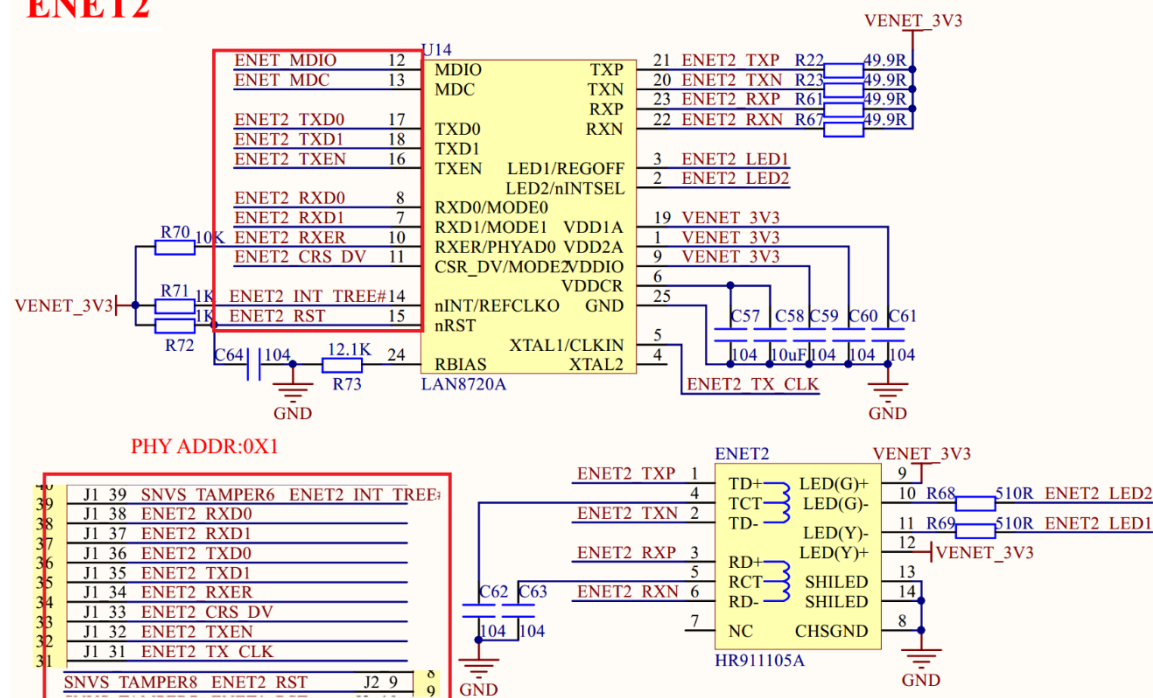


图 33.2.7.2 ENET2 原理图

关于 ENET2 网络驱动的修改也注意一下三点:

①、ENET2 的复位引脚, 从图 33.2.7.2 可以看出, ENET2 的复位引脚 ENET2_RST 接到了 I.MX6ULL 的 SNVS_TAMPER8 上。

②、ENET2 所使用的 PHY 芯片器件地址, 从图 33.2.7.2 可以看出, PHY 器件地址为 0X1。

③、LAN8720 驱动, ENET1 和 ENET2 都使用的 LAN8720, 所以驱动肯定是一样的。

2、网络 PHY 地址修改

首先修改 uboot 中的 ENET1 和 ENET2 的 PHY 地址和驱动, 打开 mx6ull_alientek_emmc.h 这个文件, 找到如下代码:

示例代码 33.2.7.1 网络默认 ID 配置参数

```

325 #ifndef CONFIG_CMD_NET
326 #define CONFIG_CMD_PING
327 #define CONFIG_CMD_DHCP
328 #define CONFIG_CMD_MII
329 #define CONFIG_FEC_MXC
330 #define CONFIG_MII
331 #define CONFIG_FEC_ENET_DEV 1
332
333 #if (CONFIG_FEC_ENET_DEV == 0)
334 #define IMX_FEC_BASE ENET_BASE_ADDR
335 #define CONFIG_FEC_MXC_PHYADDR 0x2
336 #define CONFIG_FEC_XCV_TYPE RMII
337 #elif (CONFIG_FEC_ENET_DEV == 1)
338 #define IMX_FEC_BASE ENET2_BASE_ADDR
339 #define CONFIG_FEC_MXC_PHYADDR 0x1

```

```

340 #define CONFIG_FEC_XCV_TYPE          RMII
341 #endif
342 #define CONFIG_ETHPRIME                "FEC"
343
344 #define CONFIG_PHYLIB
345 #define CONFIG_PHY_MICREL
346 #endif

```

第 331 行的宏 CONFIG_FEC_ENET_DEV 用于选择使用哪个网口，默认为 1，也就是选择 ENET2。第 335 行为 ENET1 的 PHY 地址，默认是 0x2，第 339 行为 ENET2 的 PHY 地址，默认为 0x1。根据前面的分析可知，正点原子的 I.MX6U-ALPHA 开发板 ENET1 的 PHY 地址为 0x0，ENET2 的 PHY 地址为 0x1，所以需要将第 335 行的宏 CONFIG_FEC_MXC_PHYADDR 改为 0x0。

第 345 行定了一个宏 CONFIG_PHY_MICREL，此宏用于使能 uboot 中 Micrel 公司的 PHY 驱动，KSZ8081 这颗 PHY 芯片就是 Micrel 公司生产的，不过 Micrel 已经被 Microchip 收购了。如果要使用 LAN8720A，那么就得将 CONFIG_PHY_MICREL 改为 CONFIG_PHY_SMSC，也就是使能 uboot 中的 SMSC 公司中的 PHY 驱动，因为 LAN8720A 就是 SMSC 公司生产的。所以示例代码 33.2.7.1 有三处要修改：

- ①、修改 ENET1 网络 PHY 的地址。
- ②、修改 ENET2 网络 PHY 的地址。
- ③、使能 SMSC 公司的 PHY 驱动。

修改后的网络 PHY 地址参数如下所示：

示例代码 33.2.7.2 网络 PHY 地址配置参数

```

325 #ifdef CONFIG_CMD_NET
326 #define CONFIG_CMD_PING
327 #define CONFIG_CMD_DHCP
328 #define CONFIG_CMD_MII
329 #define CONFIG_FEC_MXC
330 #define CONFIG_MII
331 #define CONFIG_FEC_ENET_DEV          1
332
333 #if (CONFIG_FEC_ENET_DEV == 0)
334 #define IMX_FEC_BASE                  ENET_BASE_ADDR
335 #define CONFIG_FEC_MXC_PHYADDR        0x0
336 #define CONFIG_FEC_XCV_TYPE          RMII
337 #elif (CONFIG_FEC_ENET_DEV == 1)
338 #define IMX_FEC_BASE                  ENET2_BASE_ADDR
339 #define CONFIG_FEC_MXC_PHYADDR        0x1
340 #define CONFIG_FEC_XCV_TYPE          RMII
341 #endif
342 #define CONFIG_ETHPRIME                "FEC"
343
344 #define CONFIG_PHYLIB
345 #define CONFIG_PHY_SMSC

```

346 #endif

3、删除 uboot 中 74LV595 的驱动代码

uboot 中网络 PHY 芯片地址修改完成以后就是网络复位引脚的驱动修改了，打开 mx6ull_alientek_emmc.c，找到如下代码：

示例代码 33.2.7.3 74LV595 引脚

```
#define IOX_SDI    IMX_GPIO_NR(5, 10)
#define IOX_STCP   IMX_GPIO_NR(5, 7)
#define IOX_SHCP   IMX_GPIO_NR(5, 11)
#define IOX_OE     IMX_GPIO_NR(5, 8)
```

示例代码 33.2.7.3 中以 IOX 开头的宏定义是 74LV595 的相关 GPIO，因为 NXP 官方 I.MX6ULL EVK 开发板使用 74LV595 来扩展 IO，两个网络的复位引脚就是由 74LV595 来控制的。正点原子的 I.MX6U-ALPHA 开发板并没有使用 74LV595，因此我们将示例代码 33.2.7.3 中的代码删除掉，替换为如下所示代码：

示例代码 33.2.7.4 修改后的网络引脚

```
#define ENET1_RESET IMX_GPIO_NR(5, 7)
#define ENET2_RESET IMX_GPIO_NR(5, 8)
```

ENET1 的复位引脚连接到 SNVS_TAMPER7 上，对应 GPIO5_IO07，ENET2 的复位引脚连接到 SNVS_TAMPER8 上，对应 GPIO5_IO08。

继续在 mx6ull_alientek_emmc.c 中找到如下代码：

示例代码 33.2.7.5 74LV595 引脚配置

```
static iomux_v3_cfg_t const iox_pads[] = {
    /* IOX_SDI */
    MX6_PAD_BOOT_MODE0__GPIO5_IO10 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_SHCP */
    MX6_PAD_BOOT_MODE1__GPIO5_IO11 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_STCP */
    MX6_PAD_SNVS_TAMPER7__GPIO5_IO07 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_nOE */
    MX6_PAD_SNVS_TAMPER8__GPIO5_IO08 | MUX_PAD_CTRL(NO_PAD_CTRL),
};
```

同理，示例代码 33.2.7.5 是 74LV595 的 IO 配置参数结构体，将其删除掉。继续在 mx6ull_alientek_emmc.c 中找到函数 iox74lv_init，如下所示：

示例代码 33.2.7.6 74LV595 初始化函数

```
static void iox74lv_init(void)
{
    int i;

    gpio_direction_output(IOX_OE, 0);

    for (i = 7; i >= 0; i--) {
        gpio_direction_output(IOX_SHCP, 0);
        gpio_direction_output(IOX_SDI, seq[qn_output[i]][0]);
        udelay(500);
    }
}
```

```

        gpio_direction_output(IOX_SHCP, 1);
        udelay(500);
    }

    .....
    /*
     * shift register will be output to pins
     */
    gpio_direction_output(IOX_STCP, 1);
};

void iox74lv_set(int index)
{
    int i;

    for (i = 7; i >= 0; i--) {
        gpio_direction_output(IOX_SHCP, 0);

        if (i == index)
            gpio_direction_output(IOX_SDI, seq[qn_output[i]][0]);
        else
            gpio_direction_output(IOX_SDI, seq[qn_output[i]][1]);
        udelay(500);
        gpio_direction_output(IOX_SHCP, 1);
        udelay(500);
    }
    .....
    /*
     * shift register will be output to pins
     */
    gpio_direction_output(IOX_STCP, 1);
};

```

iox74lv_init 函数是 74LV595 的初始化函数，iox74lv_set 函数用于控制 74LV595 的 IO 输出电平，将这两个函数全部删除掉！

在 mx6ull_alientek_emmc.c 中找到 board_init 函数，此函数是板子初始化函数，会被 board_init_r 调用，board_init 函数内容如下：

示例代码 33.2.7.7 board_init 函数

```

int board_init(void)
{
    .....
    imx_iomux_v3_setup_multiple_pads(iox_pads, ARRAY_SIZE(iox_pads));
    iox74lv_init();
    .....
}

```



```
return 0;
}
```

board_init 会调用 imx_iomux_v3_setup_multiple_pads 和 iox74lv_init 这两个函数来初始化 74lv595 的 GPIO，将这两行删除掉。至此，mx6ull_alientek_emmc.c 中关于 74LV595 芯片的驱动代码都删除掉了，接下来就是添加 I.MX6U-ALPHA 开发板两个网络复位引脚了。

4、添加 I.MX6U-ALPHA 开发板网络复位引脚驱动

在 mx6ull_alientek_emmc.c 中找到如下所示代码：

示例代码 33.2.7.8 默认网络 IO 结构体数组

```
640 static iomux_v3_cfg_t const fec1_pads[] = {
641     MX6_PAD_GPIO1_IO06__ENET1_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
642     MX6_PAD_GPIO1_IO07__ENET1_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
643     .....
649     MX6_PAD_ENET1_RX_ER__ENET1_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
650     MX6_PAD_ENET1_RX_EN__ENET1_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
651 };
652
653 static iomux_v3_cfg_t const fec2_pads[] = {
654     MX6_PAD_GPIO1_IO06__ENET2_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
655     MX6_PAD_GPIO1_IO07__ENET2_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
656     .....
664     MX6_PAD_ENET2_RX_EN__ENET2_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
665     MX6_PAD_ENET2_RX_ER__ENET2_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
666 };
```

结构体数组 fec1_pads 和 fec2_pads 是 ENET1 和 ENET2 这两个网口的 IO 配置参数，在这两个数组中添加两个网口的复位 IO 配置参数，完成以后如下所示：

示例代码 33.2.7.9 添加网络复位 IO 后的结构体数组

```
640 static iomux_v3_cfg_t const fec1_pads[] = {
641     MX6_PAD_GPIO1_IO06__ENET1_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
642     MX6_PAD_GPIO1_IO07__ENET1_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
643     .....
649     MX6_PAD_ENET1_RX_ER__ENET1_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
650     MX6_PAD_ENET1_RX_EN__ENET1_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
651     MX6_PAD_SNVS_TAMPER7__GPIO5_IO07 | MUX_PAD_CTRL(NO_PAD_CTRL),
652 };
653
654 static iomux_v3_cfg_t const fec2_pads[] = {
655     MX6_PAD_GPIO1_IO06__ENET2_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
656     MX6_PAD_GPIO1_IO07__ENET2_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
657     .....
665     MX6_PAD_ENET2_RX_EN__ENET2_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
666     MX6_PAD_ENET2_RX_ER__ENET2_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
667     MX6_PAD_SNVS_TAMPER8__GPIO5_IO08 | MUX_PAD_CTRL(NO_PAD_CTRL),
668 };
```

示例代码 33.2.7.9 中, 第 651 行和 667 行分别是 ENET1 和 ENET2 的复位 IO 配置参数。继续在文件 mx6ull_alientek_emmc.c 中找到函数 setup_iomux_fec, 此函数默认代码如下:

示例代码 33.2.7.10 setup_iomux_fec 函数默认代码

```
668 static void setup_iomux_fec(int fec_id)
669 {
670     if (fec_id == 0)
671         imx_iomux_v3_setup_multiple_pads(fec1_pads,
672             ARRAY_SIZE(fec1_pads));
673     else
674         imx_iomux_v3_setup_multiple_pads(fec2_pads,
675             ARRAY_SIZE(fec2_pads));
676 }
```

函数 setup_iomux_fec 就是根据 fec1_pads 和 fec2_pads 这两个网络 IO 配置数组来初始化 LMX6ULL 的网络 IO。我们需要在其中添加网络复位 IO 的初始化代码, 并且复位一下 PHY 芯片, 修改后的 setup_iomux_fec 函数如下:

示例代码 33.2.7.11 修改后的 setup_iomux_fec 函数

```
668 static void setup_iomux_fec(int fec_id)
669 {
670     if (fec_id == 0)
671     {
672
673         imx_iomux_v3_setup_multiple_pads(fec1_pads,
674             ARRAY_SIZE(fec1_pads));
675
676         gpio_direction_output(ENET1_RESET, 1);
677         gpio_set_value(ENET1_RESET, 0);
678         mdelay(20);
679         gpio_set_value(ENET1_RESET, 1);
680     }
681     else
682     {
683         imx_iomux_v3_setup_multiple_pads(fec2_pads,
684             ARRAY_SIZE(fec2_pads));
685         gpio_direction_output(ENET2_RESET, 1);
686         gpio_set_value(ENET2_RESET, 0);
687         mdelay(20);
688         gpio_set_value(ENET2_RESET, 1);
689     }
690 }
```

示例代码 33.2.7.11 中第 676 行~679 行和第 685 行~688 行分别对应 ENET1 和 ENET2 的复位 IO 初始化, 将这两个 IO 设置为输出并且硬件复位一下 LAN8720A, 这个硬件复位很重要! 否则可能导致 uboot 无法识别 LAN8720A。

5、修改 drivers/net/phy/phy.c 文件中的函数 genphy_update_link

大功基本上告成，还差最后一步，uboot 中的 LAN8720A 驱动有点问题，打开文件 drivers/net/phy/phy.c，找到函数 genphy_update_link，这是个通用 PHY 驱动函数，此函数用于更新 PHY 的连接状态和速度。使用 LAN8720A 的时候需要在此函数中添加一些代码，修改后的函数 genphy_update_link 如下所示：

示例代码 33.2.7.12 修改后的 genphy_update_link 函数

```
221 int genphy_update_link(struct phy_device *phydev)
222 {
223     unsigned int mii_reg;
224
225 #ifdef CONFIG_PHY_SMSC
226     static int lan8720_flag = 0;
227     int bmcrr_reg = 0;
228     if (lan8720_flag == 0) {
229         bmcrr_reg = phy_read(phydev, MDIO_DEVAD_NONE, MII_BMCR);
230         phy_write(phydev, MDIO_DEVAD_NONE, MII_BMCR, BMCR_RESET);
231         while(phy_read(phydev, MDIO_DEVAD_NONE, MII_BMCR) & 0X8000) {
232             udelay(100);
233         }
234         phy_write(phydev, MDIO_DEVAD_NONE, MII_BMCR, bmcrr_reg);
235         lan8720_flag = 1;
236     }
237 #endif
238
239     /*
240      * Wait if the link is up, and autonegotiation is in progress
241      * (ie - we're capable and it's not done)
242      */
243     mii_reg = phy_read(phydev, MDIO_DEVAD_NONE, MII_BMSR);
244     .....
291
292     return 0;
293 }
```

225 行~237 行就是新添加的代码，为条件编译代码段，只有使用 SMSC 公司的 PHY 这段代码才会执行(目前只测试了 LAN8720A，SMSC 公司其他的芯片还未测试)。第 229 行读取 LAN8720A 的 BMCR 寄存器(寄存器地址为 0)，此寄存器为 LAN8720A 的配置寄存器，这里先读取此寄存器的默认值并保存起来。230 行向寄存器 BMCR 寄存器写入 BMCR_RESET(值为 0X8000)，因为 BMCR 的 bit15 是软件复位控制位，因此 230 行就是软件复位 LAN8720A，复位完成以后此位会自动清零。第 231~233 行等待 LAN8720A 软件复位完成，也就是判断 BMCR 的 bit15 位是否为 1，为 1 的话表示还没有复位完成。第 234 行重新向 BMCR 寄存器写入以前的值，也就是 229 行读出的那个值。

至此网络的复位引脚驱动修改完成，重新编译 uboot，然后将 u-boot.bin 烧写到 SD 卡中并启动，uboot 启动信息如图 33.2.7.3 所示：

```

U-Boot 2016.03 (May 13 2019 - 16:33:05 +0800)

CPU:   Freescale i.MX6ULL rev1.1 528 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 47C
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C:   ready
DRAM:   512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc0 is current device
Net:    FEC1
Normal Boot
Hit any key to stop autoboot:  0

```

图 33.2.7.3 uboot 启动信息

从图 33.2.7.3 中可以看到“Net: FEC1”这一行，提示当前使用的 FEC1 这个网口，也就是 ENET2。在 uboot 中使用网络之前要先设置几个环境变量，命令如下：

```

setenv ipaddr 192.168.1.55          //开发板 IP 地址
setenv ethaddr b8:ae:1d:01:00:00    //开发板网卡 MAC 地址
setenv gatewayip 192.168.1.1        //开发板默认网关
setenv netmask 255.255.255.0        //开发板子网掩码
setenv serverip 192.168.1.250       //服务器地址，也就是 Ubuntu 地址
saveenv                             //保存环境变量

```

设置好环境变量以后就可以在 uboot 中使用网络了，用网线将 I.MX6U-ALPHA 上的 ENET2 与电脑或者路由器连接起来，保证开发板和电脑在同一个网段内，通过 ping 命令来测试一下网络连接，命令如下：

```
ping 192.168.1.250
```

结果如图 33.2.7.4 所示：

```

=> ping 192.168.1.250
FEC1 Waiting for PHY auto negotiation to complete.... done
Using FEC1 device
host 192.168.1.250 is alive
=>

```

图 33.2.7.4 ping 命令测试

从图 33.2.7.4 可以看出，有“host 192.168.1.250 is alive”这句，说明 ping 主机成功，说明 ENET2 网络工作正常。再来测试一下 ENET1 的网络是否正常工作，打开 mx6ull_alientek_emmc.h，将 CONFIG_FEC_ENET_DEV 改为 0，然后重新编译一下 uboot 并烧写到 SD 卡中重启。重启开发板，uboot 输出信息如图 33.2.7.5 所示：

```
U-Boot 2016.03 (May 13 2019 - 16:53:48 +0800)

CPU:   Freescale i.MX6ULL rev1.1 528 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 50C
Reset cause: POR
Board: MX6ULL 14x14 EVK
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:    serial
Out:   serial
Err:   serial
switch to partitions #0, OK
mmc0 is current device
Net:   FEC0
Normal Boot
Hit any key to stop autoboot:  0
=>
```

图 33.2.7.5 uboot 启动信息

从图 33.2.7.5 可以看出，有“Net: FEC0”这一行，说明当前使用的 FEC0 这个网卡，也就是 ENET1，同样的 ping 一下主机，结果如图 33.2.7.6 所示：

```
=> ping 192.168.1.250
FEC0 waiting for PHY auto negotiation to complete.... done
Using FEC0 device
host 192.168.1.250 is alive
=>
```

图 33.2.7.6 ping 命令测试

从图 33.2.7.6 可以看出，ping 主机也成功，说明 ENET1 网络也工作正常，至此，I.MX6U-ALPHA 开发板的两个网络都工作正常了，建议大家将 ENET2 设置为 uboot 的默认网卡！也就是将宏 CONFIG_FEC_ENET_DEV 设置为 1。

33.2.8 V2.4 及以后版本底板网络驱动修改

查看自己所使用的 ALPHA 底板版本号，V2.4 及以后版本的底板网络芯片更换为 SR8201F，并且网络 PHY 地址也有改变。因此如果您使用的底板是 V2.4 及以后的版本，请看本小节！

1、I.MX6U-ALPHA 开发板网络简介

I.MX6UL/ULL 内部有个以太网 MAC 外设，也就是 ENET，需要外接一个 PHY 芯片来实现网络通信功能，也就是内部 MAC+外部 PHY 芯片的方案。大家可能听过 DM9000 这个网络芯片，在一些没有内部 MAC 的 CPU 中，比如三星的 2440，4412 等，就会采用 DM9000 来实现联网功能。DM9000 提供了一个类似 SRAM 的访问接口，主控 CPU 通过这个接口即可与 DM9000 进行通信，DM9000 就是一个 MAC+PHY 芯片。这个方案就相当于外部 MAC+外部 PHY，那么 I.MX6U 这样的内部 MAC+PHY 芯片与 DM9000 方案比有什么优势吗？那优势大了去了！首先就是通信效率和速度，一般 SOC 内部的 MAC 是带有一个专用 DMA 的，专门用于处理网络数据包，采用 SRAM 来读写 DM9000 的速度是压根就没法和内部 MAC+外部 PHY 芯片的速度比。采用外部 DM9000 完全是无奈之举，谁让 2440，4412 这些芯片内部没有以太网外设呢，现在又想用有线网络，没有办法只能找个 DM9000 的方案。从这里也可以看出，三星的 2440、4412 这些芯片设计之初就不是给工业产品用的，他们是给消费类电子使用的，比如手机、平板等，手机或平板要上网，可以通过 WIFI 或者 4G，我是没有见过哪个手机或者平板上

网是要接根网线的。正点原子的 I.MX6U-ALPHA 开发板也可以通过 WIFI 或者 4G 上网，这个是后话了。

I.MX6UL/ULL 有两个网络接口 ENET1 和 ENET2，正点原子的 I.MX6U-ALPHA 开发板提供了这两个网络接口，其中 ENET1 和 ENET2 都使用 SR8201F 作为 PHY 芯片。NXP 官方的 I.MX6ULLEVK 开发板使用 KSZ8081 这颗 PHY 芯片，SR8201F 相比 KSZ8081 具有体积小、外围器件少、价格便宜等优点。直接使用 KSZ8081 固然可以，但是我们在实际的产品中不一定会使用 KSZ8081，有时候为了降低成本会选择其他的 PHY 芯片，这个时候就有个问题：换了 PHY 芯片以后网络驱动怎么办？为此，正点原子的 I.MX6U-ALPHA 开发板将 ENET1 和 ENET2 的 PHY 换成了 SR8201F，这样就可以给大家讲解更换 PHY 芯片以后如何调整网络驱动，使网络工作正常。

I.MX6U-ALPHA 开发板 ENET1 的网络原理图如图 33.2.8.1 所示：

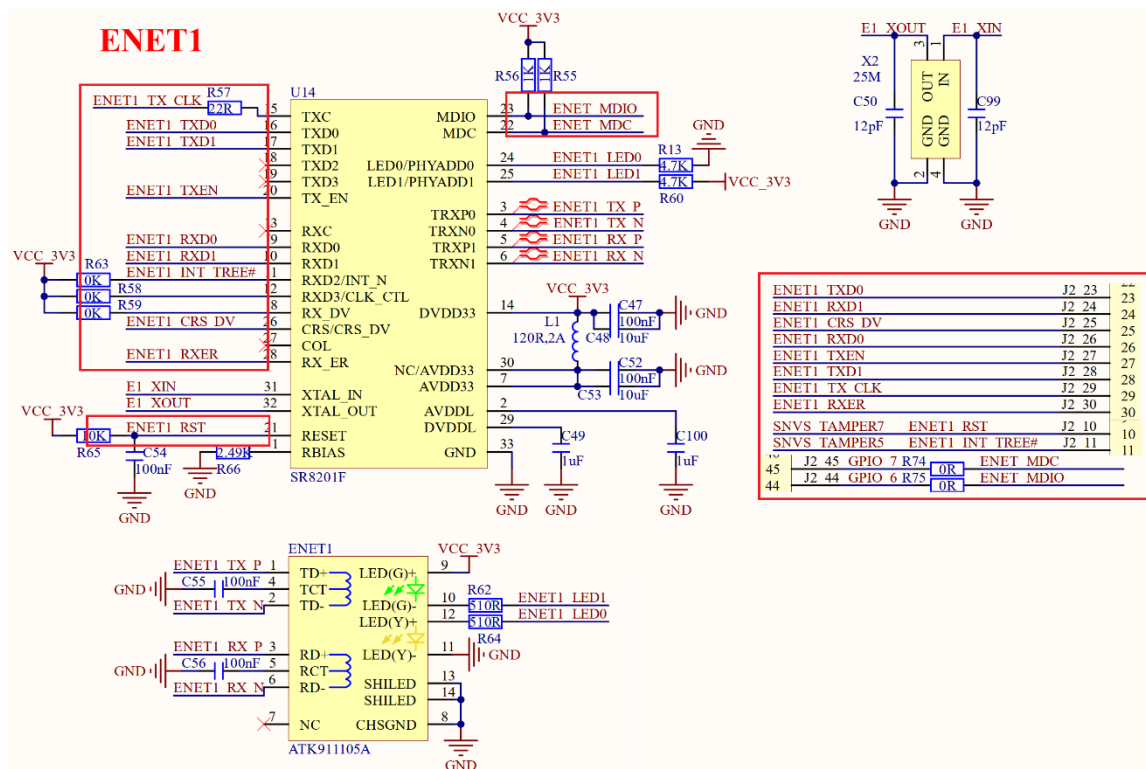


图 33.2.8.1 ENET1 原理图

ENET1 的网络 PHY 芯片为 SR8201F，通过 RMII 接口与 I.MX6ULL 相连，正点原子 I.MX6U-ALPHA 开发板的 ENET1 引脚与 NXP 官方的 I.MX6ULLEVK 开发板基本一样，唯独复位引脚不同。从图 33.2.8.1 可以看出，正点原子 I.MX6U-ALPHA 开发板的 ENET1 复位引脚 ENET1_RST 接到了 I.M6ULL 的 SNVS_TAMPER7 这个引脚上。

SR8201F 内部是有寄存器的，I.MX6ULL 会读取 SR8201F 内部寄存器来判断当前的物理链接状态、连接速度(10M 还是 100M)和双工状态(半双工还是全双工)。I.MX6ULL 通过 MDIO 接口来读取 PHY 芯片的内部寄存器，MDIO 接口有两个引脚，ENET_MDC 和 ENET_MDIO，ENET_MDC 提供时钟，ENET_MDIO 进行数据传输。一个 MDIO 接口可以管理 32 个 PHY 芯片，同一个 MDIO 接口下的这些 PHY 使用不同的器件地址来做区分，MIDO 接口通过不同的器件地址即可访问到相应的 PHY 芯片。I.MX6U-ALPHA 开发板 ENET1 上连接的 SR8201F 器件地址为 0X2，所示我们要修改 ENET1 网络驱动的话重点就三点：

①、ENET1 复位引脚初始化。

②、SR8201F 的器件 ID。

③、SR8201F 驱动

再来看一下 ENET2 的原理图，如图 33.2.8.2 所示：

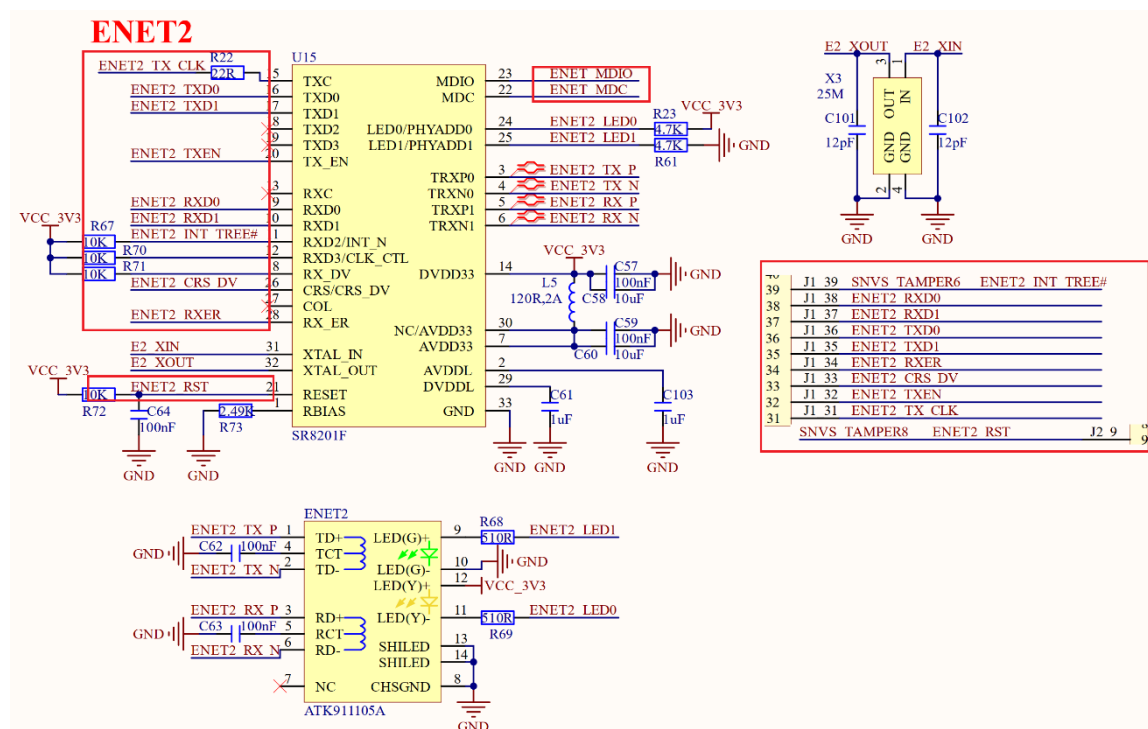


图 33.2.8.2 ENET2 原理图

关于 ENET2 网络驱动的修改也注意一下三点：

①、ENET2 的复位引脚，从图 33.2.8.2 可以看出，ENET2 的复位引脚 ENET2_RST 接到了 IMX6ULL 的 SNVS_TAMPER8 上。

②、ENET2 所使用的 PHY 芯片器件地址，从图 33.2.8.2 可以看出，PHY 器件地址为 0X1。

③、SR8201F 驱动，ENET1 和 ENET2 都使用的 SR8201F，所以驱动肯定是一样的。

2、网络 PHY 地址修改

首先修改 uboot 中的 ENET1 和 ENET2 的 PHY 地址和驱动，打开 mx6ull_alientek_emmc.h 这个文件，找到如下代码：

示例代码 33.2.8.1 网络默认 ID 配置参数

```

325 #ifdef CONFIG_CMD_NET
326 #define CONFIG_CMD_PING
327 #define CONFIG_CMD_DHCP
328 #define CONFIG_CMD_MII
329 #define CONFIG_FEC_MXC
330 #define CONFIG_MII
331 #define CONFIG_FEC_ENET_DEV 1
332
333 #if (CONFIG_FEC_ENET_DEV == 0)
334 #define IMX_FEC_BASE ENET_BASE_ADDR
335 #define CONFIG_FEC_MXC_PHYADDR 0x2
336 #define CONFIG_FEC_XCV_TYPE RMII
    
```



```

337 #elif (CONFIG_FEC_ENET_DEV == 1)
338 #define IMX_FEC_BASE          ENET2_BASE_ADDR
339 #define CONFIG_FEC_MXC_PHYADDR 0x1
340 #define CONFIG_FEC_XCV_TYPE    RMII
341 #endif
342 #define CONFIG_ETHPRIME        "FEC"
343
344 #define CONFIG_PHYLIB
345 #define CONFIG_PHY_MICREL
346 #endif

```

第 331 行的宏 CONFIG_FEC_ENET_DEV 用于选择使用哪个网口，默认为 1，也就是选择 ENET2。第 335 行为 ENET1 的 PHY 地址，默认是 0x2，第 339 行为 ENET2 的 PHY 地址，默认为 0x1。根据前面的分析可知，正点原子的 I.MX6U-ALPHA 开发板 ENET1 的 PHY 地址刚好也为 0x2，ENET2 的 PHY 地址也是 0x1，所以可以直接使用默认配置，不需要我们修改。**注意，这里和 V2.4 版本以前的开发板所使用的 LAN8720A 地址不同！**

第 345 行定了一个宏 CONFIG_PHY_MICREL，此宏用于使能 uboot 中 Micrel 公司的 PHY 驱动，KSZ8081 这颗 PHY 芯片就是 Micrel 公司生产的，不过 Micrel 已经被 Microchip 收购了。如果要使用 SR8201F，那么就得将 CONFIG_PHY_MICREL 改为 CONFIG_PHY_REALTEK。因为 SR8201F 就是 Pin to Pin 替换 Realtek 的 RTL8201F 的，因此可以直接使用 Realtek 的相关驱动。所以示例代码 33.2.8.1 中理论上有一处要修改：

- ①、修改 ENET1 网络 PHY 的地址。
- ②、修改 ENET2 网络 PHY 的地址。
- ③、使能 REALTEK 公司的 PHY 驱动。

但是，由于我们开发板上 SR8201F 的 PHY 地址和默认的配置一样，因此只需要是能 REALTEK 公司的 PHY 驱动，大家如果使用的其他品牌的开发板，那么就要根据实际情况来修改对应的 ENET1 和 ENET2 网络 PHY 地址。

修改后的网络 PHY 地址参数如下所示：

示例代码 33.2.8.2 网络 PHY 地址配置参数

```

325 #ifdef CONFIG_CMD_NET
326 #define CONFIG_CMD_PING
327 #define CONFIG_CMD_DHCP
328 #define CONFIG_CMD_MII
329 #define CONFIG_FEC_MXC
330 #define CONFIG_MII
331 #define CONFIG_FEC_ENET_DEV    1
332
333 #if (CONFIG_FEC_ENET_DEV == 0)
334 #define IMX_FEC_BASE          ENET_BASE_ADDR
335 #define CONFIG_FEC_MXC_PHYADDR 0x2
336 #define CONFIG_FEC_XCV_TYPE    RMII
337 #elif (CONFIG_FEC_ENET_DEV == 1)
338 #define IMX_FEC_BASE          ENET2_BASE_ADDR
339 #define CONFIG_FEC_MXC_PHYADDR 0x1

```

```

340 #define CONFIG_FEC_XCV_TYPE          RMII
341 #endif
342 #define CONFIG_ETHPRIME                "FEC"
343
344 #define CONFIG_PHYLIB
345 #define CONFIG_PHY_REALTEK
346 #endif

```

3、删除 uboot 中 74LV595 的驱动代码

uboot 中网络 PHY 芯片地址修改完成以后就是网络复位引脚的驱动修改了，打开 mx6ull_alientek_emmc.c，找到如下代码：

示例代码 33.2.8.3 74LV595 引脚

```

#define IOX_SDI    IMX_GPIO_NR(5, 10)
#define IOX_STCP   IMX_GPIO_NR(5, 7)
#define IOX_SHCP   IMX_GPIO_NR(5, 11)
#define IOX_OE     IMX_GPIO_NR(5, 8)

```

示例代码 33.2.8.3 中以 IOX 开头的宏定义是 74LV595 的相关 GPIO，因为 NXP 官方 I.MX6ULL EVK 开发板使用 74LV595 来扩展 IO，两个网络的复位引脚就是由 74LV595 来控制的。正点原子的 I.MX6U-ALPHA 开发板并没有使用 74LV595，因此我们将示例代码 33.2.8.3 中的代码删除掉，替换为如下所示代码：

示例代码 33.2.8.4 修改后的网络引脚

```

#define ENET1_RESET IMX_GPIO_NR(5, 7)
#define ENET2_RESET IMX_GPIO_NR(5, 8)

```

ENET1 的复位引脚连接到 SNVS_TAMPER7 上，对应 GPIO5_IO07，ENET2 的复位引脚连接到 SNVS_TAMPER8 上，对应 GPIO5_IO08。

继续在 mx6ull_alientek_emmc.c 中找到如下代码：

示例代码 33.2.8.5 74LV595 引脚配置

```

static iomux_v3_cfg_t const iox_pads[] = {
    /* IOX_SDI */
    MX6_PAD_BOOT_MODE0__GPIO5_IO10 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_SHCP */
    MX6_PAD_BOOT_MODE1__GPIO5_IO11 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_STCP */
    MX6_PAD_SNVS_TAMPER7__GPIO5_IO07 | MUX_PAD_CTRL(NO_PAD_CTRL),
    /* IOX_nOE */
    MX6_PAD_SNVS_TAMPER8__GPIO5_IO08 | MUX_PAD_CTRL(NO_PAD_CTRL),
};

```

同理，示例代码 33.2.8.5 是 74LV595 的 IO 配置参数结构体，将其删除掉。继续在 mx6ull_alientek_emmc.c 中找到函数 iox74lv_init，如下所示：

示例代码 33.2.8.6 74LV595 初始化函数

```

static void iox74lv_init(void)
{
    int i;

```

```

    gpio_direction_output(IOX_OE, 0);

    for (i = 7; i >= 0; i--) {
        gpio_direction_output(IOX_SHCP, 0);
        gpio_direction_output(IOX_SDI, seq[qn_output[i]][0]);
        udelay(500);
        gpio_direction_output(IOX_SHCP, 1);
        udelay(500);
    }

    .....
    /*
     * shift register will be output to pins
     */
    gpio_direction_output(IOX_STCP, 1);
};

void iox74lv_set(int index)
{
    int i;

    for (i = 7; i >= 0; i--) {
        gpio_direction_output(IOX_SHCP, 0);

        if (i == index)
            gpio_direction_output(IOX_SDI, seq[qn_output[i]][0]);
        else
            gpio_direction_output(IOX_SDI, seq[qn_output[i]][1]);
        udelay(500);
        gpio_direction_output(IOX_SHCP, 1);
        udelay(500);
    }

    .....
    /*
     * shift register will be output to pins
     */
    gpio_direction_output(IOX_STCP, 1);
};

```

iox74lv_init 函数是 74LV595 的初始化函数, iox74lv_set 函数用于控制 74LV595 的 IO 输出电平, 将这两个函数全部删除掉!

在 mx6ull_alientek_emmc.c 中找到 board_init 函数, 此函数是板子初始化函数, 会被 board_init_r 调用, board_init 函数内容如下:

示例代码 33.2.8.7 board_init 函数

```
int board_init(void)
{
    .....
    imx_iomux_v3_setup_multiple_pads(iox_pads, ARRAY_SIZE(iox_pads));
    iox74lv_init();
    .....
    return 0;
}
```

board_init 会调用 imx_iomux_v3_setup_multiple_pads 和 iox74lv_init 这两个函数来初始化 74lv595 的 GPIO，将这两行删除掉。至此，mx6ull_alientek_emmc.c 中关于 74LV595 芯片的驱动代码都删除掉了，接下来就是添加 I.MX6U-ALPHA 开发板两个网络复位引脚了。

4、添加 I.MX6U-ALPHA 开发板网络复位引脚驱动

在 mx6ull_alientek_emmc.c 中找到如下所示代码：

示例代码 33.2.8.8 默认网络 IO 结构体数组

```
640 static iomux_v3_cfg_t const fec1_pads[] = {
641     MX6_PAD_GPIO1_IO06__ENET1_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
642     MX6_PAD_GPIO1_IO07__ENET1_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
643     .....
649     MX6_PAD_ENET1_RX_ER__ENET1_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
650     MX6_PAD_ENET1_RX_EN__ENET1_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
651 };
652
653 static iomux_v3_cfg_t const fec2_pads[] = {
654     MX6_PAD_GPIO1_IO06__ENET2_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
655     MX6_PAD_GPIO1_IO07__ENET2_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
656     .....
664     MX6_PAD_ENET2_RX_EN__ENET2_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
665     MX6_PAD_ENET2_RX_ER__ENET2_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
666 };
```

结构体数组 fec1_pads 和 fec2_pads 是 ENET1 和 ENET2 这两个网口的 IO 配置参数，在这两个数组中添加两个网口的复位 IO 配置参数，完成以后如下所示：

示例代码 33.2.8.9 添加网络复位 IO 后的结构体数组

```
640 static iomux_v3_cfg_t const fec1_pads[] = {
641     MX6_PAD_GPIO1_IO06__ENET1_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
642     MX6_PAD_GPIO1_IO07__ENET1_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
643     .....
649     MX6_PAD_ENET1_RX_ER__ENET1_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
650     MX6_PAD_ENET1_RX_EN__ENET1_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
651     MX6_PAD_SNVS_TAMPER7__GPIO5_IO07 | MUX_PAD_CTRL(NO_PAD_CTRL),
652 };
653
654 static iomux_v3_cfg_t const fec2_pads[] = {
655     MX6_PAD_GPIO1_IO06__ENET2_MDIO | MUX_PAD_CTRL(MDIO_PAD_CTRL),
```

```

656     MX6_PAD_GPIO1_IO07__ENET2_MDC | MUX_PAD_CTRL(ENET_PAD_CTRL),
.....
665     MX6_PAD_ENET2_RX_EN__ENET2_RX_EN | MUX_PAD_CTRL(ENET_PAD_CTRL),
666     MX6_PAD_ENET2_RX_ER__ENET2_RX_ER | MUX_PAD_CTRL(ENET_PAD_CTRL),
667     MX6_PAD_SNVS_TAMPER8__GPIO5_IO08 | MUX_PAD_CTRL(NO_PAD_CTRL),
668 };

```

示例代码 33.2.8.9 中，第 651 行和 667 行分别是 ENET1 和 ENET2 的复位 IO 配置参数。继续在文件 mx6ull_alientek_emmc.c 中找到函数 setup_iomux_fec，此函数默认代码如下：

示例代码 33.2.7.10 setup_iomux_fec 函数默认代码

```

668 static void setup_iomux_fec(int fec_id)
669 {
670     if (fec_id == 0)
671         imx_iomux_v3_setup_multiple_pads(fec1_pads,
672             ARRAY_SIZE(fec1_pads));
673     else
674         imx_iomux_v3_setup_multiple_pads(fec2_pads,
675             ARRAY_SIZE(fec2_pads));
676 }

```

函数 setup_iomux_fec 就是根据 fec1_pads 和 fec2_pads 这两个网络 IO 配置数组来初始化 I.MX6ULL 的网络 IO。我们需要在其中添加网络复位 IO 的初始化代码，并且复位一下 PHY 芯片，修改后的 setup_iomux_fec 函数如下：

示例代码 33.2.8.11 修改后的 setup_iomux_fec 函数

```

668 static void setup_iomux_fec(int fec_id)
669 {
670     if (fec_id == 0)
671     {
672
673         imx_iomux_v3_setup_multiple_pads(fec1_pads,
674             ARRAY_SIZE(fec1_pads));
675
676         gpio_direction_output(ENET1_RESET, 1);
677         gpio_set_value(ENET1_RESET, 0);
678         mdelay(20);
679         gpio_set_value(ENET1_RESET, 1);
680     }
681     else
682     {
683         imx_iomux_v3_setup_multiple_pads(fec2_pads,
684             ARRAY_SIZE(fec2_pads));
685
686         gpio_direction_output(ENET2_RESET, 1);
687         gpio_set_value(ENET2_RESET, 0);
688         mdelay(20);
689         gpio_set_value(ENET2_RESET, 1);

```

```

689     }
689     mdelay(150); /* 复位结束后至少延时 150ms 才能正常使用 */
690 }

```

示例代码 33.2.8.11 中第 676 行~679 行和第 685 行~688 行分别对应 ENET1 和 ENET2 的复位 IO 初始化, 将这两个 IO 设置为输出并且硬件复位一下 LAN8720A, 这个硬件复位很重要! 第 689 行复位结束以后一定要至少延时 150ms 才能操作 SR8201F, 这个在 SR8201F 数据手册里面有详细要求的, 否则会导致 uboot 无法识别 SR8201F。

至此网络的复位引脚驱动修改完成, 重新编译 uboot, 然后将 u-boot.bin 烧写到 SD 卡中并启动, uboot 启动信息如图 33.2.8.3 所示:

```

U-Boot 2016.03 (Aug 02 2022 - 15:49:39 +0800)

CPU:   Freescale i.MX6ULL rev1.1 69 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 55C
Reset cause: POR
Board: MX6ULL ALIENTEK EMMC
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc0 is current device
Net:    FEC1
Normal Boot
Hit any key to stop autoboot: 0

```

图 33.2.8.3 uboot 启动信息

从图 33.2.8.4 中可以看到“Net: FEC1”这一行, 提示当前使用的 FEC1 这个网口, 也就是 ENET2。在 uboot 中使用网络之前要先设置几个环境变量, 命令如下:

```

setenv ipaddr 192.168.1.55      //开发板 IP 地址
setenv ethaddr b8:ae:1d:01:00:00 //开发板网卡 MAC 地址
setenv gatewayip 192.168.1.1   //开发板默认网关
setenv netmask 255.255.255.0   //开发板子网掩码
setenv serverip 192.168.1.250  //服务器地址, 也就是 Ubuntu 地址
saveenv                        //保存环境变量

```

设置好环境变量以后就可以在 uboot 中使用网络了, 用网线将 I.MX6U-ALPHA 上的 ENET2 与电脑或者路由器连接起来, 保证开发板和电脑在同一个网段内, 通过 ping 命令来测试一下网络连接, 命令如下:

```
ping 192.168.1.250
```

结果如图 33.2.8.4 所示:

```

=> ping 192.168.1.250
Using FEC1 device
host 192.168.1.250 is alive
=>

```

图 33.2.8.4 ping 命令测试

从图 33.2.8.4 可以看出, 有“host 192.168.1.250 is alive”这句, 说明 ping 主机成功, 说明 ENET2 网络工作正常。再来测试一下 ENET1 的网络是否正常工作, 打开 mx6ull_alientek_emmc.h,

将 CONFIG_FEC_ENET_DEV 改为 0, 然后重新编译一下 uboot 并烧写到 SD 卡中重启。重启开发板, uboot 输出信息如图 33.2.8.5 所示:

```
U-Boot 2016.03 (Aug 02 2022 - 16:27:20 +0800)

CPU:   Freescale i.MX6ULL rev1.1 69 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 55C
Reset cause: POR
Board: MX6ULL ALIENTEK EMMC
I2C:   ready
DRAM:  512 MiB
MMC:   FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc0 is current device
Net:    FEC0
Normal Boot
Hit any key to stop autoboot:  0
=>
```

图 33.2.8.5 uboot 启动信息

从图 33.2.8.5 可以看出, 有 “Net: FEC0” 这一行, 说明当前使用的 FEC0 这个网卡, 也就是 ENET1, 同样的 ping 一下主机, 结果如图 33.2.8.6 所示:

```
=> ping 192.168.1.250
Using FEC1 device
host 192.168.1.250 is alive
=>
```

图 33.2.8.6 ping 命令测试

从图 33.2.8.6 可以看出, ping 主机也成功, 说明 ENET1 网络也工作正常, 至此, I.MX6U-ALPHA 开发板的两个网络都工作正常了, 建议大家将 ENET2 设置为 uboot 的默认网卡! 也就是将宏 CONFIG_FEC_ENET_DEV 设置为 1。

33.2.9 其他需要修改的地方

在 uboot 启动信息中会有 “Board: MX6ULL 14x14 EVK” 这一句, 也就是说板子名字为 “MX6ULL 14x14 EVK”, 要将其改为我们所使用的板子名字, 比如 “MX6ULL ALIENTEK EMMC” 或者 “MX6ULL ALIENTEK NAND”。打开文件 mx6ull_alientek_emmc.c, 找到函数 checkboard, 将其改为如下所示内容:

示例代码 33.2.9.1 修改后的 checkboard 函数

```
int checkboard(void)
{
    if (is_mx6ull_9x9_evk())
        puts("Board: MX6ULL 9x9 EVK\n");
    else
        puts("Board: MX6ULL ALIENTEK EMMC\n");

    return 0;
}
```

修改完成以后重新编译 uboot 并烧写到 SD 卡中验证, uboot 启动信息如图 33.2.9.1 所示:


```

U-Boot 2016.03 (May 21 2019 - 10:42:19 +0800)

CPU:   Freescale i.MX6ULL rev1.1 528 MHz (running at 396 MHz)
CPU:   Industrial temperature grade (-40C to 105C) at 54C
Reset cause: POR
Board: MX6ULL ALIENTEK EMMC
I2C:   ready
DRAM:  512 MiB
MMC:    FSL_SDHC: 0, FSL_SDHC: 1
Display: TFT7016 (1024x600)
Video: 1024x600x24
In:     serial
Out:    serial
Err:    serial
switch to partitions #0, OK
mmc0 is current device
Net:    FEC1
Normal Boot
Hit any key to stop autoboot:  0
=>

```

图 33.2.9.1 uboot 启动信息

从图 33.2.9.1 可以看出，Board 变成了“MX6ULL ALIENTEK EMMC”。至此 uboot 的驱动部分就修改完成了，uboot 移植也完成了，uboot 的最终目的就是启动 Linux 内核，所以需要通启动 Linux 内核来判断 uboot 移植是否成功。在启动 Linux 内核之前我们先来学习两个重要的环境变量 bootcmd 和 bootargs。

33.3 bootcmd 和 bootargs 环境变量

uboot 中有两个非常重要的环境变量 bootcmd 和 bootargs，接下来看一下这两个环境变量。bootcmd 和 bootargs 是采用类似 shell 脚本语言编写的，里面有很多的变量引用，这些变量其实都是环境变量，有很多是 NXP 自己定义的。文件 mx6ull_alientek_emmc.h 中的宏 CONFIG_EXTRA_ENV_SETTINGS 保存着这些环境变量的默认值，内容如下：

示例代码 33.3.1.1 宏 CONFIG_EXTRA_ENV_SETTINGS 默认值

```

113 #if defined(CONFIG_SYS_BOOT_NAND)
114 #define CONFIG_EXTRA_ENV_SETTINGS \
115     CONFIG_MFG_ENV_SETTINGS \
116     "panel=TFT43AB\0" \
117     "fdt_addr=0x83000000\0" \
118     "fdt_high=0xffffffff\0" \
119     .....
126     "bootz ${loadaddr} - ${fdt_addr}\0"
127
128 #else
129 #define CONFIG_EXTRA_ENV_SETTINGS \
130     CONFIG_MFG_ENV_SETTINGS \
131     "script=boot.scr\0" \
132     "image=zImage\0" \
133     "console=ttyMXC0\0" \
134     "fdt_high=0xffffffff\0" \
135     "initrd_high=0xffffffff\0" \

```

```

136     "fdt_file=undefined\0" \
.....
194     "findfdt="\
195     "if test $fdt_file = undefined; then " \
196     "if test $board_name = EVK && test $board_rev = 9X9; then " \
197     "setenv fdt_file imx6ull-9x9-evk.dtb; fi; " \
198     "if test $board_name = EVK && test $board_rev = 14X14; then " \
199     "setenv fdt_file imx6ull-14x14-evk.dtb; fi; " \
200     "if test $fdt_file = undefined; then " \
201     "echo WARNING: Could not determine dtb to use; fi; " \
202     "fi;\0" \

```

宏 CONFIG_EXTRA_ENV_SETTINGS 是个条件编译语句，使用 NAND 和 EMMC 的时候宏 CONFIG_EXTRA_ENV_SETTINGS 的值是不同的。

33.3.1 环境变量 bootcmd

bootcmd 在前面已经说了很多次了，bootcmd 保存着 uboot 默认命令，uboot 倒计时结束以后就会执行 bootcmd 中的命令。这些命令一般都是用来启动 Linux 内核的，比如读取 EMMC 或者 NAND Flash 中的 Linux 内核镜像文件和设备树文件到 DRAM 中，然后启动 Linux 内核。可以在 uboot 启动以后进入命令行设置 bootcmd 环境变量的值。如果 EMMC 或者 NAND 中没有保存 bootcmd 的值，那么 uboot 就会使用默认的值，板子第一次运行 uboot 的时候都会使用默认值来设置 bootcmd 环境变量。打开文件 include/env_default.h，在此文件中有如下所示内容：

示例代码 33.3.1.1 默认环境变量

```

13 #ifndef DEFAULT_ENV_INSTANCE_EMBEDDED
14 env_t environment __PPCENV__ = {
15     ENV_CRC,    /* CRC Sum */
16 #ifdef CONFIG_SYS_REDUNDAND_ENVIRONMENT
17     1,         /* Flags: valid */
18 #endif
19     {
20 #elif defined(DEFAULT_ENV_INSTANCE_STATIC)
21     static char default_environment[] = {
22 #else
23     const uchar default_environment[] = {
24 #endif
25 #ifdef CONFIG_ENV_CALLBACK_LIST_DEFAULT
26     ENV_CALLBACK_VAR "=" CONFIG_ENV_CALLBACK_LIST_DEFAULT "\0"
27 #endif
28 #ifdef CONFIG_ENV_FLAGS_LIST_DEFAULT
29     ENV_FLAGS_VAR "=" CONFIG_ENV_FLAGS_LIST_DEFAULT "\0"
30 #endif
31 #ifdef CONFIG_BOOTARGS
32     "bootargs=" CONFIG_BOOTARGS "\0"
33 #endif

```

```
34 #ifdef CONFIG_BOOTCOMMAND
35     "bootcmd=" CONFIG_BOOTCOMMAND "\0"
36 #endif
37 #ifdef CONFIG_RAMBOOTCOMMAND
38     "ramboot=" CONFIG_RAMBOOTCOMMAND "\0"
39 #endif
40 #ifdef CONFIG_NFSBOOTCOMMAND
41     "nfsboot=" CONFIG_NFSBOOTCOMMAND "\0"
42 #endif
43 #if defined(CONFIG_BOOTDELAY) && (CONFIG_BOOTDELAY >= 0)
44     "bootdelay=" __stringify(CONFIG_BOOTDELAY) "\0"
45 #endif
46 #if defined(CONFIG_BAUDRATE) && (CONFIG_BAUDRATE >= 0)
47     "baudrate=" __stringify(CONFIG_BAUDRATE) "\0"
48 #endif
49 #ifdef CONFIG_LOADS_ECHO
50     "loads_echo=" __stringify(CONFIG_LOADS_ECHO) "\0"
51 #endif
52 #ifdef CONFIG_ETHPRIME
53     "ethprime=" CONFIG_ETHPRIME "\0"
54 #endif
55 #ifdef CONFIG_IPADDR
56     "ipaddr=" __stringify(CONFIG_IPADDR) "\0"
57 #endif
58 #ifdef CONFIG_SERVERIP
59     "serverip=" __stringify(CONFIG_SERVERIP) "\0"
60 #endif
61 #ifdef CONFIG_SYS_AUTOLOAD
62     "autoload=" CONFIG_SYS_AUTOLOAD "\0"
63 #endif
64 #ifdef CONFIG_PREBOOT
65     "preboot=" CONFIG_PREBOOT "\0"
66 #endif
67 #ifdef CONFIG_ROOTPATH
68     "rootpath=" CONFIG_ROOTPATH "\0"
69 #endif
70 #ifdef CONFIG_GATEWAYIP
71     "gatewayip=" __stringify(CONFIG_GATEWAYIP) "\0"
72 #endif
73 #ifdef CONFIG_NETMASK
74     "netmask=" __stringify(CONFIG_NETMASK) "\0"
75 #endif
76 #ifdef CONFIG_HOSTNAME
```

```

77     "hostname=" __stringify(CONFIG_HOSTNAME)    "\0"
78 #endif
79 #ifdef CONFIG_BOOTFILE
80     "bootfile=" CONFIG_BOOTFILE                "\0"
81 #endif
82 #ifdef CONFIG_LOADADDR
83     "loadaddr=" __stringify(CONFIG_LOADADDR)    "\0"
84 #endif
85 #ifdef CONFIG_CLOCKS_IN_MHZ
86     "clocks_in_mhz=1\0"
87 #endif
88 #if defined(CONFIG_PCI_BOOTDELAY) && (CONFIG_PCI_BOOTDELAY > 0)
89     "pcideLAY=" __stringify(CONFIG_PCI_BOOTDELAY) "\0"
90 #endif
91 #ifdef CONFIG_ENV_VARS_UBOOT_CONFIG
92     "arch="      CONFIG_SYS_ARCH                "\0"
93     "cpu="       CONFIG_SYS_CPU                 "\0"
94     "board="     CONFIG_SYS_BOARD               "\0"
95     "board_name=" CONFIG_SYS_BOARD              "\0"
96 #ifdef CONFIG_SYS_VENDOR
97     "vendor="    CONFIG_SYS_VENDOR              "\0"
98 #endif
99 #ifdef CONFIG_SYS_SOC
100    "soc="       CONFIG_SYS_SOC                  "\0"
101 #endif
102 #endif
103 #ifdef CONFIG_EXTRA_ENV_SETTINGS
104     CONFIG_EXTRA_ENV_SETTINGS
105 #endif
106     "\0"
107 #ifdef DEFAULT_ENV_INSTANCE_EMBEDDED
108     }
109 #endif
110 };

```

第 13~23 行，这段代码是个条件编译，由于没有定义 `DEFAULT_ENV_INSTANCE_EMBEDDED` 和 `CONFIG_SYS_REDUNDAND_ENVIRONMENT`，因此 `uchar default_environment[]` 数组保存环境变量。

在示例代码 33.3.1.1 中指定了很多环境变量的默认值，比如 `bootcmd` 的默认值就是 `CONFIG_BOOTCOMMAND`，`bootargs` 的默认值就是 `CONFIG_BOOTARGS`。我们可以在 `mx6ull_alientek_emmc.h` 文件中通过设置宏 `CONFIG_BOOTCOMMAND` 来设置 `bootcmd` 的默认值，NXP 官方设置的 `CONFIG_BOOTCOMMAND` 值如下：

示例代码 33.3.1.3 `CONFIG_BOOTCOMMAND` 默认值

```

204 #define CONFIG_BOOTCOMMAND \

```

```

205     "run findfdt;" \
206     "mmc dev ${mmcdev};" \
207     "mmc dev ${mmcdev}; if mmc rescan; then " \
208         "if run loadbootscript; then " \
209             "run bootscrip; " \
210         "else " \
211             "if run loadimage; then " \
212                 "run mmcboot; " \
213                 "else run netboot; " \
214             "fi; " \
215         "fi; " \
216     "else run netboot; fi"

```

看起来很复杂的样子！因为 uboot 使用了类似 shell 脚本语言的方式来编写的，我们一行一行来分析。

第 205 行，run findfdt；使用的是 uboot 的 run 命令来运行 findfdt，findfdt 是 NXP 自行添加的环境变量。findfdt 是用来查找开发板对应的设备树文件(.dtb)。IMX6ULL EVK 的设备树文件为 imx6ull-14x14-evk.dtb，findfdt 内容如下：

```

"findfdt="\
"if test $fdt_file = undefined; then " \
"if test $board_name = EVK && test $board_rev = 9X9; then " \
    "setenv fdt_file imx6ull-9x9-evk.dtb; fi; " \
    "if test $board_name = EVK && test $board_rev = 14X14; then " \
        "setenv fdt_file imx6ull-14x14-evk.dtb; fi; " \
    "if test $fdt_file = undefined; then " \
        "echo WARNING: Could not determine dtb to use; fi; " \
"fi;\0" \

```

findfdt 里面用到的变量有 fdt_file，board_name，board_rev，这三个变量内容如下：

```
fdt_file=undefined, board_name=EVK, board_rev=14X14
```

findfdt 做的事情就是判断，fdt_file 是否为 undefined，如果 fdt_file 为 undefined 的话那就要根据板子信息得出所需的.dtb 文件名。此时 fdt_file 为 undefined，所以根据 board_name 和 board_rev 来判断实际所需的.dtb 文件，如果 board_name 为 EVK 并且 board_rev=9x9 的话 fdt_file 就为 imx6ull-9x9-evk.dtb。如果 board_name 为 EVK 并且 board_rev=14x14 的话 fdt_file 就设置为 imx6ull-14x14-evk.dtb。因此 IMX6ULL EVK 板子的设备树文件就是 imx6ull-14x14-evk.dtb，

因此 run findfdt 的结果就是设置 fdt_file 为 imx6ull-14x14-evk.dtb。

第 206 行，mmc dev \${mmcdev}用于切换 mmc 设备，mmcdev 为 1，因此这行代码就是：mmc dev 1，也就是切换到 EMMC 上。

第 207 行，先执行 mmc dev \${mmcdev}切换到 EMMC 上，然后使用命令 mmc rescan 扫描看有没有 SD 卡或者 EMMC 存在，如果没有的话就直接跳到 216 行，执行 run netboot，netboot 也是一个自定义的环境变量，这个变量是从网络启动 Linux 的。如果 mmc 设备存在的话就从 mmc 设备启动。

第 208 行，运行 loadbootscrip 环境变量，此环境变量内容如下：

```
loadbootscrip=fatload mmc ${mmcdev}:${mmcpart} ${loadaddr} ${script};
```

其中 mmcdev=1，mmcpart=1，loadaddr=0x80800000，script= boot.scr，因此展开以后就是：

```
loadbootscript=fatload mmc 1:1 0x80800000 boot.src;
```

loadbootscript 就是从 mmc1 的分区 1 中读取文件 boot.src 到 DRAM 的 0X80800000 处。但是 mmc1 的分区 1 中没有 boot.src 这个文件，可以使用命令“ls mmc 1:1”查看一下 mmc1 分区 1 中的所有文件，看看有没有 boot.src 这个文件。

第 209 行，如果加载 boot.src 文件成功的话就运行 bootscript 环境变量，bootscript 的内容如下：

```
bootscript=echo Running bootscript from mmc ...;
source
```

因为 boot.src 文件不存在，所以 bootscript 也就不会运行。

第 211 行，如果 loadbootscript 没有找到 boot.src 的话就运行环境变量 loadimage，环境变量 loadimage 内容如下：

```
loadimage=fatload mmc ${mmcdev}:${mmcpart} ${loadaddr} ${image}
```

其中 mmcdev=1，mmcpart=1，loadaddr=0x80800000，image = zImage，展开以后就是：

```
loadimage=fatload mmc 1:1 0x80800000 zImage
```

可以看出 loadimage 就是从 mmc1 的分区中读取 zImage 到内存的 0X80800000 处，而 mmc1 的分区 1 中存在 zImage。

第 212 行，加载 linux 镜像文件 zImage 成功以后就运行环境变量 mmcboot，否则的话运行 netboot 环境变量。mmcboot 环境变量如下：

示例代码 33.3.1.4 mmcboot 环境变量

```
154 "mmcboot=echo Booting from mmc ...; " \
155     "run mmcargs; " \
156     "if test ${boot_fdt} = yes || test ${boot_fdt} = try; then " \
157         "if run loadfdt; then " \
158             "bootz ${loadaddr} - ${fdt_addr}; " \
159         "else " \
160             "if test ${boot_fdt} = try; then " \
161                 "bootz; " \
162             "else " \
163                 "echo WARN: Cannot load the DT; " \
164             "fi; " \
165         "fi; " \
166     "else " \
167         "bootz; " \
168     "fi;\0" \
```

第 154 行，输出信息“Booting from mmc ...”。

第 155 行，运行环境变量 mmcargs，mmcargs 用来设置 bootargs，后面分析 bootargs 的时候在学习。

第 156 行，判断 boot_fdt 是否为 yes 或者 try，根据 uboot 输出的环境变量信息可知 boot_fdt=try。因此会执行 157 行的语句。

第 157 行，运行环境变量 loadfdt，环境变量 loadfdt 定义如下：

```
loadfdt=fatload mmc ${mmcdev}:${mmcpart} ${fdt_addr} ${fdt_file}
```

展开以后就是：

```
loadfdt=fatload mmc 1:1 0x83000000 imx6ull-14x14-evk.dtb
```

因此 loadfdt 的作用就是从 mmc1 的分区 1 中读取 imx6ull-14x14-evk.dtb 文件并放到 0x83000000 处。

第 158 行, 如果读取.dtb 文件成功的话那就调用命令 bootz 启动 linux, 调用方法如下:

```
bootz ${loadaddr} - ${fdt_addr};
```

展开就是:

```
bootz 0x80800000 - 0x83000000 (注意 '-' 前后要有空格)
```

至此 Linux 内核启动, 如此复杂的设置就是为了从 EMMC 中读取 zImage 镜像文件和设备树文件。经过分析, 浓缩出来的仅仅是 4 行精华:

```
mmc dev 1 //切换到 EMMC
fatload mmc 1:1 0x80800000 zImage //读取 zImage 到 0x80800000 处
fatload mmc 1:1 0x83000000 imx6ull-14x14-evk.dtb //读取设备树到 0x83000000 处
bootz 0x80800000 - 0x83000000 //启动 Linux
```

NXP 官方将 CONFIG_BOOTCOMMAND 写的这么复杂只有一个目的: 为了兼容多个板子, 所以写了个很复杂的脚本。当我们明确知道我们所使用的板子的时候就可以大幅简化宏 CONFIG_BOOTCOMMAND 的设置, 比如我们要从 EMMC 启动, 那么宏 CONFIG_BOOTCOMMAND 就可简化为:

```
#define CONFIG_BOOTCOMMAND \
    "mmc dev 1;" \
    "fatload mmc 1:1 0x80800000 zImage;" \
    "fatload mmc 1:1 0x83000000 imx6ull-alientek-emmc.dtb;" \
    "bootz 0x80800000 - 0x83000000;"
```

或者可以直接在 uboot 中设置 bootcmd 的值, 这个值就是保存到 EMMC 中的, 命令如下:

```
setenv bootcmd 'mmc dev 1; fatload mmc 1:1 80800000 zImage; fatload mmc 1:1 83000000 imx6ull-alientek-emmc.dtb; bootz 80800000 - 83000000;'
```

33.3.2 环境变量 bootargs

bootargs 保存着 uboot 传递给 Linux 内核的参数, 在上一小节讲解 bootcmd 的时候说过, bootargs 环境变量是由 mmccargs 设置的, mmccargs 环境变量如下:

```
mmccargs=setenv bootargs console=${console},${baudrate} root=${mmccroot}
```

其中 console=ttymxc0, baudrate=115200, mmccroot=/dev/mmcblk1p2 rootwait rw, 因此将 mmccargs 展开以后就是:

```
mmccargs=setenv bootargs console= ttymxc0, 115200 root= /dev/mmcblk1p2 rootwait rw
```

可以看出环境变量 mmccargs 就是设置 bootargs 的值为 “console= ttymxc0, 115200 root= /dev/mmcblk1p2 rootwait rw”, bootargs 就是设置了很多的参数的值, 这些参数 Linux 内核会用到, 常用的参数有:

1、console

console 用来设置 linux 终端(或者叫控制台), 也就是通过什么设备来和 Linux 进行交互, 是串口还是 LCD 屏幕? 如果是串口的话应该是串口几等等。一般设置串口作为 Linux 终端, 这样我们就可以在电脑上通过 SecureCRT 来和 linux 交互了。这里设置 console 为 ttymxc0, 因为 linux 启动以后 I.MX6ULL 的串口 1 在 linux 下的设备文件就是 /dev/ttymxc0, 在 Linux 下, 一切皆文件。

ttymxc0 后面有个 “,115200”, 这是设置串口的波特率, console=ttymxc0,115200 综合起来就是设

置 `ttymxc0`（也就是串口 1）作为 Linux 的终端，并且串口波特率设置为 115200。

2、root

`root` 用来设置根文件系统的位置，`root=/dev/mmcblk1p2` 用于指明根文件系统存放在 `mmcblk1` 设备的分区 2 中。EMMC 版本的核心板启动 linux 以后会存在 `/dev/mmcblk0`、`/dev/mmcblk1`、`/dev/mmcblk0p1`、`/dev/mmcblk0p2`、`/dev/mmcblk1p1` 和 `/dev/mmcblk1p2` 这样的文件，其中 `/dev/mmcblkx(x=0~n)` 表示 mmc 设备，而 `/dev/mmcblkxpy(x=0~n,y=1~n)` 表示 mmc 设备 `x` 的分区 `y`。在 I.MX6U-ALPHA 开发板中 `/dev/mmcblk1` 表示 EMMC，而 `/dev/mmcblk1p2` 表示 EMMC 的分区 2。

`root` 后面有 “`rootwait rw`”，`rootwait` 表示等待 mmc 设备初始化完成以后再挂载，否则的话 mmc 设备还没初始化完成就挂载根文件系统会出错的。`rw` 表示根文件系统是可以读写的，不加 `rw` 的话可能无法在根文件系统中进行写操作，只能进行读操作。

3、rootfstype

此选项一般配置 `root` 一起使用，`rootfstype` 用于指定根文件系统类型，如果根文件系统为 `ext` 格式的话此选项无所谓。如果根文件系统是 `yaffs`、`jffs` 或 `ubifs` 的话就需要设置此选项，指定根文件系统的类型。

`bootargs` 常设置的选项就这三个，后面遇到其他选项的话再讲解。

33.4 uboot 启动 Linux 测试

uboot 已经移植好了，`bootcmd` 和 `bootargs` 这两个重要的环境变量也讲解了，接下来就要测试一下 uboot 能不能完成它的工作：启动 Linux 内核。我们测试两种启动 Linux 内核的方法，一种是直接从 EMMC 启动，一种是从网络启动。

33.4.1 从 EMMC 启动 Linux 系统

从 EMMC 启动也就是将编译出来的 Linux 镜像文件 `zImage` 和设备树文件保存在 EMMC 中，uboot 从 EMMC 中读取这两个文件并启动，这个是我们产品最终的启动方式。但是我们目前还没有讲解如何移植 linux 和设备树文件，以及如何将 `zImage` 和设备树文件保存到 EMMC 中。不过大家拿到手的 I.MX6U-ALPHA 开发板(EMMC 版本)已经将 `zImage` 文件和设备树文件烧写到了 EMMC 中，所以我们可以直接读取来测试。先检查一下 EMMC 的分区 1 中有没有 `zImage` 文件和设备树文件，输入命令 “`ls mmc 1:1`”，结果如图 33.4.1.1 所示：

```
=> ls mmc 1:1
6073416  zimage
37331   imx6ull-alientek-emmc.dtb
2 file(s), 0 dir(s)
=>
```

图 33.4.1.1 EMMC 分区 1 文件

从图 33.4.1.1 中可以看出，此时 EMMC 分区 1 中存在 `zimage` 和 `imx6ull-alientek-emmc.dtb` 这两个文件，所以我们可以测试新移植的 uboot 能不能启动 linux 内核。设置 `bootargs` 和 `bootcmd` 这两个环境变量，设置如下：

```
setenv bootargs 'console=ttymxc0,115200 root=/dev/mmcblk1p2 rootwait rw'
setenv bootcmd 'mmc dev 1; fatload mmc 1:1 80800000 zImage; fatload mmc 1:1 83000000
imx6ull-alientek-emmc.dtb; bootz 80800000 - 83000000;'
```

saveenv

设置好以后直接输入 boot, 或者 run bootcmd 即可启动 Linux 内核, 如果 Linux 内核启动成功的话就会输出如图 33.4.1.2 所示的启动信息:

```
switch to partitions #0, OK
mmc1(part 0) is current device
reading zImage
6073416 bytes read in 150 ms (38.6 MiB/s)
reading imx6ull-alientek-emmc.dtb
37331 bytes read in 18 ms (2 MiB/s)
Kernel image @ 0x80800000 [ 0x000000 - 0x5cac48 ]
## Flattened Device Tree blob at 83000000
   Booting using the fdt blob at 0x83000000
   Using Device Tree in place at 83000000, end 8300c1d2

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 4.1.15 (zuozhongkai@ubuntu) (gcc version 4.9.4 (Linaro GCC 4.9-2017.01) ) #10 SMP PREEMPT Mon Apr 22 15:21:47 CST 2019
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
Machine model: Freescale i.MX6 UltraLite 14x14 EVK Board
Reserved memory: created CMA memory pool at 0x8c000000, size 320 MiB
Reserved memory: initialized node linux,cma, compatible id shared-dma-pool
Memory policy: Data cache writealloc
PERCPU: Embedded 12 pages/cpu @8bb31000 s17280 r8192 d23680 u49152
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 130048
Kernel command line: console=ttyMxc0,115200 root=/dev/mmcblk1p2 rootwait rw
PID hash table entries: 2048 (order: 1, 8192 bytes)
Dentry cache hash table entries: 65536 (order: 6, 262144 bytes)
Inode-cache hash table entries: 32768 (order: 5, 131072 bytes)
Memory: 179900K/524288K available (7510K kernel code, 368K rwdata, 2596K rodata, 412K init, 442K bss, 16708K reserved, 327680K cma-reserved, 0K highmem)
Virtual kernel memory layout:
   vector : 0xffff0000 - 0xffff1000   ( 4 kB)
   fixmap : 0xffc00000 - 0xffff0000   (3072 kB)
```

图 33.4.1.2 linux 内核启动成功

33.4.2 从网络启动 Linux 系统

从网络启动 linux 系统的唯一目的就是为了调试! 不管是为了调试 linux 系统还是 linux 下的驱动。每次修改 linux 系统文件或者 linux 下的某个驱动以后都要将其烧写到 EMMC 中去测试, 这样太麻烦了。我们可以设置 linux 从网络启动, 也就是将 linux 镜像文件和根文件系统都放到 Ubuntu 下某个指定的文件夹中, 这样每次重新编译 linux 内核或者某个 linux 驱动以后只需要使用 cp 命令将其拷贝到这个指定的文件夹中即可, 这样就不用需要频繁的烧写 EMMC, 这样就加快了开发速度。我们可以通过 nfs 或者 tftp 从 Ubuntu 中下载 zImage 和设备树文件, 根文件系统的话也可以通过 nfs 挂载, 不过本小节我们不讲解如何通过 nfs 挂载根文件系统, 这个在讲解根文件系统移植的时候再讲解。这里我们使用 tftp 从 Ubuntu 中下载 zImage 和设备树文件, 前提是要将 zImage 和设备树文件放到 Ubuntu 下的 tftp 目录中, 具体方法在 30.4.4 小节讲解 tftp 命令的时候已经详细的介绍过了。

设置 bootargs 和 bootcmd 这两个环境变量, 设置如下:

```
setenv bootargs 'console=ttyMxc0,115200 root=/dev/mmcblk1p2 rootwait rw'
setenv bootcmd 'tftp 80800000 zImage; tftp 83000000 imx6ull-alientek-emmc.dtb; bootz 80800000 - 83000000'
saveenv
```

一开始是通过 tftp 下载 zImage 和 imx6ull-alientek-emmc.dtb 这两个文件, 过程如下图 33.4.2.1 所示:

[illegible]

图 33.4.2.1 下载过程

下载完成以后就是启动 Linux 内核，启动过程如图 33.4.2.2 所示：

```

Booting kernel ...

Booting Linux on physical CPU 0x0
Linux version 4.1.15 (zuozhongkai@ubuntu) (gcc version 4.9.4 (Linaro GCC 4.9-2017.01) ) #10 SMP PREEMPT Mon Apr 22 15:21:47 CST 2019
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
Machine model: Freescale i.MX6 UltraLite 14x14 EVK Board
Reserved memory: created CMA memory pool at 0x8c000000, size 320 MiB
Reserved memory: initialized node linux,cma, compatible id shared-dma-pool
Memory policy: Data cache writealloc
PERCPU: Embedded 12 pages/cpu @8bb31000 s17280 r8192 d23680 u49152
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 130048
Kernel command line: console=ttyMXC0,115200 root=/dev/mmcblkp2 rootwait rw
PID hash table entries: 2048 (order: 1, 8192 bytes)
Dentry cache hash table entries: 65536 (order: 6, 262144 bytes)
Inode-cache hash table entries: 32768 (order: 5, 131072 bytes)
Memory: 179900K/524288K available (7510K kernel code, 368K rwddata, 2596K rodata, 412K init, 442K bss, 16708K reserved, 327680K cma-reserved, 0K highmem)
Virtual kernel memory layout:
vector : 0xffff0000 - 0xffff1000   ( 4 kB)
fixmap : 0xffc00000 - 0xffff0000   (3072 kB)
vmalloc : 0xa0800000 - 0xff000000   (1512 MB)
lowmem  : 0x80000000 - 0xa0000000   ( 512 MB)
pkmap   : 0x7fe00000 - 0x80000000   ( 2 MB)
modules : 0x7f000000 - 0x7fe00000   ( 14 MB)
 .text : 0x80008000 - 0x809e6c20   (10108 kB)
 .init : 0x809e7000 - 0x80a4e000   ( 412 kB)
 .data : 0x80a4e000 - 0x80aaa220   ( 369 kB)
 .bss : 0x80aad000 - 0x80b1ba24   ( 443 kB)

```

图 33.4.2.2 Linux 启动过程

uboot 移植到此结束，简单总结一下 uboot 移植的过程：

①、不管是购买的开发板还是自己做的开发板，基本都是参考半导体厂商的 **dmeo** 板，而半导体厂商会在他们自己的开发板上移植好 **uboot**、**linux kernel** 和 **rootfs** 等，最终制作好 **BSP** 包提供给用户。我们可以在官方提供的 **BSP** 包的基础上添加我们的板子，也就是俗称的移植。

②、我们购买的开发板或者自己做的板子一般都不会原封不动的照抄半导体厂商的 demo 板，都会根据实际的情况来做修改，既然有修改就必然涉及到 uboot 下驱动的移植。

③、一般 uboot 中需要解决串口、NAND、EMMC 或 SD 卡、网络和 LCD 驱动，因为 uboot 的主要目的就是启动 Linux 内核，所以不需要考虑太多的外设驱动。

④、在 uboot 中添加自己的板子信息，根据自己板子的实际情况来修改 uboot 中的驱动。

第三十四章 U-Boot 图形化配置及其原理

在前两章中我们知道 uboot 可以通过 `mx6ull_alientek_emmc_defconfig` 来配置，或者通过文件 `mx6ull_alientek_emmc.h` 来配置 uboot。还有另外一种配置 uboot 的方法，就是图形化配置，以前的 uboot 是不支持图形化配置，只有 Linux 内核才支持图形化配置。不过不知道从什么时候开始，uboot 也支持图形化配置了，本章我们就来学习一下如何通过图形化配置 uboot，并且学习一下图形化配置的原理，因为后面学习 Linux 驱动开发的时候可能要修改图形配置文件。

34.1 U-Boot 图形化配置体验

uboot 或 Linux 内核可以通过输入“make menuconfig”来打开图形化配置界面，menuconfig 是一套图形化的配置工具，需要 ncurses 库支持。ncurses 库提供了一系列的 API 函数供调用者生成基于文本的图形界面，因此需要先在 Ubuntu 中安装 ncurses 库，命令如下：

```
sudo apt-get install build-essential
sudo apt-get install libncurses5-dev
```

menuconfig 重点会用到两个文件：.config 和 Kconfig，.config 文件前面已经说了，这个文件保存着 uboot 的配置项，使用 menuconfig 配置完 uboot 以后肯定要更新.config 文件。Kconfig 文件是图形界面的描述文件，也就是描述界面应该有什么内容，很多目录下都会有 Kconfig 文件。

在打开图形化配置界面之前，要先使用“make xxx_defconfig”对 uboot 进行一次默认配置，只需要一次即可。如果使用“make clean”清理了工程的话就那就需要重新使用“make xxx_defconfig”再对 uboot 进行一次配置。进入 uboot 根目录，输入如下命令：

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- mx6ull_alientek_emmc_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- menuconfig
```

如果已经在 uboot 的顶层 Makefile 中定义了 ARCH 和 CROSS_COMPILE 的值，那么上述命令可以简化为：

```
make mx6ull_alientek_emmc_defconfig
make menuconfig
```

打开后的图形化界面如图 34.1.1 所示：

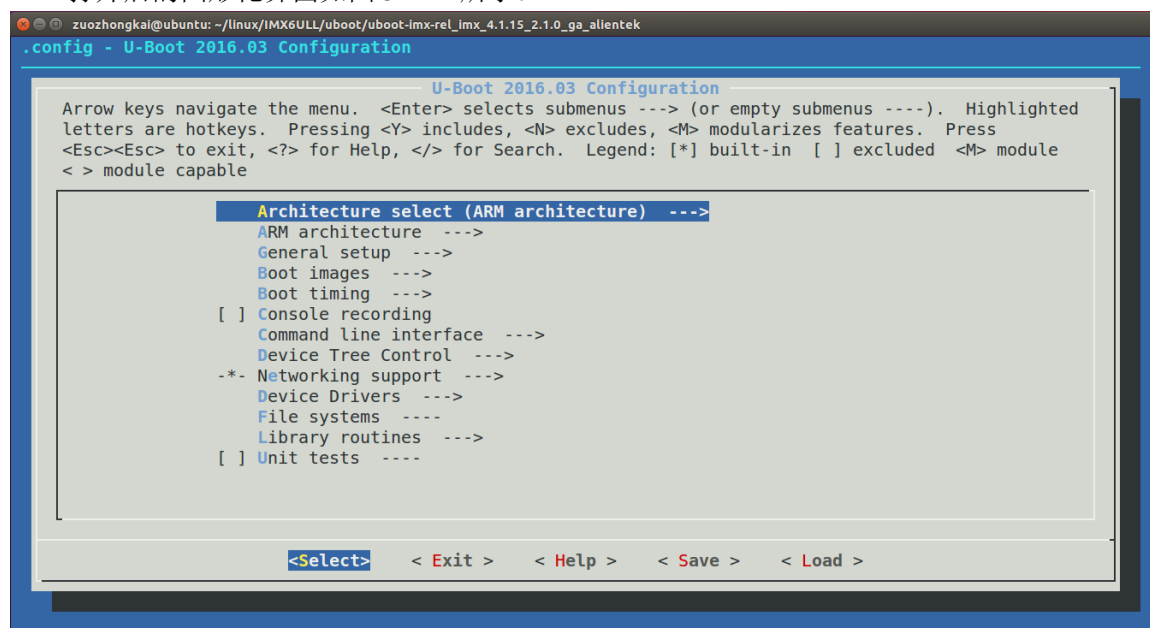


图 34.1.1 uboot 图形化配置界面

图 34.1.1 就是主界面，主界面上方的英文就是简单的操作说明，操作方法如下：

通过键盘上的“↑”和“↓”键来选择要配置的菜单，按下“Enter”键进入子菜单。菜单中高亮的字母就是此菜单的热键，在键盘上按下此高亮字母对应的键可以快速选中对应的菜单。选中子菜单以后按下“Y”键就会将相应的代码编译进 Uboot 中，菜单前面变为“<*>”。按下“N”键不编译相应的代码，按下“M”键就会将相应的代码编译为模块，菜单前面变为“<M>”。按两下“Esc”键退出，也就是返回到上一级，按下“?”键查看此菜单的帮助信息，按下“/”

键打开搜索框，可以在搜索框输入要搜索的内容。

在配置界面下方会有五个按钮，这五个按钮的功能如下：

<Select>：选中按钮，和“Enter”键的功能相同，负责选中并进入某个菜单。

<Exit>：退出按钮，和按下“Esc”键功能相同，退出当前菜单，返回到上一级。

<Help>：帮助按钮，查看选中菜单的帮助信息。

<Save>：保存按钮，保存修改后的配置文件。

<Load>：加载按钮，加载指定的配置文件。

在图 34.1.1 中共有 13 个配置主配置项，通过键盘上的上下键调节配置项。后面跟着“--->”表示此配置项是有子配置项的，按下回车键就可以进入子配置项。

我们就以如何使能 dns 命令为例，讲解一下如何通过图形化界面来配置 uboot。进入“Command line interface --->”这个配置项，此配置项用于配置 uboot 的命令，进入以后如图 34.1.2 所示：

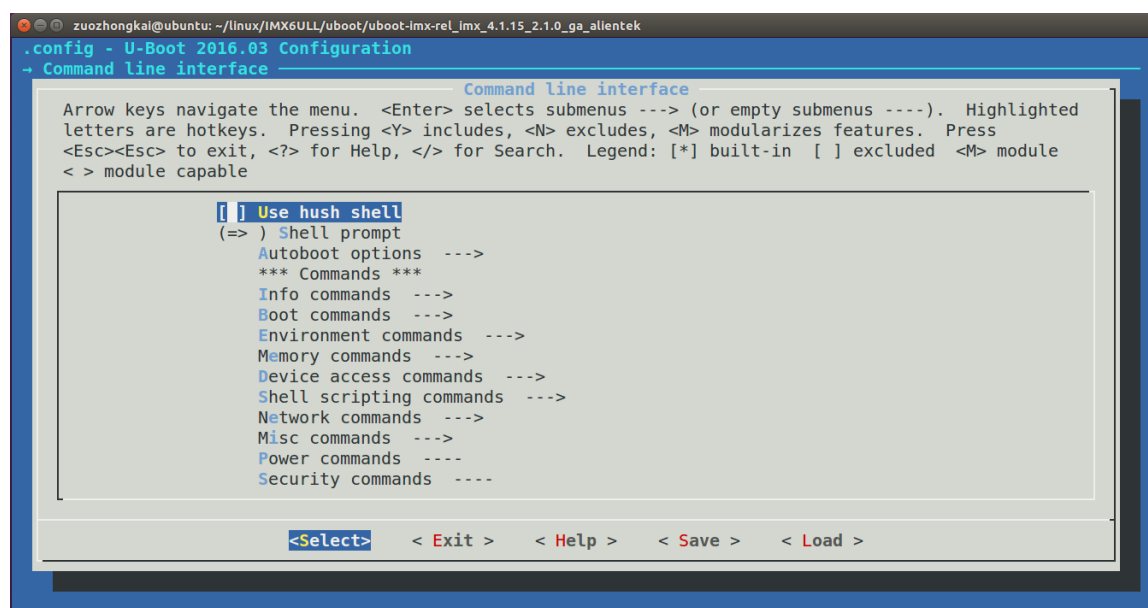


图 34.1.2 Command line interface 配置项

从图 34.1.2 可以看出，有很多配置项，这些配置项也有子配置项，选择“Network commands --->”，进入网络相关命令配置项，如图 34.1.3 所示：

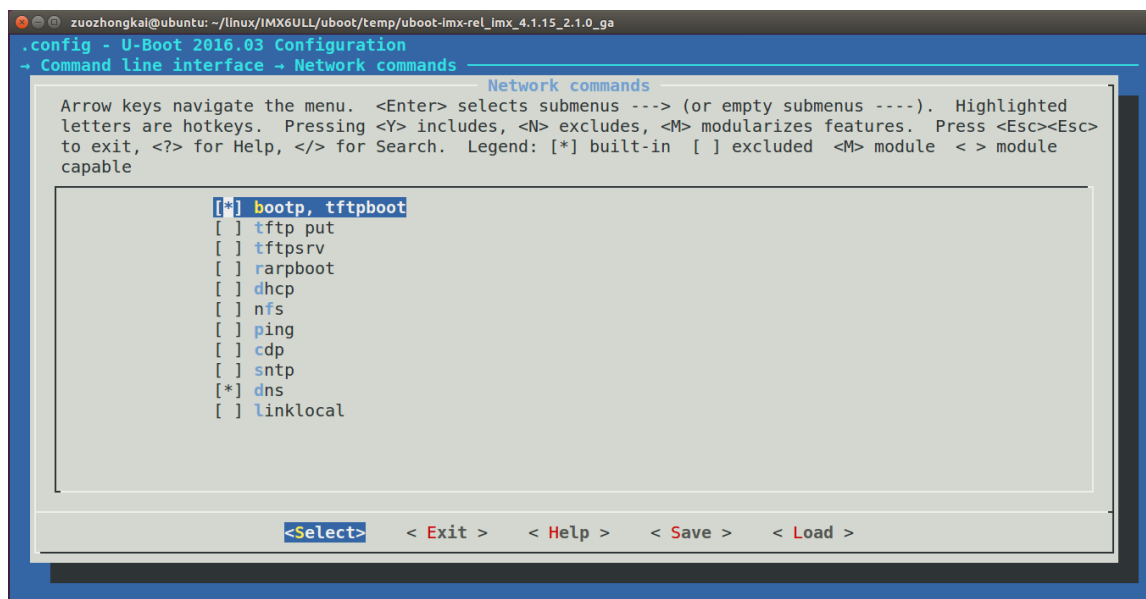


图 34.1.3 Network commands 配置项

从图 34.1.3 可以看出, uboot 中有很多和网络有关的命令, 比如 bootp、tftpboot、dhcp 等等。选中 dns, 然后按下键盘上的“Y”键, 此时 dns 前面的“[]”变成了“[*]”, 如图 34.1.4 所示:

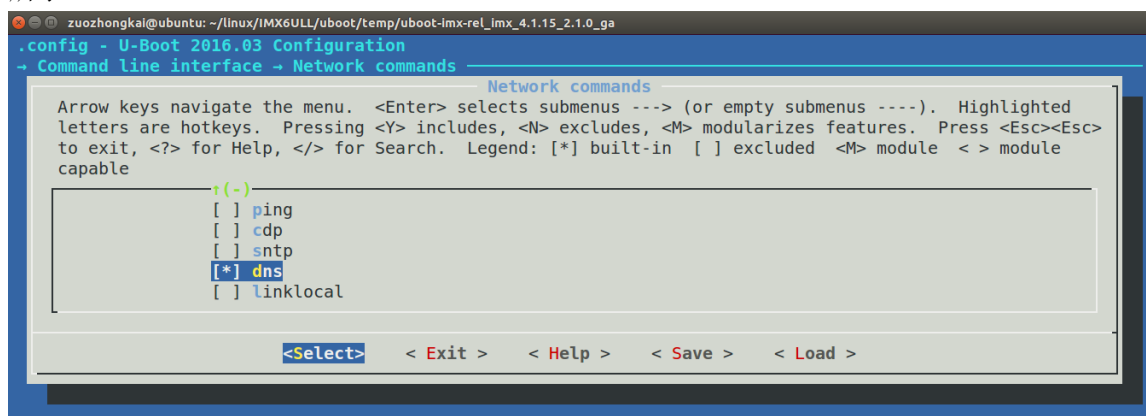


图 34.1.4 选中 dns 命令

每个选项有 3 种编译选项: 编译进 uboot 中(也就是编译进 u-boot.bin 中)、取消编译(也就是不编译这个功能模块)、编译为模块。按下“Y”键表示编译进 uboot 中, 此时“[]”变成了“[*]”; 按下“N”表示不编译, “[]”默认表示不编译; 有些功能模块是支持编译为模块的, 这个一般在 Linux 内核里面很常用, uboot 下面不使用, 如果要将某个功能编译为模块, 那就按下“M”, 此时 “[]”就会变为“<M>”。

细心的朋友应该会发现, 在 mx6ull_alientek_emmc.h 里面我们配置使能了 dhcp 和 ping 命令, 但是在图 34.1.3 中 dhcp 和 ping 前面的 “[]”并不是“[*]”, 也就是说不编译 dhcp 和 ping 命令, 这不是冲突了吗? 实际情况是 dhcp 和 ping 命令是会编译的。之所以在图 34.1.3 中没有体现出来是因为我们是直接在 mx6ull_alientek_emmc.h 中定义的宏 CONFIG_CMD_PING 和 CONFIG_CMD_DHCP, 而 menuconfig 是通过读取 .config 文件来判断使能了哪些功能, .config 里面并没有宏 CONFIG_CMD_PING 和 CONFIG_CMD_DHCP, 所以 menuconfig 就会识别出错。

选中 dns, 然后按下“H”或者“?”键可以打开 dns 命令的提示信息, 如图 34.1.5 所示:

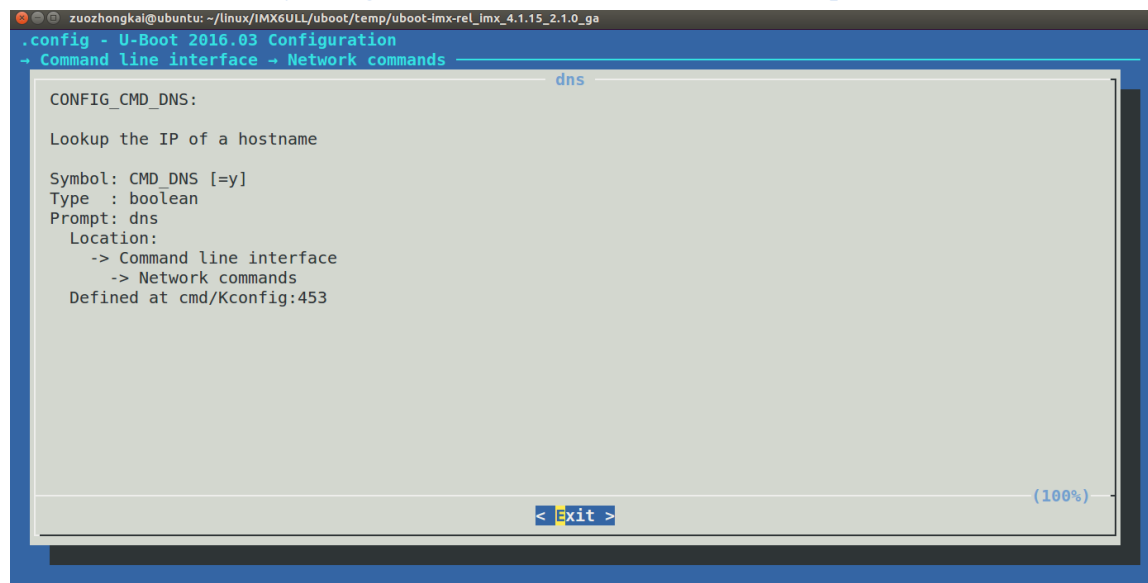


图 34.1.5 dns 命令提示信息

按两下 ESC 键即可退出提示界面，相当于返回上一层。选择 dns 命令以后，按两下 ESC 键（按两下 ESC 键相当于返回上一层），退出当前配置项，进入到上一层配置项。如果没有要修改的就按两下 ESC 键，退出到主配界面，如果也没有其他要修改的，那就再次按两下 ESC 键退出 menuconfig 配置界面。如果修改过配置的话，在退出主界面的时候会有如图 34.1.6 所示提示：

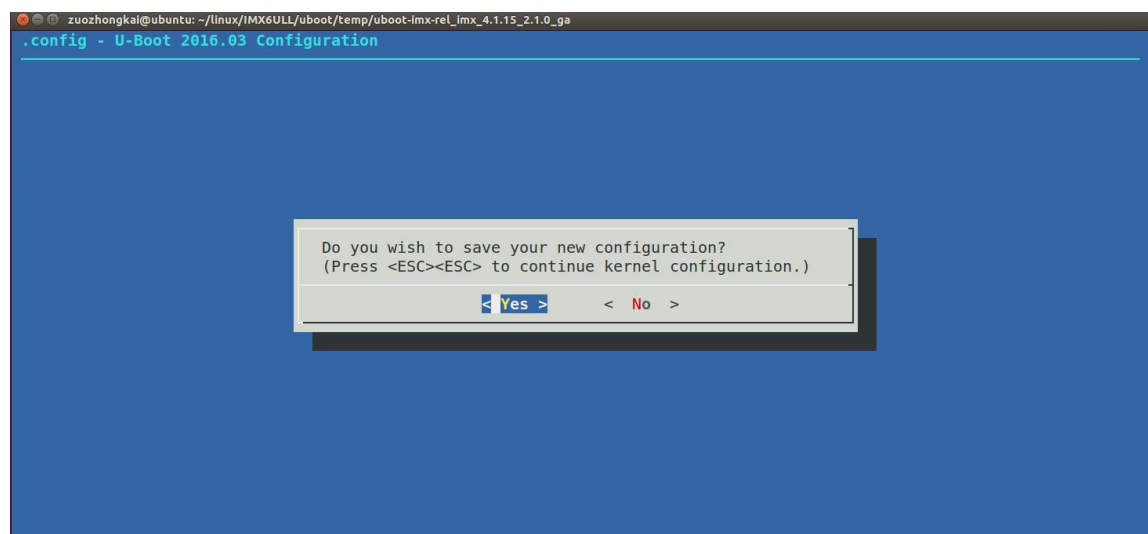


图 34.1.6 是否保存新的配置文件对话框

图 34.1.6 询问是否保存新的配置文件，通过键盘的←或→键来选择“ Yes”项，然后按下键盘上的回车键确认保存。至此，我们就完成了通过图形界面使能了 uboot 的 dns 命令，打开.config 文件，会发现多了“CONFIG_CMD_DNS=y”这一行，如图 34.1.7 中的 323 行所示：

```

zuozhongkai@ubuntu: ~/linux/IMX6ULL/uboot/temp/uboot-imx-rel_imx_4.1.15_2.1.0_ga
318 # CONFIG_CMD_DHCP is not set
319 # CONFIG_CMD_NFS is not set
320 # CONFIG_CMD_PING is not set
321 # CONFIG_CMD_CDP is not set
322 # CONFIG_CMD_SNTP is not set
323 CONFIG_CMD_DNS=y
324 # CONFIG_CMD_LINK_LOCAL is not set
325
326 #
327 # Misc commands
328 #
329 # CONFIG_CMD_TIME is not set
330 CONFIG_CMD_MISC=y
331 # CONFIG_CMD_TIMER is not set
331,12 57%

```

图 34.1.7 .config 文件

使用如下命令编译 uboot:

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j16
```

千万不能使用如下命令:

```
./mx6ull_alientek_emmc.sh
```

因为 mx6ull_alientek_emmc.sh 在编译之前会清理工程，会删除掉 .config 文件！通过图形化界面配置所有配置项都会被删除，结果就是竹篮打水一场空。

编译完成以后烧写到 SD 卡中，重启开发板进入 uboot 命令模式，输入“?”查看是否有“dns”命令，一般肯定有的。使用 dns 命令来查看一下百度官网“www.baidu.com”的 IP 地址。

注意！ 如果要与外部互联网通信，比如百度官网，这个时候要保证开发板能访问到外部互联网。如果你的开发板和电脑直接用网线连接的，那么肯定无法连接到外部网络，这个时候 dns 命令查看百度官网也会失败！所以开发板一定要连接到路由器上，而且要保证你的路由器能访问外网，比如手机连接到这个路由器上以后可以正常访问互联网。

要先设置一下 dns 服务器的 IP 地址，也就是设置环境变量 dnsip 的值，命令如下:

```
setenv dnsip 114.114.114.114
```

```
saveenv
```

设置好以后就可以使用 dns 命令查看百度官网的 IP 地址了，输入命令:

```
dns www.baidu.com
```

结果如图 34.1.8 所示:

```

=> dns www.baidu.com
FEC1 Waiting for PHY auto negotiation to complete.... done
14.215.177.38
=>

```

图 34.1.8 dns 命令

从图 34.1.7 可以看出，“www.baidu.com”的 IP 地址为 14.215.177.38，说明 dns 命令工作正常。这个就是通过图形化命令来配置 uboot，一般用来使能一些命令还是很方便的，这样就不需要到处找命令的配置宏是什么，然后在到配置文件里面去定义。

34.2 menuconfig 图形化配置原理

34.2.1 make menuconfig 过程分析

当输入 make menuconfig 以后会匹配到顶层 Makefile 的如下代码:

示例代码 34.2.1.1 顶层 Makefile 代码段

```

489 %config: scripts_basic outputmakefile FORCE
490     $(Q)$(MAKE) $(build)=scripts/kconfig $@

```

这个在 31.3.13 小节已经详细的讲解过了，其中 `build=f ./scripts/Makefile.build obj`，将 490 行的规则展开就是：

```
@ make -f ./scripts/Makefile.build obj=scripts/kconfig menuconfig
```

Makefile.build 会读取 scripts/kconfig/Makefile 中的内容，在 scripts/kconfig/Makefile 中可以找到如下代码：

示例代码 34.2.1.2 scripts/kconfig/Makefile 代码段

```
36 menuconfig: $(obj)/mconf
37     $< $(silent) $(Kconfig)
```

其中 `obj= scripts/kconfig`, `silent` 是设置静默编译的，在这里可以忽略不计，`Kconfig=Kconfig`，因此扩展以后就是：

```
menuconfig: scripts/kconfig/mconf
scripts/kconfig/mconf Kconfig
```

目标 `menuconfig` 依赖 `scripts/kconfig/mconf`，因此 `scripts/kconfig/mconf.c` 这个文件会被编译，生成 `mconf` 这个可执行文件。目标 `menuconfig` 对应的规则为 `scripts/kconfig/mconf Kconfig`，也就是说 `mconf` 会调用 `uboot` 根目录下的 `Kconfig` 文件开始构建图形配置界面。

34.2.2 Kconfig 语法简介

上一小节我们已经知道了 `scripts/kconfig/mconf` 会调用 `uboot` 根目录下的 `Kconfig` 文件开始构建图形化配置界面，接下来简单学习一下 `Kconfig` 的语法。因为后面学习 Linux 驱动开发的时候可能会涉及到修改 `Kconfig`，对于 `Kconfig` 语法我们不需要太深入的去研究，关于 `Kconfig` 的详细语法介绍，可以参考 `linux` 内核源码(不知为何 `uboot` 源码中没有这个文件)中的文件 `Documentation/kbuild/kconfig-language.txt`，本节我们大概了解其原理即可。打开 `uboot` 根目录下的 `Kconfig`，这个 `Kconfig` 文件就是顶层 `Kconfig`，我们就以这个文件为例来简单学习一下 `Kconfig` 语法。

1、mainmenu

顾名思义 `mainmenu` 就是主菜单，也就是输入 “`make menuconfig`” 以后打开的默认界面，在顶层 `Kconfig` 中有如下代码：

示例代码 34.2.2.1 顶层 Kconfig 代码段

```
5 mainmenu "U-Boot $UBOOTVERSION Configuration"
```

上述代码就是定义了一个名为 “`U-Boot $UBOOTVERSION Configuration`” 的主菜单，其中 `UBOOTVERSION=2016.03`，因此主菜单名为 “`U-Boot 2016.03 Configuration`”，如图 34.2.2.1 所示：

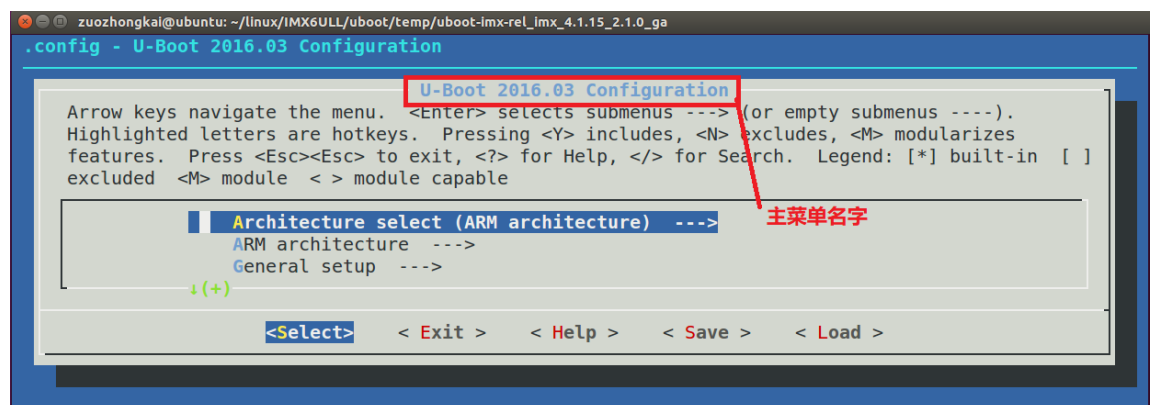


图 34.2.2.1 主菜单名字

2、调用其他目录下的 Kconfig 文件

和 makefile 一样, Kconfig 也可以调用其他子目录中的 Kconfig 文件, 调用方法如下:

```
source "xxx/Kconfig" //xxx 为具体的目录名, 相对路径
```

在顶层 Kconfig 中有如下代码:

示例代码 34.2.2.2 顶层 Kconfig 代码段

```
12 source "arch/Kconfig"
.....
225
226 source "common/Kconfig"
227
228 source "cmd/Kconfig"
229
230 source "dts/Kconfig"
231
232 source "net/Kconfig"
233
234 source "drivers/Kconfig"
235
236 source "fs/Kconfig"
237
238 source "lib/Kconfig"
239
240 source "test/Kconfig"
```

从示例代码 34.2.2.2 中可以看出, 顶层 Kconfig 文件调用了很多其他子目录下的 Kconfig 文件, 这些子目录下的 Kconfig 文件在主菜单中生成各自的菜单项。

3、menu/endmenu 条目

menu 用于生成菜单, endmenu 就是菜单结束标志, 这两个一般是成对出现的。在顶层 Kconfig 中有如下代码:

示例代码 34.2.2.3 顶层 Kconfig 代码段

```
14 menu "General setup"
15
16 config LOCALVERSION
17     string "Local version - append to U-Boot release"
18     help
19         Append an extra string to the end of your U-Boot version.
20         This will show up on your boot log, for example.
21         The string you set here will be appended after the contents of
22         any files with a filename matching localversion* in your
23         object and source tree, in that order. Your total string can
24         be a maximum of 64 characters.
25
```

```

.....
100 endmenu      # General setup
101
102 menu "Boot images"
103
104 config SUPPORT_SPL
105     bool
106
.....
224 endmenu      # Boot images

```

示例代码 34.2.2.3 中有两个 menu/endmenu 代码块，这两个代码块就是两个子菜单，第 14 行的“menu "General setup"”表示子菜单“General setup”。第 102 行的“menu "Boot images"”表示子菜单“Boot images”。体现在主菜单界面中就如同图 34.2.2.2 所示：

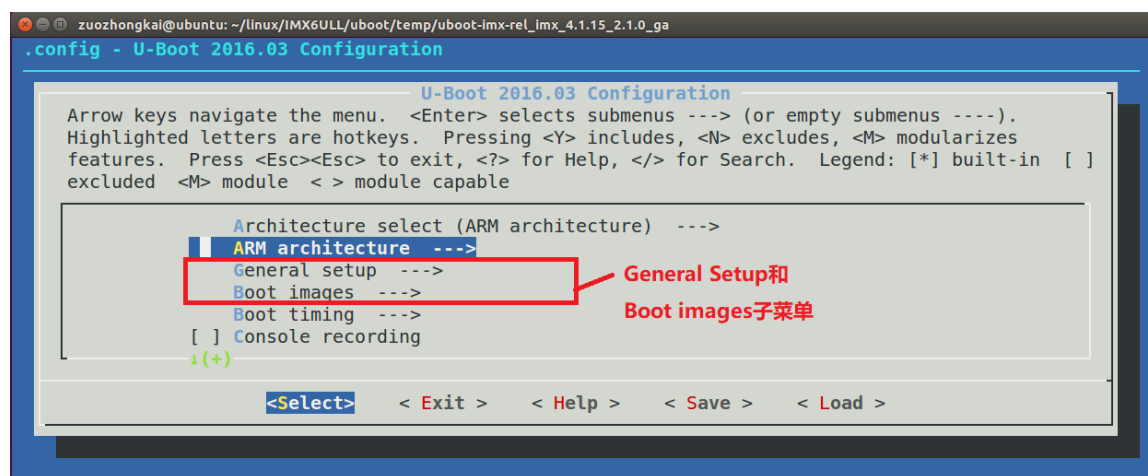


图 34.2.2.2 子菜单

在“General setup”菜单上面还有“Architecture select (ARM architecture)”和“ARM architecture”这两个子菜单，但是在顶层 Kconfig 中并没有看到这两个子菜单对应的 menu/endmenu 代码块，那这两个子菜单是怎么来的呢？这两个子菜单就是 arch/Kconfig 文件生成的。包括主界面中的“Boot timing”、“Console recording”等等这些子菜单，都是分别由顶层 Kconfig 所调用的 common/Kconfig、cmd/Kconfig 等这些子 Kconfig 文件来创建的。

3、config 条目

顶层 Kconfig 中的“General setup”子菜单内容如下：

示例代码 34.2.2.4 顶层 Kconfig 代码段

```

14 menu "General setup"
15
16 config LOCALVERSION
17     string "Local version - append to U-Boot release"
18     help
19         Append an extra string to the end of your U-Boot version.
20         This will show up on your boot log, for example.
21         The string you set here will be appended after the contents of
22         any files with a filename matching localversion* in your

```

```
23     object and source tree, in that order. Your total string can
24     be a maximum of 64 characters.
25
26 config LOCALVERSION_AUTO
27     bool "Automatically append version information to the version
string"
28     default y
29     help
.....
45
46 config CC_OPTIMIZE_FOR_SIZE
47     bool "Optimize for size"
48     default y
49     help
.....
54
55 config SYS_MALLOC_F
56     bool "Enable malloc() pool before relocation"
57     default y if DM
58     help
.....
63
64 config SYS_MALLOC_F_LEN
65     hex "Size of malloc() pool before relocation"
66     depends on SYS_MALLOC_F
67     default 0x400
68     help
.....
73
74 menuconfig EXPERT
75     bool "Configure standard U-Boot features (expert users)"
76     default y
77     help
.....
82
83 if EXPERT
84     config SYS_MALLOC_CLEAR_ON_INIT
85     bool "Init with zeros the memory reserved for malloc (slow)"
86     default y
87     help
.....
99 endif
100 endmenu      # General setup
```

可以看出，在 menu/endmenu 代码块中有大量的“config xxxx”的代码块，也就是 config 条目。config 条目就是“General setup”菜单的具体配置项，如图 34.2.2.3 所示：

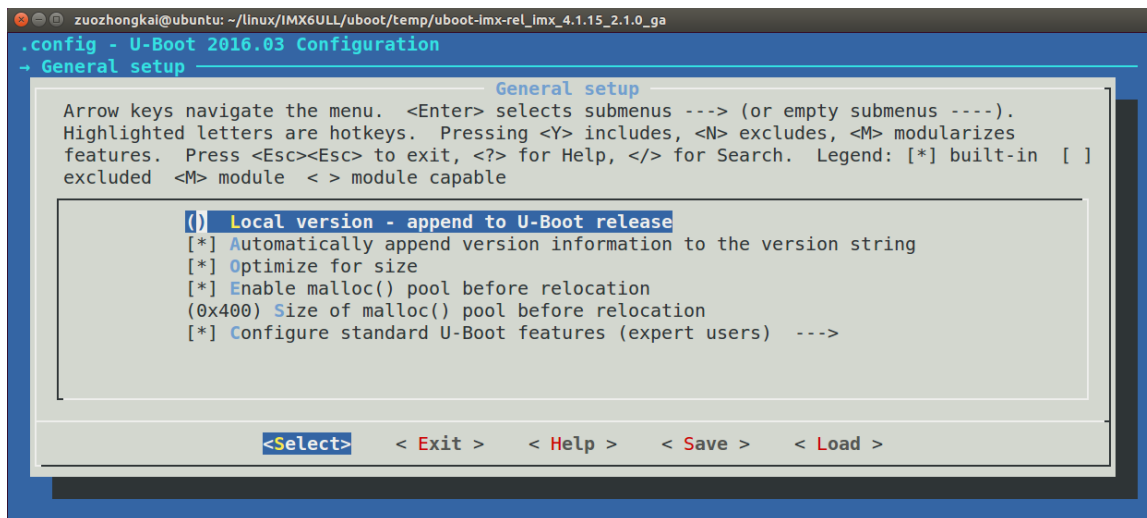


图 34.2.2.3 General setup 配置项

“config LOCALVERSION”对应着第一个配置项，“config LOCALVERSION_AUTO”对应着第二个配置项，以此类推。我们以“config LOCALVERSION”和“config LOCALVERSION_AUTO”这两个为例来分析一下 config 配置项的语法：

示例代码 34.2.2.5 顶层 Kconfig 代码段

```

16 config LOCALVERSION
17 string "Local version - append to U-Boot release"
18 help
19 Append an extra string to the end of your U-Boot version.
20 .....
21 be a maximum of 64 characters.
22
23
26 config LOCALVERSION_AUTO
27 bool "Automatically append version information to the version
string"
28 default y
29 help
30 This will try to automatically determine if the current tree is a
31 release tree by looking for git tags that belong to the current
32 .....
33
34
43
44 which is done within the script "scripts/setlocalversion".)

```

第 16 和 26 行，这两行都以 config 关键字开头，后面跟着 LOCALVERSION 和 LOCALVERSION_AUTO，这两个就是配置项名字。假如我们使能了 LOCALVERSION_AUTO 这个功能，那么就会在 .config 文件中生成 CONFIG_LOCALVERSION_AUTO，这个在上一小节讲解如何使能 dns 命令的时候讲过了。由此可知，.config 文件中的“CONFIG_xxx” (xxx 就是具体的配置项名字)就是 Kconfig 文件中 config 关键字后面的配置项名字加上“CONFIG_”前缀。

config 关键字下面的这几行是配置项属性, 17~24 行是 LOCALVERSION 的属性, 27~44 行是 LOCALVERSION_AUTO 的属性。属性里面描述了配置项的类型、输入提示、依赖关系、帮助信息和默认值等。

第 17 行的 string 是变量类型, 也就是“CONFIG_LOCALVERSION”的变量类型。可以为: bool、tristate、string、hex 和 int, 一共 5 种。最常用的是 bool、tristate 和 string 这三种, bool 类型有两种值: y 和 n, 当为 y 的时候表示使能这个配置项, 当为 n 的时候就禁止这个配置项。tristate 类型有三种值: y、m 和 n, 其中 y 和 n 的涵义与 bool 类型一样, m 表示将这个配置项编译为模块。string 为字符串类型, 所以 LOCALVERSION 是个字符串变量, 用来存储本地字符串, 选中以后即可输入用户定义的本地版本号, 如图 34.2.2.4 所示:

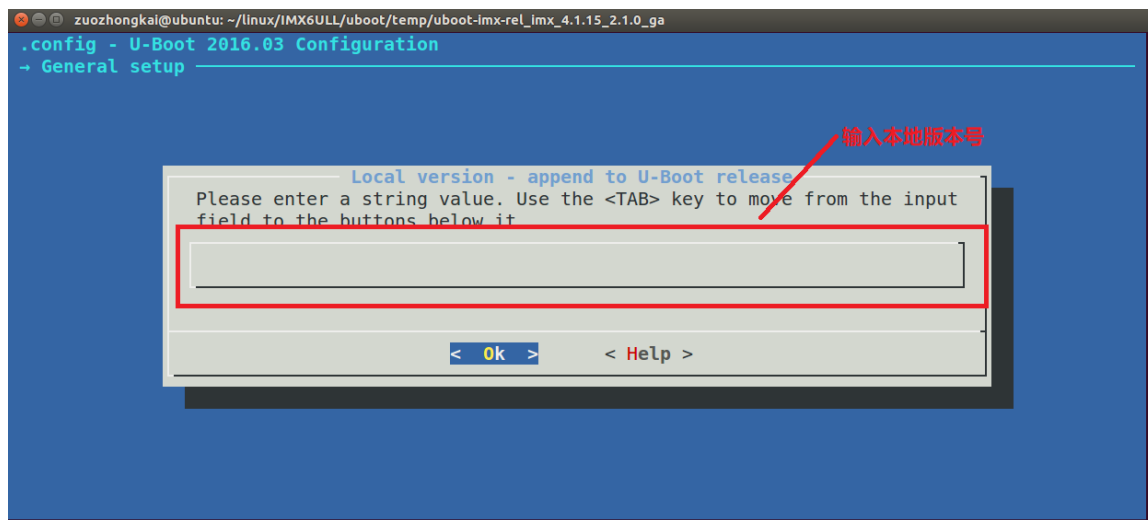


图 34.2.2.4 本地版本号配置

string 后面的“Local version - append to U-Boot release”就是这个配置项在图形界面上的显示出来的标题。

第 18 行, help 表示帮助信息, 告诉我们配置项的含义, 当我们按下“h”或“?”弹出来的帮助界面就是 help 的内容。

第 27 行, 说明“CONFIG_LOCALVERSION_AUTO”是个 bool 类型, 可以通过按下 Y 或 N 键来使能或者禁止 CONFIG_LOCALVERSION_AUTO。

第 28 行, “default y”表示 CONFIG_LOCALVERSION_AUTO 的默认值就是 y, 所以这一行默认会被选中。

4、depends on 和 select

打开 arch/Kconfig 文件, 在里面有这如下代码:

示例代码 34.2.2.6 arch/Kconfig 代码段

```

7 config SYS_GENERIC_BOARD
8     bool
9     depends on HAVE_GENERIC_BOARD
10
11 choice
12     prompt "Architecture select"
13     default SANDBOX
14
15 config ARC

```

```

16     bool "ARC architecture"
17     select HAVE_PRIVATE_LIBGCC
18     select HAVE_GENERIC_BOARD
19     select SYS_GENERIC_BOARD
20     select SUPPORT_OF_CONTROL

```

第 9 行,“depends on”说明“SYS_GENERIC_BOARD”项依赖于“HAVE_GENERIC_BOARD”,也就是说“HAVE_GENERIC_BOARD”被选中以后“SYS_GENERIC_BOARD”才能被选中。

第 17~20 行,“select”表示方向依赖,当选中“ARC”以后,“HAVE_PRIVATE_LIBGCC”、“HAVE_GENERIC_BOARD”、“SYS_GENERIC_BOARD”和“SUPPORT_OF_CONTROL”这四个也会被选中。

4、choice/endchoice

在 arch/Kconfig 文件中有如下代码:

示例代码 34.2.2.7 arch/Kconfig 代码段

```

11 choice
12     prompt "Architecture select"
13     default SANDBOX
14
15 config ARC
16     bool "ARC architecture"
17     .....
21
22 config ARM
23     bool "ARM architecture"
24     .....
29
30 config AVR32
31     bool "AVR32 architecture"
32     .....
35
36 config BLACKFIN
37     bool "Blackfin architecture"
38     .....
40
41 config M68K
42     bool "M68000 architecture"
43     .....
117
118 endchoice

```

choice/endchoice 代码段定义了一组可选择项,将多个类似的配置项组合在一起,供用户单选或者多选。示例代码 34.2.2.7 就是选择处理器架构,可以从 ARC、ARM、AVR32 等这些架构中选择,这里是单选。在 uboot 图形配置界面上选择“Architecture select”,进入以后如图 34.2.2.5 所示:

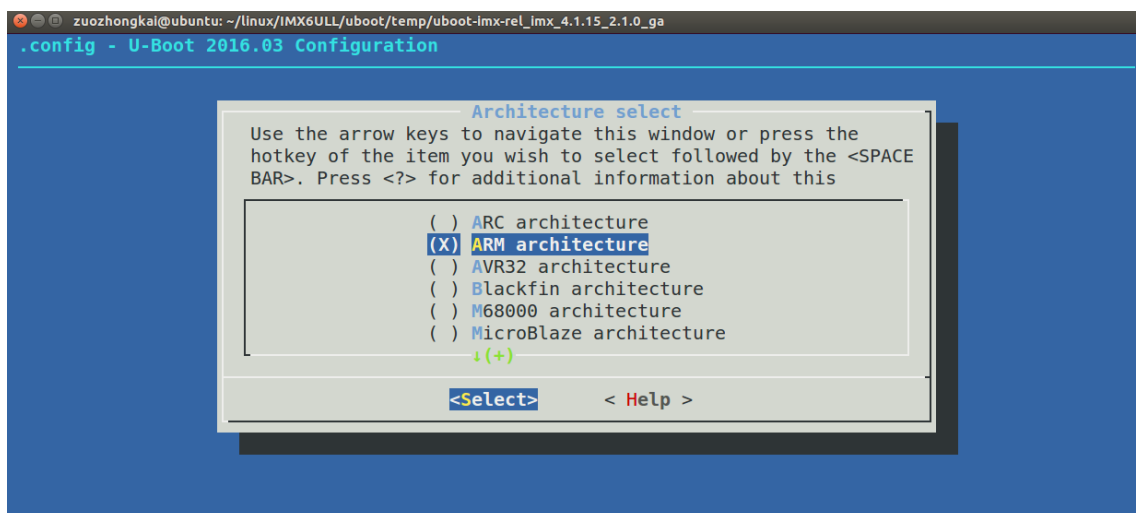


图 34.2.2.5 架构选择界面

可以在图 34.2.2.5 中通过移动光标来选择所使用的 CPU 架构。第 12 行的 prompt 给出这个 choice/endchoice 段的提示信息为“Architecture select”。

5、menuconfig

menuconfig 和 menu 很类似，但是 menuconfig 是个带选项的菜单，其一般用法为：

示例代码 34.2.2.8 menuconfig 用法

```
1 menuconfig MODULES
2     bool "菜单"
3 if MODULES
4 ...
5 endif # MODULES
```

第 1 行，定义了一个可选的菜单 MODULES，只有选中了 MODULES 第 3~5 行 if 到 endif 之间的内容才会显示。在顶层 Kconfig 中有如下代码：

示例代码 34.2.2.9 顶层 Kconfig 代码段

```
14 menu "General setup"
.....
74 menuconfig EXPERT
75     bool "Configure standard U-Boot features (expert users)"
76     default y
77     help
78         This option allows certain base U-Boot options and settings
79         to be disabled or tweaked. This is for specialized
80         environments which can tolerate a "non-standard" U-Boot.
81         Only use this if you really know what you are doing.
82
83 if EXPERT
84     config SYS_MALLOC_CLEAR_ON_INIT
85     bool "Init with zeros the memory reserved for malloc (slow)"
86     default y
87     help
```

```

88      This setting is enabled by default. The reserved malloc
89      memory is initialized with zeros, so first malloc calls
.....
98      should be replaced by calloc - if expects zeroed memory.
99  endif
100 endmenu      # General setup

```

第 74~99 行使用 menuconfig 实现了一个菜单，路径如下：

General setup

-> Configure standard U-Boot features (expert users) --->

如图 34.2.2.6 所示：

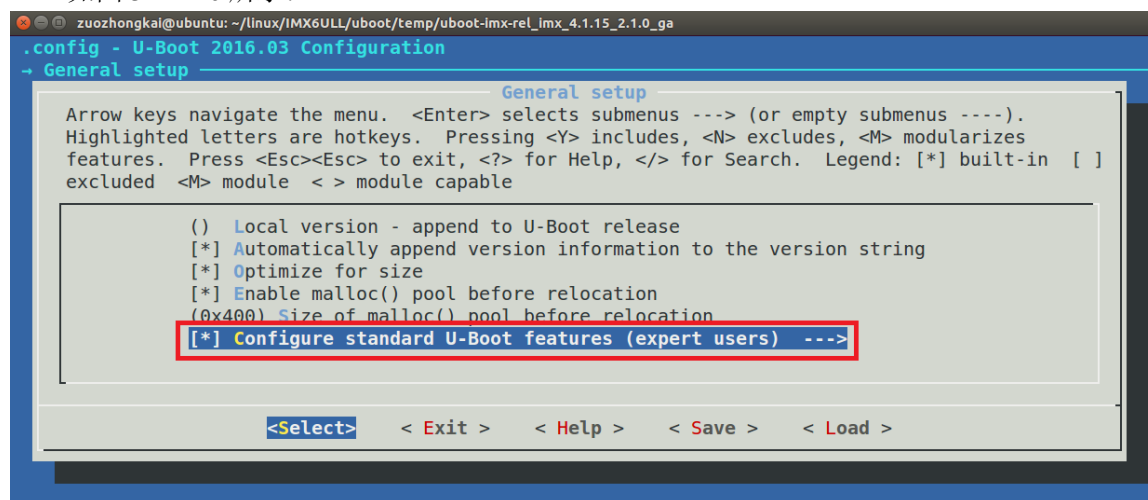


图 34.2.2.6 菜单 Configure standard U-Boot features (expert users)

从图 34.2.2.6 可以看到，前面有 “[]” 说明这个菜单是可选的，当选中这个菜单以后就可以进入到子选项中，也就是示例代码 34.2.2.9 中的第 83~99 行所描述的菜单，如图 34.2.2.7 所示：

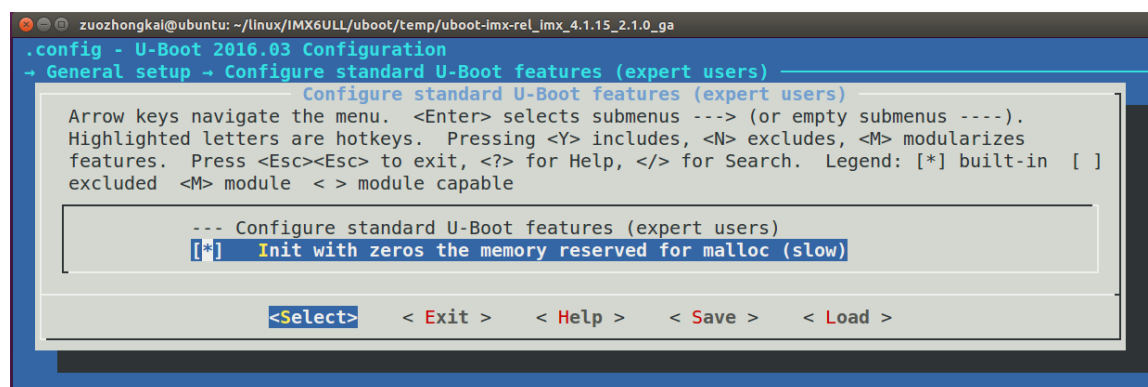


图 34.2.2.7 菜单 Init with zeros the memory reserved for malloc (slow)

如果不选择 “Configure standard U-Boot features (expert users)”，那么示例代码 34.2.2.9 中的第 83~99 行所描述的菜单就不会显示出来，进去以后是空白的。

6、comment

comment 用于注释，也就是在图形化界面中显示一行注释，打开文件 drivers/mtd/nand/Kconfig，有如下所示代码：

示例代码 34.2.2.10 drivers/mtd/nand/Kconfig 代码段

```

74 config NAND_ARASAN

```

```
75     bool "Configure Arasan Nand"
76     help
.....
80
81 comment "Generic NAND options"
```

第 81 行使用 `comment` 标注了一行注释，注释内容为：“Generic NAND options”，这行注释在配置项 `NAND_ARASAN` 的下面。在图形化配置界面中按照如下路径打开：

-> Device Drivers

-> NAND Device Support

结果如图 34.2.2.8 所示：

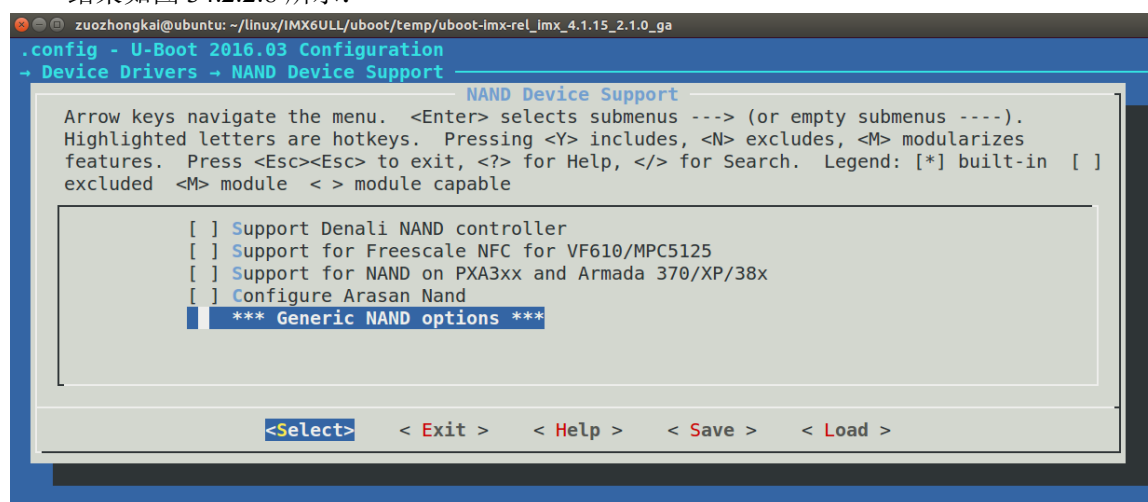


图 34.2.2.8 注释 “Generic NAND options”

从图 34.2.2.8 可以看出，在配置项 “Configure Arasan Nand” 下面有一行注释，注释内容为 “*** Generic NAND options ***”。

7、source

`source` 用于读取另一个 `Kconfig`，比如：

```
source "arch/Kconfig"
```

这个在前面已经讲过了。

`Kconfig` 语法就讲解到这里，基本上常用的语法就是这些，因为 `uboot` 相比 `Linux` 内核要小很多，所以配置项也要少很多，所以建议大家使用 `uboot` 来学习 `Kconfig`。一般不会修改 `uboot` 中的 `Kconfig` 文件，甚至都不会使用 `uboot` 的图形化界面配置工具，本小节学习 `Kconfig` 的目的主要还是为了 `Linux` 内核作准备。

34.3 添加自定义菜单

图形化配置工具的主要工作就是在 `.config` 下面生成前缀为 “`CONFIG_`” 的变量，这些变量一般都要值，为 `y`，`m` 或 `n`，在 `uboot` 源码里面会根据这些变量来决定编译哪个文件。本小节我们就来学习一下如何添加自己的自定义菜单，自定义菜单要求如下：

- ①、在主界面中添加一个名为 “My test menu”，此菜单内部有一个配置项。
- ②、配置项为 “`MY_TESTCONFIG`”，此配置项处于菜单 “My test menu” 中。
- ③、配置项的变量类型为 `bool`，默认值为 `y`。
- ④、配置项菜单名字为 “This is my test config”。

⑤、配置项的帮助内容为 “This is a empty config, just for tset!”。

打开顶层 Kconfig，在最后面加入如下代码：

示例代码 34.3.1 自定义菜单

```
1 menu "My test menu"
2
3 config MY_TESTCONFIG
4     bool "This is my test config"
5     default y
6     help
7         This is a empty config, just for test!
8
9 endmenu          # my test menu
```

添加完成以后打开图形化配置界面，如图 34.3.1 所示：

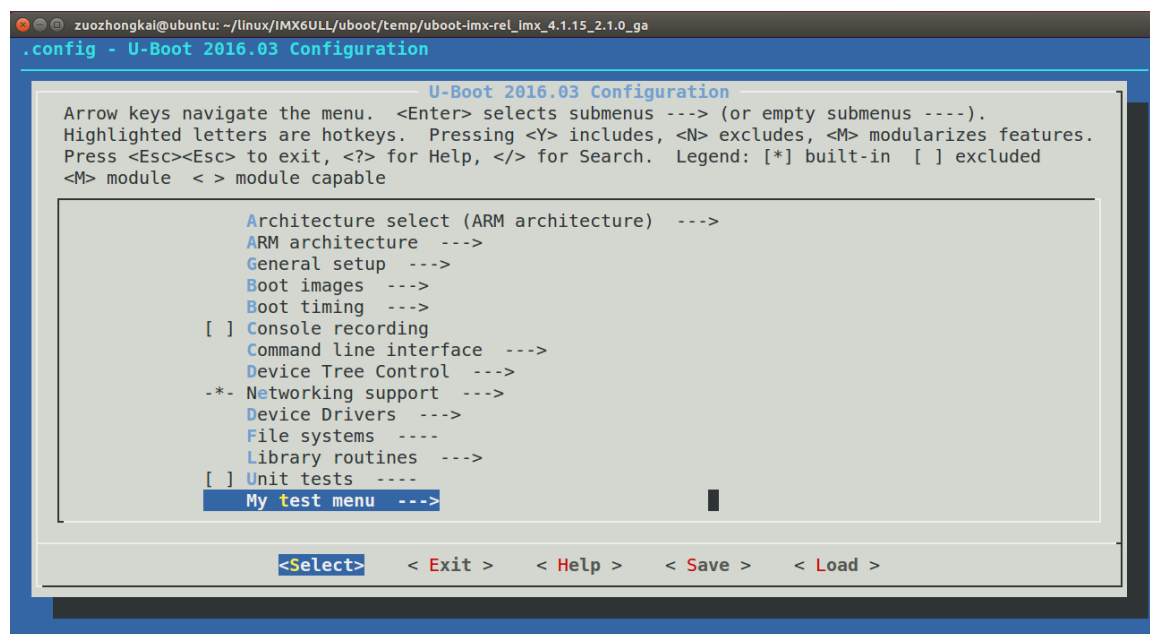


图 34.3.1 主界面

从图 34.3.1 可以看出，主菜单最后面出现了一个名为 “My test menu” 的子菜单，这个就是我们上面添加进来的子菜单。进入此子菜单，如图 34.3.2 所示：

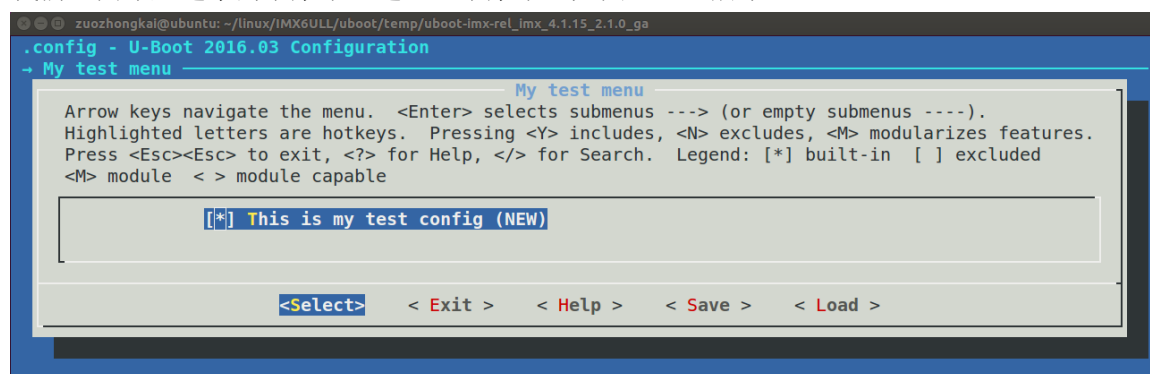


图 34.3.2 “My test menu” 子菜单

从图 34.3.2 可以看出，配置项添加成功，选中 “This is my test config” 配置项，然后按下

“H” 键打开帮助文档，如图 34.3.3 所示：

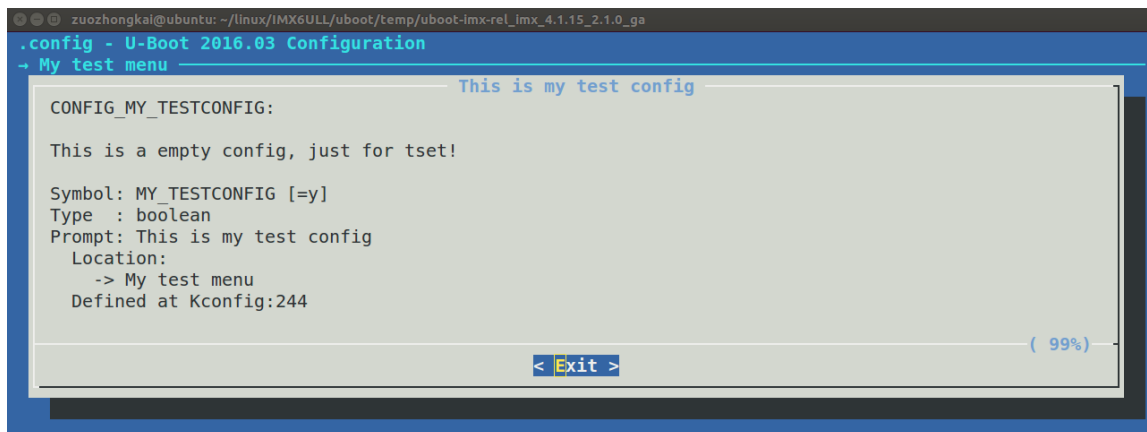


图 34.3.3 帮助信息

从图 34.3.3 可以看出，帮助信息也正确。配置项 MY_TESTCONFIG 默认也是被选中的，因此在.config 文件中肯定会有“CONFIG_MY_TESTCONFIG=y”这一行，如图 34.3.4 所示：



图 34.3.4 .config 文件

至此，我们在主菜单添加自己的自定义菜单就成功了，以后大家如果去半导体原厂工作的话，如果要编写 Linux 驱动，那么很有可能需要你来修改甚至编写 Kconfig 文件。Kconfig 语法其实不难，重要的点就是 34.2.2 小节中的那几个，最主要的是记住：Kconfig 文件的最终目的就是在.config 文件中生成以“CONFIG_”开头的变量。

第三十五章 Linux 内核顶层 Makefile 详解

前几章我们重点讲解了如何移植 uboot 到 I.MX6U-ALPHA 开发板上, 从本章开始我们就开始学习如何移植 Linux 内核。同 uboot 一样, 在具体移植之前, 我们先来学习一下 Linux 内核的顶层 Makefile 文件, 因为顶层 Makefile 控制着 Linux 内核的编译流程。

35.1 Linux 内核获取

关于 Linux 的起源以及发展历史,这里就不啰嗦了,网上相关的介绍太多了!即使写到这里也只是水一下教程页数而已,没有任何实际的意义。有限的时间还是放到有意义的事情上吧, Linux 由 Linux 基金会管理与发布, Linux 官网为 <https://www.kernel.org>, 所以你想获取最新的 Linux 版本就可以在这个网站上下载,网站界面如图 35.1.1 所示:

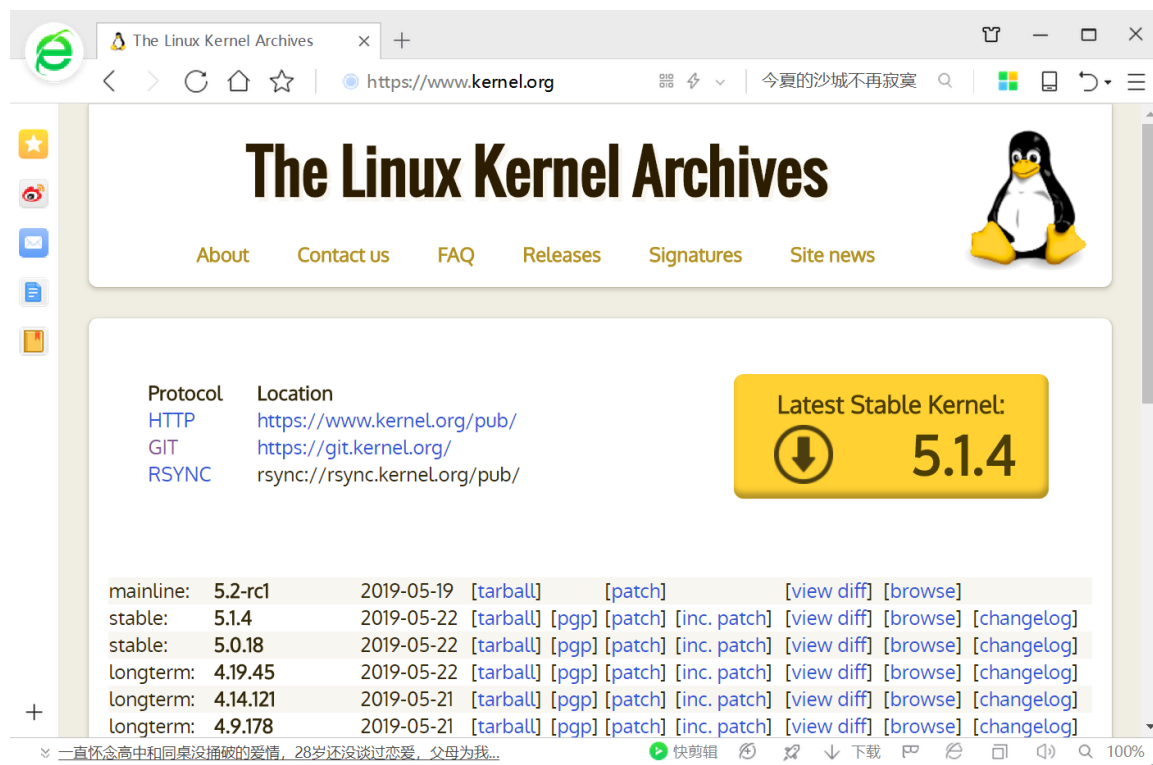


图 35.1.1 linux 官网

从图 35.1.1 可以看出最新的稳定版 Linux 已经到了 5.1.4, 大家没必要追新, 因为 4.x 版本的 Linux 和 5.x 版本没有本质上的区别, 5.x 更多的是加入了一些新的平台、新的外设驱动而已。

NXP 会从 <https://www.kernel.org> 下载某个版本的 Linux 内核, 然后将其移植到自己的 CPU 上, 测试成功以后就会将其开放给 NXP 的 CPU 开发者。开发者下载 NXP 提供的 Linux 内核, 然后将其移植到自己的产品上。本章的移植我们就使用 NXP 提供的 Linux 源码, NXP 提供 Linux 源码已经放到了开发板光盘中, 路径为: 开发板光盘->1、例程源码->4、NXP 官方原版 Uboot 和 Linux-> linux-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2

35.2 Linux 内核初次编译

编译内核之前需要先在 ubuntu 上安装 lzop 库, 否则内核编译会失败! 命令如下:

```
sudo apt-get install lzop
```

先看一下如何编译 Linux 源码, 这里编译 I.MX6U-ALPHA 开发板移植好的 Linux 源码, 已经放到了开发板光盘中, 路径为: 开发板光盘->1、例程源码->3、正点原子 Uboot 和 Linux 出厂源码-> linux-imx-4.1.15-2.1.0-g8a006db.tar.bz2, 注意, 正点原子出厂系统在不断的更新, 因此压缩包的名字可能不同, 一切以实际为准!

在 Ubuntu 中新建名为 “alientek_linux” 的文件夹, 然后将 linux-imx-4.1.15-2.1.0-g8a006db.tar.bz2 这个压缩包拷贝到前面新建的 alientek_linux 文件夹中并解压, 命令如下:

```
tar -xjvf linux-imx-4.1.15-2.1.0-g8a006db.tar.bz2
```

解压完成以后的 Linux 源码根目录如图 35.2.1 所示:

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$ ls
arch      crypto    fs        Kbuild    MAINTAINERS  README      security  virt
block     Documentation  include  Kconfig    Makefile     REPORTING-BUGS  sound
COPYING   drivers   init      kernel     mm           samples     tools
CREDITS   firmware  ipc       lib        net          scripts     usr
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$
```

图 35.2.1 正点原子提供的 Linux 源码根目录

以 EMMC 核心板为例, 讲解一下如何编译出对应的 Linux 镜像文件。新建名为“mx6ull_alientek_emmc.sh”的 shell 脚本, 然后在这个 shell 脚本里面输入如下所示内容:

示例代码 35.2.1 mx6ull_alientek_emmc.sh 文件内容

```
1 #!/bin/sh
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- imx_v7_defconfig
4 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
5 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- all -j16
```

使用 chmod 给予 mx6ull_alientek_emmc.sh 可执行权限, 然后运行此 shell 脚本, 命令如下:

```
./mx6ull_alientek_emmc.sh
```

编译的时候会弹出 Linux 图形配置界面, 如图 35.2.2 所示:

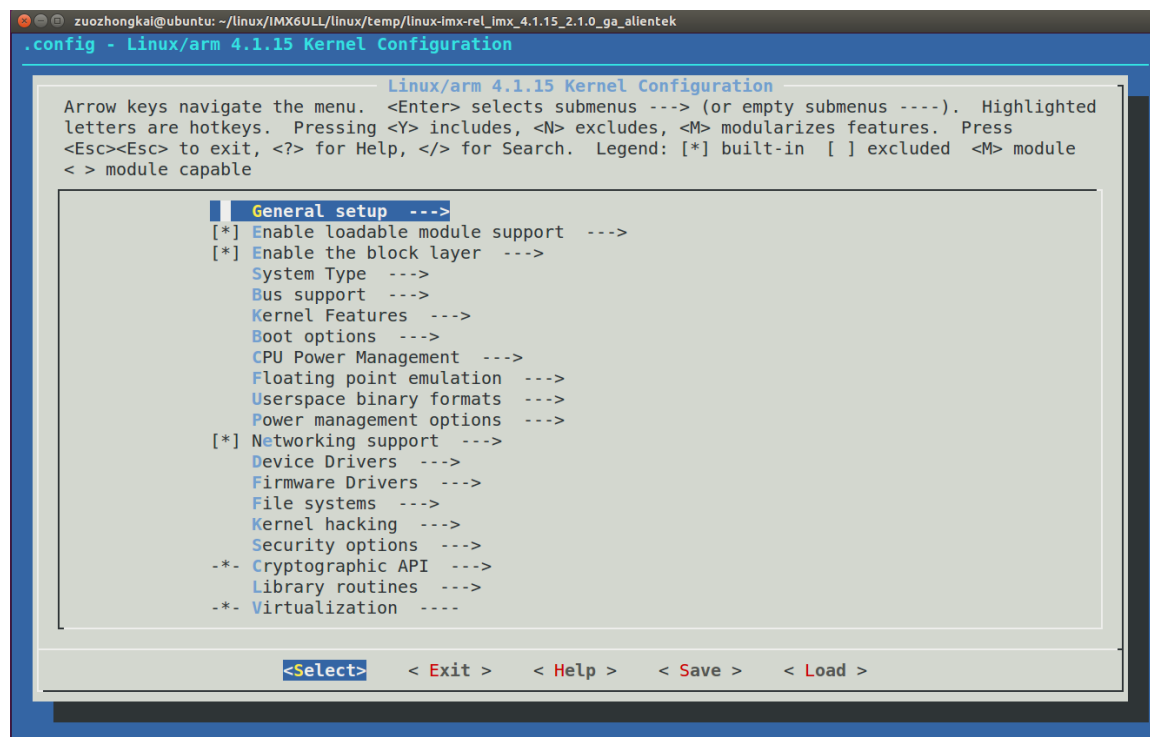


图 35.2.2 Linux 图形配置界面

Linux 的图行界面配置和 uboot 是一样的, 这里我们不需要做任何的配置, 直接按两下 ESC 键退出, 退出图形界面以后会自动开始编译 Linux。等待编译完成, 完成以后如图 35.2.3 所示:

```

zuozhongkai@ubuntu: ~/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
LD [M] fs/nls/nls_iso8859-15.ko
LD [M] lib/crc-ccitt.ko
LD [M] fs/udf/udf.ko
LD [M] fs/fat/msdos.ko
LD [M] lib/crc-itu-t.ko
LD [M] lib/crc7.ko
LD [M] lib/libcrc32c.ko
LD [M] sound/core/snd-hwdep.ko
LD [M] sound/core/snd-rawmidi.ko
LD [M] sound/usb/snd-usbmidi-lib.ko
LD [M] sound/usb/snd-usb-audio.ko
AS arch/arm/boot/compressed/piggy.lzo.o
LD arch/arm/boot/compressed/vmlinux
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
zuozhongkai@ubuntu: ~/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek$

```

图 35.2.3 Linux 编译完成

编译完成以后就会在 arch/arm/boot 这个目录下生成一个叫做 zImage 的文件，zImage 就是我们要用的 Linux 镜像文件。另外也会在 arch/arm/boot/dts 下生成很多.dtb 文件，这些.dtb 就是设备树文件。

编译 Linux 内核的时候可能会提示 “recipe for target ‘arch/arm/boot/compressed/piggy.lzo’ failed”，如图 35.2.4 所示：

```

/bin/sh: 1: lzop: not found
arch/arm/boot/compressed/Makefile:180: recipe for target 'arch/arm/boot/compressed/piggy.lzo' failed
make[2]: *** [arch/arm/boot/compressed/piggy.lzo] Error 1
make[2]: *** 正在等待未完成的任务....
CC arch/arm/boot/compressed/misc.o
arch/arm/boot/Makefile:52: recipe for target 'arch/arm/boot/compressed/vmlinux' failed
make[1]: *** [arch/arm/boot/compressed/vmlinux] Error 2
arch/arm/Makefile:316: recipe for target 'zImage' failed

```

图 35.2.4 lzop 未找到

图 35.2.4 中的错误提示 lzop 未找到，原因是没有安装 lzop 库，输入如下命令安装 lzop 库即可解决：

```
sudo apt-get install lzop
```

lzop 库安装完成以后在重新编译一下 Linux 内核即可。

看一下编译脚本 mx6ull_alientek_emmc.sh 的内容，文件内容如下：

示例代码 35.2.2 mx6ull_alientek_emmc.sh 文件内容

```

1 #!/bin/sh
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- imx_v7_defconfig
4 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
5 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- all -j16

```

第 2 行，执行 “make distclean”，清理工程，所以 mx6ull_alientek_emmc.sh 每次都会清理一下工程。如果通过图形界面配置了 Linux，但是还没保存新的配置文件，那么就要慎重使用 mx6ull_alientek_emmc.sh 编译脚本了，因为它会把你的配置信息都删除掉！

第 3 行，执行 “make xxx_defconfig”，配置工程。

第 4 行，执行 “make menuconfig”，打开图形配置界面，对 Linux 进行配置，如果不想每次编译都打开图形配置界面的话可以将这一行删除掉。

第 5 行，执行 “make”，编译 Linux 源码。

可以看出，Linux 的编译过程基本和 uboot 一样，都要先执行 “make xxx_defconfig” 来配置一下，然后在执行 “make” 进行编译。如果需要使用图形界面配置的话就执行 “make menuconfig”。

35.3 Linux 工程目录分析

将正点原子提供的 Linux 源码进行解压，解压完成以后的目录如图 35.3.1 所示：

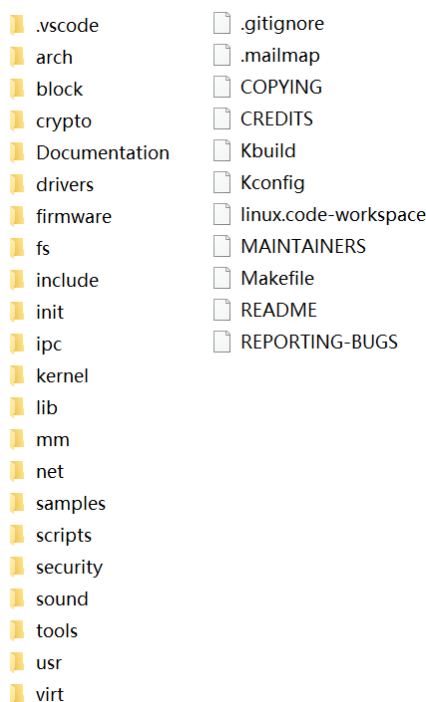


图 35.3.1 未编译的 Linux 源码目录

图 35.3.1 就是正点原子提供的未编译的 Linux 源码目录文件，我们在分析 Linux 之前一定要先在 Ubuntu 中编译一下 Linux，因为编译过程会生成一些文件，而生成的这些恰恰是分析 Linux 不可或缺的文件。编译完成以后使用 tar 压缩命令对其进行压缩并使用 Filezilla 软件将压缩后的 uboot 源码拷贝到 Windows 下。

编译后的 Linux 目录如图 35.3.2 所示：

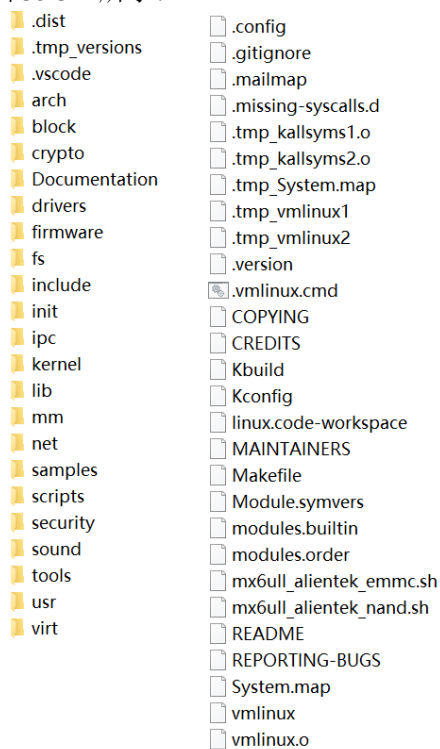


图 35.3.2 编译后的 Linux 目录

图 35.3.2 中重要的文件夹或文件的含义见表 35.3.1 所示:

类型	名字	描述	备注
文件夹	arch	架构相关目录。	Linux 自带
	block	块设备相关目录。	
	crypto	加密相关目录。	
	Documentation	文档相关目录。	
	drivers	驱动相关目录。	
	firmeare	固件相关目录。	
	fs	文件系统相关目录。	
	include	头文件相关目录。	
	init	初始化相关目录。	
	ipc	进程间通信相关目录。	
	kernel	内核相关目录。	
	lib	库相关目录。	
	mm	内存管理相关目录。	
	net	网络相关目录。	
	samples	例程相关目录。	
	scripts	脚本相关目录。	
	security	安全相关目录。	
	sound	音频处理相关目录。	
	tools	工具相关目录。	
	usr	与 initramfs 相关的目录, 用于生成 initramfs。	
	virt	提供虚拟机技术(KVM)。	
文件	.config	Linux 最终使用的配置文件。	编译生成的文件。
	.gitignore	git 工具相关文件。	Linux 自带
	.mailmap	邮件列表。	
	.missing-syscalls.d	。	编译生成的文件
	.tmp_xx	这是一系列的文件, 作用目前笔者还不是很清楚。	编译生成的文件
	.version	好像和版本有关。	
	.vmlinux.cmd	cmd 文件, 用于连接生成 vmlinux。	
	COPYING	版权声明。	Linux 自带
	CREDITS	Linux 贡献者。	
	Kbuild	Makefile 会读取此文件。	
	Kconfig	图形化配置界面的配置文件。。	
	MAINTAINERS	维护者名单。	
	Makefile	Linux 顶层 Makefile	
	Module.xx modules.xx	一系列文件, 和模块有关。	编译生成的文件
	mx6ull_alientek_emmc.sh mx6ull_alientek_nand.sh	正点原子提供的, Linux 编译脚本。	正点原子提供

	README	Linux 描述文件。	Linux 自带
	REPORTING-BUGS	BUG 上报指南	
	System.map	符号表。	编译生成的文件
	vmlinux	编译出来的、未压缩的 ELF 格式 Linux 文件	
	vmlinux.o	编译出来的 vmlinux.o 文件。	

表 35.3.1 Linux 目录

表 35.3.1 中的很多文件夹和文件我们都不需要去关心，我们要关注的文件夹或文件如下：

1、arch 目录

这个目录是和架构有关的目录，比如 arm、arm64、avr32、x86 等等架构。每种架构都对应一个目录，在这些目录中又有很多子目录，比如 boot、common、configs 等等，以 arch/arm 为例，其子目录如图 35.3.3 所示：

boot	2019/5/25 10:44	文件夹
common	2019/5/25 10:44	文件夹
configs	2019/5/25 10:44	文件夹
crypto	2019/5/25 10:44	文件夹
firmware	2019/5/25 10:44	文件夹
include	2019/5/25 10:44	文件夹
kernel	2019/5/25 10:44	文件夹
kvm	2019/5/25 10:44	文件夹
lib	2019/5/25 10:44	文件夹
mach-alpine	2019/5/25 10:44	文件夹
mach-asm9260	2019/5/25 10:44	文件夹
mach-at91	2019/5/25 10:44	文件夹
mach-axxia	2019/5/25 10:44	文件夹

图 35.3.3 arch/arm 子目录

图 35.3.3 是 arch/arm 的一部分子目录，这些子目录用于控制系统引导、系统调用、动态调频、主频设置等。arch/arm/configs 目录是不同平台的默认配置文件：xxx_defconfig，如图 35.3.4 所示：

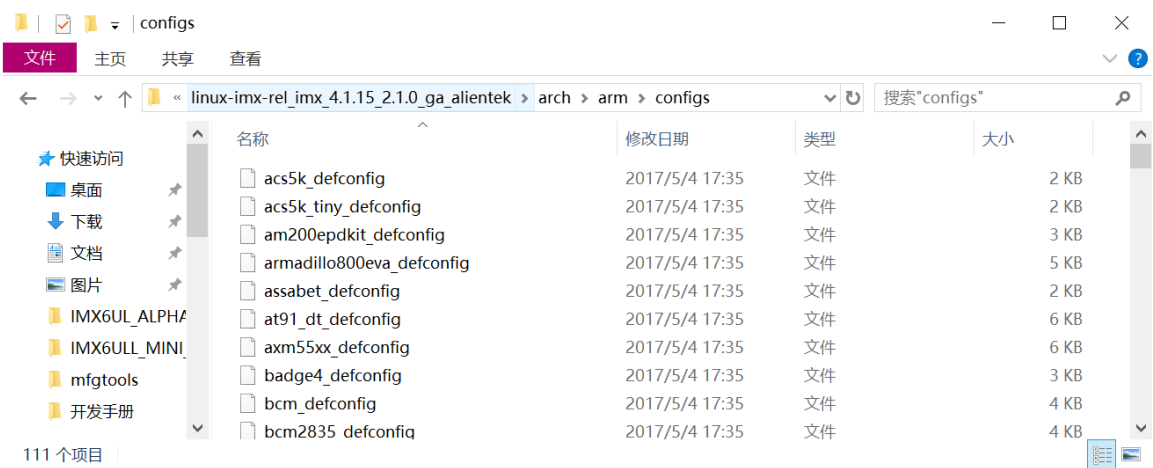


图 35.3.4 配置文件

在 arch/arm/configs 中就包含有 I.MX6U-ALPHA 开发板的默认配置文件：imx_v7_defconfig，执行“make imx_v7_defconfig”即可完成配置。arch/arm/boot/dts 目录里面是对应开发平台的设

备树文件，正点原子 I.MX6U-ALPHA 开发板对应的设备树文件如图 35.3.5 所示：

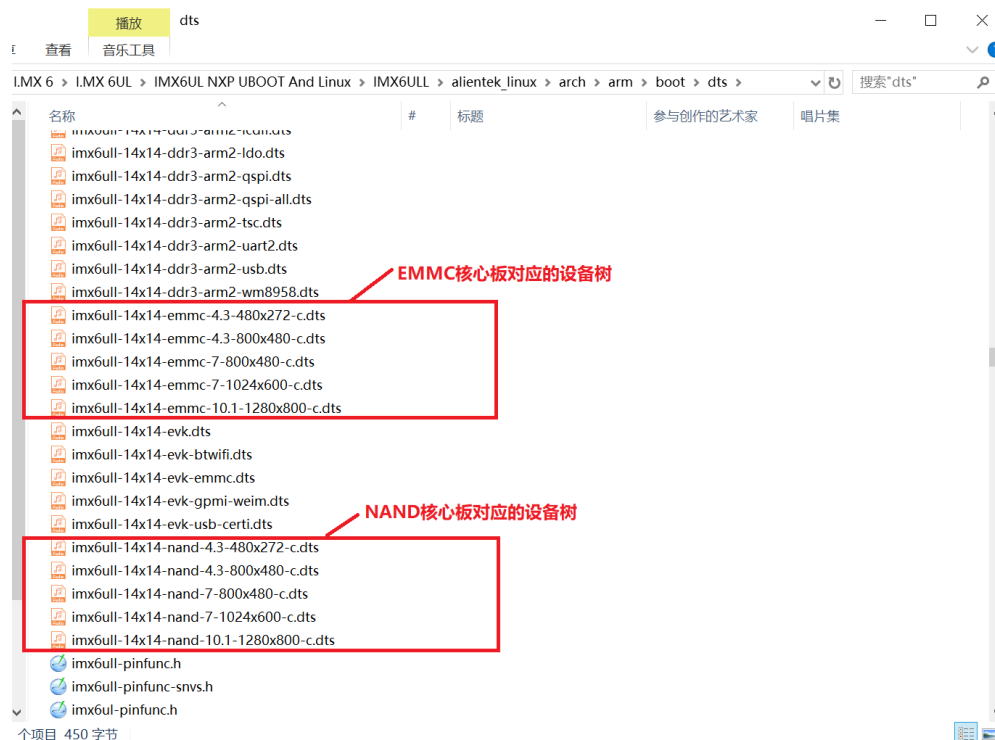


图 35.3.5 正点原子 I.MX6U 开发板对应的设备树

arch/arm/boot 目录下会保存编译出来的 Image 和 zImage 镜像文件，而 zImage 就是我们常用的 linux 镜像文件。

arch/arm/mach-xxx 目录分别为相应平台的驱动和初始化文件，比如 mach-imx 目录里面就是 I.MX 系列 CPU 的驱动和初始化文件。

2、block 目录

block 是 Linux 下块设备目录，像 SD 卡、EMMC、NAND、硬盘等存储设备就属于块设备，block 目录中存放着管理块设备的相关文件。

3、crypto 目录

crypto 目录里面存放着加密文件，比如常见的 crc、crc32、md4、md5、hash 等加密算法。

4、Documentation 目录

此目录里面存放着 Linux 相关的文档，如果要想了解 Linux 某个功能模块或驱动架构的功能，就可以在 Documentation 目录中查找有没有对应的文档。

5、drivers 目录

驱动目录文件，此目录根据驱动类型的不同，分门别类进行整理，比如 drivers/i2c 就是 I2C 相关驱动目录，drivers/gpio 就是 GPIO 相关的驱动目录，这是我们学习的重点。

6、firmware 目录

此目录用于存放固件。

7、fs 目录

此目录存放文件系统，比如 fs/ext2、fs/ext4、fs/f2fs 等，分别是 ext2、ext4 和 f2fs 等文件系统。

8、include 目录

头文件目录。

9、init 目录

此目录存放 Linux 内核启动的时候初始化代码。

10、ipc 目录

IPC 为进程间通信，ipc 目录是进程间通信的具体实现代码。

11、kernel 目录

Linux 内核代码。

12、lib 目录

lib 是库的意思，lib 目录都是一些公用的库函。

13、mm 目录

此目录存放内存管理相关代码。

14、net 目录

此目录存放网络相关代码。

15、samples 目录

此目录存放一些示例代码文件。

16、scripts 目录

脚本目录，Linux 编译的时候会用到很多脚本文件，这些脚本文件就保存在此目录中。

17、security 目录

此目录存放安全相关的文件。

18、sound 目录

此目录存放音频相关驱动文件，音频驱动文件并没有存放到 drivers 目录中，而是单独的目录。

19、tools 目录

此目录存放一些编译的时候使用到的工具。

20、usr 目录

此目录存放与 initramfs 有关的代码。

21、virt 目录

此目录存放虚拟机相关文件。

22、.config 文件

跟 uboot 一样，.config 保存着 Linux 最终的配置信息，编译 Linux 的时候会读取此文件中的配置信息。最终根据配置信息来选择编译 Linux 哪些模块，哪些功能。

23、Kbuild 文件

有些 Makefile 会读取此文件。

24、Kconfig 文件

图形化配置界面的配置文件。

25、Makefile 文件

Linux 顶层 Makefile 文件，建议好好阅读一下此文件。

26、README 文件

此文件详细讲解了如何编译 Linux 源码，以及 Linux 源码的目录信息，建议仔细阅读一下此文件。

关于 Linux 源码目录就分析到这里，接下来分析一下 Linux 的顶层 Makefile。

35.4 VSCode 工程创建

在分析 Linux 的顶层 Makefile 之前，先创建 VSCode 工程，创建过程和 uboot 一样。创建好以后将文件.vscode/settings.json 改为如下所示内容：

示例代码 35.4.1 settings.json 文件内容

```
1 {
2     "search.exclude": {
3         "**/node_modules": true,
4         "**/bower_components": true,
5         "**/*.o": true,
6         "**/*.su": true,
7         "**/*.cmd": true,
8         "Documentation": true,
9
10        /* 屏蔽不用的架构相关的文件 */
11        "arch/alpha": true,
12        "arch/arc": true,
13        "arch/arm64": true,
14        "arch/avr32": true,
15        "arch/[b-z]*": true,
16        "arch/arm/plat*": true,
17        "arch/arm/mach-[a-h]*": true,
18        "arch/arm/mach-[n-z]*": true,
19        "arch/arm/mach-i[n-z]*": true,
20        "arch/arm/mach-m[e-v]*": true,
21        "arch/arm/mach-k*": true,
22        "arch/arm/mach-l*": true,
23
24        /* 屏蔽排除不用的配置文件 */
25        "arch/arm/configs/[a-h]*": true,
26        "arch/arm/configs/[j-z]*": true,
27        "arch/arm/configs/imo*": true,
28        "arch/arm/configs/in*": true,
29        "arch/arm/configs/io*": true,
30        "arch/arm/configs/ix*": true,
```

```
31
32     /* 屏蔽掉不用的 DTB 文件 */
33     "arch/arm/boot/dts/[a-h]*":true,
34     "arch/arm/boot/dts/[k-z]*":true,
35     "arch/arm/boot/dts/in*":true,
36     "arch/arm/boot/dts/imx1*":true,
37     "arch/arm/boot/dts/imx7*":true,
38     "arch/arm/boot/dts/imx2*":true,
39     "arch/arm/boot/dts/imx3*":true,
40     "arch/arm/boot/dts/imx5*":true,
41     "arch/arm/boot/dts/imx6d*":true,
42     "arch/arm/boot/dts/imx6q*":true,
43     "arch/arm/boot/dts/imx6s*":true,
44     "arch/arm/boot/dts/imx6ul-*":true,
45     "arch/arm/boot/dts/imx6ull-9x9*":true,
46     "arch/arm/boot/dts/imx6ull-14x14-ddr*":true,
47 },
48 "files.exclude": {
49     "**/.git": true,
50     "**/.svn": true,
51     "**/.hg": true,
52     "**/CVS": true,
53     "**/.DS_Store": true,
54     "**/*.o":true,
55     "**/*.su":true,
56     "**/*.cmd":true,
57     "Documentation":true,
58
59     /* 屏蔽不用的架构相关的文件 */
60     "arch/alpha":true,
61     "arch/arc":true,
62     "arch/arm64":true,
63     "arch/avr32":true,
64     "arch/[b-z]*":true,
65     "arch/arm/plat*":true,
66     "arch/arm/mach-[a-h]*":true,
67     "arch/arm/mach-[n-z]*":true,
68     "arch/arm/mach-i[n-z]*":true,
69     "arch/arm/mach-m[e-v]*":true,
70     "arch/arm/mach-k*":true,
71     "arch/arm/mach-l*":true,
72
73     /* 屏蔽排除不用的配置文件 */
```

```

74     "arch/arm/configs/[a-h]*":true,
75     "arch/arm/configs/[j-z]*":true,
76     "arch/arm/configs/imo*":true,
77     "arch/arm/configs/in*":true,
78     "arch/arm/configs/io*":true,
79     "arch/arm/configs/ix*":true,
80
81     /* 屏蔽掉不用的 DTB 文件 */
82     "arch/arm/boot/dts/[a-h]*":true,
83     "arch/arm/boot/dts/[k-z]*":true,
84     "arch/arm/boot/dts/in*":true,
85     "arch/arm/boot/dts/imx1*":true,
86     "arch/arm/boot/dts/imx7*":true,
87     "arch/arm/boot/dts/imx2*":true,
88     "arch/arm/boot/dts/imx3*":true,
89     "arch/arm/boot/dts/imx5*":true,
90     "arch/arm/boot/dts/imx6d*":true,
91     "arch/arm/boot/dts/imx6q*":true,
92     "arch/arm/boot/dts/imx6s*":true,
93     "arch/arm/boot/dts/imx6ul-*":true,
94     "arch/arm/boot/dts/imx6ull-9x9*":true,
95     "arch/arm/boot/dts/imx6ull-14x14-ddr*":true,
96 }
97 }

```

创建好 VSCode 工程以后就可以开始分析 Linux 的顶层 Makefile 了。

35.5 顶层 Makefile 详解

Linux 的顶层 Makefile 和 uboot 的顶层 Makefile 非常相似, 因为 uboot 参考了 Linux, 前 602 行几乎一样, 所以前面部分我们大致看一下就行了。

1、版本号

顶层 Makefile 一开始就是 Linux 内核的版本号, 如下所示:

示例代码 35.5.1 顶层 Makefile 代码段

```

1 VERSION = 4
2 PATCHLEVEL = 1
3 SUBLEVEL = 15
4 EXTRAVERSION =

```

可以看出, Linux 内核版本号为 4.1.15。

2、MAKEFLAGS 变量

MAKEFLAGS 变量设置如下所示:

示例代码 35.5.2 顶层 Makefile 代码段

```

16 MAKEFLAGS += -rR --include-dir=$(CURDIR)

```

3、命令输出

Linux 编译的时候也可以通过“V=1”来输出完整的命令，这个和 uboot 一样，相关代码如下所示：

示例代码 35.5.3 顶层 Makefile 代码段

```

69 ifeq ("$(origin V)", "command line")
70     KBUILD_VERBOSE = $(V)
71 endif
72 ifndef KBUILD_VERBOSE
73     KBUILD_VERBOSE = 0
74 endif
75
76 ifeq ($(KBUILD_VERBOSE), 1)
77     quiet =
78     Q =
79 else
80     quiet=quiet_
81     Q = @
82 endif

```

4、静默输出

Linux 编译的时候使用“make -s”就可实现静默编译，编译的时候就不会打印任何的信息，同 uboot 一样，相关代码如下：

示例代码 35.5.4 顶层 Makefile 代码段

```

87 ifneq ($(filter 4.%, $(MAKE_VERSION)),) # make-4
88 ifneq ($(filter %s, $(firstword x$(MAKEFLAGS))),)
89     quiet=silent_
90 endif
91 else # make-3.8x
92 ifneq ($(filter s% -s%, $(MAKEFLAGS)),)
93     quiet=silent_
94 endif
95 endif
96
97 export quiet Q KBUILD_VERBOSE

```

5、设置编译结果输出目录

Linux 编译的时候使用“O=xxx”即可将编译产生的过程文件输出到指定的目录中，相关代码如下：

示例代码 35.5.5 顶层 Makefile 代码段

```

116 ifeq ($(KBUILD_SRC),)
117
118 # OK, Make called in directory where kernel src resides
119 # Do we want to locate output files in a separate directory?
120 ifeq ("$(origin O)", "command line")
121     KBUILD_OUTPUT := $(O)

```

122 endif

6、代码检查

Linux 也支持代码检查, 使用命令“make C=1”使能代码检查, 检查那些需要重新编译的文件。“make C=2”用于检查所有的源码文件, 顶层 Makefile 中的代码如下:

示例代码 35.5.6 顶层 Makefile 代码段

```
172 ifeq ("$(origin C)", "command line")
173     KBUILD_CHECKSRC = $(C)
174 endif
175 ifndef KBUILD_CHECKSRC
176     KBUILD_CHECKSRC = 0
177 endif
```

7、模块编译

Linux 允许单独编译某个模块, 使用命令“make M=dir”即可, 旧语法“make SUBDIRS=dir”也是支持的。顶层 Makefile 中的代码如下:

示例代码 35.5.7 顶层 Makefile 代码段

```
179 # Use make M=dir to specify directory of external module to build
180 # Old syntax make ... SUBDIRS=$PWD is still supported
181 # Setting the environment variable KBUILD_EXTMOD take precedence
182 ifdef SUBDIRS
183     KBUILD_EXTMOD ?= $(SUBDIRS)
184 endif
185
186 ifeq ("$(origin M)", "command line")
187     KBUILD_EXTMOD := $(M)
188 endif
189
190 # If building an external module we do not care about the all: rule
191 # but instead _all depend on modules
192 PHONY += all
193 ifeq ($(KBUILD_EXTMOD),)
194     _all: all
195 else
196     _all: modules
197 endif
198
199 ifeq ($(KBUILD_SRC),)
200     # building in the source tree
201     srctree := .
202 else
203     ifeq ($(KBUILD_SRC)/,$(dir $(CURDIR)))
204         # building in a subdirectory of the source tree
205         srctree := ..
```



```

206         else
207             srctree := $(KBUILD_SRC)
208         endif
209     endif
210     objtree := .
211     src := $(srctree)
212     obj := $(objtree)
213
214     VPATH := $(srctree)$(if $(KBUILD_EXTMOD),:$(KBUILD_EXTMOD))
215
216     export srctree objtree VPATH

```

外部模块编译过程和 uboot 也一样，最终导出 srctree、objtree 和 VPATH 这三个变量的值，其中 srctree=., 也就是当前目录，objtree 同样为“.”。

8、设置目标架构和交叉编译器

同 uboot 一样，Linux 编译的时候需要设置目标板架构 ARCH 和交叉编译器 CROSS_COMPILE，在顶层 Makefile 中代码如下：

示例代码 35.5.8 顶层 Makefile 代码段

```

252 ARCH           ?= $(SUBARCH)
253 CROSS_COMPILE ?= $(CONFIG_CROSS_COMPILE:"%"=%)

```

为了方便，一般直接修改顶层 Makefile 中的 ARCH 和 CROSS_COMPILE，直接将其设置为对应的架构和编译器，比如本教程将 ARCH 设置为 arm，CROSS_COMPILE 设置为 arm-linux-gnueabi-，如下所示：

示例代码 35.5.9 顶层 Makefile 代码段

```

252 ARCH           ?= arm
253 CROSS_COMPILE ?= arm-linux-gnueabi-

```

设置好以后我们就可以使用如下命令编译 Linux 了：

```

make xxx_defconfig    //使用默认配置文件配置 Linux
make menuconfig       //启动图形化配置界面
make -j16              //编译 Linux

```

9、调用 scripts/Kbuild.include 文件

同 uboot 一样，Linux 顶层 Makefile 也会调用文件 scripts/Kbuild.include，顶层 Makefile 相应代码如下：

示例代码 35.5.10 顶层 Makefile 代码段

```

348 # We need some generic definitions (do not try to remake the file).
349 scripts/Kbuild.include: ;
350 include scripts/Kbuild.include

```

10、交叉编译工具变量设置

顶层 Makefile 中其他和交叉编译器有关的变量设置如下：

示例代码 35.5.11 顶层 Makefile 代码段

```

353 AS      = $(CROSS_COMPILE)as
354 LD      = $(CROSS_COMPILE)ld
355 CC      = $(CROSS_COMPILE)gcc

```

```

356 CPP      = $(CC) -E
357 AR        = $(CROSS_COMPILE) ar
358 NM        = $(CROSS_COMPILE) nm
359 STRIP      = $(CROSS_COMPILE) strip
360 OBJCOPY    = $(CROSS_COMPILE) objcopy
361 OBJDUMP    = $(CROSS_COMPILE) objdump

```

LA、LD、CC 等这些都是交叉编译器所使用的工具。

11、头文件路径变量

顶层 Makefile 定义了两个变量保存头文件路径: USERINCLUDE 和 LINUXINCLUDE, 相关代码如下:

示例代码 35.5.12 顶层 Makefile 代码段

```

381 USERINCLUDE := \
382     -I$(srctree)/arch/$(hdr-arch)/include/uapi \
383     -Iarch/$(hdr-arch)/include/generated/uapi \
384     -I$(srctree)/include/uapi \
385     -Iinclude/generated/uapi \
386     -include $(srctree)/include/linux/kconfig.h
387
388 # Use LINUXINCLUDE when you must reference the include/ directory.
389 # Needed to be compatible with the O= option
390 LINUXINCLUDE := \
391     -I$(srctree)/arch/$(hdr-arch)/include \
392     -Iarch/$(hdr-arch)/include/generated/uapi \
393     -Iarch/$(hdr-arch)/include/generated \
394     $(if $(KBUILD_SRC), -I$(srctree)/include) \
395     -Iinclude \
396     $(USERINCLUDE)

```

第 381~386 行是 USERINCLUDE 是 UAPI 相关的头文件路径, 第 390~396 行是 LINUXINCLUDE 是 Linux 内核源码的头文件路径。重点来看一下 LINUXINCLUDE, 其中 srctree=., hdr-arch=arm, KBUILD_SRC 为空, 因此, 将 USERINCLUDE 和 LINUXINCLUDE 展开以后为:

```

USERINCLUDE := \
    -I./arch/arm/include/uapi \
    -Iarch/arm/include/generated/uapi \
    -I./include/uapi \
    -Iinclude/generated/uapi \
    -include ./include/linux/kconfig.h

LINUXINCLUDE := \
    -I./arch/arm/include \
    -Iarch/arm/include/generated/uapi \
    -Iarch/arm/include/generated \
    -Iinclude \

```

```
-I./arch/arm/include/uapi \
-Iarch/arm/include/generated/uapi \
-I./include/uapi \
-Iinclude/generated/uapi \
-include ./include/linux/kconfig.h
```

12、导出变量

顶层 Makefile 会导出很多变量给子 Makefile 使用，导出的这些变量如下：

示例代码 35.5.13 顶层 Makefile 代码段

```
417 export VERSION PATCHLEVEL SUBLEVEL KERNELRELEASE KERNELVERSION
418 export ARCH SRCARCH CONFIG_SHELL HOSTCC HOSTCFLAGS CROSS_COMPILE AS
LD CC
419 export CPP AR NM STRIP OBJCOPY OBJDUMP
420 export MAKE AWK GENKSYMS INSTALLKERNEL PERL PYTHON UTS_MACHINE
421 export HOSTCXX HOSTCXXFLAGS LDFLAGS_MODULE CHECK CHECKFLAGS
422
423 export KBUILD_CPPFLAGS NOSTDINC_FLAGS LINUXINCLUDE OBJCOPYFLAGS
LDFLAGS
424 export KBUILD_CFLAGS CFLAGS_KERNEL CFLAGS_MODULE CFLAGS_GCOV
CFLAGS_KASAN
425 export KBUILD_AFLAGS AFLAGS_KERNEL AFLAGS_MODULE
426 export KBUILD_AFLAGS_MODULE KBUILD_CFLAGS_MODULE
KBUILD_LDFLAGS_MODULE
427 export KBUILD_AFLAGS_KERNEL KBUILD_CFLAGS_KERNEL
428 export KBUILD_ARFLAGS
```

35.5.1 make xxx_defconfig 过程

第一次编译 Linux 之前都要使用“make xxx_defconfig”先配置 Linux 内核，在顶层 Makefile 中有“%config”这个目标，如下所示：

示例代码 35.5.1.1 顶层 Makefile 代码段

```
490 config-targets := 0
491 mixed-targets := 0
492 dot-config := 1
493
494 ifneq ($(filter $(no-dot-config-targets), $(MAKECMDGOALS)),)
495     ifeq ($(filter-out $(no-dot-config-targets), $(MAKECMDGOALS)),)
496         dot-config := 0
497     endif
498 endif
499
500 ifeq ($(KBUILD_EXTMOD),)
501     ifneq ($(filter config %config, $(MAKECMDGOALS)),)
502         config-targets := 1
```

```

503         ifneq ($(words $(MAKECMDGOALS)),1)
504             mixed-targets := 1
505         endif
506     endif
507 endif
508
509 ifeq ($(mixed-targets),1)
510 # =====
511 # We're called with mixed targets (*config and build targets).
512 # Handle them one by one.
513
514 PHONY += $(MAKECMDGOALS) __build_one_by_one
515
516 $(filter-out __build_one_by_one, $(MAKECMDGOALS)):
517     @:
518
519 __build_one_by_one:
520     $(Q)set -e; \
521     for i in $(MAKECMDGOALS); do \
522         $(MAKE) -f $(srctree)/Makefile $$i; \
523     done
524
525 else
526 ifeq ($(config-targets),1)
527 # =====
528 # *config targets only - make sure prerequisites are updated, and
529 # descend in scripts/kconfig to make the *config target
530
531 # Read arch specific Makefile to set KBUILD_DEFCONFIG as needed.
532 # KBUILD_DEFCONFIG may point out an alternative default
533 # configuration used for 'make defconfig'
534 include arch/$(SRCARCH)/Makefile
535 export KBUILD_DEFCONFIG KBUILD_KCONFIG
536
537 config: scripts_basic outputmakefile FORCE
538     $(Q)$(MAKE) $(build)=scripts/kconfig $@
539
540 %config: scripts_basic outputmakefile FORCE
541     $(Q)$(MAKE) $(build)=scripts/kconfig $@
542
543 else
544     .....

```

```
563 endif # KBUILD_EXTMOD
```

第 490~507 行和 uboot 一样，都是设置定义变量 config-targets、mixed-targets 和 dot-config 的值，最终这三个变量的值为：

```
config-targets= 1
mixed-targets= 0
dot-config= 1
```

因为 config-targets=1，因此第 534 行~541 行成立。第 534 行引用 arch/arm/Makefile 这个文件，这个文件很重要，因为 zImage、uImage 等这些文件就是由 arch/arm/Makefile 来生成的。

第 535 行导出变量 KBUILD_DEFCONFIG KBUILD_KCONFIG。

第 537 行，没有目标与之匹配，因此不执行。

第 540 行，“make xxx_defconfig”与目标“%config”匹配，因此执行。“%config”依赖 scripts_basic、outputmakefile 和 FORCE，“%config”真正有意义的依赖就只有 scripts_basic，scripts_basic 的规则如下：

示例代码 35.5.1.2 顶层 Makefile 代码段

```
448 scripts_basic:
449     $(Q)$(MAKE) $(build)=scripts/basic
450     $(Q)rm -f .tmp_quiet_recordmcount
```

build 定义在文件 scripts/Kbuild.include 中，值为 build := -f \$(srctree)/scripts/Makefile.build obj，因此将示例代码 35.5.1.2 展开就是：

scripts_basic:

```
@make -f ./scripts/Makefile.build obj=scripts/basic //也可以没有@，视配置而定
@rm -f .tmp_quiet_recordmcount //也可以没有@
```

接着回到示例代码 35.5.1.1 的目标“%config”处，内容如下：

```
%config: scripts_basic outputmakefile FORCE
$(Q)$(MAKE) $(build)=scripts/kconfig $@
```

将命令展开就是：

```
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

35.5.2 Makefile.build 脚本分析

从上一小节可知，“make xxx_defconfig”配置 Linux 的时候如下两行命令会执行脚本 scripts/Makefile.build：

```
@make -f ./scripts/Makefile.build obj=scripts/basic
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

我们依次来分析一下：

1、scripts_basic 目标对应的命令

scripts_basic 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/basic。打开文件 scripts/Makefile.build，有如下代码：

示例代码 35.5.2.1 Makefile.build 代码段

```
41 # The filename Kbuild has precedence over Makefile
42 kbuild-dir := $(if $(filter /%, $(src)), $(src), $(srctree)/$(src))
43 kbuild-file := $(if $(wildcard $(kbuild-dir)/Kbuild), $(kbuild-dir)/Kbuild, $(kbuild-dir)/Makefile)
```

```
44 include $(kbuild-file)
```

将 kbuild-dir 展开后为:

```
kbuild-dir=../scripts/basic
```

将 kbuild-file 展开后为:

```
kbuild-file=../scripts/basic/Makefile
```

最后将 59 行展开, 即:

```
include ../scripts/basic/Makefile
```

继续分析 scripts/Makefile.build, 如下代码:

示例代码 35.5.2.2 Makefile.build 代码段

```
94 __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target)
$(extra-y)) \
95     $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
96     $(subdir-ym) $(always)
97 @:
```

__build 是默认目标, 因为命令 “@make -f ../scripts/Makefile.build obj=scripts/basic” 没有指定目标, 所以会使用到默认目标 __build。在顶层 Makefile 中, KBUILD_BUILTIN 为 1, KBUILD_MODULES 为空, 因此展开后目标 __build 为:

```
__build:$(builtin-target) $(lib-target) $(extra-y) $(subdir-ym) $(always)
@:
```

可以看出目标 __build 有 5 个依赖: builtin-target、lib-target、extra-y、subdir-ym 和 always。这 5 个依赖的具体内容如下:

```
builtin-target =
lib-target =
extra-y =
subdir-ym =
always = scripts/basic/fixdep scripts/basic/bin2c
```

只有 always 有效, 因此 __build 最终为:

```
__build: scripts/basic/fixdep scripts/basic/bin2c
@:
```

__build 依赖于 scripts/basic/fixdep 和 scripts/basic/bin2c, 所以要先将 scripts/basic/fixdep 和 scripts/basic/bin2c.c 这两个文件编译成 fixdep 和 bin2c。

综上所述, scripts_basic 目标的作用就是编译出 scripts/basic/fixdep 和 scripts/basic/bin2c 这两个软件。

2、%config 目标对应的命令

%config 目标对应的命令为: @make -f ../scripts/Makefile.build obj=scripts/kconfig xxx_defconfig, 此命令会使用到的各个变量值如下:

```
src= scripts/kconfig
kbuild-dir = ../scripts/kconfig
kbuild-file = ../scripts/kconfig/Makefile
include ../scripts/kconfig/Makefile
```

可以看出, Makefile.build 会读取 scripts/kconfig/Makefile 中的内容, 此文件有如下所示内容:

示例代码 35.5.2.3 scripts/kconfig/Makefile 代码段

```
113 %_defconfig: $(obj)/conf
114     $(Q)$< $(silent) --defconfig=arch/$(SRCARCH)/configs/$@
$(Kconfig)
```

目标%_defconfig 与 xxx_defconfig 匹配, 所以会执行这条规则, 将其展开就是:

```
%_defconfig: scripts/kconfig/conf
    @ scripts/kconfig/conf --defconfig=arch/arm/configs/%_defconfig Kconfig
```

%_defconfig 依赖 scripts/kconfig/conf, 所以会编译 scripts/kconfig/conf.c 生成 conf 这个软件。此软件就会将%_defconfig 中的配置输出到.config 文件中, 最终生成 Linux kernel 根目录下的.config 文件。

35.5.3 make 过程

使用命令“make xxx_defconfig”配置好 Linux 内核以后就可以使用“make”或者“make all”命令进行编译。顶层 Makefile 有如下代码:

示例代码 35.5.3.1 顶层 Makefile 代码段

```
125 PHONY := _all
126 _all:
.....
192 PHONY += all
193 ifeq ($(KBUILD_EXTMOD),)
194 _all: all
195 else
196 _all: modules
197 endif
.....
608 all: vmlinux
```

第 126 行, _all 是默认目标, 如果使用命令“make”编译 Linux 的话此目标就会被匹配。

第 193 行, 如果 KBUILD_EXTMOD 为空的话 194 行的代码成立。

第 194 行, 默认目标 all 依赖 all。

第 608 行, 目标 all 依赖 vmlinux, 所以接下来的重点就是 vmlinux!

顶层 Makefile 中有如下代码:

示例代码 35.5.3.2 顶层 Makefile 代码段

```
904 # Externally visible symbols (used by link-vmlinux.sh)
905 export KBUILD_VMLINUX_INIT := $(head-y) $(init-y)
906 export KBUILD_VMLINUX_MAIN := $(core-y) $(libs-y) $(drivers-y)
$(net-y)
907 export KBUILD_LDS := arch/$(SRCARCH)/kernel/vmlinux.lds
908 export LDFLAGS_vmlinux
909 # used by scripts/pacmage/Makefile
910 export KBUILD_ALLDIRS := $(sort $(filter-out arch/%,$(vmlinux-
alldirs)) arch Documentation include samples scripts tools virt)
911
912 vmlinux-deps := $(KBUILD_LDS) $(KBUILD_VMLINUX_INIT)
$(KBUILD_VMLINUX_MAIN)
```



```

913
914 # Final link of vmlinux
915     cmd_link-vmlinux = $(CONFIG_SHELL) $< $(LD) $(LDFLAGS)
$(LDFLAGS_vmlinux)
916 quiet_cmd_link-vmlinux = LINK    $@
917
918 # Include targets which we want to
919 # execute if the rest of the kernel build went well.
920 vmlinux: scripts/link-vmlinux.sh $(vmlinux-deps) FORCE
921 ifdef CONFIG_HEADERS_CHECK
922     $(Q)$(MAKE) -f $(srctree)/Makefile headers_check
923 endif
924 ifdef CONFIG_SAMPLES
925     $(Q)$(MAKE) $(build)=samples
926 endif
927 ifdef CONFIG_BUILD_DOCSRC
928     $(Q)$(MAKE) $(build)=Documentation
929 endif
930 ifdef CONFIG_GDB_SCRIPTS
931     $(Q)ln -fsn `cd $(srctree) && /bin/pwd`/scripts/gdb/vmlinux-
gdb.py
932 endif
933     +$(call if_changed,link-vmlinux)

```

从第 920 行可以看出目标 vmlinux 依赖 scripts/link-vmlinux.sh \$(vmlinux-deps) FORCE。第 912 行定义了 vmlinux-deps，值为：

```
vmlinux-deps=$(KBUILD_LDS) $(KBUILD_VMLINUX_INIT) $(KBUILD_VMLINUX_MAIN)
```

第 905 行，KBUILD_VMLINUX_INIT=\$(head-y) \$(init-y)。

第 906 行，KBUILD_VMLINUX_MAIN=\$(core-y) \$(libs-y) \$(drivers-y) \$(net-y)。

第 907 行，KBUILD_LDS= arch/\$(SRCARCH)/kernel/vmlinux.lds，其中 SRCARCH=arm，因此 KBUILD_LDS= arch/arm/kernel/vmlinux.lds。

综上所述，vmlinux 的依赖为：scripts/link-vmlinux.sh、\$(head-y)、\$(init-y)、\$(core-y)、\$(libs-y)、\$(drivers-y)、\$(net-y)、arch/arm/kernel/vmlinux.lds 和 FORCE。

第 933 行的命令用于链接生成 vmlinux。

重点来看一下\$(head-y)、\$(init-y)、\$(core-y)、\$(libs-y)、\$(drivers-y) 和\$(net-y)这六个变量的值。

1、head-y

head-y 定义在文件 arch/arm/Makefile 中，内容如下：

示例代码 35.5.3.3 arch/arm/Makefile 代码段

```
135 head-y      := arch/arm/kernel/head$(MMUEXT).o
```

当不使能 MMU 的话 MMUEXT=-nommu，如果使能 MMU 的话为空，因此 head-y 最终的值为：

```
head-y = arch/arm/kernel/head.o
```

2、init-y、drivers-y 和 net-y

在顶层 Makefile 中有如下代码:

示例代码 35.5.3.4 顶层 Makefile 代码段

```
558 init-y      := init/
559 drivers-y   := drivers/ sound/ firmware/
560 net-y        := net/
.....
896 init-y      := $(patsubst %/, %/built-in.o, $(init-y))
898 drivers-y   := $(patsubst %/, %/built-in.o, $(drivers-y))
899 net-y        := $(patsubst %/, %/built-in.o, $(net-y))
```

从示例代码 35.5.3.4 可知, init-y、libs-y、drivers-y 和 net-y 最终的值为:

```
init-y      = init/built-in.o
drivers-y   = drivers/built-in.o  sound/built-in.o  firmware/built-in.o
net-y       = net/built-in.o
```

3、libs-y

libs-y 基本和 init-y 一样, 在顶层 Makefile 中存在如下代码:

示例代码 35.5.3.5 顶层 Makefile 代码段

```
561 libs-y      := lib/
.....
900 libs-y1     := $(patsubst %/, %/lib.a, $(libs-y))
901 libs-y2     := $(patsubst %/, %/built-in.o, $(libs-y))
902 libs-y      := $(libs-y1) $(libs-y2)
```

根据示例代码 35.5.3.5 可知, libs-y 应该等于 “lib.a built-in.o”, 这个只正确了一部分! 因为在 arch/arm/Makefile 中会向 libs-y 中追加一些值, 代码如下:

示例代码 35.5.3.6 arch/arm/Makefile 代码段

```
286 libs-y      := arch/arm/lib/ $(libs-y)
```

arch/arm/Makefile 将 libs-y 的值改为了: arch/arm/lib \$(libs-y), 展开以后为:

```
libs-y = arch/arm/lib lib/
```

因此根据示例代码 35.5.3.5 的第 900~902 行可知, libs-y 最终应该为:

```
libs-y = arch/arm/lib/lib.a  lib/lib.a  arch/arm/lib/built-in.o  lib/built-in.o
```

4、core-y

core-y 和 init-y 也一样, 在顶层 Makefile 中有如下代码:

示例代码 35.5.3.7 顶层 Makefile 代码段

```
532 core-y      := usr/
.....
887 core-y      += kernel/ mm/ fs/ ipc/ security/ crypto/ block/
```

但是在 arch/arm/Makefile 中会对 core-y 进行追加, 代码如下:

示例代码 35.5.3.8 arch/arm/Makefile 代码段

```
269 core-$(CONFIG_FPE_NWFPE) += arch/arm/nwfpe/
270 core-$(CONFIG_FPE_FASTFPE) += $(FASTFPE_OBJ)
271 core-$(CONFIG_VFP) += arch/arm/vfp/
272 core-$(CONFIG_XEN) += arch/arm/xen/
```

```

273 core-$(CONFIG_KVM_ARM_HOST) += arch/arm/kvm/
274 core-$(CONFIG_VDSO)           += arch/arm/vdso/
275
276 # If we have a machine-specific directory, then include it in the
    build.
277 core-y                        += arch/arm/kernel/ arch/arm/mm/ arch/arm/common/
278 core-y                        += arch/arm/probes/
279 core-y                        += arch/arm/net/
280 core-y                        += arch/arm/crypto/
281 core-y                        += arch/arm/firmware/
282 core-y                        += $(machdirs) $(platdirs)

```

第 269~274 行根据不同的配置向 core-y 追加不同的值，比如使能 VFP 的话就会在.config 中有 CONFIG_VFP=y 这一行，那么 core-y 就会追加 “arch/arm/vfp/”。

第 277~282 行就是对 core-y 直接追加的值。

在顶层 Makefile 中有如下行：

示例代码 35.5.3.9 顶层 Makefile 代码段

```

897 core-y      := $(patsubst %/, %/built-in.o, $(core-y))

```

经过上述代码的转换，最终 core-y 的值为：

```

core-y = usr/built-in.o          arch/arm/vfp/built-in.o \
        arch/arm/vdso/built-in.o arch/arm/kernel/built-in.o \
        arch/arm/mm/built-in.o  arch/arm/common/built-in.o \
        arch/arm/probes/built-in.o arch/arm/net/built-in.o \
        arch/arm/crypto/built-in.o arch/arm/firmware/built-in.o \
        arch/arm/mach-imx/built-in.o kernel/built-in.o \
        mm/built-in.o           fs/built-in.o \
        ipc/built-in.o          security/built-in.o \
        crypto/built-in.o       block/built-in.o

```

关于 head-y、init-y、core-y、libs-y、drivers-y 和 net-y 这 6 个变量就讲解到这里。这些变量都是一些 built-in.o 或.a 等文件，这个和 uboot 一样，都是将相应目录中的源码文件进行编译，然后在各自目录下生成 built-in.o 文件，有些生成了.a 库文件。最终将这些 built-in.o 和.a 文件进行链接即可形成 ELF 格式的可执行文件，也就是 vmlinux！但是链接是需要链接脚本的，vmlinux 的依赖 arch/arm/kernel/vmlinux.lds 就是整个 Linux 的链接脚本。

示例代码 35.5.3.2 第 933 行的命令 “+\$(call if_changed,link-vmlinux)” 表示将 \$(call if_changed,link-vmlinux) 的结果作为最终生成 vmlinux 的命令，前面的 “+” 表示该命令结果不可忽略。\$(call if_changed,link-vmlinux) 是调用函数 if_changed，link-vmlinux 是函数 if_changed 的参数，函数 if_changed 定义在文件 scripts/Kbuild.include 中，如下所示：

示例代码 35.5.3.10 scripts/Kbuild.include 代码段

```

247 if_changed = $(if $(strip $(any-prereq) $(arg-check)),          \
248     @set -e;                                                       \
249     $(echo-cmd) $(cmd_$(1));                                       \
250     printf '%s\n' 'cmd_$(1) := $(make-cmd)' > $(dot-target).cmd)

```

any-prereq 用于检查依赖文件是否有变化，如果依赖文件有变化那么 any-prereq 就不为空，否则就为空。arg-check 用于检查参数是否有变化，如果没有变化那么 arg-check 就为空。

第 248 行, “@set -e” 告诉 bash, 如果任何语句的执行结果不为 true(也就是执行出错)的话就直接退出。

第 249 行, \$(echo-cmd)用于打印命令执行过程, 比如在链接 vmlinux 的时候就会输出“LINK vmlinux”。\$(cmd \$(1))中的\$(1)表示参数, 也就是 link-vmlinux, 因此\$(cmd \$(1))表示执行 cmd_link-vmlinux 的内容。cmd_link-vmlinux 在顶层 Makefile 中有如下所示定义:

示例代码 35.5.3.11 顶层 Makefile 代码段

```
914 # Final link of vmlinux
915     cmd_link-vmlinux = $(CONFIG_SHELL) $< $(LD) $(LDFLAGS)
$(LDFLAGS_vmlinux)
916 quiet_cmd_link-vmlinux = LINK    $@
```

第 915 行就是 cmd_link-vmlinux 的值, 其中 CONFIG_SHELL=/bin/bash, \$<表示目标 vmlinux 的第一个依赖文件, 根据示例代码 35.5.3.2 可知, 这个文件为 scripts/link-vmlinux.sh。LD= arm-linux-gnueabi-hf-ld -EL, LDFLAGS 为空。LDFLAGS_vmlinux 的值由顶层 Makefile 和 arch/arm/Makefile 这两个文件共同决定, 最终 LDFLAGS_vmlinux=-p --no-undefined -X --pic-veneer --build-id。因此 cmd_link-vmlinux 最终的值为:

```
cmd_link-vmlinux = /bin/bash scripts/link-vmlinux.sh arm-linux-gnueabi-hf-ld -EL -p --no-undefined -X --pic-veneer --build-id
```

cmd_link-vmlinux 会调用 scripts/link-vmlinux.sh 这个脚本来链接出 vmlinux! 在 link-vmlinux.sh 中有如下所示代码:

示例代码 35.5.3.12 scripts/link-vmlinux.sh 代码段

```
51 vmlinux_link()
52 {
53     local lds="${objtree}/${KBUILD_LDS}"
54
55     if [ "${SRCARCH}" != "um" ]; then
56         ${LD} ${LDFLAGS} ${LDFLAGS_vmlinux} -o ${2} \
57             -T ${lds} ${KBUILD_VMLINUX_INIT} \
58             --start-group ${KBUILD_VMLINUX_MAIN} --end-group ${1}
59     else
60         ${CC} ${CFLAGS_vmlinux} -o ${2} \
61             -Wl,-T,${lds} ${KBUILD_VMLINUX_INIT} \
62             -Wl,--start-group \
63                 ${KBUILD_VMLINUX_MAIN} \
64             -Wl,--end-group \
65             -lutil ${1}
66         rm -f linux
67     fi
68 }
.....
216 info LD vmlinux
217 vmlinux_link "${kallsymso}" vmlinux
```

vmlinux_link 就是最终链接出 vmlinux 的函数, 第 55 行判断 SRCARCH 是否等于“um”, 如果不相等的话就执行 56~58 行的代码。因为 SRCARCH=arm, 因此条件成立, 执行 56~58 行的

代码。这三行代码就应该很熟悉了！就是普通的链接操作，连接脚本为 `lds= ./arch/arm/kernel/vmlinux.lds`，需要链接的文件由变量 `KBUILD_VMLINUX_INIT` 和 `KBUILD_VMLINUX_MAIN` 来决定，这两个变量在示例代码 35.5.3.2 中已经讲解过了。

第 217 行调用 `vmlinux_link` 函数来链接出 `vmlinux`。

使用命令“`make V=1`”编译 Linux，会有如图 35.5.3.1 所示的编译信息：

```
+ arm-linux-gnueabi-hf-ld -EL -p --no-undefined -X --pic-veneer --build-id -o vmlinux -T ./arch/arm/kernel/vmlinux.lds arch/arm/kernel/head.o init/built-in.o --start-group usr/built-in.o arch/arm/vfp/built-in.o arch/arm/vdso/built-in.o arch/arm/kernel/built-in.o arch/arm/mm/built-in.o arch/arm/common/built-in.o arch/arm/probes/built-in.o arch/arm/net/built-in.o arch/arm/crypto/built-in.o arch/arm/firmware/built-in.o arch/arm/mach-imx/built-in.o kernel/built-in.o mm/built-in.o fs/built-in.o ipc/built-in.o security/built-in.o crypto/built-in.o block/built-in.o arch/arm/lib/lib.a lib/lib.a arch/arm/lib/built-in.o lib/built-in.o drivers/built-in.o sound/built-in.o firmware/built-in.o net/built-in.o --end-group .tmp.kallsyms2.o
```

图 35.5.3.1 link-vmlinux.sh 链接 vmlinux 过程

至此我们基本理清了 `make` 的过程，重点就是将各个子目录下的 `built-in.o`、`.a` 等文件链接在一起，最终生成 `vmlinux` 这个 ELF 格式的可执行文件。链接脚本为 `arch/arm/kernel/vmlinux.lds`，链接过程是由 shell 脚本 `scripts/link-vmlinux.s` 来完成的。接下来的问题就是这些子目录下的 `built-in.o`、`.a` 等文件又是如何编译出来的呢？

35.5.4 built-in.o 文件编译生成过程

根据示例代码 35.5.3.2 第 920 行可知，`vmlinux` 依赖 `vmlinux-deps`，而 `vmlinux-deps=$(KBUILD_LDS) $(KBUILD_VMLINUX_INIT) $(KBUILD_VMLINUX_MAIN)`，`KBUILD_LDS` 是链接脚本，这里不考虑，剩下的 `KBUILD_VMLINUX_INIT` 和 `KBUILD_VMLINUX_MAIN` 就是各个子目录下的 `built-in.o`、`.a` 等文件。最终 `vmlinux-deps` 的值如下：

<code>vmlinux-deps =</code>	<code>arch/arm/kernel/vmlinux.lds</code>	<code>arch/arm/kernel/head.o \</code>
	<code>init/built-in.o</code>	<code>usr/built-in.o \</code>
	<code>arch/arm/vfp/built-in.o</code>	<code>arch/arm/vdso/built-in.o \</code>
	<code>arch/arm/kernel/built-in.o</code>	<code>arch/arm/mm/built-in.o \</code>
	<code>arch/arm/common/built-in.o</code>	<code>arch/arm/probes/built-in.o \</code>
	<code>arch/arm/net/built-in.o</code>	<code>arch/arm/crypto/built-in.o \</code>
	<code>arch/arm/firmware/built-in.o</code>	<code>arch/arm/mach-imx/built-in.o \</code>
	<code>kernel/built-in.o</code>	<code>mm/built-in.o \</code>
	<code>fs/built-in.o</code>	<code>ipc/built-in.o \</code>
	<code>security/built-in.o</code>	<code>crypto/built-in.o \</code>
	<code>block/built-in.o</code>	<code>arch/arm/lib/lib.a \</code>
	<code>lib/lib.a</code>	<code>arch/arm/lib/built-in.o \</code>
	<code>lib/built-in.o</code>	<code>drivers/built-in.o \</code>
	<code>sound/built-in.o</code>	<code>firmware/built-in.o \</code>
	<code>net/built-in.o</code>	

除了 `arch/arm/kernel/vmlinux.lds` 以外，其他都是要编译链接生成的。在顶层 Makefile 中有如下代码：

示例代码 35.5.4.1 顶层 Makefile 代码段

```
937 $(sort $(vmlinux-deps)) : $(vmlinux-dirs) ;
```

`sort` 是排序函数，用于对 `vmlinux-deps` 的字符串列表进行排序，并且去掉重复的单词。可以看出 `vmlinux-deps` 依赖 `vmlinux-dirs`，`vmlinux-dirs` 也定义在顶层 Makefile 中，定义如下：

示例代码 35.5.4.2 顶层 Makefile 代码段

```
889 vmlinux-dirs := $(patsubst %/,%, $(filter %/, $(init-y) $(init-m) \
```

```
890      $(core-y) $(core-m) $(drivers-y) $(drivers-m) \
891      $(net-y) $(net-m) $(libs-y) $(libs-m)) )
```

vmlinux-dirs 看名字就知道和目录有关，此变量保存着生成 vmlinux 所需源码文件的目录，值如下：

vmlinux-dirs = init	usr	arch/arm/vfp \
arch/arm/vdso	arch/arm/kernel	arch/arm/mm \
arch/arm/common	arch/arm/probes	arch/arm/net \
arch/arm/crypto	arch/arm/firmware	arch/arm/mach-imx\
kernel	mm	fs \
ipc	security	crypto \
block	drivers	sound \
firmware	net	arch/arm/lib \
lib		

在顶层 Makefile 中有如下代码：

示例代码 35.5.4.3 顶层 Makefile 代码段

```
946 $(vmlinux-dirs): prepare scripts
947     $(Q)$(MAKE) $(build)=$@
```

目标 vmlinux-dirs 依赖 prepare 和 scripts，这两个依赖不去浪费时间了，重点看一下第 947 行的命令。build 前面已经说了，值为“-f./scripts/Makefile.build obj”，因此将 947 行的命令展开就是：

```
@ make -f ./scripts/Makefile.build obj=$@
```

\$@表示目标文件，也就是 vmlinux-dirs 的值，将 vmlinux-dirs 中的这些目录全部带入到命令中，结果如下：

```
@ make -f ./scripts/Makefile.build obj=init
@ make -f ./scripts/Makefile.build obj=usr
@ make -f ./scripts/Makefile.build obj=arch/arm/vfp
@ make -f ./scripts/Makefile.build obj=arch/arm/vdso
@ make -f ./scripts/Makefile.build obj=arch/arm/kernel
@ make -f ./scripts/Makefile.build obj=arch/arm/mm
@ make -f ./scripts/Makefile.build obj=arch/arm/common
@ make -f ./scripts/Makefile.build obj=arch/arm/probes
@ make -f ./scripts/Makefile.build obj=arch/arm/net
@ make -f ./scripts/Makefile.build obj=arch/arm/crypto
@ make -f ./scripts/Makefile.build obj=arch/arm/firmware
@ make -f ./scripts/Makefile.build obj=arch/arm/mach-imx
@ make -f ./scripts/Makefile.build obj=kernel
@ make -f ./scripts/Makefile.build obj=mm
@ make -f ./scripts/Makefile.build obj=fs
@ make -f ./scripts/Makefile.build obj=ipc
@ make -f ./scripts/Makefile.build obj=security
@ make -f ./scripts/Makefile.build obj=crypto
@ make -f ./scripts/Makefile.build obj=block
@ make -f ./scripts/Makefile.build obj=drivers
```

```
@ make -f ./scripts/Makefile.build obj=sound
@ make -f ./scripts/Makefile.build obj=firmware
@ make -f ./scripts/Makefile.build obj=net
@ make -f ./scripts/Makefile.build obj=arch/arm/lib
@ make -f ./scripts/Makefile.build obj=lib
```

这些命令运行过程其实都是一样的，我们就以“@ make -f ./scripts/Makefile.build obj=init”这个命令为例，讲解一下详细的运行过程。这里又要用到 Makefile.build 这个脚本了，此脚本默认目标为__build，这个在 35.5.2 小节已经讲过了，我们再来看一下，__build 目标对应的规则如下：

示例代码 35.5.4.4 scripts/Makefile.build 代码段

```
94 __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target)
$(extra-y)) \
95 $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
96 $(subdir-ym) $(always)
97 @:
```

当只编译 Linux 内核镜像文件，也就是使用“make zImage”编译的时候，KBUILD_BUILTIN=1,KBUILD_MODULES 为空。“make”命令是会编译所有的东西，包括 Linux 内核镜像文件和一些模块文件。如果只编译 Linux 内核镜像的话，__build 目标简化为：

```
__build: $(builtin-target) $(lib-target) $(extra-y) $(subdir-ym) $(always)
@:
```

重点来看一下 builtin-target 这个依赖，builtin-target 同样定义在文件 scripts/Makefile.build 中，定义如下：

示例代码 35.5.4.5 scripts/Makefile.build 代码段

```
86 ifneq ($(strip $(obj-y) $(obj-m) $(obj-) $(subdir-m) $(lib-
target)),)
87 builtin-target := $(obj)/built-in.o
88 endif
```

第 87 行就是 builtin-target 变量的值，为“\$(obj)/built-in.o”，这就是这些 built-in.o 的来源了。要生成 built-in.o，要求 obj-y、obj-m、obj-、subdir-m 和 lib-target 这些变量不能全部为空。最后一个问题：built-in.o 是怎么生成的？在文件 scripts/Makefile.build 中有如下代码：

示例代码 35.5.4.6 顶层 Makefile 代码段

```
325 #
326 # Rule to compile a set of .o files into one .o file
327 #
328 ifdef builtin-target
329 quiet_cmd_link_o_target = LD      $@
330 # If the list of objects to link is empty, just create an empty
built-in.o
331 cmd_link_o_target = $(if $(strip $(obj-y)),\
332 $(LD) $(ld_flags) -r -o $@ $(filter $(obj-y), $^)\
333 $(cmd_secanalysis),\
334 rm -f $@; $(AR) rcs$(KBUILD_ARFLAGS) $@)
335
```



```
336 $(builtin-target): $(obj-y) FORCE
337     $(call if_changed,link_o_target)
338
339 targets += $(builtin-target)
340 endif # builtin-target
```

第 336 行的目标就是 builtin-target, 依赖为 obj-y, 命令为“\$(call if_changed,link_o_target)”, 也就是调用函数 if_changed, 参数为 link_o_target, 其返回值就是具体的命令。前面讲过了 if_changed, 它会调用 cmd_\$(1)所对应的命令(\$(1)就是函数的第 1 个参数), 在这里就是调用 cmd_link_o_target 所对应的命令, 也就是第 331~334 行的命令。cmd_link_o_target 就是使用 LD 将某个目录下的所有.o 文件链接在一起, 最终形成 built-in.o。

35.5.5 make zImage 过程

1、vmlinux、Image、zImage、uImage 的区别

前面几小节重点是讲 vmlinux 是如何编译出来的, vmlinux 是 ELF 格式的文件, 但是在实际中我们不会使用 vmlinux, 而是使用 zImage 或 uImage 这样的 Linux 内核镜像文件。那么 vmlinux、zImage、uImage 他们之间有什么区别呢?

①、vmlinux 是编译出来的最原始的内核文件, 是未压缩的, 比如正点原子提供的 Linux 源码编译出来的 vmlinux 差不多有 16MB, 如图 35.5.5.1 所示:

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$ ls vmlinux -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 16770053 Sep  3 01:44 vmlinux
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$
```

图 35.5.5.1 vmlinux 信息

②、Image 是 Linux 内核镜像文件, 但是 Image 仅包含可执行的二进制数据。Image 就是使用 objcopy 取消掉 vmlinux 中的一些其他信息, 比如符号表什么的。但是 Image 是没有压缩过的, Image 保存在 arch/arm/boot 目录下, 其大小大概在 12MB 左右如图 35.5.5.2 所示:

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$ ls arch/arm/boot/Image -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 12541952 Sep  3 01:44 arch/arm/boot/Image
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$
```

图 35.5.5.2 Image 镜像信息

相比 vmlinux 的 16MB, Image 缩小到了 12MB。

③、zImage 是经过 gzip 压缩后的 Image, 经过压缩以后其大小大概在 6MB 左右, 如图 35.5.5.3 所示:

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$ ls arch/arm/boot/zImage -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 6696768 Sep  3 01:44 arch/arm/boot/zImage
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/alientek_linux$
```

图 35.5.5.3 zImage 镜像信息

④、uImage 是老版本 uboot 专用的镜像文件, uImage 是在 zImage 前面加了一个长度为 64 字节的“头”, 这个头信息描述了该镜像文件的类型、加载位置、生成时间、大小等信息。但是新的 uboot 已经支持了 zImage 启动!所以已经很少用到 uImage 了, 除非你用的很古老的 uboot。

使用“make”、“make all”、“make zImage”这些命令就可以编译出 zImage 镜像, 在 arch/arm/Makefile 中有如下代码:

示例代码 35.5.5.1 顶层 Makefile 代码段

```
310 BOOT_TARGETS    = zImage Image xipImage bootpImage uImage
.....
315 $(BOOT_TARGETS): vmlinux
```

```
316 $ (Q) $ (MAKE) $ (build)=$ (boot) MACHINE=$ (MACHINE) $ (boot) /$@
```

第 310 行, 变量 BOOT_TARGETS 包含 zImage, Image, xipImage 等镜像文件。

第 315 行, BOOT_TARGETS 依赖 vmlinux, 因此如果使用 “make zImage” 编译的 Linux 内核的话, 首先肯定要先编译出 vmlinux。

第 316 行, 具体的命令, 比如要编译 zImage, 那么命令展开以后如下所示:

```
@ make -f ./scripts/Makefile.build obj=arch/arm/boot MACHINE=arch/arm/boot/zImage
```

看来又是使用 scripts/Makefile.build 文件来完成 vmlinux 到 zImage 的转换。

关于 Linux 顶层 Makefile 就讲解到这里, 基本和 uboot 的顶层 Makefile 一样, 重点在于 vmlinux 的生成。最后将 vmlinux 压缩成我们最常用的 zImage 或 uImage 等文件。

第三十六章 Linux 内核启动流程

看完 Linux 内核的顶层 Makefile 以后再看 Linux 内核的大致启动流程, Linux 内核的启动流程要比 uboot 复杂的多, 涉及到的内容也更多, 因此本章我们就大致的了解一下 Linux 内核的启动流程。

36.1 链接脚本 vmlinux.lds

要分析 Linux 启动流程，同样需要先编译一下 Linux 源码，因为有很多文件是需要编译才会生成的。首先分析 Linux 内核的连接脚本文件 arch/arm/kernel/vmlinux.lds，通过链接脚本可以找到 Linux 内核的第一行程序是从哪里执行的。vmlinux.lds 中有如下代码：

示例代码 36.1.1 vmlinux.lds 链接脚本

```
492 OUTPUT_ARCH (arm)
493 ENTRY (stext)
494 jiffies = jiffies_64;
495 SECTIONS
496 {
497 /*
498  * XXX: The linker does not define how output sections are
499  * assigned to input sections when there are multiple statements
500  * matching the same input section name. There is no documented
501  * order of matching.
502  *
503  * unwind exit sections must be discarded before the rest of the
504  * unwind sections get included.
505  */
506 /DISCARD/ : {
507  *(.ARM.exidx.exit.text)
508  *(.ARM.extab.exit.text)
509
510  .....
645 }
```

第 493 行的 ENTRY 指明了 Linux 内核入口，入口为 stext，stext 定义在文件 arch/arm/kernel/head.S 中，因此要分析 Linux 内核的启动流程，就得先从文件 arch/arm/kernel/head.S 的 stext 处开始分析。

36.2 Linux 内核启动流程分析

36.2.1 Linux 内核入口 stext

stext 是 Linux 内核的入口地址，在文件 arch/arm/kernel/head.S 中有如下所示提示内容：

示例代码 36.2.1.1 arch/arm/kernel/head.S 代码段

```
/*
 * Kernel startup entry point.
 * -----
 *
 * This is normally called from the decompressor code. The requirements
 * are: MMU = off, D-cache = off, I-cache = dont care, r0 = 0,
 * r1 = machine nr, r2 = atags or dtb pointer.
 * .....
 */
```

*/

根据示例代码 36.2.1.1 中的注释, Linux 内核启动之前要求如下:

- ①、关闭 MMU。
- ②、关闭 D-cache。
- ③、I-Cache 无所谓。
- ④、r0=0。
- ⑤、r1=machine nr(也就是机器 ID)。
- ⑥、r2=atags 或者设备树(dtb)首地址。

Linux 内核的入口点 `stext` 其实相当于内核的入口函数, `stext` 函数内容如下:

示例代码 36.2.1.2 arch/arm/kernel/head.S 代码段

```

80 ENTRY(stext)
.....
91     @ ensure svc mode and all interrupts masked
92     safe_svcmode_maskall r9
93
94     mrc p15, 0, r9, c0, c0      @ get processor id
95     bl __lookup_processor_type  @ r5=procinfo r9=cuid
96     movs    r10, r5             @ invalid processor (r5=0)?
97     THUMB( it eq )              @ force fixup-able long branch encoding
98     beq __error_p               @ yes, error 'p'
99
.....
107
108 #ifndef CONFIG_XIP_KERNEL
.....
113 #else
114     ldr r8, =PLAT_PHYS_OFFSET    @ always constant in this case
115 #endif
116
117     /*
118     * r1 = machine no, r2 = atags or dtb,
119     * r8 = phys_offset, r9 = cuid, r10 = procinfo
120     */
121     bl __vet_atags
.....
128     bl __create_page_tables
129
130     /*
131     * The following calls CPU specific code in a position independent
132     * manner. See arch/arm/mm/proc-*.S for details. r10 = base of
133     * xxx_proc_info structure selected by __lookup_processor_type
134     * above. On return, the CPU will be ready for the MMU to be
135     * turned on, and r0 will hold the CPU control register value.

```

```

136  */
137  ldr r13, =__mmap_switched      @ address to jump to after
138                                @ mmu has been enabled
139  adr lr, BSYM(1f)               @ return (PIC) address
140  mov r8, r4                    @ set TTBR1 to swapper_pg_dir
141  ldr r12, [r10, #PROCINFO_INITFUNC]
142  add r12, r12, r10
143  ret r12
144 1: b  __enable_mmu
145 ENDPROC(stext)

```

第 92 行, 调用函数 `safe_svcmode_maskall` 确保 CPU 处于 SVC 模式, 并且关闭了所有的中断。`safe_svcmode_maskall` 定义在文件 `arch/arm/include/asm/assembler.h` 中。

第 94 行, 读处理器 ID, ID 值保存在 r9 寄存器中。

第 95 行, 调用函数 `__lookup_processor_type` 检查当前系统是否支持此 CPU, 如果支持就获取 `procinfo` 信息。`procinfo` 是 `proc_info_list` 类型的结构体, `proc_info_list` 在文件 `arch/arm/include/asm/procinfo.h` 中的定义如下:

示例代码 36.2.1.3 `proc_info_list` 结构体

```

struct proc_info_list {
    unsigned int      cpu_val;
    unsigned int      cpu_mask;
    unsigned long     __cpu_mm_mmu_flags; /* used by head.S */
    unsigned long     __cpu_io_mmu_flags; /* used by head.S */
    unsigned long     __cpu_flush;       /* used by head.S */
    const char        *arch_name;
    const char        *elf_name;
    unsigned int      elf_hwcap;
    const char        *cpu_name;
    struct processor   *proc;
    struct cpu_tlb_fns *tlb;
    struct cpu_user_fns *user;
    struct cpu_cache_fns *cache;
};

```

Linux 内核将每种处理器都抽象为一个 `proc_info_list` 结构体, 每种处理器都对应一个 `procinfo`。因此可以通过处理器 ID 来找到对应的 `procinfo` 结构, `__lookup_processor_type` 函数找到对应处理器的 `procinfo` 以后会将其保存到 r5 寄存器中。

继续回到示例代码 36.2.1.2 中, 第 121 行, 调用函数 `__vet_atags` 验证 `atags` 或设备树(dtb)的合法性。函数 `__vet_atags` 定义在文件 `arch/arm/kernel/head-common.S` 中。

第 128 行, 调用函数 `__create_page_tables` 创建页表。

第 137 行, 将函数 `__mmap_switched` 的地址保存到 r13 寄存器中。`__mmap_switched` 定义在文件 `arch/arm/kernel/head-common.S`, `__mmap_switched` 最终会调用 `start_kernel` 函数。

第 144 行, 调用 `__enable_mmu` 函数使能 MMU, `__enable_mmu` 定义在文件 `arch/arm/kernel/head.S` 中。`__enable_mmu` 最终会通过调用 `__turn_mmu_on` 来打开 MMU, `__turn_mmu_on` 最后会执行 r13 里面保存的 `__mmap_switched` 函数。

36.2.2 __mmap_switched 函数

__mmap_switched 函数定义在文件 arch/arm/kernel/head-common.S 中，函数代码如下：

示例代码 36.2.2.1 __mmap_switched 函数

```

81 __mmap_switched:
82     adr r3, __mmap_switched_data
83
84     ldmia r3!, {r4, r5, r6, r7}
85     cmp r4, r5                @ Copy data segment if needed
86 1: cmpne r5, r6
87     ldrne fp, [r4], #4
88     strne fp, [r5], #4
89     bne 1b
90
91     mov fp, #0                @ Clear BSS (and zero fp)
92 1: cmp r6, r7
93     strcc fp, [r6], #4
94     bcc 1b
95
96 ARM( ldmia r3, {r4, r5, r6, r7, sp})
97 THUMB( ldmia r3, {r4, r5, r6, r7} )
98 THUMB( ldr sp, [r3, #16] )
99     str r9, [r4]                @ Save processor ID
100    str r1, [r5]                @ Save machine type
101    str r2, [r6]                @ Save atags pointer
102    cmp r7, #0
103    strne r0, [r7]                @ Save control register values
104    b start_kernel
105 ENDPROC(__mmap_switched)

```

第 104 行最终调用 start_kernel 来启动 Linux 内核，start_kernel 函数定义在文件 init/main.c 中。

36.2.3 start_kernel 函数

start_kernel 通过调用众多的子函数来完成 Linux 启动之前的一些初始化工作，由于 start_kernel 函数里面调用的子函数太多，而这些子函数又很复杂，因此我们简单的来看一下一些重要的子函数。精简并添加注释后的 start_kernel 函数内容如下：

示例代码 36.2.3.1 start_kernel 函数

```

asm linkage __visible void __init start_kernel(void)
{
    char *command_line;
    char *after_dashes;

    lockdep_init(); /* lockdep 是死锁检测模块，此函数会初始化

```



```

        * 两个 hash 表。此函数要求尽可能早的执行！
        */

set_task_stack_end_magic(&init_task); /* 设置任务栈结束魔术数，
                                         *用于栈溢出检测
                                         */

smp_setup_processor_id(); /* 跟 SMP 有关(多核处理器)，设置处理器 ID。
                             * 有很多资料说 ARM 架构下此函数为空函数，那是因
                             * 为他们用的老版本 Linux，而那时候 ARM 还没有多
                             * 核处理器。
                             */

debug_objects_early_init(); /* 做一些和 debug 有关的初始化 */
boot_init_stack_canary(); /* 栈溢出检测初始化 */
cgroup_init_early(); /* cgroup 初始化，cgroup 用于控制 Linux 系统资源 */
local_irq_disable(); /* 关闭当前 CPU 中断 */
early_boot_irqs_disabled = true;

/*
 * 中断关闭期间做一些重要的操作，然后打开中断
 */

boot_cpu_init(); /* 跟 CPU 有关的初始化 */
page_address_init(); /* 页地址相关的初始化 */
pr_notice("%s", linux_banner); /* 打印 Linux 版本号、编译时间等信息 */
setup_arch(&command_line); /* 架构相关的初始化，此函数会解析传递进来的
                             * ATAGS 或者设备树 (DTB) 文件。会根据设备树里面
                             * 的 model 和 compatible 这两个属性值来查找
                             * Linux 是否支持这个单板。此函数也会获取设备树
                             * 中 chosen 节点下的 bootargs 属性值来得到命令
                             * 行参数，也就是 uboot 中的 bootargs 环境变量的
                             * 值，获取到的命令行参数会保存到
                             * command_line 中。
                             */

mm_init_cpumask(&init_mm); /* 看名字，应该是和内存有关的初始化 */
setup_command_line(command_line); /* 好像是存储命令行参数 */
setup_nr_cpu_ids(); /* 如果只是 SMP (多核 CPU) 的话，此函数用于获取
                     * CPU 核心数量，CPU 数量保存在变量
                     * nr_cpu_ids 中。
                     */

setup_per_cpu_areas(); /* 在 SMP 系统中有用，设置每个 CPU 的 per-cpu 数据 */
smp_prepare_boot_cpu();

build_all_zonelists(NULL, NULL); /* 建立系统内存页区 (zone) 链表 */
page_alloc_init(); /* 处理用于热插拔 CPU 的页 */

```

```

/* 打印命令行信息 */
pr_notice("Kernel command line: %s\n", boot_command_line);
parse_early_param();          /* 解析命令行中的 console 参数 */
after_dashes = parse_args("Booting kernel",
                          static_command_line, __start__param,
                          __stop__param - __start__param,
                          -1, -1, &unknown_bootoption);
if (!IS_ERR_OR_NULL(after_dashes))
    parse_args("Setting init args", after_dashes, NULL, 0, -1, -1,
              set_init_arg);

jump_label_init();

setup_log_buf(0);              /* 设置 log 使用的缓冲区 */
pidhash_init();               /* 构建 PID 哈希表, Linux 中每个进程都有一个 ID,
                              * 这个 ID 叫做 PID。通过构建哈希表可以快速搜索进程
                              * 信息结构体。
                              */

vfs_caches_init_early();      /* 预先初始化 vfs (虚拟文件系统) 的目录项和
                              * 索引节点缓存
                              */

sort_main_extable();          /* 定义内核异常列表 */
trap_init();                  /* 完成对系统保留中断向量的初始化 */
mm_init();                    /* 内存管理初始化 */

sched_init();                 /* 初始化调度器, 主要是初始化一些结构体 */
preempt_disable();            /* 关闭优先级抢占 */
if (WARN(!irqs_disabled(), /* 检查中断是否关闭, 如果没有的话就关闭中断 */
        "Interrupts were enabled *very* early, fixing it\n"))
    local_irq_disable();
idr_init_cache();             /* IDR 初始化, IDR 是 Linux 内核的整数管理机
                              * 制, 也就是将一个整数 ID 与一个指针关联起来。
                              */

rcu_init();                   /* 初始化 RCU, RCU 全称为 Read Copy Update (读-拷贝修改) */
trace_init();                 /* 跟踪调试相关初始化 */

context_tracking_init();
radix_tree_init();            /* 基数树相关数据结构初始化 */
early_irq_init();             /* 初始中断相关初始化, 主要是注册 irq_desc 结构体变
                              * 量, 因为 Linux 内核使用 irq_desc 来描述一个中断。
                              */

init_IRQ();                   /* 中断初始化 */
tick_init();                  /* tick 初始化 */

```

```

rcu_init_nohz();
init_timers();           /* 初始化定时器 */
hrtimers_init();         /* 初始化高精度定时器 */
softirq_init();          /* 软中断初始化 */
timekeeping_init();
time_init();             /* 初始化系统时间 */
sched_clock_postinit();
perf_event_init();
profile_init();
call_function_init();
WARN(!irqs_disabled(), "Interrupts were enabled early\n");
early_boot_irqs_disabled = false;
local_irq_enable();      /* 使能中断 */

kmem_cache_init_late(); /* slab 初始化, slab 是 Linux 内存分配器 */
console_init();         /* 初始化控制台, 之前 printk 打印的信息都存放
                        * 缓冲区中, 并没有打印出来。只有调用此函数
                        * 初始化控制台以后才能在控制台上打印信息。
                        */

if (panic_later)
    panic("Too many boot %s vars at '%s'", panic_later,
          panic_param);

lockdep_info(); /* 如果定义了宏 CONFIG_LOCKDEP, 那么此函数打印一些信息。 */

locking_selftest()      /* 锁自测 */
.....
page_ext_init();
debug_objects_mem_init();
kmemleak_init();        /* kmemleak 初始化, kmemleak 用于检查内存泄漏 */
setup_per_cpu_pageset();
numa_policy_init();
if (late_time_init)
    late_time_init();
sched_clock_init();
calibrate_delay(); /* 测定 BogoMIPS 值, 可以通过 BogoMIPS 来判断 CPU 的性能
                  * BogoMIPS 设置越大, 说明 CPU 性能越好。
                  */

pidmap_init();          /* PID 位图初始化 */
anon_vma_init();        /* 生成 anon_vma slab 缓存 */
acpi_early_init();
.....
thread_info_cache_init();

```

```

cred_init();          /* 为对象的每个用于赋予资格(凭证) */
fork_init();          /* 初始化一些结构体以使用 fork 函数 */
proc_caches_init();   /* 给各种资源管理结构分配缓存 */
buffer_init();        /* 初始化缓冲缓存 */
key_init();           /* 初始化密钥 */
security_init();      /* 安全相关初始化 */
dbg_late_init();

vfs_caches_init(totalram_pages); /* 为 VFS 创建缓存 */
signals_init();        /* 初始化信号 */

page_writeback_init(); /* 页回写初始化 */
proc_root_init();      /* 注册并挂载 proc 文件系统 */
nsfs_init();
cpuset_init();         /* 初始化 cpuset, cpuset 是将 CPU 和内存资源以逻辑性
                        * 和层次性集成的一种机制, 是 cgroup 使用的子系统之一
                        */

cgroup_init();         /* 初始化 cgroup */
taskstats_init_early(); /* 进程状态初始化 */
delayacct_init();

check_bugs();          /* 检查写缓冲一致性 */

acpi_subsystem_init();
sfi_init_late();

if (efi_enabled(EFI_RUNTIME_SERVICES)) {
    efi_late_init();
    efi_free_boot_services();
}

ftrace_init();

rest_init();           /* rest_init 函数 */
}

```

start_kernel 里面调用了大量的函数, 每一个函数都是一个庞大的知识点, 如果想要学习 Linux 内核, 那么这些函数就需要去详细的研究。本教程注重于嵌入式 Linux 入门, 因此不会去讲太多关于 Linux 内核的知识。start_kernel 函数最后调用了 rest_init, 接下来简单看一下 rest_init 函数。

36.2.4 rest_init 函数

rest_init 函数定义在文件 init/main.c 中, 函数内容如下:

示例代码 36.2.4.1 rest_init 函数

```
383 static noinline void __init_refok rest_init(void)
```

```
384 {
385     int pid;
386
387     rcu_scheduler_starting();
388     smpboot_thread_init();
389     /*
390      * We need to spawn init first so that it obtains pid 1, however
391      * the init task will end up wanting to create kthreads, which,
392      * if we schedule it before we create kthreadd, will OOPS.
393      */
394     kernel_thread(kernel_init, NULL, CLONE_FS);
395     numa_default_policy();
396     pid = kernel_thread(kthreadd, NULL, CLONE_FS | CLONE_FILES);
397     rcu_read_lock();
398     kthreadd_task = find_task_by_pid_ns(pid, &init_pid_ns);
399     rcu_read_unlock();
400     complete(&kthreadd_done);
401
402     /*
403      * The boot idle thread must execute schedule()
404      * at least once to get things moving:
405      */
406     init_idle_bootup_task(current);
407     schedule_preempt_disabled();
408     /* Call into cpu_idle with preempt disabled */
409     cpu_startup_entry(CPUHP_ONLINE);
410 }
```

第 387 行, 调用函数 `rcu_scheduler_starting`, 启动 RCU 锁调度器

第 394 行, 调用函数 `kernel_thread` 创建 `kernel_init` 进程, 也就是大名鼎鼎的 `init` 内核进程。`init` 进程的 PID 为 1。`init` 进程一开始是内核进程(也就是运行在内核态), 后面 `init` 进程会在根文件系统中查找名为“`init`”这个程序, 这个“`init`”程序处于用户态, 通过运行这个“`init`”程序, `init` 进程就会实现从内核态到用户态的转变。

第 396 行, 调用函数 `kernel_thread` 创建 `kthreadd` 内核进程, 此内核进程的 PID 为 2。`kthreadd` 进程负责所有内核进程的调度和管理。

第 409 行, 最后调用函数 `cpu_startup_entry` 来进入 `idle` 进程, `cpu_startup_entry` 会调用 `cpu_idle_loop`, `cpu_idle_loop` 是个 `while` 循环, 也就是 `idle` 进程代码。`idle` 进程的 PID 为 0, `idle` 进程叫做空闲进程, 如果学过 FreeRTOS 或者 UCOS 的话应该听说过空闲任务。`idle` 空闲进程就和空闲任务一样, 当 CPU 没有事情做的时候就在 `idle` 空闲进程里面“瞎逛游”, 反正就是给 CPU 找点事做。当其他进程要工作的时候就会抢占 `idle` 进程, 从而夺取 CPU 使用权。其实大家应该可以看到 `idle` 进程并没有使用 `kernel_thread` 或者 `fork` 函数来创建, 因为它是有主进程演变而来的。

在 Linux 终端中输入“`ps -A`”就可以打印出当前系统中的所有进程, 其中就能看到 `init` 进程和 `kthreadd` 进程, 如图 36.2.4.1 所示:

```

/ # ps -A
PID      USER        TIME  COMMAND
    1      0           0:01   init
    2      0           0:00   [kthreadd]
    3      0           0:00   [ksoftirqd/0]
    5      0           0:00   [kworker/0:0H]
    7      0           0:00   [rcu_preempt]

```

图 36.2.4.1 Linux 系统当前进程

从图 36.2.4.1 可以看出, init 进程的 PID 为 1, kthreadd 进程的 PID 为 2。之所以图 36.2.4.1 中没有显示 PID 为 0 的 idle 进程, 那是因为 idle 进程是内核进程。我们接下来重点看一下 init 进程, kernel_init 就是 init 进程的进程函数。

36.2.5 init 进程

kernel_init 函数就是 init 进程具体做的工作, 定义在文件 init/main.c 中, 函数内容如下:

示例代码 36.2.5.1 kernel_init 函数

```

928 static int __ref kernel_init(void *unused)
929 {
930     int ret;
931
932     kernel_init_freeable();          /* init 进程的一些其他初始化工作 */
933     /* need to finish all async __init code before freeing the
       memory */
934     async_synchronize_full();        /* 等待所有的异步调用执行完成 */
935     free_initmem();                  /* 释放 init 段内存 */
936     mark_rodata_ro();
937     system_state = SYSTEM_RUNNING;  /* 标记系统正在运行 */
938     numa_default_policy();
939
940     flush_delayed_fput();
941
942     if (ramdisk_execute_command) {
943         ret = run_init_process(ramdisk_execute_command);
944         if (!ret)
945             return 0;
946         pr_err("Failed to execute %s (error %d)\n",
947               ramdisk_execute_command, ret);
948     }
949
950     /*
951     * We try each of these until one succeeds.
952     *
953     * The Bourne shell can be used instead of init if we are
954     * trying to recover a really broken machine.
955     */

```

```

956     if (execute_command) {
957         ret = run_init_process(execute_command);
958         if (!ret)
959             return 0;
960         panic("Requested init %s failed (error %d).",
961             execute_command, ret);
962     }
963     if (!try_to_run_init_process("/sbin/init") ||
964         !try_to_run_init_process("/etc/init") ||
965         !try_to_run_init_process("/bin/init") ||
966         !try_to_run_init_process("/bin/sh"))
967         return 0;
968
969     panic("No working init found. Try passing init= option to
kernel. "
970         "See Linux Documentation/init.txt for guidance.");
971 }

```

第 932 行, `kernel_init_freeable` 函数用于完成 `init` 进程的一些其他初始化工作, 稍后再来具体看一下此函数。

第 940 行, `ramdisk_execute_command` 是一个全局的 `char` 指针变量, 此变量值为 `"/init"`, 也就是根目录下的 `init` 程序。`ramdisk_execute_command` 也可以通过 `uboot` 传递, 在 `bootargs` 中使用 `"rdinit=xxx"` 即可, `xxx` 为具体的 `init` 程序名字。

第 943 行, 如果存在 `"/init"` 程序的话就通过函数 `run_init_process` 来运行此程序。

第 956 行, 如果 `ramdisk_execute_command` 为空的话就看 `execute_command` 是否为空, 反正不管如何一定要在根文件系统中找到一个可运行的 `init` 程序。`execute_command` 的值是通过 `uboot` 传递, 在 `bootargs` 中使用 `"init=xxxx"` 就可以了, 比如 `"init=/linuxrc"` 表示根文件系统中的 `linuxrc` 就是要执行的用户空间 `init` 程序。

第 963~966 行, 如果 `ramdisk_execute_command` 和 `execute_command` 都为空, 那么就依次查找 `"/sbin/init"`、`"/etc/init"`、`"/bin/init"` 和 `"/bin/sh"`, 这四个相当于备用 `init` 程序, 如果这四个也不存在, 那么 Linux 启动失败!

第 969 行, 如果以上步骤都没有找到用户空间的 `init` 程序, 那么就提示错误发生!

最后来简单看一下 `kernel_init_freeable` 函数, 前面说了, `kernel_init` 会调用此函数来做一些 `init` 进程初始化工作。`kernel_init_freeable` 定义在文件 `init/main.c` 中, 缩减后的函数内容如下:

示例代码 36.2.5.2 `kernel_init_freeable` 函数

```

973 static noinline void __init kernel_init_freeable(void)
974 {
975     /*
976      * Wait until kthreadd is all set-up.
977      */
978     wait_for_completion(&kthreadd_done); /* 等待 kthreadd 进程准备就绪 */
979     .....
980
981     smp_init(); /* SMP 初始化 */

```



```
1000     sched_init_smp();           /* 多核(SMP) 调度初始化 */
1001
1002     do_basic_setup();           /* 设备初始化都在此函数中完成 */
1003
1004     /* Open the /dev/console on the rootfs, this should never fail */
1005     if (sys_open((const char __user *) "/dev/console", O_RDWR, 0) <
1006         0)
1007         pr_err("Warning: unable to open an initial console.\n");
1008
1009     (void) sys_dup(0);
1010     (void) sys_dup(0);
1011     /*
1012      * check if there is an early userspace init. If yes, let it do
1013      * all the work
1014      */
1015     if (!ramdisk_execute_command)
1016         ramdisk_execute_command = "/init";
1017
1018     if (sys_access((const char __user *) ramdisk_execute_command,
1019         0) != 0) {
1020         ramdisk_execute_command = NULL;
1021         prepare_namespace();
1022     }
1023
1024     /*
1025      * Ok, we have completed the initial bootup, and
1026      * we're essentially up and running. Get rid of the
1027      * initmem segments and start the user-mode stuff..
1028      *
1029      * rootfs is available now, try loading the public keys
1030      * and default modules
1031      */
1032     integrity_load_keys();
1033     load_default_modules();
1034 }
```

第 1002 行, `do_basic_setup` 函数用于完成 Linux 下设备驱动初始化工作! 非常重要。`do_basic_setup` 会调用 `driver_init` 函数完成 Linux 下驱动模型子系统的初始化。

第 1005 行, 打开设备 “/dev/console”, 在 Linux 中一切皆为文件! 因此 “/dev/console” 也是一个文件, 此文件为控制台设备。每个文件都有一个文件描述符, 此处打开的 “/dev/console” 文件描述符为 0, 作为标准输入(0)。

第 1008 和 1009 行, `sys_dup` 函数将标准输入(0)的文件描述符复制了 2 次, 一个作为标准输出(1), 一个作为标准错误(2)。这样标准输入、输出、错误都是 `/dev/console` 了。`console` 通过 `uboot` 的 `bootargs` 环境变量设置, “`console=ttyMXC0,115200`”表示将 `/dev/ttyMXC0` 设置为 `console`, 也就是 I.MX6U 的串口 1。当然, 也可以设置其他的设备为 `console`, 比如虚拟控制台 `tty1`, 设置 `tty1` 为 `console` 就可以在 LCD 屏幕上看到系统的提示信息。

第 1020 行, 调用函数 `prepare_namespace` 来挂载根文件系统。根文件系统也是由命令行参数指定的, 就是 `uboot` 的 `bootargs` 环境变量。比如 “`root=/dev/mmcblk1p2 rootwait rw`” 就表示根文件系统在 `/dev/mmcblk1p2` 中, 也就是 EMMC 的分区 2 中。

Linux 内核启动流程就分析到这里, Linux 内核最终是需要和根文件系统打交道的, 需要挂载根文件系统, 并且执行根文件系统中的 `init` 程序, 以此来进去用户态。这里就正式引出了根文件系统, 根文件系统也是我们系统移植的最后一片拼图。Linux 移植三巨头: `uboot`、Linux `kernel`、`rootfs`(根文件系统)。关于根文件系统后面章节会详细的讲解, 这里我们只需要知道 Linux 内核移植完成以后还需要构建根文件系统即可。

第三十七章 Linux 内核移植

前两章我们简单了解了一下 Linux 内核顶层 Makefile 和 Linux 内核的启动流程, 本章我们就来学习一下如何将 NXP 官方提供的 Linux 内核移植到正点原子的 I.MX6U-ALPHA 开发板上。通过本章的学习, 我们将掌握如何将半导体厂商提供的 Linux BSP 包移植到我们自己的平台上。

37.1 创建 VSCode 工程

这里我们使用 NXP 官方提供的 Linux 源码, 将其移植到正点原子 I.MX6U-ALPHA 开发板上。NXP 官方原版 Linux 源码已经放到了开发板光盘中, 路径为: **1、例程源码->4、NXP 官方原版 Uboot 和 Linux->linux-imx-rel_imx_4.1.15_2.1.0_ga.tar.bz2**。使用 FileZilla 将其发送到 Ubuntu 中并解压, 得到名为 linux-imx-rel_imx_4.1.15_2.1.0_ga 的目录, 为了和 NXP 官方的名字区分, 可以使用 “mv” 命令对其重命名, 我这里将其重命名为 “linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek”, 命令如下:

```
mv linux-imx-rel_imx_4.1.15_2.1.0_ga linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
```

完成以后创建 VSCode 工程, 步骤和 Windows 下一样, 重点是.vscode/settings.json 这个文件。

37.2 NXP 官方开发板 Linux 内核编译

NXP 提供的 Linux 源码肯定是在自己的 I.MX6ULL EVK 开发板上运行下去的, 所以我们肯定是以 I.MX6ULL EVK 开发板为参考, 然后将 Linux 内核移植到 I.MX6U-ALPHA 开发板上的。

37.2.1 修改顶层 Makefile

修改顶层 Makefile, 直接在顶层 Makefile 文件里面定义 ARCH 和 CROSS_COMPILE 这两个的变量值为 arm 和 arm-linux-gnueabi, 结果如图 37.2.1 所示:

```
242 # CROSS_COMPILE specify the prefix used for all executables used
243 # during compilation. Only gcc and related bin-utils executables
244 # are prefixed with $(CROSS_COMPILE).
245 # CROSS_COMPILE can be set on the command line
246 # make CROSS_COMPILE=ia64-linux-
247 # Alternatively CROSS_COMPILE can be set in the environment.
248 # A third alternative is to store a setting in .config so that plain
249 # "make" in the configured kernel build directory always uses that.
250 # Default value for CROSS_COMPILE is not to prefix executables
251 # Note: Some architectures assign CROSS_COMPILE in their arch/*/Makefile
252 ARCH      ?= arm
253 CROSS_COMPILE  ?= arm-linux-gnueabi-
```

图 37.2.1 修改顶层 Makefile

图 37.2.1 中第 252 和 253 行分别设置了 ARCH 和 CROSS_COMPILE 这两个变量的值, 这样在编译的时候就不用输入很长的命令了。

37.2.2 配置并编译 Linux 内核

和 uboot 一样, 在编译 Linux 内核之前要先配置 Linux 内核。每个板子都有其对应的默认配置文件, 这些默认配置文件保存在 arch/arm/configs 目录中。imx_v7_defconfig 和 imx_v7_mfg_defconfig 都可作为 I.MX6ULLEVK 开发板所使用的默认配置文件。但是这里建议使用 imx_v7_mfg_defconfig 这个默认配置文件, 首先此配置文件默认支持 I.MX6UL 这款芯片, 而且重要的一点就是此文件编译出来的 zImage 可以通过 NXP 官方提供的 MfgTool 工具烧写!! imx_v7_mfg_defconfig 中的 “mfg” 的意思就是 MfgTool。

进入到 Ubuntu 中的 Linux 源码根目录下, 执行如下命令配置 Linux 内核:

```
make clean //第一次编译 Linux 内核之前先清理一下
```

```
make imx_v7_mfg_defconfig //配置 Linux 内核
```

配置完成以后如图 37.2.2.1 所示:

```
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek$ make imx_v7_mfg_defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/zconf.tab.o
HOSTLD scripts/kconfig/conf
#
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek$
```

图 37.2.2.1 配置 Linux 内核

配置完成以后就可以编译了, 使用如下命令编译 Linux 内核:

```
make -j16 //编译 Linux 内核
```

等待编译完成, 结果如图 37.2.2.2 所示:

```
LD [M] lib/libcrc32c.ko
LD [M] lib/crc-itu-t.ko
LD [M] sound/core/snd-rawmidi.ko
LD [M] sound/core/snd-hwdep.ko
LD [M] sound/usb/snd-usbmidi-lib.ko
LD [M] sound/usb/snd-usb-audio.ko
AS arch/arm/boot/compressed/piggy.lzo.o
LD arch/arm/boot/compressed/vmlinux
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
zuozhongkai@ubuntu:~/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek$
```

图 37.2.2.2 Linux 编译完成

Linux 内核编译完成以后会在 arch/arm/boot 目录下生成 zImage 镜像文件, 如果使用设备树的话还会在 arch/arm/boot/dts 目录下开发板对应的.dtb(设备树)文件, 比如 imx6ull-14x14-evk.dtb 就是 NXP 官方的 I.MX6ULL EVK 开发板对应的设备树文件。至此我们得到两个文件:

- ①、Linux 内核镜像文件: zImage。
- ②、NXP 官方 I.MX6ULL EVK 开发板对应的设备树文件: imx6ull-14x14-evk.dtb。

37.2.3 Linux 内核启动测试

在上一小节我们已经得到了 NXP 官方 I.MX6ULL EVK 开发板对应的 zImage 和 imx6ull-14x14-evk.dtb 这两个文件。这两个文件能不能在正点原子的 I.MX6U-ALPHA EMMC 版开发板上启动呢? 测试一下不就知道了, 在测试之前确保 uboot 中的环境变量 bootargs 内容如下:

```
console=ttyMXC0,115200 root=/dev/mmcblk1p2 rootwait rw
```

将上一小节编译出来的 zImage 和 imx6ull-14x14-evk.dtb 复制到 Ubuntu 中的 tftp 目录下, 因为我们要在 uboot 中使用 tftp 命令将其下载到开发板中, 拷贝命令如下:

```
cp arch/arm/boot/zImage /home/zuozhongkai/linux/tftpboot/ -f
cp arch/arm/boot/dts/imx6ull-14x14-evk.dtb /home/zuozhongkai/linux/tftpboot/ -f
```

拷贝完成以后就可以测试了, 启动开发板, 进入 uboot 命令行模式, 然后输入如下命令将 zImage 和 imx6ull-14x14-evk.dtb 下载到开发板中并启动:

```
tftp 80800000 zImage
tftp 83000000 imx6ull-14x14-evk.dtb
bootz 80800000 - 83000000
```

结果图 37.2.3.1 所示:

```
=> tftp 80800000 zImage
FEC1 Waiting for PHY auto negotiation to complete.... done
Using FEC1 device
TFTP from server 192.168.1.250; our IP address is 192.168.1.251
Filename 'zImage'.
Load address: 0x80800000
Loading: #####
#####
#####
#####
#####
#####
#####
#
2.4 MiB/s
done
Bytes transferred = 6680432 (65ef70 hex)
=> tftp 83000000 imx6ull-14x14-evk.dtb
Using FEC1 device
TFTP from server 192.168.1.250; our IP address is 192.168.1.251
Filename 'imx6ull-14x14-evk.dtb'.
Load address: 0x83000000
Loading: ###
1.2 MiB/s
done
Bytes transferred = 35969 (8c81 hex)
=> bootz 80800000 - 83000000
Kernel image @ 0x80800000 [ 0x000000 - 0x65ef70 ]
## Flattened Device Tree blob at 83000000
   Booting using the fdt blob at 0x83000000
   Using Device Tree in place at 83000000, end 8300bc80

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 4.1.15 (zuozhongkai@ubuntu) (gcc version 4.9.4 (Linaro GCC 4.9-2017.01))
n 8 12:26:48 CST 2019
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c53c7d
```

图 37.2.3.1 启动 Linux 内核

从图 37.2.3.1 可以看出,此时 Linux 内核已经启动了,如果 EMMC 中的根文件系统存在,我们就可以进入到 Linux 系统里面使用命令进行操作如图 37.2.3.2 所示:

```
VFS: Mounted root (ext3 filesystem) on device 1/9:10.
devtmpfs: mounted
Freeing unused kernel memory: 440K (80b18000 - 80b86000)
usb 1-1.2: new high-speed USB device number 4 using ci_hdrc
ALSA: Restoring mixer settings...

Please press Enter to activate this console.
/ #
/ #
/ #
/ # random: nonblocking pool is initialized
/ #
/ #
```

图 37.2.3.2 进入 Linux 根文件系统

37.2.4 根文件系统缺失错误

Linux 内核启动以后是需要根文件系统的,根文件系统存在哪里是由 uboot 的 bootargs 环境变量指定, bootargs 会传递给 Linux 内核作为命令行参数。比如上一小节设置 root=/dev/mmcblk1p2,也就是说根文件系统存储在/dev/mmcblk1p2 中,也就是 EMMC 的分区 2 中。这是因为正点原子的 EMMC 版本开发板出厂的时候已经 EMMC 的分区 2 中烧写好了根文件系统,所以设置 root=/dev/mmcblk1p2。如果我们不设置根文件系统路径,或者说根文件系统路径设置错误的话会出现什么问题?这个问题是很常见的,我们在实际的工作中开发一个产品,这个产品的第一版硬件出来以后我们是没有对应的根文件系统可用的,必须要自己做根文件系统。在构建出对应的根文件系统之前 Linux 内核是没有根文件系统可用的,此时 Linux 内核启动以后会出现什么问题呢?带着这个问题,我们将 uboot 中的 bootargs 环境变量改为

“console=ttymxc0,115200”，也就是不填写 root 的内容了，命令如下：

```
setenv bootargs 'console=ttymxc0,115200' //设置 bootargs
saveenv //保存
```

修改完成以后重新从网络启动，启动以后会有如图 37.2.4.1 所示错误：

```
usb 1-1.2: new full-speed USB device number 3 using ci_hdrc
mmcblk1boot1: mmc1:0001 4FTE4R partition 2 4.00 MiB
mmcblk1rpmb: mmc1:0001 4FTE4R partition 3 512 KiB
mmcblk1: p1 p2
VFS: Cannot open root device "(null)" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
0100          65536 ram0 (driver?)
0101          65536 ram1 (driver?)
0102          65536 ram2 (driver?)
0103          65536 ram3 (driver?)
0104          65536 ram4 (driver?)
0105          65536 ram5 (driver?)
0106          65536 ram6 (driver?)
0107          65536 ram7 (driver?)
0108          65536 ram8 (driver?)
0109          65536 ram9 (driver?)
010a          65536 ram10 (driver?)
010b          65536 ram11 (driver?)
010c          65536 ram12 (driver?)
010d          65536 ram13 (driver?)
010e          65536 ram14 (driver?)
010f          65536 ram15 (driver?)
b300       15558144 mmcblk0 driver: mmcblk
b301          512000 mmcblk0p1 f95b0dec-01
b302       14943744 mmcblk0p2 f95b0dec-02
b308       3817472 mmcblk1 driver: mmcblk
b309          512000 mmcblk1p1 b29f4828-01
b30a       3203072 mmcblk1p2 b29f4828-02
b320          512 mmcblk1rpmb (driver?)
b318          4096 mmcblk1boot1 (driver?)
b310          4096 mmcblk1boot0 (driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
```

图 37.2.4.1 根文件系统缺失错误

在图 37.2.4.1 中最后会有下面这一行：

```
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
```

也就是提示内核崩溃，因为 VFS(虚拟文件系统)不能挂载根文件系统，因为根文件系统目录不存在。即使根文件系统目录存在，如果根文件系统目录里面是空的依旧会提示内核崩溃。这个就是根文件系统缺失导致的内核崩溃，但是内核是启动了的，只是根文件系统不存在而已。

37.3 在 Linux 中添加自己的开发板

在 37.2 小节中我们通过编译 NXP 官方 I.MX6ULL EVK 开发板对应的 Linux 内核，发现其可以在正点原子的 EMMC 版本开发板启动，所以我们就参考 I.MX6ULL EVK 开发板的设置，在 Linux 内核中添加正点原子的 I.MX6U-ALPHA 开发板。

37.3.1 添加开发板默认配置文件

将 arch/arm/configs 目录下的 imx_v7_mfg_defconfig 重新复制一份，命名为 imx_alientek_emmc_defconfig，命令如下：

```
cd arch/arm/configs
cp imx_v7_mfg_defconfig imx_alientek_emmc_defconfig
```

打开 imx_alientek_emmc_defconfig 文件，找到“CONFIG_ARCH_MULTI_V6=y”这一行，将其屏蔽掉，如图 37.3.1.1 所示：


```

309 # CPU Core family selection
310 #
311 #CONFIG ARCH_MULTI_V6=y
312 CONFIG_ARCH_MULTI_V7=y
313 CONFIG_ARCH_MULTI_V6_V7=y
314 # CONFIG_ARCH_MULTI_CPU_AUTO is not set
315 # CONFIG_ARCH_VIRT is not set

```

图 37.3.1.1 屏蔽掉 V6 选项

因为 I.MX6ULL 是 ARMV7 架构的，因此要屏蔽掉 V6 相关选项，否则后面做驱动实验的时候可能会遇到驱动模块无法加载的情况。

以后 imx_alientek_emmc_defconfig 就是正点原子的 EMMC 版开发板默认配置文件了。完成以后如图 37.3.1.2 所示：

```

imx_alientek_emmc_defconfig
imx_v4_v5_defconfig
imx_v6_v7_defconfig
imx_v7_defconfig
imx_v7_mfg_defconfig

```

图 37.3.1.2 新添加的默认配置文件

以后就可以使用如下命令来配置正点原子 EMMC 版开发板对应的 Linux 内核了：

```
make imx_alientek_emmc_defconfig
```

37.3.2 添加开发板对应的设备树文件

添加适合正点原子 EMMC 版开发板的设备树文件，进入目录 arch/arm/boot/dts 中，复制一份 imx6ull-14x14-evk.dts，然后将其重命名为 imx6ull-alientek-emmc.dts，命令如下：

```

cd arch/arm/boot/dts
cp imx6ull-14x14-evk.dts imx6ull-alientek-emmc.dts

```

.dts 是设备树源码文件，编译 Linux 的时候会将其编译为 .dtb 文件。imx6ull-alientek-emmc.dts 创建好以后我们还需要修改文件 arch/arm/boot/dts/Makefile，找到 “dtb-\$(CONFIG_SOC_IMX6ULL)” 配置项，在此配置项中加入 “imx6ull-alientek-emmc.dtb”，如下所示：

示例代码 37.3.2.1 arch/arm/boot/dts/Makefile 代码段

```

400 dtb-$(CONFIG_SOC_IMX6ULL) += \
401     imx6ull-14x14-ddr3-arm2.dtb \
402     imx6ull-14x14-ddr3-arm2-adc.dtb \
403     imx6ull-14x14-ddr3-arm2-cs42888.dtb \
404     imx6ull-14x14-ddr3-arm2-ecspi.dtb \
405     imx6ull-14x14-ddr3-arm2-emmc.dtb \
406     imx6ull-14x14-ddr3-arm2-epdc.dtb \
407     imx6ull-14x14-ddr3-arm2-flexcan2.dtb \
408     imx6ull-14x14-ddr3-arm2-gpmi-weim.dtb \
409     imx6ull-14x14-ddr3-arm2-lcdif.dtb \
410     imx6ull-14x14-ddr3-arm2-ldo.dtb \
411     imx6ull-14x14-ddr3-arm2-qspi.dtb \
412     imx6ull-14x14-ddr3-arm2-qspi-all.dtb \

```

```

413 imx6ull-14x14-ddr3-arm2-tsc.dtb \
414 imx6ull-14x14-ddr3-arm2-uart2.dtb \
415 imx6ull-14x14-ddr3-arm2-usb.dtb \
416 imx6ull-14x14-ddr3-arm2-wm8958.dtb \
417 imx6ull-14x14-evk.dtb \
418 imx6ull-14x14-evk-btwifi.dtb \
419 imx6ull-14x14-evk-emmc.dtb \
420 imx6ull-14x14-evk-gpmi-weim.dtb \
421 imx6ull-14x14-evk-usb-certi.dtb \
422 imx6ull-alientek-emmc.dtb \
423 imx6ull-9x9-evk.dtb \
424 imx6ull-9x9-evk-btwifi.dtb \
425 imx6ull-9x9-evk-ldo.dtb

```

第 422 行为“imx6ull-alientek-emmc.dtb”，这样编译 Linux 的时候就可以从 imx6ull-alientek-emmc.dts 编译出 imx6ull-alientek-emmc.dtb 文件了。

37.3.3 编译测试

经过 37.3.1 和 37.3.2 两个小节，Linux 内核里面已经添加了正点原子 I.MX6UL-ALIPHA EMMC 版开发板了，接下接编译测试一下，我们可以创建一个编译脚本，imx6ull_alientek_emmc.sh，脚本内容如下：

示例代码 37.3.2.1 imx6ull_alientek_emmc.sh 编译脚本

```

1 #!/bin/sh
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
imx_alientek_emmc_defconfig
4 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
5 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- all -j16

```

第 2 行，清理工程。

第 3 行，使用默认配置文件 imx_alientek_emmc_defconfig 来配置 Linux 内核。

第 4 行，打开 Linux 的图形配置界面，如果不需要每次都打开图形配置界面可以删除此行。

第 5 行，编译 Linux。

执行 shell 脚本 imx6ull_alientek_emmc.sh 编译 Linux 内核，命令如下：

```

chmod 777 imx6ull_alientek_emmc.sh      //给予可执行权限
./imx6ull_alientek_emmc.sh              //执行 shell 脚本编译内核

```

编译完成以后就会在目录 arch/arm/boot 下生成 zImage 镜像文件。在 arch/arm/boot/dts 目录下生成 imx6ull-alientek-emmc.dtb 文件。将这两个文件拷贝到 tftp 目录下，然后重启开发板，在 uboot 命令模式中使用 tftp 命令下载这两个文件并启动，命令如下：

```

tftp 80800000 zImage
tftp 83000000 imx6ull-alientek-emmc.dtb
bootz 80800000 - 83000000

```

只要出现如图 37.3.3.1 所示内容就表示 Linux 内核启动成功：

```
Booting Linux on physical CPU 0x0
Linux version 4.1.15 (zuozhongkai@ubuntu) (gcc version 4.9.4 (Linaro GCC 4.9-2017.01) ) #
n 8 18:38:28 CST 2019
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c53c7d
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
Machine model: Freescale i.MX6 ULL 14x14 EVK Board
Reserved memory: created CMA memory pool at 0x8c000000, size 320 MiB
Reserved memory: initialized node linux,cma, compatible id shared-dma-pool
Memory policy: Data cache writealloc
```

图 37.3.3.1 Linux 内核启动

Linux 内核启动成功,说明我们已经在 NXP 提供的 Linux 内核源码中添加了正点原子 I.MX6UL-ALPHA 开发板。

37.4 CPU 主频和网络驱动修改

37.4.1 CPU 主频修改

正点原子 I.MX6U-ALPHA 开发板所使用的 I.MX6ULL 芯片主频都是 792MHz 的,也就是 NXP 官方宣传的 800MHz 版本。后续可能会生产 528MHz 核心板供企业级批量用户,但是开发板搭配的都是 792MHz 主频的,本节教程也就以 792MHz 的核心板为例讲解。

1、设置 I.MX6U-ALPHA 开发板工作在 792MHz

确保 EMMC 中的根文件系统可用!然后重新启动开发板,进入终端(可以输入命令),如图 37.4.1.1 所示:

```
devtmpfs: mounted
Freeing unused kernel memory: 440K (80b18000 - 80b86000)
usb 1-1.2: device no response, device descriptor read/64, error -32
usb 1-1.2: not running at top speed; connect to a high speed hub
ALSA: Restoring mixer settings...

Please press Enter to activate this console.
/ #
/ #
/ #
/ #
```

图 37.4.1.1 进入命令行

进入图 37.4.1.1 所示的命令行以后输入如下命令查看 cpu 信息:

```
cat /proc/cpuinfo
```

结果如图 37.4.2 所示:

```
/ # cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 3.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part       : 0xc07
CPU revision   : 5

Hardware      : Freescale i.MX6 Ultralite (Device Tree)
Revision     : 0000
Serial        : 0000000000000000
/ #
```

图 37.4.1.2 cpu 信息

在图 37.4.1.2 中有 BogoMIPS 这一条,此时 BogoMIPS 为 3.00, BogoMIPS 是 Linux 系统中衡量处理器运行速度的一个“尺子”,处理器性能越强,主频越高, BogoMIPS 值就越大。BogoMIPS 只是粗略的计算 CPU 性能,并不十分准确。但是我们可以通过 BogoMIPS 值来大致的判断当前处理器的性能。在图 37.4.1.2 中并没有看到当前 CPU 的工作频率,那我们就转变另

一种方法查看当前 CPU 的工作频率。进入到目录/sys/bus/cpu/devices/cpu0/cpufreq 中，此目录下会有很多文件，如图 37.4.1.3 所示：

```
/sys/devices/system/cpu/cpu0/cpufreq # ls
affected_cpus          scaling_cur_freq
cpuinfo_cur_freq       scaling_driver
cpuinfo_max_freq       scaling_governor
cpuinfo_min_freq       scaling_max_freq
cpuinfo_transition_latency scaling_min_freq
related_cpus           scaling_setspeed
scaling_available_frequencies stats
scaling_available_governors
```

图 37.4.1.3 cpufreq 目录

此目录中记录了 CPU 频率等信息，这些文件的含义如下：

cpuinfo_cur_freq: 当前 cpu 工作频率，从 CPU 寄存器读取到的工作频率。

cpuinfo_max_freq: 处理器所能运行的最高工作频率(单位: KHz)。

cpuinfo_min_freq : 处理器所能运行的最低工作频率(单位: KHz)。

cpuinfo_transition_latency: 处理器切换频率所需要的时间(单位:ns)。

scaling_available_frequencies: 处理器支持的主频率列表(单位: KHz)。

scaling_available_governors: 当前内核中支持的所有 governor(调频)类型。

scaling_cur_freq: 保存着 cpufreq 模块缓存的当前 CPU 频率，不会对 CPU 硬件寄存器进行检查。

scaling_driver: 该文件保存当前 CPU 所使用的调频驱动。

scaling_governor: governor(调频)策略，Linux 内核一共有 5 中调频策略，

①、Performance，最高性能，直接用最高频率，不考虑耗电。

②、Interactive，一开始直接用最高频率，然后根据 CPU 负载慢慢降低。

③、Powersave，省电模式，通常以最低频率运行，系统性能会受影响，一般不用这个！

④、Userspace，可以在用户空间手动调节频率。

⑤、Ondemand，定时检查负载，然后根据负载来调节频率。负载低的时候降低 CPU 频率，这样省电，负载高的时候提高 CPU 频率，增加性能。

scaling_max_freq: governor(调频)可以调节的最高频率。

cpuinfo_min_freq: governor(调频)可以调节的最低频率。

stats 目录下给出了 CPU 各种运行频率的统计情况，比如 CPU 在各频率下的运行时间以及变频次数。

使用如下命令查看当前 CPU 频率：

```
cat cpuinfo_cur_freq
```

结果如图 37.4.1.4 所示：

```
/sys/devices/system/cpu/cpu0/cpufreq # cat cpuinfo_cur_freq
198000
/sys/devices/system/cpu/cpu0/cpufreq #
```

图 37.4.1.4 当前 CPU 频率

从图 37.4.1.4 可以看出，当前 CPU 频率为 198MHz，工作频率很低！其他的值如下：

```
cpuinfo_cur_freq = 198000
```

```
cpuinfo_max_freq = 792000
```

```
cpuinfo_min_freq = 198000
```

```
scaling_cur_freq = 198000
```

```
scaling_max_freq = 792000
```

```
cat scaling_min_freq = 198000
scaling_available_frequencies = 198000 396000 528000 792000
cat scaling_governor = ondemand
```

可以看出, 当前 CPU 支持 198MHz、396MHz、528Mhz 和 792MHz 四种频率切换, 其中调频策略为 `ondemand`, 也就是定期检查负载, 然后根据负载情况调节 CPU 频率。因为当前我们开发板并没有做什么工作, 因此 CPU 频率降低为 198MHz 以省电。如果开发板做一些高负载的工作, 比如播放视频等操作那么 CPU 频率就会提升上去。查看 `stats` 目录下的 `time_in_state` 文件可以看到 CPU 在各频率下的工作时间, 命令如下:

```
cat /sys/bus/cpu/devices/cpu0/cpufreq/stats/time_in_state
```

结果如图 37.4.1.5 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq/stats # cat time_in_state
198000 57683
396000 292
528000 169
792000 63
/sys/devices/system/cpu/cpu0/cpufreq/stats # █
```

图 37.4.1.5 CPU 运行频率统计

从图 37.4.1.5 中可以看出, CPU 在 198MHz、396MHz、528MHz 和 792MHz 都工作过, 其中 198MHz 的工作时间最长! 假如我们想让 CPU 一直工作在 792MHz 那该怎么办? 很简单, 配置 Linux 内核, 将调频策略选择为 `performance`。或者修改 `imx_alientek_emmc_defconfig` 文件, 此文件中有下面几行:

示例代码 37.4.1.1 调频策略

```
41 CONFIG_CPU_FREQ_DEFAULT_GOV_ONDEMAND=y
42 CONFIG_CPU_FREQ_GOV_POWERSAVE=y
43 CONFIG_CPU_FREQ_GOV_USERSPACE=y
44 CONFIG_CPU_FREQ_GOV_INTERACTIVE=y
```

第 41 行, 配置 `ondemand` 为默认调频策略。

第 42 行, 使能 `powersave` 策略。

第 43 行, 使能 `userspace` 策略。

第 44 行, 使能 `interactive` 策略。

将示例代码 37.4.1.1 中的第 41 行屏蔽掉, 然后在 44 行后面添加:

```
CONFIG_CPU_FREQ_GOV_ONDEMAND=y
```

结果下所示:

示例代码 37.4.1.2 修改调频策略

```
41 #CONFIG_CPU_FREQ_DEFAULT_GOV_ONDEMAND=y
42 CONFIG_CPU_FREQ_GOV_POWERSAVE=y
43 CONFIG_CPU_FREQ_GOV_USERSPACE=y
44 CONFIG_CPU_FREQ_GOV_INTERACTIVE=y
45 CONFIG_CPU_FREQ_GOV_ONDEMAND=y
```

修改完成以后重新编译 Linux 内核, 编译之前先清理一下工程! 因为我们重新修改过默认配置文件了, 编译完成以后使用新的 `zImage` 镜像文件重新启动 Linux。再次查看 `/sys/devices/system/cpu/cpu0/cpufreq/ cpuinfo_cur_freq` 文件的值, 如图 37.4.1.6 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq # cat cpuinfo_cur_freq
792000
/sys/devices/system/cpu/cpu0/cpufreq #
```

图 37.4.1.6 当前 CPU 频率

从图 37.4.1.6 可以看出, 当前 CPU 频率为 792MHz 了。查看 scaling_governor 文件看一下当前的调频策略, 如图 37.4.1.7 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq # cat scaling_governor
performance
/sys/devices/system/cpu/cpu0/cpufreq #
```

图 37.4.1.7 调频策略

从图 37.4.1.7 可以看出当前的 CPU 调频策略为 performance, 也就是高性能模式, 一直以最高主频运行。

我们再来看一下如何通过图形化界面配置 Linux 内核的 CPU 调频策略, 输入 “make menuconfig” 打开 Linux 内核的图形化配置界面, 如图 37.4.1.8 所示:

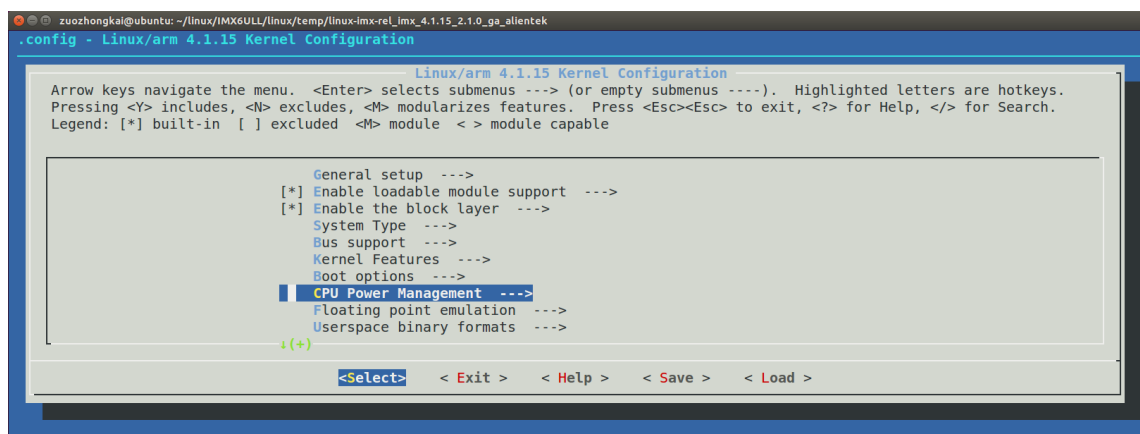


图 37.4.1.8 Linux 内核图形化配置界面

进入如下路径:

```
CPU Power Management
-> CPU Frequency scaling
-> Default CPUFreq governor
```

打开默认调频策略选择界面, 选择 “performance”, 如图 37.1.4.9 所示:

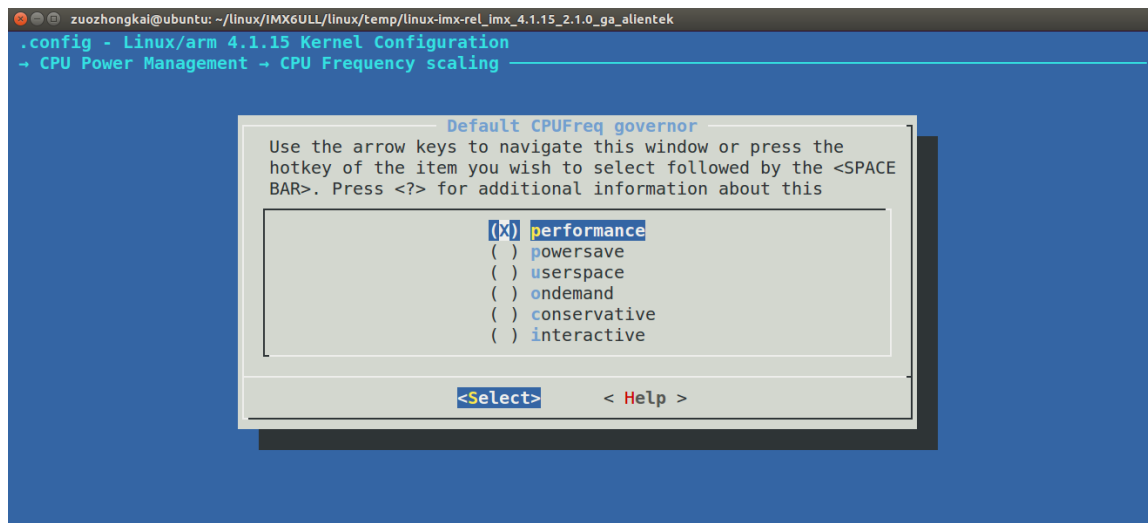


图 37.1.4.9 默认调频策略选择

在图 37.1.4.9 中选择“performance”即可，选择以后退出图形化配置界面，然后编译 Linux 内核，一定不要清理工程！否则的话我们刚刚的设置就会被清理掉。编译完成以后使用新的 zImage 重启 Linux，查看当前 CPU 的工作频率和调频策略。

我们学习的时候为了高性能，大家可以使用 performance 模式。但是在以后的实际产品开发中，从省电的角度考虑，建议大家使用 ondemand 模式，一来可以省电，二来可以减少发热。

2、超频至 700MHz

I.MX6ULL 有多种型号，按照工作频率可以分为 528MHz、700MHz(实际 696MHz)，800MHz(实际 792MHz)和 900MHz(实际频率未知，应该在 900MHz 左右)。有些朋友可能用的其他品牌的开发板，其所使用的 I.MX6ULL 主频可能是 528MHz 的，虽然芯片标称是 528MHz 主频，但是其是可以超频的 700MHz 的(这里的 700MHz 实际上只有 696MHz，但是 NXP 官方宣传其为 700MHz，所以我们就统一称为 700MHz 吧)。

声明：

对于所用的芯片为 528MHz 主频但是想体验一下高性能的朋友体验一下超频，笔者测试过 528MHz 超频到 700MHz，还没有出现过超频不稳定的现象发生，但是！毕竟是超频了的，肯定没有工作在 528MHz 稳定。

如果因为超频带来任何损坏，正点原子不负任何责任！

在实际的产品中，禁止任何超频！务必严格按照 I.MX6ULL 手册上给出的标准工作频率来运行！！如果想要更高的性能，请购买相应型号的处理器的！

看到这里，如果您还是执意要超频，那么就接着往下看，如果要放弃超频，那就跳过本小节，看下一小节。

超频设置其实很简单，修改一下设备树文件 arch/arm/boot/dts/imx6ull.dtsi 即可，打开 imx6ull.dtsi，找到下面代码：

示例代码 37.4.1.3 imx6ull.dtsi 文件代码段

```
54 cpu0: cpu@0 {
55     compatible = "arm,cortex-a7";
56     device_type = "cpu";
57     reg = <0>;
58     clock-latency = <61036>; /* two CLK32 periods */
```



```

59     operating-points = <
60         /* kHz  uV */
61         996000  1275000
62         792000  1225000
63         528000  1175000
64         396000  1025000
65         198000  950000
66     >;
67     fsl,soc-operating-points = <
68         /* KHz  uV */
69         996000  1175000
70         792000  1175000
71         528000  1175000
72         396000  1175000
73         198000  1175000
74 >;

```

示例代码 37.4.1.3 就是设置 CPU 频率的, 第 61~65 行和第 69~73 行就是 I.MX6ULL 所支持的频率, 单位为 KHz, 可以看出 I.MX6ULL(视具体型号而定)支持 996MHz、792MHz、528MHz、396MHz 和 198MHz。在上一小节中, 我们知道 Linux 内核默认支持 198MHz、396MHz、528MHz 和 792MHz, 如果是 MCIMX6Y2CVM05AB 这颗芯片的话, 默认最高只能运行在 528MHz, 我们在示例代码 37.4.2.1 中加入针对 696MHz 的支持, 修改以后代码如下:

示例代码 37.4.1.4 增加 696MHz 的支持

```

54 cpu0: cpu@0 {
55     compatible = "arm,cortex-a7";
56     device_type = "cpu";
57     reg = <0>;
58     clock-latency = <61036>; /* two CLK32 periods */
59     operating-points = <
60         /* kHz  uV */
61         996000  1275000
62         792000  1225000
63         696000  1225000
64         528000  1175000
65         396000  1025000
66         198000  950000
67     >;
68     fsl,soc-operating-points = <
69         /* KHz  uV */
70         996000  1175000
71         792000  1175000
72         696000  1175000
73         528000  1175000
74         396000  1175000

```

```
75          198000 1175000
```

```
76 >;
```

第 63 行, 加入了 “696000 1225000”, 这个就是 696MHz 的支持。

第 72 行, 加入了 “696000 1175000”, 也是对 696MHz 的支持。

修改好以后保存, 并且编译设备树, 在 Linux 内核源码根目录下输入如下命令编译设备树:

```
make dtbs
```

命令 “make dtbs” 只编译设备树文件, 也就是将 .dts 编译为 .dtb, 编译完成以后使用新的设备树文件 imx6ull-alientek_emmc.dtb 启动 Linux。重启以后查看文件 /sys/devices/system/cpu/cpu0/cpufreq/ scaling_available_frequencies 的内容, 如图 37.4.1.10 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq # cat scaling_available_frequencies
198000 396000 528000 696000
/sys/devices/system/cpu/cpu0/cpufreq #
```

图 37.4.1.10 文件 scaling_available_frequencies 内容

从图 37.4.1.11 可以看出, 此时支持了 696MHz。如果设置调频策略为 performance, 那么处理器就会一直工作在 696MHz。可以对比一下工作在 528MHz 和 696MHz 下的 BogoMIPS 的值, 528MHz 主频下的 BogoMIPS 值如图 37.4.1.12 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq # cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 8.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part       : 0xc07
CPU revision   : 5

Hardware      : Freescale i.MX6 Ultralite (Device Tree)
Revision     : 0000
Serial        : 0000000000000000
/sys/devices/system/cpu/cpu0/cpufreq #
```

图 37.4.1.12 528MHz 主频下的 BogoMIPS

696MHz 主频下的 BogoMIPS 值如图 37.4.1.13 所示:

```
/sys/devices/system/cpu/cpu0/cpufreq # cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 10.54
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part       : 0xc07
CPU revision   : 5

Hardware      : Freescale i.MX6 Ultralite (Device Tree)
Revision     : 0000
Serial        : 0000000000000000
```

图 37.4.1.13 696MHz 主频下的 BogoMIPS

从图 37.4.1.12 和图 37.2.1.13 中可以看到, 528MHz 和 696MHz 下的 BogoMIPS 值分别为 8.00 和 10.54, 相当于性能提升了 $(10.54/8)-1=31.75\%$ 。

37.4.2 修改 EMMC 驱动

1、使能 8 线 EMMC 驱动

正点原子 EMMC 版本核心板上的 EMMC 采用的 8 位数据线, 原理图如图 37.4.2.1 所示:

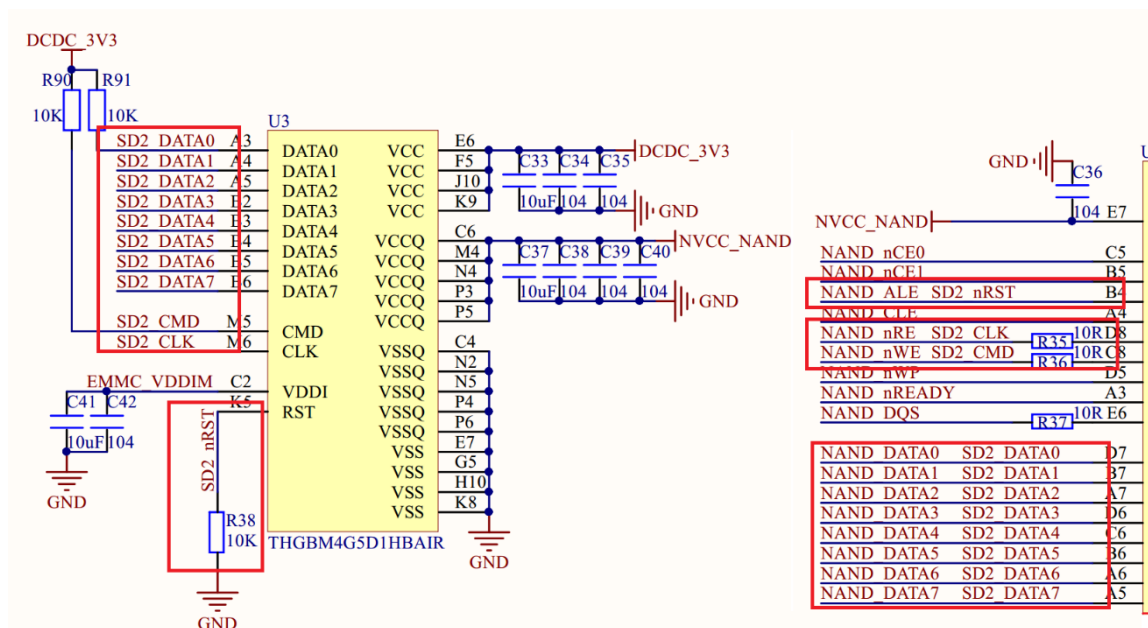


图 37.4.2.1 EMMC 原理图

Linux 内核驱动里面 EMMC 默认是 4 线模式的，4 线模式肯定没有 8 线模式的速度快，所以本节我们将 EMMC 的驱动修改为 8 线模式。修改方法很简单，直接修改设备树即可，打开文件 `imx6ull-alientek-emmc.dts`，找到如下所示内容：

示例代码 37.4.2.1 `imx6ull-alientek-emmc.dts` 代码段

```
734 &usdhc2 {
735     pinctrl-names = "default";
736     pinctrl-0 = <&pinctrl_usdhc2>;
737     non-removable;
738     status = "okay";
739 };
```

关于设备树的原理以及内容我们后面会有专门的章节讲解，示例代码 37.4.2.1 中的代码含义我们现在不去纠结，只需要将其改为如下代码即可：

示例代码 37.4.2.1 `imx6ull-alientek-emmc.dts` 代码段

```
734 &usdhc2 {
735     pinctrl-names = "default", "state_100mhz", "state_200mhz";
736     pinctrl-0 = <&pinctrl_usdhc2_8bit>;
737     pinctrl-1 = <&pinctrl_usdhc2_8bit_100mhz>;
738     pinctrl-2 = <&pinctrl_usdhc2_8bit_200mhz>;
739     bus-width = <8>;
740     non-removable;
741     status = "okay";
742 };
```

2、关闭 EMMC 1.8V 供电选项

从图 37.4.2.1 中可以看出，此时 EMMC 工作电压是 3.3V 的，因此我们要在示例代码 37.4.2.1 中的 `usdhc2` 设备树节点中添加 “no-1-8-v” 选项，也就是关闭 1.8V 这个功能选项。防止内核在运行的时候用 1.8V 去驱动 EMMC，导致 EMMC 驱动出现问题，修改后的 `usdhc2` 节点内容如

下:

示例代码 37.4.2.2 最终的 usdhc2 节点代码段

```

734 &usdhc2 {
735     pinctrl-names = "default", "state_100mhz", "state_200mhz";
736     pinctrl-0 = <&pinctrl_usdhc2_8bit>;
737     pinctrl-1 = <&pinctrl_usdhc2_8bit_100mhz>;
738     pinctrl-2 = <&pinctrl_usdhc2_8bit_200mhz>;
739     bus-width = <8>;
740     non-removable;
741     no-1-8-v;
742     status = "okay";
743 };

```

第 741 行就是加入的“no-1-8-v”选项。

修改完成以后保存一下 imx6ull-alientek-emmc.dts, 然后使用命令“make dtbs”重新编译一下设备树, 编译完成以后使用新的设备树重启 Linux 系统即可。

37.4.3 V2.4 版本以前底板网络驱动修改

注意!正点原子 I.MX6U-ALPHA 开发板 V2.4 版本以前的底板使用的网络 PHY 为 LAN8720, V2.4 及其以后的版本使用的网络 PHY 为 SR8201F, 而且网络 PHY 地址有改变, 大家一定要看准自己所使用的底板版本号!

如果用的是 Mini 板, 那么 V2.2 及以后版本 Mini 开发板请参考 V2.4 版本的 ALPHA 开发板移植。

查看自己所使用的 ALPHA 底板版本号, 如果是 V2.4 以前版本, 那么就看本结!

因为在后面学习 Linux 驱动开发的时候要用到网络调试驱动, 所以必须要把网络驱动调试好。在讲解 uboot 移植的时候就已经说过了, 正点原子开发板的网络和 NXP 官方的网络硬件上不同, 网络 PHY 芯片由 KSZ8081 换为了 LAN8720A, 两个网络 PHY 芯片的复位 IO 也不同。所以 Linux 内核自带的网络驱动是驱动不起来 I.MX6U-ALPHA 开发板上的网络的, 需要做修改。

1、修改 LAN8720 的复位以及网络时钟引脚驱动

ENET1 复位引脚 ENET1_RST 连接在 I.M6ULL 的 SNVS_TAMPER7 这个引脚上。ENET2 的复位引脚 ENET2_RST 连接在 I.MX6ULL 的 SNVS_TAMPER8 上。打开设备树文件 imx6ull-alientek-emmc.dts, 找到如下代码:

示例代码 37.4.3.1 imx6ull-alientek-emmc.dts 代码段

```

584 pinctrl_spi4: spi4grp {
585     fsl,pins = <
586         MX6ULL_PAD_BOOT_MODE0__GPIO5_IO10      0x70a1
587         MX6ULL_PAD_BOOT_MODE1__GPIO5_IO11      0x70a1
588         MX6ULL_PAD_SNVS_TAMPER7__GPIO5_IO07     0x70a1
589         MX6ULL_PAD_SNVS_TAMPER8__GPIO5_IO08     0x80000000
590     >;
591 };

```

示例代码 37.4.3.1 中第 588 和 589 行就是初始化 SNVS_TAMPER7 和 SNVS_TAMPER8 这两个

引脚的, 不过看样子好像是作为 SPI4 的 IO, 这不是我们想要的, 所以将 588 和 589 这两行删除掉! 删除掉以后继续在 imx6ull-alientek-emmc.dts 中找到如下所示代码:

示例代码 37.4.3.2 imx6ull-alientek-emmc.dts 代码段

```
125 spi4 {
126     compatible = "spi-gpio";
127     pinctrl-names = "default";
128     pinctrl-0 = <&pinctrl_spi4>;
129     pinctrl-assert-gpios = <&gpio5 8 GPIO_ACTIVE_LOW>;
.....
133     cs-gpios = <&gpio5 7 0>;
```

第 129 行, 设置 GPIO5_IO08 为 SPI4 的一个功能引脚(我也不清楚具体作为什么功能用), 而 GPIO5_IO08 就是 SNVS_TAMPER8 的 GPIO 功能引脚。

第 133 行, 设置 GPIO5_IO07 作为 SPI4 的片选引脚, 而 GPIO5_IO07 就是 SNVS_TAMPER7 的 GPIO 功能引脚。

现在我们需要 GPIO5_IO07 和 GPIO5_IO08 分别作为 ENET1 和 ENET2 的复位引脚, 而不是 SPI4 的什么功能引脚, 因此将示例代码 37.4.3.2 中的第 129 行和第 133 行处的代码删除掉!! 否则会干扰到网络复位引脚!

在 imx6ull-alientek-emmc.dts 里面找到名为 “iomuxc_snvs” 的节点(就是直接搜索), 然后在此节点下添加网络复位引脚信息, 添加完成以后的 “iomuxc_snvs” 的节点内容如下:

示例代码 37.4.3.3 iomuxc_snvs 节点添加网络复位信息

```
1 &iomuxc_snvs {
2     pinctrl-names = "default_snvs";
3     pinctrl-0 = <&pinctrl_hog_2>;
4     imx6ul-evk {
5
.....        /*省略掉其他*/
43
44        /*enet1 reset zuozhongkai*/
45        pinctrl_enet1_reset: enet1resetgrp {
46            fsl,pins = <
47                /* used for enet1 reset */
48                MX6ULL_PAD_SNVS_TAMPER7__GPIO5_IO07    0x10B0
49            >;
50        };
51
52        /*enet2 reset zuozhongkai*/
53        pinctrl_enet2_reset: enet2resetgrp {
54            fsl,pins = <
55                /* used for enet2 reset */
56                MX6ULL_PAD_SNVS_TAMPER8__GPIO5_IO08    0x10B0
57            >;
58        };
59    };
```

```
60 };
```

第 1 行, imx6ull-alientek-emmc.dts 文件中 iomuxc_snvs 节点。

第 45~50 行, ENET1 网络复位引脚配置信息。

第 53~58 行, ENET2 网络复位引脚配置信息。

最后还需要修改一下 ENET1 和 ENET2 的网络时钟引脚配置, 继续在 imx6ull-alientek-emmc.dts 中找到如下所示代码:

示例代码 37.4.3.4 imx6ull-alientek-emmc.dts 代码段

```
309 pinctrl_enet1: enet1grp {
310     fsl,pins = <
311         MX6UL_PAD_ENET1_RX_EN__ENET1_RX_EN        0x1b0b0
312         MX6UL_PAD_ENET1_RX_ER__ENET1_RX_ER        0x1b0b0
313         MX6UL_PAD_ENET1_RX_DATA0__ENET1_RDATA00    0x1b0b0
314         MX6UL_PAD_ENET1_RX_DATA1__ENET1_RDATA01    0x1b0b0
315         MX6UL_PAD_ENET1_TX_EN__ENET1_TX_EN        0x1b0b0
316         MX6UL_PAD_ENET1_TX_DATA0__ENET1_TDATA00    0x1b0b0
317         MX6UL_PAD_ENET1_TX_DATA1__ENET1_TDATA01    0x1b0b0
318         MX6UL_PAD_ENET1_TX_CLK__ENET1_REF_CLK1      0x4001b009
319     >;
320 };
321
322 pinctrl_enet2: enet2grp {
323     fsl,pins = <
324         MX6UL_PAD_GPIO1_IO07__ENET2_MDC            0x1b0b0
325         MX6UL_PAD_GPIO1_IO06__ENET2_MDIO           0x1b0b0
326         MX6UL_PAD_ENET2_RX_EN__ENET2_RX_EN        0x1b0b0
327         MX6UL_PAD_ENET2_RX_ER__ENET2_RX_ER        0x1b0b0
328         MX6UL_PAD_ENET2_RX_DATA0__ENET2_RDATA00    0x1b0b0
329         MX6UL_PAD_ENET2_RX_DATA1__ENET2_RDATA01    0x1b0b0
330         MX6UL_PAD_ENET2_TX_EN__ENET2_TX_EN        0x1b0b0
331         MX6UL_PAD_ENET2_TX_DATA0__ENET2_TDATA00    0x1b0b0
332         MX6UL_PAD_ENET2_TX_DATA1__ENET2_TDATA01    0x1b0b0
333         MX6UL_PAD_ENET2_TX_CLK__ENET2_REF_CLK2      0x4001b009
334     >;
335 };
```

第 318 和 333 行, 分别为 ENET1 和 ENET2 的网络时钟引脚配置信息, 将这两个引脚的电气属性值改为 0x4001b009, 原来默认值为 0x4001b031。

修改完成以后记得保存一下 imx6ull-alientek-emmc.dts, 网络复位以及时钟引脚驱动就修改好了。

2、修改 fec1 和 fec2 节点的 pinctrl-0 属性

在 imx6ull-alientek-emmc.dts 文件中找到名为“fec1”和“fec2”的这两个节点, 修改其中的“pinctrl-0”属性值, 修改以后如下所示:

示例代码 37.4.3.5 修改 fec1 和 fec2 的 pinctrl-0 属性

```
1 &fec1 {
```

```

2    pinctrl-names = "default";
3    pinctrl-0 = <&pinctrl_enet1
4                &pinctrl_enet1_reset>;
5    phy-mode = "rmii";
6    .....
9    status = "okay";
10 };
11
12 &fec2 {
13    pinctrl-names = "default";
14    pinctrl-0 = <&pinctrl_enet2
15                &pinctrl_enet2_reset>;
16    phy-mode = "rmii";
17    .....
36 };

```

第 3~4 行，修改后的 fec1 节点 “pinctrl-0” 属性值。

第 14~15 行，修改后的 fec2 节点 “pinctrl-0” 属性值。

3、修改 LAN8720A 的 PHY 地址

在 uboot 移植章节中，我们说过 ENET1 的 LAN8720A 地址为 0x0，ENET2 的 LAN8720A 地址为 0x1。在 imx6ull-alientek-emmc.dts 中找到如下代码：

示例代码 37.4.3.6 imx6ull-alientek-emmc.dts 代码段

```

171 &fec1 {
172    pinctrl-names = "default";
173    .....
175    phy-handle = <&ethphy0>;
176    status = "okay";
177 };
178
179 &fec2 {
180    pinctrl-names = "default";
181    .....
183    phy-handle = <&ethphy1>;
184    status = "okay";
185
186    mdio {
187        #address-cells = <1>;
188        #size-cells = <0>;
189
190        ethphy0: ethernet-phy@0 {
191            compatible = "ethernet-phy-ieee802.3-c22";
192            reg = <2>;
193        };
194

```



```

195     ethphy1: ethernet-phy@1 {
196         compatible = "ethernet-phy-ieee802.3-c22";
197         reg = <1>;
198     };
199 };
200 };

```

第 171~177 行, ENET1 对应的设备树节点。

第 179~200 行, ENET2 对应的设备树节点。但是第 186~198 行的 mdio 节点描述了 ENET1 和 ENET2 的 PHY 地址信息。将示例代码 37.4.3.6 改为如下内容:

示例代码 37.4.3.7 imx6ull-alientek-emmc.dts 代码段

```

171 &fec1 {
172     pinctrl-names = "default";
173     pinctrl-0 = <&pinctrl_enet1
174         &pinctrl_enet1_reset>;
175     phy-mode = "rmii";
176     phy-handle = <&ethphy0>;
177     phy-reset-gpios = <&gpio5 7 GPIO_ACTIVE_LOW>;
178     phy-reset-duration = <200>;
179     status = "okay";
180 };
181
182 &fec2 {
183     pinctrl-names = "default";
184     pinctrl-0 = <&pinctrl_enet2
185         &pinctrl_enet2_reset>;
186     phy-mode = "rmii";
187     phy-handle = <&ethphy1>;
188     phy-reset-gpios = <&gpio5 8 GPIO_ACTIVE_LOW>;
189     phy-reset-duration = <200>;
190     status = "okay";
191
192     mdio {
193         #address-cells = <1>;
194         #size-cells = <0>;
195
196         ethphy0: ethernet-phy@0 {
197             compatible = "ethernet-phy-ieee802.3-c22";
198             smsc,disable-energy-detect;
199             reg = <0>;
200         };
201
202         ethphy1: ethernet-phy@1 {
203             compatible = "ethernet-phy-ieee802.3-c22";

```

```

204         smsc,disable-energy-detect;
205         reg = <1>;
206     };
207 };
208 };

```

第 177 和 178 行, 添加了 ENET1 网络复位引脚所使用的 IO 为 GPIO5_IO07, 低电平有效。复位低电平信号持续时间为 200ms。

第 188 和 189 行, ENET2 网络复位引脚所使用的 IO 为 GPIO5_IO08, 同样低电平有效, 持续时间同样为 200ms。

第 198 和 204 行, “smc,disable-energy-detect” 表明 PHY 芯片是 SMSC 公司的, 这样 Linux 内核就会找到 SMSC 公司的 PHY 芯片驱动来驱动 LAN8720A。

第 196 行, 注意 “ethernet-phy@” 后面的数字是 PHY 的地址, ENET1 的 PHY 地址为 0, 所以 “@” 后面是 0(默认为 2)。

第 199 行, reg 的值也表示 PHY 地址, ENET1 的 PHY 地址为 0, 所以 reg=0。

第 202 行, ENET2 的 PHY 地址为 1, 因此 “@” 后面为 1。

第 205 行, 因为 ENET2 的 PHY 地址为 1, 所以 reg=1。

至此, LAN8720A 的 PHY 地址就改好了, 保存一下 imx6ull-alientek-emmc.dts 文件。然后使用 “make dtbs” 命令重新编译一下设备树。

3、修改 fec_main.c 文件

要在 I.MX6ULL 上使用 LAN8720A, 需要修改一下 Linux 内核源码, 打开 drivers/net/ethernet/freescale/fec_main.c, 找到函数 fec_probe, 在 fec_probe 中加入如下代码:

示例代码 37.4.3.8 fec_probe 函数代码段

```

3438 static int
3439 fec_probe(struct platform_device *pdev)
3440 {
3441     struct fec_enet_private *fep;
3442     struct fec_platform_data *pdata;
3443     struct net_device *ndev;
3444     int i, irq, ret = 0;
3445     struct resource *r;
3446     const struct of_device_id *of_id;
3447     static int dev_id;
3448     struct device_node *np = pdev->dev.of_node, *phy_node;
3449     int num_tx_qs;
3450     int num_rx_qs;
3451
3452     /* 设置 MX6UL_PAD_ENET1_TX_CLK 和 MX6UL_PAD_ENET2_TX_CLK
3453      * 这两个 IO 的复用寄存器的 SION 位为 1。
3454      */
3455     void __iomem *IMX6U_ENET1_TX_CLK;
3456     void __iomem *IMX6U_ENET2_TX_CLK;
3457
3458     IMX6U_ENET1_TX_CLK = ioremap(0X020E00DC, 4);

```

```

3459     writel(0X14, IMX6U_ENET1_TX_CLK);
3460
3461     IMX6U_ENET2_TX_CLK = ioremap(0X020E00FC, 4);
3462     writel(0X14, IMX6U_ENET2_TX_CLK);
3463
3464     .....
3656     return ret;
3657 }
    
```

第 3455~3462 就是新加入的代码，如果要在 IMX6ULL 上使用 LAN8720A 就需要设置 ENET1 和 ENET2 的 TX_CLK 引脚复位寄存器的 SION 位为 1。

4、配置 Linux 内核，使能 LAN8720 驱动

输入命令“make menuconfig”，打开图形化配置界面，选择使能 LAN8720A 的驱动，路径如下：

```

-> Device Drivers
    -> Network device support
        -> PHY Device support and infrastructure
            -> Drivers for SMSC PHYs
    
```

如图 37.4.3.1 所示：

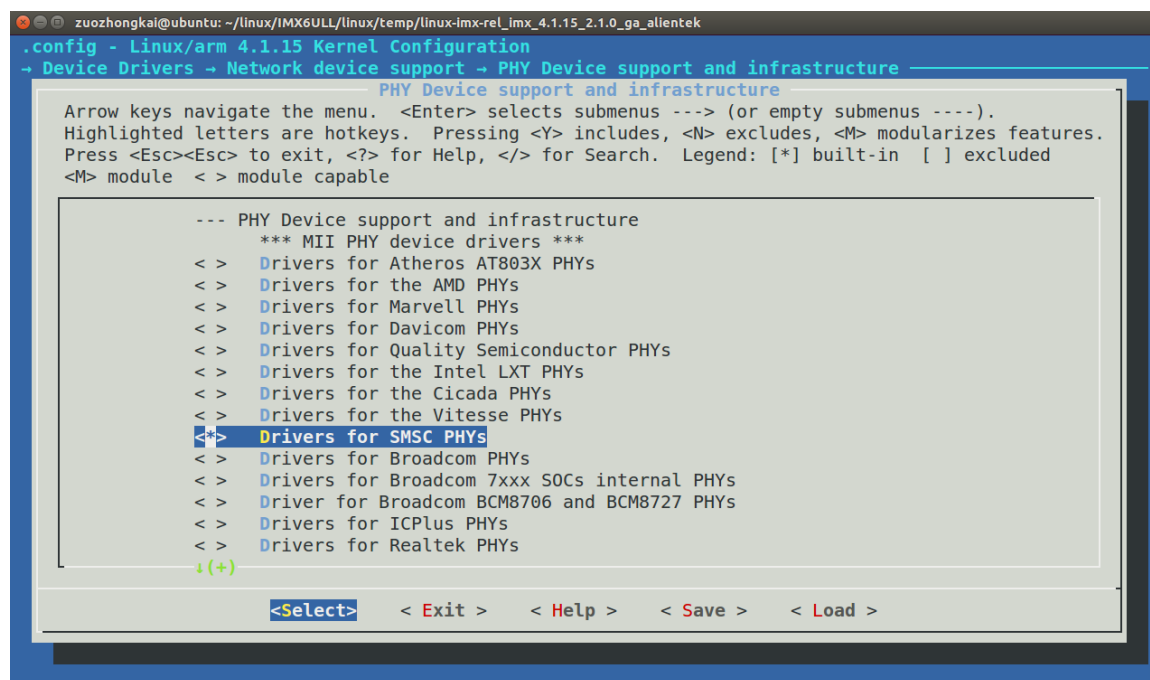


图 37.4.3.1 使能 LAN8720A 驱动

图 37.4.3.1 中选择将“Drivers for SMSC PHYs”编译到 Linux 内核中，因此“<>”里面变为了“*”。LAN8720A 是 SMSC 公司出品的，因此勾选这个以后就会编译 LAN8720 驱动，配置好以后退出配置界面，然后重新编译一下 Linux 内核。

5、修改 smsc.c 文件

在修改 smsc.c 文件之前先说点题外话，那就是我是怎么确定要修改 smsc.c 这个文件的。在写本书之前我并没有修改过 smsc.c 这个文件，都是使能 LAN8720A 驱动以后就直接使用。但是

我在测试 NFS 挂载文件系统的时候发现文件系统挂载成功率很低！老是提示 NFS 服务器找不到，三四次就有一次挂载失败！很折磨人。NFS 挂载就是通过网络来挂载文件系统，这样做的好处就是方便我们后续调试 Linux 驱动。既然老是挂载失败那么可以肯定的是网络驱动有问题，网络驱动分两部分：内部 MAC+外部 PHY，内部 MAC 驱动是由 NXP 提供的，一般不会出问题，否则的话用户早就给 NXP 反馈了。而且我用 NXP 官方的开发板测试网络是一直正常的，但是 NXP 官方的开发板所使用的 PHY 芯片为 KSZ8081。所以只有可能是外部 PHY，也就是 LAN8720A 的驱动可能出问题了。鉴于 LAN8720A 有“前车之鉴”，那就是在 uboot 中需要对 LAN8720A 进行一次软复位，要设置 LAN8720A 的 BMCR(寄存器地址为 0)寄存器 bit15 为 1。所以我猜测，在 Linux 中也需要对 LAN8720A 进行一次软复位。

首先需要找到 LAN8720A 的驱动文件，LAN8720A 的驱动文件是 drivers/net/phy/smsc.c，在此文件中有个叫做 smsc_phy_reset 的函数，看名字都知道这是 SMSC PHY 的复位函数，因此，LAN8720A 肯定也会使用到这个复位函数，修改此函数的内容，修改以后的 smsc_phy_reset 函数内容如下所示：

示例代码 37.4.3.9 smsc_phy_reset 函数

```
1 static int smsc_phy_reset(struct phy_device *phydev)
2 {
3     int err, phy_reset;
4     int msec = 1;
5     struct device_node *np;
6     int timeout = 50000;
7     if(phydev->addr == 0) /* FEC1 */ {
8         np = of_find_node_by_path("/soc/aips-bus@02100000/ethernet@
9                                     02188000");
10        if(np == NULL) {
11            return -EINVAL;
12        }
13    }
14    if(phydev->addr == 1) /* FEC2 */ {
15        np = of_find_node_by_path("/soc/aips-bus@02000000/ethernet@
16                                    020b4000");
17        if(np == NULL) {
18            return -EINVAL;
19        }
20    }
21    err = of_property_read_u32(np, "phy-reset-duration", &msec);
22    /* A sane reset duration should not be longer than 1s */
23    if (!err && msec > 1000)
24        msec = 1;
25    phy_reset = of_get_named_gpio(np, "phy-reset-gpios", 0);
26    if (!gpio_is_valid(phy_reset))
27        return;
```

```
28
29     gpio_direction_output(phy_reset, 0);
30     gpio_set_value(phy_reset, 0);
31     msleep(msec);
32     gpio_set_value(phy_reset, 1);
33
34     int rc = phy_read(phydev, MII_LAN83C185_SPECIAL_MODES);
35     if (rc < 0)
36         return rc;
37
38     /* If the SMSC PHY is in power down mode, then set it
39      * in all capable mode before using it.
40      */
41     if ((rc & MII_LAN83C185_MODE_MASK) ==
42         MII_LAN83C185_MODE_POWERDOWN) {
43
44         /* set "all capable" mode and reset the phy */
45         rc |= MII_LAN83C185_MODE_ALL;
46         phy_write(phydev, MII_LAN83C185_SPECIAL_MODES, rc);
47     }
48
49     phy_write(phydev, MII_BMCR, BMCR_RESET);
50     /* wait end of reset (max 500 ms) */
51     do {
52         udelay(10);
53         if (timeout-- == 0)
54             return -1;
55         rc = phy_read(phydev, MII_BMCR);
56     } while (rc & BMCR_RESET);
57     return 0;
58 }
```

第 7~12 行, 获取 FEC1 网卡对应的设备节点。

第 14~19 行, 获取 FEC2 网卡对应的设备节点。

第 21 行, 从设备树中获取 “phy-reset-duration” 属性信息, 也就是复位时间。

第 25 行, 从设备树中获取 “phy-reset-gpios” 属性信息, 也就是复位 IO。

第 29~32 行, 设置 PHY 的复位 IO, 复位 LAN8720A。

第 41~48 行, 以前的 `smc_phy_reset` 函数会判断 LAN8720 是否处于 Powerdown 模式, 只有处于 Powerdown 模式的时候才会软复位 LAN8720。这里我们将软复位代码移出来, 这样每次调用 `smc_phy_reset` 函数 LAN8720A 都会被软复位。

最后我们还需要在 `drivers/net/phy/smc.c` 文件中添加两个头文件, 因为修改后的 `smc_phy_reset` 函数用到了 `gpio_direction_output` 和 `gpio_set_value` 这两个函数, 需要添加的头文件如下所示:

```
#include <linux/of_gpio.h>
#include <linux/io.h>
```

6、网络驱动测试

修改好设备树和 Linux 内核以后重新编译一下，得到新的 zImage 镜像文件和 imx6ull-alientek-emmc.dtb 设备树文件，使用网线将 IMX6U-ALPHA 开发板的两个网口与路由器或者电脑连接起来，最后使用新的文件启动 Linux 内核。启动以后使用“ifconfig”命令查看一下当前活动的网卡有哪些，结果如图 37.4.3.2 所示：

```
Please press Enter to activate this console.
/ # ifconfig
/ #
```

图 37.4.3.2 ifconfig 命令结果

从图 37.4.3.2 可以看出，当前没有活动的网卡。输入命令“ifconfig -a”来查看一下开发板中存在的所有网卡，结果如图 37.4.3.3 所示：

```
/ # ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:27

can1      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:28

eth0      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

eth1      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          LOOPBACK  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

sit0      Link encap:IPv6-in-IPv4
          NOARP  MTU:1480  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 37.4.3.3 开发板所有网卡

图 37.4.3.3 中 can0 和 can1 为 CAN 接口的网卡，eth0 和 eth1 才是网络接口的网卡，其中 eth0 对应于 ENET2，eth1 对应于 ENET1。使用如下命令依次打开 eth0 和 eth1 这两个网卡：

```
ifconfig eth0 up
ifconfig eth1 up
```

网卡的打开过程如图 37.4.3.4 所示：


```

/ # ifconfig eth0 up
fec 20b4000.ethernet eth0: Freescale FEC PHY driver [SMSC LAN8710/LAN8720] (mii_bus:phy_addr=20b4000.ethernet:01, irq=-1)
/ # fec 20b4000.ethernet eth0: Link is Up - 100Mbps/Full - flow control rx/tx

/ # ifconfig eth1 up
fec 2188000.ethernet eth1: Freescale FEC PHY driver [SMSC LAN8710/LAN8720] (mii_bus:phy_addr=20b4000.ethernet:00, irq=-1)
IPv6: ADDRCONF(NETDEV_UP): eth1: link is not ready
/ # fec 2188000.ethernet eth1: Link is Up - 100Mbps/Full - flow control rx/tx
IPv6: ADDRCONF(NETDEV_CHANGE): eth1: link becomes ready
IPv6: eth1: IPv6 duplicate address fe80::204:9fff:fe04:d235 detected!

/ #

```

图 37.4.3.4 两个网卡打开过程

从图 37.4.3.4 中可以看到“SMSC LAN8710/LAN8720”字样，说明当前的网络驱动使用的就是我们前面使能的 SMSC 驱动。

再次输入“ifconfig”命令来查看一下当前活动的网卡，结果如图 37.4.3.5 所示：

```

/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          inet6 addr: fe80::204:9fff:fe04:d235/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1643 errors:0 dropped:19 overruns:0 frame:0
          TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:110575 (107.9 KiB)  TX bytes:992 (992.0 B)

eth1      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          inet6 addr: fe80::204:9fff:fe04:d235/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1394 errors:0 dropped:18 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:94453 (92.2 KiB)  TX bytes:258 (258.0 B)

```

图 37.4.3.5 当前活动的网卡。

可以看出，此时 eth0 和 eth1 两个网卡都已经打开，并且工作正常，但是这两个网卡都还没有 IP 地址，所以不能进行 ping 等操作。使用如下命令给两个网卡配置 IP 地址：

```

ifconfig eth0 192.168.1.251
ifconfig eth1 192.168.1.252

```

上述命令配置 eth0 和 eth1 这两个网卡的 IP 地址分别为 192.168.1.251 和 192.168.1.252，注意 IP 地址选择的合理性，一定要和自己的电脑处于同一个网段内，并且没有被其他的设备占用！设置好以后，使用“ping”命令来 ping 一下自己的主机，如果能 ping 通那说明网络驱动修改成功！比如我的 Ubuntu 主机 IP 地址为 192.168.1.250，使用如下命令 ping 一下：

```
ping 192.168.1.250
```

结果如图 37.4.3.6 所示：

```

/ # ping 192.168.1.250
PING 192.168.1.250 (192.168.1.250): 56 data bytes
64 bytes from 192.168.1.250: seq=0 ttl=64 time=0.906 ms
64 bytes from 192.168.1.250: seq=1 ttl=64 time=1.016 ms
64 bytes from 192.168.1.250: seq=2 ttl=64 time=0.893 ms
^C
--- 192.168.1.250 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.893/0.938/1.016 ms
/ #

```

图 37.4.3.6 ping 结果

可以看出，ping 成功，说明网络驱动修改成功！我们在后面的构建根文件系统和 Linux 驱动开发中就可以使用网络调试代码啦。

37.4.4 V2.4 及以后版本底板网络驱动修改

查看自己所使用的 ALPHA 底板版本号, V2.4 及以后版本的底板网络芯片更换为 SR8201F, 并且网络 PHY 地址也有改变。因此如果您使用的底板是 V2.4 及以后的版本, 请看本小节!

因为在后面学习 Linux 驱动开发的时候要用到网络调试驱动, 所以必须要把网络驱动调试好。在讲解 uboot 移植的时候就已经说过了, 正点原子开发板的网络和 NXP 官方的网络硬件上不同, 网络 PHY 芯片由 KSZ8081 换为了 SR8201F, 两个网络 PHY 芯片的复位 IO 也不同。所以 Linux 内核自带的网络驱动是驱动不起来 I.MX6U-ALPHA 开发板上的网络的, 需要做修改。

1、修改 SR8201F 的复位以及网络时钟引脚驱动

ENET1 复位引脚 ENET1_RST 连接在 I.M6ULL 的 SNVS_TAMPER7 这个引脚上。ENET2 的复位引脚 ENET2_RST 连接在 I.MX6ULL 的 SNVS_TAMPER8 上。打开设备树文件 imx6ull-alientek-emmc.dts, 找到如下代码:

示例代码 37.4.4.1 imx6ull-alientek-emmc.dts 代码段

```
584 pinctrl_spi4: spi4grp {
585     fsl,pins = <
586         MX6ULL_PAD_BOOT_MODE0__GPIO5_IO10    0x70a1
587         MX6ULL_PAD_BOOT_MODE1__GPIO5_IO11    0x70a1
588         MX6ULL_PAD_SNVS_TAMPER7__GPIO5_IO07   0x70a1
589         MX6ULL_PAD_SNVS_TAMPER8__GPIO5_IO08   0x80000000
590     >;
591 };
```

示例代码 37.4.4.1 中第 588 和 589 行就是初始化 SNVS_TAMPER7 和 SNVS_TAMPER8 这两个引脚的, 不过看样子好像是作为了 SPI4 的 IO, 这不是我们想要的, 所以将 588 和 589 这两行删除掉! 删除掉以后继续在 imx6ull-alientek-emmc.dts 中找到如下所示代码:

示例代码 37.4.4.2 imx6ull-alientek-emmc.dts 代码段

```
125 spi4 {
126     compatible = "spi-gpio";
127     pinctrl-names = "default";
128     pinctrl-0 = <&pinctrl_spi4>;
129     pinctrl-assert-gpios = <&gpio5 8 GPIO_ACTIVE_LOW>;
.....
133     cs-gpios = <&gpio5 7 0>;
```

第 129 行, 设置 GPIO5_IO08 为 SPI4 的一个功能引脚(我也不清楚具体作为什么功能用), 而 GPIO5_IO08 就是 SNVS_TAMPER8 的 GPIO 功能引脚。

第 133 行, 设置 GPIO5_IO07 作为 SPI4 的片选引脚, 而 GPIO5_IO07 就是 SNVS_TAMPER7 的 GPIO 功能引脚。

现在我们需要 GPIO5_IO07 和 GPIO5_IO08 分别作为 ENET1 和 ENET2 的复位引脚, 而不是 SPI4 的什么功能引脚, 因此将示例代码 37.4.4.2 中的第 129 行和第 133 行处的代码删除掉!! 否则会干扰到网络复位引脚!

在 imx6ull-alientek-emmc.dts 里面找到名为 “iomuxc_snvs” 的节点(就是直接搜索), 然后在此节点下添加网络复位引脚信息, 添加完成以后的 “iomuxc_snvs” 的节点内容如下:

示例代码 37.4.4.3 iomuxc_snvs 节点添加网络复位信息

```
1 &iomuxc_snvs {
```

```

2  pinctrl-names = "default_snvs";
3      pinctrl-0 = <&pinctrl_hog_2>;
4      imx6ul-evk {
5
6          .....          /*省略掉其他*/
7
8          .....          /*enet1 reset zuozhongkai*/
9
10         pinctrl_enet1_reset: enet1resetgrp {
11             fsl,pins = <
12                 .....          /* used for enet1 reset */
13                 MX6ULL_PAD_SNVS_TAMPER7__GPIO5_IO07      0x10B0
14             >;
15         };
16
17         .....          /*enet2 reset zuozhongkai*/
18
19         pinctrl_enet2_reset: enet2resetgrp {
20             fsl,pins = <
21                 .....          /* used for enet2 reset */
22                 MX6ULL_PAD_SNVS_TAMPER8__GPIO5_IO08      0x10B0
23             >;
24         };
25     };
26 };

```

第 1 行, imx6ull-alientek-emmc.dts 文件中 iomuxc_snvs 节点。

第 45~50 行, ENET1 网络复位引脚配置信息。

第 53~58 行, ENET2 网络复位引脚配置信息。

最后还需要修改一下 ENET1 和 ENET2 的网络时钟引脚配置, 继续在 imx6ull-alientek-emmc.dts 中找到如下所示代码:

示例代码 37.4.4.4 imx6ull-alientek-emmc.dts 代码段

```

309 pinctrl_enet1: enet1grp {
310     fsl,pins = <
311         MX6UL_PAD_ENET1_RX_EN__ENET1_RX_EN      0x1b0b0
312         MX6UL_PAD_ENET1_RX_ER__ENET1_RX_ER      0x1b0b0
313         MX6UL_PAD_ENET1_RX_DATA0__ENET1_RDATA00  0x1b0b0
314         MX6UL_PAD_ENET1_RX_DATA1__ENET1_RDATA01  0x1b0b0
315         MX6UL_PAD_ENET1_TX_EN__ENET1_TX_EN      0x1b0b0
316         MX6UL_PAD_ENET1_TX_DATA0__ENET1_TDATA00  0x1b0b0
317         MX6UL_PAD_ENET1_TX_DATA1__ENET1_TDATA01  0x1b0b0
318         MX6UL_PAD_ENET1_TX_CLK__ENET1_REF_CLK1   0x4001b031
319     >;
320 };
321
322 pinctrl_enet2: enet2grp {

```

```

323     fsl,pins = <
324         MX6UL_PAD_GPIO1_IO07__ENET2_MDC           0x1b0b0
325         MX6UL_PAD_GPIO1_IO06__ENET2_MDIO           0x1b0b0
326         MX6UL_PAD_ENET2_RX_EN__ENET2_RX_EN         0x1b0b0
327         MX6UL_PAD_ENET2_RX_ER__ENET2_RX_ER         0x1b0b0
328         MX6UL_PAD_ENET2_RX_DATA0__ENET2_RDATA00    0x1b0b0
329         MX6UL_PAD_ENET2_RX_DATA1__ENET2_RDATA01    0x1b0b0
330         MX6UL_PAD_ENET2_TX_EN__ENET2_TX_EN         0x1b0b0
331         MX6UL_PAD_ENET2_TX_DATA0__ENET2_TDATA00    0x1b0b0
332         MX6UL_PAD_ENET2_TX_DATA1__ENET2_TDATA01    0x1b0b0
333         MX6UL_PAD_ENET2_TX_CLK__ENET2_REF_CLK2     0x4001b031
334     >;
335 };

```

第 318 和 333 行, 分别为 ENET1 和 ENET2 的网络时钟引脚配置信息, 将这两个引脚的电气属性值改为 0x4001b031(原来默认值就是 0x4001b031)。

修改完成以后记得保存一下 imx6ull-alientek-emmc.dts, 网络复位以及时钟引脚驱动就修改好了。

2、修改 fec1 和 fec2 节点的 pinctrl-0 属性

在 imx6ull-alientek-emmc.dts 文件中找到名为“fec1”和“fec2”的这两个节点, 修改其中的“pinctrl-0”属性值, 修改以后如下所示:

示例代码 37.4.4.5 修改 fec1 和 fec2 的 pinctrl-0 属性

```

1  &fec1 {
2     pinctrl-names = "default";
3     pinctrl-0 = <&pinctrl_enet1
4                 &pinctrl_enet1_reset>;
5     phy-mode = "rmii";
6     .....
9     status = "okay";
10 };
11
12 &fec2 {
13     pinctrl-names = "default";
14     pinctrl-0 = <&pinctrl_enet2
15                 &pinctrl_enet2_reset>;
16     phy-mode = "rmii";
17     .....
36 };

```

第 3~4 行, 修改后的 fec1 节点“pinctrl-0”属性值。

第 14~15 行, 修改后的 fec2 节点“pinctrl-0”属性值。

3、修改 SR8201F 的 PHY 地址

在 uboot 移植章节中, 我们说过 ENET1 的 LAN8720A 地址为 0x2, ENET2 的 LAN8720A 地址为 0x1。在 imx6ull-alientek-emmc.dts 中找到如下代码:

示例代码 37.4.4.6 imx6ull-alientek-emmc.dts 代码段

```

171 &fec1 {
172     pinctrl-names = "default";
173     .....
175     phy-handle = <&ethphy0>;
176     status = "okay";
177 };
178
179 &fec2 {
180     pinctrl-names = "default";
181     .....
183     phy-handle = <&ethphy1>;
184     status = "okay";
185
186     mdio {
187         #address-cells = <1>;
188         #size-cells = <0>;
189
190         ethphy0: ethernet-phy@0 {
191             compatible = "ethernet-phy-ieee802.3-c22";
192             reg = <2>;
193         };
194
195         ethphy1: ethernet-phy@1 {
196             compatible = "ethernet-phy-ieee802.3-c22";
197             reg = <1>;
198         };
199     };
200 };

```

第 171~177 行, ENET1 对应的设备树节点。

第 179~200 行, ENET2 对应的设备树节点。但是第 186~198 行的 mdio 节点描述了 ENET1 和 ENET2 的 PHY 地址信息。将示例代码 37.4.4.6 改为如下内容:

示例代码 37.4.4.7 imx6ull-alientek-emmc.dts 代码段

```

171 &fec1 {
172     pinctrl-names = "default";
173     pinctrl-0 = <&pinctrl_enet1
174         &pinctrl_enet1_reset>;
175     phy-mode = "rmii";
176     phy-handle = <&ethphy0>;
177     phy-reset-gpios = <&gpio5 7 GPIO_ACTIVE_LOW>;
178     phy-reset-duration = <200>;
179     status = "okay";
180 };

```

```

181
182 &fec2 {
183     pinctrl-names = "default";
184     pinctrl-0 = <&pinctrl_enet2
185                 &pinctrl_enet2_reset>;
186     phy-mode = "rmii";
187     phy-handle = <&ethphy1>;
188     phy-reset-gpios = <&gpio5 8 GPIO_ACTIVE_LOW>;
189     phy-reset-duration = <200>;
190     status = "okay";
191
192     mdio {
193         #address-cells = <1>;
194         #size-cells = <0>;
195
196         ethphy0: ethernet-phy@2 {
197             compatible = "ethernet-phy-ieee802.3-c22";
198             smsc,disable-energy-detect;
199             reg = <2>;
200         };
201
202         ethphy1: ethernet-phy@1 {
203             compatible = "ethernet-phy-ieee802.3-c22";
204             smsc,disable-energy-detect;
205             reg = <1>;
206         };
207     };
208 };
    
```

第 177 和 178 行，添加了 ENET1 网络复位引脚所使用的 IO 为 GPIO5_IO07，低电平有效。复位低电平信号持续时间为 200ms。

第 188 和 189 行，ENET2 网络复位引脚所使用的 IO 为 GPIO5_IO08，同样低电平有效，持续时间同样为 200ms。

第 198 和 204 行，“smc,disable-energy-detect”表明 PHY 芯片是 SMSC 公司的，这样 Linux 内核就会找到 SMSC 公司的 PHY 芯片驱动来驱动 LAN8720A。

第 196 行，注意“ethernet-phy@”后面的数字是 PHY 的地址，ENET1 的 PHY 地址为 2，所以“@”后面是 2。

第 199 行，reg 的值也表示 PHY 地址，ENET1 的 PHY 地址为 2，所以 reg=2。

第 202 行，ENET2 的 PHY 地址为 1，因此“@”后面为 1。

第 205 行，因为 ENET2 的 PHY 地址为 1，所以 reg=1。

至此，SR8201F 的 PHY 地址就改好了，保存一下 imx6ull-alientek-emmc.dts 文件。然后使用“make dtbs”命令重新编译一下设备树。

3、修改 fec_main.c 文件

要在 IMX6ULL 上使用 SR8201F，需要修改一下 Linux 内核源码，打开

drivers/net/ethernet/freescale/fec_main.c, 找到函数 fec_reset_phy, 在 fec_reset_phy 函数中加入如下代码:

示例代码 37.4.4.8 fec_reset_phy 函数代码段

```

3330 static void fec_reset_phy(struct platform_device *pdev)
3331 {
3332     int err, phy_reset;
3333     int msec = 1;
3334     struct device_node *np = pdev->dev.of_node;
3335
3336     if (!np)
3337         return;
3338
3339     err = of_property_read_u32(np, "phy-reset-duration", &msec);
3340     /* A sane reset duration should not be longer than 1s */
3341     if (!err && msec > 1000)
3342         msec = 1;
3343
3344     phy_reset = of_get_named_gpio(np, "phy-reset-gpios", 0);
3345     if (!gpio_is_valid(phy_reset))
3346         return;
3347
3348     err = devm_gpio_request_one(&pdev->dev, phy_reset,
3349                               GPIOF_OUT_INIT_LOW, "phy-reset");
3350     if (err) {
3351         dev_err(&pdev->dev, "failed to get phy-reset-gpios: %d\n",
err);
3352         return;
3353     }
3354     msleep(msec);
3355     gpio_set_value(phy_reset, 1);
3356     msleep(200); /* 复位结束后至少再延时 150ms 才能继续操作 SR8201F */
3357 }

```

第 3356 行就是新加入的代码, 根据 SR8201F 收据手册上的要求, SR8201F 在复位结束以后需要等待至少 150ms 才能操作 SR8201F, 因此这里添加了一个 200ms 的延时。

4、网络驱动测试

修改好设备树和 Linux 内核以后重新编译一下, 得到新的 zImage 镜像文件和 imx6ull-alianteck-emmc.dtb 设备树文件, 使用网线将 IMX6U-ALPHA 开发板的两个网口与路由器或者电脑连接起来, 最后使用新的文件启动 Linux 内核。启动以后使用 “ifconfig” 命令查看一下当前活动的网卡有哪些, 结果如图 37.4.4.2 所示:

```

random: nonblocking pool is initialized

/ #
/ #

```

图 37.4.4.2 ifconfig 命令结果

从图 37.4.4.2 可以看出，当前没有活动的网卡。输入命令“ifconfig -a”来查看一下开发板中存在的所有网卡，结果如图 37.4.4.3 所示：

```
/ # ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:25

eth0      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          inet addr:192.168.6.50  Bcast:192.168.6.255  Mask:255.255.255.0
          inet6 addr: fe80::204:9fff:fe04:d235/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7085 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4098 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7280904 (6.9 MiB)  TX bytes:493144 (481.5 KiB)

eth1      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

sit0      Link encap:IPv6-in-IPv4
          NOARP  MTU:1480  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/ #
```

图 37.4.4.3 开发板所有网卡

图 37.4.4.3 中 can0 为 CAN 接口的网卡，eth0 和 eth1 才是网络接口的网卡，其中 eth0 对应于 ENET2，eth1 对应于 ENET1。使用如下命令依次打开 eth0 和 eth1 这两个网卡：

```
ifconfig eth0 up
```

```
ifconfig eth1 up
```

网卡的打开过程如图 37.4.4.4 所示：

```
root@ATK-IMX6U:~# ifconfig eth0 up
[ 217.803599] fec 20b4000.ethernet eth0: Freescale FEC PHY driver [Generic PHY] (mii_bus:phy_addr=20b4000.ethernet:01, irq=-1)
root@ATK-IMX6U:~# [ 219.856064] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 220.803918] fec 20b4000.ethernet eth0: Link is Up - 100Mbps/Full - flow control rx/tx
[ 220.812123] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready

root@ATK-IMX6U:~#
root@ATK-IMX6U:~# ifconfig eth1 up
[ 227.963552] fec 2188000.ethernet eth1: Freescale FEC PHY driver [Generic PHY] (mii_bus:phy_addr=20b4000.ethernet:02, irq=-1)
root@ATK-IMX6U:~# [ 230.022010] IPv6: ADDRCONF(NETDEV_UP): eth1: link is not ready
[ 230.963872] fec 2188000.ethernet eth1: Link is Up - 100Mbps/Full - flow control rx/tx
[ 230.972879] IPv6: ADDRCONF(NETDEV_CHANGE): eth1: link becomes ready

root@ATK-IMX6U:~#
```

图 37.4.4.4 两个网卡打开过程

从图 37.4.4.4 中可以看到“Generic PHY”字样，说明当前的网络驱动使用的就是 Linux 内

核自带的通用网络 PHY 驱动。

再次输入 “ifconfig” 命令来查看一下当前活动的网卡，结果如图 37.4.4.5 所示：

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          inet addr:192.168.6.50  Bcast:192.168.6.255  Mask:255.255.255.0
          inet6 addr: fe80::204:9fff:fe04:d235/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5722 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4145 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7177736 (6.8 MiB)  TX bytes:503166 (491.3 KiB)

eth1      Link encap:Ethernet  HWaddr 00:04:9F:04:D2:35
          inet6 addr: fe80::204:9fff:fe04:d235/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:16 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5292 (5.1 KiB)  TX bytes:258 (258.0 B)
```

图 37.4.4.5 当前活动的网卡。

可以看出，此时 eth0 和 eth1 两个网卡都已经打开，并且工作正常，但是这两个网卡都还没有 IP 地址，所以不能进行 ping 等操作。使用如下命令给两个网卡配置 IP 地址：

```
ifconfig eth0 192.168.1.251
```

```
ifconfig eth1 192.168.1.252
```

上述命令配置 eth0 和 eth1 这两个网卡的 IP 地址分别为 192.168.1.251 和 192.168.1.252，注意 IP 地址选择的合理性，一定要和自己的电脑处于同一个网段内，并且没有被其他的设备占用！设置好以后，使用 “ping” 命令来 ping 一下自己的主机，如果能 ping 通那说明网络驱动修改成功！比如我的 Ubuntu 主机 IP 地址为 192.168.1.250，使用如下命令 ping 一下：

```
ping 192.168.1.250
```

结果如图 37.4.4.6 所示：

```
/ # ping 192.168.1.250
PING 192.168.1.250 (192.168.1.250): 56 data bytes
64 bytes from 192.168.1.250: seq=0 ttl=64 time=0.906 ms
64 bytes from 192.168.1.250: seq=1 ttl=64 time=1.016 ms
64 bytes from 192.168.1.250: seq=2 ttl=64 time=0.893 ms
^C
--- 192.168.1.250 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.893/0.938/1.016 ms
/ #
```

图 37.4.4.6 ping 结果

可以看出，ping 成功，说明网络驱动修改成功！我们在后面的构建根文件系统和 Linux 驱动开发中就可以使用网络调试代码啦。

37.4.5 保存修改后的图形化配置文件

如果使用 V2.4 以前的底板，用的是 LAN8720A 这颗 PHY 芯片，我们在修改网络驱动的时候我们通过图形界面使能了 LAN8720A 的驱动，使能以后会在 .config 中存在如下代码：

```
CONFIG_SMSC_PHY=y
```

打开 drivers/net/phy/Makefile，有如下代码：

示例代码 37.4.5.1 drivers/net/phy/Makefile 代码段

```
11 obj-$(CONFIG_SMSC_PHY) += smsc.o
```

当 CONFIG_SMSC_PHY=y 的时候就会编译 smsc.c 这个文件，smsc.c 就是 LAN8720A 的驱

动文件。但是当我们执行“make clean”清理工程以后.config 文件就会被删除掉,因此我们所有的配置内容都会丢失,结果就是前功尽弃,一“删”回到解放前!所以我们在配置完图形界面以后经过测试没有问题,就必须保存一下配置文件。保存配置的方法有两个。

1、直接另存为.config 文件

既然图形化界面配置后的配置项保存在.config 中,那么就简单粗暴,直接将.config 文件另存为 imx_alientek_emmc_defconfig, 然后其复制到 arch/arm/configs 目录下, 替换以前的 imx_alientek_emmc_defconfig。这样以后执行“make imx_alientek_emmc_defconfig”重新配置 Linux 内核的时候就会使用新的配置文件,默认就会使能 LAN8720A 的驱动。

2、通过图形界面保存配置文件

相比于第 1 种直接另存为.config 文件,第 2 种方法就很“文雅”了,在图形界面中保存配置文件,在图形界面中会有“< Save >”选项,如图 37.4.5.1 所示:



图 37.4.5.1 保存配置

通过键盘的“→”键,移动到“< Save >”选项,然后按下回车键,打开文件名输入对话框,如图 37.4.5.2 所示:

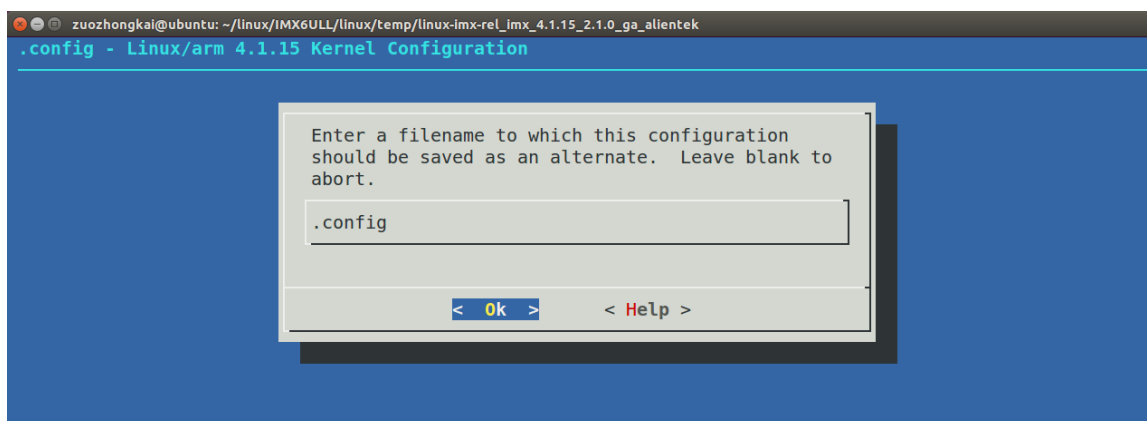


图 37.4.5.2 输入文件名

在图 37.4.5.2 中输入要保存的文件名,可以带路径,一般是相对路径(相对于 Linux 内核源码根目录)。比如我们要将新的配置文件保存到目录 arch/arm/configs 下, 文件名为 imx_alientek_emmc_defconfig, 也就是用新的配置文件替换掉老的默认配置文件。那么我们在图 37.4.5.2 中输入“arch/arm/configs/imx_alientek_emmc_defconfig”即可,如图 37.4.5.3 所示:

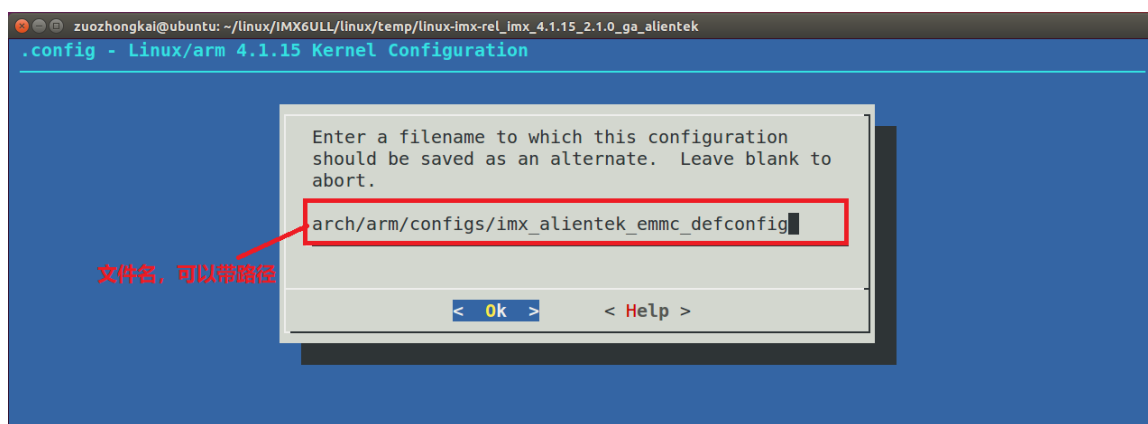


图 37.4.5.3 输入文件名

设置好文件名以后选择下方的“< Ok >”按钮，保存文件并退出。退出以后再打开 `imx_alientek_emmc_defconfig` 文件，就会在此文件中找到“`CONFIG_SMSC_PHY=y`”这一行，如图 37.4.4.4 所示：

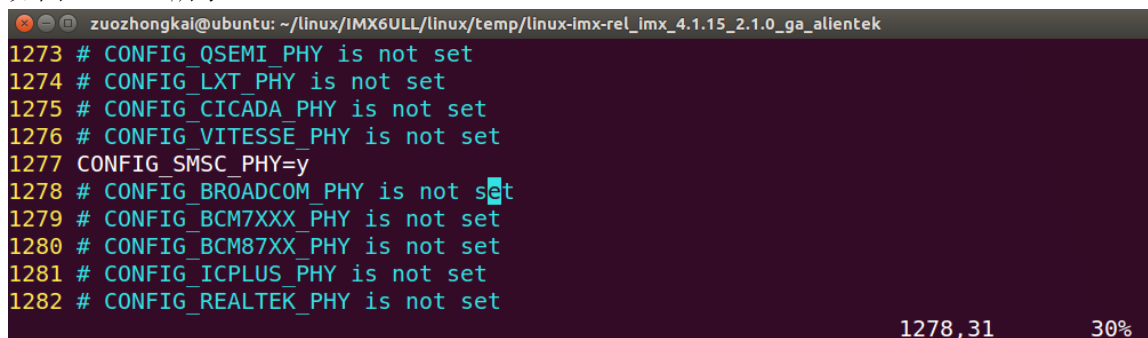


图 37.4.5.4 新的配置文件

同样的，使用“`make imx_alientek_emmc_defconfig`”重新配置 Linux 内核的时候，LAN8720A 的驱动就会使能，并被编译进 Linux 镜像文件 `zImage` 中。

关于 Linux 内核的移植就讲解到这里，简单总结一下移植步骤：

- ①、在 Linux 内核中查找可以参考的板子，一般都是半导体厂商自己做的开发板。
- ②、编译出参考板子对应的 `zImage` 和 `.dtb` 文件。
- ③、使用参考板子的 `zImage` 文件和 `.dtb` 文件在我们所使用的板子上启动 Linux 内核，看能否启动。
- ④、如果能启动的话就万事大吉，如果不能启动那就悲剧了，需要调试 Linux 内核。不过一般都会参考半导体官方的开发板设计自己的硬件，所以大部分情况下都会启动起来。启动 Linux 内核用到的外设不多，一般就 DRAM(Uboot 都初始化好的)和串口。作为终端使用的串口一般都会参考半导体厂商的 Demo 板。
- ⑤、修改相应的驱动，像 NAND Flash、EMMC、SD 卡等驱动官方的 Linux 内核都是已经提供好了，基本不会出问题。重点是网络驱动，因为 Linux 驱动开发一般都要通过网络调试代码，所以一定要确保网络驱动工作正常。如果是处理器内部 MAC+外部 PHY 这种网络方案的话，一般网络驱动都很好处理，因为在 Linux 内核中是有外部 PHY 通用驱动的。只要设置好复位引脚、PHY 地址信息基本上都可以驱动起来。
- ⑥、Linux 内核启动以后需要根文件系统，如果没有根文件系统的话肯定会崩溃，所以确定 Linux 内核移植成功以后就要开始根文件系统的构建。

第三十八章 根文件系统构建

Linux “三巨头” 已经完成了 2 个了, 就剩最后一个 rootfs(根文件系统)了, 本章我们就来学习一下根文件系统的组成以及如何构建根文件系统。这是 Linux 移植的最后一步, 根文件系统构建好以后就意味着我们已经拥有了一个完整的、可以运行的最小系统。以后我们就在这个最小系统上编写、测试 Linux 驱动, 移植一些第三方组件, 逐步的完善这个最小系统。最终得到一个功能完善、驱动齐全、相对完善的操作系统。

38.1 根文件系统简介

根文件系统一般也叫做 rootfs, 那么什么叫根文件系统? 看到“文件系统”这四个字, 很多人, 包括我第一反应就是 FATFS、FAT、EXT4、YAFFS 和 NTFS 等这样的文件系统。在这里, 根文件系统并不是 FATFS 这样的文件系统代码, EXT4 这样的文件系统代码属于 Linux 内核的一部分。Linux 中的根文件系统更像是一个文件夹或者叫做目录(在我看来就是一个文件夹, 只不过是特殊的文件夹), 在这个目录里面会有很多的子目录。根目录下和子目录中会有很多的文件, 这些文件是 Linux 运行所必须的, 比如库、常用的软件和命令、设备文件、配置文件等等。以后我们说到文件系统, 如果不特别指明, 统一表示根文件系统。对于根文件系统专业的解释, 百度百科上是这么说的(原谅我把百度百科引用为专业解释, 因为我实在找不到根文件系统的最初定义, 也不要建议我到哪些 404 网站去查找, 毕竟我胖, 我怕翻到一般梯子不稳把我摔丑了):

根文件系统首先是内核启动时所 mount(挂载)的第一个文件系统, 内核代码映像文件保存在根文件系统中, 而系统引导启动程序会在根文件系统挂载之后从中把一些基本的初始化脚本和服务等加载到内存中去运行。

百度百科上说内核代码映像文件保存在根文件系统中, 但是我们嵌入式 Linux 并没有将内核代码映像保存在根文件系统中, 而是保存到了其他地方。比如 NAND Flash 的指定存储地址、EMMC 专用分区中。根文件系统是 Linux 内核启动以后挂载(mount)的第一个文件系统, 然后从根文件系统中读取初始化脚本, 比如 rcS, inittab 等。根文件系统和 Linux 内核是分开的, 单独的 Linux 内核是没法正常工作的, 必须要搭配根文件系统。如果不提供根文件系统, Linux 内核在启动的时候就会提示内核崩溃(Kernel panic)的提示, 这个在 37.2.4 小节已经说过了。

根文件系统的这个“根”字就说明了这个文件系统的重要性, 它是其他文件系统的根, 没有这个“根”, 其他的文件系统或者软件就别想工作。比如我们常用的 ls、mv、ifconfig 等命令其实就是一个个小软件, 只是这些软件没有图形界面, 而且需要输入命令来运行。这些小软件就保存在根文件系统中, 这些小软件是怎么来的呢? 这个就是我们本章教程的目的, 教大家来构建自己的根文件系统, 这个根文件系统是满足 Linux 运行的最小根文件系统, 后续我们可以根据自己的实际工作需求不断的去填充这个最小根文件系统, 最终使其成为一个相对完善的根文件系统。

在构建根文件系统之前, 我们先来看一下根文件系统里面大概都有些什么内容, 以 Ubuntu 为例, 根文件系统的目录名字为 '/', 没看错就是一个斜杠, 所以输入如下命令就可以进入根目录中:

```
cd / //进入根目录
```

进入根目录以后输入“ls”命令查看根目录下的内容都有哪些, 结果如图 38.1.1 所示:

```
zuozhongkai@ubuntu:/$ cd /
zuozhongkai@ubuntu:/$ ls
bin          build-trusted  home          lib32         media        root         srv          var
boot         cdrom          initrd.img    lib64         mnt          run          sys          vmlinuz
build-basic  dev           initrd.img.old libx32        opt          sbin         tmp          vmlinuz.old
build-optee  etc           lib           lost+found    proc         snap         usr
```

图 38.1.1 Ubuntu 根目录

图 38.1.1 中根目录下子目录和文件不少, 但是这些都是 Ubuntu 所需要的, 其中有很多子目录和文件我们嵌入式 Linux 是用不到的, 所以这里就讲解一些常用的子目录:

1、/bin 目录

看到“bin”大家应该能想到 bin 文件, bin 文件就是可执行文件。所以此目录下存放着系统需要的可执行文件, 一般都是一些命令, 比如 ls、mv 等命令。此目录下的命令所有的客户都可以使用。

2、/dev 目录

dev 是 device 的缩写, 所以此目录下的文件都是和设备有关的, 此目录下的文件都是设备文件。在 Linux 下一切皆文件, 即使是硬件设备, 也是以文件的形式存在的, 比如 /dev/ttymx0(I.MX6ULL 根目录会有此文件)就表示 I.MX6ULL 的串口 0, 我们要想通过串口 0 发送或者接收数据就要操作文件/dev/ttymx0, 通过对文件/dev/ttymx0 的读写操作来实现串口 0 的数据收发。

3、/etc 目录

此目录下存放着各种配置文件, 大家可以进入 Ubuntu 的 etc 目录看一下, 里面的配置文件非常多! 但是在嵌入式 Linux 下此目录会很简洁。

4、/lib 目录

lib 是 library 的简称, 也就是库的意思, 因此此目录下存放着 Linux 所必须的库文件。这些库文件是共享库, 命令和用户编写的应用程序要使用这些库文件。

5、/mnt 目录

临时挂载目录, 一般是空目录, 可以在此目录下创建空的子目录, 比如/mnt/sd、/mnt/usb, 这样就可以将 SD 卡或者 U 盘挂载到/mnt/sd 或者/mnt/usb 目录中。

6、/proc 目录

此目录一般是空的, 当 Linux 系统启动以后会将此目录作为 proc 文件系统的挂载点, proc 是个虚拟文件系统, 没有实际的存储设备。proc 里面的文件都是临时存在的, 一般用来存储系统运行信息文件。

7、/usr 目录

要注意, usr 不是 user 的缩写, 而是 Unix Software Resource 的缩写, 也就是 Unix 操作系统软件资源目录。这里有个小知识点, 那就是 Linux 一般被称为类 Unix 操作系统, 苹果的 MacOS 也是类 Unix 操作系统。关于 Linux 和 Unix 操作系统的渊源大家可以直接在网上找 Linux 的发展历史来看。既然是软件资源目录, 因此/usr 目录下也存放着很多软件, 一般系统安装完成以后此目录占用的空间最多。

8、/var 目录

此目录存放一些可以改变的数据。

9、/sbin 目录

此目录用户存放一些可执行文件, 但是此目录下的文件或者说命令只有管理员才能使用, 主要用户系统管理。

10、/sys 目录

系统启动以后此目录作为 sysfs 文件系统的挂载点, sysfs 是一个类似于 proc 文件系统的特殊文件系统, sysfs 也是基于 ram 的文件系统, 也就是说它也没有实际的存储设备。此目录是系统设备管理的重要目录, 此目录通过一定的组织结构向用户提供详细的内核数据结构信息。

11、/opt

可选的文件、软件存放区, 由用户选择将哪些文件或软件放到此目录中。

关于 Linux 的根目录就介绍到这里, 接下来的构建根文件系统就是研究如何创建上面这些子目录以及子目录中的文件。

38.2 BusyBox 构建根文件系统

38.2.1 BusyBox 简介

上一小节说了，根文件系统里面就是一堆的可执行文件和其他文件组成的？难道我们得一个一个的从网上去下载这些文件？显然这是不现实的！那么有没有人或者组织专门干这个事呢？他们负责“收集”这些文件，然后将其打包，像我们这样的开发者可以直接拿来用。答案是有的，它就叫做 BusyBox！其名字分为“Busy”和“Box”，也就是忙碌的盒子。盒子是用来放东西的，忙碌的是因为它要提供根文件系统所需的文件，所以忙碌。BusyBox 是一个集成了大量的 Linux 命令和工具的软件，像 ls、mv、ifconfig 等命令 BusyBox 都会提供。BusyBox 就是一个大的工具箱，这个工具箱里面集成了 Linux 的许多工具和命令。一般下载 BusyBox 的源码，然后配置 BusyBox，选择自己想要的功能，最后编译即可。

BusyBox 可以在其官网下载到，官网地址为：<https://busybox.net/>，官网比较简陋，如图 38.2.1.1 所示：

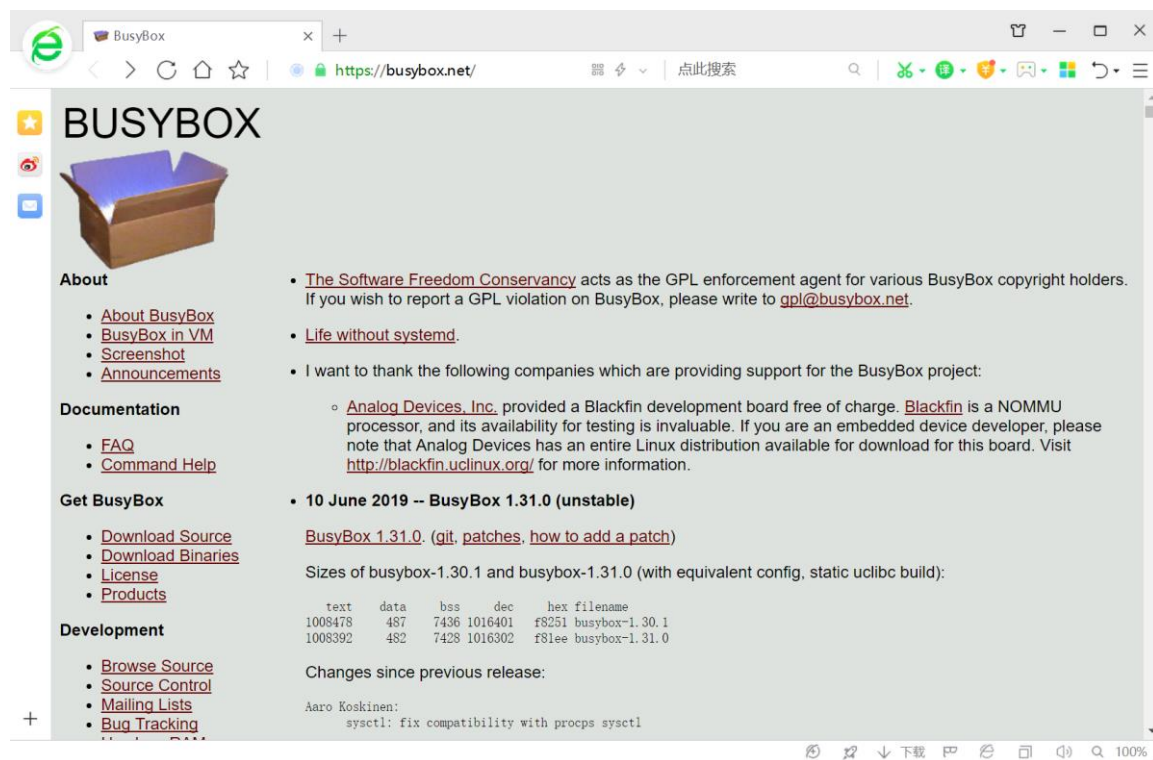


图 38.2.1.1 BusyBox 官网

在官网左侧的“Get BusyBox”栏有一行“Download Source”，点击“Download Source”即可打开 BusyBox 的下载页，如图 38.2.1.2 所示：

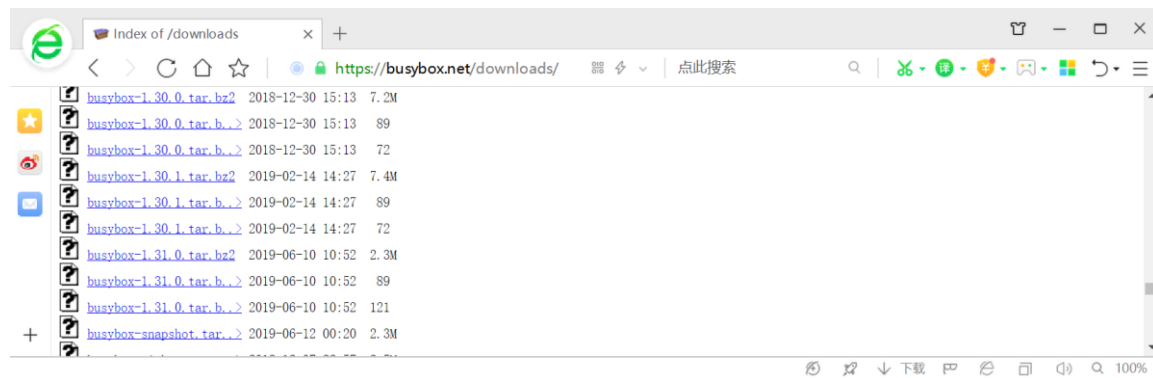


图 38.2.1.2 BusyBox 下载页

从图 38.2.1.2 可以看出,目前最新的 BusyBox 版本是 1.31.0,不过我建议大家使用我们开发板光盘里面提供的 1.29.0 版本的 BusyBox。因为笔者测试 1.29.0 版本目前还没有出现任何问题,路径为:1、例程源码->6、BusyBox 源码->busybox-1.29.0.tar.bz2, BusyBox 准备好以后就可以构建根文件系统了。

38.2.2 编译 BusyBox 构建根文件系统

一般我们在 Linux 驱动开发的时候都是通过 nfs 挂载根文件系统的,当产品最终上市开卖的时候才会将根文件系统烧写到 EMMC 或者 NAND 中。所以要在 4.2.1 小节中设置的 nfs 服务器目录中创建一个名为 rootfs 的子目录(名字大家可以随意起,为了方便就用了 rootfs),比如我的电脑中“/home/zuozhongkai/linux/nfs”就是我设置的 NFS 服务器目录,使用如下命令创建名为 rootfs 的子目录:

```
mkdir rootfs
```

创建好的 rootfs 子目录就用来存放我们的根文件系统了。

将 busybox-1.29.0.tar.bz2 发送到 Ubuntu 中,存放位置大家随便选择。然后使用如下命令将其解压:

```
tar -xjvf busybox-1.29.0.tar.bz2
```

解压完成以后进入到 busybox-1.29.0 目录中,此目录中的文件和文件夹如图 38.2.2.1 所示:

```
zuozhongkai@ubuntu:~/linux/busybox$ cd busybox-1.29.0/
zuozhongkai@ubuntu:~/linux/busybox/busybox-1.29.0$ ls
applets      coreutils   init         Makefile      NOFORK_NOEXEC.lst  selinux
applets_sh   debianutils INSTALL      Makefile.custom NOFORK_NOEXEC.sh   shell
arch         docs        klibc-utils Makefile.flags printutils          size_single_applets.sh
archival     e2fsprogs  libbb       Makefile.help  procps             sysklogd
AUTHORS      editors    libpwdgrp   make_single_applets.sh  qemu_multiarch_testing  test suite
Config.in    examples  LICENSE     miscutils      README             TODO
configs      findutils loginutils  modutils       runit           TODO_unicode
console-tools include    mailutils   networking     scripts            util-linux
```

图 38.2.2.1 busybox-1.29.0 目录内容

1、修改 Makefile, 添加编译器

同 Uboot 和 Linux 移植一样,打开 busybox 的顶层 Makefile,添加 ARCH 和 CROSS_COMPILE 的值,如下所示:

示例代码 38.2.2.1 Makefile 代码段

```
164 CROSS_COMPILE ?= /usr/local/arm/gcc-linaro-4.9.4-2017.01-
x86_64_arm-linux-gnueabihf/bin/arm-linux-gnueabihf-
.....
190 ARCH ?= arm
```

在示例代码 38.2.2.1 中 CORSS_COMPILE 使用了绝对路径！主要是为了防止编译出错。

2、busybox 中文字符支持

如果默认直接编译 busybox 的话，在使用 SecureCRT 的时候中文字符是显示不正常的，中文字符会显示为“？”，比如你的中文目录，中文文件都显示为“？”。不知道从哪个版本开始 busybox 中的 shell 命令对中文输入即显示做了限制，即使内核支持中文但在 shell 下也依然无法正确显示。

所以我们需要修改 busybox 源码，取消 busybox 对中文显示的限制，打开文件 busybox-1.29.0/libbb/printable_string.c，找到函数 printable_string，缩减后的函数内容如下：

示例代码 38.2.2.2 libbb/printable_string.c 代码段

```
12 const char* FAST_FUNC printable_string(uni_stat_t *stats, const char
*str)
13 {
14     char *dst;
15     const char *s;
16
17     s = str;
18     while (1) {
19         unsigned char c = *s;
20         if (c == '\\0') {
21             .....
22         }
23         if (c < ' ')
24             break;
25         if (c >= 0x7f)
26             break;
27         s++;
28     }
29
30 #if ENABLE_UNICODE_SUPPORT
31     dst = unicode_conv_to_printable(stats, str);
32 #else
33 {
34     char *d = dst = xstrdup(str);
35     while (1) {
36         unsigned char c = *d;
37         if (c == '\\0')
38             break;
39         if (c < ' ' || c >= 0x7f)
40             *d = '?';
41         d++;
42     }
43     .....
44 #endif
```

```
56 return auto_string(dst);
57 }
```

第 31 和 32 行, 当字符大于 0X7F 以后就跳出去了。

第 45 和 46 行, 如果支持 UNICODE 码的话, 当字符大于 0X7F 就直接输出 ‘?’。

所以我们需要对这 4 行代码进行修改, 修改以后如下所示:

示例代码 38.2.2.3 libbb/printable_string.c 代码段

```
12 const char* FAST_FUNC printable_string(uni_stat_t *stats, const char
*str)
13 {
14     char *dst;
15     const char *s;
16
17     s = str;
18     while (1) {
19         .....
20         if (c < ' ')
21             break;
22         /* 注释掉下面这个两行代码 */
23         /* if (c >= 0x7f)
24             break; */
25         s++;
26     }
27
28     #if ENABLE_UNICODE_SUPPORT
29     dst = unicode_conv_to_printable(stats, str);
30     #else
31     {
32         char *d = dst = xstrdup(str);
33         while (1) {
34             unsigned char c = *d;
35             if (c == '\\0')
36                 break;
37             /* 修改下面代码 */
38             /* if (c < ' ' || c >= 0x7f) */
39             if (c < ' ')
40                 *d = '?';
41             d++;
42         }
43         .....
44     }
45     #endif
46     return auto_string(dst);
47 }
```

示例代码 38.2.2.3 中红色部分的代码就是被修改以后的, 主要就是禁止字符大于 0X7F 以后 break 和输出 ‘?’。

接着打开文件 busybox-1.29.0/libbb/unicode.c, 找到如下内容:

示例代码 38.2.2.4 libbb/unicode.c 代码段

```
1003 static char* FAST_FUNC unicode_conv_to_printable2(uni_stat_t
*stats, const char *src, unsigned width, int flags)
1004 {
1005     char *dst;
1006     unsigned dst_len;
1007     unsigned uni_count;
1008     unsigned uni_width;
1009
1010     if (unicode_status != UNICODE_ON) {
1011         char *d;
1012         if (flags & UNI_FLAG_PAD) {
1013             d = dst = xmalloc(width + 1);
1014             .....
1022             *d++ = (c >= ' ' && c < 0x7f) ? c : '?';
1023             src++;
1024         }
1025         *d = '\\0';
1026     } else {
1027         d = dst = xstrndup(src, width);
1028         while (*d) {
1029             unsigned char c = *d;
1030             if (c < ' ' || c >= 0x7f)
1031                 *d = '?';
1032             d++;
1033         }
1034     }
1035     .....
1040     return dst;
1041 }
1042 .....
1130
1131 return dst;
1132 }
```

第 1022 行, 当字符大于 0X7F 以后, *d++就为 ‘?’。

第 1030 和 1031 行, 当字符大于 0X7F 以后, *d 也为 ‘?’。

修改示例代码 38.2.2.4, 修改后内容如下所示:

示例代码 38.2.2.5 libbb/unicode.c 代码段

```
1003 static char* FAST_FUNC unicode_conv_to_printable2(uni_stat_t
*stats, const char *src, unsigned width, int flags)
```

```

1004 {
1005     char *dst;
1006     unsigned dst_len;
1007     unsigned uni_count;
1008     unsigned uni_width;
1009
1010     if (unicode_status != UNICODE_ON) {
1011         char *d;
1012         if (flags & UNI_FLAG_PAD) {
1013             d = dst = xmalloc(width + 1);
1014             .....
1022             /* 修改下面一行代码 */
1023             /* *d++ = (c >= ' ' && c < 0x7f) ? c : '?'; */
1024             *d++ = (c >= ' ') ? c : '?';
1025             src++;
1026         }
1027         *d = '\0';
1028     } else {
1029         d = dst = xstrndup(src, width);
1030         while (*d) {
1031             unsigned char c = *d;
1032             /* 修改下面一行代码 */
1033             /* if (c < ' ' || c >= 0x7f) */
1034             if(c < ' ')
1035                 *d = '?';
1036             d++;
1037         }
1038     }
1039     .....
1044     return dst;
1045 }
1046 .....
1047
1048     return dst;
1049 }
    
```

示例代码 38.2.2.5 中红色部分的代码就是被修改以后的,同样主要是禁止字符大于 0X7F 的时候设置为 ‘?’。busybox 中文字符支持跟代码修改有关的就改好了,最后还需要配置 busybox 来使能 unicode 码,这个稍后我们配置 busybox 的时候在设置。

3、配置 busybox

根我们编译 Uboot、Linux kernel 一样,我们要先对 busybox 进行默认的配置,有以下几种配置选项:

- ①、defconfig, 缺省配置, 也就是默认配置选项。
- ②、allyesconfig, 全选配置, 也就是选中 busybox 的所有功能。

③、allnoconfig, 最小配置。

我们一般使用默认配置即可, 因此使用如下命令先使用默认配置来配置一下 busybox:

```
make defconfig
```

busybox 也支持图形化配置, 通过图形化配置我们可以进一步选择自己想要的功能, 输入如下命令打开图形化配置界面:

```
make menuconfig
```

打开以后如图 38.2.2.2 所示:

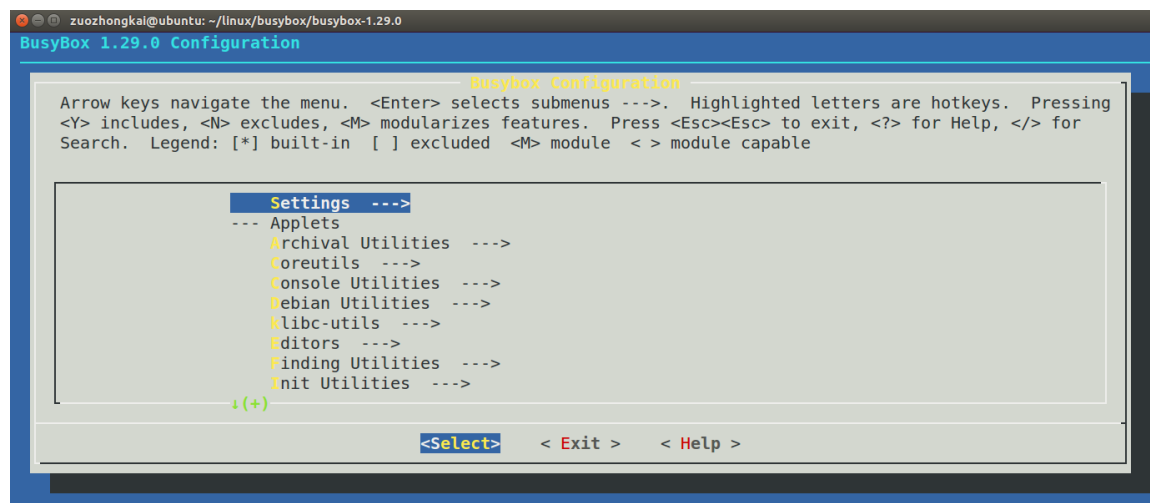


图 38.2.2.2 busybox 图形化配置界面

配置路径如下:

Location:

-> Settings

-> Build static binary (no shared libs)

选项 “Build static binary (no shared libs)” 用来决定是静态编译 busybox 还是动态编译, 静态编译的话就不需要库文件, 但是编译出来的库会很大。动态编译的话要求根文件系统中有序文件, 但是编译出来的 busybox 会小很多。这里我们不能采用静态编译! 因为采用静态编译的话 DNS 会出问题! 无法进行域名解析, 配置如图 38.2.2.3 所示:

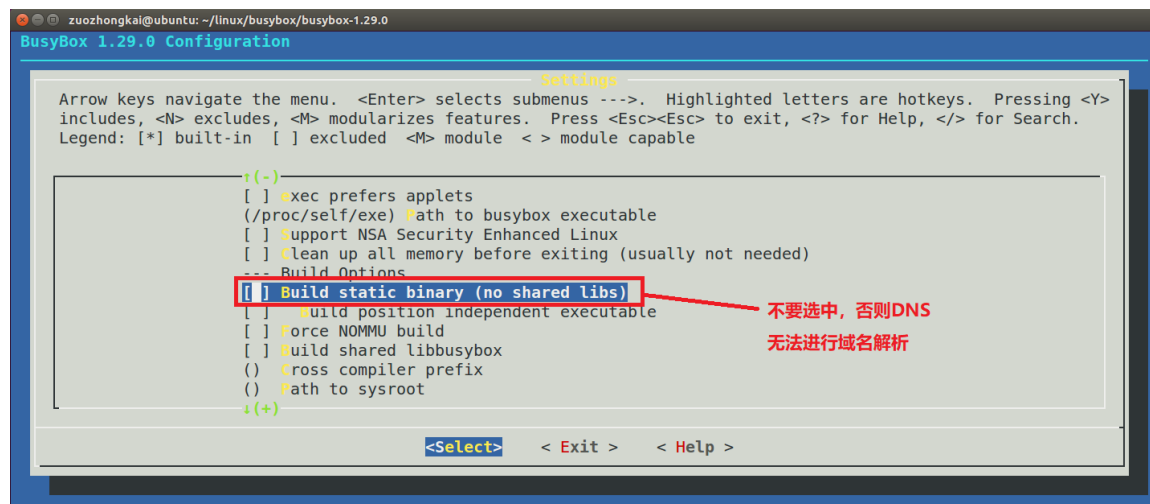


图 38.2.2.3 不选择 “Build static binary (no shared libs)”

继续配置如下路径配置项:

Location:

-> Settings

-> vi-style line editing commands

结果如图 38.2.2.4 所示:

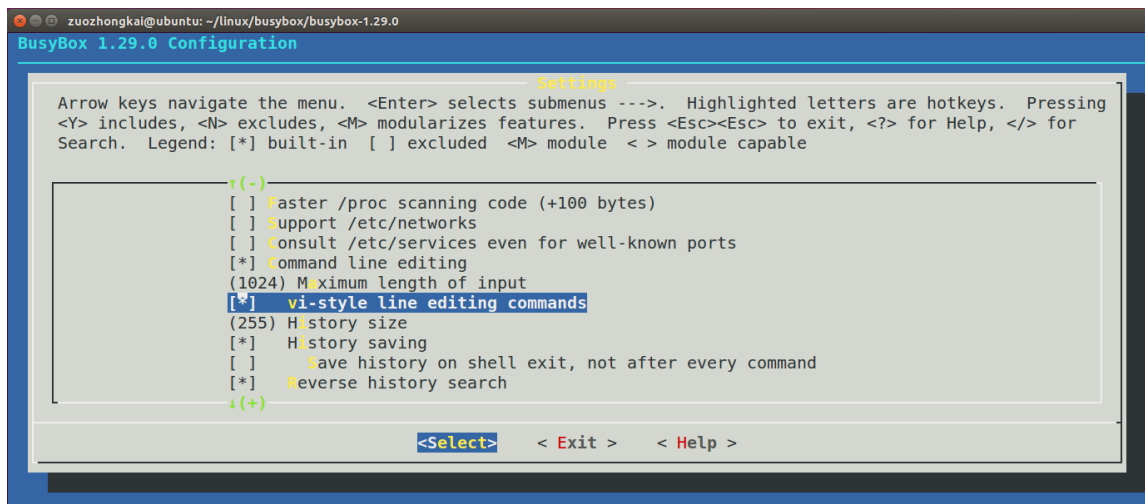


图 38.2.2.4 选择 “vi-style line editing commands”

继续配置如下路径配置项:

Location:

-> Linux Module Utilities

-> Simplified modutils

默认会选中 “Simplified modutils”，这里我们要取消勾选!! 结果如图 38.2.2.5 所示:

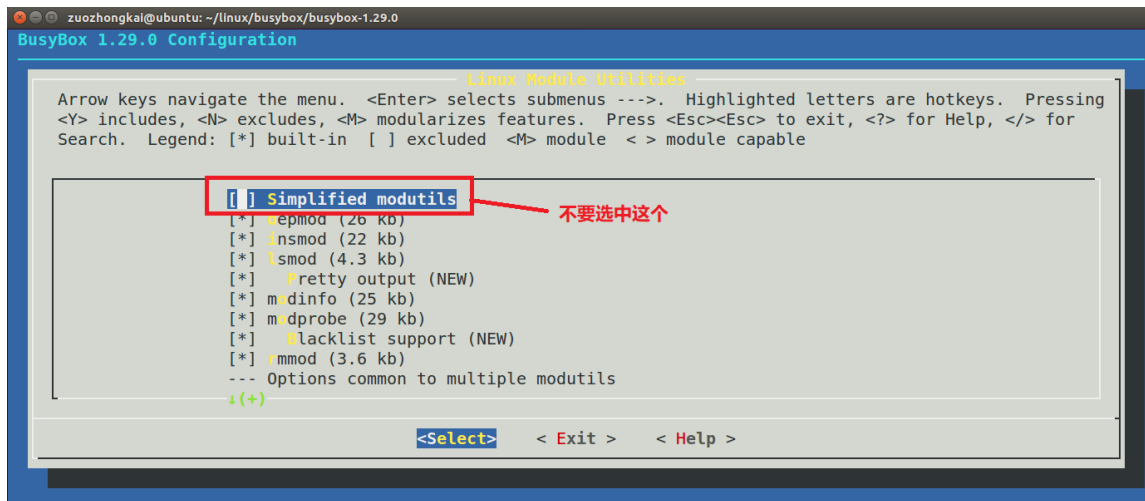


图 38.2.2.5 取消选中 “Simplified modutils”

继续配置如下路径配置项:

Location:

-> Linux System Utilities

-> mdev (16 kb) //确保下面的全部选中，默认都是选中的

结果如图 38.2.2.6 所示:

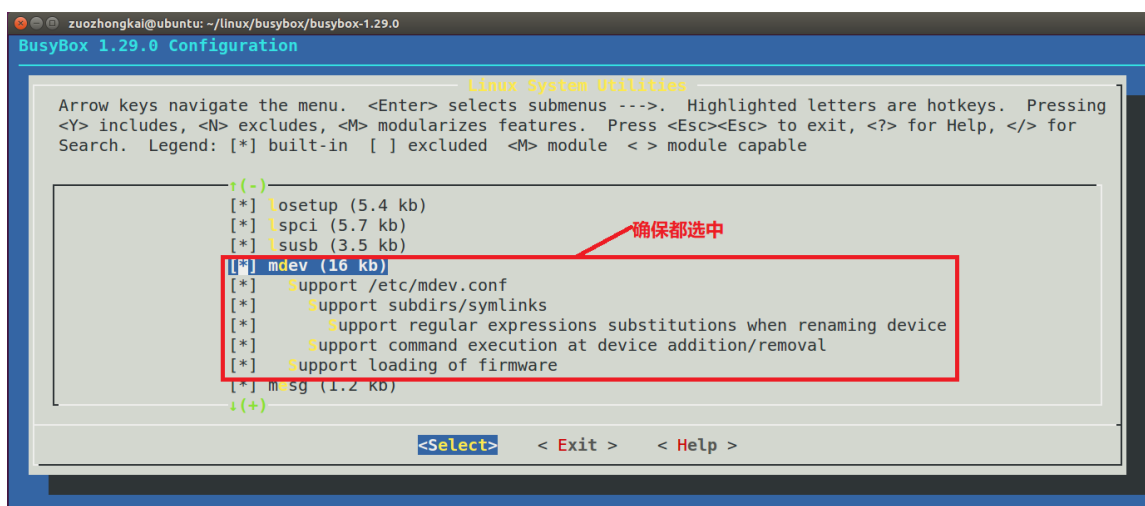


图 38.2.2.6 “mdev” 配置项

最后就是使能 busybox 的 unicode 编码以支持中文，配置路径如下：

Location:

-> Settings

-> Support Unicode

//选中

-> Check \$LC_ALL, \$LC_CTYPE and \$LANG environment variables

//选中

结果如图 38.2.2.7 所示：

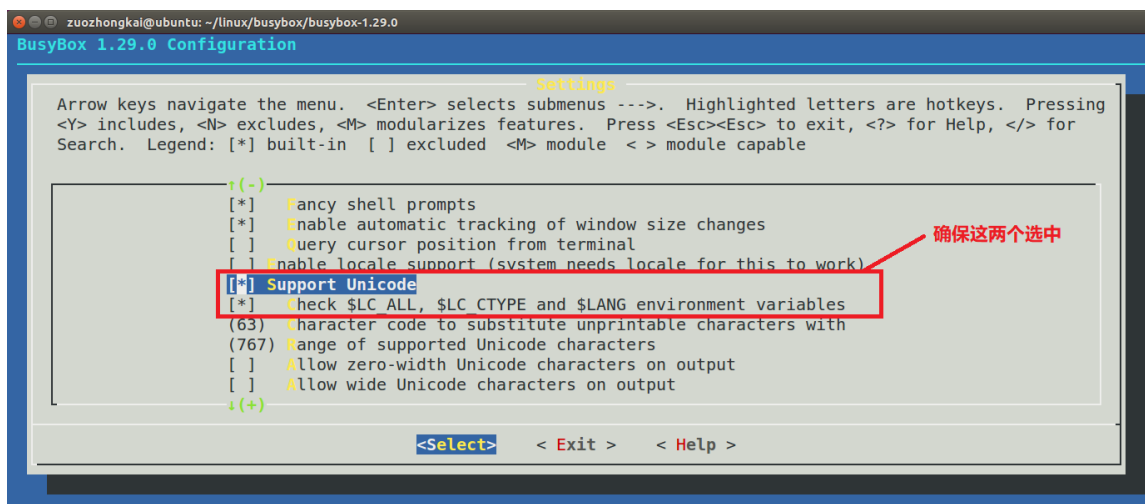


图 38.2.2.7 中文支持

busybox 的配置就到此结束了，大家也可以根据自己的实际需求选择配置其他的选项，不过对于初学者笔者不建议再做其他的修改，可能会出现编译出错的情况发生。

4、编译 busybox

配置好 busybox 以后就可以编译了，我们可以指定编译结果的存放目录，我们肯定要将编译结果存放到前面创建的 rootfs 目录中，输入如下命令：

```
make
```

```
make install CONFIG_PREFIX=/home/zuozhongkai/linux/nfs/rootfs
```

CONFIG_PREFIX 指定编译结果的存放目录，比如我存放到“/home/zuozhongkai/linux/nfs/rootfs”目录中，等待编译完成。编译完成以后如图 38.2.2.8 所示：

```

zuozhongkai@ubuntu: ~/linux/busybox/busybox-1.29.0
./_install//usr/sbin/ubirename -> ../../bin/busybox
./_install//usr/sbin/ubirmvol -> ../../bin/busybox
./_install//usr/sbin/ubirsvol -> ../../bin/busybox
./_install//usr/sbin/ubiupdatevol -> ../../bin/busybox
./_install//usr/sbin/udhcpd -> ../../bin/busybox

-----
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
-----

zuozhongkai@ubuntu:~/linux/busybox/busybox-1.29.0$

```

图 38.2.2.8 busybox 编译完成

编译完成以后会在 busybox 的所有工具和文件就会被安装到 rootfs 目录中，rootfs 目录内容如图 38.2.2.9 所示：

```

zuozhongkai@ubuntu:~/linux/nfs/rootfs$ ls
bin  linuxrc  sbin  usr
zuozhongkai@ubuntu:~/linux/nfs/rootfs$

```

图 38.2.2.9 rootfs 目录

从图 38.2.2.9 可以看出，rootfs 目录下有 bin、sbin 和 usr 这三个目录，以及 linuxrc 这个文件。前面说过 Linux 内核 init 进程最后会查找用户空间的 init 程序，找到以后就会运行这个用户空间的 init 程序，从而切换到用户态。如果 bootargs 设置 init=/linuxrc，那么 linuxrc 就是可以作为用户空间的 init 程序，所以用户态空间的 init 程序是 busybox 来生成的。

busybox 的工作就完成了，但是此时的根文件系统还不能使用，还需要一些其他的文件，我们继续来完善 rootfs。

38.2.3 向根文件系统添加 lib 库

1、向 rootfs 的“/lib”目录添加库文件

Linux 中的应用程序一般都是需要动态库的，当然你也可以编译成静态的，但是静态的可执行文件会很大。如果编译为动态的话就需要动态库，所以我们需要向根文件系统中添加动态库。在 rootfs 中创建一个名为“lib”的文件夹，命令如下：

```
mkdir lib
```

lib 文件夹创建好了，库文件从哪里来呢？lib 库文件从交叉编译器中获取，前面我们搭建交叉编译环境的时候将交叉编译器存放到了“/usr/local/arm/”目录中。交叉编译器里面有很多的库文件，这些库文件具体是做什么的我们作为初学者肯定不知道，既然我不知道那就简单粗暴的把所有的库文件都放到我们的根文件系统中。这样做出来的根文件系统肯定很大，但是我们现在是学习阶段，还做不了裁剪。这就是为什么我们推荐大家购买 512MB+8GB 版本的 EMMC 核心版，如果后面要学习 QT 的话那占用的空间将更大，不裁剪的话 512MB 的 NAND 完全不够用的！而裁剪又是需要经验的，我们都是初学者，哪里来的经验啊。所以我们推荐初学者购买 EMMC 版核心板并不是说为了多赚大家的钱，而是从实际角度考虑的。

进入如下路径对应的目录：

```
/usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi/hf/arm-linux-gnueabi/hf/libc/lib
```

此目录下有很多的*so*(是通配符)和.a 文件，这些就是库文件，将此目录下所有的*so*和.a 文件都拷贝到 rootfs/lib 目录中，拷贝命令如下：

```
cp *so* *.a /home/zuozhongkai/linux/nfs/rootfs/lib/ -d
```

后面的“-d”表示拷贝符号链接，这里有个比较特殊的库文件：ld-linux-armhf.so.3，此库文

件也是个符号链接，相当于 Windows 下的快捷方式。会链接到库 ld-2.19-2014.08-1-git.so 上，输入命令“ls ld-linux-armhf.so.3 -l”查看此文件详细信息，如图 38.2.3.1 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ cd lib/
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls ld-linux-armhf.so.3 -l
lrwxrwxrwx 1 zuozhongkai zuozhongkai 24 Jun 13 12:36 ld-linux-armhf.so.3 -> ld-2.19-2014.08-1-git.so
```

图 38.2.3.1 文件 ld-linux-armhf.so.3

从图 38.2.3.1 可以看出，ld-linux-armhf.so.3 后面有个“->”，表示其是个软连接文件，链接到文件 ld-2.19-2014.08-1-git.so，因为其是一个“快捷方式”，因此大小只有 24B。但是，ld-linux-armhf.so.3 不能作为符号链接，否则的话在根文件系统中执行程序无法执行！所以我们需要 ld-linux-armhf.so.3 完成逆袭，由“快捷方式”变为“本尊”，方法很简单，那就是重新复制 ld-linux-armhf.so.3，只是不复制软链接即可，先将 rootfs/lib 中的 ld-linux-armhf.so.3 文件删除掉，命令如下：

```
rm ld-linux-armhf.so.3
```

然后重新进入到 /usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi/lib/arm-linux-gnueabi/libc/lib 目录中，重新拷贝 ld-linux-armhf.so.3，命令如下：

```
cp ld-linux-armhf.so.3 /home/zuozhongkai/linux/nfs/rootfs/lib/
```

拷贝完成以后再到 rootfs/lib 目录下查看 ld-linux-armhf.so.3 文件详细信息，如图 38.2.3.2 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ rm ld-linux-armhf.so.3
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls ld-linux-armhf.so.3 -l
-rwxr-xr-x 1 zuozhongkai zuozhongkai 724392 Jun 13 12:59 ld-linux-armhf.so.3
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$
```

图 38.2.3.2 文件 ld-linux-armhf.so.3

从图 38.2.3.2 可以看出，此时 ld-linux-armhf.so.3 已经不是软连接了，而是实实在在的一个库文件，而且文件大小为 724392B。

继续进入如下目录中：

```
/usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi/lib
```

此目录下也有很多的*so*和.a 库文件，我们将其也拷贝到 rootfs/lib 目录中，命令如下：

```
cp *so* *.a /home/zuozhongkai/linux/nfs/rootfs/lib/ -d
```

rootfs/lib 目录的库文件就这些了，完成以后的 rootfs/lib 目录如图 38.2.3.3 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls
ld-2.19-2014.08-1-git.so      libgomp.so                  libpthread-2.19-2014.08-1-git.so
ld-linux-armhf.so.3          libgomp.so.1               libpthread.so.0
libanl-2.19-2014.08-1-git.so libgomp.so.1.0.0           libresolv-2.19-2014.08-1-git.so
libanl.so.1                  libitm.a                   libresolv.so.2
libasan.a                    libitm.so                  librt-2.19-2014.08-1-git.so
libasan.so                   libitm.so.1               librt.so.1
libasan.so.1                 libitm.so.1.0.0           libSegFault.so
libasan.so.1.0.0             libm-2.19-2014.08-1-git.so libssp.a
libatomic.a                  libmusage.so               libssp_nonshared.a
libatomic.so                 libm.so.6                 libssp.so
libatomic.so.1               libnsl-2.19-2014.08-1-git.so libssp.so.0
libatomic.so.1.1.0           libnsl.so.1               libssp.so.0.0.0
libBrokenLocale-2.19-2014.08-1-git.so libnss_compat-2.19-2014.08-1-git.so libstdc++.a
libBrokenLocale.so.1         libnss_compat.so.2        libstdc++.so
libc-2.19-2014.08-1-git.so   libnss_db-2.19-2014.08-1-git.so libstdc++.so.6
libcrypt-2.19-2014.08-1-git.so libnss_db.so.2            libstdc++.so.6.0.20
libcrypt.so.1                libnss_dns-2.19-2014.08-1-git.so libstdc++.so.6.0.20-gdb.py
libc.so.6                    libnss_dns.so.2           libsupc++.a
libdl-2.19-2014.08-1-git.so  libnss_files-2.19-2014.08-1-git.so libthread_db-1.0.so
libdl.so.2                   libnss_files.so.2         libthread_db.so.1
libgcc_s.so                  libnss_hesiod-2.19-2014.08-1-git.so libubsan.a
libgcc_s.so.1                libnss_hesiod.so.2        libubsan.so
libgfortran.a                libnss_nis-2.19-2014.08-1-git.so libubsan.so.0
libgfortran.so               libnss_nisplus-2.19-2014.08-1-git.so libubsan.so.0.0.0
libgfortran.so.3             libnss_nisplus.so.2       libutil-2.19-2014.08-1-git.so
libgfortran.so.3.0.0         libnss_nis.so.2           libutil.so.1
libgomp.a                    libpcprofile.so
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$
```

图 38.2.3.3 lib 目录

2、向 rootfs 的 “usr/lib” 目录添加库文件

在 rootfs 的 usr 目录下创建一个名为 lib 的目录,将如下目录中的库文件拷贝到 rootfs/usr/lib 目录下:

```
/usr/local/arm/gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi/hf/arm-linux-gnueabi/hf/libc/
usr/lib
```

将此目录下的 so 和.a 库文件都拷贝到 rootfs/usr/lib 目录中, 命令如下:

```
cp *so* *.a /home/zuozhongkai/linux/nfs/rootfs/usr/lib/ -d
```

完成以后的 rootfs/usr/lib 目录如图 38.2.3.4 所示:

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/usr/lib$ ls
libanl.a          libcrypt_pic.a  libnsl.a         libnss_nis_pic.a  librpcsvc_p.a
libanl_p.a        libcrypt.so     libnsl_p.a       libnss_nisplus_pic.a  librt.a
libanl_pic.a      libc.so         libnsl_pic.a     libnss_nisplus.so  librt_p.a
libanl.so         libdl.a         libnsl.so        libnss_nis.so      librt_pic.a
libBrokenLocale.a libdl_p.a       libnss_compat_pic.a  libpthread.a       librt.so
libBrokenLocale_p.a libdl_pic.a     libnss_compat.so  libpthread_nonshared.a  libthread_db_pic.a
libBrokenLocale_pic.a libdl.so        libnss_db_pic.a   libpthread_p.a       libthread_db.so
libBrokenLocale.so libg.a          libnss_db.so      libpthread.so        libutil.a
libc.a            libieee.a       libnss_dns_pic.a  libresolv.a          libutil_p.a
libc_nonshared.a libm.a          libnss_dns.so     libresolv_p.a        libutil_pic.a
libc_p.a          libmcheck.a     libnss_files_pic.a  libresolv_pic.a      libutil.so
libc_pic.a        libm_p.a        libnss_files.so   libresolv_pic.map
libcrypt.a        libm_pic.a      libnss_hesiod_pic.a  libresolv.so
libcrypt_p.a      libm.so         libnss_hesiod.so   librpcsvc.a
```

图 38.2.3.4 rootfs/usr/lib 目录

至此, 根文件系统的库文件就全部添加好了, 可以使用 “du” 命令来查看一下 rootfs/lib 和 rootfs/usr/lib 这两个目录的大小, 命令如下:

```
cd rootfs          //进入根文件系统目录
du ./lib ./usr/lib/ -sh  //查看 lib 和 usr/lib 这两个目录的大小
```

结果如图 38.2.3.5 所示:

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ du ./lib ./usr/lib/ -sh
57M      ./lib
67M      ./usr/lib/
zuozhongkai@ubuntu:~/linux/nfs/rootfs$
```

图 38.2.3.5 lib 和 usr/lib 目录大小

可以看出 lib 和 usr/lib 这两个文件的大小分别为 57MB 和 67MB, 加起来就是 57+67=124MB。非常大! 所以正点原子的 256MB 和 512MB 的 NAND 核心版就不是给初学者准备的, 而是给大批量采购的企业准备的, 还是那句话, 初学者选择 EMMC 版本的。

38.2.4 创建其他文件夹

在根文件系统中创建其他文件夹, 如 dev、proc、mnt、sys、tmp 和 root 等, 创建完成以后如图 38.2.4.1 所示:

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ ls
bin dev lib linuxrc mnt proc root sbin sys tmp usr
zuozhongkai@ubuntu:~/linux/nfs/rootfs$
```

图 38.2.4.1 创建好其他文件夹以后的 rootfs

目前来看, 这个根文件系统好像已经准备好了, 究竟有没有准备好, 直接测一下就知道了!

38.3 根文件系统初步测试

接下来我们使用测试一下前面创建好的根文件系统 rootfs, 测试方法就是使用 NFS 挂载,

uboot 里面的 bootargs 环境变量会设置 “root” 的值，所以我们将 root 的值改为 NFS 挂载即可。在 Linux 内核源码里面有相应的文档讲解如何设置，文档为 [Documentation/filesystems/nfs/nfsroot.txt](#)，格式如下：

```
root=/dev/nfs nfsroot=[<server-ip>:]<root-dir>[,<nfs-options>] ip=<client-ip>:<server-ip>:<gw-
ip>:<netmask>:<hostname>:<device>:<autoconf>:<dns0-ip>:<dns1-ip>
```

<server-ip>：服务器 IP 地址，也就是存放根文件系统主机的 IP 地址，那就是 Ubuntu 的 IP 地址，比如我的 Ubuntu 主机 IP 地址为 192.168.1.250。

<root-dir>：根文件系统的存放路径，比如我的就是 /home/zuozhongkai/linux/nfs/rootfs。

<nfs-options>：NFS 的其他可选项，一般不设置。

<client-ip>：客户端 IP 地址，也就是我们开发板的 IP 地址，Linux 内核启动以后就会使用此 IP 地址来配置开发板。此地址一定要和 Ubuntu 主机在同一个网段内，并且没有被其他的设备使用，在 Ubuntu 中使用 ping 命令 ping 一下就知道要设置的 IP 地址有没有被使用，如果不能 ping 通就说明没有被使用，那么就可以设置为开发板的 IP 地址，比如我就可以设置为 192.168.1.251。

<server-ip>：服务器 IP 地址，前面已经说了。

<gw-ip>：网关地址，我的就是 192.168.1.1。

<netmask>：子网掩码，我的就是 255.255.255.0。

<hostname>：客户机的名字，一般不设置，此值可以空着。

<device>：设备名，也就是网卡名，一般是 eth0，eth1.....，正点原子的 I.MX6U-ALPHA 开发板的 ENET2 为 eth0，ENET1 为 eth1。如果你的电脑只有一个网卡，那么基本只能是 eth0。这里我们使用 ENET2，所以网卡名就是 eth0。

<autoconf>：自动配置，一般不使用，所以设置为 off。

<dns0-ip>：DNS0 服务器 IP 地址，不使用。

<dns1-ip>：DNS1 服务器 IP 地址，不使用。

根据上面的格式 bootargs 环境变量的 root 值如下：

```
root=/dev/nfs nfsroot=192.168.1.250:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw
ip=192.168.1.251:192.168.1.250:192.168.1.1:255.255.255.0::eth0:off
```

“proto=tcp”表示使用 TCP 协议，“rw”表示 nfs 挂载的根文件系统为可读可写。启动开发板，进入 uboot 命令行模式，然后重新设置 bootargs 环境变量，命令如下：

```
setenv bootargs 'console=ttyMxc0,115200 root=/dev/nfs nfsroot=192.168.1.250:
/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.251:192.168.1.250:192.168.1.1:
255.255.255.0::eth0:off' //设置 bootargs
saveenv //保存环境变量
```

设置好以后使用 “boot” 命令启动 Linux 内核，结果如图 38.3.1 所示：


```
usb 1-1.2: new high-speed USB device number 4 using ci_hsrc
fec 20b4000.ethernet eth0: Link is Up - 100Mbps/Full - flow control rx/tx
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
IP-Config: Complete:
    device=eth0, hwaddr=00:04:9f:04:d2:35, ipaddr=192.168.1.251, mask=255.255.255.0, gw=192.168.1.
    host=192.168.1.251, domain=, nis-domain=(none)
    bootserver=192.168.1.250, rootserver=192.168.1.250, rootpath=
gpio_dvfs: disabling
VSD_3V3: disabling
can-3v3: disabling
ALSA device list:
    #0: wm8960-audio
VFS: Mounted root (nfs filesystem) on device 0:14.
devtmpfs: mounted
Freeing unused kernel memory: 444K (80b19000 - 80b88000)
can't run '/etc/init.d/rcS': No such file or directory

Please press Enter to activate this console.
/ #
/ #
/ #
/ #
/ #
```

图 38.3.1 进入根文件系统

从图 38.3.1 可以看出，我们进入了根文件系统，说明我们的根文件系统工作了！如果没有启动进入根文件系统的话可以重启一次开发板试试。我们可以输入“ls”命令测试一下，结果如图 38.3.2 所示：

```
/ # ls
bin      lib      mnt      root     sys      usr
dev      linuxrc  proc     sbin     tmp
/ #
```

图 38.3.2 ls 命令测试

可以看出 ls 命令工作正常！那么是不是说明我们的 rootfs 就制作成功了呢？大家注意，在进入根文件系统的时候会有下面这一行错误提示：

```
can't run '/etc/init.d/rcS': No such file or directory
```

提示很简单，说是无法运行“/etc/init.d/rcS”这个文件，因为这个文件不存在。如图 38.3.3 所示

```
can't run '/etc/init.d/rcS': No such file or directory

Please press Enter to activate this console.
/ #
/ #
```

图 39.3.3 “/etc/init.d/rcS”不存在

看来我们的 rootfs 还是缺文件啊，没什么说的，一步一步的完善吧。

38.4 完善根文件系统

38.4.1 创建/etc/init.d/rcS 文件

rcS 是个 shell 脚本，Linux 内核启动以后需要启动一些服务，而 rcS 就是规定启动哪些文件的脚本文件。在 rootfs 中创建/etc/init.d/rcS 文件，然后在 rcS 中输入如下所示内容：

示例代码 38.4.1.1 /etc/init.d/rcS 文件

```
1 #!/bin/sh
2
3 PATH=/sbin:/bin:/usr/sbin:/usr/bin:$PATH
4 LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib:/usr/lib
5 export PATH LD_LIBRARY_PATH
```

```

6
7 mount -a
8 mkdir /dev/pts
9 mount -t devpts devpts /dev/pts
10
11 echo /sbin/mdev > /proc/sys/kernel/hotplug
12 mdev -s

```

第 1 行，表示这是一个 shell 脚本。

第 3 行，PATH 环境变量保存着可执行文件可能存在的目录，这样我们在执行一些命令或者可执行文件的时候就不会提示找不到文件这样的错误。

第 4 行，LD_LIBRARY_PATH 环境变量保存着库文件所在的目录。

第 5 行，使用 export 来导出上面这些环境变量，相当于声明一些“全局变量”。

第 7 行，使用 mount 命令来挂载所有的文件系统，这些文件系统由文件/etc/fstab 来指定，所以我们一会还要创建/etc/fstab 文件。

第 8 和 9 行，创建目录/dev/pts，然后将 devpts 挂载到/dev/pts 目录中。

第 11 和 12 行，使用 mdev 来管理热插拔设备，通过这两行，Linux 内核就可以在/dev 目录下自动创建设备节点。关于 mdev 的详细内容可以参考 busybox 中的 docs/mdev.txt 文档。

示例代码 38.4.1.1 中的 rcS 文件内容是最精简的，大家如果去看 Ubuntu 或者其他大型 Linux 操作系统中的 rcS 文件，就会发现其非常复杂。因为我们是初次学习，所以不用搞这么复杂的，而且这么复杂的 rcS 文件也是借助其他工具创建的，比如 buildroot 等。

创建好文件/etc/init.d/rcS 以后一定要给其可执行权限！

使用如下命令给予/etc/init.d/rcS 可执行权限：

```
chmod 777 rcS
```

设置好以后就重新启动 Linux 内核，启动以后如图 38.4.1.1 所示：

```

mount: can't read '/etc/fstab': No such file or directory
/etc/init.d/rcS: line 13: can't create /proc/sys/kernel/hotplug: nonexistent directory
mdev: /sys/dev: No such file or directory
Please press Enter to activate this console.

```

图 38.4.1.1 Linux 启动过程

从图 38.4.1.1 可以看到，提示找不到/etc/fstab 文件，还有一些其他的错误，我们先把/etc/fstab 这个错误解决了。说不定把这个问题解决以后其他的错误也就解决了。前面我们说了“mount -a”挂载所有根文件系统的时候需要读取/etc/fstab，因为/etc/fstab 里面定义了该挂载哪些文件，好了，接下来就是创建/etc/fstab 文件。

38.4.2 创建/etc/fstab 文件

在 rootfs 中创建/etc/fstab 文件，fstab 在 Linux 开机以后自动配置哪些需要自动挂载的分区，格式如下：

<file system>	<mount point>	<type>	<options>	<dump>	<pass>
---------------	---------------	--------	-----------	--------	--------

<file system>：要挂载的特殊的设备，也可以是块设备，比如/dev/sda 等等。

<mount point>：挂载点。

<type>：文件系统类型，比如 ext2、ext3、proc、romfs、tmpfs 等等。

<options>：挂载选项，在 Ubuntu 中输入“man mount”命令可以查看具体的选项。一般使用 defaults，也就是默认选项，defaults 包含了 rw、suid、dev、exec、auto、nouser 和 async。

<dump>：为 1 的话表示允许备份，为 0 不备份，一般不备份，因此设置为 0。

<pass>: 磁盘检查设置, 为 0 表示不检查。根目录 ‘/’ 设置为 1, 其他的都不能设置为 1, 其他的分区从 2 开始。一般不在 fstab 中挂载根目录, 因此这里一般设置为 0。

按照上述格式, 在 fstab 文件中输入如下内容:

示例代码 38.4.2.1 /etc/fstab 文件

```

1 #<file system> <mount point> <type> <options> <dump> <pass>
2 proc /proc proc defaults 0 0
3 tmpfs /tmp tmpfs defaults 0 0
4 sysfs /sys sysfs defaults 0 0
    
```

fstab 文件创建完成以后重新启动 Linux, 结果如图 38.4.2.1 所示:

```

VFS: Mounted root (nfs filesystem) on device 0:14.
devtmpfs: mounted
Freeing unused kernel memory: 444K (80b19000 - 80b88000)

Please press Enter to activate this console.
/ #
    
```

图 38.4.2.1 Linux 启动过程

从图 38.4.2.1 可以看出, 启动成功, 而且没有任何错误提示。但是我们要还需要创建一个文件/etc/inittab。

38.4.3 创建/etc/inittab 文件

inittab 的详细内容可以参考 busybox 下的文件 examples/inittab。init 程序会读取/etc/inittab 这个文件, inittab 由若干条指令组成。每条指令的结构都是一样的, 由以 “:” 分隔的 4 个段组成, 格式如下:

<id>:<runlevels>:<action>:<process>

<id>: 每个指令的标识符, 不能重复。但是对于 busybox 的 init 来说, <id>有着特殊意义。对于 busybox 而言<id>用来指定启动进程的控制 tty, 一般我们将串口或者 LCD 屏幕设置为控制 tty。

<runlevels>: 对 busybox 来说此项完全没用, 所以空着。

<action>: 动作, 用于指定<process>可能用到的动作。busybox 支持的动作如表 38.4.3.1 所示:

动作	描述
sysinit	在系统初始化的时候 process 才会执行一次。
respawn	当 process 终止以后马上启动一个新的。
askfirst	和 respawn 类似, 在运行 process 之前在控制台上显示“Please press Enter to activate this console.”。只要用户按下 “Enter” 键以后才会执行 process。
wait	告诉 init, 要等待相应的进程执行完以后才能继续执行。
once	仅执行一次, 而且不会等待 process 执行完成。
restart	当 init 重启的时候才会执行 procee。
ctrlaltdel	当按下 ctrl+alt+del 组合键才会执行 process。
shutdown	关机的时候执行 process。

表 38.4.3.1 动作

<process>: 具体的动作, 比如程序、脚本或命令等。

参考 busybox 的 examples/inittab 文件, 我们也创建一个/etc/inittab, 在里面输入如下内容:

示例代码 38.4.3.1 /etc/inittab 文件

```

1 #etc/inittab
    
```

```
2 ::sysinit:/etc/init.d/rcS
3 console::askfirst:-/bin/sh
4 ::restart:/sbin/init
5 ::ctrlaltdel:/sbin/reboot
6 ::shutdown:/bin/umount -a -r
7 ::shutdown:/sbin/swapoff -a
```

第 2 行，系统启动以后运行/etc/init.d/rcS 这个脚本文件。

第 3 行，将 console 作为控制台终端，也就是 ttyxc0。

第 4 行，重启的话运行/sbin/init。

第 5 行，按下 ctrl+alt+del 组合键的话就运行/sbin/reboot，看来 ctrl+alt+del 组合键用于重启系统。

第 6 行，关机的时候执行/bin/umount，也就是卸载各个文件系统。

第 7 行，关机的时候执行/sbin/swapoff，也就是关闭交换分区。

/etc/inittab 文件创建好以后就可以重启开发板即可，至此！根文件系统要创建的文件就已经全部完成了。接下来就要对根文件系统进行其他的测试，比如我们自己编写的软件运行是否正常、是否支持软件开机自启动、中文支持是否正常以及能不能链接等。

38.5 根文件系统其他功能测试

38.5.1 软件运行测试

我们使用 Linux 的目的就是运行我们自己的软件，我们编译的应用软件一般都使用动态库，使用动态库的话应用软件体积就很小，但是得提供库文件，库文件我们已经添加到了根文件系统中。我们编写一个小小的测试软件来测试一下库文件是否工作正常，在根文件系统下创建一个名为“drivers”的文件夹，以后我们学习 Linux 驱动的时候就把所有的实验文件放到这个文件夹里面。

在 ubuntu 下使用 vim 编辑器新建一个 hello.c 文件，在 hello.c 里面输入如下内容：

示例代码 38.5.1.1 hello.c 文件

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     while(1) {
6         printf("hello world!\r\n");
7         sleep(2);
8     }
9     return 0;
10 }
```

hello.c 内容很简单，就是循环输出“hello world”，sleep 相当于 Linux 的延时函数，单位为秒，所以 sleep(2)就是延时 2 秒。编写好以后就是编译，因为我们是要在 ARM 芯片上运行的，所以要用交叉编译器去编译，也就是使用 arm-linux-gnueabi-gcc 编译，命令如下：

```
arm-linux-gnueabi-gcc hello.c -o hello
```

使用 arm-linux-gnueabi-gcc 将 hello.c 编译为 hello 可执行文件。这个 hello 可执行文件究竟是不是 ARM 使用的呢？使用“file”命令查看文件类型以及编码格式：

```
file hello    //查看 hello 的文件类型以及编码格式
```

结果如图 38.5.1.1 所示:

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/drivers$ arm-linux-gnueabi-gcc hello.c -o hello
zuozhongkai@ubuntu:~/linux/nfs/rootfs/drivers$ ls
hello  hello.c
zuozhongkai@ubuntu:~/linux/nfs/rootfs/drivers$ file hello
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-, for GNU/Linux 2.6.31, BuildID[sha1]=7dd1bde89e09327b11ad95e22e72f9bfafd8aebb, not stripped
zuozhongkai@ubuntu:~/linux/nfs/rootfs/drivers$
```

查看hello编码格式

图 38.5.1.1 查看 hello 编码格式

从图 38.5.1.1 可以看出, 输入 “file hello” 输入了如下所示信息:

```
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked.....
```

hello 是个 32 位的 LSB 可执行文件, ARM 架构的, 并且是动态链接的。所以我们编译出来的 hello 文件没有问题。将其拷贝到 rootfs/drivers 目录下, 在开发板中输入如下命令来执行这个可执行文件:

```
cd /drivers    //进入 drivers 目录
```

```
./hello        //执行 hello
```

结果如图 38.5.1.2 所示:

```
/ # cd drivers/
/drivers # ./hello
hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
/drivers #
```

执行hello

运行结果

图 38.5.1.2 hello 运行结果

可以看出, hello 这个软件运行正常, 说明我们的根文件系统中的共享库是没问题的, 要想终止 hello 的运行, 按下 “ctrl+c” 组合键即可。此时大家应该能感觉到, hello 执行的时候终端是没法用的, 除非使用 “ctrl+c” 来关闭 hello, 那么有没有办法既能让 hello 正常运行, 而且终端能够正常使用? 那肯定是有的, 让 hello 进入后台运行就行了, 让一个软件进入后台的方法很简单, 运行软件的时候加上 “&” 即可, 比如 “./hello &” 就是让 hello 在后台运行。在后台运行的软件可以使用 “kill -9 pid(进程 ID)” 命令来关闭掉, 首先使用 “ps” 命令查看要关闭的软件 PID 是多少, ps 命令用于查看所有当前正在运行的进程, 并且会给出进程的 PID。输入 “ps” 命令, 结果如图 38.5.1.3 所示:

```

/drivers # ps
PID  USER  TIME  COMMAND
1  0      0:01  {linuxrc} init
2  0      0:00  [kthreadd]
3  0      0:00  [ksoftirqd/0]
5  0      0:00  [kworker/0:0H]
6  0      0:00  [kworker/u2:0]
7  0      0:04  [rcu_preempt]
8  0      0:00  [rcu_sched]
9  0      0:00  [rcu_bh]
10 0      0:00  [migration/0]
11 0      0:00  [khelper]
12 0      0:00  [kdevtmpfs]
13 0      0:00  [perf]
14 0      0:00  [writeback]
15 0      0:00  [crypto]
16 0      0:00  [bioset]
17 0      0:00  [kblockd]
18 0      0:00  [ata_sff]
20 0      0:00  [cfg80211]
21 0      0:00  [rpciod]
22 0      0:00  [kswapd0]
23 0      0:00  [fsnotify_mark]
24 0      0:00  [nfsiod]
61 0      0:00  [kworker/u2:1]
66 0      0:00  [ci_otg]
67 0      0:00  [irq/21-2040000.]
68 0      0:00  [cfinteractive]
69 0      0:00  [irq/225-mmc0]
71 0      0:00  [irq/50-2190000.]
72 0      0:00  [irq/226-mmc1]
73 0      0:00  [mxs_dcp_chan/sh]
74 0      0:00  [mxs_dcp_chan/ae]
81 0      0:00  [ipv6_addrconf]
82 0      0:00  [krfcomm]
83 0      0:00  [pxp_dispatch]
84 0      0:00  [deferwq]
85 0      0:00  [irq/205-imx_the]
87 0      0:00  [kworker/0:1H]
93 0      0:00  -/bin/sh
97 0      0:00  [mmcqd/0]
103 0      0:00  [mmcqd/1]
104 0      0:00  [mmcqd/lboot0]
105 0      0:00  [mmcqd/lboot1]
106 0      0:00  [mmcqd/lrpn]
133 0      0:00  udhcpc -i eth1
163 0      0:00  [kworker/0:0]
164 0      0:00  [kworker/0:2]
165 0      0:00  [kworker/0:1]
166 0      0:00  ./hello
168 0      0:00  ps

```

图 38.5.1.3 ps 命令结果

从图 38.5.1.3 可以看出 hello 对应的 PID 为 166，因此我们使用如下命令关闭在后台运行的 hello 软件：

```
kill -9 166
```

因为 hello 在不断的输出“hello world”所以我们的输入看起来会被打断，其实是没有的，因为我们是输入，而 hello 是输出。在数据流上是没有打断的，只是显示在 SecureCRT 上就好像被打断了，所以只管输入“kill -9 166”即可。hello 被 kill 以后会有提示，如图 38.5.1.4 所示：

```

[1]+  Killed                  ./hello
/drivers #

```

图 38.5.1.4 提示 hello 被 kill 掉。

再去用 ps 命令查看一下当前的进程，发现没有 hello 了。这个就是 Linux 下的软件后台运行以及如何关闭软件的方法，重点就是 3 个操作：软件后面加“&”、使用 ps 查看要关闭的软件 PID、使用“kill -9 pid”来关闭指定的软件。

38.5.2 中文字符测试

1、设置 SecureCRT 使用 UTF-8 编码

因为 Linux 使用的编码格式为 UTF-8，因此要先设置 SecureCRT 的编码格式。打开 Options->Session Options...，打开“Session Options”对话框，选择左侧的“Appearance”，然后在右侧的“Character encoding:”栏选择 UTF-8 编码，如图 38.5.2.1 所示：

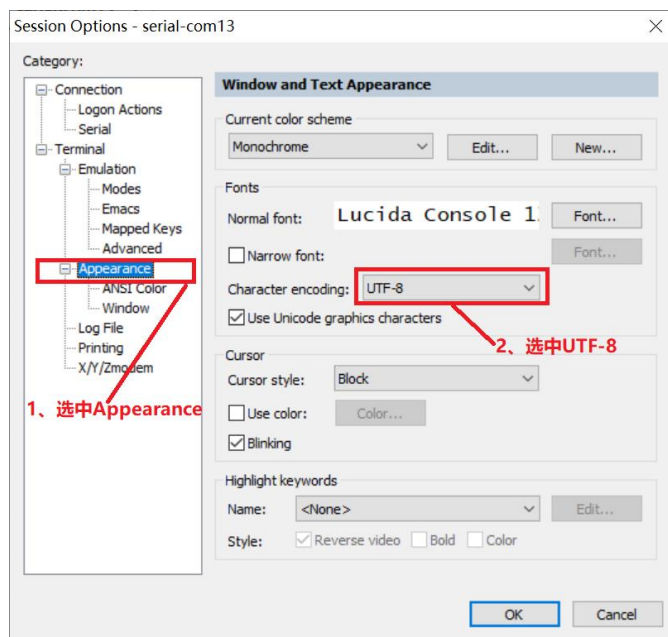


图 38.5.2.1 设置 UTF-8 编码

设置好以后点击下方的“Ok”按钮即可，SecureCRT 我们就设置好了。

2、创建中文文件

在 ubuntu 中向在 rootfs 目录新建一个名为“中文测试”的文件夹，然后在 SecureCRT 下查看中文名能不能显示正确。输入“ls”命令，结果如图 38.5.2.2 所示：

```
/drivers # cd /  
/ # ls  
bin          etc          mnt          sbin         usr  
dev          lib          proc         sys          中文测试  
drivers      linuxrc     root         tmp  
/ #
```

图 38.5.2.2 中文文件夹测试

可以看出“中文测试”这个文件夹显示正常，接着“touch”命令在“中文测试”文件夹中新建一个名为“测试文档.txt”的文件，并且使用 vim 编辑器在其中输入“这是一个中文测试文件”，借此来测试一下中文文件名和中文内容显示是否正常。在 SecureCRT 中使用“cat”命令来查看“测试文档.txt”中的内容，结果如图 38.5.2.3 所示：

```
/ # cd 中文测试/  
/中文测试 # ls  
测试文档.txt  
/中文测试 # cat 测试文档.txt  
这是一个中文测试文件  
/中文测试 #
```

1、cat命令查看“测试文档.txt”内容

2、文档内容

图 38.5.2.3 中文文档内容显示

从图 38.5.2.3 可以看出，“测试文档.txt”的中文内容显示正确，而且中文路径也完全正常，说明我们的根文件系统已经完美支持中文了！

38.5.3 开机自启动测试

在 38.5.1 小节测试 hello 软件的时候都是等 Linux 启动进入根文件系统以后手动输入命令“./hello”来完成的。我们一般做好产品以后都是需要开机自动启动相应的软件，本节我们就以

hello 这个软件为例, 讲解一下如何实现开机自启动。前面我们说过了, 进入根文件系统的时候会运行/etc/init.d/rcS 这个 shell 脚本, 因此我们可以在这个脚本里面添加自启动相关内容。添加完成以后的/etc/init.d/rcS 文件内容如下:

示例代码 38.5.3.1 rcS 文件代码

```
1 #!/bin/sh
2 PATH=/sbin:/bin:/usr/sbin:/usr/bin
3 LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib:/usr/lib
4 runlevel=S
5 umask 022
6 export PATH LD_LIBRARY_PATH runlevel
7
8 mount -a
9 mkdir /dev/pts
10 mount -t devpts devpts /dev/pts
11
12 echo /sbin/mdev > /proc/sys/kernel/hotplug
13 mdev -s
14
15 #开机自启动
16 cd /drivers
17 ./hello &
18 cd /
```

第 16 行, 进入 drivers 目录, 因为要启动的软件存放在 drivers 目录下。

第 17 行, 以后台方式执行 hello 这个软件。

第 18 行, 退出 drivers 目录, 进入到根目录下。

自启动代码添加完成以后就可以重启开发板, 看看 hello 这个软件会不会自动运行。结果如图 38.5.3.1 所示:

```
#0: wm8960-audio
VFS: Mounted root (nfs filesystem) on device 0:14.
devtmpfs: mounted
Freeing unused kernel memory: 444K (80b19000 - 80b88000)
nfs: server 192.168.1.250 not responding, still trying
nfs: server 192.168.1.250 OK

Please press Enter to activate this console. hello world!
hello world!
hello world!
hello world!
hello world!
hello world!
```

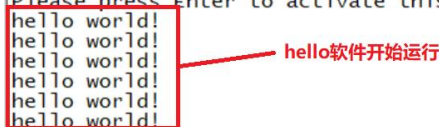


图 38.5.3.1 hello 开机自启动

从图 38.5.3.1 可以看出, hello 开机自动运行了, 说明开机自启动成功。

38.5.4 外网连接测试

这里说的外网不是外国哪些 404 网站的连接测试, 而是百度、淘宝等这些网站的测试。也就是说看看我们的开发板能不能上网, 能不能和我们的局域网外的这些网站进行通信。测试方法很简单, 就是通过 ping 命令来 ping 一下百度的官网: www.baidu.com。输入如下命令:

```
ping www.baidu.com
```

结果如图 38.5.4.1 所示:


```
/ # ping www.baidu.com
ping: bad address 'www.baidu.com'
/ #
```

图 38.5.4.1 ping 测试结果

可以看出，测试失败，提示 www.baidu.com 是个 “bad address”，也就是地址不对，显然我们的地址是正确的。之所以出现这个错误提示是因为 www.baidu.com 的地址解析失败了，并没有解析出其对应的 IP 地址。我们需要配置域名解析服务器的 IP 地址，一般域名解析地址可以设置为所处网络的网关地址，比如 192.168.1.1。也可以设置为 114.114.114.114，这个是运营商的域名解析服务器地址。

在 rootfs 中新建文件/etc/resolv.conf，然后在里面输入如下内容：

示例代码 38.5.4.1 resolv.conf 文件内容

```
1 nameserver 114.114.114.114
2 nameserver 192.168.1.1
```

设置很简单，nameserver 表示这是个域名服务器，设置了两个域名服务器地址：114.114.114.114 和 192.168.1.1，大家也可以改为其他的域名服务器试试。如果使用 “udhcpc” 命令自动获取 IP 地址，“udhcpc” 命令会修改 nameserver 的值，一般是将其设置为对应的网关地址。修改好以后保存退出，重启开发板！重启以后重新 ping 一下百度官网，结果如图 38.5.4.2 所示：

```
/ # ping www.baidu.com
PING www.baidu.com (14.215.177.39): 56 data bytes
64 bytes from 14.215.177.39: seq=0 ttl=56 time=9.591 ms
64 bytes from 14.215.177.39: seq=1 ttl=56 time=10.127 ms
64 bytes from 14.215.177.39: seq=2 ttl=56 time=13.850 ms
64 bytes from 14.215.177.39: seq=3 ttl=56 time=10.878 ms
64 bytes from 14.215.177.39: seq=4 ttl=56 time=9.816 ms
64 bytes from 14.215.177.39: seq=5 ttl=56 time=10.515 ms
^C
--- www.baidu.com ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 9.591/10.796/13.850 ms
/ #
```

图 38.5.4.2 ping 百度官网结果

可以看出 ping 百度官网成功了！域名也成功的解析了，至此！我们的根文件系统就彻底的制作完成，这个根文件系统最好打包保存一下，防止以后做实验不小心破坏了根文件系统而功亏一篑，又得从头制作根文件系统。uboot、Linux kernel、rootfs 这三个共同构成了一个完整的 Linux 系统，现在的系统至少是一个可以正常运行的系统，后面我们就可以在这个系统上完成 Linux 驱动开发的学习。

第三十九章 系统烧写

前面我们已经移植好了 uboot 和 linux kernel, 制作好了根文件系统。但是我们移植都是通过网络来测试的, 在实际的产品开发中肯定不可能通过网络来运行, 否则没网的时候产品岂不是就歇菜了。因此我们需要将 uboot、linux kernel、.dtb(设备树)和 rootfs 这四个文件烧写到板子上的 EMMC、NAND 或 QSPI Flash 等其他存储设备上, 这样不管有没有网络我们的产品都可以正常运行。本章我们就来学习一下如何使用 NXP 官方提供的 MfgTool 工具通过 USB OTG 口来烧写系统。

39.1 MfgTool 工具简介

MfgTool 工具是 NXP 提供的专门用于给 I.MX 系列 CPU 烧写系统的软件，可以在 NXP 官网下载到。此工具已经放到了开发板光盘中，路径为：**5、开发工具->3、NXP 官方原版 MFG_TOOL 烧写工具->L4.1.15_2.0.0-ga_mfg-tools.tar.gz**。此软件在 Windows 下使用，对于我们来说太友好了。将此压缩包进行解压，解压完成以后会出现一个名为 L4.1.15_2.0.0-ga_mfg-tools 的文件夹，进入此文件夹，此文件夹的内容如图 39.1.1 所示：

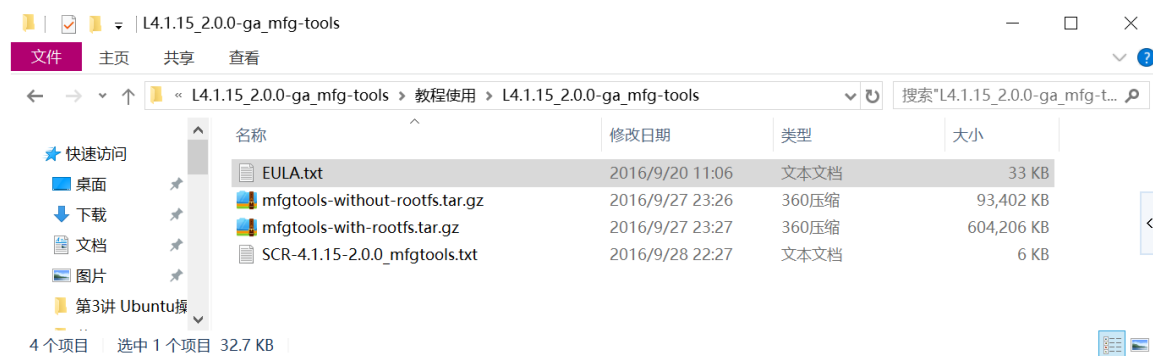


图 39.1.1 mfg_tools 工具目录

从图 39.1.1 可以看出，有两个.txt 文件和两个.gz 压缩包。.txt 文档就不去看了，重点是这两个.gz 压缩包，这两个压缩包的区别在名字上已经写的很详细了。“without-rootfs”和“with-rootfs”，一个是带 rootfs 和一个是不带 rootfs。mfg_tools 这个工具本意是给 NXP 自己的开发板设计的烧写软件，所以肯定带有自家开发板对应的 uboot、linux kernel 和 rootfs 的文件。我们肯定是要烧写文件系统的，所以选择 mfgtools-with-rootfs.tar.gz 这个压缩包，继续对其解压，解压出一个名为 mfgtools-with-rootfs 的文件夹，此文件夹就包含我们需要的烧写工具。

进入目录 mfgtools-with-rootfs\mfgtools 中，在此目录下有几个文件夹和很多的.vbs 文件，如图 39.1.2 所示：

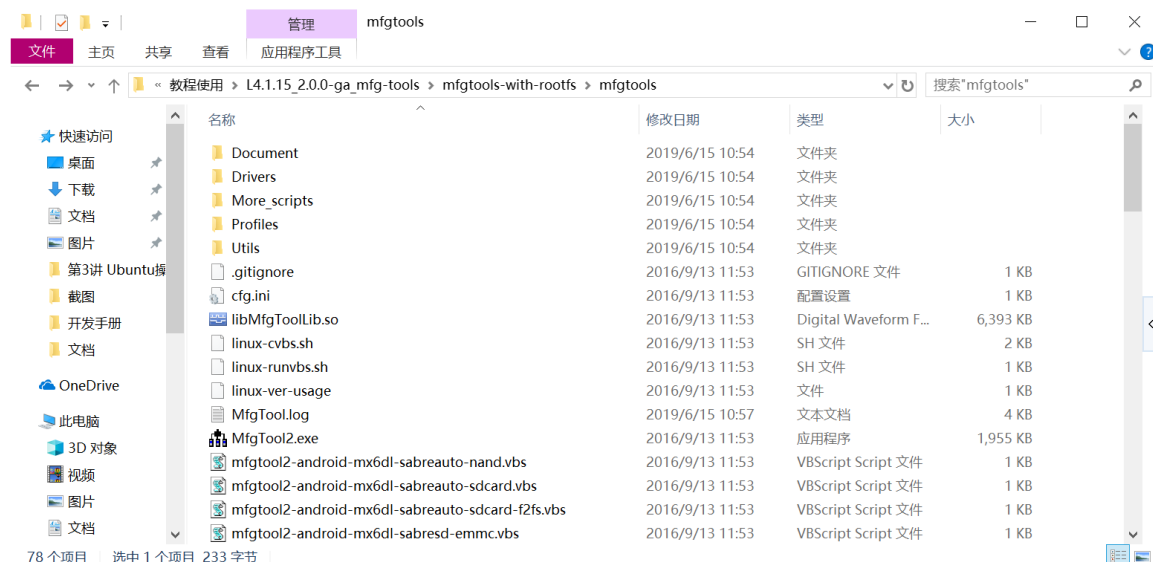


图 39.1.2 mfgtools 目录内容

我们只关心图 39.1.2 中 Profiles 这个文件夹，因为后面要烧写文件就放到这个文件夹中。MfgTool2.exe 就是烧写软件，但是我们不会直接打开这个软件烧写，mfg_tools 不仅能烧写 I.MX6U，而且也能给 I.MX7、I.MX6Q 等芯片烧写，所以在烧写之前必须要进行配置，指定烧

写的是什么芯片，烧写到哪里去？下面的这些众多的.vbs 文件就是配置脚本，烧写的时候通过双击这些.vbs 文件来打开烧写工具。这些.vbs 烧写脚本既可以根据处理器的不同，由用户选择向 I.MX6D、I.MX6Q、I.MX6S、I.MX7、I.MX6UL 和 I.MX6ULL 等的哪一款芯片烧写系统。也可以根据存储芯片的不同，选择向 EMMC、NAND 或 QSPI Flash 等的哪一种存储设备烧写，功能非常强大!! 我们现在需要向 I.MX6U 烧写系统，因此需要参考表 39.1.1 所示的 5 个烧写脚本：

脚本文件	描述
mfgtool2-yocto-mx-evk-emmc.vbs	EMMC 烧写脚本。
mfgtool2-yocto-mx-evk-nand.vbs	NAND 烧写脚本
mfgtool2-yocto-mx-evk-qspi-nor-n25q256a.vbs	QSPI Flash 烧写脚本，型号为 n25q256a
mfgtool2-yocto-mx-evk-sdcard-sd1.vbs	如果 SD1 和 SD2 接的 SD 卡，这两个文件分别向 SD1 和 SD2 上的 SD 卡烧写系统。
mfgtool2-yocto-mx-evk-sdcard-sd2.vbs	

表 39.1.1 I.MX6U 使用的烧写脚本

其他的.vbs 烧写脚本用不到，因此可以删除掉，防止干扰我们的视线。本书用的是正点原子的 EMMC 版核心板，因此只会用到 mfgtool2-yocto-mx-evk-emmc.vbs 这个烧写脚本，如果用其他的核心板请参考相应的烧写脚本。

39.2 MfgTool 工作原理简介

MfgTool 只是个工具，具体的原理不需要去深入研究，大概来了解一下其工作原理就行了，知道它的工作流程就行了。

39.2.1 烧写方式

1、连接 USB 线

MfgTool 是通过 USB OTG 接口将系统烧写进 EMMC 中的，正点原子 I.MX6U-ALPHA 开发板 V2.4 版本以前的底板上 USB OTG 口如图 39.2.1.1 所示：

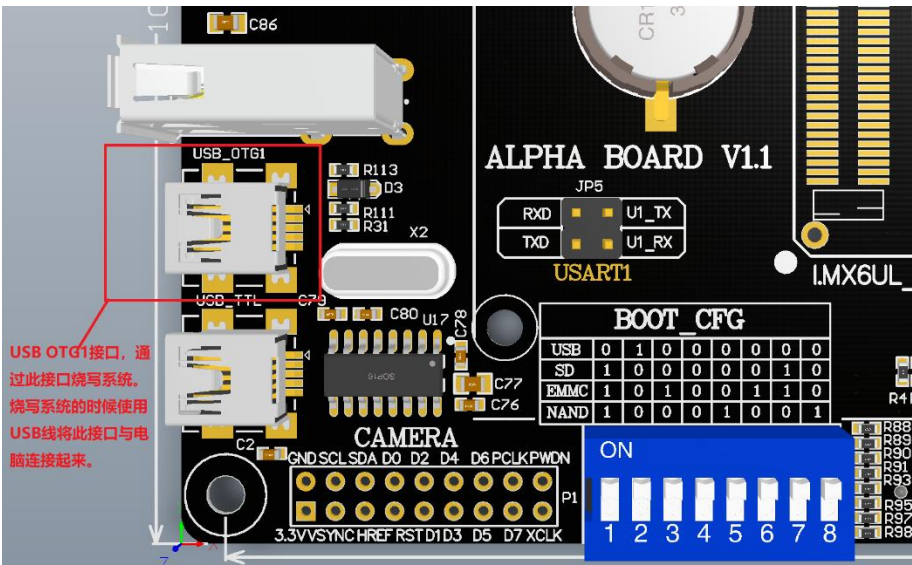


图 39.2.1.1 V2.4 以前版本底板上的 USB OTG1 接口

V2.4 及以后版本底板上 USB OTG 换为了 Type-C，接口如图 39.2.1.2 所示：

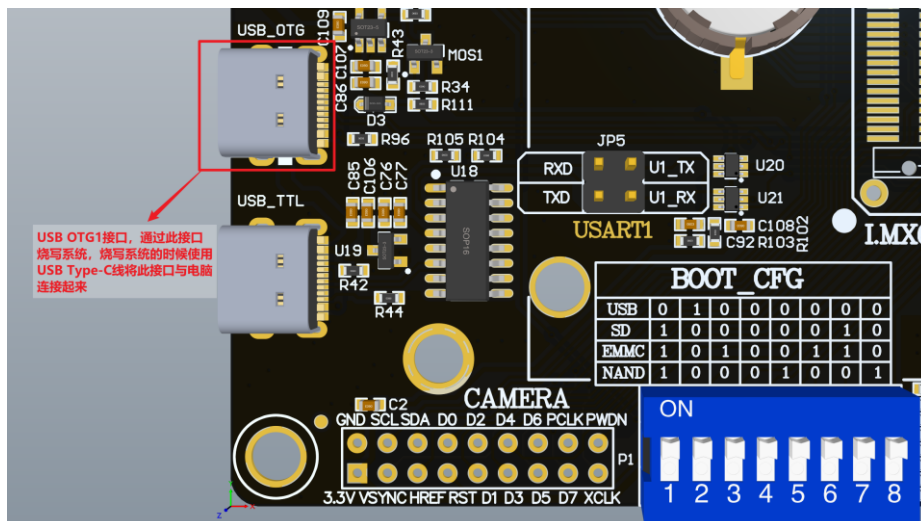


图 39.2.1.2 V2.4 及以后版本底板上的 USB OTG1 接口

在烧写之前, 需要先用 USB 线将图 39.2.1.1 或图 39.2.1.2 中的 USB_OTG1 接口与电脑连接起来。

2、拨码开关拨到 USB 下载模式

将拨码开关拨到 “USB” 模式, 如图 39.2.1.3 所示:

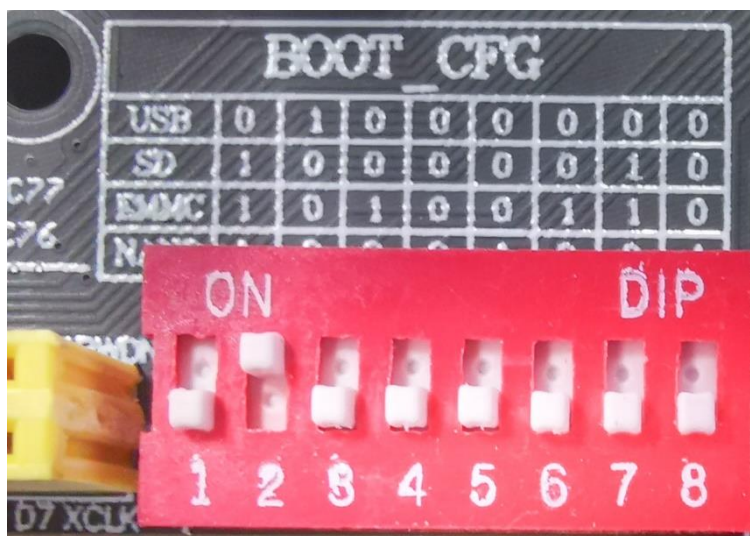


图 39.2.1.3 USB 下载模式

如果插了 TF 卡, 请弹出 TF 卡, 否则电脑不能识别 USB! 等识别出来以后再插上 TF 卡!

一切准备就绪以后, 按一下开发板的复位键, 此时就会进入到 USB 模式, 如果是第一次进入 USB 模式的话可能会久一点, 这个是免驱的, 因此不需要安装驱动。第一次进入 USB 模式会在电脑右下角有如图 39.2.1.4 所示提示:

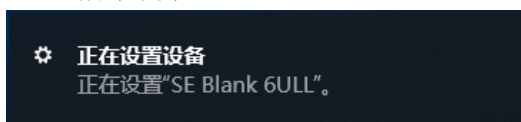


图 39.2.1.4 第一次进入 USB 模式

一旦第一次设置好设备以后, 后面每次连接都不会有任何提示了。到这里, 我们的开发板已经和电脑连接好了, 可以开始烧写系统了。

39.2.2 系统烧写原理

开发板连接电脑以后双击“mfgtool2-yocto-mx-evk-emmc.vbs”，打开下载对话框，如图 39.2.2.1 所示：



图 39.2.2.1 MfgTool 工具界面

如果出现“符合 HID 标准的供应商定义设备”就说明连接正常，可以进行烧写，如果出现其他的字符那么就要检查连接是否正确。点击“Start”按钮即可开始烧写，烧写什么东西呢？肯定是烧写 uboot、Linux kernel、.dtb 和 rootfs，那么这四个应该放到哪里 MfgTool 才能访问到呢？进入如下目录中：

L4.1.15_2.0.0-ga_mfg-tools/mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware

此目录中的文件如图 39.2.2.2 所示：

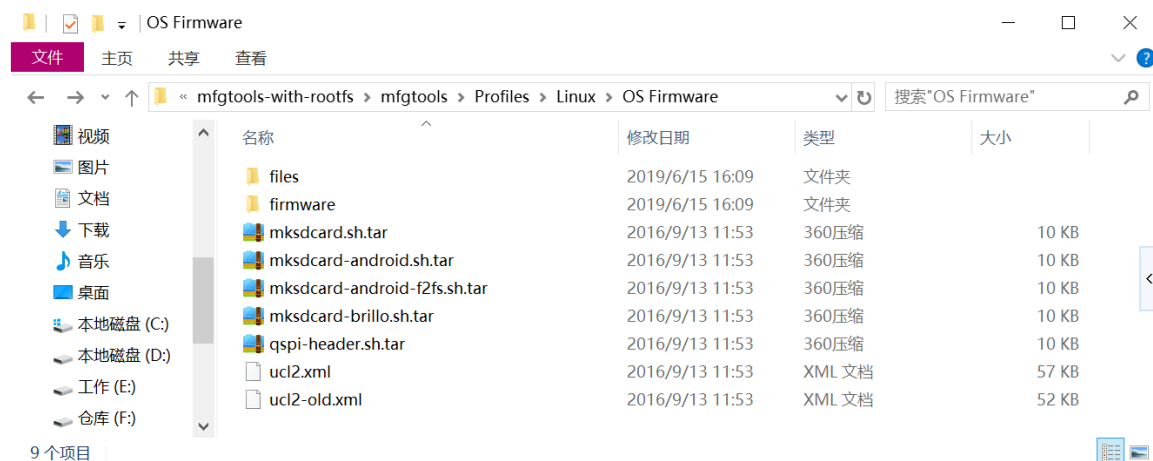


图 39.2.2.2 OS Firmware 文件夹内容

文件夹“OS Firmware”看名字就知道是存放系统固件的，我们重点关注 files、firmware 这两个文件夹，以及 ucl2.xml 这个文件。在具体看这三个文件和文件夹之前，我们先来简单了解一下 MfgTool 烧写的原理，MfgTool 其实是先通过 USB OTG 先将 uboot、kernel 和 .dtb(设备树)这三个文件下载到开发板的 DDR 中，注意不需要下载 rootfs。就相当于直接在开发板的 DDR 上启动 Linux 系统，等 Linux 系统启动以后再向 EMMC 中烧写完整的系统，包括 uboot、linux kernel、.dtb(设备树)和 rootfs，因此 MfgTool 工作过程主要分两个阶段：

①、将 firmware 目录中的 uboot、linux kernel 和 .dtb(设备树)，然后通过 USB OTG 将这个文件下载到开发板的 DDR 中，目的就是在 DDR 中启动 Linux 系统，为后面的烧写做准备。

②、经过第①步的操作，此时 Linux 系统已经运行起来了，系统运行起来以后就可以很方便的完成对 EMMC 的格式化、分区等操作。EMMC 分区建立好以后就可以从 files 中读取要烧写的 uboot、linux kernel、.dtb(设备树)和 rootfs 这 4 个文件，然后将其烧写到 EMMC 中，这个就是 MfgTool 的大概工作流程。

1、firmware 文件夹

打开 firmware 文件夹，里面有很多的.imx 结尾的 uboot 文件、一个 zImage 镜像文件、很多.dtb 结尾的设备树文件。这些文件都是 NXP 官方开发板使用的，不同的板子使用不同的文件，其中我们需要关心的只有表 39.2.2.1 中的这三个文件：

脚本文件	描述
zImage	NXP 官方 I.MX6ULL EVK 开发板的 Linux 镜像文件。
u-boot-imx6ull14x14evk_emmc.imx	NXP 官方 I.MX6ULL EVK 开发板的 uboot 文件。
zImage-imx6ull-14x14-evk-emmc.dtb	NXP 官方 I.MX6ULL EVK 开发板的设备树

表 39.2.2.1 I.MX6ULL EVK 开发板使用的系统文件

表 39.2.2.1 中的这三个文件就是 I.MX6ULL EVK 开发板烧写系统的时候第一阶段所需的文件。如果要烧写我们的系统，就需要用我们编译出来的 zImage、u-boot.imx 和 imx6ull-alientek-emmc.dtb 这三个文件替换掉表 39.2.2.1 中这三个文件。但是名字要和表 39.2.2.1 中的一致，因此需要将 u-boot.imx 重命名为 u-boot-imx6ull14x14evk_emmc.imx，将 imx6ull-alientek-emmc.dtb 重命名为 zImage-imx6ull-14x14-evk-emmc.dtb。

2、files 文件夹

将表 39.2.2.1 中的这三个文件下载到开发板的 DDR 上以后烧写的第一阶段就完成了，第二阶段就是从 files 目录中读取整个系统文件，并将其烧写到 EMMC 中。files 目录中的文件和 firmware 目录中的基本差不多，都是不同板子对应的 uboot、设备树文件，同样，我们只关心表 39.2.2.2 中的四个文件：

脚本文件	描述
zImage	NXP 官方 I.MX6ULL EVK 开发板的 Linux 镜像文件。
u-boot-imx6ull14x14evk_emmc.imx	NXP 官方 I.MX6ULL EVK 开发板的 uboot 文件。
zImage-imx6ull-14x14-evk-emmc.dtb	NXP 官方 I.MX6ULL EVK 开发板的设备树
rootfs_nogpu.tar.bz2	根文件系统，注意和另外一个 rootfs.tar.bz2 根文件系统区分开。nogpu 表示此根文件系统不包含 GPU 的内容，I.MX6ULL 没有 GPU，因此要使用此根文件系统

表 39.2.2.2 I.MX6ULL EVK 开发板烧写文件

如果要烧写我们自己编译出来的系统，就需要用我们编译出来的 zImage、u-boot.imx 和 imx6ull-alientek-emmc.dtb 和 rootfs 这四个文件替换掉表 39.2.2.2 中这四个文件。

3、ucl2.xml 文件

files 和 firmware 目录下有众多的 uboot 和设备树，那么烧写的时候究竟选择哪一个呢？这个工作就是由 ucl2.xml 文件来完成的。ucl2.xml 以“<UCL>”开始，以“</UCL>”结束。“<CFG>”和“</CFG>”之间是配置相关内容，主要是判断当前是给 I.MX 系列的哪个芯片烧写系统。“<LIST>”和“</LIST>”之间的是针对不同存储芯片的烧写命令。整体框架如下：

示例代码 39.2.2.1 ucl2.xml 框架

```
<UCL>
  <CFG>
    .....
    <!-- 判断向 I.MX 系列的哪个芯片烧写系统 -->
    .....
  </CFG>

  <LIST name="SDCard" desc="Choose SD Card as media">
```

```

<!-- 向 SD 卡烧写 Linux 系统 -->
</LIST>

<LIST name="eMMC" desc="Choose eMMC as media">
<!-- 向 EMMC 烧写 Linux 系统 -->
</LIST>

<LIST name="Nor Flash" desc="Choose Nor flash as media">
<!-- 向 Nor Flash 烧写 Linux 系统 -->
</LIST>

<LIST name="Quad Nor Flash" desc="Choose Quad Nor flash as media">
<!-- 向 Quad Nor Flash 烧写 Linux 系统 -->
</LIST>

<LIST name="NAND Flash" desc="Choose NAND as media">
<!-- 向 NAND Flash 烧写 Linux 系统 -->
</LIST>

<LIST name="SDCard-Android" desc="Choose SD Card as media">
<!-- 向 SD 卡烧写 Android 系统 -->
</LIST>

<LIST name="eMMC-Android" desc="Choose eMMC as media">
<!-- 向 EMMC 烧写 Android 系统 -->
</LIST>

<LIST name="Nand-Android" desc="Choose NAND as media">
<!-- 向 NAND Flash 烧写 Android 系统 -->
</LIST>

<LIST name="SDCard-Brillo" desc="Choose SD Card as media">
<!-- 向 SD 卡烧写 Brillo 系统 -->
</LIST>
</UCL>

```

ucl2.xml 首先会判断当前要向 I.MX 系列的哪个芯片烧写系统，代码如下：

示例代码 39.2.2.2 判断要烧写的处理器型号

```

21 <CFG>
22 <STATE name="BootStrap" dev="MX6SL" vid="15A2" pid="0063"/>
23 <STATE name="BootStrap" dev="MX6D" vid="15A2" pid="0061"/>
24 <STATE name="BootStrap" dev="MX6Q" vid="15A2" pid="0054"/>
25 <STATE name="BootStrap" dev="MX6SX" vid="15A2" pid="0071"/>
26 <STATE name="BootStrap" dev="MX6UL" vid="15A2" pid="007D"/>

```



```

27 <STATE name="BootStrap" dev="MX7D" vid="15A2" pid="0076"/>
28 <STATE name="BootStrap" dev="MX6ULL" vid="15A2" pid="0080"/>
29 <STATE name="Updater" dev="MSC" vid="066F" pid="37FF"/>
30 </CFG>

```

通过读取芯片的 VID 和 PID 即可判断出当前要烧写什么处理器的系统,如果 VID=0X15A2, PID=0080, 那么就表示要给 I.MX6ULL 烧写系统。确定了处理器以后就要确定向什么存储设备烧写系统, 这个时候就要有请 mfgtool2-yocto-mx-evk-emmc.vbs 再次登场, 此文件内容如下:

示例代码 39.2.2.3 mfgtool2-yocto-mx-evk-emmc.vbs 文件内容

```

Set wshShell = CreateObject("WScript.shell")
wshShell.run "mfgtool2.exe -c "linux" -l "eMMC" -s
"board=sabresd" -s "mmc=1" -s "6uluboot=14x14evk" -s
"6uldtb=14x14-evk"
Set wshShell = Nothing

```

重点是“wshShell.run”这一行, 这里一行调用了 mfgtool2.exe 这个软件, 并且还给出了一堆的参数, 其中就有“eMMC”字样, 说明是向 EMMC 烧写系统, 要烧写的存储设备就这样确定下来了。“wshShell.run”后面还有一堆的其他参数, 这些参数都有对应的值, 如下所示:

```

board=sabresd
mmc=1
6uluboot=14x14evk
6uldtb=14x14-evk

```

我们继续回到 ucl2.xml 中, 既然现在已经知道了是向 I.MX6ULL 的 EMMC 中烧写系统, 那么直接在 ucl2.xml 中找到相应的烧写命令就行了, 因为相应的命令太长, 为了缩小篇幅, 我们就以 uboot 的烧写为例讲解一下。前面说了烧写分两个阶段, 第一步是通过 USB OTG 向 DDR 中下载系统, 第二步才是正常的烧写。通过 USB OTG 向 DDR 下载 uboot 的命令如下:

示例代码 39.2.2.4 通过 USB OTG 下载 uboot

```

<CMD state="BootStrap" type="boot" body="BootStrap" file = "firmware/u-
boot-imx6ul%lite%%6uluboot%_emmc.imx" ifdev="MX6ULL">Loading U-boot
</CMD>

```

上面的命令就是 BootStrap 阶段, 也就是第一阶段, “file”表示要下载的文件位置, 在 firmware 目录下, 文件名字为

```
u-boot-imx6ul%lite%%6uluboot%_emmc.imx
```

在 L4.1.15_2.0.0-ga_mfg-tools\mfgtools-with-rootfs\mfgtools-with-rootfs\mfgtools 下找到 cfg.ini 文件, 该文件里包含了开发板的一些信息, 查看 cfg.ini 文件可得 lite=1 以及一些字符串代表的值。

“%lite%”和“%6uluboot%”分别表示取 lite 和 6uluboot 的值, 而 lite=1, 6uluboot=14x14evk, 因此将这个值代入以后就是:

```
u-boot-imx6ull14x14evk _emmc.imx
```

所以, 这里向 DDR 中下载的是 firmware/u-boot-imx6ull14x14evk _emmc.imx 这个 uboot 文件。同样的方法将 .dtb(设备树)和 zImage 都下载到 DDR 中以后就会跳转去运行 OS, 这个时候会在 MfgTool 工具中会有“Jumping to OS image”提示语句, ucl2.xml 中的跳转命令如下:

示例代码 39.2.2.5 跳转到 OS

```
<CMD state="BootStrap" type="jump" > Jumping to OS image. </CMD>
```

启动 Linux 系统以后就可以在 EMMC 上创建分区, 然后烧写 uboot、zImage、.dtb(设备树)

和根文件系统。

这个就是 MfgTool 的整个烧写原理，懂得了烧写原理以后就可以开始试着先将 NXP 官方的系统烧写到正点原子的 I.MX6U-ALPHA 开发板中。

39.3 烧写 NXP 官方系统

我们先试着将 NXP 官方的系统烧写到正点原子的 I.MX6U-ALPHA 开发板中，主要是先熟悉一下烧写过程。因为正点原子的 EMMC 核心版用的也是 512MB 的 DDR3 加 8G 的 EMMC，因此烧写 NXP 官方的系统是没有任何问题的。烧写步骤如下：

①、连接好 USB，拨码开关拨到 USB 下载模式。

②、弹出 TF 卡，然后按下开发板复位按键。

③、打开 SecureCRT。

③、双击“mfgtool2-yocto-mx-evk-emmc.vbs”，打开下载软件，如果出现“符合 HID 标准的供应商定义设备”等字样就说明下载软件已经准备就绪。点击“Start”按钮开发烧写 NXP 官方系统，烧写过程如图 39.3.1 所示：

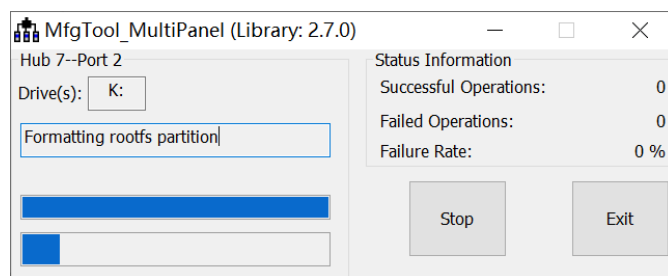


图 39.3.1 烧写过程

这个时候可以在 SecurCRT 上看到具体的烧写过程，如图 39.3.2 所示：

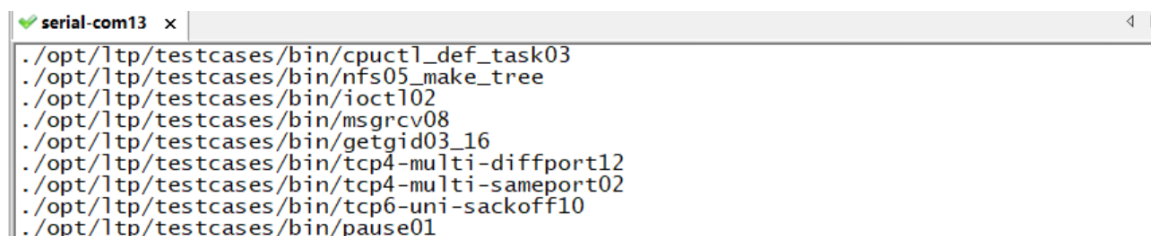


图 39.3.2 正在烧写的文件

等待烧写完成，因为 NXP 官方的根文件系统比较大，因此烧写的时候耗时会长一点。烧写完成以后 MfgTool 软件如图 39.3.3 所示：

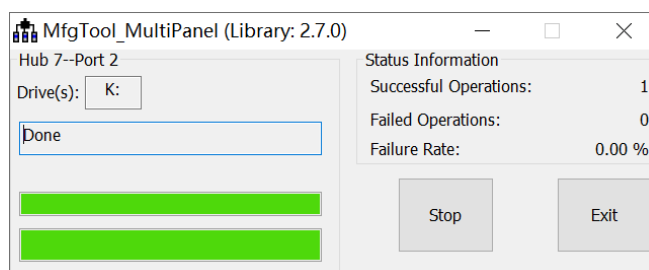


图 39.3.2 烧写完成

烧写完成以后点击“Stop”按钮停止烧写，然后点击“Exit”键退出。拔出 USB 线，将开发板上的拨码开关拨到 EMMC 启动模式，然后重启开发板，此时就会从 EMMC 启动。只是启动以后的系统是 NXP 官方给 I.MX6ULL EVK 开发板制作的，这个系统需要输入用户名，用户

名为“root”，没有密码，如图 39.3.3 所示：

```
Running local boot scripts (/etc/rc.local).
Freescaler i.MX Release Distro 4.1.15-2.0.0 imx6ul7d /dev/ttyMXc0
imx6ul7d login:
```

图 39.3.3 NXP 官方根文件系统

在“imx6ul7d login:”后面输入“root”用户名，然后点击回车键即可进入系统中，进入系统以后就可以进行其他操作了。所以说，NXP 官方的系统其实是在正点原子的 EMMC 版核心板上运行的。

39.4 烧写自制的系统

39.4.1 系统烧写

上一小节我们试着将 NXP 官方提供的系统烧写到正点原子的 I.MX6U-ALPHA 开发板中，目的是体验一下通过 MfgTool 烧写系统的过程。本小节我们就来学习如何将我们做好的系统烧写到开发板中，首先是准备好要烧写的原材料：

- ①、自己移植编译出来的 uboot 可执行文件：u-boot.imx。
- ②、自己移植编译出来的 zImage 镜像文件和开发板对应的.dtb(设备树)，对于 I.MX6U-ALPHA 开发板来说就是 imx6ull-alientek-emmc.dtb。
- ③、自己构建的根文件系统 rootfs，这里我们需要对 rootfs 进行打包，进入到 Ubuntu 中的 rootfs 目录中，然后使用 tar 命令对其进行打包，命令如下：

```
cd rootfs/
tar -vcjf rootfs.tar.bz2 *
```

完成以后会在 rootfs 目录下生成一个名为 rootfs.tar.bz2 的压缩包，将 rootfs.tar.bz2 发送到 windows 系统中。

将上面提到的这 4 个“原材料”都发送到 Windows 系统中，如图 39.4.1 所示：

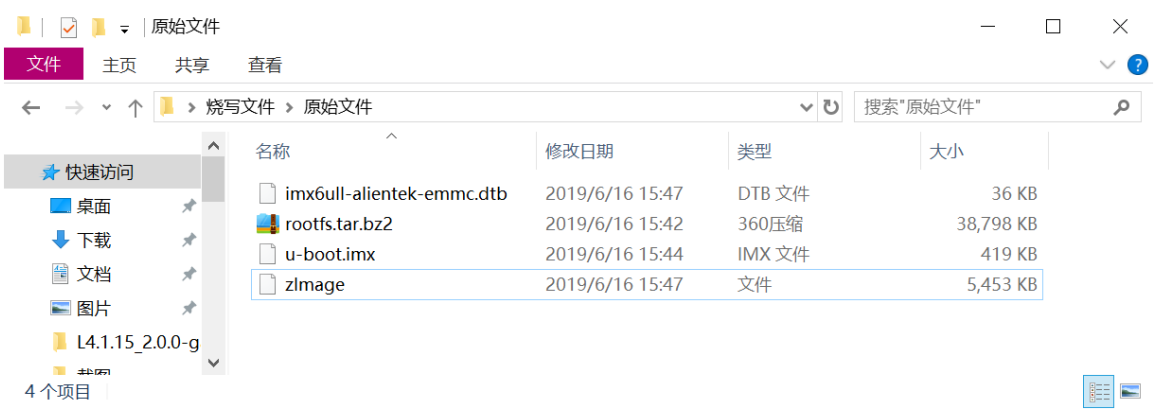


图 39.4.1 烧写原材料

材料准备好以后还不能直接进行烧写，必须对其进行重命名，否则的话 ucl2.xml 是识别不出来的，前面讲解 ucl2.xml 语法的时候已经说过了，图 39.4.1 中的这四个文件重命名见表 39.4.1：

原名字	重命名
u-boot.imx	u-boot-imx6ull14x14evk_emmc.imx
zImage	zImage(不需要重命名)
imx6ull-alientek-emmc.dtb	zImage-imx6ull-14x14-evk-emmc.dtb
rootfs.tar.bz2	rootfs_nogpu.tar.bz2

表 39.4.1 文件重命名表

完成以后如图 39.4.2 所示:

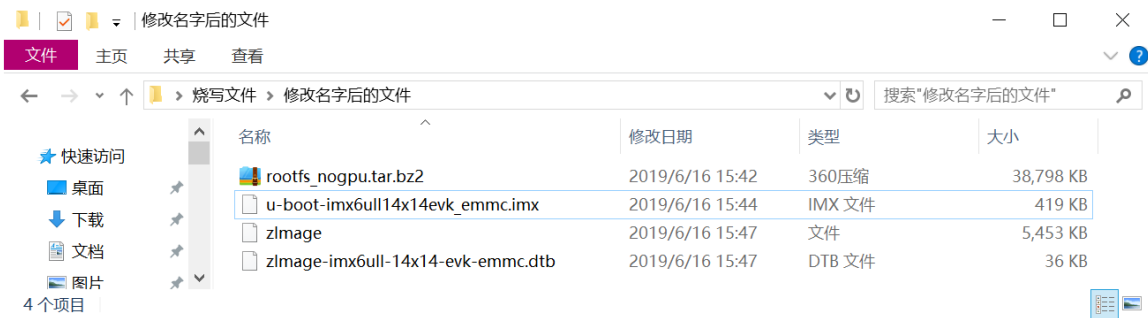


图 39.4.2 重命名以后的文件

接下来就是用我们的文件替换掉 NXP 官方的文件，先将图 39.4.2 中的 zImage、u-boot-imx6ull14x14evk_emmc.imx 和 zImage-imx6ull-14x14-evk-emmc.dtb 这三个文件拷贝到 mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware/firmware 目录中，替换掉原来的文件。然后将图 39.4.2 中的所有 4 个文件都拷贝到 mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware/files 目录中，这两个操作完成以后我们就可以进行烧写了。

双击“mfgtool2-yocto-mx-evk-emmc.vbs”，打开烧写软件，点击“Start”按钮开始烧写，由于我们自己制作的 rootfs 比较小，因此烧写相对来说会快一点。烧写完成以后设置开发板从 EMMC 启动，启动我们刚刚烧写进去的系统，测试有没有问题。

注意！一旦自己改造的 mfgtools 工具能够正常烧写系统，那么 mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware/firmware 目录下的文件以后就不能再修改！否则可能导致烧写失败！

39.4.2 网络开机自启动设置

大家在测试网络的时候可能会发现网络不能用，这并不是因为我们将系统烧写到 EMMC 中以后网络坏了。仅仅是因为网络没有打开，我们用 NFS 挂载根文件系统的时候因为要使用 NFS 服务，因此 Linux 内核会打开 eth0 这个网卡，现在我们不使用 NFS 挂载根文件系统，因此 Linux 内核也就不会自动打开 eth0 网卡了。我们可以手动打开网卡，首先输入“ifconfig -a”命令查看一下 eth0 和 eth1 是否都存在，结果如图 39.4.3 所示：


```
/ # ifconfig -a
eth0      Link encap:Ethernet  HWaddr 46:F9:14:3B:59:62
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

eth1      Link encap:Ethernet  HWaddr 2A:61:50:01:A4:3C
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          LOOPBACK  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

sit0      Link encap:IPv6-in-IPv4
          NOARP  MTU:1480  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 39.4.3 查看网络

可以看出 eth0 好 eth1 都存在，既然存在我们就打开，以打开 eth0 网卡为例，输入如下命令打开 eth0:

```
ifconfig eth0 up
```

打开网卡的时候会有如图 39.4.4 所示的提示信息:

```
/ # ifconfig eth0 up
fec 20b4000.ethernet eth0: Freescale FEC PHY driver [SMSC LAN8710/LAN8720] (mii_bus:phy_addr=20b4000.ethernet:01, irq=-1)
IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
/ # fec 20b4000.ethernet eth0: Link is Up - 100Mbps/Full - flow control rx/tx
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
```

图 39.4.5 打开 eth0 网卡

打开的时候会提示 eth0 连接成功，并且是 100Mbps 全双工，eth0 链接准备就绪。这个时候输入“ifconfig”命令就会看到 eth0 这个网卡，如图 39.4.6 所示:

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 46:F9:14:3B:59:62
          inet6 addr: fe80::44f9:14ff:fe3b:5962/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:588 errors:0 dropped:24 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:44330 (43.2 KiB)  TX bytes:680 (680.0 B)

/ #
```

图 39.4.6 当前工作的网卡

接下来就是个 eth0 设置 IP 地址，如果你的开发板连接的路由器，那么可以通过路由器自动分配 IP 地址，命令如下:

```
udhcpc -i eth0 //通过路由器分配 IP 地址
```

如果你的开发板连接着电脑，那么就可以手动设置 IP 地址，比如设置为 192.168.1.251，命令如下:

```
ifconfig eth0 192.168.1.251 netmask 255.255.255.0 //设置 IP 地址和子网掩码
route add default gw 192.168.1.1 //添加默认网关
```

推荐大家将开发板连接到路由器上, 设置好 IP 地址以后就可以测试网络了, 比如 ping 一下电脑 IP 地址, 或者 ping 一下百度官网。

每次开机以后都要自己手动打开网卡, 然后手动设置 IP 地址也太麻烦了, 有没有开机以后自动启动网卡并且设置 IP 地址的方法呢? 肯定有的, 我们将打开网卡, 设置网卡 IP 地址的命令添加到/etc/init.d/rcS 文件中就行了, 完成以后的 rcS 文件内容如下所示:

//示例代码 39.4.2.1 网络开机自启动

```
1  #!/bin/sh
2
3  PATH=/sbin:/bin:/usr/sbin:/usr/bin
4  LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib:/usr/lib
5  export PATH LD_LIBRARY_PATH runlevel
6
7  #网络开机自启动设置
8  ifconfig eth0 up
9  #udhcpc -i eth0
10 ifconfig eth0 192.168.1.251 netmask 255.255.255.0
11 route add default gw 192.168.1.1
.....
12 #cd /drivers
13 #./hello &
14 #cd /
```

第 8 行, 打开 eth0 网卡

第 9 行, 通过路由器自动获取 IP 地址。

第 10 行, 手动设置 eth0 的 IP 地址和子网掩码。

第 11 行, 添加默认网关。

修改好 rcS 文件以后保存并退出, 重启开发板, 这个时候 eth0 网卡就会在开机的时候自动启动了, 我们也不用手动添加相关设置了。

39.5 改造我们自己的烧写工具

39.5.1 改造 MfgTool

在上一小节中我们已经实现了将自己的系统烧写到开发板中, 但是使用的是“借鸡生蛋”的方法。我们通过将 NXP 官方的系统更换成我们自己制作的系统来完成系统烧写, 本节我们就来学习一下如何将 MfgTool 这个工具改造成我们自己的工具, 让其支持我们自己的开发板。要改造 MfgTool, 重点是三方面:

- ①、针对不同的核心版, 确定系统文件相关名字。
- ②、新建我们自己的.vbs 文件。
- ③、修改 ucl2.xml 文件。

1、确定系统文件名字

确定系统文件名字完全是为了兼容不同的产品, 比如某个产品有 NAND 和 EMMC 两个版

本, 那么 EMMC 和 NAND 这两个版本的 uboot、zImage、.dtb 和 rootfs 有可能不同。为了在 MfgTool 工具中同时支持 EMMC 和 NAND 这两个版本的核心板, EMMC 版本的系统文件命名如图 39.5.1.1 所示:

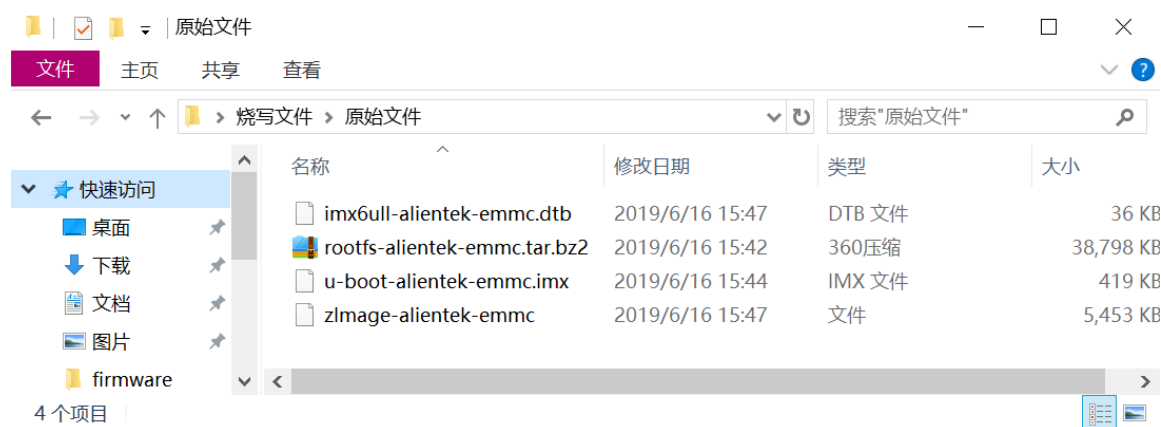


图 39.5.1.1 系统文件名

2、新建.vbs 文件

直接复制 mfgtool2-yocto-mx-evk-emmc.vbs 文件即可, 将新复制的文件重命名为 mfgtool2-alientek-alpha-emmc.vbs, 文件内容不要做任何修改, .vbs 文件我们就新建好了。

3、修改 ucl2.xml 文件

在修改 ucl2.xml 文件之前, 先保存一份原始的 ucl2.xml。将 ucl2.xml 文件改为如下所示内容:

<!-- 正点原子修改后的 ucl2.xml 文件 -->

```
<UCL>
<CFG>
  <STATE name="BootStrap" dev="MX6UL" vid="15A2" pid="007D"/>
  <STATE name="BootStrap" dev="MX6ULL" vid="15A2" pid="0080"/>
  <STATE name="Updater" dev="MSC" vid="066F" pid="37FF"/>
</CFG>

<!-- 向 EMMC 烧写系统 -->
<LIST name="eMMC" desc="Choose eMMC as media">
  <CMD state="BootStrap" type="boot" body="BootStrap" file
="firmware/u-boot-alientek-emmc.imx" ifdev="MX6ULL">Loading U-
boot</CMD>
  <CMD state="BootStrap" type="load" file="firmware/zImage-alientek-
emmc" address="0x80800000"
    loadSection="OTH" setSection="OTH" HasFlashHeader="FALSE"
ifdev="MX6SL MX6SX MX7D MX6UL MX6ULL">Loading Kernel.</CMD>
  <CMD state="BootStrap" type="load" file="firmware/%initramfs%"
address="0x83800000"
    loadSection="OTH" setSection="OTH" HasFlashHeader="FALSE"
ifdev="MX6SL MX6SX MX7D MX6UL MX6ULL">Loading Initramfs.</CMD>
```



```
<CMD state="BootStrap" type="load" file="firmware/imx6ull-alientek-
emmc.dtb" address="0x83000000"
    loadSection="OTH" setSection="OTH" HasFlashHeader="FALSE"
ifdev="MX6ULL">Loading device tree.</CMD>
<CMD state="BootStrap" type="jump" > Jumping to OS image. </CMD>

<!-- create partition -->
<CMD state="Updater" type="push" body="send"
file="mksdcard.sh.tar">Sending partition shell</CMD>
<CMD state="Updater" type="push" body="$ tar xf $FILE ">
Partitioning...</CMD>
<CMD state="Updater" type="push" body="$ sh mksdcard.sh
/dev/mmcblk%mmc%"> Partitioning...</CMD>

<!-- burn uboot -->
<CMD state="Updater" type="push" body="$ dd if=/dev/zero
of=/dev/mmcblk%mmc% bs=1k seek=768 conv=fsync count=8">clear u-boot
arg</CMD>
<!-- access boot partition -->
<CMD state="Updater" type="push" body="$ echo 0 >
/sys/block/mmcblk%mmc%boot0/force_ro">access boot partition 1</CMD>
<CMD state="Updater" type="push" body="send" file="files/u-boot-
alientek-emmc.imx" ifdev="MX6ULL">Sending u-boot.bin</CMD>
<CMD state="Updater" type="push" body="$ dd if=$FILE
of=/dev/mmcblk%mmc%boot0 bs=512 seek=2">write U-Boot to sd card</CMD>
<CMD state="Updater" type="push" body="$ echo 1 >
/sys/block/mmcblk%mmc%boot0/force_ro"> re-enable read-only access
</CMD>
<CMD state="Updater" type="push" body="$ mmc bootpart enable 1 1
/dev/mmcblk%mmc%">enable boot partion 1 to boot</CMD>

<!-- create fat partition -->
<CMD state="Updater" type="push" body="$ while [ ! -e
/dev/mmcblk%mmc%p1 ]; do sleep 1; echo \"waiting...\"; done ">Waiting
for the partition ready</CMD>
<CMD state="Updater" type="push" body="$ mkfs.vfat
/dev/mmcblk%mmc%p1">Formatting rootfs partition</CMD>
<CMD state="Updater" type="push" body="$ mkdir -p
/mnt/mmcblk%mmc%p1"/>
<CMD state="Updater" type="push" body="$ mount -t vfat
/dev/mmcblk%mmc%p1 /mnt/mmcblk%mmc%p1"/>

<!-- burn zImage -->
```

```

<CMD state="Updater" type="push" body="send" file="files/zImage-
alientek-emmc">Sending kernel zImage</CMD>

<CMD state="Updater" type="push" body="$ cp $FILE
/mnt/mmcblk%mmc%p1/zImage">write kernel image to sd card</CMD>

<!-- burn dtb -->
<CMD state="Updater" type="push" body="send" file="files/imx6ull-
alientek-emmc.dtb" ifdev="MX6ULL">Sending Device Tree file</CMD>

<CMD state="Updater" type="push" body="$ cp $FILE
/mnt/mmcblk%mmc%p1/imx6ull-alientek-emmc.dtb" ifdev="MX6ULL">write
device tree to sd card</CMD>

<CMD state="Updater" type="push" body="$ umount
/mnt/mmcblk%mmc%p1">Unmounting vfat partition</CMD>

<!-- burn rootfs -->
<CMD state="Updater" type="push" body="$ mkfs.ext3 -F -E nodiscard
/dev/mmcblk%mmc%p2">Formatting rootfs partition</CMD>

<CMD state="Updater" type="push" body="$ mkdir -p
/mnt/mmcblk%mmc%p2"/>

<CMD state="Updater" type="push" body="$ mount -t ext3
/dev/mmcblk%mmc%p2 /mnt/mmcblk%mmc%p2"/>

<CMD state="Updater" type="push" body="pipe tar -jxv -C
/mnt/mmcblk%mmc%p2" file="files/rootfs-alientek-emmc.tar.bz2"
ifdev="MX6UL MX7D MX6ULL">Sending and writting rootfs</CMD>

<CMD state="Updater" type="push" body="frf">Finishing rootfs
write</CMD>

<CMD state="Updater" type="push" body="$ umount
/mnt/mmcblk%mmc%p2">Unmounting rootfs partition</CMD>

<CMD state="Updater" type="push" body="$ echo Update
Complete!">Done</CMD>

</LIST>
</UCL>

```

ucl2.xml 文件我们仅仅保留了给 EMMC 烧写系统, 如果要支持 NAND 的话可以自行参考原版的 ucl2.xml 文件, 添加相关的内容。

39.5.2 烧写测试

MfgTool 工具修改好以后就可以进行烧写测试了, 将 imx6ull-alientek-emmc.dtb、u-boot-alientek-emmc.imx 和 zImage-alientek-emmc 这三个文件复制到 mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware/firmware 目录中。将 imx6ull-alientek-emmc.dtb、u-boot-alientek-emmc.imx、zImage-alientek-emmc 和 rootfs-alientek-emmc.tar.bz2 这四个文件复制到 mfgtools-with-rootfs/mfgtools/Profiles/Linux/OS Firmware/files 目录中。

点击“mfgtool2-alientek-alpha-emmc.vbs”打开 MfgTool 烧写系统, 等待烧写完成, 然后设置拨码开关为 EMMC 启动, 重启开发板, 系统启动信息如图 39.5.2.1 所示:

```
Normal Boot
Hit any key to stop autoboot: 0
switch to partitions #0, OK
mmc1(part 0) is current device
switch to partitions #0, OK
mmc1(part 0) is current device
reading boot.scr
** Unable to read file boot.scr **
reading zImage
5583352 bytes read in 138 ms (38.6 MiB/s)
Booting from mmc ...
reading imx6ull-14x14-evk.dtb
** Unable to read file imx6ull-14x14-evk.dtb **
Kernel image @ 0x80800000 [ 0x000000 - 0x5531f8 ]

Starting kernel ...
```

图 39.5.2.1 系统启动 log 信息

从图 39.5.2.1 可以看出，出现“Starting kernel ...”以后就再也没有任何信息输出了，说明 Linux 内核启动失败了。接下来就是解决为何 Linux 内核启动失败这个问题。

39.5.3 解决 Linux 内核启动失败

上一小节我们启动系统以后发现输出“Starting kernel ...”以后就再也没有任何信息了，难道是系统烧写错误了？可以确定的是 uboot 启动正常，就是在启动 Linux 的时候出问题了，仔细观察 uboot 输出的 log 信息，会发现如图 39.5.3.1 所示两行信息：

```
reading imx6ull-14x14-evk.dtb
** Unable to read file imx6ull-14x14-evk.dtb **
```

图 39.5.3.1 读取设备树出错

从图 39.5.3.1 可以看出，在读取“imx6ull-14x14-evk.dtb”这个设备树文件的时候出错了。重启 uboot，进入到命令行模式，输入如下命令查看 EMMC 的分区 1 里面有没有设备树文件：

```
mmc dev 1      //切换到 EMMC
ls mmc 1:1     //输出 EMMC1 分区 1 中的所有文件
```

结果如图 39.5.3.2 所示：

```
=>ls mmc 1:1
5583352  zimage
36185   imx6ull-alientek-emmc.dtb

2 file(s), 0 dir(s)

=>
```

图 39.5.3.2 EMMC 分区 1 文件

从图 39.5.3.2 可以看出，此时 EMMC 的分区 1 中是存在设备树文件的，只是文件名字为：imx6ull-alientek-emmc.dtb，因此读取 imx6ull-14x14-evk.dtb 肯定会出错的，因为根本就不存在这个文件。之所以出现这个错误的原因是因为 uboot 里面默认的设备树名字就是 imx6ull-14x14-evk.dtb，这个我们在讲解 uboot 的时候就已经说过了。解决方法很简单，有两种方法：

1、重新设置 bootcmd 环境变量值

进入 uboot 的命令行，重新设置 bootcmd 和 bootargs 这两个环境变量的值，这里要注意的是 bootargs 的值也要重新设置一下，命令如下：

```
setenv bootcmd 'mmc dev 1;fatload mmc 1:1 80800000 zImage;fatload mmc 1:1 83000000
imx6ull-alientek-emmc.dtb;bootz 80800000 - 83000000'
setenv bootargs 'console=ttyMXC0,115200 root=/dev/mmcblk1p2 rootwait rw'
```

saveenv

设置好 bootcmd 和 bootargs 这两个环境变量以后重启开发板, Linux 系统就可以正常启动。

2、修改 uboot 源码

第 1 种方法每次重新烧写系统以后都要先手动设置一下 bootcmd 的值, 这样有点麻烦, 没有一劳永逸的方法呢? 肯定是有的, 就是直接修改 uboot 源码。打开 uboot 源码中的文件 include/configs/mx6ull_alientek_emmc.h, 在宏 CONFIG_EXTRA_ENV_SETTINGS 中找到如下所示内容:

示例代码 39.5.3.1 查找设备树文件

```
194 "findfdt="\
195 "if test $fdt_file = undefined; then " \
196     "if test $board_name = EVK && test $board_rev = 9X9; then " \
197         "setenv fdt_file imx6ull-9x9-evk.dtb; fi; " \
198     "if test $board_name = EVK && test $board_rev = 14X14; then " \
199         "setenv fdt_file imx6ull-14x14-evk.dtb; fi; " \
200     "if test $fdt_file = undefined; then " \
201         "echo WARNING: Could not determine dtb to use; fi; " \
202     "fi;\0" \
```

findfdt 就是用于确定设备树文件名字的环境变量, fdt_file 环境变量保存着设备树文件名。第 196 行和 197 行用于判断设备树文件名字是否为 imx6ull-9x9-evk.dtb, 第 198 行和 199 行用于判断设备树文件名字是否为 imx6ull-14x14-evk.dtb。这两个设备树都是 NXP 官方开发板使用的, I.MX6U-ALPHA 开发板用不到, 因此直接将示例代码 39.5.3.1 中 findfdt 的值改为如下内容:

示例代码 39.5.3.1 查找设备树文件

```
194 "findfdt="\
195     "if test $fdt_file = undefined; then " \
196         "setenv fdt_file imx6ull-alientek-emmc.dtb; " \
197     "fi;\0" \
```

第 196 行, 如果 fdt_file 未定义的话, 直接设置 fdt_file=imx6ull-alientek-emmc.dtb, 简单直接, 不需要任何的判断语句。修改以后重新编译 uboot, 然后用将新的 uboot 烧写到开发板中, 烧写完成以后重启测试, Linux 内核启动正常。

关于系统烧写就讲解到这里, 本章我们使用 NXP 提供的 MfgTool 工具通过 USB OTG 口向开发板的 EMMC 中烧写 uboot、Linux kernel、.dtb(设备树)和 rootfs 这四个文件。在本章我们主要做了五个工作:

①、理解 MfgTool 工具的工作原理。

②、使用 MfgTool 工具将 NXP 官方系统烧写到 I.MX6U-ALPHA 开发板中, 主要是为了体验一下 MfgTool 软件的工作流程以及烧写方法。

③、使用 MfgTool 工具将我们自己编译出来的系统烧写到 I.MX6U-ALPHA 开发板中。

④、修改 MfgTool 工具, 使其支持我们所使用的硬件平台。

⑤、修改相应的错误。

关于系统烧写的方法就讲解到这里, 本章内容不仅仅是为了讲解如何向 I.MX6ULL 芯片中烧写系统, 更重要的是向大家详细的讲解了 MfgTool 的工作原理。如果大家在后续的工作或学习中使用 I.MX7 或者 I.MX8 等芯片, 本章同样适用。

随着本章的结束, 也宣告着本书第三篇的内容也正式结束了, 第三篇是系统移植篇, 重点

就是 uboot、Linux kernel 和 rootfs 的移植，看似简简单单的“移植”两个字，引出的却是一篇 300 多页的“爱恨情仇”。授人以鱼不如授人以渔，本可以简简单单的教大家修改哪些文件、添加哪些内容，怎么去编译，然后得到哪些文件。但是这样只能看到表象，并不能深入的了解其原理，为了让大家能够详细的了解整个流程，笔者义无反顾的选择了这条最难走的路，不管是 uboot 还是 Linux kernel，从 Makefile 到启动流程，都尽自己最大的努力去阐述清楚。奈何，笔者水平有限，还是有很多的细节没有处理好，大家有疑问的地方可以到正点原子论坛 www.openedv.com 上发帖留言，大家一起讨论学习。